

DE DÉBUTANT À DÉVELOPPEUR

C++ LE GUIDE DU MODERNE

Mehdi Benharrats et Benoît Vittupier

avec la
contribution de  **Beste de savoir**

Le guide du C++ moderne

de débutant à développeur

par
Benoît Vittupier
Mehdi Benharrats
avec la contribution de Zeste de Savoir

Le guide du C++ moderne • de débutant à développeur
par Benoît Vittupier, Mehdi Benharrats, avec la contribution de Zeste de Savoir

ISBN (PDF) : 978-2-8227-0879-1

Copyright © 2021 Éditions D-BookeR

Tous droits réservés

Conformément au Code de la propriété intellectuelle, seules les copies ou reproductions strictement réservées à l'usage privé du copiste et non destinées à une utilisation collective ainsi que les analyses et les courtes citations dans un but d'exemple et d'illustration sont autorisées. Tout autre représentation ou reproduction, qu'elle soit intégrale ou partielle, requiert expressément le consentement de l'éditeur (art L 122-4, L 122-5 2 et 3a).

Publié par les Éditions D-BookeR, 229 rue Solférino, 59000 Lille

www.d-booker.fr

contact@d-booker.fr

Les exemples (téléchargeables ou non), sauf indication contraire, sont propriété des auteurs.

Conception de la couverture : d'après une création de Marie Van Der Marlière
(www.marie-graphiste.com)

Visuel de couverture : Réalisé par putracetol (#22795181 sur VectorStock.com)

Mise en page : générée sous Calenco avec des XSLT développées par la société
NeoDoc (www.neodoc.biz)

Date de publication : Août 2021

Édition : 1

Version : 1.0

Table des matières

À propos des auteurs	xx
Bienvenue !	1
1. Codes sources des exemples	1
2. Remerciements	1
1. C'est décidé, je m'y mets !	3
1.1. Le C++, qu'est-ce que c'est ?	3
1.2. Pourquoi apprendre le C++ ?	4
1.3. Développer, un métier à part entière	5
1.4. Votre part du travail	5
1.5. Les mathématiques, indispensables ?	6
1.6. L'anglais, indispensable ?	6
1.7. La documentation	6
1.8. Récapitulons !	7
Le début du voyage	8
2. Le minimum pour commencer	9
2.1. Des outils en ligne	9
2.2. Des outils plus poussés	11
2.3. Un mot concernant Windows	15
2.4. Un mot concernant GNU/Linux	16
Éditeur de texte	16
Compilateur	16
2.5. Récapitulons !	17
3. Rencontre avec le C++	18
3.1. Compilons notre premier programme	18
3.2. Démystifions le code	19
Utiliser des fonctionnalités déjà codées	19
Le point d'entrée	19
Voici mes instructions...	20
3.3. Les commentaires	21
3.4. Récapitulons !	22
4. Tout ça est bien variable	24
4.1. Les littéraux	24

Les caractères	24
Les caractères spéciaux	26
Les nombres	27
4.2. Les variables	29
Comment créer des variables en C++ ?	30
Un peu de constance, voyons !	34
Manipulation de variables	35
4.3. Simple déduction ?	37
Avec <i>const</i>	38
Avec <i>std::string</i>	38
Quel intérêt ?	39
4.4. Les entrées	39
4.5. Récapitulons !	41
5. Le conditionnel conjugué en C++	44
5.1. Les booléens	44
5.2. <i>if</i> – si, et seulement si...	46
5.3. <i>else</i> – sinon...	48
5.4. <i>else if</i> – la combinaison des deux précédents	50
5.5. Conditions imbriquées	51
5.6. [Pratique] Gérer les erreurs d'entrée • Partie I	52
5.7. La logique booléenne	53
AND – tester si deux conditions sont vraies	53
OR – tester si au moins une condition est vraie	54
NOT – tester la négation	55
5.8. Tester plusieurs expressions	56
5.9. Entraînez-vous !	59
Une pseudo-horloge	59
Score	59
XOR – le OU exclusif	60
5.10. Récapitulons !	60
6. Des boucles qui se répètent, répètent, répètent...	67
6.1. <i>while</i> – tant que...	67
6.2. Entraînez-vous !	69
Une laverie	69
PGCD	70
Somme de nombres de 1 à <i>n</i>	70
6.3. <i>do while</i> – répéter ... tant que	70
6.4. [Pratique] Gérer les erreurs d'entrée • Partie II	71

6.5. <i>for</i> – une boucle condensée	72
6.6. Boucles imbriquées	73
6.7. Convention de nommage	74
6.8. Contrôler plus finement l'exécution de la boucle	74
<i>break</i> – je m'arrête là	74
<i>continue</i> – saute ton tour !	76
6.9. Récapitulons !	76
7. Au tableau !	83
7.1. Un tableau c'est quoi ?	83
7.2. <i>std::vector</i> – mais quel dynamisme !	83
Déclarer un <i>std::vector</i>	84
Manipuler les éléments d'un <i>std::vector</i>	85
7.3. Entraînez-vous !	93
Calcul de moyenne	93
Minimum et maximum	93
Séparer les pairs des impairs	93
Compter les occurrences d'une valeur	93
7.4. <i>std::array</i> – le tableau statique	94
Déclarer un <i>std::array</i>	94
Remplir un tableau	95
Accéder aux éléments	95
Connaître la taille	96
Vérifier si le tableau est vide	97
7.5. <i>std::string</i> – un type qui cache bien son jeu	97
Connaître la taille d'une chaîne	97
Accéder à un caractère	98
Premier et dernier caractère	98
Vérifier qu'une chaîne est vide	99
Ajouter ou supprimer un caractère à la fin	99
Supprimer tous les caractères	100
Boucler sur une chaîne	100
7.6. Récapitulons !	101
8. Déployons la toute puissance des conteneurs.....	106
8.1. La pièce maîtresse : les itérateurs	106
Déclaration d'un itérateur	106
Début et fin d'un conteneur	107
Accéder à l'élément pointé	108
Se déplacer dans une collection	108

const et les itérateurs	111
Itérer depuis la fin	112
Utilisation conjointe avec les conteneurs	113
8.2. Des algorithmes à gogo !	114
std::count – compter les occurrences d’une valeur	115
std::find – trouver un certain élément	116
std::sort – trier une collection	117
std::reverse – inverser l’ordre des éléments d’une collection	118
std::remove – suppression d’éléments	119
std::search – rechercher un sous-ensemble dans un en-semble	119
std::equal – vérifier l’égalité de deux ensembles	120
Et tant d’autres	121
8.3. Personnalisation à la demande	121
Le prédicat, pivot de la personnalisation des algorithmes	121
Trier une liste dans l’ordre décroissant	122
Somme et produit	123
Prédicats pour caractères	124
8.4. Entraînez-vous !	125
Palindrome	125
string_trim – suppression des espaces	125
Couper une chaîne	126
8.5. Récapitulons !	126
9. Des flux dans tous les sens	131
9.1. Avant toute chose	131
Prendrez-vous une extension ?	131
Vois sur ton chemin...	132
Pas d’interprétation, on est des brutes	133
9.2. std::ofstream – écrire dans un fichier	134
Ouvrir le fichier	134
Écrire	134
Ouvrir sans effacer	137
9.3. std::ifstream – lire dans un fichier	138
Ouvrir le fichier	138
Lire	138
Tout lire	141
9.4. Entraînez-vous !	141
Statistiques sur des fichiers	141
9.5. Encore plus de flux !	142

Un flux c'est quoi ?	142
Un <i>buffer</i> dites-vous ?	143
Les modificateurs de flux	144
Même les chaînes y passent !	145
9.6. Récapitulons !	146

On passe la deuxième ! 149

10. Découpons le code — les fonctions..... 150

10.1. Les éléments de base d'une fonction	150
Une fonction, c'est quoi ?	150
Une fonction incontournable	152
Les composants d'une fonction	153
10.2. Entraînez-vous !	156
Afficher un rectangle	156
Distributeur d'argent	156
Parenthésage	157
10.3. Quelles sont vos références ?	157
La problématique	157
Les références	159
Paramètres de fonctions	160
Valeur de retour	163
Un mot sur la déduction de type	164
10.4. Nos fonctions sont surchargées !	166
10.5. [Pratique] Gérer les erreurs d'entrée • Partie III	167
10.6. Dessine-moi une fonction	168
Le problème	168
La solution	169
Utilisons les prototypes	170
10.7. Récapitulons !	171

11. Erreur, erreur, erreur... 176

11.1. L'utilisateur est un idiot	176
11.2. À une condition... ou plusieurs	177
Contrats assurés par le compilateur	178
Vérifions nous aussi	178
11.3. Le développeur est un idiot	178
Préconditions et assertions	181
11.4. Les tests unitaires à la rescousse	182
Pourquoi tester ?	182

Un test unitaire, qu'est-ce que c'est ?	182
Écrire des tests unitaires	183
Les tests unitaires et les postconditions	184
11.5. [Pratique] Gérer les erreurs d'entrée • Partie IV	184
11.6. L'exception à la règle	186
La problématique	186
C'est quoi une exception ?	187
Lancement des exceptions dans 3, 2, 1...	188
Attends que je t'attrape !	189
Attrapez-les tous !	191
Un type pour les gouverner tous et dans le catch les lier	192
11.7. [Pratique] Gérer les erreurs d'entrée • Partie V	194
11.8. Récapitulons !	194
12. Des fonctions somme toute lambdas.....	199
12.1. Une lambda, c'est quoi ?	199
12.2. Pourquoi a-t-on besoin des lambdas ?	199
12.3. Syntaxe	200
Quelques exemples simples	200
12.4. Entraînez-vous !	203
Vérifier si un nombre est négatif	203
Tri par valeur absolue	203
12.5. On va stocker ça où ?	203
12.6. Paramètres génériques	204
12.7. [Pratique] Gérer les erreurs d'entrée • Partie VI	206
12.8. [Pratique] Gérer les erreurs d'entrée • Partie VII	206
12.9. Capture en cours...	207
Capture par valeur	207
Capture par référence	209
12.10. Récapitulons !	210
13. Envoyez le générique !	213
13.1. Quel beau modèle !	213
La bibliothèque standard : une fourmilière de modèles	218
Un point de vocabulaire	218
13.2. [Pratique] Gérer les erreurs d'entrée • Partie VIII	219
13.3. Instanciation explicite	220
13.4. Ce type-là, c'est un cas !	224
13.5. C++20 et les lambdas templates	228
13.6. Récapitulons !	229

14. De nouvelles structures de données.....	230
14.1. <i>struct</i> – un agrégat de données	230
Stockage d'informations personnelles	230
Analyse de la méthode	231
Introduction aux structures	231
Et maintenant, structurons !	233
14.2. <i>std::tuple</i> – une collection hétérogène	234
Déclaration	234
Accès aux éléments	234
<i>std::tuple</i> et fonctions	236
Équations horaires	237
14.3. <i>std::unordered_map</i> – une table associative	239
Un dictionnaire de langue Zestienne	240
Toujours plus de clés	242
Cette clé est unique !	243
Cherchez un élément	245
Un exemple concret	247
14.4. <i>std::unordered_set</i> – représenter un ensemble	248
Inscription sur Zeste de Savoir	248
14.5. Un peu d'ordre, voyons !	249
14.6. Une longue énumération	251
14.7. Récapitulons !	253
15. Reprendrez-vous un peu de sucre syntaxique ?	257
15.1. Ce type est trop long !	257
15.2. Décomposons tout ça	258
En pleine décomposition	258
Et avant, on faisait comment ?	260
15.3. La surcharge d'opérateurs	262
Le problème... ..	262
...et la solution	264
Mise en pratique	264
Et la bibliothèque standard ?	271
Entraînez-vous !	272
Sur le bon usage de la surcharge	273
15.4. Récapitulons !	273
16. [Pratique] Un gestionnaire de discographie	278
16.1. Cahier des charges	278
Ajout de morceaux	278

Affichage de la discographie	279
Enregistrement et chargement d'une discographie	281
Fermeture du programme	281
Gestion des erreurs	281
Dernières remarques	281
16.2. Guide de résolution	282
Analyse des besoins	282
Attaquons le code !	283
16.3. Code complet	287
16.4. Conclusion et pistes d'améliorations	297
16.5. Récapitulons !	298
17. Découpons du code – les fichiers	313
17.1. Le principe	313
Le fichier d'en-tête	313
Le fichier source	316
Créons un lien	317
17.2. La sécurité, c'est important	318
17.3. [Pratique] Découpons notre programme !	319
17.4. Les avantages du découpage en fichiers	321
17.5. Le cas des templates	323
17.6. Récapitulons !	326
Interlude : être développeur	350
18. Un coup d'œil dans le rétro	351
18.1. Au-delà du code	351
18.2. Tout est une question de paradigme	352
Le paradigme impératif	352
Le paradigme fonctionnel	353
Le paradigme générique	354
La programmation par contrat	355
18.3. L'art de la conception	355
18.4. De la persévérance avant tout	356
18.5. Récapitulons !	357
19. Mais où est la doc ?	358
19.1. Lire une page de doc	358
vector – retour sur le plus célèbre des conteneurs	359
Les algorithmes	362

Les chaînes de caractères au grand complet	363
19.2. Entraînez-vous	364
Remplacer une chaîne de caractères par une autre	364
Norme d'un vecteur	364
Nombres complexes	365
Transformations	365
19.3. Documenter son code avec Doxygen	366
Installation d'un outil dédié	366
Écrire la documentation	369
19.4. Quelques bonnes pratiques	375
Ce qu'il faut documenter	375
Les fichiers d'en-tête ou sources ?	376
Rien ne vaut un exemple	376
Commentaire vs documentation	377
19.5. Récapitulons !	379
20. Compilation en cours...	384
20.1. Le préprocesseur	384
Inclure des fichiers	384
Conditions	385
<i>Debug</i> ou <i>release</i> ?	385
20.2. La compilation	387
Les templates	387
<i>constexpr</i>	388
La compilation à proprement parler	391
Une étape intermédiaire cachée	392
Influer sur la compilation	392
20.3. Le linker	397
Une question de symboles	398
20.4. Schéma récapitulatif	400
20.5. Récapitulons !	401
21. Chasse aux bugs !	402
21.1. Le principe	402
21.2. Un code d'exemple	403
21.3. Avec Visual Studio	403
Interface	404
Pas-à-pas	406
Point d'arrêt conditionnel	407
21.4. Avec Qt Creator	408

Interface	408
Pas-à-pas	410
Point d'arrêt conditionnel	411
21.5. En ligne de commande avec gdb	412
Poser un point d'arrêt	413
Supprimer des points d'arrêt	413
Désactiver des points d'arrêt	414
Afficher l'état d'une variable	414
Pas-à-pas	414
Conditions	415
21.6. Récapitulons !	416
22. Une foule de bibliothèques	417
22.1. Quelles bibliothèques choisir ?	417
22.2. Généralités	418
Statique ou dynamique ?	418
<i>Debug</i> ou <i>release</i> ?	418
32 ou 64 bits ?	419
22.3. Installer Boost	419
GNU/Linux	420
Visual Studio 2019	421
Qt Creator et MinGW sous Windows	423
22.4. Installer SFML	425
GNU/Linux	426
Windows	427
22.5. Récapitulons !	432
23. Améliorer ses projets	433
23.1. git – sauvegarder et versionner son code	433
Installer git	434
Initialiser git	434
Créer un dépôt	434
Connaître l'état de ses fichiers	435
Ajouter un fichier	436
Valider les modifications	436
Voir l'historique	437
Bloquer certains fichiers	437
Aller plus loin	439
23.2. GitLab – partager son code	439
Création d'un projet	440

Pousser nos modifications	441
Aller plus loin	442
23.3. CMake – automatiser la compilation de ses programmes	442
Un exemple simple	442
Lier des bibliothèques externes	446
Définir des variables dynamiquement	453
Aller plus loin	453
23.4. Encore quelques outils	454
GitLab CI – de l'intégration continue	454
CppCheck – vérification du code	454
StackOverflow – la réponse à quasi toutes les questions	454
23.5. Récapitulons !	455

La programmation orientée objet 456

24. Premiers pas avec la POO	457
24.1. Le principe : des objets bien serviables	457
Penser en termes de données... ..	457
...ou de services ?	458
Exemple concret	458
24.2. Un peu de vocabulaire	459
L'objet	459
Le type	459
L'interface d'une classe	459
24.3. En C++, ça donne quoi ?	460
Penser services	460
Penser tests	460
Fonctions membres	461
Fonctions libres	462
Les attributs	463
24.4. Désolé, cet objet n'est pas modifiable	464
24.5. On ne fait pas d'exception	467
24.6. Découpage en fichiers	468
24.7. Entraînez-vous !	470
Cercle	470
Informations personnelles	471
24.8. Récapitulons !	471
25. Et qui va construire tout ça ?	475
25.1. Encapsulation et invariants	475

Cette classe, c'est open bar	475
Tout est question d'invariant	476
Encapsulons tout ça	477
Modifier sa visibilité	477
25.2. On cherche un constructeur	479
Le principe	479
La syntaxe	481
25.3. Constructeur par défaut	482
25.4. Soyons explicites	485
25.5. En toute amitié	489
Attributs publics	491
Accesseurs	491
La troisième, c'est la bonne	493
25.6. Entraînez-vous !	494
Un autre constructeur pour <i>Fraction</i>	494
Une pile	494
25.7. Récapitulons !	496
26. Une classe de grande valeur	499
26.1. Une histoire de sémantique	499
26.2. La sémantique de valeur, c'est quoi ?	501
26.3. Égalité	501
Libre ou membre ?	502
Soyons intelligents, réutilisons	506
26.4. Le retour des opérateurs	507
Les opérateurs d'affectation intégrés	507
Du deux en un	508
Les opérateurs de manipulation des flux	509
Quelques bonnes pratiques	513
Les autres opérateurs	513
26.5. Copier des objets	514
Constructeur de copie	514
Opérateur d'affectation par copie	515
26.6. Récapitulons !	515
27. [Pratique] Entrons dans la matrice	517
27.1. Qu'est-ce qu'une matrice ?	517
27.2. Cahier des charges	519
Les services	519
Les détails d'implémentation	519

Les tests	520
27.3. Réalisation	524
Les services	524
Les détails d'implémentation	526
27.4. Code complet	531
27.5. Entraînez-vous !	538
Entiers infinis	538
Des polynômes	539
27.6. Récapitulons !	540
28. Classes à sémantique d'entité	541
28.1. Des objets uniques !	541
Conséquences sur l'interface	541
Un exemple	542
28.2. Des sous-types...	544
Exemple	544
Héritage et constructeurs	545
Une question de visibilité	547
28.3. ...substituables	549
Une histoire de manchots	551
Le principe de substitution de Liskov	552
Substitution dans la bibliothèque standard	553
Substitution et programmation par contrat	553
Héritage public et LSP	555
28.4. Un monde virtuel	557
Un problème...	557
...une solution	560
En interne	564
Faisons le point	566
28.5. Interfaces et classes abstraites	567
Des fonctions virtuelles vraiment pures	567
Classe abstraite et implémentation	568
28.6. La copie, une vraie source de problème	569
28.7. Récapitulons !	571
29. Ressources, indirections et handles.....	573
29.1. Retour sur la mémoire	573
Du stockage pour tous !	573
C'est à quelle adresse ?	576
29.2. Les indirections	576

29.3. Les ressources et les handles	578
29.4. Restitution garantie 100% satisfaction	579
Le travail manuel ne paie pas toujours... ..	579
Resource Acquisition Is Initialization	581
L'exemple corrigé	583
29.5. <code>std::unique_ptr</code> – pour surveiller la mémoire	585
Mais quel rôle ce pointeur joue-t-il ?	585
Un seul propriétaire... ..	585
...et plusieurs observateurs	587
Liens entre propriétaire et observateurs	588
Transfert de responsabilité	589
Plusieurs propriétaires, plusieurs observateurs	590
29.6. Les entités et les handles	591
29.7. Un problème de destruction	594
29.8. Les pointeurs nus, on oublie !	596
<code>std::function</code> – un remplaçant pour manipuler des fonctions	596
<code>std::optional</code> – un remplaçant pour valeur optionnelle	597
29.9. De l'importance de prendre ses responsabilités	598
29.10. Récapitulons !	599
30. La sémantique de déplacement	601
30.1. T'as saisi la référence ?	601
Le besoin... ..	601
...la cause... ..	603
...et la solution	604
<code>std::move</code> , la mal-nommée	605
30.2. ♪ I like to move it, move it ♪	606
30.3. En interne	609
30.4. Comme les cinq doigts de la main	612
30.5. Récapitulons !	612
31. Oh, le bel héritage	614
31.1. NVI – Un contrat est un contrat	614
Des problèmes d'héritage... ..	614
Non Virtual Interface	614
Changements dans les contrats	618
31.2. <code>protected</code> – la portée intermédiaire	619
31.3. Mettons un point final à cet héritage	622
31.4. L'héritage multiple	624
31.5. L'héritage, solution miracle ?	627

31.6. Récapitulons !	628
32. Les classes templates	630
32.1. Les classes génériques	630
Définir une classe générique	630
Référencer une classe générique	632
Arguments types et non-types	632
Valeur par défaut	633
32.2. [Pratique] Un wrapper pour les pointeurs nus	634
32.3. Les traits	635
Exemple simple	635
Traits et programmation par contrat	636
En interne	638
32.4. Ces modèles sont bien trop génériques...	639
En voilà un bon concept !	640
Créer ses propres concepts	641
32.5. Un peu de politique	644
À la base, une classe très hospitalière	645
Points de variation	645
Lier l'implémentation et les points de variation	647
32.6. Récapitulons !	651
33. Ça, c'est du SOLID !	652
33.1. S – Single responsibility principle	652
Le principe	652
Un exemple problématique...	653
...et sa solution	654
33.2. O – Open-closed principle	654
Le principe	654
Un exemple problématique...	655
...et sa solution	656
33.3. L – Liskov substitution principle	658
Le principe	658
Un exemple problématique...	658
...et sa solution	659
33.4. I – Interface segregation principle	659
Le principe	659
Un exemple problématique...	660
...et sa solution	661
33.5. D – Dependency inversion principle	663

Le principe	663
Un exemple problématique... ..	664
...et sa solution	665
33.6. Pour quelques principes de plus	666
33.7. Récapitulons !	667
34. Le voyage ne fait que commencer	668
34.1. Davantage sur C++	668
34.2. Davantage sur le développement	669
[Pratique] Gérer les erreurs d'entrée • Solutions	670
 Index	 680

À propos des auteurs

Benoît Vittupier

Je suis un informaticien passionné par le partage et la qualité. Ayant appris au contact des autres et ayant pu m'améliorer grâce à leur aide et leurs conseils, je m'efforce de faire de même et de partager largement ce que je sais pour que d'autres en profitent. Dans l'optique d'aider les débutants à apprendre correctement le C++, je me suis lancé dans ce projet pour que le public francophone puisse bénéficier d'un ouvrage à jour, simple, progressif, tout en enseignant les bonnes pratiques. Je suis aussi présent sur Zeste de Savoir, le site libre et gratuit de partage et d'entraide.

Mehdi Benharrats

Je suis étudiant à l'École Polytechnique et passionné du langage C++. J'ai commencé très jeune à programmer et j'ai acquis une approche moderne du langage auprès d'experts sur des forums spécialisés. Après plusieurs années à donner et recevoir des conseils sur ces forums, constatant le manque de ressources à jour en français, nous nous sommes lancés avec d'autres membres du site Zeste de Savoir dans un projet d'écriture d'un cours de C++ moderne. Après quelques années de maturation, le projet aboutira finalement à cet ouvrage.

avec la contribution de Zeste de Savoir

Créé en 2014 à l'initiative d'anciens membres du Site du Zéro, Zeste de Savoir est une plateforme communautaire d'entraide et de partage dont le but est de promouvoir la transmission du savoir à travers des ressources pédagogiques gratuites et de préférence sous licence libre. Sa bibliothèque est dotée de plus de 500 publications dans des domaines allant de l'informatique aux sciences humaines.

<https://zestedesavoir.com>

Bienvenue !

Bienvenue dans ce livre sur l'apprentissage de la programmation avec le langage C++. Vous débutez complètement et n'avez qu'une vague idée de ce que peut bien être la programmation ? On vous a dit d'en apprendre plus sur le C++ ? Cela tombe bien ! Notre objectif est de vous former au C++ moderne et de vous donner des bases solides sur lesquelles continuer.

Pour rendre ce voyage plus pédagogique, nous alternerons entre théorie, explications et détails techniques d'un côté et exercices, cas pratiques et questions-réponses de l'autre. L'accent sera mis sur la rigueur, la qualité et les bonnes pratiques. Le rythme sera progressif, chaque concept étant introduit en temps voulu et en lien avec ce que nous aurons déjà vu.

URL RACCOURCIES

Dans un souci de lisibilité, et pour pouvoir les maintenir à jour, nous avons pris le parti de remplacer toutes les adresses internet par ce qu'on appelle des URL raccourcies. Une fois que vous avez accédé à la page cible, nous vous invitons à l'enregistrer avec un marque-page si vous souhaitez y revenir fréquemment. Vous disposerez alors du lien direct. Si celui-ci se périmé, n'hésitez pas à repasser par l'URL raccourcie. Si cette dernière aussi échoue, vous pouvez nous le signaler !

1. Codes sources des exemples

Tous les codes sources des exemples et des exercices proposés dans ce livre sont disponibles sur le dépôt GitLab [guide-du-cpp-moderne](#) sous licence LGPL-3.0. Si, malgré tous nos soins, vous relevez des erreurs, pensez à nous les transmettre. Les autres lecteurs pourront ainsi en profiter.

2. Remerciements

Cet ouvrage n'aurait jamais vu le jour sans l'aide et le soutien de nombreuses personnes. Nous tenons en tout premier lieu à remercier notre editrice, qui nous a donné de nombreux conseils, qui a su nous remotiver dans les moments de creux et qui a toujours été disponible pour nous aider.

Nous remercions également Luc Hermitte, pour ses nombreuses relectures, pour ses conseils, ses suggestions. En particulier, nous sommes reconnaissants de l'énorme travail

d'amélioration de la partie sur la programmation orientée objet, qui a été possible grâce à son aide et ses apports.

Il serait également injuste d'oublier Guillaume Belz, dont l'expérience dans l'enseignement du C++ et les nombreuses remarques ont façonné notre vision de la pédagogie et du C++.

Cet ouvrage doit aussi beaucoup à l'association [Zeste de Savoir](#), une association à but non-lucratif centrée sur le partage gratuit, libre et de qualité du savoir. Ce projet est né sur leur site, beaucoup de chapitres y ont été relus, corrigés, améliorés grâce aux suggestions et aux efforts de différents membres, que nous ne pouvons individuellement citer tant ils sont nombreux.

À titre personnel, je (Benoît) souhaite remercier ma femme pour sa patience et sa compréhension par rapport aux longues heures que j'ai parfois passées à écrire. Ce livre est aussi possible grâce à toi !

Quant à moi (Mehdi), je souhaite remercier ma famille qui m'a toujours soutenu dans mes projets, et mes amis qui m'ont poussé à travailler ce livre quand la motivation n'était pas là. Je n'aurais pas pu y arriver sans vous !

1

C'est décidé, je m'y mets !

Voilà, vous êtes décidé, vous voulez apprendre à programmer en C++. Mais quelques questions continuent à vous trotter dans la tête : Concrètement, le langage C++, c'est quoi ? Et puis pourquoi choisir le C++ ? Que vais-je savoir à l'issue de ce livre ? Y a-t-il des conditions pour pouvoir suivre ce livre ?

1.1. Le C++, qu'est-ce que c'est ?

Faisons un plongeon dans l'histoire et revenons dans les années 1970. À cette époque, [Dennis Ritchie](#), programmeur aux laboratoires AT&T aux États-Unis, invente [le langage C](#), conçu pour programmer le système d'exploitation UNIX. Ce langage devint très populaire à tel point qu'il est encore beaucoup utilisé aujourd'hui, notamment pour l'écriture de Linux. Un peu plus tard, au début des années 1980, [Bjarne Stroustrup](#), lui aussi développeur aux laboratoires AT&T, décide de prendre ce langage C comme base et de lui ajouter des fonctionnalités issues d'un autre langage appelé Simula. Cette première bouture est appelée le *C with classes*. Finalement, en 1983, son créateur, estimant que le nom de son langage est trop réducteur au vu de tous les ajouts faits par rapport au C, décide de le renommer C++. Mais l'histoire ne s'arrête pas là. Au contraire, le C++ continue d'évoluer à tel point qu'on décide au début des années 1990 de *le normaliser*, c'est-à-dire d'en établir les règles officielles. Ce travail de longue haleine s'acheva en 1998 ; cette version est ainsi souvent nommée C++98. Ensuite, en 2003, des corrections ont été apportées et l'on obtint C++03.

Puis de nouveau un chantier titanesque est mis en place pour améliorer encore plus le C++, ce qui aboutit huit ans plus tard, en 2011, à la sortie de C++11, jugée par beaucoup de développeurs comme la renaissance du C++. Ensuite, de nouvelles corrections et quelques ajustements ont été apportés pour donner C++14 et C++17. Enfin, dernièrement est sortie la norme C++20, nouvelle version majeure, qui apporte tout un lot de choses intéressantes ; et C++23 est déjà en chantier.

Pour ceux qui lisent l'anglais, vous trouverez plus d'informations sur l'histoire du langage, ses choix de conception, les améliorations apportées par les différentes normes, etc. dans la publication de Bjarne Stroustrup [Thriving in a Crowded and Changing World: C++ 2006–2020](#).

C++ HISTORIQUE ET C++ MODERNE

Beaucoup de programmeurs utilisent le terme C++ *historique* pour désigner les normes C++98 et C++03 et le terme C++ *moderne* pour parler de C++11 et au-delà. Cette distinction n'est pas fautive, mais la différence entre le C++ historique et le C++ moderne tient moins à une question de norme qu'à des bonnes pratiques, de meilleures façons de faire, etc.

Par exemple, on peut utiliser la dernière norme mais continuer à écrire son code comme on le faisait à l'époque du commencement de C++, dans un style proche du langage C. À l'inverse, on peut être bloqué pour une raison quelconque avec un ancien compilateur et donc être forcé d'utiliser C++98 ou C++03 et pourtant écrire du code moderne, en accord avec les bonnes pratiques issues du C++ moderne.

1.2. Pourquoi apprendre le C++ ?

- Sa *popularité* : le C++ est utilisé dans de nombreux projets importants (citons [LibreOffice](#), [7-zip](#) ou encore [KDE](#)). Il est au programme de beaucoup de formations informatiques. Il possède une communauté très vaste, beaucoup de documentation et d'aide, surtout sur l'internet anglophone.
- Sa *rapidité* : il offre un grand contrôle sur la rapidité des programmes. C'est cette caractéristique qui fait de lui un des langages de choix pour les programmes scientifiques, par exemple.
- Sa *facilité d'apprentissage* : depuis sa version de 2011, le C++ est beaucoup plus facile à apprendre que par le passé. Et ça tombe bien, c'est sur cette version et les suivantes que se base ce livre.
- Son *ancienneté* : c'est un langage ancien d'un point de vue informatique (30 ans, c'est énorme), ce qui donne une certaine garantie de maturité, de stabilité et de pérennité (il ne disparaîtra pas dans quelques années).
- Son *évolution* : C++11 est un véritable renouveau du C++ historique, qui le rend plus facile à utiliser et plus puissant dans les fonctionnalités qu'il offre aux développeurs. C++14, C++17 et C++20 améliorent encore la chose.
- Il est *multiparadigme* : il n'impose pas une façon unique de concevoir et découper ses programmes mais laisse le développeur libre de ses choix.

Bien entendu, tout n'est pas parfait et le C++ a aussi ses défauts.

- *Son héritage du C* : le C++ est un descendant du langage C, inventé dans les années 1970. Certains choix de conception, adaptés pour l'époque, sont plus problématiques aujourd'hui, et le C++ les traîne avec lui.
- *Sa complexité* : il ne faut pas se le cacher, atteindre une certaine maîtrise du C++ est très long et demandera des années d'expérience, notamment parce que certaines de ses fonctionnalités les plus puissantes requièrent de très bien connaître les bases.
- *Sa bibliothèque standard* : bien qu'elle permette de faire beaucoup de choses (d'ailleurs, nous ne ferons que l'aborder dans ce livre), elle n'offre pas de mécanisme natif pour manipuler des bases de données, faire des programmes en fenêtres, jouer avec le réseau, etc. Par rapport à d'autres langages comme Python, C++ peut paraître plus limité.

1.3. Développer, un métier à part entière

Eh oui ! Créer des logiciels, c'est un métier. Il y a des écoles et des études spécialisées pour ça. C'est que développer, ce n'est pas simplement écrire du code. Il y a aussi des phases de réflexion, de conception, d'écriture, de validation, de tests, de réécriture d'anciennes portions, etc. Et puis, même si ce n'est pas directement lié à la programmation, celle-ci requiert des connaissances en gestion de base de données, en usage du réseau, en management, en gestion de projet, etc. En bref, *être développeur c'est beaucoup de compétences différentes dans des domaines variés.*

La cruelle vérité qui se cache derrière tout ça, c'est que ce livre ne fera pas de vous des experts, ni des développeurs professionnels. Par contre, une fois fini, vous aurez des bases solides pour continuer votre apprentissage. La route du savoir est infinie.

1.4. Votre part du travail

Ce livre est écrit de façon à être le plus clair possible, sans vous noyer sous un flot de détails et d'explications, mais il arrivera parfois que vous ne compreniez pas un morceau de code ou une explication. C'est tout à fait normal, ça fait partie de l'apprentissage. Reprenez le passage à tête reposée, aidez-vous de schémas ou de dessins, demandez de l'aide sur [les forums](#), et vous ne resterez jamais bloqué longtemps.

Par contre, vous devez être prêt à fournir des efforts de votre côté. Cela signifie ne pas vous ruer sur les forums à la première difficulté rencontrée, sans chercher à essayer de la résoudre par vous-même.

1.5. Les mathématiques, indispensables ?

Une des grandes appréhensions, qui revient régulièrement dans la bouche des débutants, est de savoir si les mathématiques sont un prérequis à l'apprentissage de la programmation. La réponse est non. Il est tout à fait possible d'apprendre à programmer tout en ayant un faible niveau en mathématiques. Ce livre ne requiert aucune connaissance en mathématiques, si ce n'est celle des opérations de base et, pour quelques exemples, du sinus et du cosinus.

Bien sûr, certains aspects de la programmation, comme la sécurité, la cryptographie ou les applications scientifiques demandent un bagage mathématique solide. Mais cela n'entre pas dans le cadre de ce livre.

Par contre, avoir l'esprit logique reste un plus indéniable.

1.6. L'anglais, indispensable ?

À vrai dire, pour suivre ce livre, pas besoin de savoir parler anglais. Même si nous nous reporterons régulièrement à de la documentation écrite dans la langue de Shakespeare, nous ne le ferons jamais sans l'accompagner de quelques explications et éclaircissements. Et quand bien même il resterait quelque chose sur lequel vous butiez, vous pourriez vous servir d'un traducteur automatique.

D'un point de vue plus général, si vous souhaitez continuer dans l'informatique et la programmation, il sera très difficile d'échapper à l'anglais. Beaucoup de cours, de documents, de forums sont en anglais ou en font un usage massif. L'anglais est tout simplement *la langue de l'informatique*. Sachez cependant que l'anglais informatique est simple à comprendre, car souvent écrit par des gens dont ce n'est pas la langue maternelle. Ainsi, inutile d'être bilingue, un bon dictionnaire ou un traducteur suffiront.

1.7. La documentation

En programmation, il y a un réflexe à adopter le plus rapidement possible : si on ne sait pas comment utiliser un outil, il faut aller consulter sa documentation, et ce avant de demander de l'aide sur un forum par exemple. Voici un lien vers [une excellente documentation C++](#). Elle est en anglais, mais pas de souci, nous sommes là avec vous. Au fur et à mesure, nous vous donnerons des liens, nous vous expliquerons comment comprendre et exploiter les informations fournies pour que, par la suite, vous puissiez le faire par vous-même.

Il y a aussi un autre outil, très utile pour rechercher une information et que vous connaissez déjà : les moteurs de recherche (Google, Bing, DuckDuckGo, Qwant, etc.). Sachez aussi que les forums sont une mine d'informations. Vous pouvez par exemple utiliser ceux de [Zeste de Savoir](#), en n'oubliant bien évidemment pas d'utiliser le tag [c++] lors de la création d'un sujet.

Enfin, sachez qu'il existe une référence ultime appelée *la norme*, produite par un organisme de validation international appelé *l'International Organization for Standardization* ou ISO, qui explique tous les détails et les règles du C++ mais qui est un document complexe et très largement hors de portée quand on débute. La dernière version de ce document est sortie en 2020 et explique les règles de fonctionnement du C++ dans sa version de 2020. Nous le mentionnons simplement pour que vous soyez au courant de son existence et que vous ne soyez pas surpris si, lors de vos recherches sur Internet, des réponses s'y rapportent ou citent la norme.

1.8. Récapitulons !

- Le C++ a une longue histoire, mais le C++ moderne a véritablement vu le jour en 2011, avec des améliorations et corrections apportées en 2014, 2017 et 2020.
- Le C++ possède des atouts qui en font un des langages les plus utilisés sur la planète.
- En contrepartie, le C++ est un langage assez complexe qui demande de nombreuses années avant d'être maîtrisé.
- La programmation est une activité parfois complexe, mais très enrichissante et accessible.
- Ce livre se veut le plus accessible possible, mais vous devez jouer le jeu et faire des efforts de votre côté.
- Les mathématiques et l'anglais ne sont pas requis pour suivre ce livre, toutefois un minimum de logique et un niveau acceptable en anglais seront extrêmement utiles par la suite.
- La documentation nous aide à mieux comprendre le langage, son fonctionnement, etc. Elle est en anglais, mais est illustrée par des exemples.

Le début du voyage

Nous y voilà, nous rentrons dans le vif du sujet : nous allons apprendre à programmer ! Le but de cette première partie est de vous introduire aux concepts de base de la programmation. Ce que vous y apprendrez vous permettra de faire des programmes assez utiles. Bien entendu, votre apprentissage ne s'arrêtera pas de si tôt.

2

Le minimum pour commencer

Maintenant que nous sommes bien au fait de ce que nous allons apprendre, voyons quelques outils qui nous seront utiles.

2.1. Des outils en ligne

Le C++ est un langage compilé, nous allons donc avoir besoin d'un *compilateur*. Mais des compilateurs C++, il en existe beaucoup. Et vous, lecteurs, n'avez pas tous les mêmes systèmes d'exploitation (Windows, macOS, GNU/Linux, BSD, etc.) ni les mêmes versions (Windows XP, 7, 8, 10, distributions GNU/Linux, etc.).

LANGAGE COMPILÉ ET LANGAGE INTERPRÉTÉ

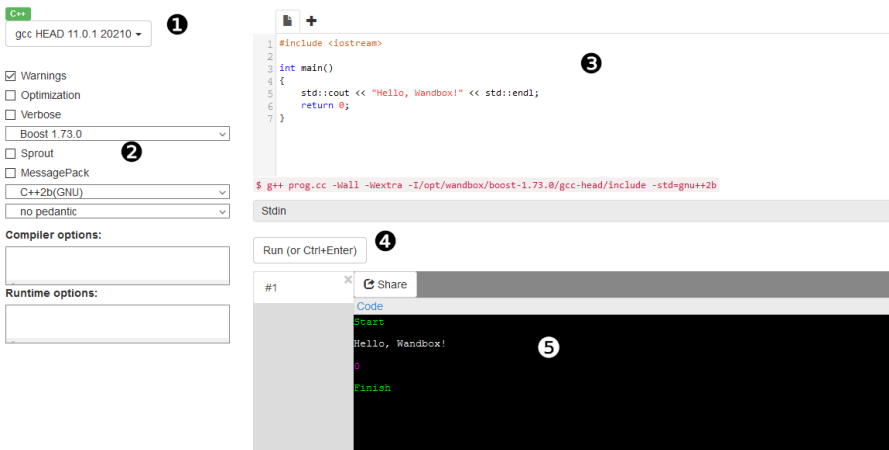
Il existe deux grandes manières d'exécuter un programme à partir de son code source : transformer le code source en code directement compréhensible par la machine à l'aide d'un compilateur, puis l'exécuter, ou alors lire et exécuter directement le code source à l'aide d'un interpréteur. Le C++ est quasi systématiquement compilé, alors que d'autres langages, comme Python, sont la plupart du temps interprétés. Généralement, un langage est utilisé dans un seul des deux modes, donc on parle souvent de langage compilé ou langage interprété, même si c'est techniquement un abus de langage puisqu'un même langage pourrait très bien subir les deux traitements.

Pour un être humain, l'équivalent d'un programme compilé serait de lire une traduction d'un livre d'une langue étrangère, alors qu'un langage interprété reviendrait à acheter la version originale et demander à un interprète de traduire chaque phrase à mesure que l'on avance dans la lecture. Le premier est bien plus rapide pour vous, mais le second a l'avantage de marcher avec n'importe qui, même s'il parle une autre langue que la vôtre ; alors que votre version du roman ne pourra être lue que par vous ou quelqu'un parlant la même langue. De même, les langages compilés sont en général plus rapides que les langages interprétés, mais ont l'inconvénient d'être compilés (traduits) différemment pour chaque plateforme cible, la traduction dépendant notamment du type de machine ainsi que du système d'exploitation.

Aujourd'hui, grâce à Internet, nous pouvons accéder à de nombreuses ressources, dont des compilateurs C++ dans leur dernière version. Le site [Wandbox](#) fournit des compilateurs en ligne pour de nombreux langages, dont deux pour C++ qui sont parmi les plus connus : [Clang](#) et [GCC](#).

Nous sommes conscients que cela demande un accès constant à Internet. La contrepartie est que l'on peut ainsi pratiquer partout (école, travail, ordinateur de quelqu'un d'autre) puisqu'aucune installation n'est requise.

Figure 2.1 : L'interface de Wandbox



- ❶ En haut à gauche, dans un encart vert, c'est *la langue de programmation* utilisé. Si un autre nom que C++ est écrit, pas de souci, il suffit de cliquer sur la liste déroulante en dessous, puis sur C++ et choisir GCC HEAD ou CLANG HEAD (qui sont les toutes dernières versions de ces compilateurs).
- ❷ Un peu plus bas, la liste déroulante commençant par C++ permet de choisir la version du langage. Dans ce livre, nous utiliserons la version C++20, qui correspond à la version 2020 du langage. Que vous choisissiez C++20 ou C++20 (GNU), cela n'a aucune importance à notre niveau.
- ❸ La grande zone de texte blanche est l'endroit où nous écrirons le code.
- ❹ Le bouton RUN (OR CTRL+ENTRÉE) servira à lancer le compilateur et lui faire compiler le code écrit dans la zone précédente.
- ❺ Quand la compilation est terminée, le résultat apparaît ici.

2.2. Des outils plus poussés

Les outils en ligne sont très bien pour des petits codes rapides ou pour faire quelques tests, mais ils sont quand même limités. Or il existe des outils très complets tant pour Windows que pour GNU/Linux. Vous pouvez aussi bien utiliser un ensemble d'outils et de logiciels différents ou bien tout faire au sein d'un IDE (un environnement de développement intégré), à vous de voir ce qui vous conviendra le mieux.

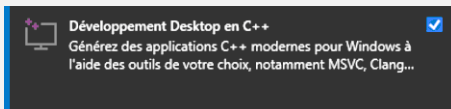
Dans le cadre de ce livre, nous nous contenterons de vous présenter Visual Studio Community (disponible sous Windows 7 et plus et sous macOS) et nous vous indiquerons également certaines manipulations sous [Qt Creator](#), un autre IDE très utilisé et multiplateforme.

VISUAL STUDIO COMMUNITY

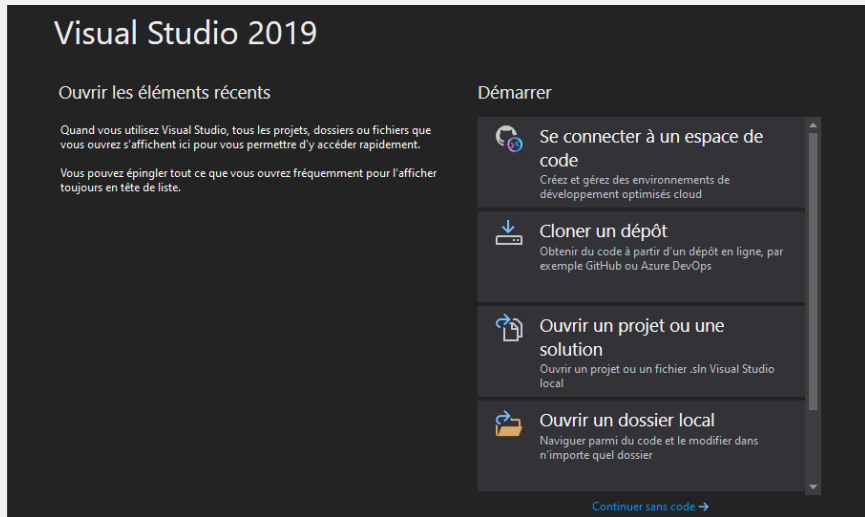
L'IDE par excellence de Microsoft, qui permet de programmer non seulement en C++, mais aussi en Python, en JavaScript, en C#, etc. Configurable, extensible, léger par défaut dans sa version 2019, il est en plus 100% gratuit dans le cadre de l'apprentissage, de la recherche ou des projets open-source. Vous n'aurez donc pas besoin de déboursier quoi que ce soit pour l'utiliser. Inconvénient ? Il n'est disponible que pour Windows (dans sa version 7 au minimum) et macOS.

Pour le télécharger et l'installer, nous vous invitons à suivre [la documentation fournie par Microsoft](#). L'installation demande une connexion internet.

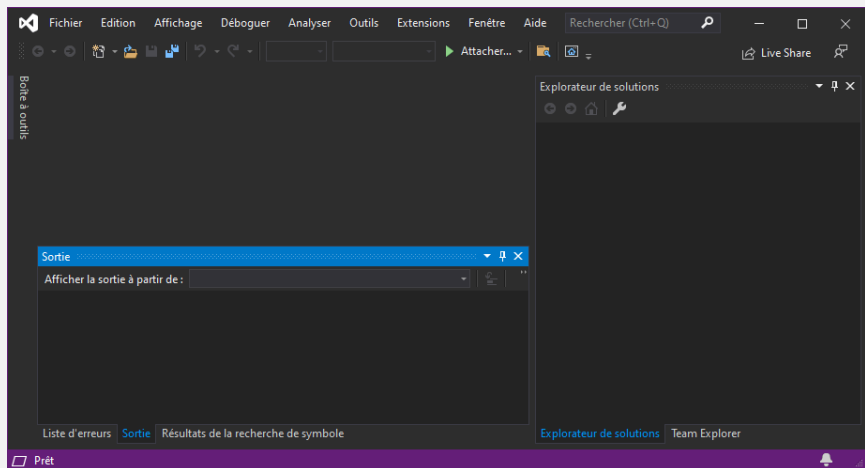
Figure 2.2 : Case C++ cochée



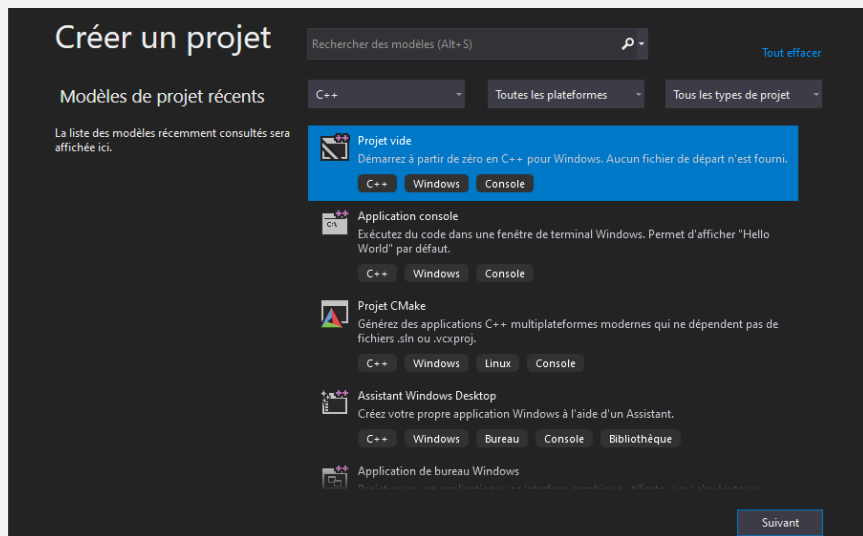
Une fois que Visual Studio est installé pour la première fois et configuré, on arrive sur l'écran d'accueil.

Figure 2.3 : Écran d'accueil de Visual Studio 2019

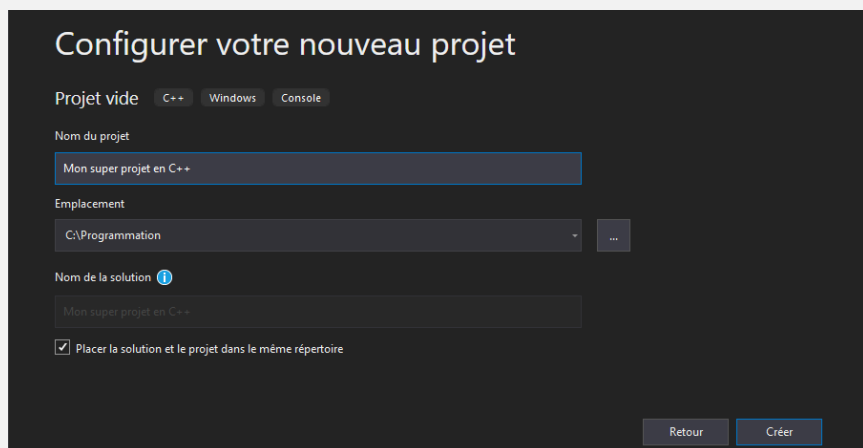
On peut directement créer un nouveau projet ou bien aller à la page principale, vide.

Figure 2.4 : Page principale

Que l'on choisisse de créer directement un nouveau projet ou que l'on clique depuis la page d'accueil sur FICHIER > NOUVEAU > PROJET..., on se retrouve sur le même écran.

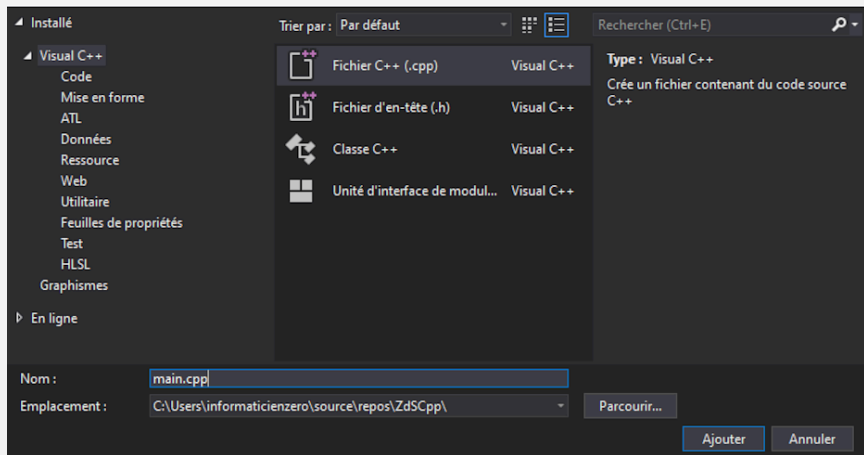
Figure 2.5 : Page de création d'un projet

Dans notre cas, nous allons créer un projet vide. Sélectionnez C++ comme langage et CONSOLE comme type. Sur l'écran suivant, choisissez son nom et son emplacement avant de cliquer sur CRÉER.

Figure 2.6 : Configuration de base du projet

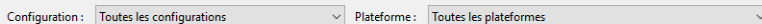
Une fois que votre projet est créé, faites un clic droit sur FICHIERS SOURCES, à droite de l'écran, puis sur AJOUTER > NOUVEL ÉLÉMENT.... Cliquez sur FICHIER C++ (.CPP), nommez le fichier `main.cpp` et validez.

Figure 2.7 : Ajout d'un fichier C++

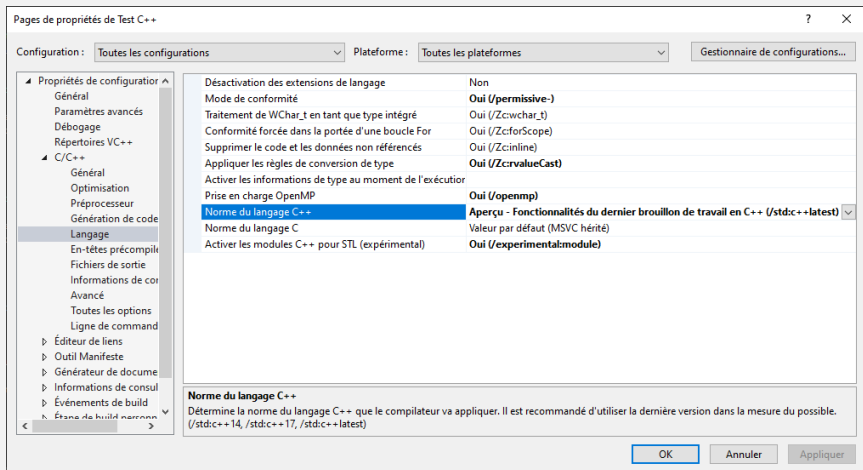


Il ne reste plus qu'à régler quelques paramètres du projet pour utiliser la dernière norme de C++ et nous serons bons. Pour cela, à droite, faites un clic droit sur le nom de votre projet, puis descendez pour cliquer sur PROPRIÉTÉS. Tout en haut, mettez l'option CONFIGURATION à la valeur Toutes les configurations et l'option PLATEFORME à la valeur Toutes les plateformes.

Figure 2.8 : Ce que vous devez avoir comme valeurs en haut de la fenêtre



Dans PROPRIÉTÉS DE CONFIGURATION, déroulez C/C++ puis LANGAGE. À la ligne NORME DU LANGAGE C++, sélectionnez APERÇU > FONCTIONNALITÉS DU DERNIER BROUILLON DE TRAVAIL EN C++ et validez.

Figure 2.9 : Fenêtre de sélection de la norme C++

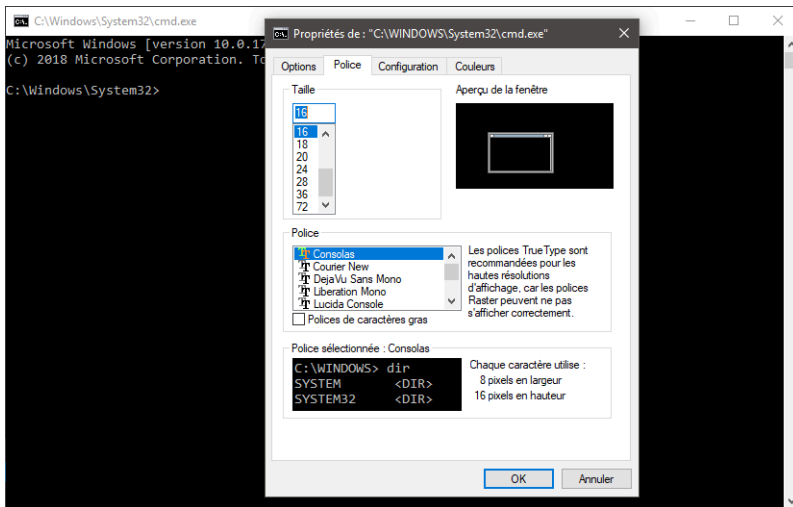
Un deuxième paramètre à modifier se trouve dans la rubrique C/C++ > EN-TÊTES PRÉ-COMPILÉS. À la rubrique EN-TÊTES PRÉ-COMPILÉS, attribuez la valeur sans utiliser les en-têtes précompilés.

Enfin, toujours dans PROPRIÉTÉS DE CONFIGURATION, mais sous la rubrique ÉDITEUR DE LIENS > SYSTÈME, attribuez à la rubrique SOUS-SYSTÈME la valeur Console (/SUBSYSTEM:CONSOLE), puis validez.

2.3. Un mot concernant Windows

Un petit paragraphe pour ceux d'entre vous qui tournent sous Windows, et ce peu importe l'IDE que vous utilisez. Par défaut, la console Windows ne sait pas bien gérer les accents de notre belle langue française. Heureusement, c'est très simple à changer.

Dans le menu DÉMARRER, tapez Cmd et ouvrez l'application. Ensuite, faites un clic droit sur la barre des tâches puis cliquez sur PROPRIÉTÉS, et sous l'onglet POLICE choisissez CONSOLAS ou LUCIDA CONSOLE ainsi que la taille, suivant vos envies.

Figure 2.10 : Police Consolas en 16 pour l'affichage console

2.4. Un mot concernant GNU/Linux

Si vous utilisez GNU/Linux au quotidien, vous êtes certainement familier de la console et connaissez sans doute les outils que nous allons présenter. Si vous avez déjà vos habitudes, vous pouvez ignorer cette section.

Éditeur de texte

Ce choix dépend de vous et peut-être en avez-vous déjà un qui vous convient très bien. Si c'est le cas, parfait, continuez à l'utiliser. Sinon, choisissez un éditeur de texte qui au moins possède la *coloration syntaxique*, c'est-à-dire la capacité de changer la couleur du texte en fonction de ce qu'un mot signifie. Parmi les plus connus, on peut citer [Visual Studio Code](#), [Atom](#) ou encore [Sublime Text](#).

Compilateur

Il en existe plusieurs, mais deux sortent du lot et sont assez populaires dans la communauté C++, il s'agit de [GCC](#) et de [Clang](#). Pour les utiliser, il faut installer soit [g++](#), soit [clang++](#). La commande exacte change en fonction du gestionnaire de paquets de votre distribution préférée.

Une fois ceci fait, reste à vous montrer comment compiler un code C++. La commande suivante est illustrée avec GCC, mais si vous utilisez Clang, il n'y a qu'à changer le compilateur, le reste étant identique.

```
g++ -std=c++20 -Wall -Wextra fichier.cpp -o fichier.out
```

Cette commande demande à GCC de compiler le fichier `fichier.cpp` et de générer le programme final avec `fichier.out` comme nom. L'option `-std=c++20` indique au compilateur qu'on veut utiliser la version 2020 de C++. Quant aux options `-Wall -Wextra`, elles permettent d'activer des vérifications et des avertissements supplémentaires, pour avoir un code encore plus robuste.

Ne reste plus qu'à écrire un code pour le compiler, ce que nous allons justement voir au chapitre suivant.

2.5. Récapitulons !

- Nous avons la possibilité de faire des tests très rapidement avec des outils en ligne, qui ne demandent rien d'autre qu'une connexion internet.
- Nous avons aussi des outils très complets qui nous facilitent la vie, comme Visual Studio 2019. Ils peuvent être utilisés sans connexion internet et font partie des outils les plus utilisés professionnellement, donc vous gagnez à apprendre à les utiliser.
- Pour que la console Windows soit capable d'afficher les accents, il ne faut pas oublier de changer la police.
- Concernant GNU/Linux, si l'on dispose d'un choix énorme d'éditeurs de texte, les deux grands compilateurs majoritairement utilisés aujourd'hui pour C++ sont GCC et Clang.

3

Rencontre avec le C++

Que diriez-vous de rentrer directement dans le vif du sujet [en écrivant votre premier code C++](#) ? Nous allons commencer par l'un des exemples les plus connus et les plus utilisés : [afficher un message](#). Et, vous verrez, il y a déjà de quoi dire.

3.1. Compilons notre premier programme

Allez donc sur [Wandbox](#), ou bien ouvrez votre projet C++ et copiez-collez ce code dans la zone de texte.

```
#include <iostream>

int main()
{
    std::cout << "Hello World !" << std::endl; // Affiche Hello World !
    return 0;
}
```

Cliquez ensuite sur RUN (pour WandBox) [ou faites Ctrl+F5 sur Visual Studio Community, ou encore compilez et exécutez en ligne de commande], et sous vos yeux émerveillés s'affiche le résultat suivant.

```
Hello World !
```

Félicitations, vous venez de compiler votre tout premier programme C++ ! Vous voulez certainement comprendre ce qui s'est passé et avoir des explications sur ce que signifie ce code.

3.2. Démystifions le code

Utiliser des fonctionnalités déjà codées

```
#include <iostream>
```

Le but de notre programme est d'afficher un message. Des développeurs experts ont déjà créé un outil qui permet de le faire facilement. Il se trouve dans un fichier nommé `iostream` (acronyme de *Input Output Stream*, soit Flux d'Entrées Sorties), qui fait partie de la bibliothèque standard C++, un ensemble de fonctionnalités déjà précodées et incluses partout avec chaque compilateur C++. Pour utiliser les fonctionnalités offertes par ce fichier, notamment écrire un message, nous devons l'importer dans notre programme. On dit qu'on l'inclut, d'où l'anglais *include*. Nous recourrons beaucoup à cette pratique tout au long du livre.

Ces fichiers déjà écrits qu'on inclut sont nommés *fichiers d'en-tête*. Nous utiliserons tout d'abord ceux de la bibliothèque standard, puis nous verrons au chapitre [Découpons du code — les fichiers](#) comment écrire les nôtres.

Dernière remarque : avez-vous noté que la ligne commence par le symbole `#` ? C'est ce qu'on appelle une *directive de préprocesseur*. Le préprocesseur est un programme qui se lance automatiquement au début de la compilation, notamment pour importer les fichiers d'en-tête. Nous vous le présenterons plus loin au chapitre [Compilation en cours...](#). Retenez pour l'instant que `#include` nous permet d'importer des fichiers que des programmeurs experts ont déjà écrits avant nous.

Le point d'entrée

```
int main()
{
}

```

Quand l'ordinateur exécute un programme, il lui faut un endroit où commencer, un début, un point d'entrée. À partir de ce point d'entrée, il exécutera des instructions, des ordres, que nous aurons au préalable écrits. Cet endroit s'appelle `main` [prononcé *meïne*] et c'est la fonction principale de tout programme C++.

Qu'est-ce qu'une *fonction* ? C'est un regroupement d'instructions. En C++, vous trouverez ainsi des fonctions pour calculer une racine carrée, comparer des durées, trier une

liste d'objets, afficher des messages, etc. Nous utiliserons d'abord des fonctions déjà existantes puis nous apprendrons à créer les nôtres.

Dans notre cas, la fonction `main` regroupe deux instructions différentes.

Voici mes instructions...

```
std::cout << "Hello World !" << std::endl;
```

La première instruction est justement celle qui nous permet d'afficher un message. Elle débute avec `std::cout` [prononcé ci-aoûte], qui représente la *sortie standard* en C++ et permet la communication du programme vers l'utilisateur, via la console dans notre cas. Le préfixe `std::` indique que vous utilisez une fonctionnalité déjà programmée et fournie dans la bibliothèque standard C++.

On note ensuite la présence de deux chevrons `<<` qui permettent d'injecter, d'envoyer, du contenu dans la sortie standard, dans `std::cout`. Dans notre code d'introduction, nous injectons deux choses :

- `"Hello World !"`, qui est le message que l'on voit s'afficher à l'écran ;
- `std::endl`, qui permet d'aller à la ligne comme son nom l'indique [*end line*, soit retour à la ligne]. De même que pour `std::cout`, `std::endl` est fourni par la bibliothèque standard.

Enfin, elle se termine par un point-virgule `;` qui est OBLIGATOIRE. Chaque instruction doit être terminée par un point-virgule. Ce sont les règles du C++ qui veulent ça.

- **3.1.** On peut cumuler les chevrons `<<` autant de fois que l'on souhaite. Essayez donc d'afficher la phrase "Bonjour, j'écris du C++" suivi d'un retour à la ligne, suivi de "Et j'aime beaucoup."

✧ [Voir la solution](#)

UN MOT SUR WINDOWS

Si vous êtes sous Windows et que vous tentez d'afficher du texte avec des accents, vous allez obtenir des caractères bizarres.

Malheureusement, la gestion des accents sur ce système d'exploitation est compliquée et la seule solution que nous avons à vous offrir est de vous passer des accents et autres caractères spéciaux. Tant pis pour la beauté de la langue française. Dans le cadre professionnel, l'anglais est majoritairement utilisé, donc c'est sans conséquence.

Sous GNU/Linux et macOS, vous n'aurez aucun problème.

```
return 0;
```

La deuxième instruction conclut la fonction `main` en renvoyant la valeur `0`. Le mot anglais pour renvoyer est *return* ; vous croiserez d'ailleurs souvent l'anglicisme retourner. Mais à qui est transmise cette valeur ? Au système d'exploitation. Le zéro, par convention, signifie que tout s'est bien passé.

Nous aurons l'occasion de comprendre mieux tout cela quand nous aborderons le chapitre sur les fonctions. Ne vous souciez donc pas de `return 0;`, sachez simplement qu'il termine la fonction `main`.

3.3. Les commentaires

Cela ne vous aura pas échappé, nous avons omis de traiter un bout de code. En effet, reste cette partie bizarre avec des slashes.

```
// Affiche Hello World !
```

C'est ce qu'on appelle un *commentaire*. Les commentaires sont complètement ignorés par le compilateur, ils ne servent qu'aux humains. On s'en sert pour documenter son code, expliquer des passages un peu ardu, décrire des passages un peu moins lisibles ou tout simplement pour offrir quelques compléments d'information.

En effet, un code est bien plus souvent lu qu'écrit ou modifié. C'est pourquoi il est important qu'il soit lisible. Les commentaires aident à gagner en lisibilité.

Enfin, contrairement au code qui contient beaucoup de mots anglais, les commentaires peuvent être écrits dans la langue que l'on souhaite. Si beaucoup de projets collaboratifs ont des commentaires écrits en anglais, dans ce livre, nous les écrivons en français afin de vous aider à bien comprendre.

CONCIS OU VOLUBILE ?

On peut écrire des commentaires de deux façons. La première, c'est celle que nous avons vue avec les deux slashes `//`. Un commentaire ainsi créé s'arrête à la fin de la ligne où il est créé. On dit que c'est un *commentaire monoligne*.

```
// Cette ligne affiche Bonjour.
std::cout << "Bonjour";

// Cette ligne permet d'afficher un retour à la ligne.
std::cout << std::endl;
```

L'autre syntaxe permet d'écrire des blocs de commentaires bien délimités entre les symboles `/*` et `*/`. On peut ainsi écrire des commentaires sur plusieurs lignes, voire entre plusieurs instructions !

```
/* Ceci est un commentaire sur une seule ligne. */
std::cout << "Hello World !" << std::endl; /* Ceci est un commentaire
à la fin d'une ligne du code. */

/* Ceci...
...est un commentaire
écrit sur plusieurs lignes...
...car il est très long. */

std::/* Ceci est un commentaire noyé dans le code. */cout << "Hello
World !";
```

Vu que les commentaires sont ignorés par le compilateur, on peut tout à fait en écrire en plein milieu d'une ligne de code. Mais ce n'est pas lisible. Aussi, nous vous déconseillons fortement de le faire, car un code est, on le rappelle, plus souvent lu qu'écrit.

3.4. Récapitulons !

- Un programme en C++ est composé d'instructions, qui sont les ordres que l'on donne à l'ordinateur.
- Ces instructions sont regroupées dans des blocs appelés fonctions. Celle par laquelle commence tout programme C++ s'appelle `main`.
- Nous avons utilisé la bibliothèque standard pour la première fois, avec le fichier `iostream`, afin d'afficher un message à l'écran.
- Les commentaires sont ignorés par le compilateur mais nous sont très utiles à nous, humains.

SOLUTION

Q : 3.1

```
cpp
#include <iostream>

int main()
{
    std::cout << "Bonjour, j'écris du C++" << std::endl
               << "Et j'aime beaucoup.";

    return 0;
}
```

4

Tout ça est bien variable

Vous savez désormais afficher des messages à l'écran. C'est un bon début, mais on est quand même assez limités, n'est-ce pas ? Rassurez-vous, C++ permet de faire bien plus que ça. Dans ce chapitre, nous allons découvrir les littéraux et les variables.

4.1. Les littéraux

Que sont les littéraux ? Une valeur écrite littéralement dans le code, d'où son nom. Nous avons rencontré un littéral dans le chapitre précédent : `"Hello world !"`. Ce littéral est ce qu'on appelle une *chaîne de caractères*. Mais ce n'est pas le seul littéral possible en C++. Examinons tout ça.

Les caractères

Nous avons fait connaissance avec les chaînes de caractères lors du chapitre précédent. Les chaînes de caractères ne sont rien d'autre que du texte. On les reconnaît parce qu'elles commencent et finissent par des doubles guillemets `"`.

- **4.1.** Entraînez-vous donc à afficher des chaînes de caractères, par exemple un littéral souhaitant une bonne journée.

✧ [Voir la solution](#)

Tout comme les mots de la langue française sont des regroupements de lettres, les chaînes de caractères sont une suite de caractères simples. En C++, un caractère est encadré par des guillemets simples `'`.

```
#include <iostream>

int main()
{
    // Un caractère peut être une lettre.
    std::cout << 'A' << std::endl;
    // Ou bien un chiffre.
    std::cout << '7' << std::endl;
    // Ou même de la ponctuation.
```



```
std::cout << '!' << std::endl;

return 0;
}
```

Les chaînes de caractères permettent de regrouper des caractères et nous facilitent ainsi la vie pour écrire du texte.

```
#include <iostream>

int main()
{
    // Que c'est fastidieux d'écrire ça !
    std::cout << 'S' << 'a'
               << 'l' << 'u' << 't'
               << ' ' << 't' << 'o'
               << 'i' << ' ' << '!'
               << std::endl;

    // C'est tellement mieux comme ça.
    std::cout << "Salut toi !" << std::endl;

    return 0;
}
```



J'ai mis plusieurs caractères entre guillemets simples, comme ceci 'abc' et mon code compile. Ça veut dire que ça marche ?

Si vous avez essayé de compiler, vous avez dû remarquer un message en rouge dans la fenêtre de résultat disant **warning: multi-character character constant**, ainsi qu'un nombre au lieu de nos caractères.

Le **warning** est un message d'avertissement que le compilateur vous envoie, car il ne sait pas si ce que vous avez fait est une erreur d'inattention ou bien une manœuvre volontaire de votre part. Dans notre cas, la norme C++ n'interdit pas de faire ça, mais ce n'est pas considéré comme une bonne pratique, comme étant du bon code.

Attention > N'ignorez pas les warnings ! Les warnings sont des messages importants signalant d'éventuels problèmes. Il ne faut surtout pas les ignorer sous prétexte que le code compile !

Les caractères spéciaux

- **4.2.** Il existe quelques caractères qui sont un peu particuliers et, pour vous les introduire, vous allez afficher un message contenant un chemin de dossier Windows [`C:\Program Files (x86)` par exemple].

✧ Voir la solution

Boum ba da boum ! Le compilateur vous crache à la figure un message de type `warning: unknown escape sequence: '\P'`. Que s'est-il passé ?

Il existe en C++ des séries de caractères, appelées *séquences d'échappement*, qui commencent toutes par `\` et dont vous trouverez la liste complète dans le [manuel de référence](#). Et pour comprendre l'intérêt de ces séquences d'échappement, essayez donc de compiler le code suivant.

```
#include <iostream>

int main()
{
    std::cout << "Il m'a demandé "Comment vas-tu ?" << std::endl;
    return 0;
}
```

Vous devez certainement avoir une erreur proche de celle-ci : `'error: expected ';' before 'Comment'`. C'est tout à fait normal. En effet, nous avons vu plus haut que les chaînes de caractères sont délimitées par les doubles guillemets `"`. Donc tout ce qui est entre cette paire de guillemets est considéré par le compilateur comme faisant partie de la chaîne de caractères.

Dans notre exemple, cela veut dire que `Comment vas-tu ?` ne fait plus partie de la chaîne. Donc le compilateur l'interprète comme des instructions C++ et comme il ne les comprend pas, il ne peut pas compiler le programme.



Mais quel rapport avec ces fameux caractères commençant par `\` ?

Eh bien, les séquences d'échappement permettent de dire au compilateur *Écoute, ce caractère est spécial, il faut l'afficher et non l'interpréter*, ce qui permet d'afficher des doubles guillemets dans une chaîne de caractères par exemple.

```
#include <iostream>

int main()
{
    std::cout << "Il m'a demandé \"Comment vas-tu ?\"" << std::endl;
    std::cout << "Dossier principal : C:\\Program Files (x86)"
              << std::endl;

    return 0;
}
```

De toutes les séquences d'échappement qui existent, les plus utilisées et celles que vous verrez le plus souvent sont les suivantes :

- `\'` qui permet d'afficher un guillemet simple `'` ;
- `\"` qui permet d'afficher un guillemet double `"` ;
- `\n` qui permet d'aller à la ligne, comme `std::endl` ;
- `\t` qui permet de faire une tabulation horizontale ;
- `\\` qui permet d'afficher un antislash `\`.

► **4.3.** Entraînez-vous donc en affichant un guillemet simple sans échappement, puis avec échappement. Et amusez-vous avec les tabulations et les retours à la ligne.

✧ [Voir la solution](#)

Les nombres

Il existe d'autres types de littéraux en C++ : les nombres entiers et les nombres à virgule, appelés *flottants*.

```
std::cout << -1 << std::endl;
std::cout << 0 << std::endl;
std::cout << 1 << std::endl;

std::cout << -1.6027 << std::endl;
std::cout << 3.14159 << std::endl;
std::cout << 2.71828 << std::endl;
```

Vous remarquerez qu'on peut utiliser des nombres négatifs sans aucun problème. Ensuite, dû à l'origine américaine de C++, les flottants s'écrivent avec un point et non une virgule. Même si l'on se fiche des chiffres après la virgule, il faut mettre le point `.` sinon C++ interprétera le nombre comme un entier.

Sinon, rien de bien étonnant, ce sont des nombres et on peut faire des opérations dessus.

```
#include <iostream>

int main()
{
    std::cout << "1 + 2 = " << 1 + 2 << std::endl;
    std::cout << "1 - 4 = " << 1 - 4 << std::endl;
    std::cout << "7.5 * 2 = " << 7.5 * 2 << std::endl;

    std::cout << "9 / 2 = " << 9 / 2 << std::endl;
    std::cout << "9. / 2 = " << 9. / 2 << std::endl;
    std::cout << "9 % 2 = " << 9 % 2 << std::endl;

    return 0;
}
```



Attends, qu'est-ce que c'est que ça ? $9/2$ et $9./2$ ne donne pas la même chose ? Et c'est quoi % ?

En fait, la raison est très simple : pour C++, si on fait une opération sur deux nombres entiers, le résultat est un nombre entier. Si l'on veut que le résultat soit un nombre à virgule, il faut qu'au moins un des deux nombres soit un flottant.

Dans le cas de la division, si les deux nombres sont des entiers, on obtient le quotient de la division euclidienne, le reste s'obtenant avec l'opérateur %, appelé *modulo* (c'est de l'[arithmétique](#)). Si l'un des deux nombres est un flottant, alors on obtient le résultat de la division réelle, donc un nombre réel.

Également, comme en maths, les calculs respectent la [distributivité](#), l'[associativité](#), la [commutativité](#) et la [priorité des opérateurs](#). L'exemple suivant a été fourni par Guillaume Belz.

```
#include <iostream>

int main()
{
    std::cout << "Commutativité :" << std::endl;
    std::cout << "2 + 3 = " << 2 + 3 << std::endl;
    std::cout << "3 + 2 = " << 3 + 2 << std::endl;

    std::cout << "Associativité :" << std::endl;
    std::cout << "2 + (3 + 4) = " << 2 + (3 + 4) << std::endl;
    std::cout << "(2 + 3) + 4 = " << (2 + 3) + 4 << std::endl;

    std::cout << "Distributivité :" << std::endl;
    std::cout << "2 * (4 + 3) = " << 2 * (4 + 3) << std::endl;
```

```
std::cout << "2 * 4 + 2 * 3 = " << 2 * 4 + 2 * 3 << std::endl;

std::cout << "Priorité des opérateurs :" << std::endl;
std::cout << "2 * 6 + 3 = " << 2 * 6 + 3 << std::endl;
std::cout << "2 * (6 + 3) = " << 2 * (6 + 3) << std::endl;
std::cout << "2 * 4 - 6 / 2 = " << 2 * 4 - 6 / 2 << std::endl;

return 0;
}
```

Nous nous sommes amusés avec les littéraux, mais on est quand même rapidement limités. Imaginons que nous voulions multiplier le résultat précédent par 2 et ce, trois fois. Nous sommes obligés d'écrire un code comme ceci.

```
#include <iostream>

int main()
{
    std::cout << "1 * 2 = " << 1 * 2 << std::endl;
    std::cout << "1 * 2 * 2 = " << 1 * 2 * 2 << std::endl;
    std::cout << "1 * 2 * 2 * 2 = " << 1 * 2 * 2 * 2 << std::endl;

    return 0;
}
```

Maintenant, imaginons qu'on ne multiplie plus par 2 mais par 4, et non plus trois fois mais cinq. Vous visualisez bien tous les changements à faire ? C'est pénible, n'est-ce pas ? N'y a-t-il pas un moyen de se souvenir de valeurs et de calculs ?

4.2. Les variables

La réponse à la question soulevée dans la section précédente se trouve dans le titre. Il s'agit des *variables*. C'est un concept commun à beaucoup de langages de programmation qui permet de *stocker une valeur* et de *lui associer un nom*, afin de faciliter tant l'écriture que la lecture du code. On peut les voir comme des enveloppes, des tiroirs, des conteneurs, bref, une zone où est stockée une valeur et à laquelle on associe un nom.



Elle est où, cette zone de stockage ?

Toutes les variables sont stockées dans la mémoire vive de l'ordinateur, [la RAM](#). Son fonctionnement est un peu complexe et ne nous intéresse pas (dans le cadre de ce livre tout du moins). Les curieux trouveront beaucoup de ressources sur Internet pour combler leur soif de savoir, allant de la [vulgarisation](#) simple à des [explications complètes](#).

Comment créer des variables en C++ ?

Pour déclarer une variable en C++, il faut trois éléments obligatoires :

- un *type*, qui indique ce que la variable va stocker (entier, chaîne de caractères, etc.) ;
- un *identificateur*, c'est-à-dire le nom associé à la variable ;
- une *valeur*, celle qu'on donne à stocker à notre variable. Pour cela, on la met entre accolades { }.

Examinons un cas réel que nous détaillerons.

```
#include <iostream>
#include <string>

int main()
{
    int reponse { 42 };
    std::cout << "La réponse à la Grande Question est "
              << reponse << std::endl;

    double pi { 3.1415926 };
    std::cout << "Voici la valeur du célèbre nombre pi : "
              << pi << std::endl;

    char lettre { 'A' };
    std::cout << "La première lettre de l'alphabet français est "
              << lettre << std::endl;

    std::string phrase { "Bonjour tout le monde !" };
    std::cout << "En entrant dans la salle, il s'écria : "
              << phrase << std::endl;

    return 0;
}
```

```
La réponse à la Grande Question est 42
Voici la valeur du célèbre nombre pi : 3.14159
La première lettre de l'alphabet français est A
En entrant dans la salle, il s'écria : Bonjour tout le monde !
```

SYNTAXE HÉRITÉE

Il existe une syntaxe alternative, de la forme `type identificateur = valeur;`. Essayez, vous verrez que ça marche.

```
#include <iostream>

int main()
{
    int variable = 42;
    std::cout << variable << std::endl;

    return 0;
}
```

Cette syntaxe est héritée du C. Elle est toujours valable en C++, ne soyez donc pas surpris si vous la croisez. Dans ce livre, nous utiliserons la forme dite *unifiée d'initialisation* utilisant les accolades `{}`.

C'est tout à fait mon type

Vous l'aurez remarqué, les types qui précèdent nos variables sont précisément les littéraux que nous avons introduits en début de chapitre :

- `int` [de l'anglais *integer* signifiant ... nombre entier] pour les *nombre entiers*. Ce type permet de stocker des entiers négatifs ou positifs ;
- `double` pour les *flottants* (les nombres à virgule), qui permet de stocker des nombres réels très grands ;
- `char` pour les *caractères simples*, qui est tellement transparent que nous ne vous ferons pas l'affront de le traduire ;
- `std::string` pour les *chaînes de caractères*. Ce type est un peu particulier car il n'existe pas nativement en C++. Ce sont des programmeurs experts qui l'ont codé afin de manipuler aisément des chaînes de caractères. Afin de pouvoir manipuler des `std::string`, il faut donc inclure le bon fichier, ce que l'on fait grâce à la ligne `#include <string>`.

Quel est ton nom, petite variable ?

Et si vous êtes un peu pointilleux sur la langue française, vous aurez également remarqué qu'il manque l'accent aigu dans l'identificateur `reponse`. Ce n'est pas un oubli de notre part. Inventé aux États-Unis dans les années 1980, C++ *n'autorise que les 26 lettres composant l'alphabet anglais* (plus les chiffres et l'underscore `_`), à savoir donc les mêmes que les nôtres, accents non compris. Dans la pratique, les programmeurs utilisent l'anglais pour nommer leurs variables, ce n'est donc pas un problème.

Note > Nous n'utilisons le français que dans le cadre de votre apprentissage. Quand vous aurez plus d'expérience, nous vous encouragerons à utiliser l'anglais.

Mais que ce soit en anglais ou en français, un identificateur doit respecter des règles bien précises :

- *débuter par une lettre* : il ne peut pas commencer par un chiffre, c'est interdit. Ni par un underscore `_` : son utilisation répond à des règles précises, donc il est plus simple de ne pas l'employer comme premier caractère ;
- *ne contenir ni espace ni signes de ponctuation* (`'`, `?`, etc.) ;
- *ne pas porter le même nom qu'un mot clé* : il est interdit de déclarer une variable s'appelant `int` ou `return`, par exemple.

```
#include <iostream>

int main()
{
    // Bon exemple.
    int avec_underscore { 0 };
    // Erreur : espace interdit.
    int avec espace { 0 };

    // Bon exemple.
    int variable1 { 42 };
    // Erreur : ne peut pas commencer par un chiffre.
    int 1variable { 42 };

    // Bon exemple.
    char lettre { 'A' };
    // Erreur : ponctuation interdite.
    char autre_lettre! { 'B' };

    // Bon exemple.
    double retour { 2.71 };
    // Erreur : mot clé réservé par C++.
    double return { 2.71 };

    return 0;
}
```

De l'importance d'un beau nom

Donner un nom clair et précis à une variable est un véritable défi même pour les programmeurs ayant de l'expérience. En effet, même si un identificateur respecte les règles imposées par C++, cela ne veut pas dire pour autant que la variable est bien nommée.

- `variable` : ne veut rien dire. Que stocke-t-elle ? Quel est son but ? Pourquoi existe-t-elle ?

- `variable_contenant_la_multiplication_de_pi_par_e_au_cube` : beaucoup trop long.
- `cIrCoNfErEnCe_CeRcLe` : le nom définit bien le pourquoi de l'existence de notre variable, mais le mélange majuscules/minuscules le rend illisible.
- `Brzeczyszczkiewicz` : aucun sens, sauf si vous aimez l'humour polonais.

Avec le temps, en progressant, vous arriverez à trouver plus rapidement et plus facilement des identificateurs qui soient clairs et simples. Dans ce livre, nous nous efforcerons de le faire afin de vous donner des exemples concrets.

Sinon, par défaut...

Abordons un dernier point. Nous vous avons dit qu'on donne une valeur entre accolades pour initialiser notre variable. Mais que se passe-t-il si on écrit simplement les accolades, sans aucune valeur dedans ? Le code va-t-il toujours compiler ?

```
#include <iostream>
#include <string>

int main()
{
    int entier {};
    std::cout << "Mon entier vaut " << entier << std::endl;

    double reel {};
    std::cout << "Mon réel vaut " << reel << std::endl;

    char lettre {};
    std::cout << "Mon caractère vaut " << lettre << std::endl;

    std::string phrase {};
    std::cout << "Ma chaîne vaut " << phrase << std::endl;

    return 0;
}
```

```
Mon entier vaut 0
Mon réel vaut 0
Mon caractère vaut
Ma chaîne vaut
```

Non, le programme n'a pas de *bugs*, il fonctionne très bien. En fait, si aucune valeur n'est spécifiée, nos variables sont initialisées à une valeur par défaut. Pour les entiers et les réels, il s'agit de zéro. Pour les caractères, c'est une valeur spéciale signifiant pas de caractère, de même que pour les chaînes de caractères, initialisée avec du vide.

***Note** > C++ vous laisse libre de vos choix. Dans ce livre, pour commencer, nous écrirons toujours les valeurs explicitement. Au fur et à mesure que vous progresserez, nous utiliserons de plus en plus souvent l'initialisation par défaut.*

Un peu de constance, voyons !

Terminons cette partie en présentant un mot clé important qui a pour nom `const`. Il permet d'empêcher toute modification de la variable sur laquelle ce mot clé s'applique. On dit qu'on crée une *constante*. Et si l'on essaye de modifier une constante, le compilateur refuse clair et net.

```
int main()
{
    double const pi { 3.141592 };
    pi = 4; // Ouch, badaboum ça ne compile pas.

    return 0;
}
```

```
[Visual Studio]
C3892 'pi' : vous ne pouvez pas assigner une variable const.

-----

[GCC]
prog.cc: In function 'int main()':
prog.cc:4:8: error: assignment of read-only variable 'pi'
   4 |     pi = 4; // Ouch, badaboum ça ne compile pas.
     |     ~~~^~~

-----

[Clang]
prog.cc:4:8: error: cannot assign to variable 'pi' with const-qualified
type 'const double'
   pi = 4; // Ouch, badaboum ça ne compile pas.
   ~ ~ ^
prog.cc:3:18: note: variable 'pi' declared const here
double const pi { 3.141592 };
~~~~~^~~~~~
1 error generated.
```

Certains choisissent de les écrire entièrement en majuscules pour bien les différencier des autres variables. C'est une pratique héritée du C et qui n'a nullement un caractère obligatoire. Nous n'imposons sur ce point aucune règle.

Quelle en est l'utilité ? Il sert pour la qualité du code. Tout ce qui n'est pas destiné à être modifié est ainsi protégé. Cela nous permet également de donner des garanties fortes à

notre code. C'est pour ça que nous allons l'utiliser dans tous les exemples. Vous verrez d'autres cas utiles plus loin dans le livre.

***Note** > Vous verrez souvent des codes qui inversent l'ordre de `const` et écrivent `const int constante {}`; . Cela est autorisé en C++, car `const` respecte une règle simple : il s'applique à ce qui est à sa gauche immédiate, sauf s'il n'y a rien, auquel cas il s'applique à ce qu'il y a à droite. Ici, cela ne change rien, mais plus tard cela aura des conséquences. Nous y reviendrons.*

Manipulation de variables

Personne n'a dit que nous devions nous contenter d'afficher nos variables. Au contraire, nous pouvons faire beaucoup d'opérations dessus, notamment les combiner entre elles et modifier leur valeur.

```
#include <iostream>

int main()
{
    int entier { 4 };
    std::cout << "Mon entier vaut : "
               << entier << std::endl;

    // On peut tout à fait changer la valeur de la variable.
    entier = 4 * (8 + 9) - 1;
    std::cout << "Finalement non, il vaut : "
               << entier << std::endl;

    // On peut même utiliser la valeur de la variable
    // et la réaffecter à la même variable.
    entier = entier + 7;
    std::cout << "Et si j'additionne 7 ? "
               << entier << std::endl;

    int autre_entier { entier * 2 };
    // On peut utiliser d'autres variables également.
    entier = autre_entier - 4;
    std::cout << "Finalement, je change de valeur : "
               << entier << std::endl;

    return 0;
}
```

Pour modifier la valeur d'une variable, on utilise l'opérateur d'affectation `=`, précédé de l'identificateur de la variable et suivi de la valeur à affecter : `identificateur = valeur`; . Et comme vous l'avez noté, cette valeur peut être un simple littéral, un calcul plus complexe, une autre variable voire la variable actuelle elle-même.

En effet, C++ s'occupe d'abord de tout ce qui se trouve à droite du signe égal. Ainsi, dans la ligne `entier = entier + 7`, il récupère l'ancienne valeur de `entier`, lui ajoute 7 et assigne ce nouveau résultat à `entier`. Essayez donc pour voir.

```
#include <iostream>

int main()
{
    int entier { 4 };
    std::cout << "Mon entier vaut : " << entier << std::endl;

    entier = entier + 2;
    std::cout << "Mon entier vaut : " << entier << std::endl;

    entier = entier - 2;
    std::cout << "Mon entier vaut : " << entier << std::endl;

    entier = entier * 2;
    std::cout << "Mon entier vaut : " << entier << std::endl;

    entier = entier / 2;
    std::cout << "Mon entier vaut : " << entier << std::endl;

    return 0;
}
```

- **4.4.** Vous souvenez-vous de l'exemple que nous avons vu à la [fin de la section sur les littéraux](#) ? Nous faisons des multiplications, mais le fait de devoir répéter de nombreuses fois les mêmes instructions était lourd. Tenez, entraînez-vous donc en faisant un programme qui utilise les variables.

✧ [Voir la solution](#)

PRENONS UN RACCOURCI

Les programmeurs étant fainéants par nature, ils n'aiment pas la répétition. Ils ont donc créé des raccourcis pour aller plus vite. Ainsi, plutôt que d'écrire `a = a + b`, on peut écrire directement `a += b`. Cette nouvelle syntaxe s'applique aux opérateurs d'addition `+`, de soustraction `-`, de multiplication `*`, de division `/` et de modulo `%`.

```
#include <iostream>

int main()
{
    int entier { 4 };
    std::cout << "Mon entier vaut : " << entier << std::endl;
```

```

entier += 2;
std::cout << "Mon entier vaut : " << entier << std::endl;

entier -= 2;
std::cout << "Mon entier vaut : " << entier << std::endl;

entier *= 2;
std::cout << "Mon entier vaut : " << entier << std::endl;

entier /= 2;
std::cout << "Mon entier vaut : " << entier << std::endl;

entier %= 2;
std::cout << "Mon entier vaut : " << entier << std::endl;

return 0;
}

```

Pire, non contents de ça, ils ont raccourci encore la syntaxe dans le cas où l'on souhaite ajouter ou soustraire 1 à un nombre. On parle d'*incréméntation* et de *décréméntation*. Il suffit de faire :

- `a++` ou `++a` pour incrémenter de 1 la valeur de la variable ;
- `a--` ou `--a` pour décrémenter de 1 la valeur de la variable.

Les deux syntaxes sont quasiment équivalentes, subsiste une différence mineure que nous passons sous silence car elle ne nous intéresse pas à l'heure actuelle.

4.3. Simple déduction ?

Lorsque nous déclarons une variable, comme vous le savez maintenant, nous précisons, entre autres, son type et sa valeur. Sauf que cette information est parfois redondante. Regardez le code d'illustration ci-dessous.

```

int main()
{
    // Comme 0 est déjà un littéral entier,
    // pourquoi donc préciser 'int' ?
    int entier { 0 };

    // Comme 3.1415 est déjà un littéral flottant,
    // pourquoi donc préciser 'double' ?
    double reel { 3.1415 };

    return 0;
}

```

Le compilateur peut en effet déduire tout seul le type d'une variable et cela grâce à un nouveau mot clé : `auto`. Celui-ci s'utilise en lieu et place du type de la variable. C'est ce qu'on appelle *l'inférence de type*.

Notre code précédent peut ainsi être réécrit de la manière qui suit.

```
int main()
{
    // Hop, le compilateur déduit que 'entier' est de type 'int'.
    auto entier { 0 };

    // Hop, le compilateur déduit que 'reel' est de type 'double'.
    auto reel { 3.1415 };

    return 0;
}
```

Avec `const`

Tout comme dans le cas d'une variable dont le type est explicitement écrit, si vous voulez déclarer une constante, il suffit de rajouter `const`.

```
int main()
{
    auto const constante { 40 };
    return 0;
}
```

Avec `std::string`

Dans le cas des chaînes de caractères, c'est un peu particulier. Déjà, il faut rajouter un `s` après les guillemets fermants `"`, et ce pour chaque chaîne de caractères que vous voulez écrire. Ensuite, il faut rajouter une ligne spéciale : `using namespace std::literals;`. L'exemple suivant va vous aider à comprendre.

```
#include <string>

int main()
{
    // Écrivez cette ligne une seule fois.
    using namespace std::literals;

    // Puis vous pouvez déclarer autant de
    // chaînes de caractères que vous voulez.
    auto chaine { "Du texte."s };
    auto autre_chaine { "Texte alternatif."s };
```

```
auto encore_une_chaine { "Allez, un dernier pour la route."s };

return 0;
}
```

Note > Si nous sommes obligés d'ajouter ce `s` en fin de littéral, c'est parce que nous avons affaire ici à un héritage du C. Si vous l'omettez, le compilateur déduit que notre littéral est une chaîne de caractères comme en C et non comme étant une `std::string`.

Quel intérêt ?

Utiliser `auto` présente deux avantages majeurs :

- il permet de raccourcir le code. Ici, remplacer `double` par `auto` n'est pas un gain de place énorme, mais, plus loin dans ce livre, nous verrons des types longs à écrire et lourds à lire, que nous serons bien contents de remplacer par ce très petit `auto` ;
- il permet un code plus évolutif. Imaginez des calculs complexes avec des types écrits explicitement. Si vous voulez changer l'un des types, il faut en changer chaque occurrence. Avec `auto`, vous laissez au compilateur le soin de s'occuper de cette tâche.



Je suis perdu. Qu'est-ce que je dois choisir entre `auto` ou le type explicite ?

C++ vous autorise à faire les deux, donc c'est à votre convenance. Certains programmeurs ne jurent que par `auto` et l'utilisent partout. D'autres préfèrent le réserver aux types longs ou complexes et écrire explicitement les types simples et courts. Dans le cadre de ce livre, nous utiliserons les deux façons de faire.

4.4. Les entrées

Dès le premier code C++ que vous avez étudié, il y avait la notion de sortie standard. Il est temps de voir le concept inverse en manipulant l'entrée standard. Maintenant que nous connaissons le concept des variables, demander des informations à l'utilisateur est à notre portée. Il faut en effet pouvoir stocker quelque part la valeur qu'a tapée l'utilisateur, ce que permettent justement les variables.

Manipuler les entrées est un peu particulier avec Wandbox. Il faut cliquer sur l'encadré STDIN et saisir la valeur que vous souhaitez rentrer. C'est moins pratique qu'avec un IDE. Peu importe ce que vous préférez, l'essentiel est que vous puissiez manipuler pour comprendre la suite du chapitre.

```
#include <iostream>

int main()
{
    std::cout << "Entre ton age : " << std::endl;
    int age { 0 };
    std::cin >> age;
    std::cout << "Tu as " << age << " ans.\n";

    return 0;
}
```

Avez-vous remarqué à quel point `cin` est semblable à `cout` ? Déjà, pour l'utiliser, il faut le préfixer par `std::`, car `cin` est un objet appartenant à la bibliothèque standard. Et on utilise les chevrons dans le sens inverse de `cout`.

GESTION DES ERREURS

Et si vous rentrez une valeur qui n'a rien à voir ? Que se passe-t-il si on décide de taper `Maurice` au lieu de `5` ? Essayez donc.

```
Entre ton age :
Maurice
Tu as 0 ans.
Appuyez sur une touche pour continuer...
```

Comme on veut récupérer un entier et qu'on a écrit une chaîne de caractères, notre variable `age` n'est pas modifiée et vaut donc toujours `0`, sa valeur par défaut.



Donc c'est bon alors ? Y'a pas de soucis particulier, puisque `std::cin` ignore nos bêtises.

Eh bien nous allons voir que si. Essayons donc de demander deux informations à l'utilisateur : son âge et son nom.

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Entre ton age : ";
    int age { 0 };
    std::cin >> age;
    std::cout << "Tu as " << age << " ans.\n";
}
```



```

std::cout << "Entre ton nom : ";
std::string nom { "" };
std::cin >> nom;
std::cout << "Tu t'appelles " << nom << ".\n";

return 0;
}

```

Lancez ce code en faisant exprès de taper une chaîne de caractères plutôt qu'un entier. Vous devriez obtenir le même résultat que ci-dessous.

```

Entre ton age : Mishka
Tu as 0 ans.
Entre ton nom : Tu t'appelles .
Appuyez sur une touche pour continuer...

```

Vous n'avez rien le temps de taper que le programme a déjà fini. C'est parce que, lorsque `std::cin` rencontre une erreur (comme le fait de rentrer du texte alors qu'on attend un entier), il passe dans un *état invalide*, tous les caractères invalides restent mémorisés et toutes les utilisations suivantes de `std::cin` sont foireuses.

Comment allons-nous gérer ça ? Comment nettoyer `std::cin` s'il a échoué ? Le chapitre suivant vous apprendra un mécanisme basique mais puissant, qu'on trouve tout le temps en programmation et qui nous aidera à régler nos problèmes.

4.5. Récapitulons !

- C++ nous permet de manipuler des caractères simples, des chaînes de caractères, des nombres entiers et des nombres réels.
- Il existe des caractères spéciaux, comme le retour à la ligne (`'\n'`) ou le caractère nul (`'\0'`).
- Les variables nous permettent d'associer un nom à une valeur, mais il faut respecter certaines règles.
- On peut utiliser `auto` pour laisser le compilateur déduire tout seul le type d'une expression.
- Nous pouvons demander des informations à l'utilisateur grâce à `std::cin`.
- Nous n'avons aucun mécanisme de protection s'il rentre n'importe quoi.

SOLUTIONS

Q : 4.1

```
cpp
#include <iostream>

int main()
{
    std::cout << "Bonne journée à toi lecteur." << std::endl;
    return 0;
}
```

Q : 4.2

```
cpp
#include <iostream>

int main()
{
    std::cout << "Dossier principal : C:\Program Files (x86)"
               << std::endl;

    return 0;
}
```

Q : 4.3

```
cpp
#include <iostream>

int main()
{
    std::cout << "Voici un exemple sans échappement : " << "''"
               << " est bien affiché." << std::endl;
    std::cout << "Maintenant, je vais l'échapper : " << '\'"
               << " est bien affiché grâce à l'échappement." << std::endl;

    std::cout << "La suite, après la tabulation : \tJe suis loin."
               << std::endl;
    std::cout << "Ces mots sont sur une ligne.\nEt ceux là sur la
    suivante.\n";

    return 0;
}
```

SOLUTIONS

Q : 4.4

```
cpp
#include <iostream>

int main()
{
    int nombre { 1 };
    int const multiplicateur { 2 };

    nombre = nombre * multiplicateur;
    std::cout << "Maintenant, mon nombre vaut " << nombre
               << std::endl;

    nombre = nombre * multiplicateur;
    std::cout << "Maintenant, mon nombre vaut " << nombre
               << std::endl;

    nombre = nombre * multiplicateur;
    std::cout << "Maintenant, mon nombre vaut " << nombre
               << std::endl;

    return 0;
}
```

Note › Vous remarquerez qu'on répète quand même plusieurs fois les mêmes instructions. Ce n'est pas grave. Nous verrons très bientôt comment remédier à ce problème. L'essentiel est que vous ayez pu voir l'avantage d'utiliser des variables.

5

Le conditionnel conjugué en C++

Le conditionnel est un mode employé pour exprimer un état soumis à une condition. Telle est la définition qu'en donne [Wikipédia](#). Cela vous rappelle-t-il vos cours de français ? Mais quel peut bien être le rapport avec la programmation ? C'est que C++ permet aussi d'exprimer des conditions et donc de modifier le comportement de notre programme.

Ce chapitre va donc introduire les conditions à notre palette d'outils et permettra de résoudre la problématique soulevée en conclusion du chapitre précédent.

5.1. Les booléens

Les conditions en C++ sont régies par un principe simple, qui est qu'une condition est soit vraie, soit fausse. Dans la réalité, c'est souvent le cas des questions fermées. Si on vous pose la question Êtes-vous majeur ?, la réponse est soit oui, soit non. De même, Faites-vous 1 m80 ou plus ? entraînera soit une réponse positive, soit négative.

C++ nous offre un type conçu exprès pour ça, `bool`. Ce type peut prendre deux valeurs : soit `true`, signifiant vrai, soit `false` qui veut dire faux. Voyez par vous-même l'exemple on ne peut plus bête ci-dessous.

```
int main()
{
    bool const vrai { true };
    bool const faux { false };

    return 0;
}
```



D'accord, mais ça sert à quoi ça ? Je vois l'intérêt de manipuler des entiers ou du texte, mais stocker juste vrai ou faux, ça sert quand ?

L'intérêt semble en effet très limité si l'on s'en tient au code précédent. Sachez que les booléens sont partout, utilisables grâce à ce que l'on appelle les opérateurs de comparaisons. En voici la liste juste ci-dessous.

Opérateur	Signification	Description	Exemple
<code>==</code>	Égalité	Compare si deux variables sont égales entre elles.	<code>a == b</code>
<code>!=</code>	Inégalité	Compare si deux variables sont de valeurs différentes.	<code>a != b</code>
<code><</code>	Strictement inférieur	Compare si la variable de gauche est strictement inférieure à celle de droite.	<code>a < b</code>
<code><=</code>	Inférieur ou égal	Compare si la variable de gauche est inférieure ou égale à celle de droite.	<code>a <= b</code>
<code>></code>	Strictement supérieur	Compare si la variable de gauche est strictement supérieure à celle de droite.	<code>a > b</code>
<code>>=</code>	Supérieur ou égal	Compare si la variable de gauche est supérieure ou égale à celle de droite.	<code>a >= b</code>

Attention > Non, `==` n'est pas une erreur. Il faut bien deux signes `=`, pour différencier l'opération de celle consistant à modifier la valeur d'une variable.

Le rapport avec les booléens ? Chacune des expressions suivantes renvoie `true` ou `false`. Faisons donc un petit test.

```
#include <iostream>

int main()
{
    int const a { 10 };
    int const b { 20 };

    // Cette directive nous permet d'afficher true ou false.
    // Par défaut, std::cout affiche 1 ou 0.
    std::cout << std::boolalpha;

    std::cout << "a == b donne " << (a == b) << std::endl;
    std::cout << "a != b donne " << (a != b) << std::endl;
    std::cout << "a < b donne " << (a < b) << std::endl;
    std::cout << "a <= b donne " << (a <= b) << std::endl;

    // On peut tout à fait stocker le résultat
    // dans une variable booléenne.
    bool const plus_grand { a > b };
    std::cout << "a > b donne " << plus_grand << std::endl;

    bool const plus_grand_ou_egal { a >= b };
    std::cout << "a >= b donne " << plus_grand_ou_egal << std::endl;
```

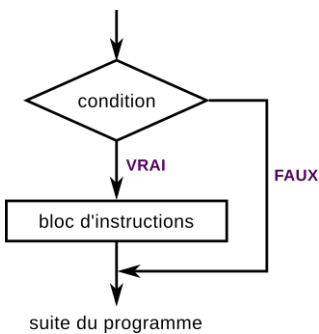
```
    return 0;
}
```

Lancez donc ce code, analysez les résultats. Sont-ils logiques ? Maintenant, changez la valeur des variables et observez de nouveau. Est-ce que c'est toujours logique ? Voyez-vous maintenant un peu plus l'utilité des booléens ?

5.2. *if* — si, et seulement si...

Maintenant, examinons la façon de poser une question en C++, si l'on peut dire. Nous allons en effet découvrir la première structure de contrôle : `if`. Ce mot clé est un mot anglais signifiant *si* et exécute des instructions *si et seulement si* la condition donnée est vraie. Voyez donc ce schéma.

Figure 5.1 : Structure de contrôle `if`



En C++, voici à quoi ressemble cette instruction. Toutes les instructions entre accolades seront exécutées si `condition` est vraie.

```
if (/* condition */)
{
    // Code à exécuter si la condition est vraie.
}
```

Prenons un exemple. Demandons à l'utilisateur la note qu'il a obtenue à un examen. Si celle-ci est supérieure ou égale à 16, nous afficherons un message de félicitations.

```
#include <iostream>

int main()
{
```

```

std::cout << "Indique-moi ta note : ";

int note { 0 };
std::cin >> note;

std::cout << "Tu as obtenu " << note << std::endl;

if (note >= 16)
{
    std::cout << "Félicitations, c'est une très bonne note !\n";
}

return 0;
}

```

Faites-le tourner. Vous verrez, le message de félicitations ne s’affiche que si vous rentrez un nombre supérieur ou égal à 16. Essayez avec 15, vous ne verrez plus le message.

À PORTÉE

Nous pouvons déclarer des variables au sein du bloc d’accolades. Cependant, celles-ci ne seront utilisables que jusqu’à la fermeture de l’accolade. Ainsi, le code ci-dessous n’est pas correct.

```

#include <iostream>

int main()
{
    if (true)
    {
        int const age { 42 };
        // D'autres instructions.

        // Aucun problème.
        if (age == 42)
        {
            // Ici non plus.
        }
    }

    // NON ! Le compilateur ne connaît pas la variable age.
    std::cout << age << std::endl;

    return 0;
}

```

Ce sont les règles de C++ qui veulent ça. Quand on écrit un nouveau bloc à base d’accolades, on dit qu’on crée une *nouvelle portée*. Et une variable n’est utilisable que dans la portée, c’est-à-dire le bloc d’accolades où elle a été déclarée.

C'est une bonne pratique de déclarer ses variables dans la plus petite portée possible. Il y a plusieurs raisons à cela.

- Dans le cas d'un code long et complexe, déclarer une variable au plus près possible de son utilisation permet de ne pas avoir à parcourir de longues lignes pour en obtenir des détails (quel est son type, sa valeur de base, etc.). Cela aide à la lecture et la compréhension du code.
- Lorsqu'on atteint la fin du bloc correspondant, le programme libère dans la mémoire les emplacements qu'il avait réservés pour les variables qui s'y trouvaient. Dans le cas d'un `int` comme ici, ça ne change rien. Mais dans le cas de variables plus complexes que nous verrons plus tard, cela peut avoir une grande incidence sur les performances du programme.

Ainsi, nous vous encourageons fortement à déclarer vos variables à l'intérieur de vos blocs de condition si elles ne sont pas destinées à être utilisées ailleurs.

5.3. *else* — sinon...

Maintenant que nous avons vu comment changer le code si une condition est vérifiée, voyons comment faire l'opposé, c'est-à-dire comment agir si celle-ci est fausse. Imaginons un programme demandant si l'utilisateur est majeur ou mineur. On peut imaginer quelque chose comme ce qui suit.

```
#include <iostream>

int main()
{
    std::cout << "Donne-moi ton âge: ";
    int age { 0 };
    std::cin >> age;

    if (age >= 18)
    {
        std::cout << "Tu es majeur." << std::endl;
    }

    if (age < 18)
    {
        std::cout << "Tu es mineur." << std::endl;
    }

    return 0;
}
```


Ce programme est fonctionnel mais un peu lourd, puisqu'on répète deux fois une condition quasiment identique. Heureusement, C++ nous offre un autre mot clé, `else`, qui exécute des instructions si la condition du `if` est fausse. Voyez vous même le code ci-dessous.

```
#include <iostream>

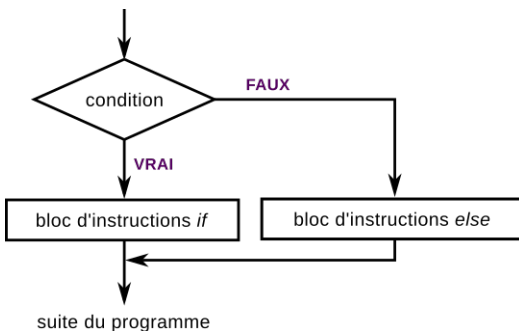
int main()
{
    std::cout << "Donne-moi ton âge: ";
    int age { 0 };
    std::cin >> age;

    if (age >= 18)
    {
        std::cout << "Tu es majeur." << std::endl;
    }
    else
    {
        std::cout << "Tu es mineur." << std::endl;
    }

    return 0;
}
```

Le code se lit ainsi en français : Si l'âge rentré par l'utilisateur est supérieur ou égal à 18 ans, alors afficher "Tu es majeur", sinon afficher "Tu es mineur". Le mot clé `else` est en effet un mot d'anglais qui signifie, comme vous l'avez compris, *sinon*.

Figure 5.2 : Structure de contrôle `else`



Notez bien que, à la différence du `if`, `else` n'a pas de parenthèses pour préciser une condition. En effet, rappelons que `else` signifie *tout le reste*. Y a-t-il donc un moyen de tester plusieurs conditions différentes avant de faire *tout le reste* ? Sommes-nous condamnés à utiliser une suite de `if` et de `else` ?

5.4. *else if* — la combinaison des deux précédents

Comme vous l’avez vu en lisant le titre de cette section, C++ propose une réponse à la problématique que nous venons de soulever. La combinaison `else if` s’utilise en effet entre un `if` et un `else` pour dire *ou si cette condition est vraie*. Voyons déjà à quoi ressemble la syntaxe pour vous donner une idée.

```
if (/* condition 1 */)
{
    // Si
    // Code à exécuter si la condition 1 est vraie.
}
else if (/* condition 2 */)
{
    // Ou si.
    // Code à exécuter si la condition 1 est fausse
    // mais que la condition 2 est vraie.
}
else if (/* condition 3 */)
{
    // Ou si.
    // Code à exécuter si ni la condition 1 ni la condition 2
    // ne sont vraies mais que la condition 3 est vraie.
}
else
{
    // Sinon.
    // Code à exécuter si aucune condition n'est vraie.
}
```

Vous voyez le principe ? Bien, passons donc à un exemple concret.

- **5.1.** Imaginons un programme qui affiche des messages différents en fonction de l’âge de l’utilisateur. Mettez le texte et choisissez les âges que vous souhaitez. Une solution parmi d’autres est proposée ci-après.

✧ [Voir la solution](#)

5.5. Conditions imbriquées

Il est tout à fait possible de tester des conditions au sein d'autres conditions. Si l'on prend l'exemple d'un menu de restaurant, on peut imaginer demander au client s'il veut de la viande ou un plat végétarien. Dans le premier cas, il faut lui demander la cuisson souhaitée ; dans le deuxième, on lui laisse le choix du légume. Nous pouvons formaliser tout à fait cela en C++.

```
#include <iostream>

int main()
{
    std::cout << "Bienvenue au restaurant \"Chez Clem\"." << std::endl;

    bool const plat_vegetarien { true };

    if (plat_vegetarien)
    {
        bool const salade { true };
        if (salade)
        {
            std::cout << "Salade grecque ? De pâtes ?" << std::endl;
        }
        else
        {
            std::cout << "Nous avons également de la purée, une macédoine de légumes, une omelette..." << std::endl;
        }
    }
    else
    {
        bool const viande { true };
        if (viande)
        {
            std::cout << "Bleue, saignante, à point, bien cuite ?" <<
std::endl;
        }
        else
        {
            std::cout << "Sinon niveau poisson il y a pavé de saumon, filet de cabillaud..." << std::endl;
        }
    }

    return 0;
}
```

INSTRUCTION CONDENSÉE

L'instruction `else if`, vue [précédemment](#), correspond en fait à un `if` dans un `else`.

```
if (/* condition 1 */)
{
}
else if (/* condition 2 */)
{
}
```

Équivaut à ceci :

```
if (/* condition 1 */)
{
}
else
{
    if (/* condition 2 */)
    {
    }
}
```

5.6. [Pratique] Gérer les erreurs d'entrée • Partie I

Vous souvenez-vous de la [problématique du chapitre précédent](#) ? Nous avons des problèmes quand l'utilisateur rentrait du texte à la place d'un entier par exemple. Eh bien, nous pouvons déjà apporter une première solution à ce problème, solution que nous améliorerons au fur et à mesure.

Les créateurs de la bibliothèque standard C++ ont déjà tout prévu. Trois fonctions, déjà programmées, vont nous intéresser :

- `std::cin >> x` est en fait une expression qui renvoie `true` si tout est correct ou `false` si on a rencontré une erreur lors de la saisie ;
- `std::cin.clear()` restaure `std::cin` à un état fonctionnel, sans erreur ;
- `std::cin.ignore()` permet d'ignorer un nombre défini de caractères, soit jusqu'à un nombre maximum (exemple 500), soit jusqu'à un caractère précis (exemple `'\n'` ou `'a'`). Dans notre cas, nous allons utiliser ceci :

```
std::cin.ignore(255, '\n');
```

Maintenant, vous avez toutes les informations nécessaires pour commencer la protection de nos entrées. À vous de jouer !

✧ [Voir la solution](#)

Si vous avez eu du mal ou n’avez pas trouvé, ce n’est pas grave, vous êtes en train d’apprendre. N’hésitez pas à relire la solution jusqu’à ce que tout vous paraisse limpide.

Voilà, nous avons déjà une petite protection. Mais elle ne marche qu’une fois : impossible de la redéclencher tant que l’utilisateur n’a pas rentré quelque chose de valide. Pour l’instant, laissez ça de côté, nous en reparlerons au [chapitre suivant](#).

5.7. La logique booléenne

Nous avons vu les différents moyens de tester des conditions, ainsi que différents opérateurs de comparaison qui retournent des booléens. Mais ce ne sont pas les seuls. Cette section va introduire [l’algèbre booléenne](#), et rassurez-vous, c’est simple à comprendre, puissant et bien utile.

AND — tester si deux conditions sont vraies

Imaginons un instant que vous souhaitez faire l’acquisition d’une voiture. Pour cela, il vous faut remplir deux conditions : avoir le permis et suffisamment d’argent. Si l’une des deux conditions n’est pas remplie, ou les deux, vous continuerez à prendre le bus. Le seul moyen de faire hurler le moteur sur l’autoroute, c’est d’avoir assez d’argent ET d’avoir le permis.

C++ nous fournit deux moyens d’exprimer cet opérateur logique ET : `&&` et `and`. Les deux sont parfaitement valables et interchangeables. Le deuxième avait été introduit à l’époque car le symbole `&` n’était pas présent sur tous les claviers.

```
#include <iostream>

int main()
{
    int const prix_voiture { 5000 };
    int const argent {2000 };
    bool const a_le_permis { true };

    if (argent >= prix_voiture && a_le_permis)
    {
        std::cout << "Voici les clés, bonne route.\n";
    }
}
```

```

else
{
    std::cout << "Désolé, vous allez devoir prendre le bus.\n";
}

return 0;
}

```

Testez donc ce programme en modifiant le prix de la voiture, l'argent disponible ou la possession ou non du permis. Comme écrit plus haut, il n'y a que si `a_le_permis` est à `true` et que vous disposez d'une somme d'argent supérieure au prix de la voiture que vous serez en mesure d'avoir les clés.

Voici ce qu'on appelle la [table de vérité](#) de l'opérateur AND, qui formalise ses entrées et ses sorties. N'hésitez pas à en lire plus sur le sujet s'il vous intéresse.

Première condition	Seconde condition	Résultat de l'opérateur AND
false	false	false
false	true	false
true	false	false
true	true	true

OR — tester si au moins une condition est vraie

Maintenant, allons visiter un musée. Au guichet, bonne surprise, l'entrée est gratuite si l'on a moins de 26 ans OU si l'on est professeur. Il suffit de remplir l'une des deux conditions pour être admis gratuitement à l'intérieur. Bien sûr, si vous remplissez les deux, l'offre reste valable.

Cet exemple va nous permettre d'introduire l'opérateur C++ OU. Pour la même raison que précédemment, vous pouvez l'écrire indifféremment de deux manières : `||` (qui se tape sur un AZERTY français en faisant AltGr+- [la touche du 6]) ou `or`.

```

#include <iostream>

int main()
{
    int const age { 25 };
    bool const est_professeur { true };

    if (age <= 26 || est_professeur)
    {
        std::cout << "Bonne visite.\n";
    }
}

```

```

else
{
    std::cout << "Désolé, il va falloir payer.\n";
}

return 0;
}

```

Encore une fois, faites votre ce programme et modifiez-le. Voyez dans quelles circonstances l'entrée est gratuite et à quels moments il faut payer. Vous allez obtenir des résultats identiques à la table de vérité ci-dessous.

Première condition	Seconde condition	Résultat de l'opérateur OR
false	false	false
false	true	true
true	false	true
true	true	true

NOT — tester la négation

Il fait chaud, vous allez à la piscine vous rafraîchir. En arrivant, vous jetez un œil au règlement et vous voyez qu'il est interdit de nager si l'on a un short de bain et non un maillot de bain. Vous raisonnez donc ainsi : *Si j'ai un maillot de bain, alors tout va bien, l'entrée m'est autorisée, je peux nager. Si j'ai un short de bain, alors l'entrée m'est interdite et je dois en acheter un avant de nager.* Comment exprimeriez-vous ça en C++ ? Peut-être avec le code ci-dessous ?

```

int main()
{
    bool const ai_je_un_maillot { true };
    if (ai_je_un_maillot)
    {
        // Je n'ai rien à faire, tout est OK.
    }
    else
    {
        // Je dois aller acheter un maillot.
    }

    // Je nage et je suis heureux.
    return 0;
}

```

Ce code fonctionne, mais il nous pose un problème : on a un `if` vide et c'est moche. L'idéal serait de ne traiter que le cas *Pas de maillot de bain*. C++ nous permet juste-

ment cela avec l'opérateur de négation `!` ou `not`. Celui-ci renvoie la valeur inverse de l'expression testée. Si l'expression vaut `true`, alors cet opérateur renverra `false` ; si, au contraire, elle valait `false`, sa négation donnerait `true`.

```
int main()
{
    bool const ai_je_un_maillot { true };
    if (!ai_je_un_maillot)
    {
        // Je dois aller acheter un maillot.
    }

    // Je nage et je suis heureux.
    return 0;
}
```

Notre code devient plus court, plus concis et plus agréable à lire, il gagne en qualité. Nous vous mettons en dessous la table de vérité, bien plus courte cette fois.

Valeur de la condition	Résultat de l'opérateur NOT
true	false
false	true



Il y a juste quelque chose que je ne comprends pas. Pourquoi ne pas créer tout simplement un booléen inverse, genre `pas_de_maillot` ? Pourquoi se casser la tête à tester la négation d'une condition et ne pas tester la condition inverse tout de suite ?

C'est une très bonne question. Dans un exemple aussi simpliste que celui-ci, monté de toute pièce, on aurait pu, c'est vrai. Mais d'autres fois nous n'avons pas le choix, car nous utilisons du code qui a été programmé d'une certaine façon.

5.8. Tester plusieurs expressions

Nous ne sommes absolument pas limités à tester deux expressions en même temps. On peut le faire avec trois, quatre, dix, soixante (bien que soixante soit un exemple extrême et vraiment trop peu lisible). Le code suivant le prouve.

```
#include <iostream>

int main()
{
    bool const ai_je_envie_de_nager { true };
    // ...
```



```

bool const ai_je_un_maillot { true };
int const argent { 20 };
int const prix { 5 };

if (ai_je_envie_de_nager && ai_je_un_maillot && argent >= prix)
{
    std::cout << "PLOUF !" << std::endl;
}
else
{
    std::cout << "Huum, un autre jour." << std::endl;
}

return 0;
}

```

Dans le cas du même opérateur, ils sont évalués de gauche à droite. C'est le cas du code précédent. Mais que se passe-t-il si l'on mélange plusieurs opérateurs différents dans la même expression ? Sauriez-vous dire ce que quelque chose comme `expression1 && expression2 || expression3` va donner ? En effet, les opérateurs logiques sont comme les opérateurs mathématiques que nous avons vus dans les chapitres précédents : *ils ont un ordre de priorité* :

1. la négation `!` ;
2. le ET `&&` ;
3. le OU `||`.

Ainsi, dans l'exemple `!a && b`, c'est d'abord `!a` qui est évalué, puis la nouvelle valeur et `b` sont testées avec l'opérateur ET `&&`. Avec le code `a && b || c && d`, dans l'ordre, on évalue `a && b`, `c && d` et enfin `a && b || c && d`.

Bien que la priorité des opérateurs soit clairement définie par C++, une bonne pratique consiste à ajouter des parenthèses autour des expressions pour rendre le code plus lisible et plus clair. Si l'on reprend nos exemples, cela donne `(!a) && b` pour le premier et `(a && b) || (c && d)` pour le deuxième. C'est clair ?

► **5.2.** Alors, prouvez-le-moi en mettant les parenthèses au bon endroit dans ces codes (piqués à Guillaume Belz).

```

a && b && c
a || b || c
a || b && c
a && b || c

!a && b
a || !b

```

```
a && b || c && d
a || b && c || d
```

Vous avez trouvé la solution ? Vous êtes sur la bonne voie ! Vous avez eu du mal ? N'hésitez pas à relire les passages précédents à tête reposée et à faire vos propres exemples.

🌟 [Voir la solution](#)

Note > À partir de maintenant, nous allons mettre des parenthèses dans le code pour le rendre clair et explicite.

ÉVALUATION EN COURT-CIRCUIT

Imaginez un formulaire à remplir, avec de multiples questions, pour demander une place de parking dans votre immeuble. Pour ça, il faut déjà habiter l'immeuble en question et avoir une voiture. Si vous habitez ailleurs, pas la peine de réfléchir à la possession ou non d'une voiture, vous êtes inéligible pour cette place de parking.

Eh bien, le compilateur procède de la même façon quand il évalue une expression contenant AND. S'il détecte qu'une condition est fausse, alors il ne sert à rien d'évaluer le reste de l'expression puisque le résultat est forcément faux. Si vous ne voyez pas pourquoi, retournez un peu plus haut jeter un œil à la [table de vérité de AND](#).

```
#include <iostream>

int main()
{
    bool const ai_je_une_voiture { false };
    bool const habite_immeuble { true };

    // Comme ai_je_une_voiture vaut false, le compilateur
    // ne va pas tester l'expression habite_immeuble.
    if (ai_je_une_voiture && habite_immeuble)
    {
        std::cout << "Voici votre place de parking.\n";
    }
    else
    {
        std::cout << "Désolé, vous n'êtes pas éligible.\n";
    }

    return 0;
}
```

Avec l'opérateur OR, le même principe s'applique : si l'une des expressions est évaluée à **true**, alors, selon la table de vérité de l'opérateur OR, le résultat sera **true**, donc pas la peine d'évaluer le reste.

Ce principe d'optimisation par le compilateur s'appelle l'*évaluation en court-circuit*. Il permet d'optimiser l'exécution du code en ne perdant pas de temps sur des instructions qui seront forcément évaluées à **false** ou à **true**. Quand nous avancerons dans notre apprentissage du C++, nous verrons des cas où l'évaluation en court-circuit est bien utile.

5.9. Entraînez-vous !

Allez, c'est l'heure de pratiquer un peu. C'est le meilleur moyen de progresser et de fixer toutes ces nouvelles notions dans votre mémoire. N'hésitez pas à faire cette section à tête reposée avec les explications sous les yeux en même temps.

Note > Certains exercices sont adaptés du cours sur le langage C paru sur Zeste de Savoir.

Une pseudo-horloge

Demandons à l'utilisateur de saisir un nombre et nous lui afficherons la période de la journée correspondante : nuit, matin, après-midi, soir. Ainsi, entre 8h et 12h, nous sommes le matin. Mais attention, nous voulons aussi gérer les cas un peu spéciaux que sont midi, minuit et un nombre qui n'est pas entre 0 et 24.

✧ [Voir la solution](#)

Score

Imaginez que vous avez un score de jeu vidéo sous la main.

- Si le score est strictement inférieur à deux mille, affichez "C'est la catastrophe !".
- Si le score est supérieur ou égal à deux mille et que le score est strictement inférieur à cinq mille, affichez : "Tu peux mieux faire !".
- Si le score est supérieur ou égal à cinq mille et que le score est strictement inférieur à neuf mille, affichez : "Tu es sur la bonne voie !".
- Sinon, affichez : "Tu es le meilleur !".

✧ [Voir la solution](#)

XOR — le OU exclusif

Le OR que nous avons vu précédemment dans ce chapitre est dit *inclusif*. Ainsi, `a || b` signifie `a`, ou `b`, ou les deux. Le but de cet exercice va être de faire un OU dit *exclusif*, c'est-à-dire que si `a` et `b` renvoient la même chose, l'expression est évaluée à `false`, sinon à `true`. Vous pouvez le faire avec les opérateurs que nous avons vus.

✎ Voir la solution

? Obtenir un indice

5.10. Récapitulons !

- Nous pouvons tester une expression avec `if`. Si celle-ci est fausse, nous pouvons tester d'autres expressions avec autant de `else if` que nécessaire. Enfin, `else` permet de tester le cas général, *tout le reste*.
- Nous pouvons comparer des expressions entre elles avec les opérateurs de comparaison.
- Les opérateurs logiques permettent de tester plusieurs expressions en même temps ou d'en obtenir la négation.
- Le compilateur optimise l'évaluation de ces multiples expressions grâce à l'évaluation en court-circuit.

SOLUTIONS

Q : 5.1

```
#include <iostream>

int main()
{
    std::cout << "Quel âge as-tu ? "";

    unsigned int age { 0 };
    std::cin >> age;

    if (age < 5u)
    {
        std::cout << "Tu n'es pas un peu jeune pour naviguer sur
internet ?" << std::endl;
    }
    else if (age < 18u)
    {
        std::cout << "Tu es mineur !" << std::endl;
    }
    else if (age < 30u)
    {
        std::cout << "Vous êtes majeur !" << std::endl;
    }
    else if (age < 50u)
    {
        std::cout << "Vous n'êtes plus très jeunes, mais vous n'êtes
pas encore vieux." << std::endl;
    }
    else if (age < 70u)
    {
        std::cout << "Vous commencez à prendre de l'âge." << std::endl;
    }
    else if (age < 120u)
    {
        std::cout << "Vous avez dû en voir, des choses, durant votre
vie !" << std::endl;
    }
    else
    {
        std::cout << "Quelle longévité !" << std::endl;
    }

    return 0;
}
```

SOLUTIONS

Q : 5.2

```
(a && b) && c  
(a || b) || c  
a || (b && c)  
(a && b) || c
```

```
(!a) && b  
a || (!b)
```

```
(a && b) || (c && d)  
(a || (b && c)) || d
```

SOLUTIONS

Une pseudo-horloge

```
#include <iostream>

int main()
{
    std::cout << "Quelle heure est-il ?" << std::endl;
    int heure { 0 };
    std::cin >> heure;

    if (heure > 0 && heure < 7)
    {
        std::cout << "Zzz..." << std::endl;
    }
    else if (heure >= 7 && heure < 12)
    {
        std::cout << "C'est le matin !" << std::endl;
    }
    else if (heure == 12)
    {
        std::cout << "Il est midi !" << std::endl;
    }
    else if (heure > 12 && heure < 18)
    {
        std::cout << "C'est l'après-midi !" << std::endl;
    }
    else if (heure >= 18 && heure < 24)
    {
        std::cout << "C'est le soir !" << std::endl;
    }
    else if (heure == 24 || heure == 0)
    {
        std::cout << "Il est minuit, dormez brave gens !" << std::endl;
    }
    else
    {
        std::cout << "Il est l'heure de réapprendre à lire l'heure !"
            << std::endl;
    }

    return 0;
}
```

SOLUTIONS

Score

```
#include <iostream>

int main()
{
    std::cout << "Quel est le score du joueur ? " << std::endl;
    int score { 0 };
    std::cin >> score;

    if (score < 2000)
    {
        std::cout << "C'est la catastrophe !" << std::endl;
    }
    else if (score >= 2000 && score < 5000)
    {
        std::cout << "Tu peux mieux faire !" << std::endl;
    }
    else if (score >= 5000 && score < 9000)
    {
        std::cout << "Tu es sur la bonne voie !" << std::endl;
    }
    else
    {
        std::cout << "Tu es le meilleur !" << std::endl;
    }

    return 0;
}
```


SOLUTIONS

XOR — le OU exclusif

La solution s'écrit ainsi : `(a && !b) || (b && !a)`. Vous pouvez vérifier avec le programme ci-dessous.

```
#include <iostream>

int main()
{
    std::cout << std::boolalpha;

    bool const deux_false { (false && !false) || (false && !false) };
    std::cout << "(false && !false) || (false && !false) == "
        << deux_false << std::endl;

    bool const true_false { (true && !false) || (false && !true) };
    std::cout << "(true && !false) || (false && !true) == "
        << true_false << std::endl;

    bool const false_true { (false && !true) || (true && !false) };
    std::cout << "(false && !true) || (true && !false) == "
        << false_true << std::endl;

    bool const deux_true { (true && !true) || (true && !true) };
    std::cout << "(true && !true) || (true && !true) == "
        << deux_true << std::endl;

    return 0;
}
```

INDICE

section XOR – le OU exclusif

Si vous bloquez, voici déjà la table de vérité de l'opérateur.

Première condition	Seconde condition	Résultat de l'opérateur XOR
false	false	false
false	true	true
true	false	true
true	true	false

6

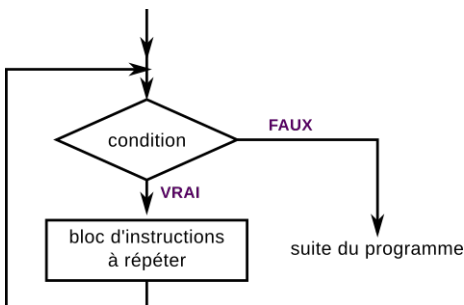
Des boucles qui se répètent, répètent, répètent...

Nous sommes maintenant capables d'exécuter des codes différents en fonction de conditions. Mais notre programme reste très linéaire : nous exécutons les instructions l'une après l'autre, du début à la fin, de haut en bas. Dans la droite ligne du chapitre précédent, nous allons donc voir un mécanisme offert par C++ pour répéter autant de fois que l'on souhaite une série d'instructions : les boucles. Elles nous permettront d'améliorer notre gestion des erreurs.

6.1. *while* — tant que...

Commençons avec une boucle *while* [*tant que* en français]. Elle exécute une série d'instructions tant que la condition est vraie (*true*). Par contre, dès que la condition est évaluée à faux (*false*), la boucle s'arrête. On dit aussi qu'on *boucle* ou qu'on *itère*.

Figure 6.1 : Boucle *while*



Prenons un exemple tout bête : notre programme va compter de 0 à 9. Voici le code correspondant.

```
#include <iostream>

int main()
{
    std::cout << "Salut, je vais compter de 0 à 9." << std::endl;
    int compteur { 0 };

    while (compteur < 10)
    {
        std::cout << compteur << std::endl;
        ++compteur;
    }

    std::cout << "Je m'arrête, après je sais pas faire." << std::endl;
    return 0;
}
```

Le code est exécuté ainsi : on évalue la condition. Tant que `compteur` est strictement inférieur à 10, alors on affiche le nombre puis on l'incrémente. Quand finalement `compteur` vaut 10, la condition devient fausse, on passe à la suite du code.

BOUCLE INFINIE

Il est important de s'assurer qu'à un moment ou un autre la condition devient fausse et qu'on sort bien de la boucle. Si l'on n'y prend pas garde, on tombe dans le piège de la *boucle infinie* !

```
#include <iostream>

int main()
{
    std::cout << "Salut, je vais compter de 0 à 9." << std::endl;
    int compteur { 0 };

    while (compteur < 10)
    {
        std::cout << compteur << std::endl;
        // Oups, on a oublié d'incrémenter le compteur.
        // Nous sommes bloqués dans la boucle.
    }

    std::cout << "Je ne serai jamais affiché." << std::endl;
    return 0;
}
```

Dans ce cas, il ne reste plus qu'à terminer le programme sauvagement en le tuant (oui, les informaticiens sont brutaux). On le répète donc : faites attention aux boucles infinies.

Comme les boucles sont basées sur les conditions, aucun problème pour en utiliser plusieurs en même temps.

```
#include <iostream>

int main()
{
    char entree { '?' };
    int compteur { 0 };

    // On boucle tant qu'on n'a pas rentré la lettre 'e'
    // ou que le compteur n'a pas atteint 5.
    while (compteur < 5 && entree != 'e')
    {
        std::cout << "Le compteur vaut " << compteur << std::endl;
        std::cout << "Rentre un caractère : ";
        std::cin >> entree;

        ++compteur;
    }

    std::cout << "Fin du programme." << std::endl;
    return 0;
}
```

6.2. Entraînez-vous !

Une laverie

Nous allons écrire un programme d'aide pour une laverie automatique. Dans celle-ci, deux types de machines sont employées : des machines économes capables de laver 5 kg de linge au maximum et d'autres, plus gourmandes, capables de laver 10 kg.

- Si le linge fait moins de 5 kg, il faut le mettre dans une machine de 5 kg.
- S'il fait entre 5 kg et 10 kg, il faut le mettre dans une machine de 10 kg.
- S'il fait plus, il faut répartir le linge entre les machines de 10 kg et, dès qu'il reste 5 kg ou moins, mettre le linge restant dans une machine de 5 kg.

✧ [Voir la solution](#)

PGCD

Allez, on va faire un peu de mathématiques ! Le PGCD¹ de deux nombres est le plus grand entier qui divise aussi bien l'un que l'autre. Ainsi, le PGCD de 427 et 84 est 7, car $427 = 7 \times 61$ et $84 = 7 \times 12$. Pour le calculer, voici l'algorithme.

- On stocke le modulo du premier nombre, appelé *a*, par l'autre nombre, appelé *b*, dans une variable *r*.
- Tant que *r* est différent de zéro, on effectue les opérations suivantes :
 - on affecte la valeur de *b* à *a*.
 - on affecte la valeur de *r* à *b*.
 - on affecte à *r* le résultat du modulo de *a* par *b*.
- Quand *r* est nul, alors *b* représente le PGCD des deux nombres donnés en entrée.

✧ [Voir la solution](#)

Somme de nombres de 1 à *n*

Disons qu'on vous donne un entier *N* au hasard. Pouvez-vous créer un programme capable de donner la somme de tous les entiers de 1 à *n*, c'est-à-dire la somme de $1 + 2 + 3 + \dots + (n - 2) + (n - 1) + n$?

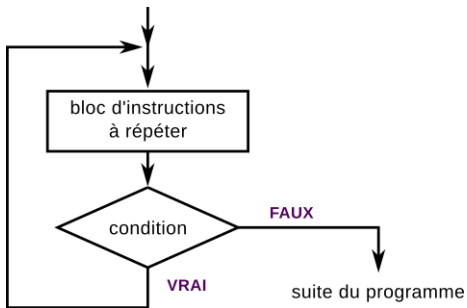
✧ [Voir la solution](#)

6.3. *do while* — répéter ... tant que

La deuxième boucle offerte par C++ est très proche de la précédente. En fait, c'est la même, à deux différences près :

- Une boucle *do while* s'exécute toujours une fois au minimum, même si la condition est fausse. La condition est en effet vérifiée après l'exécution du code.
- Il faut un point-virgule après le *while*. Voyez par vous-même le code suivant.

¹PGCD : Plus Grand Commun Diviseur.

Figure 6.2 : Boucle *do while*

```

#include <iostream>

int main()
{
    do
    {
        std::cout << "On passe quand même ici." << std::endl;
    } while (false);

    return 0;
}

```

Hormis ces deux subtiles différences, *do while* se comporte exactement comme *while* et permet elle aussi de boucler. Nous allons voir dans la section suivante comment les deux vont nous aider à résoudre le problème de sécurisation des entrées soulevé au chapitre précédent.

6.4. [Pratique] Gérer les erreurs d'entrée • Partie II

Pour l'instant, nous avons un mécanisme qui ne nous protège qu'une seule fois des erreurs. Maintenant, nous sommes en mesure de nous protéger tant que l'entrée n'est pas correcte. Améliorez donc le code que vous avez écrit pour qu'il soit capable de protéger le code suivant.

```

#include <iostream>

int main()
{
    std::cout << "Quel jour es-tu né ? ";
    int jour { 0 };
    std::cin >> jour;
}

```

```

std::cout << "Quel mois es-tu né ? ";
int mois { 0 };
std::cin >> mois;

std::cout << "Tu es né le " << jour << "/" << mois << "."
<< std::endl;
return 0;
}

```

✎ Voir la solution

Petit à petit, grâce à nos connaissances qui s'étoffent, nous améliorons la protection des entrées. Mais c'est dommage d'avoir le même code dupliqué à plusieurs endroits. C'est sale, ça nous fait perdre du temps si l'on doit en modifier un morceau : on doit le faire pour chaque endroit, avec le risque d'en oublier certains. Nous apprendrons au chapitre [Découpons le code — les fonctions](#) comment s'en sortir.

6.5. *for* — une boucle condensée

Les boucles `while` et `do while` sont très utiles et ajoutent une corde à notre arc. Mais ne trouvez-vous pas ça dommage d'avoir nos variables liées à l'itération (comme `compteur`) séparées de la boucle elle-même ? Actuellement, notre code est simple, mais imaginons un instant un code bien plus complexe. Il serait mieux de séparer les opérations liées à la boucle de nos autres variables importantes. Cela rendra le code plus clair et plus compréhensible. C'est exactement ce que permet `for` [pour en français].

```

for (/* initialisation */; /* condition */; /* itération */)
{
    // Code de la boucle.
}

// Est équivalent à la boucle ci-dessous.

// Notez la création d'une portée artificielle.
{
    // Initialisation.
    while (/* condition */)
    {
        // Code de la boucle.
        /* itération */
    }
}

```

Hormis cette syntaxe différente, `for` s'utilise comme ses consœurs. Nous sommes libres de l'initialisation, de la condition et de l'itération.


```
#include <iostream>

int main()
{
    // Compte de 10 à 1. On décrémente ici.
    for (int compteur {10}; compteur > 0; --compteur)
    {
        std::cout << compteur << std::endl;
    }

    std::cout << "Fin du programme." << std::endl;
    return 0;
}
```

Nous pouvons même effectuer plusieurs opérations en même temps, tant que les variables déclarées sont du même type. Cette possibilité est cependant à prendre avec des gants parce qu'elle peut rendre le code très peu lisible.

```
#include <iostream>

int main()
{
    // Compte de 10 à 1 et de 1 à 10 en même temps.
    for (int compteur {10}, compteur_inverse {1}; compteur > 0 &&
        compteur_inverse <= 10; --compteur, ++compteur_inverse)
    {
        std::cout << compteur << " et " << compteur_inverse << std::endl;
    }

    std::cout << "Fin du programme." << std::endl;
    return 0;
}
```



À quel moment dois-je utiliser *for* ou *while* ?

Le choix est vôtre, C++ vous laisse entièrement libre. Après, on utilise plus souvent *for* dans le cas où l'on veut itérer (compter de 1 à 100 par exemple), alors que *while* boucle pour réaliser un traitement autant de fois que nécessaire (par exemple, tant que `std::cin.fail` renvoie `true`).

*Note > Dans le chapitre suivant, nous verrons qu'il existe une autre boucle *for*.*

6.6. Boucles imbriquées

Comme pour les conditions, il est tout à fait possible d'imbriquer plusieurs boucles l'une dans l'autre. De même, on peut tout à fait mélanger les types de boucles, comme le

prouve l'exemple suivant, qui affiche les tables de multiplications de 1 à 10 en utilisant `while` et `for`.

```
#include <iostream>

int main()
{
    int facteur_gauche { 1 };
    while (facteur_gauche <= 10)
    {
        for (int facteur_droite { 1 }; facteur_droite <= 10; ++facteur_droite)
        {
            std::cout << facteur_gauche << "x"
                << facteur_droite << " = "
                << facteur_gauche * facteur_droite << std::endl;
        }
        // On saute une ligne pour séparer chaque table.
        std::cout << std::endl;
        ++facteur_gauche;
    }

    return 0;
}
```

6.7. Convention de nommage

Dernier point avant de continuer : vous verrez très souvent, tant dans ce livre que dans les divers codes C++ que vous aurez l'occasion d'examiner, que les programmeurs utilisent `i` comme identifiant de variable pour parcourir une boucle (ainsi que `j` et `k` dans le cas de boucles imbriquées). C'est une abréviation pour *index*. Ce nom est tellement court et explicite qu'il est pour ainsi dire devenu une convention de nommage en C++.

6.8. Contrôler plus finement l'exécution de la boucle

Les boucles sont très utiles, mais le code à l'intérieur des accolades est soit complètement ignoré, soit complètement exécuté. Sachez que C++ permet de contrôler un peu plus précisément l'exécution avec deux mots clés : `break` et `continue`.

Note > `break` et `continue` s'utilisent avec les boucles uniquement. Ils ne s'utilisent pas avec les conditions : pas de `break` au sein d'un `if`, par exemple.

`break` — je m'arrête là

Le premier, c'est `break`, qui en anglais signifie littéralement *casser*, *interrompre*. Celui-ci nous autorise à mettre fin à l'exécution de la boucle, peu importe où nous en étions.

Dans le code ci-dessous, vous ne verrez jamais afficher les nombres au-dessus de 3 parce que la boucle est terminée par **break** dès que notre variable vaut 3.

```
#include <iostream>

int main()
{
    for (int i { 0 }; i < 10; ++i)
    {
        // Si i vaut 3, j'arrête la boucle.
        if (i == 3)
        {
            break;
        }

        std::cout << i << std::endl;
    }

    return 0;
}
```

Dans le cas de boucles imbriquées, **break** ne stoppe l'exécution que de la boucle dans laquelle il se trouve, mais pas les boucles englobantes.

```
for (int i { 0 }; i < 5; ++i)
{
    for (int j { 0 }; j < 3; ++j)
    {
        if (j == 1)
        {
            // Quand j vaut 1, on retourne dans le for(i).
            break;
        }

        std::cout << "Voici la valeur de j : " << j << std::endl;
    }

    std::cout << "Voici la valeur de i : " << i << std::endl;
}
```

Pour illustrer son fonctionnement avec un cas plus utile, prenons un exemple mathématique. Connaissez-vous le concept du **test de primalité** ? Il s'agit d'un test pour déterminer si un entier est premier, c'est-à-dire divisible uniquement par 1 et par lui-même ; 13 est un nombre premier mais pas 10, car divisible par 1, 2, 5, et 10.

► **6.1.** Essayez donc de faire cet exercice.

✧ [Voir la solution](#)

🔍 [Obtenir un indice](#)

continue — saute ton tour !

L'autre mot clé, **continue**, permet de sauter l'itération courante. Toutes les instructions qui se trouvent après ce mot clé sont donc ignorées et la boucle continue au tour suivant.

- **6.2.** Imaginons que nous voulions un programme permettant d'afficher les nombres impairs compris entre 0 et un nombre choisi par l'utilisateur. Essayez donc de le faire, avec les connaissances que vous avez.

✎ Voir la solution

? Obtenir un indice

POTENTIELLE BOUCLE INFINIE

Attention, lorsque vous utilisez **continue** au sein d'une boucle **while** ou **do while**, TOUT le code situé après est sauté, même les instructions d'incrémentation. Le code suivant est donc un exemple de boucle infinie.

```
int i { 0 };
while (i < 10)
{
    if (i == 5)
    {
        // Ohoh...
        continue;
    }

    ++i;
}
```

6.9. Récapitulons !

- C++ nous offre trois façons de boucler : **while**, **do while** et **for**.
- **while** s'utilise plutôt pour boucler tant qu'une condition n'est pas remplie alors que **for** est utilisé pour itérer sur un nombre connu d'éléments.
- Il faut faire attention à ne pas tomber dans une boucle infinie.
- Nous disposons de deux mots clé, **break** et **continue**, pour affiner notre contrôle sur l'exécution de la boucle.

SOLUTIONS

section Une laverie

```
#include <iostream>
int main()
{
    std::cout << "Bienvenue à la laverie de Clem. Combien de kilos de linge as-tu ? ";
    int kilos { 0 };
    std::cin >> kilos;

    if (kilos <= 5)
    {
        std::cout << "Tu as peu de linge, mets-le dans une machine de 5kg." << std::endl;
    }
    else if (kilos <= 10)
    {
        std::cout << "Tu as beaucoup de linge, mets-le dans une machine de 10kg." << std::endl;
    }
    else
    {
        // On a plus que 10 kilos, il va falloir diviser le linge
        // en plusieurs machines.
        std::cout << "Dites donc, il faut laver son linge plus souvent !" << std::endl;

        int nb_machines_10_kilos { 0 };

        while (kilos > 5)
        {
            // Pour chaque machine de 10kg utilisée,
            // on enlève 10kg au total de linge restant.
            kilos -= 10;
            ++nb_machines_10_kilos;
        }

        std::cout << "Tu as besoin de " << nb_machines_10_kilos
            << " machine(s) de 10kg." << std::endl;
        if (kilos >= 0)
        {
            // S'il reste entre 0 et 5 kilos, il ne faut pas
            // oublier la dernière machine de 5kg.
            std::cout << "Le reste rentrera dans une machine de 5kg."
                << std::endl;
        }
    }
}

return 0;
}
```

SOLUTIONS

section PGCD

```
#include <iostream>

int main()
{
    std::cout << "Rentre un nombre a : ";
    int a { 0 };
    std::cin >> a;

    std::cout << "Rentre un nombre b : ";
    int b { 0 };
    std::cin >> b;

    int r { a % b };
    while (r != 0)
    {
        a = b;
        b = r;
        // r vaut le reste de la division entière de a par b.
        r = a % b;
    }

    std::cout << "PGCD vaut = " << b << std::endl;
    return 0;
}
```

SOLUTIONS

section Somme de nombres de 1 à n

```
#include <iostream>

int main()
{
    std::cout << "Donne-moi un entier : ";
    int n { 0 };
    std::cin >> n;

    int total { 0 };
    int compteur { 1 };

    while (compteur <= n)
    {
        total += compteur;
        ++compteur;
    }

    std::cout << "La somme totale vaut " << total << "." <<
    std::endl;
    return 0;
}
```

Ceux qui aiment les maths ont remarqué qu'il s'agit de la [somme des premiers entiers de 1 à n](#) et qu'il existe une formule pour calculer directement sans boucler : $\frac{N \times (N + 1)}{2}$.
Bravo à vous.

SOLUTIONS

Q : 6.1

```
#include <iostream>

int main()
{
    std::cout << "Donne-moi un nombre, je te dirai s'il est premier :
    ";
    int nombre { 0 };
    std::cin >> nombre;

    bool est_premier { true };
    for (int i { 2 }; i < nombre; ++i)
    {
        if (nombre % i == 0)
        {
            // Si i divise nombre, alors nous savons
            // que nombre n'est pas premier.
            est_premier = false;
            break;
        }
    }

    if (est_premier)
    {
        std::cout << nombre << " est un nombre premier !" << std::endl;
    }
    else
    {
        std::cout << nombre << " n'est pas premier." << std::endl;
    }

    return 0;
}
```


SOLUTIONS

Q : 6.2

```
#include <iostream>

int main()
{
    std::cout << "Donne-moi un nombre, je t'afficherai tous les
nombre impairs jusqu'à ce nombre : ";
    int maximum { 0 };
    std::cin >> maximum;

    for (int i { 0 }; i <= maximum; ++i)
    {
        if (i % 2 == 0)
        {
            // On saute ce nombre puisqu'il est pair.
            continue;
        }

        std::cout << i << std::endl;
    }

    return 0;
}
```

INDICES

Q : 6.1

Nous allons devoir tester tous les nombres entre 2 et x pour savoir s'ils sont diviseurs de x . Mais si nous trouvons un nombre y différent de x et qui divise x , alors nous savons que le nombre n'est pas premier, inutile de continuer.

Q : 6.2

Pour rappel, un nombre est pair s'il est divisible par 2 et impair autrement.

7

Au tableau !

Si nous vous demandons de faire un programme calculant la moyenne d'une série de nombres, comment feriez-vous ? Avec une boucle, comme nous avons appris dans le chapitre précédent. Mais maintenant, nous avons besoin de les stocker. Comment faire, puisque vous ne savez pas à l'avance combien de notes on veut rentrer ?

Ce chapitre va résoudre cette problématique en vous présentant les tableaux.

7.1. Un tableau c'est quoi ?

Les tableaux, en informatique, font partie de la grande famille des [structures de données](#). Ces structures de données sont des façons particulières d'organiser et de stocker des données. Il y a ainsi des structures pour stocker un nombre fixe de données du même type, d'autres pour un nombre variable, d'autre encore pour stocker des données de types différents. Ces objets, conçus spécialement pour stocker des données, sont appelés *conteneurs*.

Celles qui nous intéressent maintenant sont les tableaux. Ils sont conçus pour stocker des données de même type et ayant des points communs : notes d'un élève à l'école, titres de livres, âge d'utilisateurs. Dès que vous pensez *liste de...*, dès que des éléments ont des points communs et qu'il est logique de les regrouper, alors l'usage d'un tableau peut s'envisager.

Un tableau n'est cependant pas la solution universelle, mais ça, nous le verrons au fur et à mesure, pour ne pas vous encombrer inutilement de détails.

7.2. `std::vector` — mais quel dynamisme !

Commençons par les *tableaux dynamiques*, c'est-à-dire dont la taille ne sera connue qu'au moment de l'exécution du programme. C++ nous fournit justement un objet déjà codé pour ça et qui peut facilement s'agrandir ou se rétrécir : `std::vector`. Et le fichier d'en-tête correspondant se doit bien évidemment d'être inclus à l'aide d'un `#include <vector>`.

Déclarer un `std::vector`

Les concepteurs de `std::vector` l'ont créé de telle sorte qu'il soit utilisable avec n'importe quel type. On peut donc utiliser un `std::vector` de `int` pour stocker des âges et un `std::vector` de `double` pour stocker des moyennes. Voici la façon de faire.

```
std::vector< /* type des éléments du tableau */> identifiant {};
```

Avec maintenant trois exemples concrets :

```
#include <string>
// N'oubliez pas cette ligne.
#include <vector>

int main()
{
    std::vector<int> const tableau_de_int {};
    std::vector<double> const tableau_de_double {};
    // Même avec des chaînes de caractères c'est possible.
    std::vector<std::string> const tableau_de_string {};

    return 0;
}
```

Tous nos `std::vector` (ou tableaux) sont vides avec ces déclarations. Mais on peut très bien leur affecter des valeurs, comme ci-dessous. Chacun d'eux a un nombre d'éléments différents dès sa création.

```
#include <string>
#include <vector>

int main()
{
    // On stocke 5 entiers.
    std::vector<int> const tableau_de_int { 1, 2, 3, 4, 5 };
    // Et ici 2 flottants.
    std::vector<double> const tableau_de_double { 2.71828, 3.14159 };
    // Maintenant 3 chaînes de caractères.
    std::vector<std::string> const tableau_de_string {
        "Une phrase.",
        "Et une autre !",
        "Allez, une pour la fin."
    };

    return 0;
}
```

Rien ne nous empêche non plus de copier un tableau dans un autre, tout comme on le fait pour des `int`.

```
#include <vector>

int main()
{
    std::vector<int> const tableau_de_int { 1, 2, 3 };
    // Maintenant, copie possède les mêmes éléments que tableau_de_int.
    std::vector<int> const copie { tableau_de_int };

    return 0;
}
```

Manipuler les éléments d'un `std::vector`

Accès aux éléments

Pour l'instant, nos tableaux sont inutiles, nous ne faisons rien avec. Commençons donc à nous amuser un peu en récupérant un élément du tableau pour l'afficher. On utilise pour cela les crochets devant l'identifiant de notre tableau, avec entre crochets la position de l'élément à récupérer.

```
identificateur[/ * un entier */];

#include <vector>
#include <iostream>

int main()
{
    std::vector<int> const nombres { 2, 4, 6, 8 };
    std::cout << "Deuxième élément: "
               << nombres[1] << '\n';

    return 0;
}
```



Pourquoi récupère-t-on le deuxième élément en écrivant `1` ? Le deuxième élément, ça devrait être `2` non ?

Justement non. La subtilité des tableaux en C++, c'est qu'ils commencent à l'indice 0 et non à l'indice 1. Pourquoi ça ? C'est un héritage du C que nous n'allons pas détailler car il demande des connaissances que vous n'avez pas encore. Souvenez-vous simplement que le premier élément d'un tableau est à l'indice 0 et que donc le $x^{\text{ème}}$ élément d'un tableau est à l'indice $x - 1$.

```

#include <iostream>
#include <vector>

int main()
{
    std::vector<int> const tableau_de_int { 1, 2, 3 };

    std::cout << "Le premier élément est "
    << tableau_de_int[0] << "." << std::endl;
    std::cout << "Le deuxième élément est "
    << tableau_de_int[1] << "." << std::endl;
    std::cout << "Le troisième élément est "
    << tableau_de_int[2] << "." << std::endl;

    return 0;
}

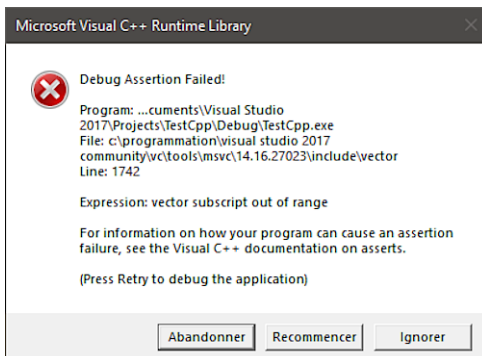
```



Et si j'accède à l'élément 4, que se passe-t-il ?

Potentiellement, tout et n'importe quoi. Parfois, le programme va continuer sans rien dire, d'autre fois vous allez avoir des erreurs et des plantages, comme illustré ci-dessous. Parfois, le programme peut aussi péter complètement un câble et lancer un missile nucléaire sur un pays pris au hasard. C'est ce qu'on appelle un *comportement indéterminé* (en anglais *undefined behavior* ou UB). Impossible donc de faire du code de qualité dans ces conditions. Voilà pourquoi il ne faut pas utiliser d'indice inférieur à 0 ou plus grand que la taille de notre tableau moins un.

Figure 7.1 : Un exemple avec Visual Studio 2019 de ce qui arrive.



Attention > Un UB est le signe d'une erreur de programmation. Vous avez mal fait quelque chose, quelque part dans le code, et le programme ne peut plus être considéré comme fiable désormais. Il faut donc à tout prix les éviter et faire attention en codant !

Premier et dernier élément

Il existe d'ailleurs deux fonctions dont le but est d'accéder spécifiquement au premier ou au dernier élément d'un tableau. Elles se nomment, respectivement, `front` et `back`. Leur utilisation est très simple.

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> const tableau_de_int { 1, 2, 3, 4, 5, 6, 7, 8, 9 };

    std::cout << "Le premier élément est "
               << tableau_de_int.front() << "." << std::endl;
    std::cout << "Le dernier élément est "
               << tableau_de_int.back() << "." << std::endl;

    return 0;
}
```

Obtenir la taille d'un tableau

Comment récupérer-t-on la taille de notre tableau ? En utilisant la fonction `std::size`, qui renvoie le nombre d'éléments de n'importe quel conteneur.

Le type de retour de cette fonction est un type nouveau, que nous n'avons pas rencontré jusque-là et qui s'appelle `std::size_t`. C'est un type entier conçu, entre autres, pour être capable de stocker la taille de n'importe quel tableau, aussi grand soit-il. En vrai, un `int` est suffisant et `std::size_t` est présent car historiquement déjà là en C. De toute façon, avec `auto`, nous n'avons aucune raison de nous casser la tête là-dessus.

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> const tableau_de_int { 1, 2, 3 };

    auto const taille { std::size(tableau_de_int) };
    std::cout << "Mon tableau contient "
               << taille << " éléments." << std::endl;
}
```

```
    return 0;
}
```

Afficher les éléments d'un tableau

Comme nous l'avons dit dans le chapitre sur les boucles, depuis l'arrivée de C++ 11, il existe une variante de la boucle `for`, conçue exclusivement pour parcourir les éléments d'un conteneur. Ci-dessous, voici un pseudo-code illustrant cette possibilité.

```
for (/* type d'un élément du tableau ou auto */ identifiant : /* tableau
à parcourir */)
{
    // Manipuler identifiant, en l'affichant par exemple.
}
```

Voici ce que cela donne avec un exemple concret. Pas besoin de récupérer la taille, la boucle est intelligente. Cela nous facilite la vie et rend notre code plus concis et plus facile à lire.

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> tableau_entiers { 1, 2, 3 };

    std::cout << "On va afficher le tableau en entier.\n";
    for (auto const element : tableau_entiers)
    {
        std::cout << element << std::endl;
    }

    return 0;
}
```

Note > Quant à utiliser `auto` ou préciser explicitement le type d'un élément (`int` par exemple), c'est une question personnelle. Certains ne vont jurer que par `auto`, d'autres ne le mettront que si le type est long à écrire. Vous êtes libre de choisir ce qu'il vous plaît.

Vérifier si un tableau est vide

Fonction bien utile, `std::empty` permet de vérifier si un tableau est vide, c'est-à-dire qu'il ne contient aucun élément et donc qu'il a une taille de zéro. Elle renvoie un booléen. Cela signifie qu'elle renverra `true` si elle est appliquée à un tableau vide et `false` pour un tableau avec trois éléments, par exemple.


```

#include <iostream>
#include <vector>

int main()
{
    std::vector<int> const tableau_vide {};
    std::vector<int> const tableau_rempli { 1, 2, 3 };

    std::cout << std::boolalpha;
    std::cout << "Est-ce que tableau_vide est vide ? Réponse : ""
    << std::empty(tableau_vide) << std::endl;
    std::cout << "Est-ce que tableau_rempli est vide ? Réponse : ""
    << std::empty(tableau_rempli) << std::endl;

    return 0;
}

```

Ajouter des éléments

Pour l'instant, nos `std::vector` ne sont pas vraiment dynamiques. Mais cela va changer avec la fonction `push_back`, qui nous donne la possibilité d'ajouter un élément à la fin de notre tableau. Celui-ci doit cependant être du même type que les autres éléments du tableau. On ne pourra ainsi pas ajouter de `int` à un tableau de `std::string`.

```

#include <iostream>
#include <string>
#include <vector>

int main()
{
    std::vector<int> tableau_de_int { 12, 24 };
    // On ajoute un élément...
    tableau_de_int.push_back(36);
    // ...mais on peut en ajouter encore d'autres.
    tableau_de_int.push_back(48);
    tableau_de_int.push_back(100);

    // On affiche pour vérifier.
    for (auto const valeur : tableau_de_int)
    {
        std::cout << valeur << std::endl;
    }

    std::vector<std::string> tableau_de_string {
        "Salut !",
        "Voici une phrase."
    };
    tableau_de_string.push_back("Mais je vais en ajouter une autre.");
    // Ceci ne compilera pas.
    // tableau_de_string.push_back(5);
}

```

```

    for (auto const chaine : tableau_de_string)
    {
        std::cout << chaine << std::endl;
    }

    return 0;
}

```

Et pour vous prouver que la taille change bien, il suffit d'utiliser `std::size` comme vu plus haut.

```

#include <iostream>
#include <vector>

int main()
{
    std::vector<int> tableau_de_int { 12, 24 };

    std::cout << "Taille avant l'ajout : "
              << tableau_de_int.size() << std::endl;

    tableau_de_int.push_back(36);
    tableau_de_int.push_back(48);
    tableau_de_int.push_back(100);

    std::cout << "Taille après l'ajout : "
              << std::size(tableau_de_int) << std::endl;

    return 0;
}

```

Supprimer des éléments

On ajoute, on ajoute, mais on a aussi envie de pouvoir retirer un voire tous les éléments. Cette possibilité nous est offerte avec les fonctions `pop_back` et `clear`. Ces fonctions sont simples, comme l'illustre l'exemple ci-dessous.

```

#include <iostream>
#include <vector>

int main()
{
    std::vector<int> tableau_de_int { 12, 24 };

    std::cout << "Taille avant l'ajout : "
              << std::size(tableau_de_int) << std::endl;

    tableau_de_int.push_back(36);
    tableau_de_int.push_back(48);
    tableau_de_int.push_back(100);
}

```

```

std::cout << "Taille après l'ajout : "
    << std::size(tableau_de_int) << std::endl;

// On retire le dernier élément.
tableau_de_int.pop_back();
std::cout << "Taille après avoir retiré le dernier élément : "
    << std::size(tableau_de_int) << std::endl;

// Finalement vidons le tableau.
tableau_de_int.clear();
std::cout << "Taille après avoir vidé le tableau : "
    << std::size(tableau_de_int) << std::endl;

return 0;
}

```

Attention > Appeler `pop_back` sur un tableau vide est aussi un comportement indéterminé.

Modifier des éléments

Pour l’instant, nous ne faisons que lire les valeurs de notre tableau, mais il n’y a aucun problème pour les modifier, tant que le tableau n’est pas déclaré comme `const` bien évidemment. Et la bonne nouvelle, c’est que vous savez déjà comment faire : les crochets `[]`. En effet, s’ils sont suivis d’un `=` puis d’une valeur, alors l’élément du tableau auquel on accède se voit modifié.

```

#include <iostream>
#include <vector>

int main()
{
    std::vector<int> tableau { 1, 2, 3 };

    // Finalement, on aimerait que la première valeur soit un 15.
    tableau[0] = 15;

    for (auto const valeur : tableau)
    {
        std::cout << valeur << std::endl;
    }

    return 0;
}

```

De même que pour la lecture, n’essayez pas de modifier la valeur d’un élément en dehors du tableau. Rappelez-vous, c’est un comportement indéterminé et qui est potentiellement dangereux !

Remplir le tableau avec une valeur unique

Dernière fonction que nous allons vous montrer, celle-ci est bien utile pour remplir d'un coup un tableau d'une même valeur. La fonction `assign` a besoin de savoir combien de fois vous souhaitez le faire et quelle est la valeur en question.

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> valeurs {};
    std::cout << "Taille actuelle de mon tableau : "
              << std::size(valeurs) << " éléments.\n";

    // On ajoute dix fois la valeur 42.
    valeurs.assign(10, 42);
    std::cout << "Taille actuelle de mon tableau : "
              << std::size(valeurs) << " éléments.\n";

    for (auto const valeur : valeurs)
    {
        std::cout << valeur << std::endl;
    }

    std::cout << '\n';

    // Finalement on réduit en assignant cinq fois la valeur 3.
    valeurs.assign(5, 3);
    std::cout << "Taille actuelle de mon tableau : "
              << std::size(valeurs) << " éléments.\n";

    for (auto const valeur : valeurs)
    {
        std::cout << valeur << std::endl;
    }

    std::cout << '\n';

    // Mais on peut très bien augmenter.
    valeurs.assign(12, 78);
    std::cout << "Taille actuelle de mon tableau : "
              << std::size(valeurs) << " éléments.\n";

    for (auto const valeur : valeurs)
    {
        std::cout << valeur << std::endl;
    }

    return 0;
}
```

7.3. Entraînez-vous !

Calcul de moyenne

Dans cet exercice, vous allez laisser l'utilisateur entrer autant de valeurs qu'il le souhaite puis en faire la moyenne, c'est-à-dire diviser la somme des notes par le nombre de notes. L'utilisateur pourra taper autant de valeurs positives qu'il le souhaite. Cependant, un nombre négatif signifiera qu'il ne souhaite plus entrer de notes.

✧ [Voir la solution](#)

Minimum et maximum

Le but de ce programme est tout simple : trouver le plus petit élément du tableau, ainsi que le plus grand. Pour vous simplifier la vie, vous pouvez utiliser un tableau déjà rempli.

✧ [Voir la solution](#)

Séparer les pairs des impairs

Cette fois, on veut créer deux nouveaux tableaux qui contiendront, pour le premier, les nombres pairs uniquement, pour le second, les nombres impairs.

✧ [Voir la solution](#)

Compter les occurrences d'une valeur

Une opération très courante avec un tableau consiste à compter le nombre de fois que revient une certaine valeur. Faites donc un programme qui permet de compter combien de fois revient une lettre donnée dans un tableau de `char`.

✧ [Voir la solution](#)

7.4. `std::array` — le tableau statique

Abordons maintenant l'autre catégorie de tableaux qui existe en C++ : les *tableaux statiques*, c'est-à-dire dont la taille est connue à la compilation et qui ne peut varier. En contrepartie, ce tableau est plus rapide à créer qu'un `std::vector` puisqu'il n'y a pas d'opération d'ajout ou de retrait d'éléments.

Pour l'utiliser, il faut inclure le fichier d'en-tête `<array>`.

Déclarer un `std::array`

Comme pour `std::vector`, notre nouveau conteneur attend qu'on lui indique non seulement quel type de données il doit stocker, mais également *combien*. Rappelez-vous que la taille doit être définie à la compilation. Vous ne pouvez pas déclarer un `std::array` et lui passer une taille ultérieurement. Voyez vous-même l'exemple ci-dessous.

```
#include <array>
#include <iostream>

int main()
{
    // Tableau contenant cinq entiers explicitement précisés.
    std::array<int, 5> const tableau_de_5_entiers { 1, 2, 3, 4, 5 };
    for (auto const element : tableau_de_5_entiers)
    {
        std::cout << element << std::endl;
    }

    // Tableau contenant deux décimaux définis puis cinq zéros.
    std::array<double, 7> const tableau_de_7_decimaux { 3.14159, 2.1878 };
    for (auto const element : tableau_de_7_decimaux)
    {
        std::cout << element << std::endl;
    }

    // Tableau qui a trop d'éléments et ne compilera pas.
    //std::array<int, 2> const tableau_qui_ne_compilera_pas { 1, 2, 3,
    4 };

    // Il est tout à fait possible de copier un tableau.
    auto copie { tableau_de_5_entiers };
    for (auto const element : copie)
    {
        std::cout << element << std::endl;
    }

    return 0;
}
```

Avez-vous remarqué que, contrairement à `std::vector`, un tableau statique défini avec `std::array` contient des éléments par défaut ? Ainsi, même si l'on ne précise pas tous les éléments, comme pour `tableau_de_7_decimaux`, nous avons la garantie que les autres seront initialisés à zéro.

Remplir un tableau

Il existe une fonction permettant de remplir le tableau, mais contrairement à `std::vector` qui peut être redimensionné et donc attend le nouveau nombre d'éléments, la fonction `fill` pour `std::array` n'attend que la valeur utilisée pour remplir le tableau.

```
#include <array>
#include <iostream>

int main()
{
    std::array<int, 5> valeurs { 1, 2, 3, 4, 5 };

    std::cout << "Avant" << std::endl;
    for (auto const valeur : valeurs)
    {
        std::cout << valeur << std::endl;
    }

    // On remplit de 42.
    valeurs.fill(42);

    std::cout << std::endl << "Après" << std::endl;
    for (auto const valeur : valeurs)
    {
        std::cout << valeur << std::endl;
    }

    return 0;
}
```

Accéder aux éléments

Comme précédemment, pour accéder à un élément du tableau, nous allons utiliser les crochets `[]`. Et comme pour `std::vector`, faites attention à ne pas accéder à un élément *en dehors des limites du tableau*.

```
#include <array>
#include <iostream>

int main()
{
    std::array<double, 7> tableau_de_7_decimaux { 3.14159, 2.1878 };
}
```

```
// C'est le nombre d'or.
tableau_de_7_decimaux[2] = 1.61803;

// Aïe aïe aïe, catastrophe !
//tableau_de_7_decimaux[7] = 1.0;

for (auto const element : tableau_de_7_decimaux)
{
    std::cout << element << std::endl;
}

return 0;
}
```

Les fonctions `front` et `back` sont toujours de la partie.

```
#include <array>
#include <iostream>

int main()
{
    std::array<int, 9> const tableau_de_int { 1, 2, 3, 4, 5, 6, 7, 8, 9 };

    std::cout << "Le premier élément est "
        << tableau_de_int.front() << "." << std::endl;
    std::cout << "Le dernier élément est "
        << tableau_de_int.back() << "." << std::endl;

    return 0;
}
```

Connaître la taille

Sans surprise, c'est la même fonction que pour `std::vector`, c'est-à-dire `std::size`. Le code qui suit ne devrait donc pas vous surprendre.

```
#include <array>
#include <iostream>

int main()
{
    std::array<int, 5> const valeurs { 1, 2, 3, 4, 5 };
    std::cout << "Et la taille de mon tableau vaut bien "
        << std::size(valeurs) << "." << std::endl;
    return 0;
}
```


Vérifier si le tableau est vide

De même que vu plus haut, rien de dépayçant, la fonction `std::empty` renvoie `false` si le tableau contient des éléments et `true` s'il est vide. En pratique, cette fonction ne vous renverra jamais `true` car avoir un tableau vide signifie ici que la taille donnée est 0. Autant vous dire que c'est inutile d'avoir un tableau vide.

```
#include <array>
#include <iostream>

int main()
{
    std::array<int, 5> const valeurs { 1, 2, 3, 4, 5 };
    std::cout << std::boolalpha;
    std::cout << "Est-ce que mon tableau est vide ? "
        << std::empty(valeurs)
        << " Ah, je le savais !" << std::endl;
    return 0;
}
```

7.5. `std::string` — un type qui cache bien son jeu

Vous vous demandez peut-être toujours à quoi servent VRAIMENT les tableaux. Vous voudriez un exemple concret d'un tableau qui rend de grands services. Ça tombe très bien puisque vous en utilisez un depuis le début de ce livre : `std::string`.

En effet, vous vous souvenez que C++ utilise le type `char` pour stocker un caractère ? De là, stocker du texte se fait en stockant un ensemble de `char`. Le type `std::string` n'est donc rien d'autre qu'un tableau de `char`, conçu et optimisé spécialement pour le stockage de texte.

Connaître la taille d'une chaîne

En toute originalité, on peut récupérer la taille d'une `std::string` grâce à la fonction `std::size`.

```
#include <cassert>
#include <iostream>
#include <string>

int main()
{
    std::string const phrase { "Voici une phrase normale." };
    std::cout << "Voici la longueur de cette phrase : "
        << std::size(phrase) << " caractères."
        << std::endl;
}
```

```
    return 0;
}
```

Accéder à un caractère

Comme pour les deux conteneurs que nous avons découverts plus tôt dans ce chapitre, nous pouvons utiliser les crochets `[]` pour accéder à un caractère en particulier. Et, exactement comme pour les deux autres, il faut faire attention à bien utiliser un indice qui est inférieur à la taille de la chaîne.

```
#include <cassert>
#include <iostream>
#include <string>

int main()
{
    std::string const phrase { "Ceci me servira d'exemple." };
    auto const indice { 5 };

    std::cout << "Voici le caractère à la position "
                << indice << " : "
                << phrase[indice] << std::endl;
    return 0;
}
```

On peut bien entendu modifier un caractère sans problème.

```
#include <iostream>
#include <string>

int main()
{
    std::string verbe { "Estamper" };
    verbe[3] = 'o';
    std::cout << "Verbe choisi : "
                << verbe << std::endl;

    return 0;
}
```

Premier et dernier caractère

Les deux fonctions `front` et `back` nous rendent le même service que précédemment, en nous permettant de récupérer le premier et le dernier caractère de la chaîne.

```
#include <iostream>
#include <string>
```

```
int main()
{
    std::string const phrase { "Bonjour tout le monde !" };
    std::cout << "Première lettre : "
              << phrase.front() << std::endl;
    std::cout << "Dernière lettre : "
              << phrase.back() << std::endl;

    return 0;
}
```

Vérifier qu'une chaîne est vide

Toujours sans surprise pour vous, nous allons utiliser la même fonction `std::empty` que pour `std::vector` et `std::array`.

```
#include <iostream>
#include <string>

int main()
{
    std::string const texte { "Du texte." };

    std::cout << std::boolalpha;
    std::cout << "Est ce que 'texte' est vide ? "
              << std::empty(texte) << std::endl;

    std::string const rien { "" };
    std::cout << "Est ce que 'rien' est vide ? "
              << std::empty(rien) << std::endl;

    return 0;
}
```

Ajouter ou supprimer un caractère à la fin

Nous bénéficions également des fonctions `push_back` et `pop_back` pour, respectivement, ajouter et supprimer un caractère à la fin de la chaîne. Faites toujours attention à ne pas retirer le dernier caractère d'une chaîne vide, c'est un comportement indéfini.

```
#include <iostream>
#include <string>

int main()
{
    std::string phrase { "Une phrase !" };

    phrase.pop_back();
}
```

```

phrase.pop_back();

phrase.push_back(' ');

std::cout << phrase << std::endl;

return 0;
}

```

Supprimer tous les caractères

La fonction `clear` (que nous avons déjà vu avec les `std::vector`) permet de laisser une chaîne vide.

```

#include <cassert>
#include <iostream>
#include <string>

int main()
{
    std::string phrase { "Blablabla du texte." };
    phrase.clear();
    std::cout << "Rien ne va s'afficher : "
              << phrase << std::endl;

    return 0;
}

```

Boucler sur une chaîne

On peut bien évidemment boucler sur tous les caractères qui composent une chaîne à l'aide de `for`.

```

#include <iostream>
#include <string>

int main()
{
    std::string const phrase { "Voici un ensemble de lettres." };
    for (char lettre : phrase)
    {
        std::cout << lettre << std::endl;
    }

    return 0;
}

```

Bien entendu, `std::string` est encore plus fourni que ça et permet de faire de nombreuses autres choses. Nous aurons l'occasion d'en reparler au chapitre sur la docu-

mentation ([section Les chaînes de caractères au grand complet](#)). Retenez simplement que `std::string` peut être manipulé comme un tableau.

7.6. Récapitulons !

- Les tableaux nous permettent de stocker un ensemble de valeurs liées entre elles (notes, âges, moyennes, caractères, etc.).
- Il existe plusieurs types de tableaux en C++ : dynamique quand la taille est connue à l'exécution et statique quand elle est connue à la compilation.
- `std::vector` est un tableau dynamique, dont le nombre d'éléments peut augmenter ou diminuer à volonté.
- `std::array` est un tableau statique, dont la taille est fixée définitivement à la compilation, et dont on ne peut pas ajouter ni supprimer d'éléments.
- `std::string` est un tableau dynamique spécialement conçu pour la gestion du texte.

SOLUTIONS

section Calcul de moyenne

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<double> notes {};

    while (true)
    {
        std::cout << "Entre une note (inférieur à 0 pour arrêter) : ";
        double note { 0.0 };

        // N'oublions pas de protéger notre entrée.
        while (!(std::cin >> note))
        {
            std::cout << "Entrée invalide. Recommence." << std::endl;
            std::cin.clear();
            std::cin.ignore(255, '\n');
        }

        if (note < 0)
        {
            // On s'arrête si la note est inférieur à 0.
            break;
        }

        // On ajoute la note à la liste des notes.
        notes.push_back(note);
    }

    if (!std::empty(notes))
    {
        double total{};
        for (auto const note : notes)
        {
            total += note;
        }

        std::cout << "Ta moyenne est de " << total / notes.size()
        << "." << std::endl;
    }
    else
    {
        std::cout << "Tu n'as pas rentré de notes, je ne peux pas
        calculer ta moyenne." << std::endl;
    }
}
```

SOLUTIONS

```
    return 0;
}
```

Aviez-vous pensé à prendre en compte si l'utilisateur ne rentre aucune note car sa seule entrée est négative ?

section Minimum et maximum

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> const entiers { 5, -2, 74, 455, -12 };
    int minimum { entiers[0] };
    int maximum { entiers[0] };

    for (auto const entier : entiers)
    {
        if (entier < minimum)
        {
            minimum = entier;
        }

        if (entier > maximum)
        {
            maximum = entier;
        }
    }

    std::cout << "Le minimum est " << minimum << "." << std::endl;
    std::cout << "Le maximum est " << maximum << "." << std::endl;

    return 0;
}
```

SOLUTIONS

section Séparer les pairs des impairs

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> const entiers{ 5, -2, 74, 455, -12, 10, 11 };
    std::vector<int> pairs {};
    std::vector<int> impairs {};

    for (auto const entier : entiers)
    {
        if (entier % 2 == 0)
        {
            pairs.push_back(entier);
        }
        else
        {
            impairs.push_back(entier);
        }
    }

    std::cout << "Les pairs sont : " << std::endl;
    for (int const entier_pair : pairs)
    {
        std::cout << entier_pair << std::endl;
    }

    std::cout << std::endl << "Les impairs sont : " << std::endl;
    for (int const entier_impair : impairs)
    {
        std::cout << entier_impair << std::endl;
    }

    return 0;
}
```


SOLUTIONS

section Compter les occurrences d'une valeur

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<char> const lettres
    { 'a', 'b', 'b', 'c', 'b', 'd', 'd', 'a' };
    char const lettre_a_compter { 'b' };
    int total { 0 };

    for (auto const lettre : lettres)
    {
        if (lettre == lettre_a_compter)
        {
            ++total;
        }
    }

    std::cout << "La lettre '" << lettre_a_compter
               << "' est présente " << total << " fois." << std::endl;

    return 0;
}
```

8

Déployons la toute puissance des conteneurs

Depuis le début du livre, nous avons étudié deux types de conteneurs très utiles : les tableaux dynamiques `std::vector` et les tableaux statiques `std::array`. Nous avons vu qu'on pouvait accéder à un élément grâce aux crochets `[]`. Cependant cette solution n'est pas très générique, puisque avec d'autres conteneurs C++, comme `std::list`, les crochets ne sont pas disponibles.

Ce chapitre va donc introduire une solution générique qui porte le nom d'*itérateurs*, largement employés dans la bibliothèque standard notamment avec les *algorithmes*.

8.1. La pièce maîtresse : les itérateurs

Qu'est-ce qu'un itérateur ? Littéralement, il s'agit de quelque chose dont on se sert pour *itérer*. Dans notre cas, c'est sur des conteneurs que nous voulons itérer. Et qui dit concept de boucle, dit accès aux éléments un par un. Nous avons vu précédemment les crochets `[]` avec `std::array` et `std::vector`. Il s'agit ici du même concept, mais encore plus puissant, car totalement indépendant du conteneur utilisé !

Un itérateur peut accomplir ceci puisqu'il sait à quel endroit de la séquence regarder. On dit que l'itérateur est un *curseur sur l'élément*. Et en se déplaçant d'emplacement en emplacement, il peut ainsi parcourir l'ensemble d'une collection de données.

Déclaration d'un itérateur

Pour déclarer un itérateur, rien de plus simple, vous connaissez déjà la moitié de la syntaxe. Schématiquement, le code ressemblera à ça. Notez que le type de l'identificateur doit correspondre à celui du conteneur associé : on ne peut pas utiliser un itérateur de `std::vector<double>` pour parcourir un `std::vector<char>`.

```
// Dans le cas d'un conteneur modifiable.  
std::/* Le conteneur utilisé. */::iterator identificateur { /* L'élément  
    sur lequel pointer. */ };
```

```
// Dans le cas d'un conteneur const.
std::/* Le conteneur utilisé. */::const_iterator identificateur { /*
    L'élément sur lequel pointer. */ };
```

Pour raccourcir le code et se simplifier la vie, on peut bien entendu faire appel à l'[inférence de type](#), à l'aide d'[auto](#). C'est ce que nous allons faire dans le reste de ce livre.

```
auto identificateur { /* L'élément sur lequel pointer. */ };
```

Début et fin d'un conteneur

Notez, dans la déclaration précédente, le commentaire `/* L'élément sur lequel pointer. */`. Deux fonctions, dans un premier temps, vont nous intéresser :

- `std::begin`, un curseur vers le premier élément du conteneur ;
- `std::end`, un curseur vers un *élément virtuel inexistant*, qui se trouve après le dernier élément.



Pourquoi ne pas pointer vers le dernier élément de la collection plutôt ?

Pour une raison simple : comment faites-vous donc pour déterminer que vous êtes à la fin de la collection ? En effet, si `std::end` fait référence au dernier élément valide de la collection, qu'est-ce qui vous prouve que vous êtes à la fin ? C'est pour cela que `std::end` renvoie un itérateur sur un élément inexistant, qui indique la fin de la collection.

```
#include <vector>

int main()
{
    std::vector<int> const tableau { -1, 28, 346 };
    // Déclaration explicite d'un itérateur de std::vector d'entiers.
    std::vector<int>::const_iterator debut_tableau
    { std::begin(tableau) };
    // Déclaration plus courte en utilisant auto.
    auto fin_tableau { std::end(tableau) };

    return 0;
}
```

Accéder à l'élément pointé

Nous savons comment pointer vers le premier élément de la collection, ainsi que signaler sa fin, mais ça ne nous avance pas vraiment. Il nous manque l'accès aux éléments. Pour cela, il faut *déréférencer* l'itérateur en plaçant une étoile `*` devant celui-ci. L'exemple suivant affiche la valeur sur laquelle pointe l'itérateur, c'est-à-dire le premier élément du tableau (ici `T`).

```
#include <iostream>
#include <string>

int main()
{
    std::string tableau { "Tu texte." };
    auto debut_tableau { std::begin(tableau) };

    // On affiche le premier élément du tableau.
    std::cout << *debut_tableau << std::endl;
    // Et on peut même le modifier, comme ce qu'on fait
    // avec les références, ou les crochets.
    *debut_tableau = 'D';
    // On vérifie.
    std::cout << tableau << std::endl;

    return 0;
}
```

Attention > Ne tentez pas de *déréférencer* `std::end` ! Vous accéderiez à un espace mémoire inconnu, ce qui entraînerait un comportement indéterminé, donc tout et n'importe quoi potentiellement. Nous insistons, ne tentez pas d'accéder à la valeur pointée par `std::end`, ni de la modifier.

Se déplacer dans une collection

Déplacement vers l'élément suivant

Maintenant que nous sommes capables d'afficher le premier élément, il serait bien de faire de même avec les autres. Nous pouvons, à partir d'un itérateur sur un élément, récupérer le suivant en appliquant l'opérateur d'incréméntation `++`.

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> tableau { -1, 28, 346 };
    auto debut_tableau { std::begin(tableau) };
}
```

```

std::cout << *debut_tableau << std::endl;

// On incrémente l'itérateur pour pointer sur l'élément suivant.
++debut_tableau;

// On affiche le deuxième élément.
std::cout << *debut_tableau << std::endl;

return 0;
}

```

Utilisation concrète

Nous avons maintenant tout ce qu'il nous faut pour afficher l'ensemble d'une collection. L'exemple suivant montre comment parcourir un conteneur à l'aide d'une boucle `for` classique et d'itérateurs. Vous noterez l'utilisation d'un identifiant `it`, qui est l'équivalent du `i` rencontré dans les boucles.

```

#include <iostream>
#include <vector>

int main()
{
    std::vector<int> tableau { -1, 28, 346, 84 };

    for (auto it { std::begin(tableau) }; it != std::end(tableau); ++it)
    {
        std::cout << *it << std::endl;
    }

    return 0;
}

```

Déplacement vers l'élément précédent

La plupart des conteneurs fournissent aussi l'opération inverse, c'est-à-dire récupérer l'élément précédent, grâce à l'opérateur de décrémentation `--`. Celui-ci permet, par exemple, de récupérer le dernier élément d'un conteneur et l'afficher.

```

#include <iostream>
#include <vector>

int main()
{
    std::vector<int> tableau { -1, 28, 346, 84 };
    auto iterateur_dernier_element { std::end(tableau) };

    // On déplace l'itérateur sur l'élément précédent.
    --iterateur_dernier_element;
}

```

```
// Et on affiche pour vérifier.
std::cout << *iterateur_dernier_element << std::endl;

return 0;
}
```

Se déplacer de plusieurs éléments

Admettons que vous vouliez vous déplacer de plusieurs éléments, disons pour récupérer le cinquième. N’y a-t-il pas un autre moyen que d’utiliser cinq fois l’opérateur `++` ? La réponse dépend du conteneur que vous utilisez. En effet, en fonction de celui-ci, les itérateurs derrière ne proposent pas exactement les mêmes fonctionnalités.

Les conteneurs comme `std::vector`, `std::array` ou `std::string`, que nous avons découverts au chapitre précédent, sont ce qu’on appelle des *conteneurs à accès direct* [*random access* en anglais], c’est-à-dire qu’on peut accéder directement à n’importe quel élément. Cela est vrai en utilisant les crochets `[]` (c’est même à leur présence qu’on reconnaît qu’un conteneur est à accès direct), et c’est également vrai pour les itérateurs. Ainsi, on peut directement écrire ceci quand on utilise un conteneur à accès direct.

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> tableau { -1, 28, 346, 84, -94, 31, 784 };
    // Notez le +4 pour accéder au cinquième élément,
    // car comme avec les crochets, on commence à 0.
    auto itérateur_cinquieme_element { std::begin(tableau) + 4 };

    std::cout << *itérateur_cinquieme_element << std::endl;

    return 0;
}
```

Il existe d’autres conteneurs (comme `std::list`) qui sont dit *bidirectionnels*, parce qu’ils autorisent le déplacement vers un élément précédent ou suivant, mais pas plusieurs d’un coup. Si vous remplacez, dans le code précédent, `std::vector` par `std::list`, vous verrez qu’il ne compilera pas.

Note > Il existe, dans la bibliothèque standard, cinq types différents d’itérateurs. Nous en avons déjà vu deux ici. Nous aurons l’occasion de voir les trois autres au fur et à mesure que nous progresserons dans ce livre.

const et les itérateurs

Le concept d'itérateur réclame des explications supplémentaires concernant l'utilisation de `const`. En effet, avec les itérateurs, on distingue la constance de l'itérateur lui-même de la constance de l'objet pointé. Pour parler plus concrètement, appliquer `const` à un itérateur rend celui-ci constant, c'est-à-dire qu'on ne peut pas, une fois assigné, le faire pointer sur autre chose. Par contre, la valeur pointée est tout à fait modifiable.

```
#include <string>
#include <iostream>

int main()
{
    std::string mot { "Bonjour" };

    // Itérateur constant sur un élément modifiable.
    auto const it { std::begin(mot) };

    // Ceci est impossible.
    // it = std::end(mot);

    // Ceci est parfaitement autorisé.
    *it = 'B';

    std::cout << mot << std::endl;
    return 0;
}
```

Pour garantir la constance des éléments pointés, on va utiliser deux fonctions très proches de celles vues plus haut. Il s'agit de `std::cbegin` et de `std::cend`, pour `const begin` et `const end`. L'exemple suivant vous montre la différence.

```
#include <string>

int main()
{
    std::string mot { "Bonjour" };

    // Itérateur constant sur un élément modifiable.
    auto const itereur_constant { std::begin(mot) };

    // Ceci est impossible.
    // it = std::end(mot);

    // Ceci est parfaitement autorisé.
    *itereur_constant = 'B';

    auto itereur_non_constant { std::cbegin(mot) };
}
```

```
// Ceci est impossible.
// *itérateur_non_constant = 'D';

// Ceci est parfaitement autorisé.
itérateur_non_constant = std::cend(mot) - 1;

return 0;
}
```

On peut, bien entendu, combiner les deux formes pour obtenir un itérateur constant sur un élément constant.

Itérer depuis la fin

Il existe encore une troisième paire de fonctions, qui vont nous autoriser à parcourir une collection de la fin vers le début. Elles se nomment `std::rbegin` (reverse begin) et `std::rend` (reverse end). Elles s'utilisent exactement comme les précédentes, leur utilisation est transparente.

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> const tableau { -1, 28, 346, 84 };

    // Affiche depuis la fin, en commençant par 84.
    for (auto it { std::rbegin(tableau) }; it != std::rend(tableau); ++it)
    {
        std::cout << *it << std::endl;
    }

    return 0;
}
```

Et comme pour les autres, les itérateurs peuvent être déclarés comme constants avec `std::crbegin` et `std::crend`.

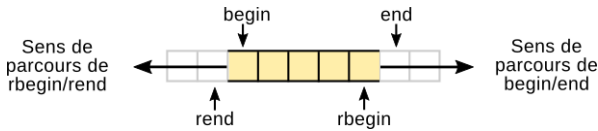


Pourquoi avoir des itérateurs spéciaux ? Pourquoi ne pas partir de `std::end` et itérer jusqu'à `std::begin` ?

La convention est qu'en C et en C++, les algorithmes de parcours s'écrivent toujours du premier élément inclus jusqu'au dernier exclu. Si l'on part de `std::end`, on va se déplacer vers la droite sans jamais retomber sur `std::begin` ; le programme plantera bien avant. Voilà pourquoi il existe des itérateurs spécialement conçus pour parcourir une collection de la fin vers le début.

Un schéma créé par Guillaume Belz pour son cours C++ illustre bien cela.

Figure 8.1 : *Itérateurs classiques et reverses*



Alors, pourquoi ne pas partir de la fin et aller vers le début en décrémentant ?

Vous pourriez... mais ça serait tellement plus pénible. Il faudrait gérer les cas spéciaux que sont la valeur de début et la valeur de fin. Ou bien réécrire tous les algorithmes pour prendre en compte la décrémentation. Ou bien... utiliser `std::rbegin` et `std::rend` de manière totalement transparente.

Utilisation conjointe avec les conteneurs

Terminons cette section en présentant une possibilité d'initialisation des conteneurs qui nous était inconnue jusque-là. Grâce aux itérateurs, nous pouvons facilement créer des sous-ensembles à partir d'une collection d'éléments, comme un tableau qui ne contiendrait que les éléments 2 à 5 d'un autre tableau. Cela se fait très simplement : il suffit de donner en argument, entre accolades {}, l'itérateur de début et l'itérateur de fin.

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> collection { 1, -8, 4, 654, -7785, 0, 42 };
    // On passe l'itérateur de début et celui de fin.
    std::vector<int> sous_ensemble {
        std::begin(collection) + 1,
        std::begin(collection) + 5
    };

    // On affiche pour vérifier.
    for (auto element : sous_ensemble)
    {
        std::cout << element << std::endl;
    }

    return 0;
}
```

Certains conteneurs, comme `std::vector`, `std::list` ou encore `std::string`, proposent également une fonction nommée `erase`, qui se base sur le principe des itérateurs pour supprimer plusieurs éléments d'une collection. On peut ainsi supprimer tous les éléments à partir du cinquième jusqu'à la fin, ou bien les trois premiers.

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main()
{
    std::vector<int> nombres { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    // Certains éléments.
    nombres.erase(std::begin(nombres) + 2, std::begin(nombres) + 5);

    for (auto nombre : nombres)
    {
        std::cout << nombre << std::endl;
    }

    std::cout << std::endl;

    std::string phrase { "Voici une phrase beurk !" };
    // Jusqu'à la fin.
    phrase.erase(std::begin(phrase) + 16, std::end(phrase));
    std::cout << phrase << std::endl;

    return 0;
}
```

8.2. Des algorithmes à gogo !

Vous allez voir maintenant, les itérateurs vont devenir vraiment intéressants et concrets, car les algorithmes de la bibliothèque standard en tirent pleinement parti. Ces derniers, assez nombreux, sont des fonctions déjà programmées et fortement optimisées pour manipuler des collections, permettant ainsi de les trier, de récupérer des éléments satisfaisant une condition, etc. Nous n'en verrons que quelques-uns dans ce chapitre, mais une fois le principe compris, vous n'aurez aucune difficulté à utiliser les autres.

Note > La plupart des algorithmes se trouvent dans le fichier d'en-tête `<algorithm>`, pensez donc à l'inclure.

`std::count` — compter les occurrences d'une valeur

Commençons avec un algorithme simple, mais on ne peut plus utile : compter les occurrences d'une certaine valeur dans une collection. Cela se fait avec l'algorithme `std::count`. Celui-ci attend trois paramètres :

- un itérateur pointant sur le début de la séquence ;
- un itérateur pointant sur la fin de la séquence ;
- l'élément à compter.

La force de l'algorithme, c'est qu'il ne nous impose pas de chercher dans toute la collection. En effet, comme on manipule ici des itérateurs, rien ne nous empêche de compter les occurrences d'une valeur seulement entre le troisième et le septième élément de la collection, ou entre le cinquième et le dernier.

```
#include <algorithm>
#include <iostream>
#include <string>

int main()
{
    std::string const phrase {
        "Exemple illustrant le Guide du C++ moderne."
    };

    // Toute la collection.
    auto const total_phrase {
        std::count(std::begin(phrase), std::end(phrase), 'e')
    };
    std::cout << "Dans la phrase entière, il y a "
        << total_phrase << " 'e' minuscule.\n";

    // Un sous-ensemble seulement de la collection.
    auto const total_premier_mot {
        std::count(std::begin(phrase), std::begin(phrase) + 7, 'e')
    };
    std::cout << "Dans le premier mot, il y a "
        << total_premier_mot << " 'e' minuscule.\n";

    return 0;
}
```

Cet exemple utilise `std::string`, mais on aurait tout aussi bien pu employer `std::vector` ou `std::array`. Par contre, `std::list` ne convient pas parce que... oui ! on a écrit `std::begin(phrase) + 7`, ça suit dans le fond. Heureusement, une solution existe, que nous allons vous montrer de suite.

`std::find` — trouver un certain élément

Le code précédent n'est pas de très bonne qualité, puisque nous avons écrit **+7** directement, en dur. Outre qu'on ne peut utiliser `std::list`, cela n'est pas très flexible : si le premier mot devient plus long ou plus court, on fausse les résultats. Alors qu'avec un itérateur qui pointerait sur le premier espace, on serait assuré d'avoir plus de souplesse. C'est justement ce que propose `std::find`, qui prend trois paramètres.

- un itérateur pointant sur le début de la séquence ;
- un itérateur pointant sur la fin de la séquence ;
- l'élément à rechercher.

Cette fonction retourne un *itérateur sur le premier élément correspondant à la recherche*, ou bien le deuxième argument si rien n'est trouvé.

```
#include <algorithm>
#include <iostream>
#include <string>

int main()
{
    std::string const phrase
    {
        "Exemple illustrant le Guide du C++ moderne."
    };

    // On obtient un itérateur pointant sur le premier espace trouvé.
    // Si l'on n'avait rien trouvé, on aurait obtenu std::end(phrase)
    // comme valeur de retour.
    auto const itérateur_premier_mot
    {
        std::find(std::begin(phrase),
            std::end(phrase), ' ')
    };

    // On peut donc s'en servir ici. Même si l'on modifie la longueur
    // du premier mot, on est assuré que cette solution marche.
    auto const total_premier_mot
    {
        std::count(std::begin(phrase),
            itérateur_premier_mot, 'e')
    };

    std::cout << "Dans le premier mot, il y a "
        << total_premier_mot << " 'e' minuscule."
        << std::endl;

    return 0;
}
```

On précise de nouveau pour être bien clair : `std::find` retourne un itérateur sur le PREMIER élément trouvé, pas le deuxième ou le troisième. Mais avec une simple boucle, rien ne nous empêche de récupérer la deuxième occurrence, ou la troisième.

Cela nous donne d'ailleurs une excellente idée d'exercice.

- **8.1.** En utilisant les deux fonctions que nous venons de voir, essayez d'écrire un programme qui compte le nombre de 'e' dans chaque mot d'une phrase.

Astuce > On peut initialiser une chaîne de caractères avec un itérateur de début et de fin, comme n'importe quel autre conteneur.

✧ Voir la solution

`std::sort` — trier une collection

L'algorithme `std::sort` (fonctionnant comme les autres avec une paire d'itérateurs) permet de trier dans l'ordre croissant une collection d'éléments (nombres, caractères, etc.). Bien entendu, il ne marchera que sur des conteneurs modifiables, c'est-à-dire non `const`. Voici un exemple tout simple qui ne devrait guère vous surprendre.

```
#include <algorithm>
#include <iostream>
#include <vector>

int main()
{
    std::vector<double> constantes_mathematiques
    {
        2.71828,
        3.1415,
        1.0836,
        1.4142,
        1.6180
    };

    std::sort(
        std::begin(constantes_mathematiques),
        std::end(constantes_mathematiques)
    );

    for (auto constante : constantes_mathematiques)
    {
        std::cout << constante << std::endl;
    }

    return 0;
}
```

Si vous avez essayé de changer `std::vector` en `std::list`, vous vous êtes rendu compte que le code ne compile plus. En effet, `std::sort` attend en paramètre des itérateurs à accès direct, comme ceux de `std::array`, `std::vector` et `std::string`. À l’opposé, `std::list` n’utilise que des itérateurs bidirectionnels. Il est donc impossible de trier une `std::list` en utilisant cet algorithme.

LE CAS DE `STD::LIST`

Le type `std::list` possède une fonction `sort` qui lui est propre et fait la même chose.

```
#include <iostream>
#include <list>

int main()
{
    std::list<int> liste { 4, -8, 45, 2 };
    liste.sort();

    for (int i : liste)
    {
        std::cout << i << std::endl;
    }
    return 0;
}
```

`std::reverse` — inverser l’ordre des éléments d’une collection

Parlons de `std::reverse`, qui inverse la position des éléments compris entre l’itérateur de début et l’itérateur de fin. Un exemple va vous montrer clairement ce qu’il fait.

```
#include <algorithm>
#include <iostream>
#include <list>

int main()
{
    std::list<int> nombres { 2, 3, 1, 7, -4 };
    std::cout << "Avant :\n";
    for (auto nombre : nombres)
    {
        std::cout << nombre << std::endl;
    }

    std::reverse(std::begin(nombres), std::end(nombres));

    std::cout << "\nAprès :\n";
    for (auto nombre : nombres)
```

```

{
    std::cout << nombre << std::endl;
}

return 0;
}

```

`std::remove` — suppression d'éléments

Admettons maintenant que vous voulez supprimer toutes les lettres 'b' d'une chaîne. Avec `std::remove`, il n'y a aucun problème à ça. L'algorithme va supprimer toutes les occurrences de la lettre dans la chaîne. Et comme l'algorithme ne peut pas modifier la taille du conteneur, il nous renvoie un itérateur qui nous signale la fin de la collection. Il ne reste plus qu'à supprimer le tout grâce à `erase`.

```

#include <algorithm>
#include <iostream>
#include <string>

int main()
{
    std::string paroles
    {
        "I'm blue, da ba dee da ba daa !"
    };

    auto itérateur_fin
    {
        std::remove(std::begin(paroles), std::end(paroles), 'b')
    };
    paroles.erase(itérateur_fin, std::end(paroles));

    std::cout << paroles << std::endl;
    return 0;
}

```

`std::search` — rechercher un sous-ensemble dans un ensemble

Cette fois, avec `std::search`, ça n'est pas un unique élément que nous allons chercher, mais bien un sous-ensemble dans un autre ensemble, comme, par exemple, un mot dans une phrase. Cette fois, pas moins de quatre arguments sont attendus :

- l'itérateur de début de la collection à fouiller ;
- l'itérateur de fin ;
- l'itérateur de début de la sous-collection à rechercher ;
- l'itérateur de fin.

Tout comme `std::find`, si rien n'est trouvé, le deuxième argument (l'itérateur de fin de la collection à parcourir) est renvoyé. Voici donc un exemple pour illustrer le fonctionnement de l'algorithme.

```
#include <algorithm>
#include <iostream>
#include <string>

int main()
{
    std::string const phrase
    {
        "Un exemple de phrase avec plusieurs mots."
    };
    auto const debut { std::begin(phrase) };
    auto const fin { std::end(phrase) };

    // Rappel : std::boolalpha permet d'afficher
    // 'true' ou 'false' et non '1' ou '0'.
    std::cout << std::boolalpha;

    std::string mot { "mot" };
    std::cout << (
        std::search(debut, fin, std::begin(mot), std::end(mot))
        != fin) << std::endl;

    mot = "exemple";
    std::cout << (
        std::search(debut, fin, std::begin(mot), std::end(mot))
        != fin) << std::endl;

    return 0;
}
```

`std::equal` — vérifier l'égalité de deux ensembles

Comment testeriez-vous que deux ensembles sont équivalents, autrement dit qu'ils possèdent les mêmes éléments ? La réponse vient du titre : avec l'algorithme `std::equal`. Celui-ci prend plusieurs paramètres :

- l'itérateur de début du premier ensemble ;
- l'itérateur de fin du premier ensemble ;
- l'itérateur de début du deuxième ensemble ;
- l'itérateur de fin du deuxième ensemble.

Si les deux ensembles n'ont pas la même taille, alors l'égalité est forcément impossible. Sinon, l'algorithme va comparer les deux ensembles élément par élément.


```

#include <algorithm>
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> premier { 1, 2, 3, 4 };
    std::vector<int> second { 1, 2, 3, 4, 5 };
    std::vector<int> troisieme { 1, 2, 3, 4 };

    std::cout << std::boolalpha;

    std::cout << std::equal(
        std::begin(premier), std::end(premier),
        std::begin(second), std::end(second))
        << std::endl;

    std::cout << std::equal(
        std::begin(premier), std::end(premier),
        std::begin(troisieme), std::end(troisieme))
        << std::endl;
}

```

Et tant d'autres

Il y a tellement d'algorithmes différents que nous n'allons pas tous les présenter. Vous en avez déjà vu plusieurs, et tous fonctionnent sur le même principe : avec des itérateurs. Utiliser les autres algorithmes ne devrait donc pas vous poser de problème.

8.3. Personnalisation à la demande

Jusqu'ici, nos algorithmes sont figés. Par exemple, `std::sort` ne trie que dans l'ordre croissant. Mais les concepteurs de la bibliothèque standard ont pensé à ça et ont écrit leurs algorithmes de telle manière qu'on peut personnaliser leur comportement. Mine de rien, cette possibilité les rend encore plus intéressants ! Voyons ça ensemble.

Le prédicat, pivot de la personnalisation des algorithmes

Un prédicat est une expression qui prend, ou non, des arguments et renvoie un booléen. Le résultat sera donc `true` ou `false`, selon que la condition est vérifiée ou non. Littéralement, un prédicat répond à des questions ressemblant à celles ci-dessous.

- Est-ce que l'élément est impair ?
- Est-ce que les deux éléments `a` et `b` sont égaux ?

- Est-ce que les deux éléments sont triés par ordre croissant ?

La très grande majorité des algorithmes utilisent les prédicats, même ceux vus plus haut, sans que vous le sachiez. Par exemple, `std::sort` utilise, par défaut, le prédicat `<`, qui compare si un élément `a` est strictement inférieur à un autre élément `b`, ce qui explique pourquoi l'algorithme trie une collection dans l'ordre croissant. Mais il est tout à fait possible de lui indiquer de faire autre chose.

La bibliothèque standard contient déjà quelques prédicats, regroupés dans le fichier d'en-tête `<functional>`.

Trier une liste dans l'ordre décroissant

Faisons connaissance avec `std::greater`. Il s'agit d'un *foncteur*, c'est-à-dire un objet qui agit comme une fonction. Les détails internes ne nous intéressent pas pour l'instant, nous y reviendrons *plus tard*, quand nous aurons les connaissances requises.

Ce foncteur compare deux éléments et retourne `true` si le premier est plus grand que le deuxième, sinon `false`. Voici quelques exemples. Notez qu'on doit préciser, entre chevrons `<>`, le type des objets comparés.

```
#include <functional>
#include <iostream>

int main()
{
    std::cout << std::boolalpha;
    // On peut l'utiliser directement.
    std::cout
        << std::greater<int>{}(2, 3) << std::endl;
    std::cout
        << std::greater<int>{}(6, 3) << std::endl;
    std::cout
        << std::greater<double>{}(0.08, 0.02)
        << std::endl;

    // Ou bien à travers une variable,
    // qu'on appelle comme une fonction.
    std::greater<int> foncteur {};
    std::cout << foncteur(-78, 9) << std::endl;
    std::cout << foncteur(78, 8) << std::endl;

    return 0;
}
```

Afin de personnaliser le comportement de `std::sort` (notre fonction de tri vue plus haut), il suffit de lui passer ce foncteur. Observez par vous-même.

```

#include <algorithm>
#include <functional>
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> tableau { -8, 5, 45, -945 };
    std::sort(
        std::begin(tableau), std::end(tableau), std::greater<int> {}
    );

    for (auto x : tableau)
    {
        std::cout << x << std::endl;
    }

    return 0;
}

```

Somme et produit

Dans le fichier d'en-tête `<numeric>`, il existe une fonction nommée `std::accumulate`, qui permet de faire la somme d'un ensemble de valeurs à partir d'une valeur de départ. Ça, c'est l'opération par défaut, mais on peut très bien lui donner un autre prédicat, comme `std::multiplies`, afin de calculer le produit des éléments d'un ensemble.

```

#include <functional>
#include <iostream>
#include <numeric>
#include <vector>

int main()
{
    std::vector<int> const nombres { 1, 2, 4, 5, 10 };

    // On choisit 0 comme élément de départ
    // pour calculer la somme, car 0 est
    // l'élément neutre de l'addition.
    auto somme
    {
        std::accumulate(std::cbegin(nombres), std::cend(nombres), 0)
    };
    std::cout << somme << std::endl;

    // On choisit 1 comme élément de départ pour
    // calculer le produit, car 1 est l'élément
    // neutre de la multiplication.
    auto produit
    {
        std::accumulate(

```

```

        std::cbegin(nombres),
        std::cend(nombres),
        1,
        std::multiplies<int> {}
    )
};
std::cout << produit << std::endl;

return 0;
}

```

Prédicats pour caractères

Il existe, dans le fichier d'en-tête `<cctype>`, des prédicats vérifiant des conditions *uniquement sur un caractère*. Ils permettent de vérifier si un caractère est une lettre, un chiffre, un signe de ponctuation, etc. Voyez par vous-même.

```

#include <cctype>
#include <iostream>

int main()
{
    char const lettre { 'A' };

    std::cout << "Est-ce que "
        << lettre << " est une minuscule ? "
        << islower(lettre) << std::endl;
    std::cout << "Est-ce que "
        << lettre << " est une majuscule ? "
        << isupper(lettre) << std::endl;
    std::cout << "Est-ce que "
        << lettre << " est un chiffre ? "
        << isdigit(lettre) << std::endl;

    char const chiffre { '7' };

    std::cout << "Est-ce que "
        << chiffre << " est un chiffre ? "
        << isdigit(chiffre) << std::endl;
    std::cout << "Est-ce que "
        << chiffre << " est un signe de ponctuation ? "
        << ispunct(chiffre) << std::endl;

    return 0;
}

```

Note > Ces fonctions sont héritées du C, c'est pour cela qu'elles ne sont pas précédées de l'habituel `std::`. C'est également la raison pour laquelle elles renvoient des entiers et non des booléens, comme on aurait pu s'y attendre.

On peut faire des choses intéressantes avec ça. Par exemple, en utilisant l'algorithme `std::all_of`, qui vérifie que tous les éléments d'un ensemble respectent une propriété, on peut vérifier si tous les caractères d'une chaîne sont des chiffres, pour valider qu'il s'agit d'un numéro de téléphone.

```
#include <algorithm>
#include <cctype>
#include <iostream>
#include <string>

int main()
{
    std::string const numero { "0185017204" };
    if (std::all_of(std::cbegin(numero), std::cend(numero), isdigit))
    {
        std::cout
            << "C'est un numéro de téléphone correct."
            << std::endl;
    }
    else
    {
        std::cout << "Entrée invalide." << std::endl;
    }

    return 0;
}
```

Note > Au chapitre [Des fonctions somme toute lambdas](#), nous apprendrons à écrire nos propres prédicats, ce qui nous donnera encore plus de pouvoir sur nos algorithmes.

8.4. Entraînez-vous !

Palindrome

Vous devez tester si une chaîne de caractères est un palindrome, c'est-à-dire un mot pouvant être lu indifféremment de gauche à droite ou de droite à gauche. Un exemple : kayak.

✧ [Voir la solution](#)

`string_trim` — suppression des espaces

Allez, un exercice plus dur, pour vous forger le caractère. Le but est de reproduire une fonction qui existe dans de nombreux autres langages, comme C# ou Python, et qui permet de supprimer les espaces, au début et à la fin d'une chaîne de caractères. Cela

signifie non seulement les espaces ' ' mais aussi les tabulations '\t', les retours à la ligne '\n', etc. Pour vous aider, vous allez avoir besoin de deux choses :

- le prédicat `isspace`, pour savoir si un caractère est un type d'espace ou non ;
- l'algorithme `std::find_if_not`, qui permet de trouver, dans un ensemble délimité par deux itérateurs, le premier caractère qui ne respecte pas une condition.

Pour vous faciliter la vie, découpez l'exercice en deux. Faites d'abord la fonction permettant de supprimer tous les espaces à gauche, puis pour ceux à droite.

✧ [Voir la solution](#)

Couper une chaîne

Une autre opération courante, et qui est fournie nativement dans d'autres langages comme Python ou C#, consiste à découper une chaîne de caractères selon un caractère donné. Ainsi, si on coupe la chaîne "Salut, ceci est une phrase." en fonction des espaces, on obtient en résultat ["Salut,", "ceci", "est", "une", "phrase.].

Le but est donc que vous codiez un algorithme qui permet de faire ça. Vous allez notamment avoir besoin de la fonction `std::distance`, qui retourne la distance entre deux itérateurs, sous forme d'un nombre entier.

✧ [Voir la solution](#)

8.5. Récapitulons !

- Les itérateurs sont des abstractions représentant un pointeur sur un élément d'un conteneur. Ils rendent ainsi l'usage du conteneur plus transparent.
- Les fonctions `std::begin` et `std::end` sont extrêmement utilisées avec les itérateurs, car elles retournent un pointeur sur le début et la fin d'une collection.
- Les itérateurs demandent de la rigueur pour ne pas manipuler des éléments qui sont hors de la collection, ce qui entraînerait des comportements indéterminés.
- La bibliothèque standard contient des algorithmes, déjà programmés et très optimisés, qu'on peut appliquer à des collections.
- Nous pouvons les personnaliser grâce à des prédicats fournis dans la bibliothèque standard.

SOLUTIONS

Q : 8.1

```
#include <algorithm>
#include <iostream>
#include <string>

int main()
{
    std::string const phrase { "Exemple illustrant le Guide du C++
    moderne, mais un peu plus long." };

    // Pour garder en mémoire le début de chaque mot.
    auto itérateur_precedent { std::begin(phrase) };
    auto itérateur_espace { std::find(std::begin(phrase),
    std::end(phrase), ' ') };

    // Tant qu'on n'est pas à la fin de la phrase.
    while (itérateur_espace != std::end(phrase))
    {
        std::string const mot { itérateur_precedent,
    itérateur_espace };
        auto const total_e { std::count(std::begin(mot),
    std::end(mot), 'e') };

        std::cout << "Dans le mot '" << mot << "', il y a "
        << total_e << " fois la lettre 'e'." << std::endl;

        // On incrémente pour ne pas garder l'espace au début,
        // car itérateur_espace pointe déjà sur un espace.
        ++itérateur_espace;
        // On met à jour notre itérateur de sauvegarde.
        itérateur_precedent = itérateur_espace;
        // On cherche la 1re occurrence dans le nouveau sous-ensemble.
        itérateur_espace = std::find(itérateur_espace,
    std::end(phrase), ' ');
    }

    // Une dernière fois pour analyser les caractères restants.
    std::string const dernier_mot { itérateur_precedent,
    std::end(phrase) };
    std::cout << "Dans le dernier mot '" << dernier_mot
    << "', il y a " << std::count(itérateur_precedent,
    std::end(phrase), 'e')
    << " fois la lettre 'e'." << std::endl;

    return 0;
}
```

SOLUTIONS

section Palindrome

L'astuce consiste à regarder si les deux ensembles de lettres sont les mêmes, en partant de la gauche vers la droite pour le premier (itérateur classique) et l'opposé pour le deuxième (*reverse iterator*).

```
#include <algorithm>
#include <iostream>
#include <string>

int main()
{
    std::string const s1 { "azerty" };
    std::string const s2 { "radar" };

    std::cout << std::boolalpha;
    std::cout << std::equal(std::begin(s1), std::end(s1),
std::rbegin(s1), std::rend(s1)) << std::endl;
    std::cout << std::equal(std::begin(s2), std::end(s2),
std::rbegin(s2), std::rend(s2)) << std::endl;

    return 0;
}
```


SOLUTIONS

section `string_trim` — suppression des espaces

```
#include <algorithm>
#include <cctype>
#include <iostream>
#include <string>

int main()
{
    std::string test { "\n\tHello !\n\t" };
    std::cout << "Avant modification : " << test << std::endl;

    // On utilise l'itérateur fourni par std::find_if_not, qui
    // pointe sur le premier élément qui n'est pas un espace.
    auto premier_non_espace { std::find_if_not(std::begin(test),
std::end(test), isspace) };
    test.erase(std::begin(test), premier_non_espace);
    // On affiche pour tester.
    std::cout << "Suppression au début : " << test << std::endl;

    // En inversant, tous les espaces de fin se retrouvent au début.
    std::reverse(std::begin(test), std::end(test));
    // On reprend le même algorithme.
    premier_non_espace = std::find_if_not(std::begin(test),
std::end(test), isspace);
    test.erase(std::begin(test), premier_non_espace);
    // On revient à l'état initial.
    std::reverse(std::begin(test), std::end(test));
    // On affiche pour tester.
    std::cout << "Suppression à la fin : " << test << std::endl;

    return 0;
}
```

Il est vrai que nous répétons deux fois les instructions pour supprimer les espaces à gauche. Laissez ça en l'état, nous verrons très bientôt une solution pour corriger ce fâcheux problème.

SOLUTIONS

section Couper une chaîne

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main()
{
    std::string texte { "Voici une phrase que je vais couper." };
    char const delimiteur { ' ' };
    std::vector<std::string> parties {};

    auto debut = std::begin(texte);
    auto fin = std::end(texte);
    auto itereur = std::find(debut, fin, delimiteur);

    while (itereur != fin)
    {
        // Grâce à std::distance, nous obtenons la taille du mot.
        std::string mot { debut, debut + std::distance(debut,
            itereur) };

        parties.push_back(mot);
        // +1 pour sauter le délimiteur.
        debut = itereur + 1;
        // Et on recommence.
        itereur = std::find(debut, fin, delimiteur);
    }

    // Ne pas oublier le dernier mot.
    std::string mot { debut, debut + std::distance(debut,
        itereur) };
    parties.push_back(mot);

    // On affiche pour vérifier.
    for (std::string mot : parties)
    {
        std::cout << mot << std::endl;
    }

    return 0;
}
```

9

Des flux dans tous les sens

Depuis le début de ce livre, toutes les données que nous manipulons ont un point commun : leur volatilité. Elles sont en effet stockées dans la mémoire vive de l'ordinateur, et celle-ci se vide dès qu'on l'éteint. Impossible dans ces conditions de sauvegarder des informations, comme une sauvegarde de jeu vidéo ou un résumé sur l'utilisateur.

Le but de ce chapitre est donc d'apprendre à lire et écrire dans des fichiers et, de manière plus générale, dans des flux.

Note > Pour manipuler des fichiers, il faut inclure le fichier d'en-tête `<fstream>`.

9.1. Avant toute chose

Votre ordinateur contient énormément de fichiers, de tous les genres. Vos musiques préférées sont des fichiers, tout comme vos photos de vacances, votre CV Word ou LibreOffice, les fichiers de configuration de vos jeux vidéos, etc. Même les programmes que vous utilisez sont des fichiers !

- Chaque fichier a *un nom*.
- Chaque fichier a *une extension*.

Prendrez-vous une extension ?

Il existe tout un tas d'extensions de fichiers, pour chaque type qui existe. En voici quelques exemples.

- Fichiers audios : `.mp3`, `.oga`, etc.
- Fichiers vidéos : `.mp4`, `.avi`, etc.
- Images : `.jpg`, `.png`, `.gif`, etc.
- Exécutables : `.exe`, `.msi`, `.out`, etc.
- Fichiers textes : `.txt`, `.ini`, `.log`, etc.

Celles-ci sont surtout utiles pour nous, humains, afin de savoir quel type de fichier nous allons manipuler. Mais elles n'influencent en rien le contenu du fichier.

Certains fichiers, comme les fichiers textes, sont très simples à ouvrir et donne accès directement à l'information. D'autres, comme les `.png` ou les `.docx`, doivent être lus en respectant une certaine façon de faire, une norme, afin que l'information fasse sens. Dans le cadre de ce livre, nous ne verrons que les documents textes simples.

Vois sur ton chemin...

Afin de faciliter leur localisation et leur gestion, les fichiers sont classés et organisés sur leur support suivant un *système de fichiers*. C'est lui qui permet à l'utilisateur de répartir ses fichiers dans une arborescence de dossiers et de localiser ces derniers à partir d'un chemin d'accès.

Un *chemin d'accès* permet d'indiquer où est situé un fichier dans ledit système de fichiers. Il contient forcément le nom du fichier concerné, mais également, si nécessaire, un ou plusieurs noms de dossiers, qu'il est nécessaire de traverser pour accéder au fichier depuis la racine.

La *racine* d'un système de fichiers est le point de départ de l'arborescence des fichiers et dossiers. Sous GNU/Linux, il s'agit du répertoire / tandis que sous Windows, chaque lecteur est une racine (comme `c:` par exemple). Si un chemin d'accès commence par la racine, alors celui-ci est dit *absolu* car il permet d'atteindre le fichier depuis n'importe qu'elle position dans l'arborescence. Si le chemin d'accès ne commence pas par la racine, il est dit *relatif* et ne permet de parvenir au fichier que depuis un point précis dans la hiérarchie de fichiers.

Prenons un exemple. Nous avons un fichier `code_coffre_fort.txt` qui se trouve dans un dossier `top_secret`, lui-même dans `documents`, qui se trouve à la racine. Le chemin absolu ressemblera à ça.

- Windows : `C:/documents/top_secret/code_coffre_fort.txt`
- GNU/Linux : `/documents/top_secret/code_coffre_fort.txt`

Par contre, si nous sommes déjà dans le dossier `documents`, alors nous pouvons nous contenter d'utiliser un chemin relatif. Ce chemin ne marchera cependant plus si nous sortons du dossier `documents`.

- `./top_secret/code_coffre_fort.txt`

Le point `.` représente le dossier courant, c'est-à-dire `documents` dans notre exemple.

UN MOT SUR WINDOWS

Depuis les années 1990, Windows supporte les chemins utilisant les slashes /. Si vous ouvrez l'explorateur de fichier et que vous écrivez `C:/Windows/System32`, ça marchera sans problème. C'est la solution que nous vous conseillons.

Pas d'interprétation, on est des brutes

Le caractère `\` est un caractère d'échappement, qu'on utilise pour le retour à la ligne `"\n"` par exemple. Si on veut afficher une chaîne de caractères qui contient un antislash `\`, on est obligé de le doubler et d'écrire `\\`.

Utiliser les séquences d'échappement peut alourdir le code, en plus d'être casse-pieds à écrire. C++ offre une solution alternative qui se nomme *chaînes brutes* [de l'anglais *raw strings*]. Par défaut, une chaîne brute commence par `R"(` et termine par `)"`. Tout ce qui se trouve entre ces deux séquences n'est pas interprété, mais tout simplement ignoré.

```
#include <iostream>

int main()
{
    std::cout << R"(Elle a dit "Va dans le dossier C:\Windows".)"
              << std::endl;
    return 0;
}
```

Et si jamais notre chaîne contient le délimiteur de fin, on peut définir un autre délimiteur. Pour ça, on ajoute les caractères qu'on souhaite entre les guillemets et les parenthèses.

```
#include <iostream>

int main()
{
    // Oups, ça ne marche pas...
    //std::cout << R"(Il a écrit "f(x)" au tableau.)"
    // << std::endl;

    // Heureusement, on peut choisir soi-même ses délimiteurs.
    // Ici, tout ce qui est entre "&( et )&"
    // n'est pas interprété.
    std::cout << R"&(Il a écrit "f(x)" au tableau.)&"
              << std::endl;
    return 0;
}
```

9.2. `std::ofstream` — écrire dans un fichier

Commençons par apprendre à écrire dans un fichier. Nous en aurons besoin si nous voulons sauvegarder un score, les paramètres de configuration d’une application ou n’importe quel résultat de notre programme. Et pour ce faire, nous allons utiliser le type `std::ofstream` (abréviation de *output file stream*, ce qui signifie *flux fichier de sortie*).

Ouvrir le fichier

`std::ofstream` attend en argument le nom du fichier à ouvrir. Par défaut, si le fichier n’existe pas, il est créé.

```
#include <fstream>

int main()
{
    // On veut ouvrir un fichier nommé 'sortie.txt',
    // qui se trouve dans le dossier du projet.
    // S'il existe, il sera ouvert.
    // Sinon, il sera d'abord créé puis ouvert.
    std::ofstream fichier { "sortie.txt" };
    return 0;
}
```

Écrire

Pour écrire dedans, c’est très simple, il suffit d'utiliser les chevrons, comme avec `std::cout`. Voyez par vous-même.

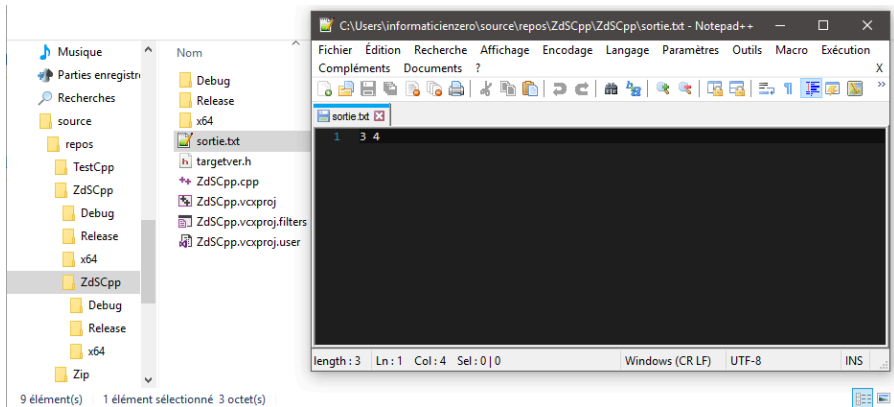
```
#include <fstream>

int main()
{
    std::ofstream fichier { "sortie.txt" };
    // On écrit un 3, un espace et un 4.
    fichier << 3 << " " << 4;

    return 0;
}
```

Comme sur la [Figure 9.1](#), vos deux valeurs et l’espace doivent apparaître dans le fichier.

Figure 9.1 : Écrire dans un fichier



Et, exactement comme pour `std::cout`, nous pouvons écrire des littéraux, mais aussi des variables et des expressions.

```
#include <fstream>
#include <string>

int main()
{
    std::ofstream fichier { "sortie.txt" };
    fichier << 3 << " " << 4;

    int x { 0 };
    // On va à la ligne puis on écrit une équation.
    fichier << '\n' << x << " + 2 = " << x + 2;

    // Pas de problème avec le texte non plus.
    std::string texte { "Voici une phrase." };
    fichier << '\n' << texte;

    return 0;
}
```



J'ai essayé d'écrire une phrase avec des accents et j'ai obtenu un texte avec des caractères bizarres. Que s'est-il passé ?

Normalement, les Linuxiens sont épargnés par ce problème et seuls ceux utilisant Windows sont concernés (pas de chance). En fait, c'est une histoire d'encodage.

Sans entrer dans les détails, il existe différents encodages qui permettent de stocker plus ou moins de caractères. L'ASCII (le premier inventé dans les années 1960)¹ ne permet de stocker que 127 caractères, dont les 26 lettres de l'alphabet anglais, mais pas nos accents français, le ñ espagnol, le ß allemand, etc. D'autres sont apparus et permettent de stocker plus de caractères, dont [UTF-8](#).

Par défaut sous Windows, en C++, quand on manipule une chaîne de caractères, celle-ci est interprétée comme étant de l'ASCII. Comme nos accents français ne sont pas représentables, on obtient du grand n'importe quoi en sortie. La solution consiste à utiliser UTF-8, ce qui se fait en précisant `u8` devant nos chaînes de caractères, puis enregistrez votre code source en UTF-8.

```
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

int main()
{
    std::vector<std::string> const phrases
    {
        u8"Voici un mot important.\n",
        u8"Russe : резиновые перчатки",
        u8"Polonais : gumowe rękawiczki",
        u8"Roumain : mănuși de cauciuc"
    };

    std::ofstream fichier { "sortie.txt" };
    for (auto const & phrase : phrases)
    {
        fichier << phrase << std::endl;
    }

    return 0;
}
```

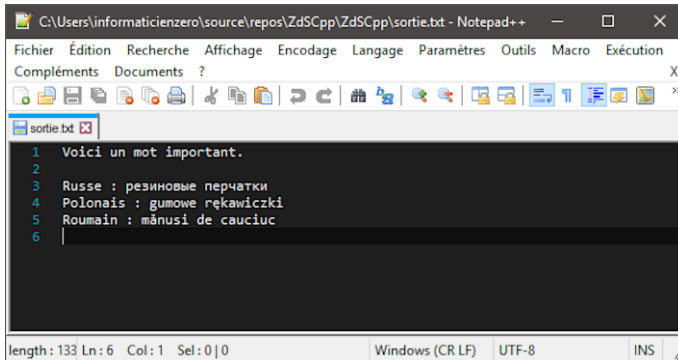
ENREGISTRER EN UTF-8 SOUS VISUAL STUDIO

Pour enregistrer en UTF-8, cliquez sur FICHIER > ENREGISTRER MAIN.CPP SOUS... puis sur la petite flèche à côté du bouton ENREGISTRER et sélectionnez l'option ENREGISTRER AVEC ENCODAGE.... Pour le champ ENCODAGE, choisissez la valeur UNICODE (UTF-8 AVEC SIGNATURE) - PAGE DE CODES 65001 et validez.

¹American Standard Code for Information Interchange.

Il ne vous reste plus qu'à lancer et voir le résultat. Hop, plus de soucis, ça marche maintenant.

Figure 9.2 : Fichier enregistré en UTF-8



Ouvrir sans effacer

Essayez de modifier le fichier vous-même, avec votre éditeur de texte préféré, et ajoutez au tout début une phrase quelconque. Lancez le programme, rouvrez le fichier et horreur ! Votre texte a disparu.

En fait, par défaut, `std::ofstream` tronque le contenu du fichier. Autrement dit, s'il y avait des données écrites précédemment, elles seront perdues dès que le programme rouvrira le fichier. Mais il est possible de régler ce problème très simplement. Lorsque nous initialisons notre objet `std::ofstream`, nous pouvons lui donner un deuxième argument qui est le mode d'ouverture. Dans notre cas, c'est `std::ios::app`. Ici, `ios` veut dire *Input Output Stream* [flux d'entrée et de sortie] ; `app` est le diminutif de *append* [soit *ajouter*].

Notre contenu n'est plus effacé et les données supplémentaires sont ajoutées à la suite.

```
#include <fstream>
#include <iostream>
#include <string>

int main()
{
    std::ofstream fichier { "sortie.txt", std::ios::app };
    fichier << 3 << " " << 4;

    int x { 0 };
    fichier << '\n' << x << " + 2 = " << x + 2;
```

```
std::string texte { u8"Voici une phrase." };
fichier << '\n' << texte;

return 0;
}
```

9.3. `std::ifstream` — lire dans un fichier

Abordons maintenant la lecture, opération elle aussi ô combien importante. Elle permet de charger des paramètres, d'analyser des fichiers, etc. Pour ce faire, nous allons utiliser le type `std::ifstream` (abréviation de *input file stream*, ce qui signifie *flux fichier d'entrée*).

Ouvrir le fichier

L'ouverture se fait de la même façon que pour `std::ofstream`, c'est-à-dire en donnant le nom du fichier à ouvrir. Sauf que cette fois, il faut que le fichier existe déjà. C'est logique : vous ne pouvez pas lire un fichier qui n'existe pas.

Créez donc avec moi un fichier appelé `texte.txt`.

```
#include <fstream>

int main()
{
    std::ifstream fichier { "texte.txt" };
    return 0;
}
```

Lire

Écrivez dans le fichier `texte.txt` un entier, puis un retour à la ligne, puis un mot.

42

Bonjour

La lecture se fait très simplement. On utilise les chevrons comme pour `std::cin`.

```
#include <fstream>
#include <iostream>
#include <string>

int main()
{
```

```

std::ifstream fichier { "texte.txt" };

int entier { 0 };
fichier >> entier;
std::cout << "Mon entier vaut : " << entier << std::endl;

std::string mot { "" };
fichier >> mot;
std::cout << "Mon mot vaut : " << mot << std::endl;

return 0;
}

```



J'avais dans mon fichier une phrase entière, et seul le premier mot est apparu dans la console. Que s'est-il passé ?

En fait, c'est un comportement tout à fait normal, qu'on retrouve également avec `std::cin`. Par défaut, les espaces et les retours à la ligne sont considérés comme étant des délimiteurs, des séparateurs. Ainsi, là où vous voyez une phrase complète, le programme va voir un ensemble de mots séparés par des espaces.

Les concepteurs de C++ ne nous laissent néanmoins pas sans solution. Il existe une fonction, `std::getline`, qui permet de lire une ligne entière. Elle attend le flux d'entrée à lire ainsi qu'une `std::string` pour écrire le résultat.

```

#include <fstream>
#include <iostream>
#include <string>

int main()
{
    std::ifstream fichier { "texte.txt" };

    int entier { 0 };
    fichier >> entier;
    std::cout << "Mon entier vaut : " << entier << std::endl;

    std::string phrase { "" };
    std::getline(fichier, phrase);
    std::cout << "Ma phrase vaut : " << phrase << std::endl;

    return 0;
}

```



Ça ne marche toujours pas ! Pourquoi je ne vois plus rien maintenant ?

Vicieux comme nous sommes, nous vous avons encore fait tomber dans un piège !

Le problème vient de l'utilisation, juste avant `std::getline`, des chevrons. Ceux-ci lisent la valeur entière, pas de soucis, mais le caractère de retour à la ligne est laissé de côté. Si l'on refait encore cette opération avec les chevrons, alors ce caractère est consommé et ne nous gêne pas. Mais ce n'est pas le cas avec `std::getline`.

Pour régler ce problème, nous avons plusieurs solutions :

- placer un `fichier.ignore(255, '\n')` après l'utilisation des chevrons, comme nous l'avons appris dans [notre exemple de gestion des erreurs d'entrée](#). Ainsi, nous sommes sûrs de supprimer le retour à la ligne qui nous embête ;
- utiliser le *modificateur de flux* (c'est-à-dire qui modifie son comportement) `std::ws` [de l'anglais *white spaces*, espaces blancs]. Il supprime tous les espaces et retours à la ligne qu'il trouve et s'utilise comme suit.

```
// On peut l'utiliser directement
// en même temps que la lecture.
int entier { 0 };
fichier >> entier >> std::ws;
std::cout << "Mon entier vaut : " << entier << std::endl;
```

```
// On peut en faire une instruction à part.
fichier >> std::ws;
```

```
// Ou même directement avec std::getline !
std::getline(fichier >> std::ws, phrase);
```

Maintenant, en complétant le code précédent, nous n'avons plus aucun problème.

```
#include <fstream>
#include <iostream>
#include <string>

int main()
{
    std::ifstream fichier { "texte.txt" };

    int entier { 0 };
    fichier >> entier;
    std::cout << "Mon entier vaut : " << entier << std::endl;

    std::string phrase { "" };
    // On est sûr de ne pas oublier en l'utilisant
    // directement dans std::getline.
    std::getline(fichier >> std::ws, phrase);
    std::cout << "Ma phrase vaut : " << phrase << std::endl;
```

```
    return 0;
}
```

Tout lire

Vous êtes un lecteur vorace et vous voulez lire tout le fichier ? Aucun problème. Tant les chevrons (mot par mot) que `std::getline` (ligne par ligne) peuvent être utilisés dans une boucle, pour lire jusqu'à la fin du fichier.

```
#include <fstream>
#include <iostream>
#include <string>

int main()
{
    std::ifstream fichier { "texte.txt" };
    std::string ligne { "" };

    while (std::getline(fichier, ligne))
    {
        std::cout << "Ligne lue : " << ligne << std::endl;
    }

    return 0;
}
```

Remarquez la syntaxe de la boucle : `while (std::getline(fichier, ligne))`. Cela devrait vous rappeler notre utilisation de `std::cin` dans les boucles (et plus généralement dans les conditions). Ce parallèle est dû au fait que la fonction `std::getline` renvoie un flux ; plus précisément, elle renvoie le flux qu'on lui donne en paramètre (ici, le fichier ouvert), dont l'état a donc été modifié. C'est ce flux qui est alors converti en booléen pour nous dire si tout s'est bien passé. En particulier, lorsqu'on a atteint la fin du fichier, le flux renvoyé par `std::getline` est évalué à `false`, ce qui permet de sortir de la boucle en fin de lecture de fichier.

9.4. Entraînez-vous !

Statistiques sur des fichiers

Quand vous écrivez avec un logiciel de traitement de texte, celui-ci vous donne en direct certaines informations, telles que le nombre de caractères ou de mots. Faisons donc un petit programme qui va ouvrir un fichier et obtenir les informations suivantes :

- le nombre de lignes ;

- le nombre de caractères (sans les espaces) ;
- le nombre de mots.

✧ [Voir la solution](#)

9.5. Encore plus de flux !

Nous savons lire et écrire dans des fichiers, ce qui est déjà bien. Mais allons un peu plus loin, voulez-vous ? En fait, la manipulation de fichiers et celle des entrées/sorties ont beaucoup plus de points communs que ce que vous pourriez penser. D'ailleurs, dans les deux cas, on parle de *flux*.

Un flux c'est quoi ?

Il existe de très nombreux composants électroniques. Que ce soient des processeurs, des cartes-mères, des claviers, des écrans, des téléphones, une multitude s'offre à nous. Et tous viennent avec leur propre façon de communiquer. Écrire un programme qui devrait savoir communiquer avec tous ces périphériques est impossible. C'est pour cela qu'on utilise des abstractions. Par exemple, votre système d'exploitation (Windows, GNU/Linux, Android, etc.) est une abstraction. Notre programme ne communique plus avec un nombre potentiellement très grand de matériels et composants, mais uniquement avec le système d'exploitation, qui se charge lui-même de faire le lien avec le matériel (la mémoire vive, le processeur, le disque dur, etc.). Cela nous simplifie grandement la vie.

La bibliothèque standard est aussi une abstraction. Elle nous fournit un lot unique de fonctionnalités précises, qui marcheront quel que soit le système d'exploitation. Ainsi, en utilisant la bibliothèque standard, notre programme pourra être compilé pour fonctionner tant avec Windows que GNU/Linux, Android, etc.

Les flux sont eux aussi une abstraction. Grâce à eux, on peut envoyer ou recevoir des flux de données (d'où le nom), d'une taille potentiellement infinie, à des composants. Dans la bibliothèque standard, nous avons ainsi de quoi recevoir des informations du clavier (`std::cin`), d'en envoyer à l'écran (`std::cout`) mais aussi avec des fichiers (`std::ifstream` et `std::ofstream`). L'avantage, c'est qu'ils offrent une façon transparente et unifiée d'envoyer et de recevoir des données. Vous l'avez vu vous-même plus haut, il n'y a aucune différence entre écrire sur la sortie standard ou dans un fichier de notre choix. On pourrait même coder nos propres flux de façon à lire et écrire sur le réseau, ils s'utiliseraient de la même façon que les autres !

Un *buffer* dites-vous ?

Les flux sont, selon le terme anglophone largement employé, *bufferisé*. Le problème est qu'accéder à des composants externes, des périphériques, comme la carte graphique, un appareil externe ou autre, peut être long. Ainsi, dans le code suivant, on accéderait potentiellement trois fois à un composant externe.

```
flux << 1 << 'A' << 3.1415;
```

Pour éviter ça, tout ce qu'on envoie ou reçoit d'un flux n'est pas directement traité mais mis en mémoire dans un *tampon* [en anglais *buffer*]. Pour que les données soient concrètement et physiquement envoyées à l'écran, ou écrites sur un fichier ou autre, il faut *vider le tampon* [en anglais *flush*]. C'est notamment ce que fait `std::endl`, en plus d'afficher un retour à la ligne. C'est également ce qui se passe quand le fichier est fermé, quand notre objet arrive à la fin de sa portée et est détruit.

Tenez, faisons un test. Compilez le code suivant et, avant de taper une phrase quelconque en guise d'entrée, ouvrez le fichier concerné et regardez son contenu. Vous le trouverez vide, car les données sont toujours dans le *buffer*.

```
#include <fstream>
#include <iostream>
#include <string>

int main()
{
    std::ofstream fichier { "sortie.txt" };
    // Notez qu'on ne vide pas le tampon.
    fichier << "Hey, salut toi !\n";
    fichier << 42 << " " << 2.718;

    // Une entrée pour vous laisser le temps
    // d'ouvrir le fichier sortie.txt.
    std::string phrase { "" };
    std::cout << "Tape une phrase quelconque : ";
    std::cin >> phrase;

    // On vide explicitement le tampon.
    // Maintenant les données sont écrites.
    fichier << std::flush;
    return 0;
}
```

Note > Pour éviter ce phénomène parfois coûteux, certains codes remplacent `std::endl` par `'\n'`, pour éviter de flusher trop souvent. Dans le cadre de ce livre, ces questions de performance ne nous intéressent pas, donc nous continuerons à utiliser en alternance `std::endl` et `'\n'`.

Les modificateurs de flux

Nous avons déjà vu un modificateur de flux à la [Section 5.1, Les booléens](#) : `std::boolalpha`. Il permet d'afficher `true` ou `false` au lieu de `1` ou `0` quand on manipule des booléens. Il est tout à fait possible de l'utiliser avec d'autres types de flux. L'exemple suivant vous le démontre.

```
#include <fstream>

int main()
{
    std::ofstream fichier { "sortie.txt" };
    // On va écrire true.
    fichier << std::boolalpha << true << std::endl;
    // On revient comme avant et on va écrire 0.
    fichier << std::noboolalpha << false << std::endl;

    return 0;
}
```

Note > Le modificateur `std::noboolalpha` fait l'opération inverse de `std::boolalpha`. La plupart des modificateurs sont ainsi disponibles par paire, l'un activant le comportement désiré, l'autre le désactivant.

Voici ce que vous allez obtenir dans le fichier de sortie.

```
true
0
```

Examinons un autre exemple. Vous savez que, si vous afficher dans la console un nombre positif, vous le verrez tel quel. Mais on peut aussi demander à afficher le signe `+`, tout comme le signe `-` est affiché pour les nombres négatifs.

```
#include <iostream>

int main()
{
    std::cout << std::showpos << 42
              << " " << 89 << " "
              << -48 << std::endl;
    return 0;
}
```

```
+42 +89 -48
```


Même les chaînes y passent !

Maintenant, admettons que vous vouliez avoir une chaîne de caractères représentant un nombre réel. Mais, contraintes supplémentaires, le signe doit toujours être affiché et le nombre écrit en [notation scientifique](#). Comment faire ? Il y a bien `std::scientific` et `std::showpos`, mais ils n'existent que pour les flux. Justement, la solution va être de traiter les chaînes de caractères comme des flux. Cela se fait en utilisant `std::ostream` comme flux de sortie et `std::istream` comme flux d'entrée. Les deux sont disponibles dans le fichier `<sstream>`.

```
#include <iostream>
#include <sstream>
#include <string>

int main()
{
    double const reel { 10245.789 };

    std::ostream flux_chaine;
    // Notez comment l'utilisation est identique
    // à ce que vous connaissez.
    flux_chaine << std::scientific
        << std::showpos << reel << std::endl;
    // On récupère une chaîne de caractères en appelant str().
    std::string resultat { flux_chaine.str() };

    std::cout << "Affichage par défaut : " << reel << std::endl;
    std::cout << "Affichage modifié : " << resultat << std::endl;

    return 0;
}
```

Affichage par défaut : 10245.8

Affichage modifié : +1.024579e+04

Maintenant, abordons `std::istream`, qui permet de traiter une chaîne de caractères comme un flux, avec un exemple concret. En informatique, beaucoup de valeurs sont écrites suivant le système hexadécimal. Que ce soit l'identifiant d'une carte réseau ou le codage des couleurs sur un site web, il est courant de l'utiliser.

***Note** > Le système hexadécimal est un système de numération qui utilise 16 chiffres au lieu des 10 que nous connaissons dans le système décimal. Les chiffres vont de 0 à 9, comme pour le décimal, puis on utilise les lettres A, B, C, D, E et F. Ainsi, des nombres comme 2A et FD sont des nombres écrits en hexadécimal, qui valent respectivement 42 et 253 dans notre système décimal. Si vous souhaitez en savoir plus, lisez l'article [Wikipédia](#), ainsi que d'autres ressources.*

Par exemple, l'orange agrume utilisé sur Zeste de Savoir est écrit `f8ad32` dans son code source. Imaginons donc que nous récupérions cette valeur. C'est une chaîne de caractères.

tères et nous voulons en déterminer le niveau de rouge, de vert et de bleu. Il existe un modificateur de flux appelé `std::hex` qui permet de dire au flux concerné que les nombres qu'il va extraire utilisent le système hexadécimal et non décimal. En traitant une chaîne de caractères comme un flux, on résout alors notre problème.

```
#include <iostream>
#include <sstream>

int main()
{
    std::istringstream flux_entree { "f8 ad 32" };
    int rouge { 0 };
    int vert { 0 };
    int bleu { 0 };

    flux_entree >> std::hex
        >> rouge >> vert >> bleu;
    std::cout << "Niveau de rouge : " << rouge << std::endl;
    std::cout << "Niveau de vert : " << vert << std::endl;
    std::cout << "Niveau de bleu : " << bleu << std::endl;

    return 0;
}
```

Niveau de rouge : 248

Niveau de vert : 173

Niveau de bleu : 50

***Note** > On utilisait beaucoup `std::ostringstream` et `std::istringstream` avant C++11 pour convertir des nombres en chaînes de caractères et vice-versa. D'autres méthodes existent maintenant, mais nous en parlerons [plus tard](#).*

9.6. Récapitulons !

- Pour ouvrir un fichier, il faut préciser son chemin, qui peut être relatif ou absolu.
- On écrit dans un fichier avec `std::ofstream` et on le lit avec `std::ifstream`.
- Les flux sont une abstraction permettant d'envoyer ou recevoir un flux de données à des composants quelconques.
- Les flux sont bufferisés.
- Il existe des flux pour communiquer avec des fichiers (`std::ifstream` et `std::ofstream`), avec des chaînes de caractères (`std::istringstream` et `std::ostringstream`), avec l'entrée et la sortie standards (`std::cin` et `std::cout`).
- On utilise des modificateurs de flux pour en modifier le comportement.

SOLUTION

section Statistiques sur des fichiers

```
#include <fstream>
#include <iostream>
#include <string>

int main()
{
    std::ifstream fichier { "fichier_1.txt" };
    std::string ligne { "" };

    int total_lignes { 0 };
    int total_caracteres { 0 };
    int total_mots { 0 };

    bool espace_caractere_precedent { false };
    while (std::getline(fichier, ligne))
    {
        ++total_lignes;

        for (char c : ligne)
        {
            if (isspace(c))
            {
                // Si le précédent n'était pas un espace, alors c'était une
                partie de mot.
                if (!espace_caractere_precedent)
                {
                    ++total_mots;
                }

                espace_caractere_precedent = true;
            }
            else
            {
                ++total_caracteres;
                espace_caractere_precedent = false;
            }
        }

        // Ne pas oublier de regarder pour l'éventuel dernier mot.
        if (!espace_caractere_precedent)
        {
            ++total_mots;
            espace_caractere_precedent = true;
        }
    }
}
```

SOLUTION

```
std::cout << "Total de caractères : " << total_caracteres <<
std::endl;
std::cout << "Total de lignes : " << total_lignes << std::endl;
std::cout << "Total de mots : " << total_mots << std::endl;

return 0;
}
```

Félicitations, vous avez déjà codé une ébauche de [wc](#), un programme UNIX existant.

On passe la deuxième !

Depuis les débuts de l'informatique et de la programmation, découper son code en sous-unités est un principe de base. Celui-ci permet de mieux diviser le travail, résoudre les problèmes plus facilement, et aide les développeurs à mieux s'y retrouver. En bref, plein d'avantages. Cette partie aborde toutes les notions de découpage et de regroupement en C++, ainsi que différentes techniques qui vont grandement élargir notre champ de possibilités.

10

Découpons le code — les fonctions

Nos programmes petit à petit se complexifient, nous sommes en mesure de faire de plus en plus de choses. Mais, chaque fois que nous avons besoin de réutiliser un morceau de code, comme dans le cadre de la protection des entrées, nous sommes amenés à le dupliquer. Autant dire que c'est contraignant, ça alourdit le programme et c'est une source d'erreurs potentielles.

Le moment est donc venu pour vous de découvrir les fonctions, qui sont à la base de nombreux codes et la solution à notre problème.

10.1. Les éléments de base d'une fonction

Une fonction, c'est quoi ?

Une fonction est un *ensemble d'instructions* pour réaliser une tâche précise. Cet ensemble d'instructions est isolé dans une partie spécifique nommé *identificateur*. Une fonction peut ou non prendre des informations en entrée, comme elle peut ou non en fournir en sortie.

Depuis le début de ce livre, nous manipulons des fonctions incluses dans la bibliothèque standard. Revenons un peu sur différents exemples qui vont illustrer la définition théorique vue ci-dessus.

Cas le plus simple

Nous avons vu des fonctions simples qui ne renvoyaient ni ne prenaient de valeur. Ce genre de fonctions n'a pas besoin qu'on lui fournisse des données en entrée, et elle n'en fournit pas en retour.

```
std::string chaine { "Salut toi" };  
// On supprime le dernier caractère.  
chaine.pop_back();  
// On supprime la chaîne.
```

```
chaine.clear();

std::list<int> liste { 4, -9, 45, 3 };
// On trie la liste.
liste.sort();
```

Fonction renvoyant une valeur

Certaines fonctions nous fournissent des informations que nous pouvons, au choix, récupérer ou ignorer. Le [chapitre sur les tableaux](#) nous en fournit plusieurs exemples.

```
std::vector<int> const tableau_de_int
{
    1, 2, 3, 4, 5, 6, 7, 8, 9
};
// Récupérer le premier élément.
int premier { tableau_de_int.front() };
// Récupérer le dernier élément.
int dernier { tableau_de_int.back() };
```

Nous ne sommes pas obligés de stocker la valeur qu’une fonction renvoie. Par exemple, `std::getline` peut s’utiliser tel quel, sans se soucier de ce que la fonction a renvoyé.

Fonction attendant une valeur

D’autres fois, certaines fonctions ont besoin d’informations extérieures pour travailler. C’est le cas notamment des algorithmes que nous avons vus, mais aussi d’autres fonctions.

```
std::string texte { "Du texte." };
// Ajouter une lettre à la fin.
texte.push_back('!');

std::vector<char> tableau {};
// On remplit le tableau avec cinq fois la valeur 'C'.
tableau.assign(5, 'C');
```

POINT DE VOCABULAIRE

Dans le cas où une fonction ne retourne aucune valeur, on parle dans la littérature informatique de *procédure*. Cette appellation est valable que la fonction attende ou non des données.

Fusion !

Enfin, dernière possibilité, il y a les fonctions qui attendent des informations en entrée et qui en fournissent en sortie. Là encore, nous en avons déjà vu dans ce livre.

```
std::vector<int> tableau { 1, 2, 3, 4 };
// Renvoie un itérateur sur le premier élément.
auto it = std::begin(tableau);
// On compte le nombre d'éléments valant deux.
std::count(std::begin(tableau), std::end(tableau), 2);

std::string texte { "" };
// std::getline attend qu'on lui donne le flux à lire
// et la chaîne qui servira à stocker le résultat.
// Elle renvoie ce flux en guise de retour.
std::getline(std::cin, texte);
// Attend un conteneur en entrée et renvoie sa taille en sortie.
auto taille { std::size(texte) };
```

Une fonction incontournable

Nous avons un exemple de fonction sous nos yeux depuis le premier code que vous avez écrit : la fonction `main`. Celle-ci est le point d'entrée de tous les programmes que nous codons. La voici.

```
int main()
{
    // Instructions diverses.
    return 0;
}
```

Vous noterez que :

- `main` est le nom de la fonction, son *identificateur* ;
- le `int` situé juste avant définit le type de *retour*. Dans notre cas, la fonction renvoie une valeur entière, qui est le `0` que nous voyons dans `return 0` ; et qui indique au système d'exploitation que tout s'est bien passé ;
- nous voyons une paire de parenthèses vide, ce qui signifie que la fonction n'attend aucune information en entrée ;
- entre les accolades `{}`, nous avons le *corps de la fonction*, les instructions qui la composent.

Les composants d'une fonction

Schéma d'une fonction

En écrivant en pseudo-code, voici à quoi ressemble une fonction en C++.

```
type_de_retour identificateur(paramètres)
{
    instructions
}
```

Votre identité, s'il vous plaît

Le point le plus important est de donner un nom à notre fonction. Les règles sont les mêmes que pour nommer nos variables. Nous vous les réécrivons ci-dessous.

- Il doit commencer par une lettre. Il ne peut pas commencer par un chiffre, c'est interdit. Il ne doit pas commencer non plus par *underscore* `_`, car les fonctions commençant par ce caractère obéissent à des règles spéciales et sont souvent réservées à l'usage du compilateur.
- Les espaces et les signes de ponctuation sont interdits (`'` , `?` , etc.).
- On ne peut pas utiliser un mot clé du langage comme identificateur. Ainsi, il est interdit de déclarer une fonction s'appelant `int` ou `return`, par exemple.

Nous vous rappelons également l'importance de donner un nom clair à vos fonctions, qui définit clairement ce qu'elles font. Évitez donc les noms trop vagues, les abréviations obscures, etc.

Mais de quel type es-tu ?

Comme nous l'avons vu à l'instant, une fonction peut ou non renvoyer une valeur de retour. Si elle doit renvoyer une valeur quelconque, alors il suffit d'écrire le type de cette valeur. À l'inverse, si votre fonction ne doit rien renvoyer, alors on utilise le mot clé `void`, qui signifie *vide* et qui, dans ce contexte, indique que la fonction renvoie du vide, c'est-à-dire rien.

Pour renvoyer une valeur, on utilise le mot clé `return`, exactement comme dans la fonction `main`. Il peut y en avoir plusieurs dans une fonction, par exemple deux `return`, un si un `if` est vrai, l'autre s'il est faux. Par contre, lorsqu'une instruction `return` est exécutée, on sort de la fonction en cours et tout le code restant n'est pas exécuté. On dit qu'un

retour de fonction coupe le flot d'exécution de la fonction. Oui, comme `break` dans une boucle. Vous suivez, c'est très bien.

Paramétrage en cours...

Maintenant, parlons des paramètres. C'est par leur intermédiaire qu'une fonction indique au monde extérieur ce qu'elle attend pour travailler. Par exemple, `std::sort` attend qu'on lui donne un itérateur de début et un itérateur de fin. Quand nous ajoutons un élément à un tableau à l'aide de `push_back`, nous devons lui fournir la valeur à ajouter.

Comment les déclarer ? Tout simplement avec un type et un identificateur. S'il y en a plusieurs, ils seront séparés par des virgules.

Exemples

Voici en exemple quelques fonctions que nous avons créées en reprenant les codes écrits au cours des chapitres précédents.

```
int pgcd(int a, int b)
{
    int r { a % b };
    while (r != 0)
    {
        a = b;
        b = r;
        r = a % b;
    }

    // On peut tout à fait renvoyer la valeur d'un paramètre.
    return b;
}

int somme(int n)
{
    // On peut renvoyer le résultat d'un calcul
    // ou d'une expression directement.
    return (n * (n + 1)) / 2;
}

bool ou_exclusif(bool a, bool b)
{
    return (a && !b) || (b && !a);
}

int main()
{
    int const a { 845 };
    int const b { 314 };
}
```

```

std::cout << "Le PGCD de " << a << " et "
    << b << " vaut "
    << pgcd(a, b) << "." << std::endl;

std::cout << "La somme de tous les entiers de 1 à 25 vaut "
    << somme(25) << "." << std::endl;

std::cout << std::boolalpha;
std::cout << "XOR(true, false) vaut "
    << ou_exclusif(true, false)
    << "." << std::endl;

return 0;
}

```

Notez qu'on peut très bien donner en argument à une fonction une variable qui porte le même identificateur qu'un des paramètres. Il n'y a aucun problème à ça, car nous sommes dans deux portées différentes : `main` et la fonction en question.

Conséquence du point précédent, il n'est pas possible d'utiliser un paramètre de fonction en dehors de celle-ci. Le code suivant produit donc une erreur.

```

void fonction(int parametre)
{
    // Aucun problème.
    parametre = 5;
}

int main()
{
    // La variable parametre n'existe pas !
    parametre = 410;
    return 0;
}

```

ARGUMENT OU PARAMÈTRE ?

Vous êtes nombreux à confondre paramètres et arguments :

- Un paramètre (aussi appelé paramètre formel), c'est ce qu'attend une fonction pour travailler ; il est inscrit dans sa déclaration.
- Un argument (aussi appelé paramètre réel), c'est la valeur transmise à la fonction quand on l'utilise.

```
void fonction(int entier)
```

```

{
    // entier est le paramètre de la fonction.
}

int main()
{
    // 4 est l'argument de la fonction.
    fonction(4);

    int const valeur { 5 };
    // valeur est l'argument de la fonction.
    fonction(valeur);

    return 0;
}

```

Dans la pratique, beaucoup utilisent l'un ou l'autre de manière interchangeable et vous serez parfaitement compris si vous faites de même. Nous tenions néanmoins à vous expliquer cela rigoureusement.

10.2. Entraînez-vous !

Il est temps de pratiquer toutes ces nouvelles notions que vous avez acquises, afin de mieux les comprendre et bien les retenir.

Afficher un rectangle

Commençons par une fonction simple qui affichera dans la console un rectangle composé d'étoiles *. Voici un exemple en lui donnant une longueur de 4 et une largeur de 3.

```

***
***
***
***

```

 [Voir la solution](#)

Distributeur d'argent

Le but ici est de donner une quantité de coupures correspondant à une somme donnée, en utilisant les plus grosses en premier. Par exemple, 285 € avec des coupures de 500 €, 200 €, 100 €, 50 €, 20 €, 10 €, 5 €, 2 € et 1 € donne un total d'un billet de 200 €, un billet de 50 €, un billet de 20 €, un billet de 10 € et enfin un billet de 5 €. Par contre, 346 € avec uniquement 20 €, 10 €, 5 €, 2 € et 1 € donne dix-sept billets de 20 €, un

billet de 5 € et une pièce de 1 €. Il faut donc prendre en compte les coupures disponibles dans le résultat.

✧ [Voir la solution](#)

Bonus : Faire une fonction qui affiche de façon lisible le total. Par exemple, pour 285 €, on peut imaginer la sortie suivante.

```
1 x 200 euros
1 x 50 euros
1 x 20 euros
1 x 10 euros
1 x 5 euros
```

✧ [Voir la solution](#)

Parenthésage

Imaginons une expression de plusieurs dizaines de caractères, avec des parenthèses ouvrantes (et fermantes). Il serait fastidieux de vérifier manuellement qu'elle est correctement parenthésée, c'est-à-dire qu'elle a le même nombre de symboles ouvrants et fermants et qu'ils sont bien dans l'ordre.

Ainsi, l'expression (((()))) est correcte, alors que (((())) est incorrecte parce qu'il manque une parenthèse fermante. Quant à) (, elle est également incorrecte parce que le symbole fermant est avant le symbole ouvrant.

✧ [Voir la solution](#)

10.3. Quelles sont vos références ?

La problématique

Nous avons vu [précédemment](#) que les paramètres d'une fonction étaient dans une portée différente, ce qui autorise à faire ceci.

```
void fonction(int a)
{
    a = 5;
}

int main()
{
```

```

int a { 8 };
// Aucun problème.
fonction(a);

return 0;
}

```

Il faut cependant savoir que ceci implique une copie. Pour chaque paramètre, le compilateur va réserver un espace en mémoire de la taille nécessaire et copier la variable qu'on lui passe. Nous allons donc manier des variables totalement différentes. C'est ce qui s'appelle le *passage par valeur*.

```

#include <iostream>
#include <vector>

void fonction(int a, double b, std::vector<int> c)
{
    a = 5;
    b = 2.18781;
    c.assign(7, 0);

    // Ici, nous manipulons donc des copies.
    std::cout << a << std::endl;
    std::cout << b << std::endl;
    for (auto element : c)
    {
        std::cout << element << " ";
    }
    std::cout << std::endl << std::endl;
}

int main()
{
    int const entier { 42 };
    double const reel { 3.14159 };
    std::vector<int> const tableau { 1, 782, 4, 5, 71 };

    // Chaque variable va être copiée.
    fonction(entier, reel, tableau);

    // Affichons pour prouver que les variables originales
    // n'ont pas changé.
    std::cout << entier << std::endl;
    std::cout << reel << std::endl;
    for (auto element : tableau)
    {
        std::cout << element << " ";
    }

    std::cout << std::endl;
    return 0;
}

```

Le côté négatif, c'est qu'on doit forcément copier l'argument, même si on ne le modifie pas. Dans le cas d'un entier ou d'un caractère, c'est idéal, mais dans le cas d'un tableau de plusieurs milliers d'éléments, on perd du temps inutilement. C'est comme si on demandait à consulter un certain original dans des archives et qu'on nous faisait des photocopies à la place. S'il s'agit d'un post-it, pas grave, mais s'il s'agit d'un livre de 400 pages, bonjour le gâchis !

Les références

La solution consiste à ne pas passer une copie de la variable originale mais un alias, une *référence* vers celle-ci, qui soit manipulable comme si nous avions l'original entre les mains. Cela revient à donner le document original et éventuellement, selon le contexte, le droit de le modifier.

Une référence en C++ est simple à déclarer mais doit réunir plusieurs conditions :

- être précédée par une esperluette `&` ;
- être de même type que la variable cible (une référence sur un `double` ne peut pas se voir associée à une variable de type `int`) ;
- cibler une seule et unique variable (on ne peut pas créer de référence qui ne cible rien, ni changer la cible d'une référence une fois qu'on l'a créée) ;

```
#include <iostream>

int main()
{
    int entier { 40 };
    // On initialise notre référence pour qu'elle référence entier.
    int & reference_entier { entier };

    std::cout << "Entier vaut : "
                << entier << std::endl;
    std::cout << "Référence vaut : "
                << reference_entier << std::endl;

    reference_entier += 2;
    std::cout << "Entier vaut : "
                << entier << std::endl;
    std::cout << "Référence vaut : "
                << reference_entier << std::endl;

    int const entier_constant { 0 };
    // On peut également déclarer des références constantes
    // sur des entiers constants.
    int const & reference_const_entier_constant { entier_constant };
    // Ce qui interdit ce qui suit,
```

```

// car le type référencé est un int const.
// reference_const_entier_constant = 1;

// On ne peut pas déclarer de référence non constante
// sur un entier constant.
// int & reference_entier_constant { entier_constant };

// On peut par contre tout à fait déclarer une
// référence constante sur un objet non constant.
// On ne peut donc pas modifier la valeur de l'objet en utilisant
// la référence, mais l'entier original reste tout à fait modifiable.
int const & reference_const_entier { entier };
std::cout << "Entier vaut : "
    << entier << std::endl;
std::cout << "Référence constante vaut : "
    << reference_const_entier << std::endl;

entier = 8;
std::cout << "Entier vaut : "
    << entier << std::endl;
std::cout << "Référence constante vaut : "
    << reference_const_entier << std::endl;

return 0;
}

```

TABLEAUX ET RÉFÉRENCES

Vous avez déjà utilisé des références sans le savoir. En effet, quand nous [accédons à un élément d'un tableau](#), c'est en fait une référence qui est renvoyée, ce qui autorise à modifier un tableau comme nous l'avions vu.

```

// On change la valeur du premier élément.
tableau[0] = 5;

```

C++ est un langage qui cache plein de concepts sous des airs innocents.

Paramètres de fonctions

On peut tout à fait déclarer les paramètres d'une fonction comme étant des références. Ainsi, celles-ci ne recevront plus une copie mais bel et bien des références sur les objets. On parle donc de *passage par référence*. On peut, bien entendu, préciser si la référence est constante ou non. Dans le premier cas, toute modification du paramètre sera interdite, alors que le deuxième cas autorise à le modifier et conserver ces changements.


```

#include <iostream>
#include <string>

void fonction_reference_const(std::string const & texte)
{
    std::cout << "J'ai reçu une référence comme paramètre. "
               << "Pas de copie, youpi !" << std::endl;
    // Interdit.
    // texte = "Changement";
}

void fonction_reference(std::string & modifiable)
{
    std::cout << "Je peux modifier la chaîne de caractères. "
               << "Les changements seront conservés." << std::endl;
    modifiable = "Exemple";
}

int main()
{
    std::string modifiable { "Du texte modifiable." };
    std::string const non_modifiable { "Du texte non modifiable." };

    std::cout << "Avant : "
               << modifiable << std::endl;
    fonction_reference(modifiable);
    std::cout << "Après : "
               << modifiable << std::endl;

    fonction_reference_const(non_modifiable);
    // Possible.
    fonction_reference_const(modifiable);
    // Impossible, parce que la fonction attend une référence
    // sur un std::string et nous lui donnons une référence
    // sur un std::string const.
    //fonction_reference(non_modifiable);

    return 0;
}

```



Quand dois-je utiliser une référence constante ou une référence simple ?

Il suffit de se poser la question : *Est-ce que ce paramètre est destiné à être modifié ou non ?* Si c'est le cas, alors on utilisera une référence simple. Dans le cas contraire, il faut utiliser une référence constante. Ainsi, l'utilisateur de la fonction aura des garanties que son objet ne sera pas modifié dans son dos.

Le cas des littéraux

Analysez le code suivant, sans le lancer. Que pensez-vous qu'il va faire ? Si vous répondez qu'il va simplement afficher deux messages... dommage, vous avez perdu.

```
#include <iostream>

void test(int & a)
{
    std::cout << "Je suis dans la fonction test.\n";
}

void test_const(int const & a)
{
    std::cout << "Je suis dans la fonction test_const.\n";
}

int main()
{
    test(42); ❶
    test_const(42);
    return 0;
}
```

En fait, ce code ne va même pas compiler, à cause de la ligne ❶. Il y a une très bonne raison à ça : nous passons un littéral, qui est invariable et n'existe nulle part en mémoire, or nous tentons de le donner en argument à une fonction qui attend une référence, un alias, sur une variable modifiable. Mais un littéral n'est pas modifiable et tenter de le faire n'a aucun sens. Le compilateur refuse donc et ce code est invalide.

Par contre, supprimez-la et le code compilera. Pourtant, la seule différence entre les deux fonctions, c'est la présence d'un `const`. Eh bien, c'est lui qui rend le code valide. En effet, le compilateur va créer un objet temporaire anonyme, qui n'existera que le temps que la fonction prendra pour travailler, et va lui assigner le littéral comme valeur. On peut réécrire le code précédent comme ceci pour mieux comprendre.

```
#include <iostream>
#include <string>

void test_const(int const & a)
{
    std::cout << "Je suis dans la fonction test_const.\n";
}

void test_string_const(std::string const & texte)
{
    std::cout << "Je suis dans la fonction test_string_const.\n";
}
```

```

int main()
{
    // test_const(42);
    {
        // Nouvelle portée.
        int const variable_temporaire { 42 };
        int const & reference_temporaire { variable_temporaire };
        test_const(reference_temporaire);
        // Nos variables disparaissent.
    }

    // Exemple avec une chaîne de caractères.
    // test_string_const("Salut toi !");
    {
        std::string const texte_temporaire { "Salut toi !" };
        std::string const & reference_temporaire { texte_temporaire };
        test_string_const(reference_temporaire);
    }

    return 0;
}

```

Le cas des types standards

Utiliser des références pour `std::vector` ou `std::string` d'accord, mais qu'en est-il des types simples comme `int`, `char` et `double` ? Y a-t-il un intérêt à utiliser des références constantes ? Doit-on bannir toutes les copies de nos codes sous prétexte d'efficacité ? Non. L'usage en C++, c'est que les types natifs sont toujours passés par copie. Pourquoi cela ? Parce qu'ils sont très petits et le coût de la création d'une référence sur des types aussi élémentaires est souvent plus élevé que celui d'une simple copie. En effet, le compilateur peut optimiser la copie et la rendre extrêmement rapide, bien plus qu'avec les références.

Donc ne tombez pas dans le piège de l'optimisation prématurée, qui consiste à vouloir améliorer les performances de son programme en faisant des changements qui n'apporteront aucune amélioration. Laissez plutôt ce travail au compilateur.

Valeur de retour

Les références ont une contrainte particulière : elles doivent toujours être valides. Dans le cas d'un paramètre de fonction, il n'y a aucun danger. En effet, l'argument transmis à la fonction existera toujours quand celle-ci se terminera, comme le montre le code suivant.

```

void fonction(int & reference)
{
    // On modifie une référence sur une variable qui existe, c'est OK.
}

```

```

    reference = 42;
}

int main()
{
    int variable { 24 };
    // On transmet variable...
    fonction(variable);
    // ... qui existe toujours après la fin de la fonction.

    return 0;
}

```

Cependant, dans le cas où l'on souhaite qu'une fonction renvoie une référence, il faut faire très attention parce qu'on peut en renvoyer une sur un objet qui n'existe plus ! Ceci va donc entraîner un comportement indéterminé.

```

int& fonction()
{
    int variable { 42 };
    // Ohoh, variable sera détruite quand la fonction sera terminée !
    return variable;
}

int main()
{
    // Grosse erreur, notre référence est invalide !
    int& reference { fonction() };

    return 0;
}

```

Attention > Ne renvoyez JAMAIS de référence sur une variable locale.

Un mot sur la déduction de type

Certains lui reprochent son côté rigide. En effet, `auto` seul dégage la référence. À l'inverse, `auto &` sera toujours une référence. Et il en est de même pour `const` : `auto const` sera toujours constant et `auto const &` sera toujours une référence sur un objet constant.

```

int main()
{
    int variable { 2 };
    int & reference { variable };

    // Sera toujours une variable et jamais une référence.
    auto toujours_variable_1 { variable };
    auto toujours_variable_2 { reference };
}

```

```

// Sera toujours une référence et jamais une variable.
auto & toujours_reference_1 { variable };
auto & toujours_reference_2 { reference };

// On pourrait rajouter des exemples avec const,
// mais vous comprenez le principe.
return 0;
}

```

Il existe cependant un mot clé qui s'adapte toujours au type en paramètre parce qu'il conserve la présence (ou non) de `const` et de références. Il s'appelle `decltype` et s'utilise de deux façons. La première, c'est avec une expression explicite, comme un calcul, une autre variable ou le retour d'une fonction, ce qui donne `decltype(expression) identifiant { valeur };` et est illustré ci-dessous.

```

#include <iostream>
#include <vector>

int main()
{
    std::vector<int> tableau { -8, 45, 35, 9 };

    // Sera toujours une variable.
    auto premier_element_variable { tableau[0] }; ❶
    // Est en fait une référence sur cet élément,
    // ce qui autorise à le modifier.
    decltype(tableau[1]) deuxieme_element_reference { tableau[1] }; ❷

    // Modifions et regardons.
    premier_element_variable = 89;
    deuxieme_element_reference = 0;

    for (auto const & element : tableau)
    {
        std::cout << element << std::endl;
    }

    return 0;
}

```

La deuxième façon de l'utiliser, c'est de combiner `decltype` et `auto`, donnant `decltype(auto) identifiant { valeur };` afin d'avoir le type exact de `valeur` et de ne pas se répéter. Les lignes ❶ et ❷ peuvent ainsi être fusionnées :

```

std::vector<int> tableau { -8, 45, 35, 9 };
// Sera de type int& si tableau est modifiable, de type int const &
// si tableau est déclaré comme const.
decltype(auto) deuxieme_element_reference { tableau[1] };

```

Ces deux mots clés ne sont pas deux concurrents s'opposant pour faire le même travail, bien au contraire. Ils sont une preuve de la liberté que C++ laisse au développeur. Vous verrez en continuant votre apprentissage d'autres usages qui rendront ces mots clés encore plus concrets pour vous. En attendant, reprenez ceci pour vous aider à choisir.

- `auto` : lorsque vous souhaitez explicitement une valeur (par copie) ;
- `auto &` (`const` ou non) : lorsque vous souhaitez explicitement une référence ;
- `decltype` : lorsque vous souhaitez le type exact.

10.4. Nos fonctions sont surchargées !

Dans le [chapitre sur les flux](#), nous avons vu qu'on pouvait convertir un nombre en chaîne de caractères à l'aide de `std::ostream` et `std::istream`. Il existe d'autres méthodes pour le faire. Voici justement comment l'une d'elles est déclarée dans la bibliothèque standard :

```
std::string to_string(int value);
std::string to_string(double value);
```



Attends une seconde ! Comment c'est possible qu'on ait deux fonctions avec le même nom ?

La réponse est simple : ce qui différencie deux fonctions, ça n'est pas seulement leur identificateur mais également leurs paramètres. C'est ce qu'on appelle la *signature d'une fonction*. Dans notre cas, le compilateur voit deux signatures différentes : `to_string(int)` et `to_string(double)`, sans que ça ne pose le moindre problème.

Cette possibilité s'appelle *surcharge*, de l'anglais *overloading*. Elle rend le code plus clair car on laisse au compilateur le soin d'appeler la bonne surcharge.

```
#include <iostream>
#include <string>

int main()
{
    int const entier { -5 };
    double const flottant { 3.14156 };

    std::string affichage { "Voici des nombres : " };
    affichage += std::to_string(entier) + ", ";
    affichage += std::to_string(flottant);
}
```

```
std::cout << affichage << std::endl;

return 0;
}
```

Par contre, le type de retour n'entre pas en compte dans la signature d'une fonction. Ainsi, deux fonctions avec le même identifiant, les mêmes paramètres et des types de retour différents seront rejetées par le compilateur car il ne saura pas laquelle utiliser. Et c'est logique, car comment, en lisant le code, sauriez-vous ce que la fonction va vous renvoyer ? C'est pour ça que les fonctions convertissant des chaînes de caractères en nombres ont des noms bien distincts : `std::stoi` pour convertir une chaîne en `int` et `std::stod` pour `double`.

```
#include <iostream>
#include <string>

int main()
{
    int const entier { std::stoi("-8") };
    double const flottant { std::stod("2.71878") };

    std::cout << entier + flottant << std::endl;
    return 0;
}
```

10.5. [Pratique] Gérer les erreurs d'entrée • Partie III

Avec ce que nous avons vu, nous sommes maintenant capables d'écrire des fonctions qui nous épargnent des duplications de code chaque fois que nous souhaitons sécuriser nos entrées. Nous pourrions ainsi sécuriser le code suivant sans duplication.

```
#include <iostream>

int main()
{
    int jour { 0 };
    std::cout << "Quel jour es-tu né ? ";
    // Entrée.

    int mois { 0 };
    std::cout << "Quel mois ? ";
    // Entrée.

    int annee { 0 };
    std::cout << "Quelle année ? ";
    // Entrée.
}
```

```
double taille { 0.0 };
std::cout << "Quelle taille ? ";
// Entree.

std::cout << "Tu es né le "
  << jour << "/"
  << mois << "/"
  << annee << " et tu mesures "
  << taille << "m." << std::endl;
return 0;
}
```

✧ [Voir la solution](#)

10.6. Dessine-moi une fonction

Nous allons nous attarder sur un petit problème qui peut apparaître lorsque l'on crée des fonctions : les appels croisés.

Le problème

Introduisons ce problème par un exemple : essayons d'implémenter deux fonctions `est_pair` et `est_impair`, dont l'objectif est, comme leur nom l'indique, de dire si un entier est pair ou impair. Une manière élégante de procéder est de faire en sorte que chacune appelle l'autre.

```
bool est_pair(int i)
{
    if (i == 0)
    {
        return true;
    }
    else if (i < 0)
    {
        return est_pair(-i);
    }
    else
    {
        return est_impair(i - 1);
    }
}

bool est_impair(int i)
{
    if (i == 0)
    {
        return false;
    }
}
```



```

if (i < 0)
{
    return est_impair(-i);
}
else
{
    return est_pair(i - 1);
}
}

```

L'idée est de s'approcher pas à pas du cas 0 (dont on connaît la parité) en appelant la fonction avec l'entier `i-1`. C'est ce qu'on appelle de la programmation récursive, rappelant l'idée d'une récurrence en mathématiques.

Bref, ce qui est important ici, c'est que ce code ne compile pas. En effet, comme le compilateur lit le code de haut en bas, il ne connaît pas la fonction `est_impair` lorsqu'on l'utilise dans `est_pair`. Et ça ne marche pas non plus en échangeant les deux fonctions, puisque `est_impair` appelle aussi `est_pair`.

Heureusement, nos fonctions ne sont pas à jeter puisque C++ est bien fait, et qu'il a donc une solution à ce problème : les prototypes.

La solution

Un *prototype* est une expression qui décrit toutes les informations qui définissent une fonction, en dehors de son implémentation.

- son identificateur ;
- son type de retour ;
- ses paramètres.

On peut résumer en disant que c'est tout simplement la première ligne de la définition d'une fonction, à laquelle on ajoute un point-virgule.

```

type_de_retour identifiant(paramètres);

```

Ces informations sont suffisantes pour que le compilateur sache que la fonction existe et qu'il sache comment elle s'utilise. Cela nous permet alors d'écrire, pour notre exemple précédent :

```

bool est_impair(int i);

bool est_pair(int i)

```

```

{
    if (i == 0)
    {
        return true;
    }
    else if (i < 0)
    {
        return est_pair(-i);
    }
    else
    {
        return est_impair(i - 1);
    }
}

bool est_impair(int i)
{
    if (i < 0)
    {
        return est_impair(-i);
    }
    else
    {
        return est_pair(i - 1);
    }
}

```

Et maintenant, ça compile !

Utilisons les prototypes

Les prototypes ne servent pas qu'à résoudre les problèmes d'appels croisés. En effet, ils peuvent être utilisés pour améliorer la lisibilité du code. Tout comme le compilateur, nous n'avons pas besoin de l'implémentation pour savoir comment s'utilise une fonction. En plus, le nom nous suffit souvent, lorsqu'il est bien choisi, pour savoir à quoi sert la fonction.

Constat : nous ne voulons pas voir l'implémentation. Derrière cette phrase un tantinet abusive, l'idée est de réunir l'essentiel de l'information dans le prototype.

On peut donc tirer parti de cela pour rendre nos programmes plus lisibles. Par exemple, on peut mettre tous les prototypes avant la fonction `main` et reléguer les implémentations à la fin du fichier, juste après cette dernière. Ainsi, lorsque l'on veut utiliser une fonction, le prototype n'est plus noyé au milieu des implémentations et on peut en un coup d'œil récupérer toutes les informations qui nous intéressent.

Cette idée de faire valoir les interfaces plutôt que les implémentations est très importante en programmation et l'on aura l'occasion de la recroiser plusieurs fois.

10.7. Récapitulons !

- Une fonction permet de factoriser un morceau de code utilisé régulièrement. Nous en utilisons beaucoup depuis le début de ce livre.
- Une fonction a un identifiant et peut prendre ou non plusieurs paramètres, ainsi que renvoyer ou non une valeur. Dans le cas où elle ne renvoie rien, elle est déclarée avec le mot clé `void`.
- Les références sont des alias de variables qui permettent, à travers un autre nom, de les manipuler directement. On s'en sert, notamment, dans les paramètres des fonctions pour éviter des copies inutiles ou pour conserver les modifications faites sur un objet.
- Il faut faire attention à ce qu'une référence soit toujours valide. Cela nous interdit donc de renvoyer une référence sur une variable locale.
- Le mot clé `auto` ne conserve pas les références, mais le nouveau mot clé que nous avons introduit, `decltype`, si.
- La signature d'une fonction est ce qui la rend unique et est composée de son identifiant et de ses paramètres. On peut donc avoir plusieurs fonctions avec le même identifiant si les paramètres sont différents, ce qu'on appelle la surcharge.
- Le prototype d'une fonction permet, entre autres, de résoudre le problème des appels croisés.
- Le prototype contenant l'identificateur, le type de retour et les paramètres d'une fonction, il donne suffisamment d'informations pour ne pas avoir besoin de lire l'implémentation.

SOLUTIONS

section Afficher un rectangle

```
#include <iostream>

void rectangle(int longueur, int largeur)
{
    for (auto i { 0 }; i < longueur; ++i)
    {
        for (auto j { 0 }; j < largeur; ++j)
        {
            std::cout << "**";
        }

        std::cout << std::endl;
    }
}

int main()
{
    rectangle(2, 5);
    rectangle(4, 3);
    return 0;
}
```

SOLUTIONS

section Distributeur d'argent

```
#include <iostream>
#include <vector>

std::vector<int> distributeur(int total, std::vector<int>
coupures_disponibles)
{
    std::vector<int> resultat {};

    for (auto coupure : coupures_disponibles)
    {
        resultat.push_back(total / coupure);
        total %= coupure;
    }

    return resultat;
}

int main()
{
    std::vector<int> const coupures_disponibles { 500, 200, 100, 50,
20, 10, 5, 2, 1 };
    auto const coupures { distributeur(285, coupures_disponibles) };

    for (auto coupure : coupures)
    {
        std::cout << coupure << std::endl;
    }

    return 0;
}
```

Bonus :

```
#include <iostream>
#include <vector>

std::vector<int> distributeur(int total, std::vector<int>
coupures_disponibles)
{
    std::vector<int> resultat {};

    for (auto coupure : coupures_disponibles)
    {
        resultat.push_back(total / coupure);
        total %= coupure;
    }

    return resultat;
}
```

SOLUTIONS

```

void affichage_distributeur(std::vector<int> total,
    std::vector<int> coupures_disponibles)
{
    int i { 0 };
    for (auto coupure : total)
    {
        // Si 0, ça veut dire qu'il n'y a aucune coupure de ce type
        // dans le total, donc nous n'afficherons rien.
        if (coupure != 0)
        {
            std::cout << coupure << " x " << coupures_disponibles[i] << "
euros" << std::endl;
        }
        ++i;
    }
}

int main()
{
    std::vector<int> const coupures_disponibles { 500, 200, 100, 50,
    20, 10, 5, 2, 1 };
    auto const coupures_285 { distributeur(285,
    coupures_disponibles) };
    affichage_distributeur(coupures_285, coupures_disponibles);

    // Moins de coupures disponibles.
    std::vector<int> const autre_coupures_disponibles { 200, 50, 10,
    1 };
    auto const coupures_45874 { distributeur(45874,
    autre_coupures_disponibles) };
    affichage_distributeur(coupures_45874,
    autre_coupures_disponibles);

    return 0;
}

```

SOLUTIONS

section Parenthésage

```
#include <iostream>

bool parentheses(std::string expression)
{
    int ouvrantes { 0 };
    int fermantes { 0 };

    for (auto caractere : expression)
    {
        if (caractere == '(')
        {
            ++ouvrantes;
        }
        else
        {
            ++fermantes;
        }

        if (fermantes > ouvrantes)
        {
            // Pas la peine de continuer, l'expression est invalide.
            return false;
        }
    }

    return ouvrantes == fermantes;
}

int main()
{
    std::cout << std::boolalpha;
    std::cout << parentheses("((()))") << std::endl;
    std::cout << parentheses("(()())") << std::endl;
    std::cout << parentheses(")(")(") << std::endl;

    return 0;
}
```

11

Erreur, erreur, erreur...

Depuis le début de ce livre, nous avons été épargnés par les erreurs. Nous avons certes appris à sécuriser un peu les entrées, mais c'est tout. Ça ne sera pas toujours le cas. Il serait de bon ton que nos programmes soient *résistants aux erreurs*, afin de les rendre fiables et robustes.

Le but de ce chapitre est d'introduire la gestion des erreurs ainsi que la réflexion à mener lors de l'écriture de son code.

11.1. L'utilisateur est un idiot

Derrière ce titre un poil provoquant se cache une vérité informatique universelle : peu importe que ce soit consciemment ou non, il arrivera forcément que l'utilisateur rentre une information incomplète, fausse ou totalement différente de ce qu'on attendait. Que ce soit parce qu'il veut s'amuser avec votre programme ou bien parce qu'il a mal compris une consigne, a oublié un caractère ou s'est trompé de ligne, votre programme doit être capable de gérer ça.

Vérifier les entrées veut bien sûr dire s'assurer qu'elles soient du bon type, mais aussi contrôler que la donnée est cohérente et valide. Ainsi, vous ne pouvez pas dire que vous êtes né un 38 avril, car un mois a 31 jours au maximum. Si la donnée n'est pas correcte, même si elle est du bon type, il faut demander à l'utilisateur de la ressaisir. Cela se fait relativement facilement.

```
#include <iostream>

int main()
{
    int jour { 0 };
    std::cout << "Donne un jour entre 1 et 31 : ";

    while (!(std::cin >> jour) || jour < 0 || jour > 31)
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();
        std::cin.ignore(255, '\n');
    }
}
```



```
std::cout << "Jour donné : " << jour << std::endl;
return 0;
}
```

Tant que l'entrée met `std::cin` dans un état invalide ou qu'elle n'est pas dans la plage de valeurs autorisées, on demande à l'utilisateur de saisir une nouvelle donnée. Vous remarquerez que ce code est tiré de notre fonction `entree_securisee`. Nous l'avons extraite pour pouvoir ajouter des conditions supplémentaires. Au chapitre suivant ([Section 12.7, \[Pratique\] Gérer les erreurs d'entrée • Partie VI](#)), nous verrons une façon d'améliorer ça.

Pour l'instant, reprenez qu'il faut vérifier qu'une donnée est cohérente en plus de s'assurer qu'elle est du bon type.

11.2. À une condition... ou plusieurs

Écrire une fonction peut être plus ou moins simple, selon ce qu'elle fait, mais beaucoup de développeurs oublient une étape essentielle qui consiste à *réfléchir au(x) contrat(s) que définit une fonction*. Un contrat est un accord entre deux parties, ici celui qui utilise la fonction et celui qui la code.

- La fonction mathématique `std::sqrt`, qui calcule la racine carrée d'un nombre, attend qu'on lui donne un réel nul ou positif. Si ce contrat est respecté, elle s'engage à retourner un réel supérieur ou égal à zéro dont le carré est égal à la valeur donnée en argument. Le contrat inhérent à la fonction `std::sqrt` se résume donc ainsi : *Donne-moi un entier positif, et je te renverrai sa racine carrée*. Ça paraît évident, mais c'est important de le garder en tête.
- Quand on récupère un élément d'un tableau avec `[]`, il faut fournir un index valide, compris entre 0 et la taille du tableau moins un. Si on fait ça, on a la garantie d'obtenir un élément valide.
- Si l'on a une chaîne de caractères ayant au moins un caractère, on respecte le contrat de `pop_back` et on peut retirer en toute sécurité le dernier caractère.

On voit, dans chacun de ces exemples, que la fonction attend qu'on respecte une ou plusieurs conditions, que l'on nomme les *préconditions*. Si celles-ci sont respectées, la fonction s'engage en retour à respecter sa part du marché, qu'on appelle les *postconditions*. Les passionnés de mathématiques feront un parallèle avec les domaines de définition des fonctions.

Le but de cette réflexion sur les conditions est de produire du code plus robuste, plus fiable et plus facilement testable, et ainsi toujours tendre vers une plus grande qualité.

En effet, en prenant quelques minutes pour penser à ce qu'on veut que la fonction fasse ou ne fasse pas, on sait plus précisément quand elle fonctionne et quand il y a un bug.

Contrats assurés par le compilateur

Le typage

Un exemple simple de contrat, que nous avons déjà vu sans même le savoir, s'exprime avec le typage. Si une fonction s'attend à recevoir un entier, on ne peut pas passer un `std::vector`. Le compilateur s'assure qu'on passe en arguments des types compatibles avec ce qui est attendu.

Avec `const`

Un autre type de contrat qu'on a déjà vu implique l'utilisation de `const`. Ainsi, une fonction qui attend une référence sur une chaîne de caractères constante garantit que celle-ci restera intacte, sans aucune modification, à la fin de la fonction. Cette postcondition est vérifiée par le compilateur, puisqu'il est impossible de modifier un objet déclaré comme étant `const`.

Vérifions nous aussi

Le compilateur fait une partie du travail certes, mais la plus grosse part nous revient. Pour vérifier nos contrats, nous avons dans notre besace plusieurs outils que nous allons voir.

11.3. Le développeur est un idiot

Vous souvenez vous du chapitre sur les tableaux ? Quand nous avons vu l'accès aux éléments avec les crochets, nous vous avons dit de vérifier que l'indice que vous utilisez est bien valide, sous peine de comportements indéterminés. Si vous ne le faites pas, alors c'est de votre faute si le programme plante, car vous n'avez pas respecté une précondition. C'est ce qu'on appelle une erreur de programmation ou *bug*. Lorsque ce genre de problème arrive, rien ne sert de continuer, mieux vaut arrêter le programme et corriger le code. Il existe justement un outil qui va nous y aider : les assertions.

Note > Pour utiliser les assertions, il faut inclure le fichier d'en-tête `<cassert>`.

Une assertion fonctionne très simplement. Il s'agit d'évaluer une condition quelconque. Si la condition est vraie, le programme continue normalement. Par contre, si elle se révèle fausse, le programme s'arrête brutalement. Voyez par vous-même ce qu'il se passe avec le code suivant.

```
#include <cassert>

int main()
{
    // Va parfaitement fonctionner et passer à la suite.
    assert(1 == 1);
    // Va faire planter le programme.
    assert(1 == 2);
    return 0;
}
```

```
Assertion failed: 1 == 2, file d:\documents\visual studio 2017\projects
\zdscpp\zdscpp\main.cpp, line 8
```

Le programme plante et nous indique quel fichier, quelle ligne et quelle condition exactement lui ont posé problème. On peut même ajouter une chaîne de caractères pour rendre le message d'erreur plus clair. Cela est possible car, pour le compilateur, une chaîne de caractères est toujours évaluée comme étant **true**.

```
#include <cassert>

int main()
{
    // Va parfaitement fonctionner et passer à la suite.
    assert(1 == 1 && "1 doit toujours être égal à 1.");
    // Va faire planter le programme.
    assert(1 == 2 && "Ouh là, 1 n'est pas égal à 2.");
    return 0;
}
```

```
[Visual Studio]
Assertion failed: 1 == 2 && "Ouh là, 1 n'est pas égal à 2."
```

```
-----
```

```
[GCC]
prog.exe: prog.cc:8: int main(): Assertion `1 == 2 && "Ouh là, 1 n'est
pas égal à 2."' failed.
```

```
Aborted
```

```
-----
```

```
[Clang]
prog.exe: prog.cc:8: int main(): Assertion `1 == 2 && "Ouh là, 1 n'est
pas égal à 2."' failed.
```

```
Aborted
```

Pour donner un exemple concret, on va utiliser `assert` pour vérifier les préconditions de l'accès à un élément d'un tableau. En faisant cela, nous éviterons les comportements indéterminés qui surviennent si nous violons le contrat de la fonction.

```
#include <cassert>
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> const tableau { -4, 8, 452, -9 };
    int const index { 2 };

    assert(index >= 0 && "L'index ne doit pas être négatif.");
    assert(index < std::size(tableau)
           && "L'index ne doit pas être plus grand que la taille du tableau.");

    std::cout << "Voici l'élément "
               << index << " : "
               << tableau[index] << std::endl;

    return 0;
}
```

Si par inadvertance nous utilisons un indice trop grand, alors le plantage provoqué par `assert` nous le signalera et nous poussera ainsi à corriger le code. La documentation est, pour cela, une aide très précieuse, car elle indique comment réagit une fonction en cas d'arguments invalides. Dans la suite du livre, nous apprendrons à utiliser ces informations.



Mais je ne veux pas que mon programme plante. Pourquoi utiliser `assert` ?

En effet, plus un programme est résistant aux erreurs, mieux c'est. Cependant, `assert` s'utilise non pas pour les erreurs de l'utilisateur mais bel et bien pour celles du programmeur. Quand le développeur est distrait et écrit du code qui engendre des comportements indéterminés (comme un dépassement d'indice pour un tableau), tout peut arriver et, justement, ce n'est pas ce qu'on attend du programme. Cela n'a donc pas de sens de vouloir continuer l'exécution.

Par contre, les erreurs de l'utilisateur (par exemple rentrer une chaîne de caractères à la place d'un entier) ne sont absolument pas à traiter avec `assert`. On ne veut pas que notre programme plante brutalement parce que l'utilisateur s'est trompé d'un caractère.

Préconditions et assertions

Pour nos propres fonctions, le principe sera le même. Nous allons utiliser les assertions pour vérifier que les paramètres reçus respectent bien les contrats que nous avons préalablement définis.

Prenons un exemple élémentaire avec une fonction chargée de renvoyer le résultat de la division de deux réels.

```
#include <cassert>
#include <iostream>

double division(double numérateur, double dénominateur)
{
    assert(dénominateur != 0.0
        && "Le dénominateur ne peut pas valoir 0.");
    return numérateur / dénominateur;
}

int main()
{
    std::cout << "Numérateur : ";
    double numérateur { 0.0 };
    std::cin >> numérateur;

    double dénominateur { 0.0 };
    // La responsabilité de vérifier que le contrat
    // est respecté appartient au code appelant.
    do
    {
        std::cout << "Dénominateur : ";
        std::cin >> dénominateur;

    } while (dénominateur == 0.0);

    std::cout << "Résultat : "
        << division(numérateur, dénominateur) << std::endl;
    return 0;
}
```

La fonction `division` pose ici une précondition, qui est que le dénominateur soit non nul. Si elle est respectée, alors elle s'engage, en postcondition, à retourner le résultat de la division du numérateur par le dénominateur. En revanche, si le contrat est violé, le programme va s'interrompre sur le champ, car une division par zéro est mathématiquement impossible. Si le programme continuait, tout et n'importe quoi pourrait arriver.

11.4. Les tests unitaires à la rescousse

Pourquoi tester ?

C'est une bonne question. Pourquoi ne pas se contenter de lancer le programme et de voir soi-même avec quelques valeurs si tout va bien ? Parce que cette méthode est mauvaise. Déjà, nous n'allons tester que quelques valeurs, en général des valeurs qui sont correctes, sans forcément penser aux valeurs problématiques. De plus, ces tests sont consommateurs en temps, puisqu'il faut les faire à la main.

À l'opposé, en écrivant des tests nous-mêmes, nous bénéficions de plusieurs avantages.

- Nos tests sont reproductibles, on peut les lancer à l'infini et vérifier autant que l'on veut. Ils sont déjà écrits, pas de perte de temps donc.
- Conséquence du point précédent, on peut les lancer au fur et à mesure que l'on développe pour vérifier l'absence de régressions, c'est-à-dire des problèmes qui apparaissent suite à des modifications, comme une fonction qui ne renvoie plus un résultat correct alors qu'auparavant, c'était le cas.
- Comme on ne perd pas de temps manuellement, on peut écrire davantage de tests, qui permettent donc de détecter plus facilement et rapidement d'éventuels problèmes. Ceux-ci étant détectés plus tôt, ils sont plus faciles à corriger et le code voit sa qualité progresser.

Un test unitaire, qu'est-ce que c'est ?

Un test unitaire, abrégé TU, est un morceau de code dont le seul but est de tester un autre morceau de code, afin de vérifier si celui-ci respecte ses contrats. Il est dit *unitaire* car il teste ce morceau de code en toute indépendance du reste du programme. Pour cela, les morceaux de code qui seront testés unitairement doivent être relativement courts et facilement isolables. Voici des exemples d'identifiants explicites qui vont vous aider à voir ce à quoi peut ressembler un test unitaire.

- `test_sinus_0_retourne_0`
- `test_cosinus_pi_retourne_moins1`
- `test_parenthesage_expression_simple_retourne_true`

Notez que les noms des fonctions sont un peu longs mais très clairs. Cela est important, car quand certains tests échouent, s'ils ont un nom clair, qui définit sans ambiguïté ce que chacun d'eux vérifient, il est plus facile de corriger ce qui ne va pas. Ainsi, si la fonction `test_cosinus_zero` échoue, vous savez où regarder. À l'inverse, un nom trop

vague, trop générique, comme `test_cosinus`, ne nous donnerait aucune indication sur ce qui n'est pas correct.

Et si ce genre de test est nommé unitaire, c'est qu'il en existe d'autres types, que nous ne verrons pas dans le cadre de ce livre, mais dont vous entendrez parler au fur et à mesure que vous continuerez en programmation.

- *Tests d'intégration* : ils vérifient que différents modules s'intègrent bien entre eux. On peut tester, par exemple, que le module de paiement s'intègre bien au reste de l'application déjà développé.
- *Tests fonctionnels* : ils vérifient que le produit correspond bien à ce qu'on a demandé. Des tests fonctionnels sur une calculatrice vérifieront que celle-ci offre bien les fonctionnalités demandées (addition, logarithme, etc.) par rapport à ce qui avait été défini et prévu (si l'on avait demandé le calcul de cosinus, il faut que la calculatrice l'implémente).
- *Tests d'UI* [interface utilisateur, de l'anglais *user interface*] : ils vérifient (dans le cas d'une application graphique) que les boutons sont bien au bon endroit, les raccourcis, les menus et sous-menus, etc.
- Et encore de nombreux autres que nous passerons sous silence.

Écrire des tests unitaires

Il existe de nombreuses bibliothèques pour écrire des tests unitaires en C++, certaines basiques, d'autres très complètes, en bref, pour tous les goûts. Dans notre cas, nous n'avons pas besoin d'une solution aussi poussée. Nous allons écrire nos tests très simplement : quelques fonctions que nous lancerons dans le `main` pour vérifier que tout va bien.

Et pour illustrer cette section, nous allons tester un type que nous connaissons bien, `std::vector`. Bien entendu, il a déjà été testé bien plus complètement et profondément par les concepteurs de la bibliothèque standard, mais ce n'est pas grave.

Par exemple, pour tester que la fonction `push_back` fonctionne correctement, on va vérifier qu'elle augmente bien le nombre d'éléments du tableau de un, et que l'élément ajouté se trouve bien à la fin du tableau. On obtient un code qui ressemble à ça.

```
#include <cassert>
#include <iostream>
#include <vector>

void test_push_back_taille_augmentee()
{
    std::vector<int> tableau { 1, 2, 3 };
```

```

assert(std::size(tableau) == 3
    && "La taille doit être de 3 avant l'insertion.");

tableau.push_back(-78);
assert(std::size(tableau) == 4
    && "La taille doit être de 4 après l'insertion.");
}

void test_push_back_element_bonne_position()
{
    std::vector<int> tableau { 1, 2, 3, 4};
    int const element { 1 };

    tableau.push_back(element);

    assert(tableau[std::size(tableau) - 1] == element
        && "Le dernier élément doit être 1.");
}

int main()
{
    test_push_back_taille_augmentee();
    test_push_back_element_bonne_position();

    return 0;
}

```

Si on lance ce code, on voit que rien ne s'affiche et c'est normal. Comme le `std::vector` fourni dans la bibliothèque standard est de qualité, on est assuré qu'il se comportera bien comme il doit. Cependant, si nous avions dû écrire nous-mêmes `std::vector`, alors ces tests nous auraient assurés de la qualité de notre code.

Vous pouvez vous entraîner en écrivant les tests unitaires des différents exemples et exercices des chapitres passés.

Les tests unitaires et les postconditions

Rappelez-vous, nous avons défini les postconditions comme étant les contrats que doit respecter une fonction si ses préconditions le sont. Nous pouvons maintenant, grâce aux tests unitaires, nous assurer que la fonction fera bien son travail en lui fournissant des paramètres corrects et en vérifiant que les résultats sont valides et correspondent à ce qui est attendu.

11.5. [Pratique] Gérer les erreurs d'entrée • Partie IV

Un autre genre de mauvaise pratique consiste à écrire des littéraux en dur dans le code. Un exemple, que nous avons nous-mêmes introduit dans ce livre (honte à nous !), se trouve dans les précédents cas pratiques sur la gestion des entrées.


```
std::cin.ignore(255, '\n');
```

On décide d'ignorer jusqu'à 255 caractères, mais que se passe-t-il si l'utilisateur décide d'écrire un roman en guise d'entrée ? Nous ne supprimons pas tous les caractères et notre fonction échoue à remplir sa mission : c'est un bug, une erreur de notre part. Afin de le corriger, il faudrait vider le bon nombre de caractères. Comment faire ?

Dans le fichier d'en-tête `<limits>`, il existe un type `std::numeric_limits` qui permet pour chaque type (comme `int`, `double` et autres) d'en obtenir quelques propriétés, telles que la valeur maximale ou la valeur minimale que peut stocker ce type. Comme ces valeurs peuvent varier selon la plateforme et le compilateur, utiliser `std::numeric_limits` à la place de constantes écrites en dur est un gage d'une meilleure robustesse et donc de qualité.

```
std::numeric_limits</* type en question */>::/*fonction à appeler */

#include <iostream>
#include <limits>

int main()
{
    std::cout << "Valeur minimale d'un int : "
              << std::numeric_limits<int>::min() << std::endl;
    std::cout << "Valeur maximale d'un int : "
              << std::numeric_limits<int>::max() << std::endl;

    std::cout << "Valeur minimale d'un double : "
              << std::numeric_limits<double>::min()
              << std::endl;
    std::cout << "Valeur maximale d'un double : "
              << std::numeric_limits<double>::max()
              << std::endl;

    return 0;
}
```

Pour résoudre le problème soulevé en introduction à cette section, il faut juste savoir combien de caractères peut stocker au maximum `std::cin`. Un `int`, par exemple, pourrait être trop petit et ne pas suffire, ou être trop grand et donc nous créer des problèmes. Le type `std::streamsize` est justement là pour ça.

En se basant sur ces informations, vous avez toutes les clés pour rendre notre fonction de gestion des entrées plus robuste.

✧ [Voir la solution](#)

11.6. L'exception à la règle

La problématique

Au fur et à mesure que vous coderez des programmes de plus en plus complexes, vous verrez qu'il existe plein de cas d'erreurs extérieures. Parfois, un fichier sera manquant ou supprimé pendant la lecture. D'autres fois, il n'y aura plus assez de mémoire pour stocker un gros tableau. Ce sont des situations qui n'arrivent que rarement, mais que le programme doit être capable de gérer.

Prenons un exemple très simple, avec l'ouverture et la lecture d'un fichier. Ce programme récupère un tableau de chaînes de caractères, lues depuis un fichier quelconque.

```
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

std::vector<std::string> lire_fichier(std::string const & nom_fichier)
{
    std::vector<std::string> lignes {};
    std::string ligne { "" };
    std::ifstream fichier { nom_fichier };

    while (std::getline(fichier, ligne))
    {
        lignes.push_back(ligne);
    }

    return lignes;
}

int main()
{
    std::string nom_fichier { "" };
    std::cout << "Donnez un nom de fichier : ";
    std::cin >> nom_fichier;

    auto lignes = lire_fichier(nom_fichier);
    std::cout << "Voici le contenu du fichier : "
              << std::endl;
    for (auto const & ligne : lignes)
    {
        std::cout << ligne << std::endl;
    }

    return 0;
}
```

Si nous ne pouvons pas ouvrir le fichier parce que celui-ci n'existe pas, par exemple, alors il faut trouver un moyen de le signaler à l'utilisateur.

On peut modifier la fonction pour qu'elle renvoie un booléen, et le tableau sera un argument transmis par référence. C'est la façon de faire en C, mais elle comporte un gros défaut : on peut très bien ignorer la valeur de retour de la fonction et ainsi ne pas détecter l'erreur potentielle. De plus, elle est peu élégante car elle surcharge le code de paramètres et de valeurs de retour inutiles (au sens où elles ne servent qu'à transmettre les erreurs et ne font pas partie du fonctionnement "théorique" du programme) ainsi que de code de vérification d'erreur lourd. Cette solution n'est donc pas bonne, puisqu'elle nuit à la robustesse et à l'expressivité du code.

On peut aussi afficher un message et demander de saisir un nouveau nom. C'est une solution parmi d'autres, mais elle implique de *rajouter des responsabilités* à notre fonction, puisque en plus de lire un fichier, elle doit maintenant gérer des entrées. De plus, si nous voulons changer nos entrées/sorties de la console vers un fichier, ou même un programme graphique, nous sommes coincés.

La solution consiste à utiliser un nouveau concept que nous allons introduire : les exceptions.

C'est quoi une exception ?

C'est un mécanisme qui permet à un morceau de code, comme une fonction, de signaler au morceau de code qui l'a appelé que quelque chose d'exceptionnel – c'est-à-dire indépendant du développeur – s'est passé. On dit du premier morceau qu'il *lance*/*lève l'exception* [en anglais *throw*] et que le second *la rattrape* [en anglais *catch*]. Cela permet de gérer l'erreur dans le morceau de code appelant.

Contrairement aux assertions qui vont signaler la présence d'erreurs internes (de bugs à corriger), les exceptions s'utilisent pour les erreurs externes (erreurs qui peuvent arriver même si votre programme est parfaitement codé). Ne les voyez donc pas comme deux méthodes opposées, mais, bien au contraire, complémentaires.

Les exceptions sont notamment utilisées dans le cadre de la programmation par contrat : lorsqu'une fonction dont les préconditions sont vérifiées est dans l'incapacité de vérifier une de ses postconditions à cause d'un événement exceptionnel (un fichier inexistant par exemple), on lance une exception.

Lancement des exceptions dans 3, 2, 1...

Tout d'abord et comme pour beaucoup de choses en C++, il faut inclure un fichier d'en-tête, qui se nomme ici `<stdexcept>`. Ensuite, le reste est simple. On utilise le mot clé `throw` suivi du type de l'exception qu'on veut lancer. Car, de même qu'il existe des types pour stocker des caractères, des entiers, des réels et autres, il existe différents types d'exceptions pour signaler une erreur de taille, une erreur de mémoire, une erreur d'argument, etc.

Celle que nous allons utiliser s'appelle `std::runtime_error` et représente n'importe quelle erreur à l'exécution. Elle attend une chaîne de caractères, le message d'erreur.

```
#include <fstream>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

std::vector<std::string> lire_fichier(std::string const & nom_fichier)
{
    std::vector<std::string> lignes {};
    std::string ligne { "" };

    std::ifstream fichier { nom_fichier };
    if (!fichier)
    {
        // Si le fichier ne s'ouvre pas, alors on lance une exception
        // pour le signaler.
        throw std::runtime_error("Fichier impossible à ouvrir.");
    }

    while (std::getline(fichier, ligne))
    {
        lignes.push_back(ligne);
    }

    return lignes;
}

int main()
{
    std::string nom_fichier { "" };
    std::cout << "Donnez un nom de fichier : ";
    std::cin >> nom_fichier;

    auto lignes = lire_fichier(nom_fichier);
    std::cout << "Voici le contenu du fichier : "
        << std::endl;

    for (auto const & ligne : lignes)
    {
```

```

        std::cout << ligne << std::endl;
    }

    return 0;
}

```

Note > Pour tester si notre fichier a bien été ouvert, il suffit d'un simple `if`.

Lancez le programme et regardez-le ... planter lamentablement. La vie est cruelle.

Que s'est-il passé ? Lors du déroulement du programme, dès qu'on tombe sur une ligne `throw`, l'exécution de la fonction où l'on se trouve s'arrête immédiatement (on dit que l'exception *suspend l'exécution*) et on revient à la fonction appelante. Si celle-ci n'est pas en mesure de gérer l'exception, alors on revient à la précédente encore. Quand enfin on arrive à la fonction `main` et que celle-ci ne sait pas comment gérer l'exception elle non plus, la norme C++ prévoit de tuer tout simplement le programme.

Attends que je t'attrape !

Il nous faut maintenant apprendre à rattraper (on dit aussi *catcher*) l'exception afin de pouvoir la traiter. Pour cela, on utilise la syntaxe `try catch` [`try` étant le mot anglais pour essayer].

Concrètement, si un code est situé dans un bloc `try` et que ce code lève une exception, l'exécution du programme s'arrête à cette ligne et on part directement dans le bloc `catch` pour traiter ladite exception. Quant à ce dernier, il faut lui préciser (en plus du mot clé `catch`) le type d'exception à rattraper, par référence constante. Une fois une exception attrapée, on peut afficher le message qu'elle contient en appelant la fonction `what`.

```

#include <fstream>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

std::vector<std::string> lire_fichier(std::string const & nom_fichier)
{
    std::vector<std::string> lignes {};
    std::string ligne { "" };

    std::ifstream fichier { nom_fichier };
    if (!fichier)
    {
        throw std::runtime_error("Fichier impossible à ouvrir.");
    }

    while (std::getline(fichier, ligne))

```

```

    {
        lignes.push_back(ligne);
    }

    return lignes;
}

int main()
{
    std::string nom_fichier { "" };
    std::cout << "Donnez un nom de fichier : ";
    std::cin >> nom_fichier;

    try
    {
        // Dans le try, on est assuré que toute exception levée
        // pourra être traitée dans le bloc catch situé après.

        auto lignes = lire_fichier(nom_fichier);
        std::cout << "Voici le contenu du fichier : "
            << std::endl;

        for (auto const & ligne : lignes)
        {
            std::cout << ligne << std::endl;
        }
    }
    // Notez qu'une exception s'attrape par référence constante.
    catch (std::runtime_error const & exception)
    {
        // On affiche la cause de l'exception.
        std::cout << "Erreur : "
            << exception.what() << std::endl;
    }

    return 0;
}

```

Attention > Nous l'avons écrit plus haut, mais on le remet bien lisiblement ici : dès qu'un appel à une fonction (dans le `try`) lève une exception, le code suivant n'est pas exécuté et on file directement dans le `catch`. C'est pour ça que le message **"Voici le contenu du fichier : "** n'est pas affiché.

Voilà, notre programme est maintenant capable de rattraper et de gérer cette situation exceptionnelle, bien qu'on se contente ici de simplement afficher la cause de l'erreur. On peut facilement imaginer un programme qui redemande de saisir un nom de fichier valide tant qu'on rencontre une erreur.

► 11.1. Essayez donc de le faire.

✧ [Voir la solution](#)

Attrapez-les tous !

Dans notre exemple, le `catch` qui suit le `try` permet d'attraper des exceptions de type `std::runtime_error`. Mais certaines fonctions peuvent lancer plusieurs types d'exceptions. Il suffit cependant d'enchaîner les `catch` pour attraper autant d'exceptions différentes qu'on le souhaite.

Preons l'exemple de la fonction `std::stoi`, vue au [chapitre précédent](#), permettant de convertir une chaîne de caractères en entier. Cette fonction peut lever deux types d'exceptions :

- `std::invalid_argument` si la chaîne de caractères est invalide et ne peut pas être convertie ;
- `std::out_of_range` si le nombre à convertir est trop grand.

```
#include <iostream>
#include <stdexcept>
#include <string>

int main()
{
    try
    {
        // Essayez avec un nombre très très grand ou bien avec des
        // lettres pour observer le comportement de std::stoi.
        int entier {

std::stoi("100000000000000000000000000000000000000000000000000000000000000")
        };
        std::cout << "Entier : " << entier << std::endl;
    }
    catch (std::invalid_argument const & exception)
    {
        std::cout << "Argument invalide : "
            << exception.what() << std::endl;
    }
    catch (std::out_of_range const & exception)
    {
        std::cout << "Chaîne trop longue : "
            << exception.what() << std::endl;
    }

    return 0;
}
```

Note > L'ordre dans lequel vous mettez les `catch` n'a, ici, pas d'importance. En effet, si le `catch` est du même type que l'exception lancée, on rentre dedans, sinon on le saute.

Lancez le programme, modifiez la valeur donnée en argument à `std::stoi` et voyez par vous-même comment on passe dans le `catch` attrapant l'exception lancée.

Un type pour les gouverner tous et dans le catch les lier

Il arrive, dans certains cas, qu'on ne veuille pas traiter différemment les exceptions qu'on attrape. Cela signifie-t-il alors qu'on doit écrire autant de `catch` qu'il y a d'exceptions potentiellement lançables ? Non, on peut utiliser une exception *générique*.

Le type `std::exception` décrit justement une exception générique, une erreur quelconque. Nous reviendrons **plus tard** sur les propriétés qui rendent ce résultat possible, mais, en attendant, retenir qu'on peut attraper n'importe quelle exception avec.

```
#include <iostream>
#include <stdexcept>
#include <string>

int main()
{
    try
    {
        // Essayez avec un nombre très très grand ou bien avec des
        // lettres pour observer le comportement de std::stoi.
        int entier {

std::stoi("10000000000000000000000000000000000000000000000000000000000000000000")
        };
        std::cout << "Entier : " << entier << std::endl;
    }
    catch (std::exception const & exception)
    {
        std::cout << "Une erreur est survenue : "
            << exception.what() << std::endl;
    }

    return 0;
}
```

Son utilisation n'est absolument pas contraire à l'utilisation de types d'exceptions précis. On peut ainsi décider d'attraper certaines exceptions bien précises en vue d'un traitement particulier et d'attraper toutes les autres avec `std::exception` et leur faire suivre le même processus. Il n'y a qu'une chose à garder en tête : le catch générique doit toujours se trouver en dernier.


```

#include <iostream>
#include <stdexcept>
#include <string>

int main()
{
    try
    {
        int entier { std::stoi("+a") };
        std::cout << "Entier : "<< entier << std::endl;
    }
    catch (std::invalid_argument const & exception)
    {
        // Message particulier affiché seulement dans le cas
        // d'une exception std::invalid_argument.
        std::cout << "Argument invalide : "
                  << exception.what() << std::endl;
    }
    catch (std::exception const & exception)
    {
        // Message générique pour tous les autres types
        // d'exceptions possibles.
        std::cout << "Erreur : "
                  << exception.what() << std::endl;
    }

    return 0;
}

```

LES DANGERS DU CATCH GÉNÉRIQUE

- Répétons-le : le `catch` générique doit toujours être en dernier. En effet, les exceptions sont filtrées dans l'ordre de déclaration des `catch`. Comme toutes les exceptions sont des versions plus spécifiques de `std::exception`, elles seront toutes attrapées par le `catch` générique et jamais par les éventuels autres `catch`.
- Ne prenez pas la mauvaise habitude de toujours tout attraper. En effet, vous pourriez attraper par inadvertance des exceptions que vous ne saurez pas gérer, car pas assez d'informations sur le problème ou pas moyen à cet endroit du code de corriger ledit problème. N'attrapez donc que celles que vous êtes en mesure de gérer.

11.7. [Pratique] Gérer les erreurs d'entrée • Partie V

Puisque nous en sommes à améliorer notre cas pratique, que diriez-vous d'afficher un meilleur message d'erreur ? En effet, peu importe qu'on soit dans le cas d'une entrée de type invalide (une chaîne de caractères à la place d'un nombre) ou dans le cas d'une entrée incorrecte (8 alors qu'on demande un numéro entre 0 et 5), on affiche toujours le même message. Que diriez-vous de le rendre plus précis ? C'est une chose importante que de signaler clairement à l'utilisateur l'erreur qui s'est produite.

Pour nous aider, il existe deux fonctions. Nous les appliquerons ici à `std::cin` mais elles sont disponibles également pour un flux de fichier ou de chaîne.

- `std::cin.eof()` renvoie `true` si le flux d'entrée est fermé, notamment dans le cas où on arrive à la fin d'un fichier. Cela peut aussi arriver quand vous tapez Ctrl+D dans le terminal sous GNU/Linux ou Ctrl+Z sous Windows ;
- `std::cin.fail()` renvoie `true` si une erreur quelconque d'entrée apparaît. On va s'en servir notamment dans le cas où une entrée n'est pas du bon type.

Avec ces éléments, vous êtes tout bon pour améliorer notre code. Bon courage.

✎ Voir la solution

? Obtenir un indice

11.8. Récapitulons !

- Les entrées de l'utilisateur peuvent poser problème, c'est pour ça que nous les vérifions avec des conditions.
- Créer une fonction demande une réflexion sur ses préconditions et postconditions.
- Les assertions sont là pour les erreurs de programmation, les fautes du programmeur. On s'en sert pour les préconditions, notamment.
- Les tests unitaires présentent de nombreux avantages, notamment celui de vérifier les postconditions de nos fonctions, et donc leur bon fonctionnement.
- Les valeurs écrites en dur sont une mauvaise pratique et peuvent même créer des problèmes.
- Les exceptions nous permettent de signaler des erreurs externes, des situations sur lesquelles le programme n'a pas prise.
- Une exception se lève avec le mot clé `throw`.
- Une exception se rattrape dans un bloc `catch`, qui suit immédiatement un bloc `try`.

- On peut avoir autant de blocs `catch` qu'on veut, un pour chaque type d'exception que l'on veut attraper.
- On peut attraper toutes les exceptions de manière générique en *catchant* `std::exception`.

SOLUTION

Q : 11.1

```
#include <fstream>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

std::vector<std::string> lire_fichier(std::string const &
    nom_fichier)
{
    std::vector<std::string> lignes {};
    std::string ligne { "" };

    std::ifstream fichier { nom_fichier };
    // Pour tester si un fichier est ouvert et valide,
    // un simple if suffit.
    if (!fichier)
    {
        throw std::runtime_error("Fichier impossible à ouvrir.");
    }

    while (std::getline(fichier, ligne))
    {
        lignes.push_back(ligne);
    }

    return lignes;
}

std::string demander_nom_fichier()
{
    std::string nom_fichier { "" };
    std::cout << "Donnez un nom de fichier : ";
    std::cin >> nom_fichier;
    return nom_fichier;
}

int main()
{
    std::string nom_fichier { demander_nom_fichier() };
    bool fichier_valide { true };

    do
    {
        try
        {
            auto lignes = lire_fichier(nom_fichier);
            std::cout << "Voici le contenu du fichier :" << std::endl;
            for (auto const & ligne : lignes)
            {
```

SOLUTION

```

        std::cout << ligne << std::endl;
    }

    // Si on atteint cette ligne, c'est qu'aucune exception
    // n'a été levée et donc que le fichier est OK.
    fichier_valide = true;
}
catch (std::runtime_error const & exception)
{
    std::cout << "Erreur : " << exception.what() << std::endl;
    // Si on atteint cette ligne, c'est qu'une exception
    // a été levée car le nom n'est pas bon.
    fichier_valide = false;
    // Donc on redemande le nom.
    nom_fichier = demander_nom_fichier();
}

} while (!fichier_valide);

return 0;
}

```

INDICE

Section 11.7, [Pratique] Gérer les erreurs d'entrée • Partie V

N'oubliez pas que s'il y a fermeture de flux, il faudra lancer une exception puisque nous serons dans l'impossibilité de vérifier la postcondition *affecter une valeur correcte fournie par le flux à la variable*.

12

Des fonctions somme toute lambdas

Vous rappelez-vous du [chapitre sur les algorithmes](#) ? Nous avons vu alors que nous pouvions, dans une certaine mesure, personnaliser leur comportement. Mais nous n'avions pas d'autres choix que d'utiliser les prédicats déjà fournis par la bibliothèque standard. Ce chapitre va corriger cela.

Nous allons découvrir comment écrire nos propres prédicats en utilisant les lambdas.

12.1. Une lambda, c'est quoi ?

Une lambda est une fonction, potentiellement anonyme, destinée à être utilisée pour des opérations locales. Détaillons la signification de ces caractéristiques :

- *une fonction* : c'est-à-dire un regroupement de plusieurs instructions, comme nous l'avons vu au [chapitre dédié](#) — rien de bien nouveau ;
- *anonyme* : contrairement aux fonctions classiques qui portent obligatoirement un nom (un identificateur), une lambda peut être utilisée sans qu'on lui donne de nom ;
- *pour des opérations locales* : une lambda peut être directement écrite dans une fonction ou un algorithme, au plus près de l'endroit où elle est utilisée, à la différence d'une fonction qui est à un endroit bien défini, à part.

***Note** > Si les syntaxes que nous allons découvrir sont celles de C++, les lambdas sont répandues et utilisées par de nombreux langages, comme C#, Haskell ou encore Python. Ce concept vient de la [programmation fonctionnelle](#), une autre façon de programmer et dont C++ a repris plusieurs éléments.*

12.2. Pourquoi a-t-on besoin des lambdas ?

En effet, c'est une question à se poser. Après tout, nous avons déjà les fonctions et donc rien ne nous empêche de les utiliser pour personnaliser nos algorithmes. C'est tout à fait possible. Seulement voilà, les fonctions présentent deux inconvénients :

- *argument mineur* : le compilateur n'arrive pas à optimiser aussi bien que si l'on utilise les lambdas ;

- argument majeur : on crée une fonction visible et utilisable par tout le monde, alors qu'elle n'a pour objectif que d'être utilisée localement. On dit que la fonction a une *portée globale*, alors qu'il vaudrait mieux qu'elle ait une *portée locale*.

Les lambdas sont donc une solution à ces deux inconvénients. Bien entendu, C++ vous laisse libre de vos choix. Si vous voulez utiliser des fonctions là où des lambdas sont plus adaptées, rien ne vous l'interdit. Mais s'il y a des outils plus efficaces, autant les utiliser.

12.3. Syntaxe

La syntaxe pour déclarer une lambda a des points communs avec celle des fonctions classiques, mais aussi quelques différences. Examinons-la ensemble.

`[zone de capture](paramètres de la lambda) -> type de retour { instructions }`

- zone de capture* : par défaut, une lambda est en totale isolation et ne peut manipuler aucune variable de l'extérieur. Grâce à cette zone, la lambda va pouvoir modifier des variables extérieures. Nous allons [y revenir](#) ;
- paramètres de la lambda* : exactement comme pour les fonctions, les paramètres de la lambda peuvent être présents ou non, avec utilisation de références et/ou `const` possible ;
- type de retour* : encore un élément qui nous est familier. Il est écrit après la flèche `->`. Il peut être omis dans quelques cas, mais nous allons l'écrire à chaque fois dans une volonté de concevoir un code clair et explicite ;
- instructions* : nous ne vous ferons pas l'affront de détailler.

Quelques exemples simples

Ainsi, l'exemple le plus simple et minimaliste qu'on puisse vous donner serait celui-ci.

```
int main()
{
    []{};
    // Ou version explicite :
    []() -> void {};

    return 0;
}
```

Personnalisons la lambda pour qu'elle prenne un message à afficher.

```
#include <iostream>
```



```
#include <string>

int main()
{
    [](std::string const & message) -> void {
        std::cout << "Message reçu : " << message << std::endl;
    };
    return 0;
}
```

Maintenant, que diriez-vous de la tester avec un algorithme pour en voir tout l'intérêt. Disons que l'on veut parcourir un tableau de chaînes de caractères.

Nous avons formaté le code pour qu'il soit plus lisible, notamment en allant à la ligne pour les instructions de la lambda. Bien évidemment, vous n'êtes pas obligé de faire comme ça, mais sachez que c'est une écriture lisible et répandue.

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main()
{
    std::vector<std::string> const chaines {
        "Un mot",
        "Autre chose",
        "Du blabla",
        "Du texte",
        "Des lettres"
    };

    std::for_each(std::begin(chaines), std::end(chaines),
        [](std::string const & message) -> void
        {
            std::cout << "Message reçu : " << message << std::endl;
        });

    return 0;
}
```

Cet exemple montre une lambda qui ne prend qu'un paramètre. En effet, `std::for_each` attend un *prédicat unaire*, c'est-à-dire que l'algorithme travaille sur un seul élément à la fois. La lambda ne prend donc qu'un seul paramètre.

L'exemple suivant reprend le tri par ordre décroissant qu'on a déjà vu avec des foncteurs, mais cette fois avec une lambda. Notez que celle-ci prend deux paramètres. C'est parce que `std::sort` attend un *prédicat binaire*, parce que l'algorithme manipule et compare deux éléments en même temps.

```

#include <algorithm>
#include <array>
#include <iostream>

int main()
{
    std::array<double, 4u> tableau { 1, 3.1415, 2.1878, 1.5 };

    // Tri dans l'ordre décroissant.
    std::sort(std::begin(tableau), std::end(tableau),
        [](double a, double b) -> bool
        {
            return a > b;
        });

    for (auto nombre : tableau)
    {
        std::cout << nombre << std::endl;
    }

    return 0;
}

```

Continuons avec l'algorithme `std::find_if`. Celui-ci fonctionne exactement comme `std::find`, mais prend en plus un prédicat pour rechercher tout élément le vérifiant. Que diriez-vous du coup de chercher tous les éléments impairs d'une collection d'entiers ? Comme on regarde élément par élément, il s'agit d'un prédicat unaire.

```

#include <algorithm>
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> const nombres { 2, 4, 6, 8, 9, 10, 11, 13, 16 };
    auto itereur { std::begin(nombres) };

    do
    {
        itereur = std::find_if(itereur, std::end(nombres),
            [](int nombre) -> bool
            {
                return nombre % 2 != 0;
            });

        if (itereur != std::end(nombres))
        {
            std::cout << "Nombre impair : " << *itereur << std::endl;
            // On n'oublie pas d'incrémenter,
            // sinon on bouclera indéfiniment.
            ++itereur;
        }
    } while (true);
}

```

```

    }

    } while (it != std::end(nombres));

    return 0;
}

```

Avez-vous noté, dans ces exemples, l'élégance des lambdas ? On les a juste sous les yeux, on voit directement ce que fait l'algorithme. Et en plus, on ne pollue pas l'espace global avec des fonctions à usage unique.

12.4. Entraînez-vous !

Vérifier si un nombre est négatif

Dans le chapitre consacré aux conteneurs, nous avons vu `std::all_of` qui permet de vérifier que tous les éléments respectent un prédicat. Il y a un algorithme semblable, `std::any_of`, qui s'utilise de la même manière mais examine si au moins un élément vérifie le prédicat.

Le but est donc de dire si une collection d'entiers est entièrement positive ou non.

✧ Voir la solution

Tri par valeur absolue

Rien de très compliqué, mais que diriez-vous de trier un tableau dans l'ordre croissant en utilisant la **valeur absolue** des nombres ? Cela se fait en utilisant la fonction `std::abs`, disponible dans le fichier d'en-tête `<cmath>`.

✧ Voir la solution

12.5. On va stocker ça où ?

Imaginons maintenant que l'on veuille appliquer une lambda plusieurs fois. Va-t-on copier-coller le code ? Vous connaissez déjà la réponse. Il suffit de créer une variable, tout simplement, comme nous l'avons fait jusque-là. La façon la plus simple est d'utiliser `auto`.

***Note** > On peut, comme le démontre le code, utiliser la syntaxe de C++ moderne, à base d'accolades {}, ou bien utiliser l'initialisation héritée du C à base du signe égal =. Tout est une question de goût. Nous avons choisi la deuxième syntaxe car nous la trouvons plus lisible, mais vous restez libre de vos choix.*

```

#include <algorithm>
#include <iostream>
#include <vector>

int main()
{
    auto lambda_avec_accolades { [](int nombre) -> void {
        std::cout << "Nombre reçu : " << nombre << std::endl;
    }};
    auto lambda_avec_egal = [](int nombre) -> void {
        std::cout << "Nombre reçu : " << nombre << std::endl;
    };

    // S'utilise comme une fonction classique,
    // de façon totalement transparente.
    lambda_avec_accolades(5);

    std::vector<int> const nombres { 1, 2, 3, 8, 94, 548 };
    // Ou bien dans un algorithme, aucun problème.
    std::for_each(std::cbegin(nombres), std::cend(nombres),
        lambda_avec_egal);

    return 0;
}

```

Stocker une lambda permet de garder les avantages de localité tout en évitant de répéter du code, dans le cas où elle est utilisée plusieurs fois.



Par curiosité, quel est le type explicite d'une lambda ?

C'est un type complexe qui n'est pas spécifié par la norme. Le compilateur transforme la lambda en autre chose, et cet autre chose dépend donc du compilateur que vous utilisez. Les lambdas nous donnent donc un excellent exemple de l'intérêt de `auto` et de la simplicité qu'il apporte.

12.6. Paramètres génériques

Tout comme il est possible de laisser le compilateur deviner le type de retour, on peut lui laisser le soin de déduire les paramètres de la lambda, en utilisant `auto` à la place du type explicite des paramètres. Bien entendu, le fait de l'utiliser pour certains paramètres ne nous empêche absolument pas d'avoir des paramètres au type explicitement écrit.

```

#include <iostream>
#include <string>

```

```

int main()
{
    // Le premier paramètre est générique, mais le deuxième
    // est explicitement marqué comme étant une chaîne.
    auto lambda = [](auto const & parametre, std::string const & message)
    -> void
    {
        std::cout << "Paramètre générique reçu : "
        << parametre << std::endl;
        std::cout << "Message reçu : " << message << std::endl;
    };

    lambda(1, "Test avec un entier.");
    lambda("str", "Test avec une chaîne de caractères.");
    lambda(3.1415, "Test avec un flottant.");

    return 0;
}

```

Du coup, grâce à ce nouveau pouvoir que nous avons, nous sommes en mesure d'utiliser une même lambda pour afficher deux collections de types différents.

```

#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main()
{
    auto enumeration = [](auto const & parametre) -> void
    {
        std::cout << "Paramètre reçu : " << parametre << std::endl;
    };

    // On affiche une collection de std::string.
    std::vector<std::string> const chaines {
        "Un mot",
        "Autre chose",
        "Du blabla",
        "Du texte",
        "Des lettres"
    };
    std::for_each(std::begin(chaines), std::end(chaines), enumeration);

    std::cout << std::endl;

    // Pas de problème non plus avec des entiers.
    std::vector<int> const nombres { 1, 5, -7, 67, -4454, 7888 };
    std::for_each(std::begin(nombres), std::end(nombres), enumeration);

    return 0;
}

```

12.7. [Pratique] Gérer les erreurs d'entrée • Partie VI

Avec les connaissances que vous avez, nous pouvons reprendre le code où nous l'avons laissé et commencer à réduire la duplication en utilisant une lambda. Faites donc une entrée sécurisée mais sans dupliquer le code.

```
#include <iostream>

int main()
{
    int jour { 0 };
    std::cout << "Quel jour es-tu né ? ";
    // Entrée.

    int mois { 0 };
    std::cout << "Quel mois ? ";
    // Entrée.

    int annee { 0 };
    std::cout << "Quelle année ? ";
    // Entrée.

    double taille { 0.0 };
    std::cout << "Quelle taille ? ";
    // Entree.

    std::cout << "Tu es né le "
        << jour << "/"
        << mois << "/"
        << annee << " et tu mesures "
        << taille << "m." << std::endl;
    return 0;
}
```

✧ [Voir la solution](#)

12.8. [Pratique] Gérer les erreurs d'entrée • Partie VII

Eh oui, nous allons encore enchaîner deux cas pratiques d'affilée ! Nous l'avons découpé pour que vous puissiez l'aborder avec plus de facilité.

Relisons le [chapitre sur les erreurs](#). Nous y avions vu une méthode pour non seulement vérifier que l'entrée ne faisait pas planter `std::cin`, mais en plus que la valeur donnée avait un sens. Nous pouvions ainsi vérifier qu'un jour était compris entre 1 et 31, ou qu'un mois était inclus entre 1 et 12. Mais pour cela, nous avons dû extraire notre code de la fonction `entree_securisee`.

Cependant, nous venons de voir que les lambdas acceptent des paramètres génériques avec `auto`. Alors, soyons fous, pourquoi ne pas essayer de passer une autre lambda ?

```
#include <iostream>

int main()
{
    auto lambda = [](auto parametre, auto fonction) -> void
    {
        fonction(parametre);
    };

    lambda(42, [](int entier) -> void {
        std::cout << entier << std::endl;
    });
    return 0;
}
```

Ça marche parfaitement ! Et si nous utilisons ce concept puissant en combinaison avec notre cas pratique précédent ? On pourrait ainsi vérifier que le mois et le jour sont corrects, ou bien même que la personne est née après 1900. Vous avez à présent tous les indices qu'il vous faut pour résoudre ce cas.

✧ [Voir la solution](#)

12.9. Capture en cours...

Capture par valeur

Abordons maintenant cette fameuse zone de capture. Par défaut, une lambda est en totale isolation du reste du code. Elle ne peut pas, par exemple, accéder aux variables de la fonction dans laquelle elle est écrite... sauf si l'on utilise la zone de capture pour dire à quels objets la lambda peut accéder. On peut bien sûr en capturer plusieurs.

```
#include <iostream>

int main()
{
    bool const un_booleen { true };
    int const un_entier { 42 };
    double const un_reel { 3.1415 };

    auto lambda = [un_booleen, un_entier, un_reel]() -> void
    {
        std::cout << "Le booléen vaut " << std::boolalpha
            << un_booleen << "." << std::endl;
    };
}
```

```

std::cout << "L'entier vaut " << un_entier << "." << std::endl;
std::cout << "Le réel vaut " << un_reel << "." << std::endl;
};

lambda();

return 0;
}

```

Notez que si l'on supprime `un_booleen` de la zone de capture, le code ne compile plus. C'est pourquoi on parle de zone de capture : la lambda capture notre variable pour l'utiliser. Cette capture est dite *par valeur*, ou *par copie*, car chaque variable est copiée.

Attention > Il est possible, notamment si vous utilisez GCC ou MinGW (avec Qt Creator, entre autres) que vous ayez deux warnings comme ceux ci-dessous.

```

warning: lambda capture 'un_booleen' is not required to be captured for
this use. [-Wunused-lambda-capture]
warning: lambda capture 'un_entier' is not required to be captured for
this use. [-Wunused-lambda-capture]

```

Selon la norme, il y a effectivement certains cas où l'on n'est pas obligé de capturer une variable pour l'utiliser. Cependant, ces cas ne sont pas très nombreux, difficiles à expliquer à ce stade de votre apprentissage et présents surtout par volonté de compatibilité avec l'ancienne norme C++03. Visual Studio 2019 ne respecte pas la norme sur ce point et donc nous sommes obligés de capturer toutes les variables que nous voulons utiliser si nous voulons que notre code compile avec. Voilà pourquoi nous vous invitons, très exceptionnellement, à ignorer le warning et à utiliser les mêmes codes que dans ce livre.

Modifier une variable capturée

Reprenons le code précédent et adaptons-le pour ne garder qu'un entier modifiable (sans `const` donc). Capturez-le et essayez de le modifier. Vous allez obtenir une belle erreur, et c'est fait exprès !



Je ne comprends pas pourquoi je n'ai pas le droit de les modifier. Si c'est une capture par valeur, alors je manipule des copies bien distinctes des objets originaux, il ne devrait pas y avoir de problème. Pourquoi ?

Il y a plusieurs réponses à cette question. L'une est apportée par [Herb Sutter](#), un grand programmeur, très investi dans C++ et l'écriture de sa norme : *Le fait qu'une variable capturée par copie ne soit pas modifiable semble avoir été ajouté parce qu'un utilisateur peut ne pas se rendre compte qu'il modifie une copie et non l'original.*

Il y en a d'autres, mais il nous manque encore des notions pour les comprendre. Vous les découvrirez par vous-même en continuant votre exploration de C++.

Retenez qu'on ne peut pas, par défaut, modifier une variable capturée par copie. Heureusement, le mot clé `mutable` placé juste après la liste des arguments et avant le type de retour rend malgré tout cela possible.

```
#include <iostream>

int main()
{
    int un_entier { 42 };

    // Le mot clé 'mutable' se place après les paramètres
    // de la lambda et avant le type de retour.
    auto lambda = [un_entier]() mutable -> void
    {
        un_entier = 0;
        std::cout << "L'entier vaut "
                   << un_entier << " dans la lambda.\n"
    };

    lambda();
    std::cout << "L'entier vaut "
               << un_entier << " dans main.\n";

    return 0;
}
```

Comme vous le montre le résultat de ce code, la modification est locale à la lambda, exactement comme avec les fonctions. Mais ce n'est pas le seul mode de capture existant, comme vous vous en doutez peut-être.

Capture par référence

Le principe est exactement le même, sauf que cette fois on a affaire directement à la variable capturée et non à sa copie et qu'on fait cela en ajoutant l'esperluette `&` devant. Cela permet, comme pour les paramètres, d'éviter une copie potentiellement lourde ou bien de sauvegarder les modifications faites à la variable.

```
#include <algorithm>
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> const nombres { 8, 7, 5, 4, 31, 98, 2, 77, 648 };

    int const diviseur { 2 };
    int somme { 0 };
}
```

```

// Il n'y a aucun problème à mélanger les captures
// par valeur et par référence.
std::for_each(std::cbegin(nombres), std::cend(nombres),
    [diviseur, &somme](int element) -> void
    {
        if (element % diviseur == 0)
        {
            somme += element;
        }
    });

std::cout << "La somme de tous les éléments divisibles par "
    << diviseur << " vaut "
    << somme << "." << std::endl;
return 0;
}

```

Notez qu'on peut capturer certaines variables par référence, d'autres par valeur. C'est exactement comme pour les paramètres d'une fonction, donc cela ne doit pas vous surprendre.

12.10. Récapitulons !

- Les lambdas sont des fonctions anonymes, destinées à être utilisées localement.
- Elles réduisent la taille du code, le rendent plus lisible et évitent de créer des fonctions à portée globale alors qu'elles sont à usage local.
- Les paramètres peuvent être explicitement typés ou automatiquement déduits en utilisant `auto`.
- La véritable nouveauté des lambdas, par rapport aux fonctions, est la zone de capture, qui permet de manipuler par référence et/ou valeur des objets de la portée englobante.
- On peut capturer explicitement certaines variables ou capturer toutes celles de la portée englobante.

SOLUTIONS

section Vérifier si un nombre est négatif

```
#include <algorithm>
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> const nombres { 1, 2, 3, 4, 5, 6, -7 };

    if (std::any_of(std::cbegin(nombres), std::cend(nombres), [](int
nombre) -> bool { return nombre < 0; }))
    {
        std::cout << "Au moins un nombre est négatif." << std::endl;
    }
    else
    {
        std::cout << "Tous les nombres sont positifs." << std::endl;
    }

    return 0;
}
```

SOLUTIONS

section Tri par valeur absolue

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <string>
#include <vector>

int main()
{
    std::vector<int> nombres { -8, 5, 97, -789, 0, 87, -88, -8555,
10000, 3 };
    std::sort(std::begin(nombres), std::end(nombres), [](int a, int
b) -> bool
    {
        return std::abs(a) < std::abs(b);
    });

    // Affichons pour vérifier.
    for (int nombre : nombres)
    {
        std::cout << nombre << std::endl;
    }

    return 0;
}
```

Ne soyez pas surpris par la sortie du programme. Rappelez-vous que nous avons trié le résultat en utilisant la valeur absolue, et que l’affichage nous montre les éléments triés, mais originaux.

13

Envoyez le générique !

Nous avons vu avec les lambdas qu'il est possible d'écrire du code générique. Cela évite notamment la duplication de code. Par exemple, notre fonction `entree_securisee` a pu être écrite une seule fois pour tous les types à la différence de notre ancienne version, qu'il fallait écrire une fois pour les `int` et une fois pour les `double`. L'intérêt donc est double : le code est plus facile à maintenir (puisque les modifications se font une seule fois) et plus besoin de réécrire une nouvelle version pour chaque nouveau type. Le problème, c'est qu'on ne sait faire ça qu'avec des lambdas. Or il serait dommage de se limiter à elles seules. Heureusement, en C++, il est possible de rendre n'importe quelle fonction générique. C'est justement ce que nous allons voir.

13.1. Quel beau modèle !

Pour créer des fonctions génériques en C++, on écrit ce que l'on appelle des *templates* [en français *modèle* ou *patron*] : au lieu d'écrire directement une fonction, avec des types explicites, on va en écrire une version générique avec des types génériques. C'est ensuite le compilateur qui, quand on en aura besoin, utilisera ce modèle pour créer une fonction avec les bons types. On dit qu'il *instancie le modèle*.

Allons droit au but, voici la syntaxe générale, qui ressemble beaucoup à celle utilisée pour créer une fonction.

```
template <typename T1, typename T2, ...>
type_de_retour identificateur(paramètres)
{
    instructions
}
```

Notez l'apparition de deux mots clés :

- `template` prévient le compilateur qu'on n'écrit pas une fonction classique mais un modèle ;
- `typename` permet de déclarer un nouveau nom de type générique.

Le principe est le suivant : au lieu d'utiliser uniquement des types précis (`int`, `double`, `std::string`, etc.) comme on en a l'habitude, on va définir des noms de types quel-

conques, c'est-à-dire *pouvant être remplacés par n'importe quel type* au moment de l'utilisation de la fonction. Le compilateur va ensuite créer la fonction à notre place, en remplaçant les types génériques par les types réels des arguments qu'on va lui passer.

Note > On peut ici remarquer un parallèle avec les concepts de paramètre et d'argument :

- *Le type générique représente ce que le template attend comme type, jouant un rôle similaire à celui d'un paramètre de fonction.*
- *Le type réel est ce qui va être utilisé pour créer la fonction en remplaçant le type générique, à l'instar de l'argument qui est la valeur avec laquelle est appelée la fonction.*

Un exemple valant mieux que mille mots, voici un template permettant d'afficher tous les éléments d'un `std::vector` dans la console. Nous avons déjà vu une telle fonction, mais ici la différence est qu'elle va fonctionner *quel que soit le type des éléments dans le tableau*.

```
#include <iostream>
#include <vector>

template <typename T>
void afficher(std::vector<T> const & v)
{
    for (auto const & e : v)
    {
        std::cout << e << std::endl;
    }
}

int main()
{
    std::vector<int> const tableau_entiers { 1, 3, 5, 7, 9 };
    afficher(tableau_entiers);

    std::cout << std::endl;

    std::vector<double> const tableau_reels { 1.2, 3.1415, 12.5, 2.7 };
    afficher(tableau_reels);

    std::cout << std::endl;

    std::vector<std::string> const tableau_chaines {
        "Hello",
        "World",
        "Les templates",
        "C'est super !"
    };
    afficher(tableau_chaines);

    return 0;
}
```

Quand le compilateur voit ce template, il comprend qu'il faut l'instancier trois fois, car il y a trois appels avec trois types différents. Bien sûr, la mécanique interne dépend du compilateur, mais on peut imaginer qu'il génère un code comme celui-ci.

```
#include <iostream>
#include <vector>

void afficher(std::vector<int> const & v)
{
    for (auto const & e : v)
    {
        std::cout << e << std::endl;
    }
}

void afficher(std::vector<double> const & v)
{
    for (auto const & e : v)
    {
        std::cout << e << std::endl;
    }
}

void afficher(std::vector<std::string> const & v)
{
    for (auto const & e : v)
    {
        std::cout << e << std::endl;
    }
}

int main()
{
    std::vector<int> const tableau_entiers { 1, 3, 5, 7, 9 };
    afficher(tableau_entiers);

    std::cout << std::endl;

    std::vector<double> const tableau_reels { 1.2, 3.1415, 12.5, 2.7 };
    afficher(tableau_reels);

    std::cout << std::endl;

    std::vector<std::string> const tableau_chaines {
        "Hello",
        "World",
        "Les templates",
        "C'est super !" };
    afficher(tableau_chaines);

    return 0;
}
```

Note > Il s'agit ici d'un principe de base de ce qu'on appelle la programmation générique : on se fiche du type réel des éléments du tableau, du moment qu'ils sont affichables ! En programmation générique, ce qui compte, ce n'est pas le type exact, mais ce que l'on peut faire avec.

On peut faire encore mieux : on se fiche du type réel du conteneur du moment qu'il est itérable. Essayons d'appliquer ce principe et d'écrire une fonction `afficher` qui marche avec n'importe quel type de conteneur itérable.

```
#include <array>
#include <iostream>
#include <list>
#include <string>
#include <vector>

template <typename Collection>
void afficher(Collection const & iterable)
{
    for (auto const & e : iterable)
    {
        std::cout << e << std::endl;
    }
}

int main()
{
    // Un std::vector est itérable, donc aucun problème.
    std::vector<int> tableau_entiers { 1, 3, 5, 7, 9 };
    afficher(tableau_entiers);

    std::cout << std::endl;

    // De même que pour un std::array.
    std::array<double, 4> const tableau_reels { 1.2, 3.1415, 12.5, 2.7 };
    afficher(tableau_reels);

    std::cout << std::endl;

    // Ou bien une std::list.
    std::list<std::string> const liste_chaines {
        "Hello",
        "World",
        "Les templates",
        "C'est super !"
    };
    afficher(liste_chaines);

    std::cout << std::endl;

    // Une chaîne de caractères est tout à fait itérable.
    std::string const chaine {
        u8"Ça marche même avec des chaînes de caractères !"
    };
    afficher(chaine);
}
```



```
    return 0;
}
```

Pour vous aider à bien visualiser ce qui se passe, voici ce que peut produire le compilateur quand il a fini d’instancier le modèle.

```
#include <array>
#include <iostream>
#include <list>
#include <string>
#include <vector>

void afficher(std::vector<int> const & iterable)
{
    for (auto const & e : iterable)
    {
        std::cout << e << std::endl;
    }
}

void afficher(std::array<double, 4> const & iterable)
{
    for (auto const & e : iterable)
    {
        std::cout << e << std::endl;
    }
}

void afficher(std::list<std::string> const & iterable)
{
    for (auto const & e : iterable)
    {
        std::cout << e << std::endl;
    }
}

void afficher(std::string const & iterable)
{
    for (auto const & e : iterable)
    {
        std::cout << e << std::endl;
    }
}

int main()
{
    std::vector<int> tableau_entiers { 1, 3, 5, 7, 9 };
    afficher(tableau_entiers);

    std::cout << std::endl;

    std::array<double, 4> const tableau_reels { 1.2, 3.1415, 12.5, 2.7 };
    afficher(tableau_reels);
}
```

```

std::cout << std::endl;

std::list<std::string> const liste_chaines {
    "Hello",
    "World",
    "Les templates",
    "C'est super !"
};
afficher(liste_chaines);

std::cout << std::endl;

std::string const chaine {
    u8"Ça marche même avec des chaînes de caractères !"
};
afficher(chaine);

return 0;
}

```

Magique, non ?

La bibliothèque standard : une fourmilière de modèles

En fait, on a vu et utilisé un nombre incalculable de fonctions génériques dans la bibliothèque standard : celles renvoyant des itérateurs (`std::begin`, `std::end`, etc.), les conteneurs, les algorithmes, etc. En effet, toutes ces fonctions peuvent s'utiliser avec des arguments de types différents. Pour `std::begin`, par exemple, il suffit que le type passé en argument ait la capacité de fournir un itérateur sur le premier élément.

Le principe même des templates est d'oublier le type pour se concentrer sur ce que peut faire ce type, et c'est très intéressant pour la bibliothèque standard puisqu'elle a vocation à être utilisée dans des contextes variés. En fait, sans les templates, il serait très difficile de concevoir la bibliothèque standard du C++. Cela nous donne un exemple concret de leur intérêt.

Un point de vocabulaire

Lorsqu'on parle de templates, on utilise un vocabulaire spécifique pour évoquer chaque étape dans leur processus de fonctionnement. Reprenons-le pour que vous le mémorisiez bien.

- *Template* : description générique de ce que fait la fonction.
- *Instanciation* : lorsque le compilateur crée une fonction à partir du modèle pour l'utiliser avec des types donnés.

- *Appel de fonction* : lorsqu'on appelle la fonction instanciée, le nom ne change pas de ce dont on a l'habitude.

13.2. [Pratique] Gérer les erreurs d'entrée • Partie VIII

Nous avons déjà, dans la [partie précédente](#), éliminé la duplication de code en utilisant une lambda. Nous avons également appris à transmettre une fonction, ce qui permet d'ajouter des vérifications supplémentaires. Cependant, notre code était dans une portée locale. Maintenant, vous disposez de toutes les clés pour corriger ce problème.

Le code à sécuriser est toujours le même.

```
#include <iostream>

int main()
{
    int jour { 0 };
    std::cout << "Quel jour es-tu né ? ";
    // Entrée.

    int mois { 0 };
    std::cout << "Quel mois ? ";
    // Entrée.

    int annee { 0 };
    std::cout << "Quelle année ? ";
    // Entrée.

    double taille { 0.0 };
    std::cout << "Quelle taille ? ";
    // Entree.

    std::cout << "Tu es né le "
        << jour << "/"
        << mois << "/"
        << annee << " et tu mesures "
        << taille << "m." << std::endl;
    return 0;
}
```

✧ [Voir la solution](#)

Et voilà ! Maintenant, `entree_securisee` est générique et utilisable à une portée globale, on a tout ce que l'on voulait ! Comme vous voyez, nous sommes de plus en plus efficaces pour éviter la duplication de code.

13.3. Instanciation explicite

Il arrive parfois que le compilateur ne puisse pas déterminer avec quels types il doit instancier un template, typiquement lorsque les types génériques n'apparaissent pas dans les paramètres de la fonction. C'est en particulier le cas lorsqu'il n'y a tout simplement pas de paramètre.

Par exemple, imaginons que notre fonction `entree_securisee`, au lieu de prendre la variable à modifier en paramètre, crée une variable du bon type et renvoie le résultat.

```
template <typename T>
T entree_securisee()
{
    // Initialisation par défaut, qui marche pour tous les types.
    T variable {};

    while (!(std::cin >> variable))
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }

    return variable;
}
```

Jusque-là, pas de problème. Essayons d'appeler la fonction.

```
#include <iostream>
#include <limits>

template <typename T>
T entree_securisee()
{
    T variable {};
    while (!(std::cin >> variable))
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }

    return variable;
}

int main()
{
    std::cout << "Quel jour es-tu né ? ";
    int jour { entree_securisee() };

    std::cout << "Quel mois ? ";
```

```

int mois { entree_securisee() };

std::cout << "Quelle année ? ";
int annee { entree_securisee() };

std::cout << "Quelle taille ? ";
double taille { entree_securisee() };

std::cout << "Tu es né le "
  << jour << "/"
  << mois << "/"
  << annee << " et tu mesures "
  << taille << "m." << std::endl;
return 0;
}

```

Et là, c'est le drame : impossible de compiler, il y a des erreurs remontées.

```

[Visual Studio]
Erreur C2783 'T entree_securisee(void)' : impossible de déduire
l'argument modèle pour 'T'.
Error C2783 'T entree_securisee(void)' : could not deduce template
argument for 'T'.

-----

[GCC]
prog.cc: In function 'int main()':
prog.cc:21:33: error: no matching function for call to
'entree_securisee()'
    21 | int jour { entree_securisee() };
        |             ^
prog.cc:5:3: note: candidate: 'template<class T> T entree_securisee()'
    5 | T entree_securisee()
        | ^~~~~~
prog.cc:5:3: note: template argument deduction/substitution failed:
prog.cc:21:33: note: couldn't deduce template parameter 'T'
    21 | int jour { entree_securisee() };
        |             ^
prog.cc:24:33: error: no matching function for call to
'entree_securisee()'
    24 | int mois { entree_securisee() };
        |             ^
prog.cc:5:3: note: candidate: 'template<class T> T entree_securisee()'
    5 | T entree_securisee()
        | ^~~~~~
prog.cc:5:3: note: template argument deduction/substitution failed:
prog.cc:24:33: note: couldn't deduce template parameter 'T'
    24 | int mois { entree_securisee() };
        |             ^
prog.cc:27:34: error: no matching function for call to
'entree_securisee()'
    27 | int annee { entree_securisee() };
        |             ^
prog.cc:5:3: note: candidate: 'template<class T> T entree_securisee()'

```

```

    5 | T entree_securisee()
      | ^~~~~~
prog.cc:5:3: note: template argument deduction/substitution failed:
prog.cc:27:34: note: couldn't deduce template parameter 'T'
    27 | int annee { entree_securisee() };
      |               ^
prog.cc:30:38: error: no matching function for call to
'entree_securisee()'
    30 | double taille { entree_securisee() };
      |                   ^
prog.cc:5:3: note: candidate: 'template<class T> T entree_securisee()'
    5 | T entree_securisee()
      | ^~~~~~
prog.cc:5:3: note: template argument deduction/substitution failed:
prog.cc:30:38: note: couldn't deduce template parameter 'T'
    30 | double taille { entree_securisee() };
      |                   ^

```

[Clang]

```

prog.cc:21:16: error: no matching function for call to
'entree_securisee'
    int jour { entree_securisee() };
               ^~~~~~
prog.cc:5:3: note: candidate template ignored: couldn't infer template
argument 'T'
T entree_securisee()
  ^
prog.cc:24:16: error: no matching function for call to
'entree_securisee'
    int mois { entree_securisee() };
               ^~~~~~
prog.cc:5:3: note: candidate template ignored: couldn't infer template
argument 'T'
T entree_securisee()
  ^
prog.cc:27:17: error: no matching function for call to
'entree_securisee'
    int annee { entree_securisee() };
               ^~~~~~
prog.cc:5:3: note: candidate template ignored: couldn't infer template
argument 'T'
T entree_securisee()
  ^
prog.cc:30:21: error: no matching function for call to
'entree_securisee'
    double taille { entree_securisee() };
                   ^~~~~~
prog.cc:5:3: note: candidate template ignored: couldn't infer template
argument 'T'
T entree_securisee()
  ^
4 errors generated.

```

Leur origine est simple : le compilateur n'a aucun moyen de savoir avec quel type instancier le template. Ce cas se présente aussi quand, pour un type générique attendu, on en donne plusieurs différents.

```
#include <iostream>

template <typename T>
T addition(T x, T y)
{
    return x + y;
}

int main()
{
    // Aucun problème, on a deux int ici.
    std::cout << addition(1, 2) << std::endl;
    // Oups. Le type déduit, c'est quoi alors ? int ou double ?
    std::cout << addition(1, 3.1415) << std::endl;

    return 0;
}
```

Dans le deuxième cas, il y a ambiguïté. Pour la lever, il suffit de préciser explicitement les types devant être utilisés pour l'instanciation, en les mettant entre chevrons juste avant les parenthèses de paramètres. La syntaxe ne vous est pas totalement inconnue, puisqu'elle est proche de celle utilisée pour les conteneurs (`std::vector<int>`, `std::list<double>`, etc.).

```
#include <iostream>
#include <limits>

template <typename T>
T entree_securisee()
{
    T variable {};
    while (!(std::cin >> variable))
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }

    return variable;
}

template <typename T>
T addition(T x, T y)
{
    return x + y;
}
```

```

int main()
{
    std::cout << "Quel jour es-tu né ? ";
    int jour { entree_securisee<int>() };

    std::cout << "Quel mois ? ";
    int mois { entree_securisee<int>() };

    std::cout << "Quelle année ? ";
    int annee { entree_securisee<int>() };

    std::cout << "Quelle taille ? ";
    double taille { entree_securisee<double>() };

    std::cout << "Tu es né le "
        << jour << "/"
        << mois << "/"
        << annee << " et tu mesures "
        << taille << "m." << std::endl;

    std::cout << "1 + 3.1415 = "
        << addition<double>(1, 3.1415) << std::endl;
    return 0;
}

```

Note > Cette précision explicite des types pour l'instanciation sert à lever les ambiguïtés, mais elle peut aussi s'utiliser dans d'autres cas. Par exemple, vous pouvez tout à fait préciser les types si vous trouvez ça plus lisible, même s'il n'y a pas d'ambiguïté.

13.4. Ce type-là, c'est un cas !

On a vu que les templates sont une fonctionnalité très puissante pour écrire du code générique, c'est-à-dire ne dépendant pas des types des paramètres mais des services, des fonctionnalités proposées par ces types. Cependant, on aimerait parfois que, pour un type particulier, la fonction se comporte différemment.

Revenons au template `afficher` qu'on a défini précédemment.

```

#include <array>
#include <iostream>
#include <list>
#include <string>
#include <vector>

template <typename Collection>
void afficher(Collection const & iterable)
{
    for (auto const & e : iterable)
    {
        std::cout << e << std::endl;
    }
}

```



```

}

int main()
{
    std::vector<int> tableau_entiers { 1, 3, 5, 7, 9 };
    afficher(tableau_entiers);

    std::cout << std::endl;

    std::array<double, 4> const tableau_reels { 1.2, 3.1415, 12.5, 2.7 };
    afficher(tableau_reels);

    std::cout << std::endl;

    std::list<std::string> const liste_chaines {
        "Hello",
        "World",
        "Les templates",
        "C'est super !"
    };
    afficher(liste_chaines);

    std::cout << std::endl;

    std::string const chaine {
        u8"Ça marche même avec des chaînes de caractères !"
    };
    afficher(chaine);

    return 0;
}

```

Ce code donne la sortie suivante.

```

1
3
5
7
9

1.2
3.1415
12.5
2.7

Hello
World
Les templates.
C'est super !

Ç
a

m
a

```

```
r  
c  
h  
e  
  
m  
ê  
m  
e  
  
a  
v  
e  
c  
  
d  
e  
s  
  
c  
h  
a  
î  
n  
e  
s  
  
d  
e  
  
c  
a  
r  
a  
c  
t  
è  
r  
e  
s  
  
!
```

Le résultat est assez désagréable à lire lorsqu'on utilise la fonction avec le type `std::string`. Heureusement, on a un moyen très simple de modifier le comportement de notre template pour un type de paramètre en particulier. C'est une notion que nous avons abordée dans le chapitre sur les fonctions : [la surcharge](#). On va surcharger la fonction `afficher` pour les chaînes de caractères. On appelle ça la *spécialisation*.

```

#include <array>
#include <iostream>
#include <list>
#include <string>
#include <vector>

template <typename Collection>
void afficher(Collection const & iterable)
{
    for (auto const & e : iterable)
    {
        std::cout << e << std::endl;
    }
}

// Version non-template ne fonctionnant que pour les chaînes de
// caractères.
void afficher(std::string const & str)
{
    std::cout << str << std::endl;
}

int main()
{
    std::vector<int> tableau_entiers { 1, 3, 5, 7, 9 };
    afficher(tableau_entiers);

    std::cout << std::endl;

    std::array<double, 4> const tableau_reels { 1.2, 3.1415, 12.5, 2.7 };
    afficher(tableau_reels);

    std::cout << std::endl;

    std::list<std::string> const liste_chaines {
        "Hello",
        "World",
        "Les templates",
        "C'est super !"
    };
    afficher(liste_chaines);

    std::cout << std::endl;

    std::string const chaine {
        u8"Ça marche même avec des chaînes de caractères !"
    };
    afficher(chaine);

    return 0;
}

```

Ce qui donne un résultat bien plus lisible en sortie.

```

1
3
5
7
9

1.2
3.1415
12.5
2.7

hello
world
Les templates.
C'est super !

Ça marche même avec des chaînes de caractères !

```

On a donc un template qui permet d’afficher toutes sortes de collections et un traitement à part pour les `std::string`. Bien entendu, on peut créer d’autres surcharges pour d’autres types, si on veut qu’ils aient un comportement bien précis.

Note > Le changement du comportement par surcharge fonctionne car le compilateur prend en priorité les fonctions non-templates lors du choix de la bonne surcharge. En effet, a priori, il devrait y avoir ambiguïté (et donc erreur), car il y a deux fonctions possibles : celle générée à partir du modèle et celle écrite explicitement. Heureusement, le choix a été fait par les concepteurs du C++ d’accorder la priorité aux fonctions écrites explicitement. Cela permet donc de rendre valide ce genre de code.

13.5. C++20 et les lambdas templates

C++14 a apporté avec lui l’utilisation du mot clé `auto` dans les paramètres d’une lambda, comme nous l’avons vu dans le chapitre précédent, ce qui permet une certaine genericité. C’est pratique, mais on reste limité. Par exemple, comment créer une lambda pour travailler sur un `std::vector<T>`, avec `T` pouvant être un `int`, un `std::string`, etc. ?

Avec C++20, ce problème est résolu, puisqu’il est maintenant possible d’appliquer la syntaxe d’une fonction générique à une lambda.

```

#include <iostream>
#include <string>
#include <vector>

using namespace std::literals;

int main()
{
    std::vector<int> const nombres { 1, 2, 3, 4, 5, 6 };
    std::vector<std::string> const mots {
        "Mot"s,

```

```

    "lettres"s,
    "test"s
};

auto const lambda = [<typename T>(std::vector<T> const& tableau) ->
void
{
    std::cout << "Le tableau donné contient "
    << std::size(tableau) << " éléments.\n";
};

lambda(nombres);
lambda(mots);

return 0;
}

```

```

Le tableau donné contient 6 éléments.
Le tableau donné contient 3 éléments.

```

13.6. Récapitulons !

- Les templates permettent de gagner en généricité et donc de réduire la duplication de code.
- C'est une généralisation du principe de généricité, qu'on a déjà observé avec les lambdas.
- Cela permet une plus grande flexibilité ainsi que l'écriture de fonctions génériques dans la portée globale.
- La bibliothèque standard regorge de templates, car la généricité en est un aspect important.
- On peut préciser, ou non, explicitement les types lors de l'instanciation, mais certains cas rendent cette précision obligatoire.
- On peut surcharger une fonction générique pour personnaliser son comportement pour certains types explicitement donnés.
- Les templates peuvent également s'utiliser avec les lambdas.

14

De nouvelles structures de données

Jusque-là, nous avons eu l'occasion d'utiliser des données sous de nombreuses formes : types de base, chaînes de caractères, collections de données, etc. Cependant, toutes ces formes de données sont finalement assez rudimentaires, et on souhaite parfois avoir plus de souplesse dans la manière dont sont structurées nos données.

Dans ce chapitre, nous allons justement découvrir de nouvelles structures de données qui vont nous offrir plus de flexibilité dans nos programmes. Pour chaque structure, nous verrons comment elle s'utilise et en quoi elle est utile.

14.1. *struct* — un agrégat de données

En C++, il existe une structure de données permettant de regrouper plusieurs données en un seul bloc. Dans sa grande originalité, elle s'appelle... *structure*.

*Note > À partir de maintenant, nous emploierons le terme *structure* pour désigner l'agrégat que nous allons introduire et l'expression *structure de données* pour désigner les formes complexes de données de manière générale.*

Les structures sont très utiles pour regrouper des données qui ont un lien entre elles et travailler dessus de manière uniforme, et non sur chaque donnée séparément. Voyons un exemple concret par le biais d'un petit exercice.

Stockage d'informations personnelles¹

Essayez de créer un programme qui demande des informations sur une personne et les stocke dans un fichier `Prenom.Nom.csv` sous le format suivant :

```
nom, prenom, sexe, age
```

En sortie, la console devrait ressembler à quelque chose comme ci-dessous.

¹En total accord avec le RGPD [Règlement Européen pour la Protection des Données].

```
Nom ? Lagrume
Prénom ? Clem
Sexe ? Fruit
Age ? 4
```

```
Ces données seront enregistrées dans le fichier Clem.Lagrume.csv.
```

Le programme va ensuite enregistrer ces informations dans un fichier `Clem.Lagrume.csv`, organisé comme suit.

```
Lagrume,Clem,Fruit,4
```

 [Voir la solution](#)

Analyse de la méthode

Notre méthode marche, mais on peut remarquer des pistes d'amélioration :

- Les fonctions `demandeur_infos` et `enregistrer_infos` prennent beaucoup de paramètres. Mais, à la limite, c'est du détail.
- Plus grave, si on ajoute une information sur une personne, il faudra modifier les prototypes de toutes nos fonctions ! Cela porte gravement atteinte à l'évolutivité de nos programmes.
- Les données que l'on manipule sont liées et pourraient s'utiliser en tant qu'ensemble.

Introduction aux structures

La solution est justement d'utiliser une structure. Cela va nous permettre de créer un type (que l'on va par exemple appeler `InformationsPersonnelles`) regroupant l'intégralité des informations que l'on souhaite enregistrer. La syntaxe de création est la suivante.

```
struct identifiant
{
    *membres*
};
```

- L'identifiant de la structure suit les mêmes [règles de nommage](#) que les identifiants de variables et de fonctions.

- Nous suivrons une convention appelée CamelCase : on colle les mots en commençant chaque mot par une majuscule. Il en existe d'autres, mais nous utiliserons celle-là car elle permet facilement de distinguer les variables et les types.
- La définition d'une structure doit se *terminer par un point virgule*. Il est fréquent de l'oublier.

Par exemple, une structure représentant les données personnelles d'une personne (dont on pourrait se servir dans l'exercice précédent) peut être définie comme suit.

```
struct InformationsPersonnelles
{
    std::string prenom;
    std::string nom;
    std::string sexe;
    int age;
};
```

Initialisation

Pour initialiser une structure, nous disposons de plusieurs façons de faire, qui ne vont pas vous surprendre puisque nous les connaissons déjà pour les variables classiques.

```
// Initialisation par défaut.
InformationsPersonnelles infos {};

// Initialisation à des valeurs choisies.
InformationsPersonnelles infos { "Clem", "Lagrum", "Fruit", 4 };

// Autre syntaxe, utilisant le signe = comme en C.
InformationsPersonnelles infos = { "Clem", "Lagrum", "Fruit", 4 };
```

Notez que, lorsqu'on initialise avec des valeurs choisies, il faut les donner dans l'ordre dans lequel elles sont définies dans la structure. On ne peut pas donner un entier comme première valeur si celle-ci est déclarée dans la définition comme étant une chaîne de caractères.

Voici les membres

Pour manipuler un membre, c'est-à-dire une variable appartenant à la structure, il suffit d'utiliser la syntaxe `structure.membre`, qui fonctionne tant en lecture qu'en écriture.

```
#include <iostream>
#include <string>
```



```

struct InformationsPersonnelles
{
    std::string prenom;
    std::string nom;
    std::string sexe;
    int age;
};

int main()
{
    InformationsPersonnelles info_clem { "Clem", "Lagrum", "Fruit", 5 };

    // Modification.
    info_clem.age = 4;
    // Utilisation en lecture.
    std::cout << "Je m'appelle " << info_clem.prenom
        << " " << info_clem.nom
        << " et j'ai " << info_clem.age
        << " ans." << std::endl;

    return 0;
}

```

```
Je m'appelle Clem Lagrum et j'ai 4 ans.
```

Bien entendu, comme toujours, la modification d'un membre n'est possible que si la variable n'est pas déclarée comme étant `const`.

Et maintenant, structurons !

Sans plus attendre, reprenons le code de l'[exercice précédent](#) et utilisons une structure au lieu de nous trimballer plein de variables.

✧ [Voir la solution](#)

Et voilà ! Le code est plus lisible et plus évolutif. Vous pouvez essayer d'ajouter une information à enregistrer, par exemple la couleur préférée ou la date de naissance, et vous verrez que vous aurez uniquement des ajouts à faire. C'est un critère important d'évolutivité du code. En particulier, les prototypes des fonctions ne changeront pas, seules les implémentations changeront. Du point de vue de l'utilisation, c'est transparent, on ne voit rien de changé, tout le code les utilisant n'aura donc pas à être réécrit.

Astuce > Maintenant qu'on peut directement renvoyer le résultat de `demander_infos`, on n'est plus obligé de prendre en paramètre des références vers les variables à modifier. Cela permet donc d'écrire, non sans élégance : `auto infos = demander_infos();` .

14.2. `std::tuple` — une collection hétérogène

Les structures ne sont pas le seul moyen de représenter un ensemble de données de types variés. Il existe un autre outil nommé `std::tuple`, qui est une collection hétérogène de taille fixée. Hétérogène, car on peut stocker des informations de types différents (comme `struct`), mais dont la taille ne varie pas (comme `std::array`).

Note > Pour utiliser les tuples, il faut inclure le fichier `<tuple>`.

Déclaration

Pour préciser les types des différents éléments, on utilise une syntaxe désormais familière. Par exemple, si on veut créer un tuple pour contenir des informations personnelles, comme dans l'exercice précédent, on procède comme suit.

```
using namespace std::literals;
std::tuple { "Clem"s, "Lagrumes", "Fruit"s, 4 };
```

Attention > N'oubliez pas le s. Souvenez-vous qu'avec les littéraux, pour que le type déduit soit bien `std::string`, vous devez ajouter ce s après chaque littéral (ainsi que la ligne `using namespace std::literals`;) C'est ici obligatoire pour créer un `std::tuple` contenant des `std::string` et non des chaînes héritées du C.

On a aussi une fonction `std::make_tuple` permettant de créer un tuple en déduisant les types.

```
using namespace std::literals;
auto infos = std::make_tuple("Clem"s, "Lagrumes", "Fruit"s, 4);
```

Accès aux éléments

Comme pour toute collection, il faut aussi être capable d'accéder aux éléments. Pour cela, on utilise la fonction `std::get`. Celle-ci prend entre chevrons l'indice de l'élément auquel on veut accéder.

```
#include <iostream>
#include <string>
#include <tuple>

int main()
{
    using namespace std::literals;
    auto informations = std::make_tuple("Clem"s, "Lagrumes", "Fruit"s, 4);
```

```

// Accès en lecture.
std::cout << "Prénom : "
  << std::get<0>(informations) << std::endl;
std::cout << "Âge : "
  << std::get<3>(informations) << std::endl;

// Accès en écriture.
std::get<3>(informations) += 1; // Clem a maintenant 5 ans !
std::cout << "Âge maintenant : "
  << std::get<3>(informations) << std::endl;

return 0;
}

```

```

Prénom : Clem
Âge : 4
Âge maintenant : 5

```

Dans le cas où tous les éléments d'un `std::tuple` ont un type unique, et seulement dans ce cas, on peut aussi y accéder en donnant à `std::get` le type de l'élément désiré.

```

#include <iostream>
#include <string>
#include <tuple>

int main()
{
    using namespace std::literals;
    // Prénom et salaire.
    auto informations = std::make_tuple("Clem"s, 50000);

    std::cout << "Prénom : "
      << std::get<std::string>(informations)
      << std::endl;
    std::cout << "Salaire : "
      << std::get<int>(informations) << "€"
      << std::endl;

    return 0;
}

```

```

Prénom : Clem
Salaire : 50000€

```

Si vous essayez d'utiliser cette syntaxe alors qu'il y a plusieurs éléments de même type, le code ne compilera tout simplement pas.

```

#include <iostream>
#include <string>
#include <tuple>

int main()
{
    using namespace std::literals;
    auto identite = std::make_tuple("Clem"s, "Lagrumes"s);
    // Ne compile pas car il y a deux
    // éléments de même type. Comment choisir ?
    std::get<std::string>(identite);

    return 0;
}

```

`std::tuple` et fonctions

On peut tout à fait créer une fonction qui renvoie un `std::tuple`. C'est même une astuce qu'on utilise pour contourner la limite qu'impose C++ de ne pouvoir retourner qu'une seule valeur à la fois. Il suffit de déclarer que la fonction renvoie un `std::tuple` et de préciser, entre chevrons, les types de ses éléments.

Notez, dans l'exemple suivant, la syntaxe alternative et concise pour renvoyer un `std::tuple`, introduite depuis C++17.

```

// Si on compile avec C++20, la valeur pi est fournie dans la
// bibliothèque standard avec le fichier d'en-tête <numbers>.
// Pour utiliser la valeur pi, il faut écrire une ligne spéciale,
// using namespace std::numbers;
// Ensuite on peut utiliser pi, ainsi que d'autres constantes
// mathématiques : std::sqrt(pi) + std::pow(e, ln2), etc.

// Sinon, on peut s'appuyer sur la propriété que PI = acos(-1).
// double const pi { std::acos(-1) };

#include <cmath>
#include <iostream>
#include <string>
#include <tuple>

std::tuple<int, std::string> f()
{
    using namespace std::literals;
    // On retourne un std::tuple qui contient un entier
    // et une chaîne de caractères.
    return std::make_tuple(20, "Clem"s);
}

std::tuple<double, double> g(double angle)

```

```

{
    // On retourne un std::tuple qui contient deux flottants.
    // Notez la nouvelle syntaxe à base d'accolades.
    return { std::cos(angle), std::sin(angle) };
}

int main()
{
    std::tuple resultats_scolaires = f();
    std::cout << "Tu t'appelles "
        << std::get<std::string>(resultats_scolaires)
        << " et tu as obtenu "
        << std::get<int>(resultats_scolaires)
        << " / 20." << std::endl;

    double const pi { std::acos(-1) };
    std::tuple calculs = g(pi / 2.);
    std::cout << "Voici le cosinus de PI / 2 : "
        << std::get<0>(calculs) << std::endl;
    std::cout << "Voici le sinus de PI / 2 : "
        << std::get<1>(calculs) << std::endl;

    return 0;
}

```

```

Tu t'appelles Clem et tu as obtenu 20 / 20.
Voici le cosinus de PI / 2 : 0
Voici le sinus de PI / 2 : 1

```

Équations horaires

Vous voulez un exemple concret d'utilisation des `std::tuple` ? Vous aimez la physique ? Vous allez être content ! Nous allons coder une fonction retournant les coordonnées $(x; y)$ d'un objet lancé à une vitesse v_0 et à un angle α , en fonction du temps t . Sa position est obtenue à l'aide de deux équations.

$$\begin{cases} x(t) = v_0 \cdot \cos \alpha \cdot t \\ y(t) = -g \cdot \frac{t^2}{2} + v_0 \cdot \sin \alpha \cdot t \end{cases}$$

Note > C'est ce qu'on appelle l'équation horaire de l'objet. Ne vous inquiétez pas si vous ne connaissez pas, nous n'allons pas aller plus loin que cela. Et pour ceux que cela intéresse, vous pouvez consulter [ce cours de physique écrit par un ancien élève de Télécom ParisTech](#).

Si on traduit tout ça en C++, on obtient quelque chose de ce style.

```

#include <cmath>
#include <iostream>
#include <string>
#include <tuple>

std::tuple<double, double> equations_mouvement_temps(double
vitesse_initiale, double angle, double t)
{
    // Accélération sur Terre.
    double const g { 9.80665 };
    double const x { vitesse_initiale * std::cos(angle) * t };
    double const y {
        (-g * (t * t) / 2.) + (vitesse_initiale * std::sin(angle) * t)
    };

    return { x, y };
}

int main()
{
    // Vitesse de 8 mètres par seconde.
    double const vitesse_initiale { 8.0 };
    // On tire à un angle de  $\pi / 4$ , soit 45°.
    double const pi { std::acos(-1) };
    double const angle { pi / 4 };

    // On simule sur 10 secondes
    // avec un calcul toutes les 100 millisecondes.
    for (double t { 0.0 }; t < 10; t += 0.1)
    {
        std::tuple coordonnees {
            equations_mouvement_temps(vitesse_initiale, angle, t)
        };

        if (std::get<1>(coordonnees) < 0)
        {
            // On ne continue plus la simulation
            // dès que y est au niveau du sol.
            break;
        }

        std::cout << "À l'instant t = "
            << t << ", on a : " << std::endl;
        std::cout << "x = "
            << std::get<0>(coordonnees) << std::endl;
        std::cout << "y = "
            << std::get<1>(coordonnees) << std::endl
            << std::endl;
    }

    return 0;
}

```



Pourquoi s'encombrer des tuples ? Que dois-je choisir ?

Il est vrai que `std::tuple` semble présenter plus d'inconvénients qu'autre chose. Le code n'est pas très lisible avec tous ces `std::get`. De plus, contrairement à une structure dont l'accès à un élément est clair (comme `informations.prenom`), il ne saute pas aux yeux que `std::get<1>(informations)` représente potentiellement un prénom.

En fait, chaque solution a ses avantages et ses inconvénients, et on va donc choisir en conséquence selon les situations.

- Le tuple permet de créer rapidement une collection hétérogène sans avoir à définir un nouveau type. Il est particulièrement utile lorsque l'on a besoin d'une collection hétérogène uniquement pour une portée locale, dans le corps d'une fonction par exemple. Par contre, la syntaxe d'utilisation, notamment pour l'accès aux éléments, est peu lisible.
- La structure permet une plus grande expressivité, notamment par le fait que les éléments ont des noms. Elle est particulièrement adaptée pour une utilisation sur une portée plus globale.

En fait, le tuple est à la structure ce que la lambda est à la fonction. C++ vous laisse libre, c'est vous qui choisirez en fonction des cas.

14.3. `std::unordered_map` — une table associative

Depuis le début de ce livre, nous avons vu les conteneurs `std::vector`, `std::array` et `std::string`. Nous avons également, très brièvement, survolé `std::list`. Maintenant, nous aimerions vous en présenter d'autres, qui sont un peu différents, mais très utiles.

Le premier conteneur se nomme `std::unordered_map`. Ce nom peut être traduit par *tableau associatif*, *table associative* ou *carte associative*, car son principe est d'associer une valeur à une clé. Ce principe, nous le connaissons déjà avec `std::vector`, qui associe une valeur quelconque à une clé entière. Ainsi, quand vous faites `tableau[4]`, vous utilisez une clé de type entier et qui vaut 4. Grâce à `std::unordered_map`, nous avons le choix du type de la clé.

Note > Pour utiliser ce nouveau conteneur, pensez à inclure le fichier `<unordered_map>`.

Un dictionnaire de langue Zestienne²

Imaginez que vous vouliez créer un dictionnaire. Comme dans un dictionnaire classique, nous voulons associer un mot à un ensemble de phrases. En C++, on peut traduire ça par une `std::unordered_map` dont la clé est de type `std::string`, ainsi que la valeur. On précise le type de la clé et de la valeur entre chevrons, car notre conteneur utilise les templates.

```
#include <iostream>
#include <string>
#include <unordered_map>

int main()
{
    using namespace std::literals;

    // Notez que chaque élément ajouté est lui-même entre accolades {}.
    std::unordered_map<std::string, std::string> dictionnaire
    {
        // Ajout d'une clé 'Clem' avec sa valeur 'La mascotte du site
        // Zeste de Savoir, toute gentille et tout mignonne.'
        { "Clem"s, "La mascotte du site Zeste de Savoir, toute gentille et
        toute mignonne."s },
        { "mehdidou99"s, "Un des auteurs du tutoriel C++."s },
        { "informaticienzero"s, "Un des auteurs du tutoriel C++."s },
        { "Taurre"s, "Un super valideur !"s },
        { "Arius"s, "Un membre super sympa mais qui mord."s },
        { "Gants de vaisselle"s, "Objets présents sur le site et dont
        personne ne sait pourquoi."s }
    };

    return 0;
}
```

Pour afficher le contenu de notre conteneur, on va récupérer les éléments un par un. On peut faire ça avec une simple boucle `for`, comme on en a l'habitude. Chaque élément est un `std::pair<T, U>`, où `T` est de même type que la clé et `U` de même type que la valeur. Ce type n'est rien d'autre qu'un cas particulier de tuple à deux valeurs uniquement.

Ce type fournit deux propriétés qui nous intéressent, `first` et `second`, renvoyant respectivement la clé et la valeur récupérée dans la `std::unordered_map`. On a donc maintenant toutes les informations pour écrire la boucle et afficher les éléments de notre tableau associatif.

```
#include <iostream>
```

²Petit hommage à la communauté de Zeste de Savoir, qui a hébergé et contribué par sa relecture à la première version de ce livre.


```

#include <string>
#include <unordered_map>

int main()
{
    using namespace std::literals;

    std::unordered_map<std::string, std::string> dictionnaire
    {
        { "Clem"s, "La mascotte du site Zeste de Savoir, toute gentille et
toute mignonne."s },
        { "mehdidou99"s, "Un des auteurs du tutoriel C++."s },
        { "informaticienzero"s, "Un des auteurs du tutoriel C++."s },
        { "Taurre"s, "Un super validateur !"s },
        { "Arius"s, "Un membre super sympa mais qui mord."s },
        { "Gants de vaisselle"s, "Objets présents sur le site et dont
personne ne sait pourquoi."s }
    };

    for (auto const & paire : dictionnaire)
    {
        std::cout << "Clé : " << paire.first << std::endl;
        std::cout << "Valeur : " << paire.second << std::endl << std::endl;
    }

    return 0;
}

```

```

Clé : mehdidou99
Valeur : Un des auteurs du tutoriel C++.

Clé : Clem
Valeur : La mascotte du site Zeste de Savoir, toute gentille et toute
mignonne.

Clé : informaticienzero
Valeur : Un des auteurs du tutoriel C++.

Clé : Taurre
Valeur : Un super validateur !

Clé : Arius
Valeur : Un membre super sympa mais qui mord.

Clé : Gants de vaisselle
Valeur : Objets présents sur le site et dont personne ne sait pourquoi.

```

Note > Nous vous avons expliqué ce qu'était `std::pair`, mais nous avons utilisé `auto` à la place. Le code est ainsi plus court et plus lisible. Vous êtes néanmoins libre d'écrire explicitement `std::pair<std::string, std::string>` si vous le souhaitez.

Toujours plus de clés

Bien entendu, si notre conteneur n'est pas déclaré comme `const`, il est possible de le modifier, lui ajouter ou lui supprimer des éléments. Voyez par vous-même plusieurs exemples ci-dessous.

```
#include <iostream>
#include <string>
#include <unordered_map>

int main()
{
    using namespace std::literals;

    std::unordered_map<std::string, std::string> dictionnaire
    {
        { "Clem"s, "La mascotte du site Zeste de Savoir, toute gentille et
toute mignonne."s },
        { "mehdidou99"s, "Un des auteurs du tutoriel C++."s },
        { "informaticienzero"s, "Un des auteurs du tutoriel C++."s },
        { "Taurre"s, "Un super validateur !"s },
        { "Arius"s, "Un membre super sympa mais qui mord."s },
        { "Gants de vaisselle"s, "Objets présents sur le site et dont
personne ne sait pourquoi."s }
    };

    // Insertion avec la fonction 'insert'.
    dictionnaire.insert({ "C++"s, "Le langage que vous apprenez."s });

    // Plusieurs valeurs en même temps. Notez les doubles accolades {}.
    dictionnaire.insert({
        { "Pacman++"s, "C'est un des avatars de mehdidou99."s },
        { "Un truc"s, "Personne ne sait ce que c'est."s }
    });

    // Ajout d'une nouvelle paire avec les crochets.
    dictionnaire["nohar"] = "Un lama ancestral."s;

    // Utilisation des crochets pour modifier
    // la valeur associée à une clé existante.
    dictionnaire["Taurre"] = "Un validateur vraiment cool !"s;
    // Accès en lecture.
    std::cout << "Qui est Taurre ? "
        << dictionnaire["Taurre"]
        << std::endl << std::endl;

    // Suppression par clé.
    dictionnaire.erase("Un truc"s);

    for (auto const & paire : dictionnaire)
    {
        std::cout << "Clé : "
            << paire.first << std::endl;
```

```

    std::cout << "Valeur : "
    << paire.second << std::endl << std::endl;
}

return 0;
}

```

Qui est Taurre ? Un validateur vraiment cool !

Clé : mehdidou99

Valeur : Un des auteurs du tutoriel C++.

Clé : C++

Valeur : Le langage que vous apprenez.

Clé : Clem

Valeur : La mascotte du site Zeste de Savoir, toute gentille et toute mignonne.

Clé : informaticienzero

Valeur : Un des auteurs du tutoriel C++.

Clé : Taurre

Valeur : Un validateur vraiment cool !

Clé : Arius

Valeur : Un membre super sympa mais qui mord.

Clé : Pacman++

Valeur : C'est un des avatars de mehdidou99.

Clé : Gants de vaisselle

Valeur : Objets présents sur le site et dont personne ne sait pourquoi.

Clé : nohar

Valeur : Un lama ancestral.

Cette clé est unique !

Abordons maintenant un point important de `std::unordered_map`, mais que nous avons passé sous silence. Il s'agit de l'*unicité* de la clé. Il ne peut y avoir qu'une seule valeur associée à une seule clé. Dit autrement, on ne peut pas assigner plusieurs valeurs à une même clé.

Quand on ajoute une nouvelle paire, il ne se passe rien de visible dans le cas où la clé existe déjà. C'est un peu gênant pour signaler l'erreur. Heureusement, la fonction `insert` retourne un `std::pair` contenant deux informations qui nous intéressent :

- `first`, qui est un itérateur sur l'élément inséré si la clé n'existe pas au préalable, ou sur l'élément correspondant à la clé si elle existe déjà. Lui-même est un `std::pair`, donc on accède à la clé avec `first` et à la valeur avec `second` ;
- `second`, qui est un booléen indiquant si l'élément a été inséré (`true`) ou non (`false`).

```
#include <iostream>
#include <string>
#include <unordered_map>

int main()
{
    using namespace std::literals;

    std::unordered_map<std::string, double> constantes
    {
        { "pi"s, 3.14159 },
        { "e"s, 2.71828 }
    };

    auto paire_phi = constantes.insert({ "phi"s, 1.61803 });
    if (paire_phi.second)
    {
        std::cout << "La valeur de "
            << paire_phi.first->first
            << " a bien été ajoutée." << std::endl;
    }

    auto paire_pi = constantes.insert({ "pi"s, 3.0 });
    if (!paire_pi.second)
    {
        std::cout << "La clé "
            << paire_pi.first->first
            << " existe déjà et vaut "
            << paire_pi.first->second
            << "." << std::endl;
    }

    return 0;
}
```

```
La valeur de phi a bien été ajoutée.
La clé pi existe déjà et vaut 3.14159.
```

Dans le cas où vous souhaitez ajouter une valeur si elle n'existe pas, ou bien mettre à jour la valeur associée à une clé si celle-ci existe, utilisez la fonction `insert_or_assign`, qui fonctionne presque pareil. Elle renvoie une paire semblable, avec toujours le booléen signalant l'insertion (`true`) ou la mise à jour (`false`).

```

#include <iostream>
#include <string>
#include <unordered_map>

int main()
{
    using namespace std::literals;

    std::unordered_map<std::string, double> constantes
    {
        { "pi"s, 3.14159 },
        { "e"s, 2.0 }
    };

    auto paire_e = constantes.insert_or_assign("e"s, 2.71828);
    if (paire_e.second)
    {
        std::cout << "La valeur a bien été ajoutée.\n";
    }
    else
    {
        std::cout << "La valeur a bien été mise à jour.\n";
    }

    for (auto const & paire : constantes)
    {
        std::cout << "La constante "
                    << paire.first << " vaut "
                    << paire.second << ".\n";
    }

    return 0;
}

```

```

La valeur a bien été mise à jour.
La constante pi vaut 3.14159.
La constante e vaut 2.71828.

```

Cherchez un élément

Comme l'accès à un élément par les crochets permet également d'ajouter une nouvelle paire clé/valeur, l'utiliser s'avère parfois délicat. Voyez par vous-même l'exemple suivant.

```

#include <iostream>
#include <string>
#include <unordered_map>

int main()
{

```

```

using namespace std::literals;

std::unordered_map<std::string, std::string> membres
{
    { "mehdidou99"s, "Un auteur du cours C++."s },
    { "informaticienzero"s, "Un auteur du cours C++."s }
};

// Notez l'accent.
std::string const pseudo { "informaticienzéro" };
// ah, ça ne va pas afficher ce que vous attendiez.
std::cout << "Qui est informaticienzero ? "
    << membres[pseudo] << std::endl;

return 0;
}

```

Qui est informaticienzero ?

Il existe heureusement une fonction `find`, qui attend une clé et qui renvoie un itérateur sur l'élément concerné si la clé existe, ou bien un itérateur sur la fin de la collection sinon.

```

#include <iostream>
#include <string>
#include <unordered_map>

int main()
{
    using namespace std::literals;

    std::unordered_map<std::string, std::string> membres
    {
        { "mehdidou99"s, "Un auteur du cours C++."s },
        { "informaticienzero"s, "Un auteur du cours C++."s }
    };

    // Notez l'accent.
    std::string const pseudo { "informaticienzéro" };
    if (membres.find(pseudo) != std::end(membres))
    {
        // On a trouvé le membre.
        std::cout << "Qui est informaticienzero ? "
            << membres[pseudo] << std::endl;
    }
    else
    {
        std::cout << "Ce membre n'existe pas.\n";
    }

    return 0;
}

```

Ce membre n'existe pas.

Un exemple concret

Grâce à ce que nous savons, nous pouvons faire un petit programme qui compte les occurrences de chaque mot au sein d'une phrase. Il suffit d'avoir un tableau associatif avec `std::string` pour clé et `int` pour valeur.

```
#include <iostream>
#include <sstream>
#include <string>
#include <unordered_map>

void occurrences_mots(std::string const & phrase)
{
    std::istringstream iss { phrase };
    std::string mot {};

    std::unordered_map<std::string, int> occurrences {};
    while (iss >> mot)
    {
        ++occurrences[mot];
    }

    for (auto const & paire : occurrences)
    {
        std::cout << "Le mot '" << paire.first
                    << "' apparait " << paire.second
                    << " fois." << std::endl;
    }
}

int main()
{
    occurrences_mots("Bonjour, ceci est un test avec une phrase qui est
longue et est une phrase en français.");
    return 0;
}
```

```
Le mot 'Bonjour,' apparait 1 fois.
Le mot 'ceci' apparait 1 fois.
Le mot 'est' apparait 3 fois.
Le mot 'un' apparait 1 fois.
Le mot 'une' apparait 2 fois.
Le mot 'test' apparait 1 fois.
Le mot 'qui' apparait 1 fois.
Le mot 'avec' apparait 1 fois.
Le mot 'français.' apparait 1 fois.
Le mot 'phrase' apparait 2 fois.
Le mot 'longue' apparait 1 fois.
```

```
Le mot 'et' apparaît 1 fois.
Le mot 'en' apparaît 1 fois.
```

14.4. `std::unordered_set` — représenter un ensemble

Le deuxième conteneur se nomme `std::unordered_set`. C'est la représentation en C++ d'un ensemble au sens mathématique du terme, c'est-à-dire un ensemble d'éléments du même type, où chacun n'est présent qu'une seule et unique fois. C'est un peu comme une table associative dans laquelle il n'y aurait que des clés et pas de valeurs.

Note > Pour utiliser ce conteneur, il faut inclure le fichier d'en-tête `<unordered_set>`.

Inscription sur Zeste de Savoir

Sur Zeste de Savoir³, vous pouvez vous inscrire, mais il faut que votre pseudo soit unique. Vous ne pouvez donc pas réutiliser le pseudo de quelqu'un d'autre. On pourrait donc imaginer stocker les pseudos dans un `std::unordered_set`. Vous allez voir que l'utilisation de ce conteneur ressemble fortement à celle de `std::unordered_map`.

```
#include <iomanip>
#include <iostream>
#include <string>
#include <unordered_set>

void ajouter_membre(std::unordered_set<std::string> & membres,
    std::string const & pseudo)
{
    auto it = membres.insert(pseudo);
    if (it.second)
    {
        std::cout << "Bienvenu sur Zeste de Savoir, "
            << pseudo << "." << std::endl;
    }
    else
    {
        std::cout << "Désolé, le pseudo "
            << std::quoted(pseudo)
            << " est déjà pris." << std::endl;
    }
}

int main()
{
    std::unordered_set<std::string> pseudos {
        "mehdidou99",
```

³Oui, nous les aimons beaucoup 😊.


```

    "informaticienzero"
};

// Ajouts.
pseudos.insert("Dwayn");
pseudos.insert("artragis");
pseudos.insert("Taurre");

ajouter_membre(pseudos, "gbdivers");
ajouter_membre(pseudos, "informaticienzero");

// Suppression.
pseudos.erase("Dwayn");

// Parcours for.
for (auto const & cle : pseudos)
{
    std::cout << cle << std::endl;
}

// Recherche de membres.
if (pseudos.find("Taurre") != std::end(pseudos))
{
    std::cout << "Taurre est un membre." << std::endl;
}

if (pseudos.find("Gabbro") == std::end(pseudos))
{
    std::cout << "Le pseudo "
        << std::quoted("Gabbro")
        << " est libre.\n";
}

return 0;
}

```

Comme vous le voyez, l'utilisation est très proche. La différence se trouve simplement dans l'absence de valeurs liées aux clés. Pour choisir entre les deux conteneurs, il faut simplement réfléchir à ce que vous voulez stocker. Un ensemble de clés uniques ? Choisissez `std::unordered_set`. Si vous devez associer à ces clés des valeurs, alors partez sur `std::unordered_map`.

14.5. Un peu d'ordre, voyons !

Ces deux conteneurs sont très utiles mais, comme vous l'avez remarqué, ils ne sont pas ordonnés. Si, par exemple, vous avez un `std::unordered_set` de chaînes de caractères, vous verrez qu'elles ne sont pas affichées dans l'ordre alphabétique.

```

#include <iostream>
#include <string>

```

```
#include <unordered_set>

int main()
{
    std::unordered_set<std::string> mots {
        "Archive",
        "Base",
        "Conduite",
        "Développement",
        "Entrain"
    };

    for (auto const & mot : mots)
    {
        std::cout << mot << std::endl;
    }

    return 0;
}
```

```
Développement
Archive
Entrain
Conduite
Base
```

En interne, les clés sont ordonnées et triées, mais selon leur hash. Non, nous n'avons pas fait de faute de français. Un *hash* est le résultat d'une [fonction de hachage](#). En très bref, il s'agit d'une fonction qui transforme une donnée quelconque en une donnée de taille fixée. Pour notre conteneur, cela signifie qu'il ne va plus travailler avec des chaînes de caractères ou autres clés potentiellement grosses, mais avec une version de taille fixe et plus petite, ce qui est beaucoup plus rapide.

Il existe une version alternative des conteneurs `std::unordered_map` et `std::unordered_set`, qui n'ordonne pas les éléments selon le hash des clés mais directement avec les clés : `std::map` et `std::set`.

Note > Pour les utiliser, il faut respectivement inclure les fichiers `<map>` et `<set>`.

Dans l'exemple précédent, si on remplace `std::unordered_set` par sa version triée par clé, on obtient bien un affichage par ordre alphabétique.

```
#include <iostream>
#include <string>
#include <set>

int main()
{
```

```

std::set<std::string> const mots {
    "Archive",
    "Base",
    "Conduite",
    "Développement",
    "Entrain"
};

for (auto const & mot : mots)
{
    std::cout << mot << std::endl;
}

return 0;
}

```

```

Archive
Base
Conduite
Développement
Entrain

```



Du coup, la seule différence, c'est une question d'affichage et d'ordre ?

La structure interne et les algorithmes employés pour ajouter, supprimer ou chercher des éléments ne sont pas les mêmes. De manière générale, les versions non ordonnées sont plus rapides d'utilisation et ce sont celles qui sont utilisées dans 90% des cas. Les 10% restants se répartissent entre deux situations :

- on veut conserver l'ordre des clés ;
- on a vraiment énormément de données (de l'ordre du million d'éléments), auquel cas la version non ordonnée est moins rapide que la version ordonnée, de part leur structure interne.

Leur utilisation est très semblable à leurs cousins non ordonnés, vous n'aurez donc aucun mal à les manipuler.

14.6. Une longue énumération

Terminons ce chapitre avec les énumérations. Elles permettent de créer des listes fixes de valeurs fortement typées. Fixes car le nombre d'éléments est fixé et immuable. Fortement typées car les valeurs sont d'un type bien précis et défini.

On s'en sert typiquement pour *limiter le champ des possibilités* d'un type. Ainsi, au lieu de stocker les jours de la semaine sous forme de `std::string`, ce qui n'offre pas de protection si l'utilisateur rentre n'importe quoi, on va utiliser une énumération. De la sorte, celui-ci n'a d'autre choix que d'utiliser une des valeurs que nous avons nous-mêmes définies. C'est beaucoup plus souple que de devoir faire plein de vérifications sur une chaîne de caractères.

Pour déclarer une énumération, c'est très simple.

```
enum class Nom_enumeration
{
    Valeur1,
    Valeur2
    // Etc.
};
```

Concrètement appliqué à notre exemple des jours de la semaine, on obtient le code suivant.

```
enum class Jour
{
    Lundi,
    Mardi,
    Mercredi,
    Jeudi,
    Vendredi,
    Samedi,
    Dimanche
};
```

Pour utiliser l'une des valeurs de l'énumération, c'est aussi très simple. On fait `Nom_enumeration::Valeur` (notez les deux-points `::`, comme dans `std::`). Examinez le code suivant qui montre un cas d'utilisation.

```
#include <iostream>

enum class Jour
{
    Lundi,
    Mardi,
    Mercredi,
    Jeudi,
    Vendredi,
    Samedi,
    Dimanche
};

void message(Jour jour_semaine)
{
```

```

    if (jour_semaine == Jour::Samedi || jour_semaine == Jour::Dimanche)
    {
        std::cout << "Bon week-end !" << std::endl;
    }
    else
    {
        std::cout << "Courage !" << std::endl;
    }
}

int main()
{
    Jour const aujourd'hui { Jour::Mardi };
    message(aujourd'hui);
    return 0;
}

```

Ainsi, si vous avez une liste de choix de valeurs qui est limitée, une énumération peut être la bonne solution. Les mois de l'année, les couleurs d'un jeu de cartes, les directions d'une boussole sont de bons cas d'utilisation. En plus de moins se casser la tête pour la vérification des valeurs, vous créez un nouveau type, clairement défini ; en effet, `JourSemaine` est bien plus parlant qu'une `std::string`. Le code est rendu plus clair et c'est une très bonne pratique.

Note > Une énumération est très légère car elle utilise en interne des entiers. C'est pour cette raison qu'il est inutile de la passer par référence.

14.7. Récapitulons !

- Les structures sont des agrégats de données permettant de lier et de manipuler plus aisément des données liées entre elles.
- Les tuples sont des collections hétérogènes et fixées, qui permettent de lier des données entre elles sans créer pour autant de nouveau type.
- Grâce aux tuples et aux structures, nous sommes en mesure de retourner plusieurs variables d'un coup.
- Les `std::unordered_map` sont des tables associatives, qui permettent d'associer une clé de type quelconque à une valeur de type quelconque.
- Les `std::unordered_set` sont des ensembles au sens mathématique du terme et permettent de stocker un ensemble de clés uniques.
- Il existe des versions triées par clé de ces conteneurs, `std::map` et `std::set`, mais qui sont beaucoup plus lentes.
- Une énumération permet de définir un type avec un ensemble de valeurs fixe et clairement défini.

SOLUTIONS

section Stockage d'informations personnelles

```
#include <fstream>
#include <iostream>
#include <limits>
#include <string>

template<typename T>
void entree_securisee(T & variable)
{
    while (!(std::cin >> variable))
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();

        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
}

template <typename T, typename Predicat>
void entree_securisee(T & variable, Predicat predicat)
{
    while (!(std::cin >> variable) || !predicat(variable))
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();

        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
}

void demander_infos(std::string & nom, std::string & prenom,
std::string & sexe, int & age)
{
    std::cout << "Nom ? ";
    entree_securisee(nom);
    std::cout << "Prénom ? ";
    entree_securisee(prenom);
    std::cout << "Sexe ? ";
    entree_securisee(sexe);
    std::cout << "Age ? ";
    entree_securisee(age, [](int & age) { return age >= 0; });
}

std::string enregistrer_infos(std::string const & nom,
std::string const & prenom, std::string const & sexe, int age)
{
    std::string nom_fichier { prenom + "." + nom + ".csv" };
    std::ofstream fichier { nom_fichier };
```

SOLUTIONS

```

    fichier << nom << ',' << prenom << ',' << sexe << ',' << age;

    return nom_fichier;
}

int main()
{
    std::string nom { "" };
    std::string prenom { "" };
    std::string sexe { "" };
    int age { 0 };

    demander_infos(nom, prenom, sexe, age);
    auto nom_fichier = enregistrer_infos(nom, prenom, sexe, age);
    std::cout << std::endl
        << "Ces données seront enregistrées dans le fichier "
        << nom_fichier << std::endl;

    return 0;
}

```

section Et maintenant, structurons !

```

#include <fstream>
#include <iostream>
#include <limits>
#include <string>

template<typename T>
void entree_securisee(T & variable)
{
    while (!(std::cin >> variable))
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();

        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
}

template <typename T, typename Predicat>
void entree_securisee(T & variable, Predicat predicat)
{
    while (!(std::cin >> variable) || !predicat(variable))
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();

        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
}

```

SOLUTIONS

```

}

struct InformationsPersonnelles
{
    std::string prenom;
    std::string nom;
    std::string sexe;
    int age;
};

InformationsPersonnelles demander_infos()
{
    InformationsPersonnelles infos {};
    std::cout << "Nom ? ";
    entree_securisee(infos.nom);
    std::cout << "Prénom ? ";
    entree_securisee(infos.prenom);
    std::cout << "Sexe ? ";
    entree_securisee(infos.sexe);
    std::cout << "Age ? ";
    entree_securisee(infos.age, [](int & age) { return age >= 0; });

    return infos;
}

std::string enregistrer_infos(InformationsPersonnelles const &
    infos)
{
    std::string nom_fichier { infos.prenom + "." + infos.nom + ".csv" };
    std::ofstream fichier { nom_fichier };

    fichier << infos.nom << ',' << infos.prenom << ',' << infos.sexe
        << ',' << infos.age;

    return nom_fichier;
}

int main()
{
    auto infos = demander_infos();
    auto nom_fichier = enregistrer_infos(infos);
    std::cout << std::endl << "Ces données seront enregistrées dans
    le fichier " << nom_fichier << std::endl;

    return 0;
}

```


15

Reprenez-vous un peu de sucre syntaxique ?

Le sucre se trouve partout aujourd'hui. Dans les gâteaux, dans les biscuits, dans les boissons et même dans notre code ! Sauf que dans ce dernier cas, on parle de *sucre syntaxique*. Ce terme, inventé par [Peter J. Landin](#), un informaticien britannique, désigne des syntaxes plus lisibles ou plus faciles à écrire qu'on utilise pour se faciliter la vie.

Le but de ce chapitre va donc être d'utiliser et même d'écrire du sucre syntaxique.

15.1. Ce type est trop long !

Vous vous souvenez du [chapitre sur les flux](#) ? Nous y avons découvert comment traiter des chaînes de caractères comme des flux, avec les types `std::istream` et `std::ostream`. Mais il faut avouer que c'est assez long à écrire et fatigant à la longue.

Heureusement, en C++, on peut définir des alias de types, des synonymes, qui permettent d'éviter l'utilisation de types longs à écrire ou bien de types composés. Cela se fait très simplement, en utilisant le mot clé `using`.

```
// Désormais, écrire 'oss' dans le code revient
// à écrire 'std::ostream'.
using oss = std::ostream;

// Désormais, écrire 'ListeResultats' dans le code revient à écrire
// 'std::vector<std::tuple<std::string, int>>'.
using ListeResultats = std::vector<std::tuple<std::string, int>>;
```

Même la bibliothèque standard utilise le concept. Ainsi, `std::string` est lui-même l'alias d'un type `std::basic_string<char>`. Notez que c'est quand même beaucoup plus agréable à écrire.

15.2. Décomposons tout ça

Dans le [chapitre précédent](#), nous avons souligné la lourdeur et le manque d'expressivité de l'accès aux éléments d'un tuple. Il est vrai qu'une expression comme `std::get<2>(tuple)` ne nous donne guère d'information sur le sens de ce que nous manipulons. Heureusement, il existe des syntaxes alternatives, que nous allons voir.

En pleine décomposition

Une des nouveautés de C++17, ce sont les décompositions [en anglais *structured bindings*]. Ce concept est très simple. Son but est de déclarer et initialiser plusieurs variables d'un coup, depuis un tuple ou une structure. Et comme un code vaut mille mots, regardez par vous-même. Nous avons repris [l'un des exemples de tuples](#) du chapitre précédent, en l'adaptant.

```
#include <cmath>
#include <iostream>
#include <string>
#include <tuple>

std::tuple<int, std::string> f()
{
    using namespace std::literals;
    return { 20, "Clem"s };
}

std::tuple<double, double> g(double angle)
{
    return { std::cos(angle), std::sin(angle) };
}

int main()
{
    // AVANT
    // std::tuple resultats_scolaires = f();
    // std::cout << "Tu t'appelles "
    // << std::get<std::string>(resultats_scolaires)
    // << " et tu as obtenu "
    // << std::get<int>(resultats_scolaires)
    // << " / 20."
    // << std::endl;

    // Maintenant, on sait qu'on a affaire à un nom et un entier.
    auto[note, nom] = f();
    std::cout << "Tu t'appelles " << nom
              << " et tu as obtenu " << note << " / 20." << std::endl;

    double const pi { std::acos(-1) };

    // AVANT
```

```
// std::tuple calculs = g(pi / 2.);
// std::cout << "Voici le cosinus de PI / 2 : "
// << std::get<0>(calculs) << std::endl;
// std::cout << "Voici le sinus de PI / 2 : "
// << std::get<1>(calculs) << std::endl;

// Maintenant, on sait qu'on a affaire à deux valeurs mathématiques.
auto[cosinus, sinus] = g(pi / 4.);
std::cout << "Voici le cosinus de PI / 4 : "
<< cosinus << std::endl;
std::cout << "Voici le sinus de PI / 4 : "
<< sinus << std::endl;

return 0;
}
```

Notez comment le code s'en trouve simplifié et clarifié. Nous avons remplacé une ligne aussi peu claire que `std::get<0>(calculs)` par `cosinus`. D'un coup d'œil, vous voyez ce que fait la variable et ce que nous manipulons. Le code est plus lisible et donc de meilleure qualité. N'oubliez pas qu'un code est bien plus souvent lu qu'écrit.

On peut bien entendu utiliser des références si l'on veut que des modifications sur les variables se répercutent sur le tuple lié. L'exemple suivant est tiré [d'un billet de Florent Chapelle](#).

```
#include <cassert>
#include <iostream>
#include <string>
#include <tuple>

int main()
{
    using namespace std::literals;
    auto bookmark = std::make_tuple(
        "https://zestedesavoir.com"s,
        "Zeste de Savoir"s,
        30
    );

    auto&[url, titre, visites] = bookmark;
    ++visites;

    assert(std::get<2>(bookmark) == 31);
    return 0;
}
```

Enfin, ce que nous venons de voir est tout à fait applicable aux structures. L'exemple suivant, bien que bidon, vous le montre.

```
#include <iostream>
```

```

#include <string>
#include <tuple>

struct Personne
{
    std::string nom;
    std::string prenom;
    int age;
};

Personne f()
{
    return { "Lagrume", "Clem", 4 };
}

int main()
{
    auto[nom, prenom, age] = f();
    std::cout << "Voici " << prenom << " " << nom
        << " et elle a " << age << " ans." << std::endl;
    return 0;
}

```

```
Voici Clem Lagrume et elle a 4 ans.
```

Une décomposition peut déclarer exactement autant de variables que d'éléments composant la structure ou le tuple, ni plus, ni moins. Même si nous ne voulons récupérer que certaines variables, il faut tout de même créer des variables inutiles qui ne seront plus utilisées par la suite.

Et avant, on faisait comment ?

Les décompositions sont apparues en C++17, mais avant, il existait une autre façon de faire, qui existe toujours soit dit en passant. Il s'agit de `std::tie`. Le principe est identique, puisqu'on veut assigner à des variables les différents éléments d'un tuple ou d'une structure. Sauf qu'ici, les variables doivent être déclarées au préalable.

```

#include <cmath>
#include <iostream>
#include <tuple>

std::tuple<double, double, double> f(double angle)
{
    return { std::cos(angle), std::sin(angle), std::tan(angle) };
}

int main()

```

```

{
    // Création au préalable.
    double cosinus {};
    double sinus {};
    double tangente {};

    double const pi { std::acos(-1) };
    std::tie(cosinus, sinus, tangente) = f(pi);

    std::cout << "Cosinus de pi : " << cosinus << std::endl;
    std::cout << "Sinus de pi : " << sinus << std::endl;
    std::cout << "Tangente de pi : " << tangente << std::endl;

    return 0;
}

```

```

Cosinus de pi : -1
Sinus de pi : 0
Tangente de pi : 0

```

Cette méthode a comme inconvénient majeur que les variables doivent être déclarées à l'avance et donc qu'on ne peut pas profiter de l'inférence de type. Par contre, l'avantage qu'elle a sur la décomposition, c'est qu'elle permet d'ignorer certaines valeurs qui ne nous intéressent pas et que nous ne voulons pas récupérer, grâce à `std::ignore`.

```

#include <cmath>
#include <iostream>
#include <tuple>

std::tuple<double, double, double> f(double angle)
{
    return { std::cos(angle), std::sin(angle), std::tan(angle) };
}

int main()
{
    // Seul le cosinus m'intéresse.
    double cosinus {};

    double const pi { std::acos(-1) };
    std::tie(cosinus, std::ignore, std::ignore) = f(pi);

    std::cout << "Cosinus de pi : " << cosinus << std::endl;
    return 0;
}

```

Un autre avantage, c'est de pouvoir réutiliser une même variable plusieurs fois, alors que la décomposition C++17 ne nous le permet pas.

15.3. La surcharge d'opérateurs

Lorsque l'on crée un type, il arrive que l'on souhaite réaliser certaines opérations communes sur ce type, comme l'injection dans un flux (pour de l'affichage par exemple), la réalisation d'opérations arithmétiques (addition, soustraction, etc.) ou encore d'opérations de comparaison.

Le problème...

Imaginons que nous voulions créer un type pour manipuler des fractions. On aimerait pouvoir les additionner, les soustraire, etc., en bref, faire des opérations dessus. Sauf qu'on ne peut pas se contenter de faire un simple `+` ou `-`, comme c'est le cas avec des `int` ou des `double`.

```
#include <iostream>
#include <string>

struct Fraction
{
    int numérateur;
    int dénominateur;
};

int main()
{
    Fraction const un_demi { 1, 2 };
    Fraction const trois_quarts { 3, 4 };
    Fraction const resultat { un_demi + trois_quarts };

    std::cout << "1/2 + 3/4 font " << resultat.numérateur
               << "/" << resultat.dénominateur << std::endl;
    return 0;
}
```

```
[Visual Studio]
Erreur C2676 '+' binaire : 'const Fraction' ne définit pas cet
opérateur ou une conversion vers un type acceptable pour l'opérateur
prédéfini
Erreur (active) E0349 aucun opérateur "+" ne correspond à ces opérandes
```

```
-----

[GCC]
prog.cc: In function 'int main()':
prog.cc:14:39: error: no match for 'operator+' (operand types are 'const
Fraction' and 'const Fraction')
   14 | Fraction const resultat { un_demi + trois_quarts };
      |                               ~~~~~^~~~~~
```

```
-----
[Clang]
prog.cc:14:39: error: invalid operands to binary expression ('const
Fraction' and 'const Fraction')
    Fraction const resultat { un_demi + trois_quarts };
                                ~~~~~ ^ ~~~~~
1 error generated.
```

On doit alors écrire des fonctions pour faire ça. Mais si on les mélange, ça peut très vite devenir indigeste à lire.

```
#include <iostream>
#include <string>

struct Fraction
{
    int numerateur;
    int denominateur;
};

Fraction additionner(Fraction const & a, Fraction const & b)
{
    int numerateur {
        a.numerateur * b.denominateur + b.numerateur * a.denominateur
    };
    int denominateur { a.denominateur * b.denominateur };

    return { numerateur, denominateur };
}

Fraction soustraire(Fraction const & a, Fraction const & b)
{
    int numerateur {
        a.numerateur * b.denominateur - b.numerateur * a.denominateur
    };
    int denominateur { a.denominateur * b.denominateur };

    return { numerateur, denominateur };
}

int main()
{
    Fraction const un_demi { 1, 2 };
    Fraction const trois_quarts { 3, 4 };
    Fraction const deux_huitiemes { 2, 8 };

    // Arf, c'est dur à lire.
    Fraction const resultat { soustraire(additionner(un_demi,
    trois_quarts), deux_huitiemes) };
    std::cout << "1/2 + 3/4 - 2/8 font "
        << resultat.numerateur << "/"
        << resultat.denominateur << std::endl;
```

```
    return 0;
}
```

Sans parler de l’affichage et de bien d’autres choses encore. Il faut donc trouver une solution à cet énorme problème de lisibilité. L’idéal, ça serait de pouvoir utiliser les opérateurs avec nos types personnels. Et heureusement, C++ ne nous laisse pas seuls face à ce problème !

...et la solution

Dans le chapitre [introduisant les fonctions](#), nous avons vu qu’il était possible de surcharger une fonction pour l’utiliser avec des paramètres de types différents. Eh bien, bonne nouvelle, les opérateurs sont des fonctions comme les autres.

Autrement dit, tous les opérateurs, aussi bien arithmétiques (+, *, etc.), de comparaison (<, >, ==, etc.) ou de flux (<<, >>), sont appelables et surchargeables. Simplement, ce sont des fonctions avec une syntaxe spéciale faite pour rendre le code plus lisible.

Ainsi, on va pouvoir surcharger nos opérateurs pour nos types personnels et ainsi éliminer le problème d’expressivité du code qu’on a soulevé précédemment.

Mise en pratique

Les opérateurs arithmétiques

Nous allons aborder ici les opérateurs +, -, * et /, qui nous permettront de réaliser, respectivement, des additions, des soustractions, des multiplications et des divisions de fractions. Voyons déjà à quoi ressemblent, de manière générale, les opérateurs en question.

```
T operator+(T const & lhs, T const & rhs);
T operator-(T const & lhs, T const & rhs);
T operator*(T const & lhs, T const & rhs);
T operator/(T const & lhs, T const & rhs);
```

Dans notre cas, ils se traduisent ainsi. Notez que les prototypes de + et de - ressemblent beaucoup aux fonctions [additionner](#) et [soustraire](#) que nous avons écrites.

```
#include <iostream>
#include <string>

struct Fraction
{
    int numérateur;
```



```

    int denominateur;
};

// Ça ressemble à quelque chose de connu, hein ?
Fraction operator+(Fraction const & a, Fraction const & b)
{
    int numerateur {
        a.numerateur * b.denominateur + b.numerateur * a.denominateur
    };
    int denominateur { a.denominateur * b.denominateur };

    return { numerateur, denominateur };
}

Fraction operator-(Fraction const & a, Fraction const & b)
{
    int numerateur {
        a.numerateur * b.denominateur - b.numerateur * a.denominateur
    };
    int denominateur { a.denominateur * b.denominateur };

    return { numerateur, denominateur };
}

int main()
{
    Fraction const un_demi { 1, 2 };
    Fraction const trois_quarts { 3, 4 };
    Fraction const deux_huitiemes { 2, 8 };

    Fraction const resultat { un_demi + trois_quarts - deux_huitiemes };
    std::cout << "1/2 + 3/4 - 2/8 font "
        << resultat.numerateur << "/"
        << resultat.denominateur << std::endl;

    return 0;
}

```

1/2 + 3/4 - 2/8 font 64/64

Note > Le résultat de 64/64 peut vous surprendre, mais il est tout à fait normal, parce qu'on ne simplifie pas la fraction.

Entraînez-vous en implémentant aussi la multiplication et la division. Le code est très proche de celui qu'on vient d'écrire.

Les opérateurs de comparaison

Continuons en essayant maintenant de comparer des fractions entre elles (leurs valeurs sont-elles égales, l'une est-elle supérieure à l'autre, etc.). C'est le même principe, les prototypes peuvent se déduire assez facilement.

```
bool operator==(T const& lhs, T const& rhs);
bool operator!=(T const& lhs, T const& rhs);
bool operator<(T const& lhs, T const& rhs);
bool operator>(T const& lhs, T const& rhs);
bool operator<=(T const& lhs, T const& rhs);
bool operator>=(T const& lhs, T const& rhs);
```

Et si vous êtes effrayé à l'idée d'implémenter autant d'opérateurs différents, rappelez-vous de la logique booléenne. Vous verrez, ça réduit énormément la quantité de code.

```
#include <iostream>
#include <string>

struct Fraction
{
    int numérateur;
    int dénominateur;
};

// On vérifie que a et b sont égaux.
bool operator==(Fraction const & a, Fraction const & b)
{
    Fraction const gauche {
        a.numérateur * b.dénominateur, a.dénominateur * b.dénominateur
    };
    Fraction const droite {
        b.numérateur * a.dénominateur, b.dénominateur * a.dénominateur
    };

    return gauche.numérateur == droite.numérateur
        && gauche.dénominateur == droite.dénominateur;
}

// Pour aller plus vite, il suffit de
// retourner la négation de la condition 'a == b'.
bool operator!=(Fraction const & a, Fraction const & b)
{
    return !(a == b);
}

// On vérifie si a est plus petit que b.
bool operator<(Fraction const & a, Fraction const & b)
{
    if (a.numérateur == b.numérateur)
```

```

{
    return a.denominateur > b.denominateur;
}
else if (a.denominateur == b.denominateur)
{
    return a.numérateur < b.numérateur;
}
else
{
    Fraction const gauche {
        a.numérateur * b.denominateur, a.denominateur * b.denominateur
    };
    Fraction const droite {
        b.numérateur * a.denominateur, b.denominateur * a.denominateur
    };

    return gauche.numérateur < droite.numérateur;
}
}

// Pour simplifier, il suffit de vérifier
// que a n'est ni inférieur, ni égal à b.
bool operator>(Fraction const & a, Fraction const & b)
{
    return !(a < b) && a != b;
}

// Pour simplifier, il suffit de vérifier
// que a est inférieur ou égal à b.
bool operator<=(Fraction const & a, Fraction const & b)
{
    return a < b || a == b;
}

// Pour simplifier, il suffit de vérifier
// que a est supérieur ou égal à b.
bool operator>=(Fraction const & a, Fraction const & b)
{
    return a > b || a == b;
}

int main()
{
    Fraction const trois_septieme { 3, 7 };
    Fraction const un_quart { 1, 4 };

    std::cout << std::boolalpha;
    std::cout << "Est-ce que 3/7 est plus petit que 1/4 ? "
        << (trois_septieme < un_quart) << std::endl;
    std::cout << "Est ce que 3/7 est plus grand que 1/4 ? "
        << (trois_septieme > un_quart) << std::endl;

    Fraction const neuf_vingtetunieme { 9, 21 };
    std::cout << "Est-ce que 3/7 et 9/21 sont égaux ? "
        << (trois_septieme == neuf_vingtetunieme) << std::endl;
}

```

```
    return 0;
}
```

```
Est-ce que 3/7 est plus petit que 1/4 ? false
Est ce que 3/7 est plus grand que 1/4 ? true
Est-ce que 3/7 et 9/21 sont égaux ? true
```

Aviez-vous pensé à l’astuce consistant à réutiliser le code déjà écrit pour simplifier celui à écrire ? Si oui, alors c’est que vous commencez à acquérir les bons réflexes de développeur.

Les opérateurs de manipulation des flux

Il nous reste le problème d’affichage de notre type `Fraction`. Pour l’instant, nous sommes obligés d’aller chercher le numérateur et le dénominateur nous-mêmes. Si notre type contient plein d’informations, aller toutes les récupérer devient vite pénible.

Les opérateurs de manipulation des flux `<<` et `>>` sont, fort heureusement, surchargeables. Voyez leur prototype.

```
std::ostream& operator<<(std::ostream & os, T const & t);
std::istream& operator>>(std::istream & is, T & t);
```

Ce qui peut surprendre, c’est le fait que l’on renvoie une référence sur un flux à chaque fois. En fait, c’est le flux que l’on reçoit en argument qui est renvoyé. Ceci a pour but d’enchaîner des appels à l’opérateur pour pouvoir écrire du code tel que celui-ci.

```
std::cout << t << std::endl;
```

Il sera interprété ensuite comme une suite d’appel à l’opérateur `<<` par le compilateur.

```
operator<<(operator<<(std::cout, t), std::endl);
```

***Note** > Notez que les types retournés sont `std::ostream` et `std::istream`. Ce sont des versions génériques de flux de sortie et d’entrée. Ainsi, en renvoyant ces types, on s’assure que notre type peut être utilisé avec des flux vers l’entrée et la sortie standards, vers des fichiers, des chaînes de caractères, etc.*

Dans notre cas, celui qui nous intéresse le plus, c’est `operator<<`, afin de pouvoir afficher nos fractions sur la sortie standard.

```
#include <iostream>
#include <string>
```

```

struct Fraction
{
    int numerateur;
    int denominateur;
};

Fraction operator+(Fraction const & a, Fraction const & b)
{
    int numerateur {
        a.numerateur * b.denominateur + b.numerateur * a.denominateur
    };
    int denominateur { a.denominateur * b.denominateur };

    return { numerateur, denominateur };
}

std::ostream & operator<<(std::ostream & flux, Fraction const &
fraction)
{
    return flux << fraction.numerateur
        << "/" << fraction.denominateur;
}

int main()
{
    Fraction const un_quart { 1, 4 };
    Fraction const neuf_vingtetunieme { 9, 21 };

    std::cout << "1/4 + 9/21 = "
        << un_quart + neuf_vingtetunieme << std::endl;
    return 0;
}

```

1/4 + 9/21 = 57/84

Si vous le souhaitez, vous pouvez vous amuser à implémenter l'opérateur `>>` pour être capable par exemple de créer des `Fraction` depuis des chaînes de caractères ou en lisant un fichier.

Plus et moins unaires

Il existe encore beaucoup d'opérateurs, nous n'allons donc pas pouvoir tous les aborder dans ce chapitre. Nous aimerions néanmoins vous parler du plus et du moins unaires, qui permettent de faire des opérations comme `+variable` ou `-variable`, exactement comme avec des entiers (entre autres).

Le modèle est très simple, le voici ci-dessous pour les deux opérateurs.

```
T operator+(T const & t);
T operator-(T const & t);
```

Dans notre code, cela nous permet donc de créer, notamment, des fractions négatives.

```
#include <cmath>
#include <iostream>
#include <string>

struct Fraction
{
    int numérateur;
    int dénominateur;
};

Fraction operator-(Fraction const & fraction)
{
    return {
        -std::abs(fraction.numérateur), std::abs(fraction.dénominateur)
    };
}

std::ostream & operator<<(std::ostream & flux, Fraction const &
fraction)
{
    return flux << fraction.numérateur
        << "/" << fraction.dénominateur;
}

int main()
{
    Fraction const un_quart { 1, 4 };
    std::cout << "L'opposé de 1/4 est " << -un_quart << "." << std::endl;
    return 0;
}
```

```
L'opposé de 1/4 est -1/4.
```

Aller plus loin

Pour pratiquer, nous vous proposons de finir ce petit projet de fractions. Voici quelques idées en vrac.

- Il n'y a aucune gestion des erreurs. Comment y remédier ?
- Il n'y a pas de tests unitaires.
- Renvoyer l'opposé d'une fraction.
- Inverser une fraction.

- Rendre une fraction irréductible. Pour cela, il faut diviser le numérateur et le dénominateur par leur PGCD.

Et la bibliothèque standard ?

Elle utilise à foison la surcharge d'opérateurs. Et vous en avez utilisé énormément sans le savoir. Examinons quelques cas.

Concaténation de chaînes

```
std::string const hello { "Hello" };
std::string const world { "World" };

std::string const texte { hello + " " + world };
```

Quand vous concaténez des chaînes de caractères, vous faites en réalité appel à `operator+` appliqué au type `std::string`. L'utilisation de cet opérateur est parfois décrite par certains, qui le jugent peu adapté. Reste que, avec celui-ci ou un autre, le code est raccourci et plus agréable à lire.

Accès à un élément

```
std::vector<int> const tableau { 1, 2, 3, 4 };
int const nombre { tableau[0] };
```

Dans l'exemple précédent, vous pouvez remplacer `std::vector` par `std::array` ou `std::map`, le principe est toujours le même. Derrière, c'est en effet `operator[]` qui est appelé. Celui-ci attend un entier comme argument, ce qui ne doit pas vous surprendre, puisque vous êtes habitué à la manipulation de tableaux maintenant.

Définition de littéraux `std::string`

```
using namespace std::literals;
auto chaine { "Hello, chaine C++s" }
```

Vous vous demandez où est la surcharge dans le code précédent ? Allez, on vous aide, c'est le `s` en fin de chaîne. Eh oui, même quelque chose d'aussi simple est en fait une surcharge de `operator""`. Ce dernier, arrivé avec C++11, permet de définir des suffixes, qui s'ajoutent après des littéraux afin d'en changer le type.

Dans le cas exposé ci-dessus, le suffixe `s` permet de transformer le type du littéral `"Hello, chaine C++"`, qui était un type hérité du C, en un `std::string`. Sans cette surcharge, il aurait fallu écrire `std::string { "Hello, chaine C++" }`, ce qui est plus long et moins lisible.

Entraînez-vous !

Calculs avec des durées

Pratiquons pour mieux retenir avec un petit exercice, dans lequel on va implémenter quelques opérations pour un type `Duree` qui, comme son nom l'indique, représente une durée. Voici déjà comment on définit une durée dans notre code.

```
struct Duree
{
    int secondes;
};
```

Comme vous voyez, on stocke les durées sous forme de secondes. L'exercice consiste à implémenter une liste d'opérations :

- créer une durée à partir de secondes, minutes (optionnel), heures (optionnel) ;
- renvoyer la durée sous forme de secondes ;
- renvoyer la durée sous forme de tuple {minutes, secondes}. Le signe de la durée sera porté par les minutes ;
- renvoyer la durée sous forme de tuple {heures, minutes, secondes}. Le signe de la durée sera porté par les heures ;
- additionner des durées ;
- renvoyer l'opposé d'une durée ;
- soustraire des durées ;
- comparer des durées ;
- injecter une durée dans un flux sortant, au format `H:mm'ss"`. Le signe de la durée sera mis devant les heures.

***Note** > Ici, on considère une durée comme un intervalle de temps entre deux événements. Par conséquent, une durée peut être négative ; les opérations comme prendre l'opposé d'une durée ont donc un sens.*

✧ [Voir la solution](#)

? [Obtenir un indice](#)

Sur le bon usage de la surcharge

La surcharge d'opérateurs est un outil puissant, il faut donc faire attention à ne pas l'utiliser n'importe comment. Notamment, C++ ne vérifie pas ce que vous mettez dans votre implémentation. Vous pourriez alors surcharger l'opérateur `/` pour effectuer une addition. Dans ce cas, vous violeriez la sémantique des opérateurs, c'est-à-dire que vous leur feriez faire autre chose que ce pour quoi ils sont prévus normalement. C'est une très mauvaise pratique, à bannir absolument.

L'autre point, c'est que, même si vous pouvez surcharger des opérateurs, cela n'a pas toujours de sens de le faire. Ainsi, additionner deux personnes ou deux comptes en banque n'a aucun sens et n'est pas correct. Nous reviendrons sur cet aspect au chapitre [Classes à sémantique d'entité](#). Retenez que ce n'est pas parce que vous pouvez que vous devez.

15.4. Récapitulons !

- On peut définir des alias de type grâce au mot clé `using`.
- On peut simplifier l'utilisation de `std::tuple` et même de structures grâce aux décompositions.
- La surcharge d'opérateurs permet d'augmenter la lisibilité et la qualité de notre code.
- Il est possible de surcharger de nombreux opérateurs.
- Elle doit être utilisée à bon escient, en ne faisant pas faire aux opérateurs des opérations qui violent leur sémantique.
- La surcharge d'opérateurs est un bonus qui, s'il est appréciable, ne doit pas pour autant être constamment et systématiquement utilisé.

SOLUTION

section Calculs avec des durées

Voici le corrigé avec les implémentations.

```
#include <iostream>
#include <tuple>

struct Duree
{
    int secondes;
};

Duree creer_duree(int secondes, int minutes = 0, int heures = 0);
int en_secondes(Duree d);
std::tuple<int, int> en_minutes(Duree d);
std::tuple<int, int, int> en_heures(Duree d);

Duree operator+(Duree const& lhs, Duree const& rhs);
Duree operator-(Duree const& d);
Duree operator-(Duree const& lhs, Duree const& rhs);

bool operator==(Duree const& lhs, Duree const& rhs);
bool operator!=(Duree const& lhs, Duree const& rhs);
bool operator<(Duree const& lhs, Duree const& rhs);
bool operator>(Duree const& lhs, Duree const& rhs);
bool operator<=(Duree const& lhs, Duree const& rhs);
bool operator>=(Duree const& lhs, Duree const& rhs);

std::ostream& operator<<(std::ostream& os, Duree d);

int main()
{
    return 0;
}

Duree creer_duree(int secondes, int minutes, int heures)
{
    return Duree{secondes + 60 * minutes + 3600 * heures};
}

int en_secondes(Duree d)
{
    return d.secondes;
}

std::pair<int, int> div_eucl_ordinaire(int i, int div)
{
    if(i >= 0)
        return {i / div, i % div};

    return {i / div - 1, i % div + 60};
}
```

SOLUTION

```

    }

    std::tuple<int, int> en_minutes(Duree d)
    {
        return div_eucl_ordinaire(en_secondes(d), 60);
    }

    std::tuple<int, int, int> en_heures(Duree d)
    {
        auto [minutes, secondes] = en_minutes(d);
        auto heures{0};
        std::tie(heures, minutes) = div_eucl_ordinaire(minutes, 60);
        return {heures, minutes, secondes};
    }

    Duree operator+(Duree const& lhs, Duree const& rhs)
    {
        return Duree{lhs.secondes + rhs.secondes};
    }

    Duree operator-(Duree const& d)
    {
        return Duree{-d.secondes};
    }

    Duree operator-(Duree const& lhs, Duree const& rhs)
    {
        return lhs + (-rhs);
    }

    bool operator==(Duree const& lhs, Duree const& rhs)
    {
        return lhs.secondes == rhs.secondes;
    }

    bool operator!=(Duree const& lhs, Duree const& rhs)
    {
        return !(lhs == rhs);
    }

    bool operator<(Duree const& lhs, Duree const& rhs)
    {
        return lhs.secondes < rhs.secondes;
    }

    bool operator>(Duree const& lhs, Duree const& rhs)
    {
        return lhs.secondes > rhs.secondes;
    }

    bool operator<=(Duree const& lhs, Duree const& rhs)
    {
        return !(lhs > rhs);
    }

```

SOLUTION

```

bool operator>=(Duree const& lhs, Duree const& rhs)
{
    return !(lhs < rhs);
}

std::ostream& operator<<(std::ostream& os, Duree const & duree)
{
    auto [heures, minutes, secondes] = en_heures(duree);

    auto format = [](int i)
    {
        auto res = std::to_string(i);
        if(std::size(res) == 1)
            res = '0' + res;
        return res;
    };

    return os << heures << ':' << format(minutes) << ':' <<
        format(secondes) << "'";
}

```

INDICE

section Calculs avec des durées

Voici, pour vous aider, les prototypes des fonctions à implémenter.

```
Duree creer_duree(int secondes, int minutes = 0, int heures = 0);
int en_secondes(Duree d);
std::tuple<int, int> en_minutes(Duree d);
std::tuple<int, int, int> en_heures(Duree d);

Duree operator+(Duree const& lhs, Duree const& rhs);
Duree operator-(Duree const& d);
Duree operator-(Duree const& lhs, Duree const& rhs);

bool operator==(Duree const& lhs, Duree const& rhs);
bool operator!=(Duree const& lhs, Duree const& rhs);
bool operator<(Duree const& lhs, Duree const& rhs);
bool operator>(Duree const& lhs, Duree const& rhs);
bool operator<=(Duree const& lhs, Duree const& rhs);
bool operator>=(Duree const& lhs, Duree const& rhs);

std::ostream& operator<<(std::ostream& os, Duree const & d);
```

16

[Pratique] Un gestionnaire de discographie

Depuis le début de ce livre, nous avons eu l'occasion de découvrir de nombreux concepts, que nous avons mis en pratique dans des micro-situations, en particulier pour la gestion des erreurs d'entrée.

Dans ce chapitre, nous allons pour la première fois écrire un petit programme complet et fonctionnel, qui sera l'occasion d'utiliser toutes les notions acquises jusqu'à présent. C'est votre baptême du feu, en quelque sorte. Mais ne vous inquiétez pas, nous allons y aller pas à pas.

16.1. Cahier des charges

Lors de ce projet, vous allez créer un gestionnaire de discographie en ligne de commande. Ce logiciel doit permettre d'ajouter des morceaux avec album et artiste à votre discographie. Il offrira la possibilité d'afficher votre discographie de plusieurs manières, ainsi que de l'enregistrer et de la charger à partir de fichiers.

L'utilisateur disposera d'une petite invite de commande pour entrer ses ordres. Toutes les actions devront être faisables en ligne de commande.

```
>
```

Ajout de morceaux

Pour ajouter un morceau à la discographie, l'utilisateur le fera simplement en tapant une commande de cette forme :

```
> ajouter morceau | album | artiste
```

Les informations sont optionnelles. Par exemple, l'utilisateur doit pouvoir taper ça :

```
> ajouter morceau | album
```

Ou ça :

```
> ajouter morceau | | artiste
```

Ou encore ceci :

```
> ajouter morceau
```

Et même ceci :

```
> ajouter
```

Lorsque des informations sont omises, la discographie enregistrera le morceau avec les mentions *Morceau inconnu*, *Album inconnu* ou *Artiste inconnu*.

Affichage de la discographie

L'utilisateur doit pouvoir afficher sa discographie de trois manières différentes, que l'on va illustrer par un exemple. Les commandes d'affichage commenceront toutes par le même mot.

```
> afficher
```

Prenons pour exemple la discographie suivante.

Morceau	Album	Artiste
Chanson A	Mes supers chansons	Bob le chanteur
Un chat qui dormait	Comptines d'Alice	Alice la chanteuse
L'amour	Comptines d'Alice	Alice la chanteuse
La mousse au chocolat	Cuisine en musique	Papa cuistot
Chanson B	Mes supers chansons	Bob le chanteur

Affichage par ordre alphabétique des morceaux

L'utilisateur pourra taper `> afficher morceaux`, auquel cas le programme devra lui afficher la liste des morceaux par ordre alphabétique.

```
> afficher morceaux
--> Chanson A | Mes supers chansons | Bob le chanteur
--> Chanson B | Mes supers chansons | Bob le chanteur
--> L'amour | Comptines d'Alice | Alice la chanteuse
--> La mousse au chocolat | Cuisine en musique | Papa cuistot
--> Un chat qui dormait | Comptines d'Alice | Alice la chanteuse
```

Affichage par album

Si l'utilisateur tape `> afficher albums`, le programme devra lui répondre avec la liste des albums, avec une sous-liste des morceaux de ces albums, le tout étant trié par ordre alphabétique.

```
> afficher albums
--> Comptines d'Alice | Alice la chanteuse
    /--> L'amour
    /--> Un chat qui dormait
--> Cuisine en musique | Papa cuistot
    /--> La mousse au chocolat
--> Mes supers chansons | Bob le chanteur
    /--> Chanson A
    /--> Chanson B
```

Affichage par artiste

C'est le même principe : si l'utilisateur tape `> afficher artistes`, le programme renverra une liste des auteurs, avec une sous-liste d'albums contenant elle-même une sous-liste de morceaux.

```
> afficher artistes
--> Alice la chanteuse
    /--> Comptines d'Alice
        /--> L'amour
        /--> Un chat qui dormait
--> Bob le chanteur
    /--> Mes supers chansons
        /--> Chanson A
        /--> Chanson B
--> Papa cuistot
    /--> Cuisine en musique
    /--> La mousse au chocolat
```

Bien sûr, si vous voulez adapter un peu l'affichage pour qu'il vous plaise davantage, vous pouvez. Après tout, l'un des intérêts de savoir programmer, c'est de pouvoir faire des programmes qui nous plaisent !

Enregistrement et chargement d'une discographie

L'utilisateur doit pouvoir enregistrer sa discographie sur le disque, pour pouvoir la sauvegarder lorsqu'il arrête le programme. Il doit aussi pouvoir la charger depuis un fichier. Ces deux fonctionnalités s'utiliseront à travers deux commandes.

```
> enregistrer nom_fichier
```

```
> charger nom_fichier
```

Fermeture du programme

Pour quitter le programme, l'utilisateur aura simplement à taper la commande qui suit.

```
> quitter
```

Gestion des erreurs

Le programme devra être robuste aux erreurs de saisie : si l'utilisateur tape une commande invalide, le programme ne devra pas crasher ou avoir un comportement bizarre. Il doit ignorer la commande et signaler à l'utilisateur que la commande est invalide.

```
> afficher qssqds  
Erreur : Commande invalide.
```

Le programme devra aussi être capable de gérer les erreurs dues à l'impossibilité d'ouvrir un fichier, que ce soit en lecture ou en écriture.

Dernières remarques

Ce projet est votre première confrontation à la conception logicielle. La difficulté ne réside plus seulement dans la compréhension et l'utilisation du langage, mais aussi dans la conception : il va vous falloir réfléchir à comment organiser votre code.

L'objectif est que vous réfléchissiez à une manière d'avoir un code propre, robuste, agréable à lire, ouvert aux évolutions et bien sûr fonctionnel. C'est une tâche difficile, mais rassurez-vous : comme c'est votre première fois, nous vous proposons un guide pour vous accompagner dans les différentes étapes. Prenez cependant le temps de réfléchir et de vous casser la tête avant de le consulter. Cela vous confrontera à la difficulté

et vous fera progresser. Ensuite seulement, lisez ce guide pour vous aiguiller vers une solution. Et pour finir, une fois que vous avez écrit votre propre code, examinez celui que nous vous proposons.

16.2. Guide de résolution

Analyse des besoins

Pour concevoir un programme efficacement, il faut analyser les besoins auxquels il doit répondre – autrement dit les tâches qu’il doit être capable de réaliser – pour arriver à des problèmes plus atomiques, auxquels on peut répondre directement par un morceau de code, une fonction, une structure de données...

Ici, les besoins sont définis par le cahier des charges ; on va donc partir de ce dernier et le découper en problèmes élémentaires.

Interpréter les commandes

L'utilisateur donne ses instructions au moyen d'une ligne de commande. Notre logiciel doit donc être capable de lire et comprendre les commandes de l'utilisateur, ainsi que de détecter les commandes invalides.

Il y a plusieurs types de commande : les commandes d'ajout de morceau, les commandes de sauvegarde/chargement de la bibliographie, et bien sûr les commandes d'affichage. Celles-ci se décomposent ensuite en plusieurs modes d'affichage. On a donc deux tâches distinctes :

- lire et interpréter les commandes ;
- exécuter les commandes.

Nous allons avoir une fonction d'exécution pour chaque type de commande, et des fonctions qui se chargeront de lire les commandes et d'appeler la fonction d'exécution adéquate ; ce seront nos fonctions d'interprétation. Nous reviendrons [un peu plus loin](#) dans ce chapitre sur l'implémentation, mais cela nous donne déjà une idée du schéma de fonctionnement global.

Stocker la discographie

Ensuite, il nous faut un moyen de mémoriser la discographie, que ce soit pendant l'exécution du programme (stockage en mémoire de travail dans une variable) ou sur le

long terme (stockage sur disque dans un fichier). On dégage donc ici deux besoins techniques :

- représenter la bibliographie dans notre programme : en C++, cela revient à peu de choses près à définir un type adapté. C'est par cet aspect que nous entrerons en premier dans les détails techniques. En effet, cette représentation est centrale et sera omniprésente dans le reste du code ;
- sauvegarder et charger cette représentation dans un fichier : nous devons entre autres choisir une représentation de la bibliothèque sous un format texte.

Gérer les erreurs d'entrée

Enfin, nous devons gérer les erreurs d'entrée — et ce de manière *user friendly*. Plus précisément, il faut que nos fonctions d'interprétation détectent les erreurs. Elles doivent aussi pouvoir les remonter. Pour cela, nous pouvons tout simplement utiliser le système d'exceptions.

Attaquons le code !

Nous pouvons maintenant aborder la conception plus précise des différents points vus précédemment, ainsi que l'implémentation. Comme on l'a dit, nous allons commencer par un point central : la représentation de la discographie.

Représentation de la discographie

Une discographie peut simplement être vue comme une collection de morceaux. Au regard du cahier des charges, les informations dont on a besoin pour un morceau sont son nom, son album et son artiste. Ainsi :

- un artiste a comme unique donnée son nom ;
- un album a lui aussi un nom ;
- un morceau a pour données importantes son nom, ainsi que l'album et l'artiste correspondants ;
- une discographie est une collection de morceaux, de taille variable évidemment.

Ces constats étant faits, cela devrait vous aider à trouver des structures de données adaptées.

✧ [Voir un exemple de structures de données](#)

Fonctions de base

Une fois nos structures de données définies, il est commun de créer des fonctions de base qui vont nous permettre de les manipuler de manière agréable. C'est typiquement le moment de créer des surcharges pour certains opérateurs pour nos types nouvellement créés. Notamment, on aimerait pouvoir afficher facilement nos différentes structures, ainsi que les lire et les écrire dans un fichier. On va donc avoir besoin de surcharger les opérateurs de flux.

Nous vous suggérons dans un premier temps d'essayer de trouver les prototypes de telles fonctions, puis d'essayer de les implémenter.

✧ [Voir un exemple de prototypes](#)

✧ [Voir un exemple d'implémentation](#)

Comme nous sommes des programmeurs consciencieux, nous voulons écrire des tests pour nous assurer que ce que nous avons déjà écrit fonctionne bien. En plus de vérifier qu'une entrée valide est acceptée, nous devons aussi penser aux cas limites ainsi qu'aux erreurs, pour mesurer la résistance et la bonne conception de notre programme.

Par exemple, "Chanson A | Mes supers chansons | Bob le chanteur" et "Chanson B | Mes supers chansons | Bob le chanteur" sont des entrées valides et complètes, même si dans la seconde il y a beaucoup d'espaces. De même, "Un chat qui dormait | | Alice la chanteuse" et "L'amour | |" aussi, quand bien même certaines informations sont manquantes.

✧ [Voir un exemple de tests unitaires](#)

Stockage dans un fichier

Maintenant que nous avons construit nos structures de base pour pouvoir représenter une bibliographie en mémoire, nous pouvons créer des fonctions pour sauvegarder et charger une discographie dans un fichier.

✧ [Voir un exemple d'implémentation](#)

Affichage de la discographie

Autre point qu'on a soulevé, l'affichage. Nous voudrions pouvoir afficher la discographie par album, par artiste ou par titre. Plus précisément, on remarque que pour chaque mode d'affichage, il y a en réalité deux composantes distinctes : la manière d'afficher à proprement parler, mais aussi l'ordre dans lequel sont affichés les morceaux, c'est-à-dire le tri de la discographie. Nous dégageons ainsi six fonctions à implémenter : une de tri et une d'affichage pour chaque mode d'affichage.

✧ [Voir un exemple de prototypes](#)

✧ [Voir un exemple d'implémentation](#)

L'AVANTAGE DE `CONST` POUR LA CONCEPTION

La manière dont nous nous sommes rendu compte qu'il y avait deux parties bien distinctes à envisager (et donc à traiter dans des fonctions séparées) est très intéressante, donc nous aimerions vous l'expliquer.

Au départ, le tri se faisait directement dans les fonctions d'affichage. Or des fonctions d'affichage, vu leurs rôles, ne devraient pas avoir à modifier la discographie ; nous souhaitons donc que ces trois fonctions prennent en paramètre une `Discographie const&` et non pas une `Discographie&`. Mais le tri modifie la discographie, donc c'était impossible. C'est ainsi que nous avons compris que la logique d'affichage comprenait en fait deux sous-tâches indépendantes, à traiter séparément.

Ainsi, on est dans un cas où faire attention à l'utilisation de `const` permet d'améliorer la conception du code.

Pour lier tout ça, nous pouvons créer une quatrième fonction qui lancera les fonctions du mode d'affichage demandé. Comme nos possibilités d'affichage sont limitées, nous pouvons les regrouper dans une énumération.

✧ [Voir un exemple de prototypes](#)

✧ [Voir un exemple d'implémentation](#)

Système de commandes

Il nous reste encore à interagir avec l'utilisateur, à l'aide du système de commandes. On a fait ressortir deux problèmes principaux, l'interprétation et l'exécution des commandes. On gardera en tête le fait qu'il faut gérer les erreurs.

✧ [Voir un exemple de prototypes](#)

Essayez maintenant d'implémenter de telles fonctions.

✧ [Voir un exemple d'implémentation](#)

Finalisation

Maintenant que toutes nos fonctionnalités sont programmées, il suffit de tout intégrer en écrivant une fonction `main`, qui aura pour principale mission d'initialiser le programme (en lançant d'abord les tests) et de faire tourner une boucle d'invite de commandes jusqu'à ce que l'utilisateur décide de quitter.

✧ [Voir un exemple de fonction principale](#)

Et voilà ! Nous avons construit pas à pas toutes les briques de notre programme, jusqu'à avoir tout le code pour le faire fonctionner. La démarche à retenir est la suivante : il faut réfléchir à une manière de découper notre problème initial (ici, *faire une discographie*) en sous-problèmes (par exemple *lire et comprendre une commande*), qui eux-mêmes seront découpés en sous-problèmes, jusqu'à arriver à des briques de base simples à réaliser (par exemple *créer un type représentant les morceaux*).

16.3. Code complet

Nous espérons que vos phases de recherches et d'avancées à l'aide du guide ont été fructueuses ! Que vous ayez réussi ou non à obtenir un résultat fonctionnel, l'important est que vous ayez bien réfléchi aux différentes problématiques. C'est en vous confrontant à vos difficultés que vous progresserez.

Voici maintenant une manière de programmer notre gestionnaire, issue du guide.

```
#include <algorithm>
#include <cassert>
#include <cctype>
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <tuple>
#include <vector>

std::string traitement_chaine(std::string const & chaine)
{
    std::string copie { chaine };

    // D'abord on enlève les espaces au début...
    auto premier_non_espace{ std::find_if_not(std::begin(copie),
std::end(copie), isspace) };
    copie.erase(std::begin(copie), premier_non_espace);

    // ...puis à la fin.
    std::reverse(std::begin(copie), std::end(copie));
    premier_non_espace = std::find_if_not(std::begin(copie),
std::end(copie), isspace);
    copie.erase(std::begin(copie), premier_non_espace);
    std::reverse(std::begin(copie), std::end(copie));

    return copie;
}

struct Artiste
{
    std::string nom;
};

// On surcharge l'opérateur << pour pouvoir afficher le type Artiste.
std::ostream & operator<<(std::ostream & sortie, Artiste const &
artiste)
{
    sortie << artiste.nom;
    return sortie;
}

struct Album
{

```

```

    std::string nom;
};

// On surcharge l'opérateur << pour pouvoir afficher le type Album.
std::ostream & operator<<(std::ostream & sortie, Album const & album)
{
    sortie << album.nom;
    return sortie;
}

struct Morceau
{
    std::string nom;
    Artiste compositeur;
    Album album;
};

// On surcharge l'opérateur << pour pouvoir afficher le type Morceau.
std::ostream & operator<<(std::ostream & sortie, Morceau const &
morceau)
{
    sortie << morceau.nom << " | " << morceau.album << " | " <<
morceau.compositeur;
    return sortie;
}

// Conversion d'un flux d'entrée (std::cin, chaîne, fichier) en type
Morceau.
std::istream & operator>>(std::istream & entree, Morceau & morceau)
{
    std::string mot {};
    std::ostringstream flux {};

    // Récupération du nom du morceau.
    while (entree >> mot && mot != "|")
    {
        flux << mot << " ";
    }

    std::string nom_morceau { flux.str() };
    if (std::empty(nom_morceau))
    {
        nom_morceau = "Morceau inconnu";
    }
    morceau.nom = traitement_chaine(nom_morceau);
    flux.str("");

    // Récupération du nom de l'album.
    while (entree >> mot && mot != "|")
    {
        flux << mot << " ";
    }

    std::string nom_album { flux.str() };
    if (std::empty(nom_album))
    {

```



```

    nom_album = "Album inconnu";
}
morceau.album.nom = traitement_chaine(nom_album);
flux.str("");

// Récupération du nom de l'artiste.
while (entree >> mot)
{
    flux << mot << " ";
}

std::string nom_artiste { flux.str() };
if (std::empty(nom_artiste))
{
    nom_artiste = "Artiste inconnu";
}
morceau.compositeur.nom = traitement_chaine(nom_artiste);
flux.str("");

return entree;
}

// Test qu'une entrée complète est valide.
void test_creation_morceau_entree_complete()
{
    std::istringstream entree { "Chanson A | Mes supers chansons | Bob le
chanteur" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "Chanson A" && u8"Le nom du morceau doit être
Chanson A.");
    assert(morceau.album.nom == "Mes supers chansons" && u8"Le nom de
l'album doit être Mes supers chansons.");
    assert(morceau.compositeur.nom == "Bob le chanteur" && u8"Le nom du
compositeur doit être Bob le chanteur.");
}

// Test d'une entrée complète avec beaucoup d'espaces.
void test_creation_morceau_entree_espaces_partout()
{
    std::istringstream entree { "Chanson B          | Mes supers chansons
|      Bob le chanteur" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "Chanson B" && u8"Le nom du morceau doit être
Chanson B.");
    assert(morceau.album.nom == "Mes supers chansons" && u8"Le nom de
l'album doit être Mes supers chansons.");
    assert(morceau.compositeur.nom == "Bob le chanteur" && u8"Le nom du
compositeur doit être Bob le chanteur.");
}

```

```

// Test d'une entrée avec seulement le nom de la chanson et de
// l'artiste.
void test_creation_morceau_entree_chanson_artiste()
{
    std::istringstream entree { "Un chat qui dormait | | Alice la
    chanteuse" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "Un chat qui dormait" && u8"Le nom du morceau
    doit être Un chat qui dormait.");
    assert(morceau.album.nom == "Album inconnu" && u8"Le nom de l'album
    doit être Album inconnu.");
    assert(morceau.compositeur.nom == "Alice la chanteuse" && u8"Le nom du
    compositeur doit être Alice la chanteuse.");
}

// Test d'une entrée avec seulement le nom de la chanson.
void test_creation_morceau_entree_chanson_uniquement()
{
    std::istringstream entree { "L'amour | |" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "L'amour" && u8"Le nom du morceau doit être
    L'amour.");
    assert(morceau.album.nom == "Album inconnu" && u8"Le nom de l'album
    doit être Album inconnu.");
    assert(morceau.compositeur.nom == "Artiste inconnu" && u8"Le nom du
    compositeur doit être Artiste inconnu.");
}

// Test d'une entrée vide.
void test_creation_morceau_entree_vide()
{
    std::istringstream entree{ "| |" };
    Morceau morceau{};

    entree >> morceau;

    assert(morceau.nom == "Morceau inconnu" && u8"Le nom du morceau doit
    être Morceau inconnu.");
    assert(morceau.album.nom == "Album inconnu" && u8"Le nom de l'album
    doit être Album inconnu.");
    assert(morceau.compositeur.nom == "Artiste inconnu" && u8"Le nom du
    compositeur doit être Artiste inconnu.");
}

using Discographie = std::vector<Morceau>;

void tri_morceau(Discographie& discographie)
{

```

```

    std::sort(std::begin(discographie), std::end(discographie), []
(Morceau const & lhs, Morceau const & rhs)
    {
        return lhs.nom < rhs.nom;
    });
}

void tri_album(Discographie& discographie)
{
    std::sort(std::begin(discographie), std::end(discographie), []
(Morceau const & lhs, Morceau const & rhs)
    {
        if (lhs.album.nom < rhs.album.nom)
        {
            return true;
        }

        return lhs.album.nom == rhs.album.nom && lhs.nom < rhs.nom;
    });
}

void tri_artiste(Discographie& discographie)
{
    std::sort(std::begin(discographie), std::end(discographie),
    [] (Morceau const & lhs, Morceau const & rhs)
    {
        if (lhs.compositeur.nom < rhs.compositeur.nom)
        {
            return true;
        }

        if (lhs.compositeur.nom == rhs.compositeur.nom)
        {
            if (lhs.album.nom < rhs.album.nom)
            {
                return true;
            }

            return lhs.album.nom == rhs.album.nom
                && lhs.nom < rhs.nom;
        }

        return false;
    });
}

// Affichage par morceau, dans l'ordre alphabétique.
void affichage_morceau(Discographie const & discographie)
{
    for (Morceau const & morceau : discographie)
    {
        std::cout << "--> " << morceau << std::endl;
    }
}

// Affichage par album, dans l'ordre alphabétique.

```

```

void affichage_album(Discographie const & discographie)
{
    Album album_precedent{};
    for (Morceau const & morceau : discographie)
    {
        if (morceau.album.nom != album_precedent.nom)
        {
            std::cout << "--> "
                << morceau.album << " | "
                << morceau.compositeur << std::endl;
        }

        std::cout << "\t/--> " << morceau.nom << std::endl;

        album_precedent = morceau.album;
    }
}

// Affichage par artiste, dans l'ordre alphabétique.
void affichage_artiste(Discographie const & discographie)
{
    Artiste artiste_precedent {};
    Album album_precedent {};

    for (Morceau const & morceau : discographie)
    {
        if (morceau.compositeur.nom != artiste_precedent.nom)
        {
            std::cout << "--> " << morceau.compositeur << std::endl;
        }

        if (morceau.album.nom != album_precedent.nom)
        {
            std::cout << "\t/--> " << morceau.album << std::endl;
        }

        std::cout << "\t\t/--> " << morceau.nom << std::endl;

        artiste_precedent = morceau.compositeur;
        album_precedent = morceau.album;
    }
}

// L'ensemble des possibilités d'affichage de la discographie.
enum class Affichage { Artiste, Album, Morceau };

void affichage(Discographie& discographie, Affichage type_affichage)
{
    if (type_affichage == Affichage::Album)
    {
        tri_album(discographie);
        affichage_album(discographie);
    }
    else if (type_affichage == Affichage::Artiste)
    {
        tri_artiste(discographie);
    }
}

```

```

    affichage_artiste(discographie);
}
else if (type_affichage == Affichage::Morceau)
{
    tri_morceau(discographie);
    affichage_morceau(discographie);
}
else
{
    // Par exemple si on met à jour l'énumération
    // mais qu'on oublie d'ajouter la fonction correspondante.
    throw std::runtime_error("Commande d'affichage inconnue.");
}
}

void enregistrement(Discographie const & discographie, std::string const
& nom_fichier)
{
    std::ofstream fichier { nom_fichier };
    if (!fichier)
    {
        throw std::runtime_error("Impossible d'ouvrir le fichier en
écriture.");
    }

    for (Morceau const & morceau : discographie)
    {
        fichier << morceau << std::endl;
    }
}

void chargement(Discographie & discographie, std::string const &
nom_fichier)
{
    std::ifstream fichier { nom_fichier };
    if (!fichier)
    {
        throw std::runtime_error("Impossible d'ouvrir le fichier en
lecture.");
    }

    std::string ligne {};
    while (std::getline(fichier, ligne))
    {
        Morceau morceau {};

        std::istringstream flux { ligne };
        flux >> morceau;

        discographie.push_back(morceau);
    }
}

// Récupération de ce que l'utilisateur écrit.
std::string recuperer_commande()
{

```

```

std::cout << "> ";

std::string commande {};
std::getline(std::cin, commande);
return commande;
}

// L'ensemble des actions qu'il est possible de faire.
enum class Commande { Afficher, Ajouter, Enregistrer, Charger,
    Quitter };

// Analyse du texte reçu en entrée.
std::tuple<Commande, std::string> analyser_commande(std::string const &
    commande_texte)
{
    // Pour traiter la chaîne comme étant un flux,
    // afin d'en extraire les données.
    std::istringstream flux { commande_texte };

    std::string premier_mot {};
    flux >> premier_mot;
    premier_mot = traitement_chaine(premier_mot);

    std::string instructions {};
    std::getline(flux, instructions);
    instructions = traitement_chaine(instructions);

    if (premier_mot == "afficher")
    {
        // Le mode d'affichage.
        return { Commande::Afficher, instructions };
    }
    else if (premier_mot == "ajouter")
    {
        // Les informations à ajouter.
        return { Commande::Ajouter, instructions };
    }
    else if (premier_mot == "enregistrer")
    {
        // Le fichier à écrire.
        return { Commande::Enregistrer, instructions };
    }
    else if (premier_mot == "charger")
    {
        // Le fichier à lire.
        return { Commande::Charger, instructions };
    }
    else if (premier_mot == "quitter")
    {
        // Chaîne vide, car on quitte le programme sans autre instruction.
        return { Commande::Quitter, std::string {} };
    }
    else
    {
        // On a reçu du texte qui est incorrect.
        throw std::runtime_error("Commande invalide.");
    }
}

```

```

    }
}

// La commande à exécuter ainsi que la suite des instructions.
bool executer_commande(Discographie & discographie, Commande commande,
    std::string const & instructions)
{
    if (commande == Commande::Afficher)
    {
        if (instructions == "artistes")
        {
            affichage(discographie, Affichage::Artiste);
        }
        else if (instructions == "albums")
        {
            affichage(discographie, Affichage::Album);
        }
        else if (instructions == "morceaux")
        {
            affichage(discographie, Affichage::Morceau);
        }
        else
        {
            // Si on se trouve ici, alors c'est qu'il y a
            // une erreur dans les instructions d'affichage.
            throw std::runtime_error("Commande d'affichage inconnue.");
        }
    }
    else if (commande == Commande::Ajouter)
    {
        std::istringstream flux { instructions };
        Morceau morceau {};
        flux >> morceau;
        discographie.push_back(morceau);
    }
    else if (commande == Commande::Charger)
    {
        chargement(discographie, instructions);
    }
    else if (commande == Commande::Enregistrer)
    {
        enregistrement(discographie, instructions);
    }
    else if (commande == Commande::Quitter)
    {
        // Plus rien à faire, on quitte.
        return false;
    }

    return true;
}

int main()
{
    // Lancement préalable des tests unitaires.
    test_creation_morceau_entree_complete();
}

```

```

test_creation_morceau_entree_espaces_partout();
test_creation_morceau_entree_chanson_artiste();
test_creation_morceau_entree_chanson_uniquement();
test_creation_morceau_entree_vide();

// On préremplit la discographie.
Artiste const bob_le_chanteur { "Bob le chanteur " };
Artiste const alice_la_chanteuse { "Alice la chanteuse" };
Artiste const papa_cuistot { "Papa cuistot" };

Album const mes_supers_chansons { "Mes supers chansons" };
Album const comptines_alice { "Comptines d'Alice" };
Album const cuisine_en_musique { "Cuisine en musique" };

Morceau const chanson_a {
    "Chanson A",
    bob_le_chanteur,
    mes_supers_chansons
};
Morceau const chanson_b {
    "Chanson B",
    bob_le_chanteur,
    mes_supers_chansons
};
Morceau const un_chat_qui_dormait {
    "Un chat qui dormait",
    alice_la_chanteuse,
    comptines_alice
};
Morceau const amour {
    "L'amour",
    alice_la_chanteuse,
    comptines_alice
};
Morceau const mousse_au_chocolat {
    "La mousse au chocolat",
    papa_cuistot,
    cuisine_en_musique
};

Discographie discographie {
    chanson_a,
    chanson_b,
    un_chat_qui_dormait,
    amour,
    mousse_au_chocolat
};

bool continuer { true };
do
{
    try
    {
        std::string entree { recuperer_commande() };
        auto[commande, instructions] = analyser_commande(entree);
        instructions = traitement_chaine(instructions);
    }
    catch (const std::exception& e)
    {
        std::cout << "Erreur : " << e.what() << std::endl;
    }
} while (continuer);

```



```

        continuer = executer_commande(discographie, commande,
instructions);
    }
    catch (std::runtime_error const & exception)
    {
        std::cout << "Erreur : " << exception.what() << std::endl;
    }

    } while (continuer);

    return 0;
}

```

Note > Ce code n'est pas LA solution, mais une parmi tant d'autres. Le vôtre est différent, mais vous êtes malgré tout arrivé à la solution ? Bravo, vous pouvez être fier.

16.4. Conclusion et pistes d'améliorations

Voilà qui conclut ce cas pratique, à travers lequel vous avez pu vous confronter pour la première fois à la conception d'un programme complet, plus ambitieux que les exercices précédents. Nous avons pu ainsi voir la démarche qu'il faut avoir en tête pour pouvoir créer du code de qualité, c'est-à-dire expressif et surtout évolutif.

Cette évolutivité vous permet d'ajouter à votre guise des fonctionnalités à votre programme, et ce de manière efficace. Les possibilités d'ajout sont infinies, vous pouvez faire ce que vous voulez. Voici quelques idées ci-dessous.

- Ajouter des options d'affichage (comme l'affichage des morceaux d'un seul album ou artiste) grâce à des commandes du type `> afficher artiste Bob le chanteur` par exemple.
- Ajouter des options de tri des morceaux, par BPM par exemple.
- Avoir un fichier de configuration du logiciel permettant à l'utilisateur de choisir son invite de commandes, son séparateur dans les affichages et dans les commandes d'ajout, les puces à utiliser dans les listes affichées, etc. En bref, rendre le logiciel configurable. Un tel fichier pourrait ressembler à ça, pour les réglages donnant les mêmes affichages que ceux qu'on a codés en dur.

```

invite=">"
separateur_affichage="|"
separateur_ajout="|"
puce_niveau1="->"
puce_niveau2="/->"
puce_niveau3="/->"

```

- Notre code n'est pas entièrement testé. Il ne tient qu'à vous d'écrire les tests manquants.

N'hésitez pas, lorsque vous voulez ajouter des fonctionnalités, à continuer avec la même démarche : commencez par réfléchir, décomposez ce que vous voulez faire en petits problèmes, créez des types pratiques s'il le faut, etc.

Note > Nous vous conseillons de lire le chapitre suivant avant de faire vos ajouts. En effet, vous serez alors capable de mieux organiser votre code, ce qui vous aidera à le faire évoluer pour ajouter des nouvelles fonctionnalités.

16.5. Récapitulons !

- Vous vous êtes confronté pour la première fois à la réalité de la conception d'un programme informatique.
- On a pu en tirer une démarche pour avoir un code lisible et évolutif.
- La réflexion est importante si on veut obtenir un code de qualité.
- Les notions vues jusqu'à présent peuvent en pratique se combiner pour en tirer le meilleur et ainsi obtenir cette qualité.
- Il ne tient qu'à vous d'imaginer des pistes d'amélioration et d'ajouter vos propres fonctionnalités.

REPRÉSENTATION DE LA DISCOGRAPHIE

Structures de données

Un choix possible est le suivant : représenter les artistes, albums et morceaux par des structures simples, et la discographie par un alias, à l'aide de `using`.

```
struct Artiste
{
    std::string nom;
};

struct Album
{
    std::string nom;
};

struct Morceau
{
    std::string nom;
    Album album;
    Artiste compositeur;
};
// Une discographie n'est rien d'autre qu'un ensemble de morceaux.
using Discographie = std::vector<Morceau>;
```

FONCTIONS DE BASE

Prototypes

```
std::ostream & operator<<(std::ostream & sortie, Artiste const &
    artiste);
std::ostream & operator<<(std::ostream & sortie, Album const &
    album);
std::ostream & operator<<(std::ostream & sortie, Morceau const &
    morceau);
std::istream & operator>>(std::istream & entree, Morceau &
    morceau);
void ajouter_morceau(Discographie& disco, std::string const&
    nom_morceau, std::string const& nom_album, std::string const&
    nom_artiste);
```

FONCTIONS DE BASE

Implémentations

```
// Fonction utilisée pour nettoyer les entrées.
// Elle sera réutilisée à plusieurs endroits.
std::string traitement_chaine(std::string const & chaine)
{
    std::string copie { chaine };

    // D'abord on enlève les espaces au début...
    auto premier_non_espace { std::find_if_not(std::begin(copie),
std::end(copie), isspace) };
    copie.erase(std::begin(copie), premier_non_espace);

    // ...puis à la fin.
    std::reverse(std::begin(copie), std::end(copie));
    premier_non_espace = std::find_if_not(std::begin(copie),
std::end(copie), isspace);
    copie.erase(std::begin(copie), premier_non_espace);
    std::reverse(std::begin(copie), std::end(copie));

    return copie;
}

// On surcharge l'opérateur << pour pouvoir afficher le type
Artiste.
std::ostream & operator<<(std::ostream & sortie, Artiste const &
artiste)
{
    sortie << artiste.nom;
    return sortie;
}

// On surcharge l'opérateur << pour pouvoir afficher le type
Album.
std::ostream & operator<<(std::ostream & sortie, Album const &
album)
{
    sortie << album.nom;
    return sortie;
}

// On surcharge l'opérateur << pour pouvoir afficher le type
Morceau.
std::ostream & operator<<(std::ostream & sortie, Morceau const &
morceau)
{
    sortie << morceau.nom << " | " << morceau.album << " | " <<
morceau.compositeur;
    return sortie;
}
```

FONCTIONS DE BASE

```
// Conversion d'un flux d'entrée (std::cin, chaîne, fichier) en
// type Morceau.
std::istream & operator>>(std::istream & entree, Morceau &
morceau)
{
    std::string mot {};
    std::ostringstream flux {};

    // Récupération du nom du morceau.
    while (entree >> mot && mot != "|")
    {
        flux << mot << " ";
    }
    std::string nom_morceau { flux.str() };
    if (std::empty(nom_morceau))
    {
        nom_morceau = "Morceau inconnu";
    }
    morceau.nom = traitement_chaine(nom_morceau);
    flux.str(std::string {});

    // Récupération du nom de l'album.
    while (entree >> mot && mot != "|")
    {
        flux << mot << " ";
    }
    std::string nom_album { flux.str() };
    if (std::empty(nom_album))
    {
        nom_album = "Album inconnu";
    }
    morceau.album.nom = traitement_chaine(nom_album);
    flux.str(std::string {});

    // Récupération du nom de l'artiste.
    while (entree >> mot)
    {
        flux << mot << " ";
    }
    std::string nom_artiste { flux.str() };
    if (std::empty(nom_artiste))
    {
        nom_artiste = "Artiste inconnu";
    }
    morceau.compositeur.nom = traitement_chaine(nom_artiste);
    flux.str(std::string {});

    return entree;
}
```

FONCTIONS DE BASE

Vous remarquerez l'utilisation des flux de chaînes de caractères pour gagner en clarté et en élégance. Ils seront beaucoup utilisés dans ce programme. Vous pouvez aussi voir que l'on a défini une fonction supplémentaire (`traitement_chaine`) pour éviter la répétition de code.

Tests unitaires

```
// Test d'une entrée complète.
void test_creation_morceau_entree_complete()
{
    std::istringstream entree { "Chanson A | Mes supers chansons |
    Bob le chanteur" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "Chanson A" && u8"Le nom du morceau doit
    être Chanson A.");
    assert(morceau.album.nom == "Mes supers chansons" && u8"Le nom de
    l'album doit être Mes supers chansons.");
    assert(morceau.compositeur.nom == "Bob le chanteur" && u8"Le nom
    du compositeur doit être Bob le chanteur.");
}

// Test d'une entrée complète avec beaucoup d'espaces.
void test_creation_morceau_entree_espaces_partout()
{
    std::istringstream entree { "Chanson B      | Mes supers chansons
    |   Bob le chanteur" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "Chanson B" && u8"Le nom du morceau doit
    être Chanson B.");
    assert(morceau.album.nom == "Mes supers chansons" && u8"Le nom de
    l'album doit être Mes supers chansons.");
    assert(morceau.compositeur.nom == "Bob le chanteur" && u8"Le nom
    du compositeur doit être Bob le chanteur.");
}

// Test d'une entrée avec seulement le nom de la chanson et
// l'artiste.
void test_creation_morceau_entree_chanson_artiste()
{
    std::istringstream entree { "Un chat qui dormait | | Alice la
    chanteuse" };
    Morceau morceau {};
```

FONCTIONS DE BASE

```

    entree >> morceau;

    assert(morceau.nom == "Un chat qui dormait" && u8"Le nom du
    morceau doit être Un chat qui dormait.");
    assert(morceau.album.nom == "Album inconnu" && u8"Le nom de
    l'album doit être Album inconnu.");
    assert(morceau.compositeur.nom == "Alice la chanteuse" && u8"Le
    nom du compositeur doit être Alice la chanteuse.");
}

// Test d'une entrée avec seulement le nom de la chanson.
void test_creation_morceau_entree_chanson_uniquement()
{
    std::istringstream entree { "L'amour | |" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "L'amour" && u8"Le nom du morceau doit être
    L'amour.");
    assert(morceau.album.nom == "Album inconnu" && u8"Le nom de
    l'album doit être Album inconnu.");
    assert(morceau.compositeur.nom == "Artiste inconnu" && u8"Le nom
    du compositeur doit être Artiste inconnu.");
}

// Test d'une entrée vide.
void test_creation_morceau_entree_vide()
{
    std::istringstream entree{ "| |" };
    Morceau morceau{};

    entree >> morceau;

    assert(morceau.nom == "Morceau inconnu" && u8"Le nom du morceau
    doit être Morceau inconnu.");
    assert(morceau.album.nom == "Album inconnu" && u8"Le nom de
    l'album doit être Album inconnu.");
    assert(morceau.compositeur.nom == "Artiste inconnu" && u8"Le nom
    du compositeur doit être Artiste inconnu.");
}

```

STOCKAGE DANS UN FICHIER

Implémentation

```

void enregistrement(Discographie const & discographie,
std::string const & nom_fichier)
{
    std::ofstream fichier { nom_fichier };
    if (!fichier)
    {
        throw std::runtime_error("Impossible d'ouvrir le fichier en
écriture.");
    }

    for (Morceau const & morceau : discographie)
    {
        fichier << morceau << std::endl;
    }
}

void chargement(Discographie & discographie, std::string const &
nom_fichier)
{
    std::ifstream fichier { nom_fichier };
    if (!fichier)
    {
        throw std::runtime_error("Impossible d'ouvrir le fichier en
lecture.");
    }

    std::string ligne {};
    while (std::getline(fichier, ligne))
    {
        Morceau morceau {};

        std::istringstream flux { ligne };
        flux >> morceau;

        discographie.push_back(morceau);
    }
}

```


AFFICHAGE DE LA DISCOGRAPHIE

Prototypes

```
void tri_morceau(Discographie& discographie);
void tri_album(Discographie& discographie);
void tri_artiste(Discographie& discographie);

void affichage_morceau(Discographie const & discographie)
void affichage_album(Discographie const & discographie);
void affichage_artiste(Discographie const & discographie);
```

Implémentation

```
void tri_morceau(Discographie& discographie)
{
    std::sort(std::begin(discographie), std::end(discographie), []
(Morceau const & lhs, Morceau const & rhs)
    {
        return lhs.nom < rhs.nom;
    });
}

void tri_album(Discographie& discographie)
{
    std::sort(std::begin(discographie), std::end(discographie), []
(Morceau const & lhs, Morceau const & rhs)
    {
        if (lhs.album.nom < rhs.album.nom)
            return true;

        return lhs.album.nom == rhs.album.nom && lhs.nom < rhs.nom;
    });
}

void tri_artiste(Discographie& discographie)
{
    std::sort(std::begin(discographie), std::end(discographie), []
(Morceau const & lhs, Morceau const & rhs)
    {
        if (lhs.compositeur.nom < rhs.compositeur.nom)
            return true;

        if (lhs.compositeur.nom == rhs.compositeur.nom)
        {
            if (lhs.album.nom < rhs.album.nom)
                return true;

            return lhs.album.nom == rhs.album.nom && lhs.nom < rhs.nom;
        }
    })
}
```

AFFICHAGE DE LA DISCOGRAPHIE

```

        return false;
    });
}

void affichage_morceau(Discographie const & discographie)
{
    for (Morceau const & morceau : discographie)
    {
        std::cout << "--> " << morceau << std::endl;
    }
}

void affichage_album(Discographie const & discographie)
{
    Album album_precedent{};
    for (Morceau const & morceau : discographie)
    {
        if (morceau.album.nom != album_precedent.nom)
        {
            std::cout << "--> " << morceau.album << " | "
                << morceau.compositeur << std::endl;
        }
        std::cout << "\t/--> " << morceau.nom << std::endl;

        album_precedent = morceau.album;
    }
}

void affichage_artiste(Discographie const & discographie)
{
    Artiste artiste_precedent{};
    Album album_precedent{};

    for (Morceau const & morceau : discographie)
    {
        if (morceau.compositeur.nom != artiste_precedent.nom)
        {
            std::cout << "--> " << morceau.compositeur << std::endl;
        }

        if (morceau.album.nom != album_precedent.nom)
        {
            std::cout << "\t/--> " << morceau.album << std::endl;
        }

        std::cout << "\t\t/--> " << morceau.nom << std::endl;

        artiste_precedent = morceau.compositeur;
        album_precedent = morceau.album;
    }
}

```

AFFICHAGE DE LA DISCOGRAPHIE

Prototypes

```
enum class Affichage { Artiste, Album, Morceau };
void affichage(Discographie& discographie, Affichage
type_affichage);
```

Implémentation

```
// L'ensemble des possibilités d'affichage de la discographie.
enum class Affichage { Artiste, Album, Morceau };

// Fonction liant ce qui précède pour gérer toute la logique
d'affichage.
void affichage(Discographie& discographie, Affichage
type_affichage)
{
    if (type_affichage == Affichage::Album)
    {
        tri_album(discographie);
        affichage_album(discographie);
    }
    else if (type_affichage == Affichage::Artiste)
    {
        tri_artiste(discographie);
        affichage_artiste(discographie);
    }
    else if (type_affichage == Affichage::Morceau)
    {
        tri_morceau(discographie);
        affichage_morceau(discographie);
    }
    else
    {
        // Par exemple si on met à jour l'énumération mais qu'on oublie
        // d'ajouter la fonction correspondante.
        throw std::runtime_error("Commande d'affichage inconnue.");
    }
}
```

On utilise ici une exception qui ne devrait en principe jamais être lancée, car nous ne tomberons jamais dans le `else`. Mais si, plus tard, on modifiait le programme pour ajouter une nouvelle forme d'affichage, sans mettre à jour cette partie du code, l'exception nous le rappellerait.

SYSTÈME DE COMMANDES

Prototypes

```
// L'ensemble des actions qu'il est possible de faire.
enum class Commande { Afficher, Ajouter, Enregistrer, Charger,
    Quitter };

// Affichage et récupération de ce que l'utilisateur écrit.
std::string recuperer_commande();
// Analyse du texte reçu en entrée.
std::tuple<Commande, std::string>
analyser_commande(std::string const & commande_texte);
// La commande à exécuter ainsi que la suite des instructions.
bool executer_commande(Discographie & discographie, Commande
    commande, std::string const & instructions);
```

Remarquez la création d'une énumération pour représenter les différents types de commandes ; ce n'est pas nécessaire, c'est juste un type de base de plus pour pouvoir lire élégamment le type de commande dans la console.

Implémentations

```
// Récupération de ce que l'utilisateur écrit.
std::string recuperer_commande()
{
    std::cout << "> ";

    std::string commande {};
    std::getline(std::cin, commande);
    return commande;
}

// L'ensemble des actions qu'il est possible de faire.
enum class Commande { Afficher, Ajouter, Enregistrer, Charger,
    Quitter };

// Analyse du texte reçu en entrée.
std::tuple<Commande, std::string>
analyser_commande(std::string const & commande_texte)
{
    // Pour traiter la chaîne comme étant un flux,
    // afin d'en extraire les données.
    std::istringstream flux { commande_texte };

    std::string premier_mot {};
    flux >> premier_mot;
    premier_mot = traitement_chaine(premier_mot);
```

SYSTÈME DE COMMANDES

```

std::string instructions {};
std::getline(flux, instructions);
instructions = traitement_chaine(instructions);

if (premier_mot == "afficher")
{
    // Le mode d'affichage.
    return { Commande::Afficher, instructions };
}
else if (premier_mot == "ajouter")
{
    // Les informations à ajouter.
    return { Commande::Ajouter, instructions };
}
else if (premier_mot == "enregistrer")
{
    // Le fichier à écrire.
    return{ Commande::Enregistrer, instructions };
}
else if (premier_mot == "charger")
{
    // Le fichier à lire.
    return { Commande::Charger, instructions };
}
else if (premier_mot == "quitter")
{
    // Chaîne vide, car on quitte le programme sans autre
    instruction.
    return { Commande::Quitter, std::string {} };
}
else
{
    // On a reçu du texte qui est incorrect.
    throw std::runtime_error("Commande invalide.");
}
}

// La commande à exécuter ainsi que la suite des instructions.
bool executer_commande(Discographie & discographie, Commande
commande, std::string const & instructions)
{
    if (commande == Commande::Afficher)
    {
        if (instructions == "artistes")
        {
            affichage(discographie, Affichage::Artiste);
        }
        else if (instructions == "albums")
        {
            affichage(discographie, Affichage::Album);
        }
    }
}

```

SYSTÈME DE COMMANDES

```

    else if (instructions == "morceaux")
    {
        affichage(discographie, Affichage::Morceau);
    }
    else
    {
        // Si on se trouve ici, alors c'est qu'il y a une erreur
        // dans les instructions d'affichage.
        throw std::runtime_error("Commande d'affichage inconnue.");
    }
}
else if (commande == Commande::Ajouter)
{
    std::istringstream flux { instructions };
    Morceau morceau {};
    flux >> morceau;
    discographie.push_back(morceau);
}
else if (commande == Commande::Charger)
{
    chargement(discographie, instructions);
}
else if (commande == Commande::Enregistrer)
{
    enregistrement(discographie, instructions);
}
else if (commande == Commande::Quitter)
{
    // Plus rien à faire, on quitte.
    return false;
}

return true;
}

```

Comme on l'a dit, on utilise les exceptions pour faire remonter les erreurs en cas de commande invalide. C'est un usage logique des exceptions, puisque les erreurs de commande de l'utilisateur sont indépendantes de la volonté du programmeur. Les exceptions sont donc toutes indiquées pour gérer ce genre de cas.

FINALISATION

Fonction principale

```
int main()
{
    // Lancement préalable des tests unitaires.
    test_creation_morceau_entree_complexe();
    test_creation_morceau_entree_espaces_partout();
    test_creation_morceau_entree_chanson_artiste();
    test_creation_morceau_entree_chanson_uniquement();
    test_creation_morceau_entree_vide();

    // On préremplit la discographie.
    Artiste const bob_le_chanteur { "Bob le chanteur " };
    Artiste const alice_la_chanteuse { "Alice la chanteuse" };
    Artiste const papa_cuistot { "Papa cuistot" };

    Album const mes_supers_chansons { "Mes supers chansons" };
    Album const comptines_alice { "Comptines d'Alice" };
    Album const cuisine_en_musique { "Cuisine en musique" };

    Morceau const chanson_a { "Chanson A", bob_le_chanteur,
    mes_supers_chansons };
    Morceau const chanson_b { "Chanson B", bob_le_chanteur,
    mes_supers_chansons };
    Morceau const un_chat_qui_dormait { "Un chat qui dormait",
    alice_la_chanteuse, comptines_alice };
    Morceau const amour { "L'amour", alice_la_chanteuse,
    comptines_alice };
    Morceau const mousse_au_chocolat { "La mousse au chocolat",
    papa_cuistot, cuisine_en_musique};

    Discographie discographie { chanson_a, chanson_b,
    un_chat_qui_dormait, amour, mousse_au_chocolat };

    bool continuer { true };
    do
    {
        try
        {
            std::string entree { recuperer_commande() };
            auto[commande, instructions] = analyser_commande(entree);
            instructions = traitement_chaine(instructions);
            continuer = executer_commande(discographie, commande,
            instructions);
        }
        catch (std::runtime_error const & exception)
        {
            std::cout << "Erreur : " << exception.what() << std::endl;
        }
    } while (continuer);
}
```

FINALISATION

```
    return 0;  
}
```

Il ne faut pas oublier de gérer les erreurs remontées par notre programme. Ici ce n'est pas compliqué, il suffit d'expliquer l'erreur à l'utilisateur et de lui redemander une entrée.

17

Découpons du code — les fichiers

Lors du chapitre précédent, nous avons codé notre premier programme d'assez grande ampleur. Et avec l'ampleur vient un problème qui jusque-là était caché par la taille réduite de nos premiers essais : l'organisation du code. En effet, nous avons écrit tout notre code dans le fichier `main.cpp`. Le plus sophistiqué qu'on ait entrepris en matière d'organisation est d'utiliser les `prototypes` pour pouvoir reléguer les implémentations après la fonction `main` et ainsi alléger un peu la lecture du code. Et cela suffisait. Mais on le voit avec notre gestionnaire de discographie, dès qu'un projet prend un peu d'ampleur, ça devient vite un sacré fouillis. Et encore, ce n'est rien comparé à certains gros projets. Par exemple, à l'heure où nous écrivons ces lignes, le code du noyau Linux comporte 17 millions de lignes !

C'est pourquoi nous allons maintenant apprendre comment séparer son code en plusieurs fichiers, ce qui va nous permettre de mieux l'organiser.

17.1. Le principe

Nous savons séparer les prototypes de leur implémentation, ce qui est déjà une première étape vers une réorganisation. Mais ce n'est pas suffisant. Il faut maintenant séparer le code en plusieurs fichiers. Et il en existe deux types.

Le fichier d'en-tête

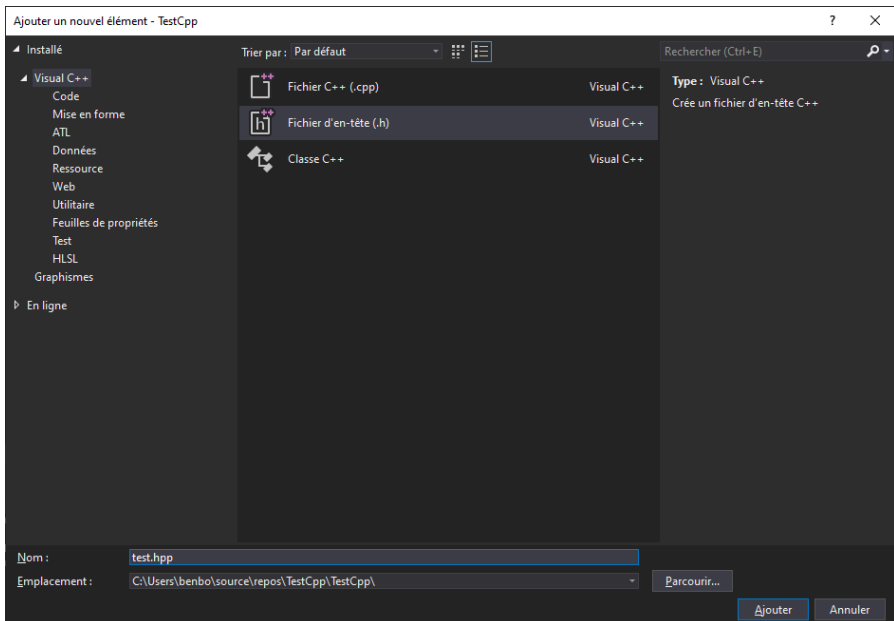
Les fichiers d'en-tête contiennent les déclarations des types et fonctions que l'on souhaite créer. Nous sommes habitués à les manipuler depuis le début de notre aventure avec C++. C'est grâce à eux que nous sommes en mesure d'utiliser la bibliothèque standard. Le concept n'est pas nouveau, on passe simplement de consommateur à créateur.

Les fichiers d'en-tête peuvent tout à fait inclure d'autres fichiers d'en-tête, tant standards que personnels. C'est le cas par exemple lorsque notre fichier d'en-tête fait appel à des composants de la bibliothèque standard comme `std::string` ou `std::vector`.

Sur Visual Studio

Ouvrez votre projet avec Visual Studio. Cliquez droit sur le nom du projet (ici, TestCpp), puis sur AJOUTER > NOUVEL ÉLÉMENT.... Vous arrivez alors sur l'écran de la [Figure 17.1](#).

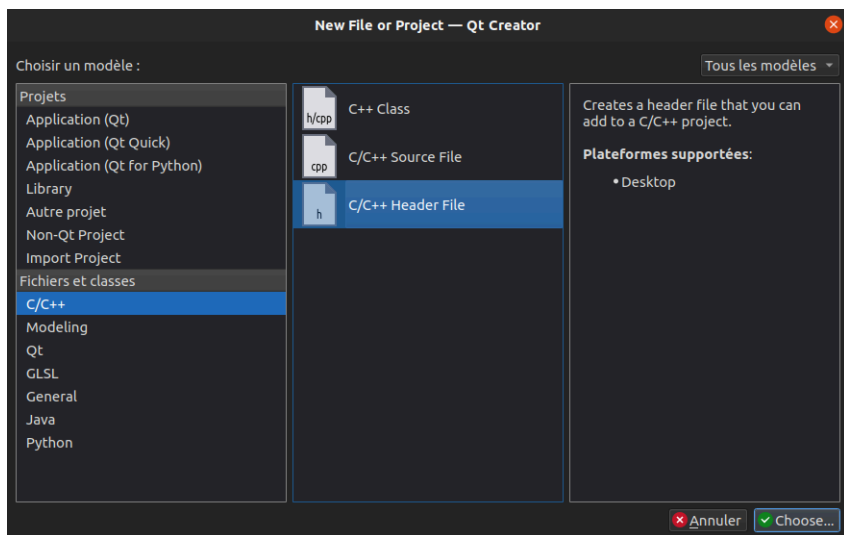
Figure 17.1 : Ajout d'un fichier d'en-tête



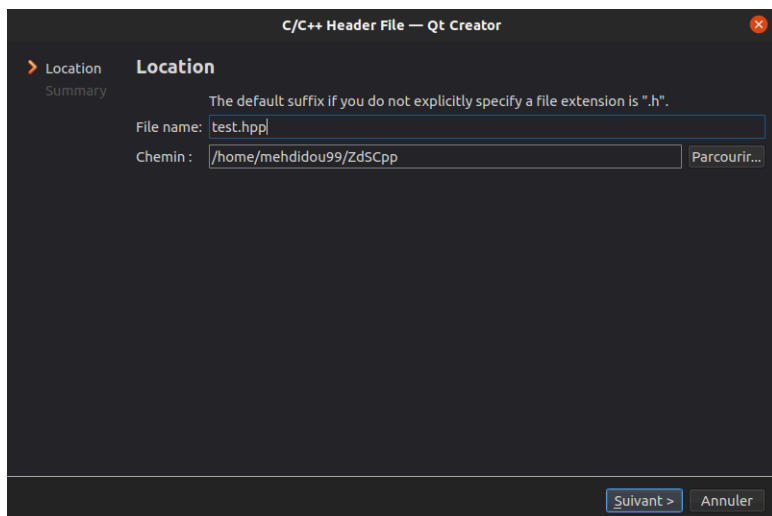
Choisissez bien FICHIER D'EN-TÊTE (.H) puis donnez un nom à votre fichier. Ici, nous avons choisi test .hpp. Notez que nous avons changé l'extension, de .h vers .hpp, et nous vous invitons à faire de même. Enfin, cliquez sur AJOUTER. Vous arriverez sur l'écran principal, avec le fichier ajouté au projet.

Sur Qt Creator

Ouvrez votre projet avec Qt Creator. Cliquez droit sur le nom du projet, puis sur ADD NEW.... Vous arrivez sur la fenêtre de la [Figure 17.2](#).

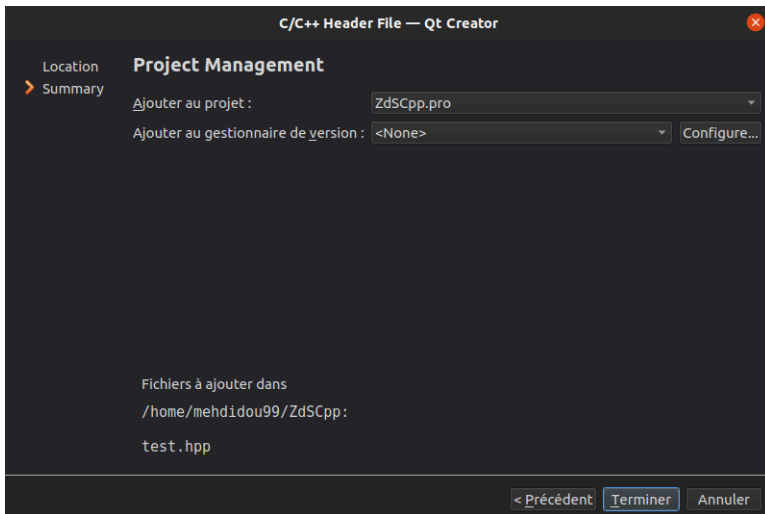
Figure 17.2 : Création d'un fichier d'en-tête

Cliquez sur C/C++ HEADER FILE, puis sur CHOOSE... en bas à droite. Vous passez au deuxième écran.

Figure 17.3 : Nom du fichier

Maintenant, donnez un nom à votre fichier. Ici, pour l'exemple, nous l'avons appelé `test.hpp`. Cliquez ensuite sur SUIVANT et vous arriverez sur le troisième écran.

Figure 17.4 : Récapitulatif des éléments de création



Tout est bon, il ne reste plus qu'à cliquer sur TERMINER et nous en avons fini avec l'ajout de fichiers d'en-tête. On retombe sur la page principale : le fichier apparaît désormais dans l'arborescence du projet.

En ligne de commande

Rien de compliqué, il suffit de créer un simple fichier `.hpp` et c'est tout.

Le fichier source

Le fichier source va contenir les implémentations de nos fonctions. La création d'un fichier source est exactement identique à celle de la création d'un fichier d'en-tête.

Un fichier source peut inclure autant de fichiers d'en-tête que nécessaire. Il y a bien sûr celui qui contient les prototypes et les types qu'il définit, mais il peut en inclure d'autres.

Créons un lien

Il reste une étape importante : lier les fichiers d'en-tête et sources. Pour cela, il suffit d'inclure le fichier d'en-tête concerné dans le fichier source concerné. Cependant, à la différence des fichiers de la bibliothèque standard qui s'incluent avec des chevrons, on utilise ici les guillemets doubles ". On doit aussi écrire le nom complet, *extension comprise*. Celle-ci est `.hpp`.

Ainsi, si vous nommez votre fichier `test.hpp`, vous l'incluez dans le code en faisant comme suit.

```
#include "test.hpp"
```

***Note** > Les fichiers d'en-tête peuvent aussi utiliser l'extension `.h`. C'est parfaitement valide. La différence n'est qu'une histoire de goût. Nous préférons `.hpp` à `.h` par analogie aux fichiers sources, qui se terminent par `.cpp` en C++ mais `.c` en C.*

Voici un exemple tout bête mais fonctionnel de séparation en plusieurs fichiers. Pour le reproduire, créez les fichiers concernés dans votre projet et copiez-collez les codes ci-dessous.

`test.hpp`

```
// Parce que le compilateur a besoin de savoir ce qu'est std::string.
#include <string>

// On ne met ici que le prototype.
void afficher_message(std::string const & message);
```

`test.cpp`

```
// On a besoin de iostream pour utiliser std::cout,
// std::quoted et std::endl.
#include <iostream>
// On inclut le fichier d'en-tête correspondant.
#include "test.hpp"

// Par contre, comme 'test.hpp' inclut déjà '<string>',
// on n'a pas besoin de l'inclure de nouveau ici,
// dans le fichier source.

// On définit la fonction ici.
void afficher_message(std::string const & message)
{
    std::cout << "J'ai reçu un message : "
               << std::quoted(message) << std::endl;
}
```

main.cpp

```
// Pour appeler la fonction, il faut connaître son prototype,
// donc on doit inclure le fichier d'en-tête correspondant.
#include "test.hpp"

int main()
{
    // Ici, on va utiliser la fonction.
    afficher_message("Salut les lecteurs !");
    return 0;
}
```

Pour compiler ce programme avec Visual Studio ou Qt Creator, rien de bien compliqué, on lance la compilation et c'est tout. Pour ceux qui sont en console, il faut ajouter chaque fichier .cpp à la liste des fichiers à compiler. Quant aux .hpp, il ne faut pas les inclure, nous verrons pourquoi [Section 20.1, Le préprocesseur](#).

L'exemple suivant se compile ainsi.

```
> g++ -std=c++20 test.cpp main.cpp -o programme.out
> clang++ -std=c++20 test.cpp main.cpp -o programme.out
```

17.2. La sécurité, c'est important

Avez-vous noté ces lignes nouvelles qui étaient automatiquement présentes lors de la création du fichier d'en-tête ? Ce sont surtout celles de Qt Creator qui nous intéressent (celle de Visual Studio n'est pas du C++ standard). Nous vous les réécrivons ci-dessous.

```
#ifndef TEST_HPP
#define TEST_HPP

#endif // TEST_HPP
```

C'est qu'en fait, lorsque le compilateur voit une ligne `#include`, il ne réfléchit pas et inclut le fichier en question. Sauf que, si, par erreur, ou même volontairement, on inclut plusieurs fois le même fichier, on va se retrouver avec plusieurs fois les mêmes déclarations de fonctions, de types, etc. Autant dire que le compilateur ne va que très moyennement apprécier.

Afin de s'en prémunir, il faut écrire ces trois lignes, qui sont des directives de préprocesseur et ce dans chaque fichier d'en-tête que vous créerez. Vous pouvez remplacer `TEST_HPP` par ce que vous voulez, à condition que ce nom soit unique. Le plus simple

est de le remplacer par le nom du fichier en question : s'il s'appelle `autre.hpp`, alors écrivez `AUTRE_HPP`. Le code de l'en-tête ira entre la deuxième et la troisième ligne.

Ainsi, le `fichier d'en-tête` de l'exemple de la section précédente n'est pas correct. Il faut le corriger en utilisant ce que nous venons d'apprendre.

```
#ifndef TEST_HPP
#define TEST_HPP

#include <string>

void afficher_message(std::string const & message);

#endif
```

17.3. [Pratique] Découpons notre programme !

Pour nous exercer, nous allons prendre le `code de notre gestionnaire de discographie` et le réorganiser en plusieurs fichiers. Commençons par le commencement, séparons prototypes et implémentations. Nous y verrons ainsi plus clair une fois que nous aurons uniquement les prototypes sous les yeux.

✧ Voir séparation prototypes/implémentations

On peut former des groupes de types et fonctions liées :

- les fonctions utilitaires : pour l'instant, nous n'avons que `traitement_chaine`, mais d'autres pourraient la rejoindre par la suite lors de l'évolution du projet ;
- les types de base `Artiste`, `Album` et `Morceau` et les fonctions les manipulant ;
- les tests unitaires correspondants ;
- le type `Discographie` et ses fonctions de manipulation ;
- les types et fonctions concernant l'exécution des commandes.

Nous allons donc créer cinq paires de fichiers pour réorganiser notre code en conséquence : `utils`, `donnees_disco`, `donnees_disco_tests`, `discographie`, `systeme_commandes`. Cela donne les fichiers suivants.

Note > Comme d'habitude, il n'y a pas qu'une seule façon de faire en programmation. L'organisation que nous proposons en est une, mais l'essentiel est juste d'avoir un code organisé de manière cohérente.

✧ Utilitaire : `utils.hpp/utils.cpp`

✧ Données discographie : `donnees_disco.cpp/donnees_disco.hpp`

- ✧ Tests unitaires : `donnees_disco_tests.cpp/donnees_disco_tests.hpp`
- ✧ Discographie: `discographie.hpp/discographie.cpp`
- ✧ Commandes : `systeme_commande.hpp/systeme_commande.cpp`

Notre fichier `main.cpp` devient alors très simple.

```
#include "donnees_disco_tests.hpp"
#include "discographie.hpp"
#include "systeme_commandes.hpp"
#include "utils.hpp"

#include <stdexcept>

int main()
{
    test_creation_morceau_entree_complete();
    test_creation_morceau_entree_espaces_partout();
    test_creation_morceau_entree_chanson_artiste();
    test_creation_morceau_entree_chanson_uniquement();
    test_creation_morceau_entree_vide();

    // On préremplit la discographie.
    Artiste const bob_le_chanteur { "Bob le chanteur " };
    Artiste const alice_la_chanteuse { "Alice la chanteuse" };
    Artiste const papa_cuistot { "Papa cuistot" };

    Album const mes_supers_chansons { "Mes supers chansons" };
    Album const comptines_alice { "Comptines d'Alice" };
    Album const cuisine_en_musique { "Cuisine en musique" };

    Morceau const chanson_a {
        "Chanson A",
        bob_le_chanteur,
        mes_supers_chansons
    };
    Morceau const chanson_b {
        "Chanson B",
        bob_le_chanteur,
        mes_supers_chansons
    };
    Morceau const un_chat_qui_dormait {
        "Un chat qui dormait",
        alice_la_chanteuse,
        comptines_alice
    };
    Morceau const amour {
        "L'amour",
        alice_la_chanteuse,
        comptines_alice
    };
    Morceau const mousse_au_chocolat {
        "La mousse au chocolat",
```



```

    papa_cuistot,
    cuisine_en_musique
};

Discographie discographie {
    chanson_a,
    chanson_b,
    un_chat_qui_dormait,
    amour,
    mousse_au_chocolat
};

bool continuer { true };
do
{
    try
    {
        std::string entree { recuperer_commande() };
        auto[commande, instructions] = analyser_commande(entree);
        instructions = traitement_chaine(instructions);
        continuer = executer_commande(discographie, commande,
instructions);
    }
    catch (std::runtime_error const & exception)
    {
        std::cout << "Erreur : "
        << exception.what() << std::endl;
    }

} while (continuer);

return 0;
}

```

Et voilà ! Nous avons un code organisé en plusieurs fichiers, prêt à évoluer !

17.4. Les avantages du découpage en fichiers

Le découpage en fichiers permet de mieux organiser le code, mais ce n'est pas tout ; il présente d'autres avantages que nous allons voir maintenant.

Une structure de projet plus apparente

Imaginez un projet plus conséquent encore que notre gestionnaire de discographie précédent. Tout est dans le même fichier. Pas facile d'y voir un ordre quelconque. Si, par contre, le projet est découpé intelligemment en plusieurs fichiers, vous pourrez, d'un coup d'œil, comprendre sa structure et mieux orienter vos recherches.

Le projet [Zeste de Savoir](#), par exemple, est découpé en plein de fichiers, qui séparent les tests, l'affichage, la manipulation de la base de données, etc. Un déve-

veloppeur ayant des connaissances dans les technologies utilisées sait donc, d'un rapide regard, où il doit orienter ses recherches.

Une meilleure abstraction

L'avantage de séparer l'implémentation des prototypes et des définitions de type, notamment avec les fichiers d'en-tête, c'est qu'on peut masquer toute la complexité et les détails au développeur qui utilisera notre code. Lui ne verra que ce qu'on choisit d'exposer. Si besoin est, on pourra changer du tout au tout la façon d'écrire nos fonctions, pour les rendre plus sûres ou plus rapides et le développeur n'y verra rien.

Vous, apprenti développeur C++, dépendez des prototypes et abstractions de la bibliothèque standard. Par contre, en coulisse, l'implémentation n'est pas la même selon que vous utilisez Visual Studio, GCC ou Clang. Mais ça n'est pas un problème. Nous dépendons d'abstractions, non de détails techniques. C'est un des [grands principes](#) d'un programme bien conçu.

De plus, comme les détails techniques sont cachés dans les fichiers sources, les fichiers d'en-tête sont bien plus lisibles et n'exposent que les fonctions et données publiques dont a besoin l'utilisateur.

Une meilleure modularité

Séparer en plusieurs fichiers, c'est aussi autoriser des portions de code à être réutilisées à d'autres endroits. Vous l'avez déjà expérimenté en faisant des `#include` d'en-tête de la bibliothèque standard. Il serait plus difficile et bien plus lourd d'utiliser `std::string` ou `std::vector` s'il n'y avait pas les fichiers d'en-tête correspondants, n'est-ce pas ?

Pour le gestionnaire de discographie, rien que le fait d'avoir séparé la fonction de traitement des chaînes de caractères et l'avoir placée dans un fichier à part nous autorise à la réutiliser ailleurs que ce qu'on avait originellement prévu.

17.5. Le cas des templates

Il nous reste un cas particulier à aborder, celui des *templates*. Dans le chapitre [Envoyez le générique I](#), nous avons dit que le compilateur instanciat une fonction *template* chaque fois qu'elle était appelée avec un type réel différent. Mais pour l'instancier, le compilateur a besoin de son implémentation complète. Du coup, un code comme celui-ci ne peut pas fonctionner.

neutre conteneur.hpp

```
#ifndef CONTENEUR_HPP
#define CONTENEUR_HPP

#include <iostream>

template <typename Collection>
void afficher(Collection const & iterable);

#endif
```

neutre conteneur.cpp

```
#include "conteneur.hpp"

template <typename Collection>
void afficher(Collection const & iterable)
{
    for (auto const & e : iterable)
    {
        std::cout << e << std::endl;
    }
}
```

neutre main.cpp

```
#include <iostream>
#include <vector>
#include "conteneur.hpp"

int main()
{
    std::vector<int> const tableau { 1, 2, 3, 4 };
    afficher(tableau);
    return 0;
}
```

```
[Visual Studio]
Erreur LNK2019 symbole externe non résolu "void __cdecl
afficher<class std::vector<int,class std::allocator<int> >
>(class std::vector<int,class std::allocator<int> > const &)" (??
$aafficher@V?$vector@HV?$allocator@H@std@@@std@@@YAXABV?$vector@HV?
$allocator@H@std@@@std@@@Z) référencé dans la fonction _main

-----

[GCC]
/usr/bin/ld: /tmp/cc7jhqie.o: in function `main':
main.cpp:(.text+0x6e): undefined reference to `void
afficher<std::vector<int, std::allocator<int> > >(std::vector<int,
std::allocator<int> > const&)'
collect2: error: ld a retourné le statut de sortie 1

-----

[Clang]
/usr/bin/ld: /tmp/main-27208d.o: in function `main':
main.cpp:(.text+0x80): undefined reference to `void
afficher<std::vector<int, std::allocator<int> > >(std::vector<int,
std::allocator<int> > const&)'
clang-7: error: linker command failed with exit code 1 (use -v to see
invocation)
```

Le compilateur va en effet se plaindre (avec une erreur cryptique, soit dit en passant), car il essaye d'instancier le template `afficher` mais ne trouve nulle part son implémentation. Afin de compiler, il faut que toutes nos fonctions templates soient déclarées et implémentées directement dans les fichiers d'en-tête. C'est contraire à ce que nous venons d'apprendre dans ce chapitre, mais c'est bien ce qu'il faut faire.

neutre conteneur.hpp

```
#ifndef CONTENEUR_HPP
#define CONTENEUR_HPP

#include <iostream>

template <typename Collection>
void afficher(Collection const & iterable)
{
    for (auto const & e : iterable)
    {
        std::cout << e << std::endl;
    }
}

#endif
```

```
1
2
3
4
```



Il n'y a vraiment pas d'autres solutions ? Parce que tu as dit toi-même que c'était bien que les détails techniques restent cachés dans les fichiers sources.

Eh bien, figurez-vous que si ! En fait, il y a une astuce toute simple, qui consiste à déclarer les fonctions dans le fichier d'en-tête, mais à les implémenter dans un fichier séparé (souvent avec l'extension `.hpp`), fichier qui sera inclus à la fin du fichier d'en-tête. Ainsi, on cache les détails techniques dans un autre fichier tout en laissant la compilation se faire.

neutre conteneur_impl.hpp

```
// Implémentation des fonctions templates.

template <typename Collection>
void afficher(Collection const & iterable)
{
    for (auto const & e : iterable)
    {
        std::cout << e << std::endl;
    }
}
```

neutre conteneur.hpp

```
#ifndef CONTENEUR_HPP
#define CONTENEUR_HPP

#include <iostream>

template <typename Collection>
void afficher(Collection const & iterable);

// A la fin, on rajoute le fichier d'implémentation.
#include "conteneur_impl.hpp"

#endif
```

neutre main.cpp

```
#include <iostream>
#include <vector>
```

```
// Le fichier d'implémentation est lui aussi inclus, du coup.
#include "conteneur.hpp"

int main()
{
    std::vector<int> const tableau { 1, 2, 3, 4 };
    afficher(tableau);
    return 0;
}
```

```
1
2
3
4
```

17.6. Récapitulons !

- Découper en fichiers permet de séparer les définitions des implémentations.
- Les fichiers d'en-tête contiennent les définitions et portent, dans ce livre, l'extension `.hpp`.
- Les fichiers sources contiennent les implémentations et les détails techniques. Ils terminent par l'extension `.cpp`.
- Lors de la création de fichiers d'en-tête, il faut bien penser à les sécuriser avec les directives de préprocesseur abordées.
- Découper son projet en fichiers permet une meilleure modularité, une meilleure réutilisation, une structure de projet plus visible.
- Dans le cas des templates, l'implémentation doit être écrite soit directement dans le fichier d'en-tête, soit dans un fichier séparé qui lui-même sera inclus dans le fichier d'en-tête concerné.

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

Séparation prototypes/implémentations

(retour à la Section 17.3, [Pratique] Découpons notre programme !)

```
#include <algorithm>
#include <cassert>
#include <cctype>
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <tuple>
#include <vector>

std::string traitement_chaine(std::string const & chaine);

struct Artiste
{
    std::string nom;
};

std::ostream & operator<<(std::ostream & sortie, Artiste const &
    artiste);

struct Album
{
    std::string nom;
};

std::ostream & operator<<(std::ostream & sortie, Album const &
    album);

struct Morceau
{
    std::string nom;
    Artiste compositeur;
    Album album;
};

std::ostream & operator<<(std::ostream & sortie, Morceau const &
    morceau);
std::istream & operator>>(std::istream & entree, Morceau &
    morceau);

void test_creation_morceau_entree_complete();
void test_creation_morceau_entree_espaces_partout();
void test_creation_morceau_sans_album();
void test_creation_morceau_entree_chanson_uniquement();
void test_creation_morceau_entree_vide();

using Discographie = std::vector<Morceau>;
```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

void tri_morceau(Discographie& discographie);
void tri_album(Discographie& discographie);
void tri_artiste(Discographie& discographie);

void affichage_morceau(Discographie const & discographie);
void affichage_album(Discographie const & discographie);
void affichage_artiste(Discographie const & discographie);

enum class Affichage { Artiste, Album, Morceau };

void affichage(Discographie& discographie, Affichage
    type_affichage);

void enregistrement(Discographie const & discographie,
    std::string const & nom_fichier);
void chargement(Discographie & discographie, std::string const &
    nom_fichier);

std::string recuperer_commande();

enum class Commande { Afficher, Ajouter, Enregistrer, Charger,
    Quitter };

std::tuple<Commande, std::string>
    analyser_commande(std::string const & commande_texte);
bool executer_commande(Discographie & discographie, Commande
    commande, std::string const & instructions);

int main()
{
    test_creation_morceau_entree_complete();
    test_creation_morceau_entree_espaces_partout();
    test_creation_morceau_sans_album();
    test_creation_morceau_entree_chanson_uniquement();
    test_creation_morceau_entree_vide();

    // On préremplit la discographie.
    Artiste const bob_le_chanteur { "Bob le chanteur " };
    Artiste const alice_la_chanteuse { "Alice la chanteuse" };
    Artiste const papa_cuistot { "Papa cuistot" };

    Album const mes_supers_chansons { "Mes supers chansons" };
    Album const comptines_alice { "Comptines d'Alice" };
    Album const cuisine_en_musique { "Cuisine en musique" };

    Morceau const chanson_a { "Chanson A", bob_le_chanteur,
        mes_supers_chansons };
    Morceau const chanson_b { "Chanson B", bob_le_chanteur,
        mes_supers_chansons };
    Morceau const un_chat_qui_dormait { "Un chat qui dormait",
        alice_la_chanteuse, comptines_alice };

```


RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

    Morceau const amour { "L'amour", alice_la_chanteuse,
    comptines_alice };
    Morceau const mousse_au_chocolat { "La mousse au chocolat",
    papa_cuistot, cuisine_en_musique};

    Discographie discographie { chanson_a, chanson_b,
    un_chat_qui_dormait, amour, mousse_au_chocolat };

    bool continuer { true };
    do
    {
        try
        {
            std::string entree { recuperer_commande() };
            auto[commande, instructions] = analyser_commande(entree);
            instructions = traitement_chaine(instructions);
            continuer = executer_commande(discographie, commande,
instructions);
        }
        catch (std::runtime_error const & exception)
        {
            std::cout << "Erreur : " << exception.what() << std::endl;
        }

    } while (continuer);

    return 0;
}

std::string traitement_chaine(std::string const & chaine)
{
    std::string copie { chaine };

    auto premier_non_espace { std::find_if_not(std::begin(copie),
std::end(copie), isspace) };
    copie.erase(std::begin(copie), premier_non_espace);

    std::reverse(std::begin(copie), std::end(copie));
    premier_non_espace = std::find_if_not(std::begin(copie),
std::end(copie), isspace);
    copie.erase(std::begin(copie), premier_non_espace);
    std::reverse(std::begin(copie), std::end(copie));

    return copie;
}

std::ostream & operator<<(std::ostream & sortie, Artiste const &
artiste)
{
    sortie << artiste.nom;
    return sortie;
}

```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

std::ostream & operator<<(std::ostream & sortie, Album const &
album)
{
    sortie << album.nom;
    return sortie;
}

std::ostream & operator<<(std::ostream & sortie, Morceau const &
morceau)
{
    sortie << morceau.nom << " | " << morceau.album << " | " <<
morceau.compositeur;
    return sortie;
}

std::istream & operator>>(std::istream & entree, Morceau &
morceau)
{
    std::string mot {};
    std::ostringstream flux {};

    while (entree >> mot && mot != "|")
    {
        flux << mot << " ";
    }

    std::string nom_morceau { flux.str() };
    if (std::empty(nom_morceau))
    {
        nom_morceau = "Morceau inconnu";
    }
    morceau.nom = traitement_chaine(nom_morceau);
    flux.str(std::string {});

    while (entree >> mot && mot != "|")
    {
        flux << mot << " ";
    }

    std::string nom_album { flux.str() };
    if (std::empty(nom_album))
    {
        nom_album = "Album inconnu";
    }
    morceau.album.nom = traitement_chaine(nom_album);
    flux.str(std::string {});

    while (entree >> mot)
    {
        flux << mot << " ";
    }
}

```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

    std::string nom_artiste { flux.str() };
    if (std::empty(nom_artiste))
    {
        nom_artiste = "Artiste inconnu";
    }
    morceau.compositeur.nom = traitement_chaine(nom_artiste);
    flux.str(std::string {});

    return entree;
}

void test_creation_morceau_entree_complete()
{
    std::istringstream entree { "Chanson A | Mes supers chansons |
Bob le chanteur" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "Chanson A" && u8"Le nom du morceau doit
être Chanson A.");
    assert(morceau.album.nom == "Mes supers chansons" && u8"Le nom de
l'album doit être Mes supers chansons.");
    assert(morceau.compositeur.nom == "Bob le chanteur" && u8"Le nom
du compositeur doit être Bob le chanteur.");
}

// Test d'une entrée complète avec beaucoup d'espaces.
void test_creation_morceau_entree_espaces_partout()
{
    std::istringstream entree { "Chanson B      | Mes supers chansons
|   Bob le chanteur" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "Chanson B" && u8"Le nom du morceau doit
être Chanson B.");
    assert(morceau.album.nom == "Mes supers chansons" && u8"Le nom de
l'album doit être Mes supers chansons.");
    assert(morceau.compositeur.nom == "Bob le chanteur" && u8"Le nom
du compositeur doit être Bob le chanteur.");
}

// Test d'une entrée avec seulement le nom de la chanson et de
l'artiste.
void test_creation_morceau_entree_chanson_artiste()
{
    std::istringstream entree { "Un chat qui dormait | | Alice la
chanteuse" };
    Morceau morceau {};

```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

    entree >> morceau;

    assert(morceau.nom == "Un chat qui dormait" && u8"Le nom du
morceau doit être Un chat qui dormait.");
    assert(morceau.album.nom == "Album inconnu" && u8"Le nom de
l'album doit être Album inconnu.");
    assert(morceau.compositeur.nom == "Alice la chanteuse" && u8"Le
nom du compositeur doit être Alice la chanteuse.");
}

// Test d'une entrée avec seulement le nom de la chanson.
void test_creation_morceau_entree_chanson_uniquement()
{
    std::istringstream entree { "L'amour | |" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "L'amour" && u8"Le nom du morceau doit être
L'amour.");
    assert(morceau.album.nom == "Album inconnu" && u8"Le nom de
l'album doit être Album inconnu.");
    assert(morceau.compositeur.nom == "Artiste inconnu" && u8"Le nom
du compositeur doit être Artiste inconnu.");
}

// Test d'une entrée vide.
void test_creation_morceau_entree_vide()
{
    std::istringstream entree{ "| |" };
    Morceau morceau{};

    entree >> morceau;

    assert(morceau.nom == "Morceau inconnu" && u8"Le nom du morceau
doit être Morceau inconnu.");
    assert(morceau.album.nom == "Album inconnu" && u8"Le nom de
l'album doit être Album inconnu.");
    assert(morceau.compositeur.nom == "Artiste inconnu" && u8"Le nom
du compositeur doit être Artiste inconnu.");
}

void tri_morceau(Discographie& discographie)
{
    std::sort(std::begin(discographie), std::end(discographie), []
(Morceau const & lhs, Morceau const & rhs)
    {
        return lhs.nom < rhs.nom;
    });
}

```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

void tri_album(Discographie& discographie)
{
    std::sort(std::begin(discographie), std::end(discographie), []
(Morceau const & lhs, Morceau const & rhs)
    {
        if (lhs.album.nom < rhs.album.nom)
            return true;

        return lhs.album.nom == rhs.album.nom && lhs.nom < rhs.nom;
    });
}

void tri_artiste(Discographie& discographie)
{
    std::sort(std::begin(discographie), std::end(discographie), []
(Morceau const & lhs, Morceau const & rhs)
    {
        if (lhs.compositeur.nom < rhs.compositeur.nom)
            return true;

        if (lhs.compositeur.nom == rhs.compositeur.nom)
        {
            if (lhs.album.nom < rhs.album.nom)
                return true;

            return lhs.album.nom == rhs.album.nom && lhs.nom < rhs.nom;
        }

        return false;
    });
}

// Affichage par morceau, dans l'ordre alphabétique.
void affichage_morceau(Discographie const & discographie)
{
    for (Morceau const & morceau : discographie)
    {
        std::cout << "--> " << morceau << std::endl;
    }
}

// Affichage par album, dans l'ordre alphabétique.
void affichage_album(Discographie const & discographie)
{
    Album album_precedent{};
    for (Morceau const & morceau : discographie)
    {
        if (morceau.album.nom != album_precedent.nom)
        {
            std::cout << "--> " << morceau.album << "
" << morceau.compositeur << std::endl;
        }
    }
}

```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

        std::cout << "\t/--> " << morceau.nom << std::endl;

        album_precedent = morceau.album;
    }
}

// Affichage par artiste, dans l'ordre alphabétique.
void affichage_artiste(Discographie const & discographie)
{
    Artiste artiste_precedent{};
    Album album_precedent{};

    for (Morceau const & morceau : discographie)
    {
        if (morceau.compositeur.nom != artiste_precedent.nom)
        {
            std::cout << "--> " << morceau.compositeur << std::endl;
        }

        if (morceau.album.nom != album_precedent.nom)
        {
            std::cout << "\t/--> " << morceau.album << std::endl;
        }

        std::cout << "\t\t/--> " << morceau.nom << std::endl;

        artiste_precedent = morceau.compositeur;
        album_precedent = morceau.album;
    }
}

void affichage(Discographie& discographie, Affichage
type_affichage)
{
    if (type_affichage == Affichage::Album)
    {
        tri_album(discographie);
        affichage_album(discographie);
    }
    else if (type_affichage == Affichage::Artiste)
    {
        tri_artiste(discographie);
        affichage_artiste(discographie);
    }
    else if (type_affichage == Affichage::Morceau)
    {
        tri_morceau(discographie);
        affichage_morceau(discographie);
    }
}

```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

        else
        {
            // Par exemple si on met à jour l'énumération mais qu'on oublie
            d'ajouter la fonction correspondante.
            throw std::runtime_error("Commande d'affichage inconnue.");
        }
    }

    void enregistrement(Discographie const & discographie,
        std::string const & nom_fichier)
    {
        std::ofstream fichier { nom_fichier };
        if (!fichier)
        {
            throw std::runtime_error("Impossible d'ouvrir le fichier en
            écriture.");
        }

        for (Morceau const & morceau : discographie)
        {
            fichier << morceau << std::endl;
        }
    }

    void chargement(Discographie & discographie, std::string const &
        nom_fichier)
    {
        std::ifstream fichier { nom_fichier };
        if (!fichier)
        {
            throw std::runtime_error("Impossible d'ouvrir le fichier en
            lecture.");
        }

        std::string ligne {};
        while (std::getline(fichier, ligne))
        {
            Morceau morceau {};

            std::istringstream flux { ligne };
            flux >> morceau;

            discographie.push_back(morceau);
        }
    }

    std::string recuperer_commande()
    {
        std::cout << "> ";

        std::string commande {};
        std::getline(std::cin, commande);
    }

```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

    return commande;
}

std::tuple<Commande, std::string>
analyser_commande(std::string const & commande_texte)
{
    std::istringstream flux { commande_texte };

    std::string premier_mot {};
    flux >> premier_mot;
    premier_mot = traitement_chaine(premier_mot);

    std::string instructions {};
    std::getline(flux, instructions);
    instructions = traitement_chaine(instructions);

    if (premier_mot == "afficher")
    {
        return { Commande::Afficher, instructions };
    }
    else if (premier_mot == "ajouter")
    {
        return { Commande::Ajouter, instructions };
    }
    else if (premier_mot == "enregistrer")
    {
        return { Commande::Enregistrer, instructions };
    }
    else if (premier_mot == "charger")
    {
        return { Commande::Charger, instructions };
    }
    else if (premier_mot == "quitter")
    {
        return { Commande::Quitter, std::string {} };
    }
    else
    {
        throw std::runtime_error("Commande invalide.");
    }
}

bool executer_commande(Discographie & discographie, Commande
commande, std::string const & instructions)
{
    if (commande == Commande::Afficher)
    {
        if (instructions == "artistes")
        {
            affichage(discographie, Affichage::Artiste);
        }
    }
}

```


RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```
    else if (instructions == "albums")
    {
        affichage(discographie, Affichage::Album);
    }
    else if (instructions == "morceaux")
    {
        affichage(discographie, Affichage::Morceau);
    }
    else
    {
        throw std::runtime_error("Commande d'affichage inconnue.");
    }
}
else if (commande == Commande::Ajouter)
{
    std::istringstream flux { instructions };
    Morceau morceau {};
    flux >> morceau;
    discographie.push_back(morceau);
}
else if (commande == Commande::Charger)
{
    chargement(discographie, instructions);
}
else if (commande == Commande::Enregistrer)
{
    enregistrement(discographie, instructions);
}
else if (commande == Commande::Quitter)
{
    return false;
}

return true;
}
```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

Fichiers d'en-tête et fichiers sources

utils.hpp ([retour à la Section 17.3, \[Pratique\] Découpons notre programme !](#))

```
#ifndef UTILS_HPP_INCLUDED
#define UTILS_HPP_INCLUDED

#include <string>

std::string traitement_chaine(std::string const & chaine);

#endif
```

utils.cpp

```
#include "utils.hpp"
#include <algorithm>

std::string traitement_chaine(std::string const & chaine)
{
    std::string copie { chaine };

    auto premier_non_espace { std::find_if_not(std::begin(copie),
std::end(copie), isspace) };
    copie.erase(std::begin(copie), premier_non_espace);

    std::reverse(std::begin(copie), std::end(copie));
    premier_non_espace = std::find_if_not(std::begin(copie),
std::end(copie), isspace);
    copie.erase(std::begin(copie), premier_non_espace);
    std::reverse(std::begin(copie), std::end(copie));

    return copie;
}
```

donnees_disco.hpp ([retour à la Section 17.3, \[Pratique\] Découpons notre programme !](#))

```
#ifndef DONNEES_DISCO_HPP_INCLUDED
#define DONNEES_DISCO_HPP_INCLUDED

#include <string>
#include <iostream>
#include <vector>

struct Artiste
{
    std::string nom;
};
```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```
std::ostream & operator<<(std::ostream & sortie, Artiste const &
    artiste);

struct Album
{
    std::string nom;
};

std::ostream & operator<<(std::ostream & sortie, Album const &
    album);

struct Morceau
{
    std::string nom;
    Artiste compositeur;
    Album album;
};

std::ostream & operator<<(std::ostream & sortie, Morceau const &
    morceau);
std::istream & operator>>(std::istream & entree, Morceau &
    morceau);

using Discographie = std::vector<Morceau>;

#endif
```

donnees_disco.cpp

```
#include "donnees_disco.hpp"
#include "utils.hpp"
#include <sstream>

std::ostream & operator<<(std::ostream & sortie, Artiste const &
    artiste)
{
    sortie << artiste.nom;
    return sortie;
}

std::ostream & operator<<(std::ostream & sortie, Album const &
    album)
{
    sortie << album.nom;
    return sortie;
}

std::ostream & operator<<(std::ostream & sortie, Morceau const &
    morceau)
{
    sortie << morceau.nom << " | " << morceau.album << " | " <<
    morceau.compositeur;
```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

    return sortie;
}

std::istream & operator>>(std::istream & entree, Morceau &
morceau)
{
    std::string mot {};
    std::ostringstream flux {};

    while (entree >> mot && mot != "|")
    {
        flux << mot << " ";
    }

    std::string nom_morceau { flux.str() };
    if (std::empty(nom_morceau))
    {
        nom_morceau = "Morceau inconnu";
    }
    morceau.nom = traitement_chaine(nom_morceau);
    flux.str(std::string {});

    while (entree >> mot && mot != "|")
    {
        flux << mot << " ";
    }

    std::string nom_album { flux.str() };
    if (std::empty(nom_album))
    {
        nom_album = "Album inconnu";
    }
    morceau.album.nom = traitement_chaine(nom_album);
    flux.str(std::string {});

    while (entree >> mot)
    {
        flux << mot << " ";
    }

    std::string nom_artiste { flux.str() };
    if (std::empty(nom_artiste))
    {
        nom_artiste = "Artiste inconnu";
    }
    morceau.compositeur.nom = traitement_chaine(nom_artiste);
    flux.str(std::string {});

    return entree;
}

```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

donnees_disco_tests.hpp (retour à la Section 17.3, [Pratique] Découpons notre programme !)

```
#ifndef DONNEES_DISCO_TESTS_HPP_INCLUDED
#define DONNEES_DISCO_TESTS_HPP_INCLUDED

void test_creation_morceau_entree_complete();
void test_creation_morceau_entree_espaces_partout();
void test_creation_morceau_sans_album();
void test_creation_morceau_entree_chanson_uniquement();
void test_creation_morceau_entree_vide();

#endif
```

donnees_disco_tests.cpp

```
#include "donnees_disco_tests.hpp"
#include <sstream>
#include <cassert>
#include "donnees_disco.hpp"

void test_creation_morceau_entree_complete()
{
    std::istringstream entree { "Chanson A | Mes supers chansons |  
Bob le chanteur" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "Chanson A" && u8"Le nom du morceau doit être Chanson A.");
    assert(morceau.album.nom == "Mes supers chansons" && u8"Le nom de l'album doit être Mes supers chansons.");
    assert(morceau.compositeur.nom == "Bob le chanteur" && u8"Le nom du compositeur doit être Bob le chanteur.");
}

// Test d'une entrée complète avec beaucoup d'espaces.
void test_creation_morceau_entree_espaces_partout()
{
    std::istringstream entree { "Chanson B      | Mes supers chansons  
| Bob le chanteur" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "Chanson B" && u8"Le nom du morceau doit être Chanson B.");
    assert(morceau.album.nom == "Mes supers chansons" && u8"Le nom de l'album doit être Mes supers chansons.");
}
```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

    assert(morceau.compositeur.nom == "Bob le chanteur" && u8"Le nom
    du compositeur doit être Bob le chanteur.");
}

// Test d'une entrée avec seulement le nom de la chanson et de
// l'artiste.
void test_creation_morceau_entree_chanson_artiste()
{
    std::istringstream entree { "Un chat qui dormait | | Alice la
    chanteuse" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "Un chat qui dormait" && u8"Le nom du
    morceau doit être Un chat qui dormait.");
    assert(morceau.album.nom == "Album inconnu" && u8"Le nom de
    l'album doit être Album inconnu.");
    assert(morceau.compositeur.nom == "Alice la chanteuse" && u8"Le
    nom du compositeur doit être Alice la chanteuse.");
}

// Test d'une entrée avec seulement le nom de la chanson.
void test_creation_morceau_entree_chanson_uniquement()
{
    std::istringstream entree { "L'amour | |" };
    Morceau morceau {};

    entree >> morceau;

    assert(morceau.nom == "L'amour" && u8"Le nom du morceau doit être
    L'amour.");
    assert(morceau.album.nom == "Album inconnu" && u8"Le nom de
    l'album doit être Album inconnu.");
    assert(morceau.compositeur.nom == "Artiste inconnu" && u8"Le nom
    du compositeur doit être Artiste inconnu.");
}

// Test d'une entrée vide.
void test_creation_morceau_entree_vide()
{
    std::istringstream entree{ "| |" };
    Morceau morceau{};

    entree >> morceau;

    assert(morceau.nom == "Morceau inconnu" && u8"Le nom du morceau
    doit être Morceau inconnu.");
    assert(morceau.album.nom == "Album inconnu" && u8"Le nom de
    l'album doit être Album inconnu.");
    assert(morceau.compositeur.nom == "Artiste inconnu" && u8"Le nom
    du compositeur doit être Artiste inconnu.");
}

```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

discographie.hpp ([retour à la Section 17.3](#), [\[Pratique\]](#) Découpons notre programme !)

```
#ifndef DISCOGRAPHIE_HPP_INCLUDED
#define DISCOGRAPHIE_HPP_INCLUDED

#include "donnees_disco.hpp"
#include <string>

void tri_morceau(Discographie& discographie);
void tri_album(Discographie& discographie);
void tri_artiste(Discographie& discographie);

void affichage_morceau(Discographie const & discographie);
void affichage_album(Discographie const & discographie);
void affichage_artiste(Discographie const & discographie);

enum class Affichage { Artiste, Album, Morceau };

void affichage(Discographie& discographie, Affichage
    type_affichage);

void enregistrement(Discographie const & discographie,
    std::string const & nom_fichier);
void chargement(Discographie & discographie, std::string const &
    nom_fichier);

#endif
```

discographie.cpp

```
#include "discographie.hpp"
#include <algorithm>
#include <fstream>
#include <sstream>
#include <stdexcept>

void tri_morceau(Discographie& discographie)
{
    std::sort(std::begin(discographie), std::end(discographie), [])
    (Morceau const & lhs, Morceau const & rhs)
    {
        return lhs.nom < rhs.nom;
    });
}

void tri_album(Discographie& discographie)
{
    std::sort(std::begin(discographie), std::end(discographie), [])
    (Morceau const & lhs, Morceau const & rhs)
    {
```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

        if (lhs.album.nom < rhs.album.nom)
            return true;

        return lhs.album.nom == rhs.album.nom && lhs.nom < rhs.nom;
    });
}

void tri_artiste(Discographie& discographie)
{
    std::sort(std::begin(discographie), std::end(discographie), []
(Morceau const & lhs, Morceau const & rhs)
    {
        if (lhs.compositeur.nom < rhs.compositeur.nom)
            return true;

        if (lhs.compositeur.nom == rhs.compositeur.nom)
        {
            if (lhs.album.nom < rhs.album.nom)
                return true;

            return lhs.album.nom == rhs.album.nom && lhs.nom < rhs.nom;
        }

        return false;
    });
}

// Affichage par morceau, dans l'ordre alphabétique.
void affichage_morceau(Discographie const & discographie)
{
    for (Morceau const & morceau : discographie)
    {
        std::cout << "--> " << morceau << std::endl;
    }
}

// Affichage par album, dans l'ordre alphabétique.
void affichage_album(Discographie const & discographie)
{
    Album album_precedent{};
    for (Morceau const & morceau : discographie)
    {
        if (morceau.album.nom != album_precedent.nom)
        {
            std::cout << "--> " << morceau.album << " | " <<
morceau.compositeur << std::endl;
        }
        std::cout << "\t/--> " << morceau.nom << std::endl;

        album_precedent = morceau.album;
    }
}

// Affichage par artiste, dans l'ordre alphabétique.
```


RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

void affichage_artiste(Discographie const & discographie)
{
    Artiste artiste_precedent{};
    Album album_precedent{};

    for (Morceau const & morceau : discographie)
    {
        if (morceau.compositeur.nom != artiste_precedent.nom)
        {
            std::cout << "--> " << morceau.compositeur << std::endl;
        }

        if (morceau.album.nom != album_precedent.nom)
        {
            std::cout << "\t--> " << morceau.album << std::endl;
        }

        std::cout << "\t\t--> " << morceau.nom << std::endl;

        artiste_precedent = morceau.compositeur;
        album_precedent = morceau.album;
    }
}

// L'ensemble des possibilités d'affichage de la discographie.
enum class Affichage { Artiste, Album, Morceau };

void affichage(Discographie& discographie, Affichage
type_affichage)
{
    if (type_affichage == Affichage::Album)
    {
        tri_album(discographie);
        affichage_album(discographie);
    }
    else if (type_affichage == Affichage::Artiste)
    {
        tri_artiste(discographie);
        affichage_artiste(discographie);
    }
    else if (type_affichage == Affichage::Morceau)
    {
        tri_morceau(discographie);
        affichage_morceau(discographie);
    }
    else
    {
        // Par exemple si on met à jour l'énumération mais qu'on
        // oublie
        // d'ajouter la fonction correspondante.
        throw std::runtime_error("Commande d'affichage inconnue.");
    }
}

```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

void enregistrement(Discographie & discographie,
std::string const & nom_fichier)
{
    std::ofstream fichier { nom_fichier };
    if (!fichier)
    {
        throw std::runtime_error("Impossible d'ouvrir le fichier en
écriture.");
    }

    for (Morceau const & morceau : discographie)
    {
        fichier << morceau << std::endl;
    }
}

void chargement(Discographie & discographie, std::string const &
nom_fichier)
{
    std::ifstream fichier { nom_fichier };
    if (!fichier)
    {
        throw std::runtime_error("Impossible d'ouvrir le fichier en
lecture.");
    }

    std::string ligne {};
    while (std::getline(fichier, ligne))
    {
        Morceau morceau {};

        std::istringstream flux { ligne };
        flux >> morceau;

        discographie.push_back(morceau);
    }
}

```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

systeme_commandes.hpp ([retour à la Section 17.3, \[Pratique\]](#) Découpons notre programme !)

```
#ifndef SYSTEME_COMMANDES_HPP_INCLUDED
#define SYSTEME_COMMANDES_HPP_INCLUDED

#include "donnees_disco.hpp"
#include <string>
#include <tuple>

std::string recuperer_commande();

enum class Commande { Afficher, Ajouter, Enregistrer, Charger,
    Quitter };

std::tuple<Commande, std::string>
analyser_commande(std::string const & commande_texte);
bool executer_commande(Discographie & discographie, Commande
    commande, std::string const & instructions);

#endif
```

systeme_commandes.cpp

```
#include "discographie.hpp"
#include "systeme_commandes.hpp"
#include "utils.hpp"
#include <sstream>
#include <stdexcept>
#include <string>

std::string recuperer_commande()
{
    std::cout << "> ";

    std::string commande {};
    std::getline(std::cin, commande);
    return commande;
}

std::tuple<Commande, std::string>
analyser_commande(std::string const & commande_texte)
{
    std::istringstream flux { commande_texte };

    std::string premier_mot {};
    flux >> premier_mot;
    premier_mot = traitement_chaine(premier_mot);

    std::string instructions {};
```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```

std::getline(flux, instructions);
instructions = traitement_chaine(instructions);

if (premier_mot == "afficher")
{
    return { Commande::Afficher, instructions };
}
else if (premier_mot == "ajouter")
{
    return { Commande::Ajouter, instructions };
}
else if (premier_mot == "enregistrer")
{
    return { Commande::Enregistrer, instructions };
}
else if (premier_mot == "charger")
{
    return { Commande::Charger, instructions };
}
else if (premier_mot == "quitter")
{
    return { Commande::Quitter, std::string {} };
}
else
{
    throw std::runtime_error("Commande invalide.");
}
}

bool executer_commande(Discographie & discographie, Commande
commande, std::string const & instructions)
{
    if (commande == Commande::Afficher)
    {
        if (instructions == "artistes")
        {
            affichage(discographie, Affichage::Artiste);
        }
        else if (instructions == "albums")
        {
            affichage(discographie, Affichage::Album);
        }
        else if (instructions == "morceaux")
        {
            affichage(discographie, Affichage::Morceau);
        }
        else
        {
            throw std::runtime_error("Commande d'affichage inconnue.");
        }
    }
    else if (commande == Commande::Ajouter)
    {
        std::istringstream flux { instructions };
    }
}

```

RÉORGANISATION DU CODE DU GESTIONNAIRE DE DISCOGRAPHIE

```
    Morceau morceau {};  
    flux >> morceau;  
    discographie.push_back(morceau);  
}  
else if (commande == Commande::Charger)  
{  
    chargement(discographie, instructions);  
}  
else if (commande == Commande::Enregistrer)  
{  
    enregistrement(discographie, instructions);  
}  
else if (commande == Commande::Quitter)  
{  
    return false;  
}  
  
return true;  
}
```

Interlude : être développeur

L'art de développer ne se limite pas à la maîtrise du langage utilisé. Il y a, en effet, bien d'autres aspects dans le développement que la pure programmation et c'est ce que nous allons aborder dans cette partie. Les outils que vous y découvrirez vous donneront une grande autonomie pour mener à bien des projets seul. Vous serez alors à même de créer des programmes mine de rien déjà assez complexes.

18

Un coup d'œil dans le rétro

Nous avons abordé de très nombreuses notions depuis le début de ce livre, et avec elles nous avons touché du doigt certains aspects essentiels du développement informatique. Il est temps de revenir sur ce que nous avons fait et de l'analyser avec un peu de recul. Nous insistons sur ces points, car être un bon développeur c'est beaucoup plus que simplement savoir écrire du C++.

18.1. Au-delà du code

Jusqu'à présent, nous avons abordé différents aspects de la programmation en C++, que ce soit des notions de base (comme les conditions ou les boucles) ou des notions plus avancées (comme l'utilisation d'outils fournis par la bibliothèque standard, ainsi que la création de fonctions et de structures de données personnalisées). Vous avez ainsi déjà acquis un arsenal de techniques assez complet pour faire des programmes relativement complexes. La création du gestionnaire de discographie de la partie précédente est là pour le prouver.

Cependant, nous avons vu dans ces deux parties bien plus que de simples notions de C++. Nous avons découvert une certaine manière de réfléchir qui traduit les qualités d'un bon développeur : le désir d'avoir toujours un code de qualité, à travers un travail de conception poussé et des bonnes pratiques. Citons, parmi celles-ci, les tests unitaires pour vérifier que notre code fait ce qu'on lui demande, les contrats pour définir clairement ce que les différents intervenants attendent comme entrées et sorties et les exceptions pour remonter des problèmes à l'utilisateur.

Ce sont ces qualités qui feront qu'un développeur sera non seulement capable d'utiliser son langage, mais aussi de mener ses projets à bien. Pour être un bon marin, il ne suffit pas de savoir utiliser la voile et le gouvernail, encore faut-il savoir naviguer, s'orienter sur la mer. De même, il ne suffit pas pour un développeur de maîtriser ses outils pour mener sa barque à bon port.

Vous avez commencé à vous rendre compte de ces aspects-là, notamment grâce à la deuxième partie qui insiste fortement sur l'amélioration de la qualité du code. Et c'est l'une des choses les plus importantes à retenir de ces deux premières parties. Dans cet

interlude, nous allons aborder de nouvelles notions qui s'inscrivent dans cette optique, à savoir faire de vous de bons développeurs et pas simplement des personnes qui savent écrire en C++.

18.2. Tout est une question de paradigme

Nous avons souvent dit que tout problème de développement a de nombreuses solutions, l'essentiel étant que les choix que l'on fait soient guidés par la volonté de produire un code propre et robuste à travers une bonne conception.

Cette diversité se manifeste notamment à travers la manière de programmer. Il existe différentes approches, appelées *paradigmes*. Le C++ est un langage *multiparadigme*. Il permet de programmer de manière très libre et nous laisse choisir parmi plusieurs de ces manières de programmer. Nous en avons déjà croisé quelques-unes.

Le paradigme impératif

Le **paradigme impératif** est le principal paradigme que l'on a utilisé. C'est aussi celui de beaucoup d'autres langages de programmation, en particulier des premiers inventés. Selon cette approche, le programme est une suite d'instructions qui s'exécutent *linéairement* et le font avancer. Il comprend des conditions, boucles, fonctions, etc. Une grande partie des notions que l'on a abordées s'inscrit, au moins partiellement, dans ce paradigme.

Voici un exemple d'un code écrit typiquement avec le paradigme impératif, qui met chaque élément d'un tableau au carré.

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> nombres { -8, 7, 48, 366, 1, 4, 3 };
    for (int i = 0; i < nombres.size(); ++i)
    {
        nombres[i] *= nombres[i];
    }

    for (auto nombre : nombres)
    {
        std::cout << nombre << std::endl;
    }

    return 0;
}
```



```

64
49
2304
133956
1
16
9

```

Le paradigme fonctionnel

Le **paradigme fonctionnel** est aussi en partie couvert par le langage C++. C'est un paradigme dans lequel, comme son nom l'indique, les fonctions jouent un rôle prépondérant. D'ailleurs, dans les langages dit *fonctionnellement purs*, la modification de variable peut ne pas exister et tout est réalisé avec des calculs sur résultats de fonctions ! Nous avons principalement vu ce paradigme à travers **les lambdas**, qui sont un concept issu tout droit des langages fonctionnels. De manière plus générale, le fait de pouvoir manipuler les fonctions comme des variables est un emprunt à ce paradigme.

Le code suivant est une réécriture de l'exemple précédent, dans un style plus fonctionnel.

```

#include <algorithm>
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> nombres { -8, 7, 48, 366, 1, 4, 3 };
    std::for_each(std::begin(nombres), std::end(nombres),
        [](int & nombre) -> void
        {
            nombre *= nombre;
        });

    for (auto nombre : nombres)
    {
        std::cout << nombre << std::endl;
    }

    return 0;
}

```

```

64
49
2304
133956
1
16
9

```

Le paradigme générique

Le **paradigme générique** s'appuie essentiellement sur l'idée que, parfois, on se fiche du type réel des données que l'on manipule. Ce qui compte, c'est la manière dont elles peuvent être utilisées. Nous l'avons principalement appréhendé sous deux formes :

- les algorithmes standards qui peuvent s'appliquer à tout un tas de collections différentes ;
- les fonctions templates qui permettent d'écrire du code fonctionnant avec plusieurs types, du moment qu'on peut les manipuler de cette manière.

En reprenant les exemples précédents, utilisons le paradigme générique. Voici un résultat qu'on peut obtenir.

```
#include <algorithm>
#include <iostream>
#include <list>
#include <vector>

template <typename Collection>
void au_carre(Collection & collection)
{
    std::for_each(std::begin(collection), std::end(collection),
        [](auto & nombre) -> void
        {
            nombre *= nombre;
        });
}

template <typename Collection>
void affichage(Collection const & collection)
{
    for (auto element : collection)
    {
        std::cout << element << std::endl;
    }
}

int main()
{
    std::vector<int> nombres { -8, 7, 48, 366, 1, 4, 3 };
    au_carre(nombres);
    affichage(nombres);

    std::cout << std::endl;

    std::list<int> autres { 10, 30, 60 };
    au_carre(autres);
    affichage(autres);
}
```

```
    return 0;
}
```

```
64
49
2304
133956
1
16
9

100
900
3600
```

La programmation par contrat

Bien que ce ne soit pas aussi poussé qu'avec d'autres langages comme Eiffel ou le D, il est possible en C++ de faire de la [programmation par contrat](#), ainsi que nous l'avons vu au chapitre [Erreur, erreur, erreur....](#) Les principes de la programmation par contrat se combinent très bien avec les autres paradigmes, alors n'hésitez pas à en user et en abuser.

***Note** > C++23 devrait inclure de nouveaux outils pour faire de la programmation par contrat, directement intégrés dans le langage, ce qui permettra de faire des choses encore plus poussées.*

Comme vous le voyez, à la différence d'autres langages plus restrictifs, le C++ met à notre disposition tout un panel d'outils très variés pour écrire du code. Nous pouvons prendre un peu de chaque paradigme et nous servir de ses points forts. Dans la quatrième partie du livre, nous aborderons un nouveau paradigme, lui aussi très important en C++ : la programmation orientée objet.

***Attention** > Ce n'est pas une critique envers d'autres langages que de dire qu'ils sont moins permissifs. Il est parfois préférable d'utiliser un langage qui fait des choix forts. Simplement, l'un des attraits du C++ est précisément cette permissivité. Mais c'est aussi un écueil. Comme on peut faire beaucoup de choses, on peut aussi très facilement faire n'importe quoi ! Vous devez donc être très vigilant et toujours vous efforcer de produire du code propre et robuste. D'où notre insistance sur ce point.*

18.3. L'art de la conception

Nous l'avons compris, le C++ offre de nombreux outils, ainsi que la liberté d'utiliser et mélanger plusieurs paradigmes. Mais ce ne sont là qu'outils et instruments. Il faut les assembler intelligemment et harmonieusement pour produire de bons programmes. Et pour cela, avant de se ruer sur son clavier et de martyriser énergiquement ses pauvres touches, il faut *réfléchir et concevoir*.

On prend un stylo et un papier (ou tout autre support qui vous convient) , et on se pose des questions sur le programme sur lequel on travaille. On réfléchit à ses fonctionnalités, ce qu'il rendra comme services, ce qu'il ne fera pas, ce qu'il sera capable de gérer, etc. On se demande quels sont les cas d'erreurs, les entrées invalides, ce qui peut potentiellement poser problème, etc.

***Note** > Nous avons déjà commencé à faire ça, notamment avec les contrats et la réflexion sur nos entrées et sorties.*

On découpe aussi de grosses fonctionnalités en plus petites afin qu'elles soient plus facilement abordables. Un logiciel de traitement d'images peut ainsi être découpé en sous-tâches comme l'ouverture d'un fichier PNG, la modification d'un unique pixel, la sauvegarde du PNG modifié, etc.

C'est là que le fait de noter ses idées se révèle utile. On peut comparer plusieurs solutions à un même problème, on peut visualiser plus facilement à quoi ressemblera le programme final, on peut écrire du pseudo-code pour s'aider à mieux comprendre un algorithme complexe, etc. Et puis, personne n'ayant une mémoire infailible, noter ses idées aide à s'en rappeler.

Cependant concevoir, anticiper ne signifie pas que le programme est figé et qu'il n'évoluera pas d'un pouce. Au contraire, réfléchir avant de coder permet de mieux prévoir des aspects qui pourront être amenés à changer. Cela rend le programme plus évolutif.

***Attention** > C'est un exercice délicat que de trouver l'équilibre entre le néant et trop de spécifications, entre une trop grande rigidité et une absence de réflexion et de conception. Ne vous inquiétez pas. Comme beaucoup de choses, tout vient en pratiquant.*

18.4. De la persévérance avant tout

Être un bon développeur, c'est être autonome et se débrouiller. Cela ne veut pas dire que les développeurs ne se parlent jamais entre eux et restent chacun dans leur coin, coupés les uns des autres. Au contraire, l'existence même du tutoriel à l'origine de ce livre et de la plateforme Zeste de Savoir sont des preuves que l'échange est important.

Mais il ne faut pas vous attendre à ce que tout vous tombe cuit dans le bec. Un bon développeur ne compte pas sur les autres pour faire son travail. Il n'attend pas de réponse toute faite. Quand il est bloqué, il réfléchit d'abord et demande ensuite. Dans cette partie (chapitre [Chasse aux bugs](#) !), nous verrons aussi comment débusquer vos erreurs en exécutant pas à pas votre code. C'est ce qu'on appelle le débogage. Avec cette technique, vous pourrez déjà bien avancer seul dans la recherche de la solution.

Si vous êtes vraiment bloqué sur un problème, vous pouvez bien entendu demander un coup de pouce, notamment sur Zeste de Savoir. Mais quand cela arrivera, montrez ce

que vous avez tenté, les erreurs qui vous bloquent, si vous avez déjà fait des recherches dessus, etc. Et vous verrez, vous aurez de l'aide et des réponses 😊 .

En conclusion, laissez-moi vous lister quelques qualités, en plus de l'autonomie, qui feront de vous un meilleur développeur.

- *Patience.* Il en faut pour apprendre, pour comprendre, pour faire et corriger des erreurs, pour chercher des solutions, pour ne pas s'énerver quand ça ne marche pas.
- *Persévérance et motivation.* Développer, c'est parfois rencontrer des problèmes qui nous dépassent, qui semblent trop complexes à première vue, ou bien des notions trop abstraites. Ces qualités aideront à ne pas abandonner à la première difficulté.
- *Humilité.* Un bon développeur accepte de ne pas avoir réponse à tout et donc de demander de l'aide quand il n'y arrive vraiment pas. Il accepte les conseils et les corrections qu'on lui donne. Il accepte de ne pas se reposer sur ses acquis, se remet régulièrement à jour et en question. Cela lui évite les réponses méprisantes, arrogantes et hautaines qu'on voit parfois sur certains forums.

18.5. Récapitulons !

- Vous avez déjà beaucoup appris sur le langage C++, mais aussi sur des aspects plus généraux du développement, comme l'écriture de tests unitaires, de contrats ou la réflexion à mener sur les entrées/sorties.
- Le C++ est un langage très riche qui permet de programmer de différentes manières.
- Cette liberté est une force mais aussi un danger, c'est pourquoi une bonne conception et du code de qualité sont primordiaux.
- Être un bon développeur demande aussi plusieurs qualités, dont la persévérance, l'autonomie et la patience.
- La suite de cette partie vous aidera à tendre vers cet idéal en vous apprenant de nouvelles notions pour mieux développer.

19

Mais où est la doc ?

Cette chère documentation est une des plus vieilles amies du développeur. Parfois très complète, parfois un peu brève, parfois totalement absente (pas de bol !), elle est là pour le guider, répondre à ses questions et lui apprendre comment utiliser tel ou tel type, ce qu’attend telle ou telle fonction, etc. Heureusement, dans notre cas, il existe une [excellente documentation pour C++](#), en anglais certes (sachez que la [version française](#) est une traduction automatique), mais simple à comprendre. Et elle n’aura bientôt plus de secret pour vous.

Nous allons apprendre à lire la documentation C++ ainsi qu’à écrire celle de nos programmes.

19.1. Lire une page de doc

Vous voilà donc sur la documentation et tous ces liens vous laissent perplexe. Comment on s’y retrouve là-dedans ? Par où chercher l’information ? Le site étant organisé en plusieurs parties, tout dépend de ce que vous cherchez. Certaines vont plus nous intéresser, dans le cadre de ce livre, que d’autres.

- **HEADERS** : fichiers d’en-tête standard. Cette page recense tous les fichiers d’en-tête de la bibliothèque standard. Nous en reconnaissons plusieurs, comme `<iostream>`, `<string>` ou `<vector>`. Ils sont classés par catégorie, en fonction de ce à quoi ils sont dédiés : la manipulation de chaînes de caractères, les conteneurs, etc.
- **STRING LIBRARY** : chaînes de caractères. Ici, nous allons retrouver tout ce qui concerne le célèbre `std::string`.
- **CONTAINERS LIBRARY** : conteneurs. Ici, vous avez un lien direct vers tous les conteneurs qui existent. Vous reconnaîtrez notamment `vector`, `array` et `list`.
- **ALGORITHMS LIBRARY** : algorithmes. Vous en connaissez certains, mais il en reste d’autres, tous listés ici.

Nous vous citons ici les autres catégories, pour information et pour satisfaire votre curiosité débordante.

- UTILITIES LIBRARY : correspond aux utilitaires pour gérer les types, la mémoire, les erreurs, les dates et temps, etc.
- ITERATORS LIBRARY : itérateurs. Vous connaissez déjà les [itérateurs](#). Ici, on retrouve la documentation de tous ceux qu'on connaît, plus bien d'autres encore.
- NUMERICS LIBRARY : numérique. C'est tout ce qui touche à la manipulation des nombres (complexes, aléatoires, etc.).
- INPUT/OUTPUT LIBRARY : entrées/sorties. On en a déjà un peu manipulé, mais il reste encore beaucoup de choses.
- LOCALIZATIONS LIBRARY : internationalisation. Nous n'en parlerons pas, mais il y a ici des éléments permettant d'adapter son programme à d'autres langues et cultures.
- FILESYSTEM LIBRARY : système de fichiers. Pour permettre la manipulation de dossiers et de fichiers de manière standard.
- D'autres encore, d'un niveau trop avancé pour être expliqués maintenant.

Au fur et à mesure que vous progresserez en C++ et gagnerez en autonomie, vous pourrez vous débrouiller seul dans la documentation, car celle-ci est organisée de manière logique.

vector – retour sur le plus célèbre des conteneurs

Commençons en examinant la [page dédiée à vector](#). Il y a une très grande quantité d'informations, mais toutes ne nous sont pas utiles. Les premières lignes indiquent à quoi ressemble un **vector**.

```
template<
    class T,
    class Allocator = std::allocator<T>
> class vector;
```

Vous reconnaissez la syntaxe des templates. Seulement, vous aviez l'habitude jusque-là de déclarer des objets de type tableau de la manière suivante : `std::vector<int> tableau`, alors qu'ici vous voyez qu'il y a deux types génériques attendus. En fait, le deuxième sert, à l'aide d'une valeur par défaut, à réserver un espace mémoire pour stocker les éléments. On ne le modifie qu'en cas de besoins très particuliers, notamment quand on veut manipuler la mémoire d'une manière précise, donc pour nous ça ne pré-

sente aucun intérêt. On laisse donc la valeur par défaut et on retrouve ainsi la syntaxe habituelle, `T` indiquant le type des éléments.

Les fonctions membres

Sautons plusieurs détails et passons directement à la section MEMBER FUNCTIONS [fonctions membres]. Ici sont listées toutes les fonctions que l'on peut appliquer à un `vector` avec la syntaxe `mon_vecteur.fonction()`. Certaines vous sont familières, comme `push_back` ou `clear`. Si vous cliquez sur l'une d'elles, vous aurez tous les détails la concernant.

Chaque page de fonction présente d'abord tous ses prototypes. Comme vous le savez déjà, avec les surcharges, il peut exister plusieurs fonctions avec le même identificateur mais des paramètres différents. C'est le cas ici pour `push_back`. Si une fonction est apparue avec une nouvelle norme de C++, ou si elle a été supprimée, cela est précisé à la fin de la ligne.

```
void push_back( const T& value ); (1)
void push_back( T&& value );      (2) (since C++11)
```

Dans le cas de la fonction `push_back`, nous avons deux surcharges, dont la deuxième n'est disponible qu'à partir de C++11.

Note > La syntaxe du deuxième prototype doit vous surprendre. En effet, nous ne l'avons pas encore abordée. Nous le ferons plus loin dans le livre, mais sachez que, entre autres, la deuxième version est appelée quand vous lui donnez directement un littéral [`push_back(5);`], alors que si vous passez une variable, ça sera le premier prototype [`int const a { 5 }; push_back(a);`]. Pour faire simple, cela permet d'optimiser le code lorsqu'il est appelé avec une valeur temporaire.

Les fonctions non-membres

Sur la page de `std::vector`, on trouve d'autres fonctions, dites non-membres (section NON-MEMBER FUNCTIONS). Celles-ci utilisent la syntaxe `fonction(mon_vecteur)`, à la différence de ce que nous avons vu à la section précédente mais de manière similaire à des fonctions que vous connaissez, comme `std::size` ou `std::begin`. C'est ici que vous trouverez la plupart des opérateurs surchargés, notamment.

On trouve par exemple `std::swap`, qui permet d'échanger le contenu de deux vecteurs, ou bien des opérateurs de `comparaison`.

Les paramètres

Revenons sur la page de `push_back`. La section `PARAMETERS` y décrit les différents paramètres attendus par la fonction. Rien de bien surprenant pour vous dans le cas de `push_back`, il s'agit simplement de la valeur à insérer. Ensuite, la section `RETURN VALUE` décrit le résultat, qui, dans notre cas, n'existe pas puisque la fonction renvoie un `void`.

Les exceptions

Intéressons-nous maintenant à la section `EXCEPTIONS`. Comme vous le savez, C++ permet d'utiliser les exceptions et certaines fonctions de la bibliothèque standard le font. Cette section décrit le comportement attendu dans le cas où une fonction viendrait à lancer une exception. En effet, toutes ne réagissent pas de la même manière et ne présentent pas les mêmes garanties.

- *Strong exception guarantee* — garantie forte. Elles garantissent que, si une exception est levée, alors rien n'est modifié et tout reste comme avant le lancement de l'exception. C'est le cas de `push_back` : si une exception est levée, alors le tableau reste inchangé.
- *Basic exception guarantee* — garantie basique. Dans ce cas, la seule chose dont on est sûr, c'est qu'il n'y a pas de fuite de mémoire. Par contre, l'objet peut avoir été modifié.
- *No-throw guarantee* — garantie mais pas d'exception. Aucune exception ne sera levée. Ces fonctions possèdent le mot clé `noexcept` dans leur prototype. C'est le cas de la fonction `empty` par exemple.
- *No exception safety* — aucune garantie. Tout peut se passer potentiellement. Ce genre de fonctions sont celles qui offrent le moins de garantie et de protection.

Dans le cas où la fonction est `noexcept`, la section `EXCEPTIONS` peut être absente de la page de documentation. Cela dépend des cas.

Attention > Si une fonction spécifiée `noexcept` lève quand même une exception, la fonction `std::terminate` est appelée et celle-ci provoque un arrêt brutal du programme.

Exemple(s)

Enfin, la section `EXAMPLE` donne un cas d'utilisation concret de la fonction exposée. On peut même modifier et exécuter le code en cliquant sur `RUN THIS CODE`.

Les algorithmes

Continuons notre apprentissage en nous rendant sur [la page des algorithmes](#). Nous en connaissons déjà quelques-uns, ce qui va nous faciliter la compréhension de la documentation. Notez que les algorithmes sont eux-mêmes classés en sous-catégories, comme NON-MODIFYING SEQUENCE OPERATIONS [Non modifiants], MODIFYING SEQUENCE OPERATIONS [Modifiants], SORTING OPERATIONS [Tri], etc. Au début de chaque sous-catégorie se trouve le fichier d'en-tête à inclure pour utiliser les algorithmes sous-cités.

Prenons l'exemple de `std::sort`, que vous connaissez déjà. Vous le trouverez dans la catégorie SORTING OPERATIONS, puisqu'il s'agit d'un algorithme de tri. On note en haut que le fichier d'en-tête requis est `<algorithm>`. Maintenant que nous avons ces informations, nous pouvons nous rendre sur [sa page](#).

Une fois là-bas, on remarque qu'il existe quatre [signatures](#) différentes, chacune étant ordonnée avec un numéro. La première (1) est celle que nous connaissons le mieux, puisque nous [l'avons déjà vue](#). On remarque que le prototype change selon qu'on compile en C++20 ou antérieur. La différence réside dans le mot clé `constexpr`, que nous détaillerons [plus tard](#).

```
template <class RandomIt>
void sort(RandomIt first, RandomIt last);
```

La troisième (3) lui ressemble, à ceci près qu'on a une possibilité de personnaliser l'algorithme de comparaison. Là encore, la seule différence entre pré- et post-C++20, c'est `constexpr`.

```
template <class RandomIt, class Compare>
void sort(RandomIt first, RandomIt last, Compare comp);
```

Les deux autres, à savoir les (2) et le (4), ont été introduites avec C++17 et permettent de traiter plusieurs instructions en parallèle plutôt que les unes à la suite des autres. C'est encore compliqué à notre niveau, donc nous n'irons pas plus loin sur le sujet.

La section PARAMETERS nous donne des indications supplémentaires sur les arguments attendus par `std::sort`. On y trouve notamment la forme que doit prendre la fonction transmise en paramètre de `comp`. Dans le cas présent, il se trouve que vous le saviez déjà, mais cela pourrait vous servir pour d'autres algorithmes.

Enfin, on nous informe que la fonction ne renvoie rien et qu'elle peut éventuellement lever une exception de type `std::bad_alloc`, nous indiquant par là de quoi nous protéger. Puis vous avez un exemple de code.

Les chaînes de caractères au grand complet

Examinons maintenant la [page de documentation de `std::string`](#). Comme déjà dit lors de l'[introduction de `using`](#), le type `std::string` est un alias du type `basic_string`, instancié pour les `char`. Ne vous étonnez donc pas en voyant la documentation.

Exemple de fonction membre

Vous pouvez voir, dans la section MEMBER FUNCTIONS, tout un tas de fonctions que nous ne connaissions pas encore. Certaines sont familières, comme `push_back` ou `find`. D'autres nouvelles, comme `insert`, que nous allons étudier.

En regardant les différents prototypes proposés, on distingue pas moins de onze signatures différentes. On peut insérer plusieurs fois un même caractère (1), insérer une chaîne entière (2)(4), des morceaux de chaînes grâce aux itérateurs (8) ou à des index (5), etc. Chacune est détaillée ensuite, avec explication de chaque paramètre attendu.

Toutes ces fonctions peuvent lever des exceptions. La documentation cite `std::out_of_range` si on transmet un index plus grand que la taille de la chaîne (logique) et `std::length_error` si on tente de créer une trop grande chaîne. Dans tous les cas, on a une garantie forte.

Exemple de fonction non-membre

Avec C++20 a été introduite la fonction `std::erase`, qui permet d'effacer toutes les occurrences d'un caractère, ainsi que sa cousine `std::erase_if`, qui est paramétrable.

Exemple d'autres fonctions associées

Tout ce qui permet de convertir une chaîne de caractères en entiers ou réels (et vice-versa) est listé sur cette page. Ces fonctions ne font pas partie du type `std::string`, mais sont tellement liées qu'elles sont réunies au sein du même fichier d'en-tête. On retrouve ainsi `stoi`, `stod` et `to_string`.

19.2. Entraînez-vous

Pour gagner en autonomie et être capable de vous débrouiller seul dans la documentation, vous devez vous entraîner. Le but de cette section d'exercices est donc que vous appreniez à chercher une information dans la documentation. C'est important, car vous n'aurez pas toujours un guide pour vous expliquer quoi et où chercher.

Remplacer une chaîne de caractères par une autre

On aimerait pouvoir remplacer un morceau d'une chaîne de caractères par un autre. Pour être plus précis, disons qu'on veut remplacer une portion commençant à un index précis et d'une longueur précise. Le code suivant devra être complété pour pouvoir s'exécuter.

```
#include <cassert>
#include <iostream>
#include <string>

int main()
{
    std::string phrase { "J'aimerais annuler ce texte." };
    std::string const mot { "remplacer" };

    // Le code à écrire.

    assert(phrase == "J'aimerais remplacer ce texte."
        && "Le texte n'a pas été modifié correctement.");
    return 0;
}
```

 [Voir la solution](#)

Norme d'un vecteur

Pour ceux qui ont quelques connaissances en vecteurs, vous savez comment en calculer la norme. Pour les autres, sachez que la norme d'un vecteur correspond à sa longueur, que l'on calcule simplement à l'aide de la formule suivante.

Si $\vec{V}$ est un vecteur de coordonnées (x, y) , alors sa norme $\|\vec{V}\|$ vaut $\sqrt{x^2 + y^2}$.

Il nous semble qu'il y a une fonction pour ça en C++, mais laquelle ?

```
#include <cassert>

int main()
{
    int const x { 5 };
    int const y { 12 };
    int norme { 0 };

    // Le code pour résoudre l'exercice.

    assert(norme == 13 && "La norme du vecteur v(5;12) vaut 13.");
    return 0;
}
```

✎ Voir la solution

? Obtenir un indice

Nombres complexes

Ah, les nombres complexes ! Saviez-vous que C++ permettait d'en utiliser ? Eh oui, on peut calculer la racine carrée de -1 si ça nous chante. Que diriez-vous de faire ce calcul puis d'afficher la partie réelle puis la partie imaginaire du résultat ?

✎ Voir la solution

? Obtenir un indice

Transformations

Vous connaissez déjà l'algorithme `std::for_each`, qui applique une opération sur chaque élément d'un conteneur. Mais supposons que nous voulons non pas modifier le tableau lui-même mais une copie. Est-ce possible ? À vous de trouver l'algorithme adapté.

```
#include <iostream>
#include <string>

int main()
{
```

```

std::string const phrase {
    "Voici une phrase longue que je veux en majuscules."
};
std::string sortie { "" };

// L'algorithme à utiliser.

std::cout << sortie << std::endl;
return 0;
}

```

✧ Voir la solution

Cet exercice vous a semblé plus difficile ? C'était le cas. Quand on ne connaît pas quelque chose, bien souvent, une seule page de documentation n'est pas suffisant. Il faut faire des recherches complémentaires. Heureusement pour vous, la documentation C++ présente sur ce site est accompagnée d'exemples qui vous aideront à bien saisir les difficultés et particularités potentielles.

19.3. Documenter son code avec Doxygen

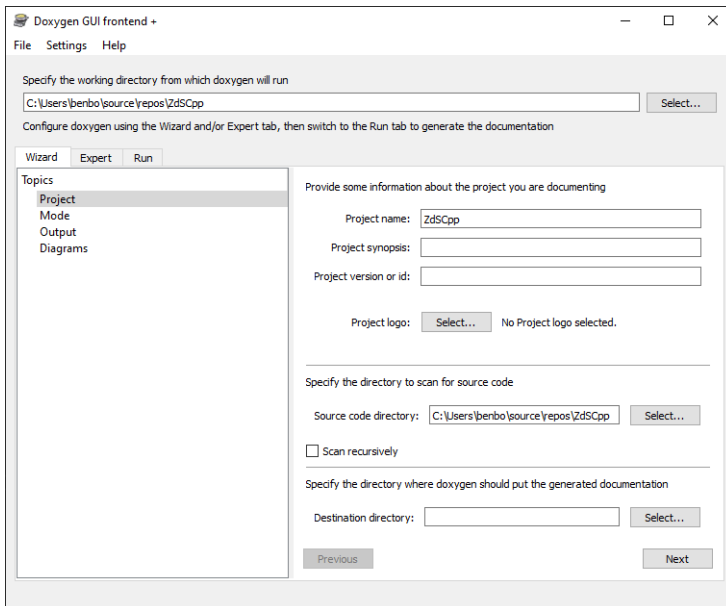
Vous commencez maintenant à vous débrouiller avec la documentation officielle de C++. C'est une très bonne chose, car savoir lire et comprendre une documentation est un gros atout en informatique vu que la très grande majorité des langages et des programmes en fournissent une. Et les vôtres alors ? Il ne manquerait pas quelque chose ? En toute logique, nous devons aussi apprendre à écrire la documentation de notre code.

Installation d'un outil dédié

Cela va se faire en utilisant un outil appelé [Doxygen](#). Il en existe plusieurs, mais celui-ci est très répandu et utilisable avec d'autres langages de programmation. Vous gagnez donc à apprendre à vous en servir.

Pour l'installer sur Windows, rendez-vous sur la page des [téléchargements](#) et prenez la dernière version disponible. Pour les utilisateurs de GNU/Linux, regardez dans vos dépôts. Le paquet devrait s'appeler `doxygen`, accompagné ou non de `doxygen-doxywizard`¹. Une fois ceci fait, nous allons utiliser le *Doxywizard*, l'interface graphique pour lancer la génération de la documentation.

¹Référez-vous à la documentation de votre distribution favorite pour savoir exactement quoi installer.

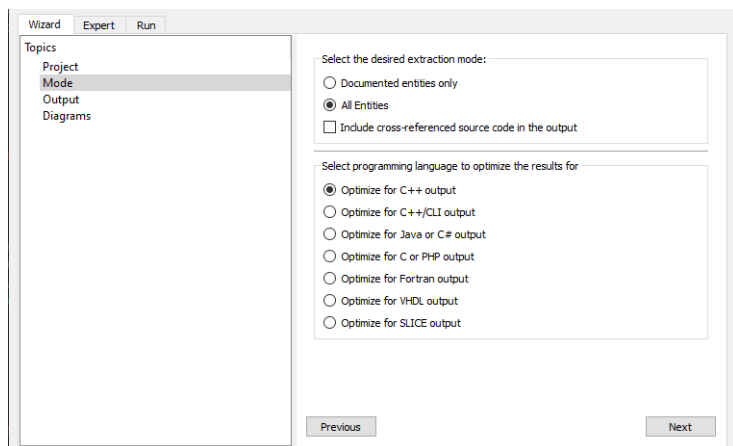
Figure 19.1 : Voici le magicien Doxygen, si l'on traduit son nom en français

Pour commencer, tout en haut de la fenêtre, on définit le dossier dans lequel Doxygen mettra la documentation produite.

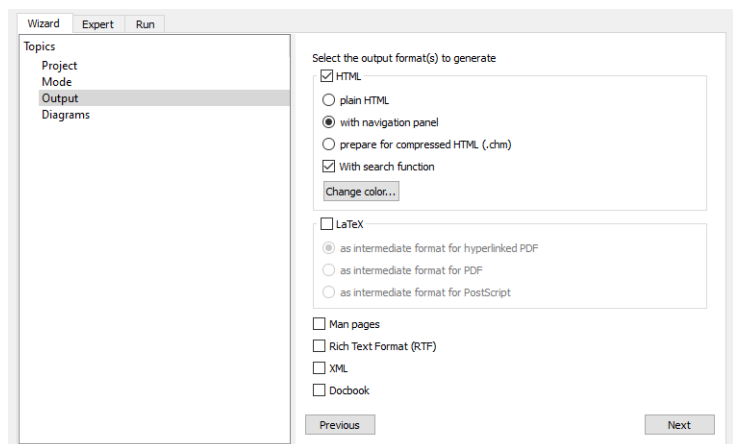
Puis, on s'assure qu'on est bien sur l'onglet WIZARD, car c'est ici qu'on va pouvoir configurer plusieurs options. Sous la rubrique PROJECT (voir [Figure 19.1](#)), les trois premiers champs sont facultatifs, ils permettent de définir le nom du projet, son but et son numéro de version (très utile s'il y a plusieurs versions du programme et donc de documentations disponibles). Vous pouvez vous amuser à mettre un logo aussi.

Il faut ensuite préciser où se situe précisément le code source. Dans l'exemple donné, il est dans le même dossier qu'indiqué précédemment, mais dans d'autres, il peut être rangé dans un sous-dossier. Mettez le chemin du dossier qui contient votre code source.

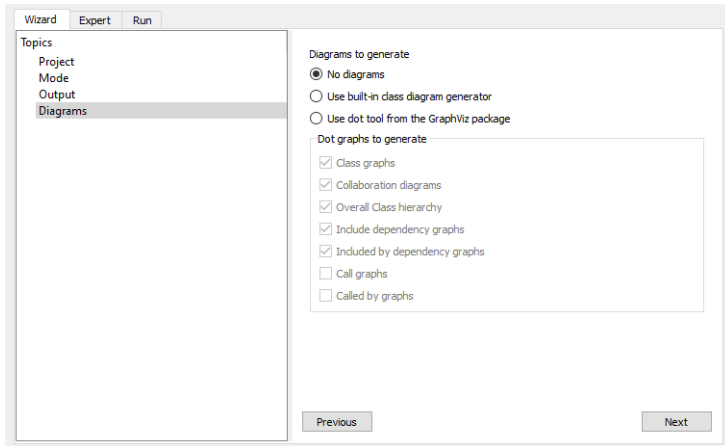
Sous la rubrique MODE, on va indiquer que c'est du C++ (sélectionnez OPTIMIZE FOR C++ OUTPUT) et qu'on veut tout extraire (sélectionnez ALL ENTITIES).

Figure 19.2 : Rubrique Mode

Sous la rubrique OUTPUT, ne cochez que HTML et sélectionnez WITH NAVIGATION PANEL parmi les options proposées, car les autres options de sortie (LaTeX, Docbook, etc) ne nous intéressent pas.

Figure 19.3 : Rubrique Output

Enfin, sous la rubrique DIAGRAMS, sélectionnez NO DIAGRAMS, et tout est bon. Nous pouvons passer à l'onglet RUN, d'où nous allons lancer la génération de la documentation en cliquant sur RUN DOXYGEN. À l'issue de cette étape, un dossier HTML est créé.

Figure 19.4 : Rubrique Diagrams

Astuce > Sauvegardez votre configuration en quittant l'application DoxyWizard. Vous pourrez ainsi la recharger à la prochaine utilisation, cela vous fera gagner du temps.

Par défaut, la documentation est générée en anglais, mais vous pouvez sélectionner la langue de votre choix en cliquant sur EXPERT, puis PROJECT. Là, mettez l'option OUTPUT_LANGUAGE à la valeur que vous voulez, *French* pour le français.

Écrire la documentation

Mais concrètement, comment on écrit la documentation ? En utilisant des balises spéciales, que Doxygen comprendra et qu'il traitera de façon appropriée. Ainsi, on a des attributs spéciaux pour décrire les arguments d'une fonction, sa valeur de retour, la présence de *bugs*, etc. La liste [est longue](#), donc nous n'en décrivons que certains. Libre à vous d'explorer la documentation de Doxygen pour découvrir les autres.

À noter également qu'on peut utiliser le Markdown dans les blocs de commentaires (pour entourer un morceau de code, mettre en gras, etc.).

Comment fait-on ? Il existe plusieurs manières de procéder. Nous avons choisi de vous en montrer deux : la forme `/** Commentaire */` ou la forme `/// Commentaire`. On retrouve ici un parallèle avec les commentaires `/* */` et `//`. C'est une affaire de goût. Pour notre part, quand le commentaire fait plus d'une ligne, nous utilisons la forme longue `/** */`, sinon les trois slashes `///`.

Description d'une fonction

Et si nous reprenions notre fonction d'entrée sécurisée pour illustrer cela ? Nous vous remettons ci-dessous sa dernière version, à base de templates.

```
template <typename T, typename Predicat>
void entree_sécurisée(T & variable, Predicat predicat)
{
    while (!(std::cin >> jour) || !predicat(variable))
    {
        if (std::cin.eof())
        {
            throw std::runtime_error("Le flux a été fermé !");
        }
        else if (std::cin.fail())
        {
            std::cout << "Entrée invalide. Recommence.\n";
            std::cin.clear();

            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        }
        else
        {
            std::cout << "Le prédicat n'est pas respecté !\n";
        }
    }
}
```

Pour commencer, il convient d'expliquer ce que fait la fonction, sa raison d'être. Pour cela, Doxygen fournit deux attributs :

- `@brief` pour décrire brièvement, en une ou deux phrases, la fonction et ce qu'elle fait ;
- `@details` pour décrire en détail la fonction, expliquer ce qu'elle fait, ce qu'elle ne fait pas, les exceptions qu'elle peut lever potentiellement, etc.

Appliquons donc ce que nous venons d'apprendre à notre code. Nous avons écrit un certain texte, mais vous pouvez très bien mettre le vôtre.

```
/**
 * @brief Saisie sécurisée avec `std::cin`.
 * @details Vérifie que la variable demandée est du bon type et respecte
 * un prédicat donné.
 * En cas de fermeture du flux d'entrée, une exception
 * `std::runtime_error` est levée.
 * En cas de type invalide, le flux d'entrée est vidé.
 * En cas de prédicat non respecté, un message d'erreur est affiché sur
 * la sortie standard.
```

```

*/
template <typename T, typename Predicat>
void entree_securisee(T & variable, Predicat predicat)
{
    // Le code qui ne change pas.
}

```

Si vous naviguez dans les fichiers HTML créés, vous allez trouver, dans la documentation du fichier C++ où se trouve la fonction (`main.cpp` par exemple), un encart présentant la fonction avec sa signature, une description courte puis une plus détaillée.

Les paramètres

Maintenant, attaquons un autre morceau très important, à savoir la description des paramètres. Il est important que l'utilisateur sache ce que signifie chaque paramètre et s'il y a des préconditions à respecter. Ça tombe bien, Doxygen propose des attributs spécifiques pour ça :

- `@param[type] paramètre` permet de décrire un paramètre de manière générale. Le type entre crochets peut prendre trois valeurs : `in` si le paramètre est une valeur lue mais non modifiée par la fonction, `out` si le paramètre est modifié par la fonction et `in,out` dans le cas où le paramètre est lu mais également modifié ;
- `@pre` décrit une précondition par ligne. Si plusieurs attributs `@pre` sont présents les uns après les autres, ils sont regroupés dans un même bloc à la sortie ;
- `@post` décrit une postcondition par ligne. Si plusieurs attributs `@post` sont présents les uns après les autres, ils sont regroupés dans un même bloc à la sortie ;
- `@tparam paramètre` permet de faire la même opération pour les templates.

Ceci appliqué à notre code, on obtient quelque chose comme ci-dessous. Pareil, le texte que vous avez mis peut varier.

```

/**
 * @brief Saisie sécurisée avec `std::cin`.
 * @details Vérifie que la variable demandée est du bon type et respecte
 * un prédicat donné.
 * En cas de fermeture du flux d'entrée, une exception
 * `std::runtime_error` est levée.
 * En cas de type invalide, le flux d'entrée est vidé.
 * En cas de prédicat non respecté, un message d'erreur est affiché sur
 * la sortie standard.
 *
 * @tparam T Le type de la valeur à récupérer.
 * @tparam Predicat Un prédicat à passer pour personnaliser la
 * vérification.

```

```

*
* @param[out] variable Une référence sur la variable de type `T` à
récupérer.
* @param[in] predicat Un prédicat que l'entrée saisie doit respecter.
*
* @pre Le type `T` doit être utilisable avec `std::cin`.
* @pre Le prédicat doit être de la forme `bool predicat(T variable)`.
* @post La variable sera initialisée correctement avec la valeur entrée.
*/
template <typename T, typename Predicat>
void entree_securisee(T & variable, Predicat predicat)
{
    // Le code qui ne change pas.
}

```

Les exceptions

Notre fonction peut lever des exceptions, que ce soit parce que nous en lançons nous-mêmes explicitement, ou parce qu'une des fonctions appelées lance une exception en interne. Il est donc de bon ton de décrire ce qui peut potentiellement arriver. Bien sûr, être totalement exhaustif est impossible. Mais décrire les exceptions que nous lançons explicitement est une bonne habitude à prendre. Cela se fait très simplement avec l'attribut `@exception` `exception_lancée`.

```

/**
* @brief Saisie sécurisée avec `std::cin`.
* @details Vérifie que la variable demandée est du bon type et respecte
un prédicat donné.
* En cas de fermeture du flux d'entrée, une exception
`std::runtime_error` est levée.
* En cas de type invalide, le flux d'entrée est vidé.
* En cas de prédicat non respecté, un message d'erreur est affiché sur
la sortie standard.
*
* @tparam T Le type de la valeur à récupérer.
* @tparam Predicat Un prédicat à passer pour personnaliser la
vérification.
*
* @param[out] variable Une référence sur la variable de type `T` à
récupérer.
* @param[in] predicat Un prédicat que l'entrée saisie doit respecter.
*
* @pre Le type `T` doit être utilisable avec `std::cin`.
* @pre Le prédicat doit être de la forme `bool predicat(T variable)`.
* @post La variable sera initialisée correctement avec la valeur entrée.
*
* @exception std::runtime_error Si le flux d'entrée est fermé.
*/
template <typename T, typename Predicat>
void entree_securisee(T & variable, Predicat predicat)

```

```
{
    // Le code qui ne change pas.
}
```

Le type de retour

Nous n'en avons pas dans notre cas, car la fonction `entree_securisee` ne retourne rien. Certains aiment utiliser l'attribut `@returns` même dans ce cas, d'autres jugent que c'est inutile. Nous vous laissons trancher. Voici néanmoins un exemple d'utilisation de cet attribut, avec une [fonction vérifiant le parenthésage](#) que nous avons écrite au chapitre [Découpons le code – les fonctions](#) (légèrement modifiée).

```
/**
 * @brief Détermine si une expression est correctement parenthésée.
 * @details La fonction regarde si on a bien le même nombre de
 * parenthèses ouvrantes et fermantes et si celles-ci sont bien
 * ordonnées.
 *
 * @param[in] expression Une chaîne de caractères à analyser.
 *
 * @returns `true` si l'expression est bien parenthésée, `false` sinon.
 */
bool parentheses(std::string const & expression)
{
    int ouvrantes { 0 };
    int fermantes { 0 };

    for (auto const & caractere : expression)
    {
        if (caractere == '(')
        {
            ++ouvrantes;
        }
        else
        {
            ++fermantes;
        }

        if (fermantes > ouvrantes)
        {
            return false;
        }
    }

    return ouvrantes == fermantes;
}
```

D'autres attributs utiles

Terminons cet aperçu par quelques attributs de documentation d'une fonction qui permettent de fournir des informations supplémentaires, ne rentrant pas dans le cadre des ceux vus précédemment. Il arrive, par exemple, qu'on veuille signaler la présence d'un bug encore non corrigé ou rarissime, une modification ou amélioration prévue mais pas encore faite, un avertissement concernant un aspect particulier du code, etc.

- `@bug` sert à décrire la présence d'un bug dans le code. Cet attribut aurait pu, par exemple, être utilisé dans nos premiers cas pratiques où nous ne pensions pas à prendre en compte la fermeture potentielle du flux. Nous aurions pu ajouter un commentaire du type `@bug Si le flux est fermé, la fonction rentre dans une boucle infinie.`
- `@todo` sert à décrire une action à faire plus tard. Nous pourrions nous en servir pour nous rappeler, dans notre [premier cas pratique](#), de mettre à jour le code pour utiliser un flux d'entrée générique à la place de `std::cin`.
- `@note` permet d'ajouter des informations génériques. Si votre code implémente une fonction mathématique, vous aurez peut-être envie de l'écrire clairement. Ce bloc est tout à fait adapté à ça.
- `@warning` permet d'ajouter des avertissements. On peut vouloir prévenir, par exemple, qu'une fonction ne doit pas être utilisée en même temps qu'une autre, ou bien qu'un paramètre doit être d'abord correctement initialisé sous peine de voir le programme planter.

Les structures et les énumérations

Bien évidemment, il ne faut pas oublier de commenter nos types personnalisés. Non seulement le type lui-même, mais chacun des champs. Pour cela, nous disposons des attributs `@struct` et `@enum`. Chacun fonctionne sur le même principe. On écrit dans l'ordre :

1. l'attribut puis le nom du type ;
2. une description brève ;
3. une description plus détaillée.

De même, chaque champ peut faire l'objet de description courte, détaillée, d'avertissements, etc. En général, seule une description succincte est nécessaire. Examinez par vous-même un exemple de structure et d'énumération documentées.

```

/** @enum Couleur
 * @brief Liste des couleurs possibles.
 * @details La liste des trois couleurs applicables à l'écran de fond du
 * programme.
 */
enum class Couleur
{
    /// @brief Un bleu profond.
    Bleu,
    /// @brief Un rouge vif.
    Rouge,
    /// @brief Un vert magnifique.
    Vert
};

/** @struct InformationsPersonnelles
 * @brief Les informations personnelles d'un membre.
 * @details Permet de stocker, sous un même type, les nom/prénom ainsi
 * que le genre et l'âge d'un membre.
 * Il n'y a pas de limite à la taille du nom ou du prénom du membre.
 */
struct InformationsPersonnelles
{
    /// @brief Le prénom du membre.
    std::string prenom;
    /// @brief Le nom de famille du membre.
    std::string nom;
    /// @brief Femme ou homme ?
    std::string sexe;
    /// @brief L'âge du membre.
    int age;
};

```

19.4. Quelques bonnes pratiques

Vous savez maintenant documenter votre code et c'est une excellente chose. Une bonne documentation fait gagner du temps et constitue la meilleure alliée d'un développeur découvrant un nouveau code. Mais doit-on tout documenter ? Absolument tout ? Y a-t-il des bonnes pratiques ?

Ce qu'il faut documenter

Dans l'idéal, toutes vos fonctions devraient être documentées. Utilisez Doxygen pour détailler vos fonctions, leur utilisation, ce qu'elles attendent, les exceptions qu'elles peuvent lancer, etc. C'est assez fastidieux, on ne va pas le nier. Mais ne pas le faire est une très mauvaise pratique. Qui n'a jamais pesté contre un appareil qu'il a dû utiliser sans manuel ni instructions ? Si quelqu'un vient à utiliser votre code, il n'aimera pas non plus avoir à deviner ce que celui-ci fait, les exceptions qu'il lance, etc.



Mais si je suis tout seul, n'est-ce pas un peu inutile ?

Cela vous servira même si vous êtes seul. Ne croyez pas que vous vous souviendrez de ce que font vos 4000 lignes de code quand vous rouvrirez le projet après plusieurs mois d'absence s'il n'y a pas de documentation. Ce qui vous semble clair maintenant ne le sera peut-être pas plus tard. Épargnez-vous des problèmes, documentez. Pour ne pas vous démotiver face à l'immensité de cette tâche, commencez tôt et faites la petit à petit. Ainsi, au fur et à mesure que votre base de code augmentera, elle sera bien documentée et facile à comprendre.

Les fichiers d'en-tête ou sources ?

Nous n'aimons pas la recopie. C'est une source d'erreurs et d'incompréhensions. Imaginez que la documentation d'un prototype dans un fichier d'en-tête soit légèrement différente de celle de son implémentation dans le fichier source. Laquelle est à jour ? Un vrai nœud à problèmes que de maintenir deux fois la même documentation à jour.

Heureusement, Doxygen est assez intelligent et affiche la documentation même si on demande à voir celle du fichier source. Il comprend qu'on a affaire aux mêmes fonctions, avec d'un côté le prototype et de l'autre l'implémentation.

Rien ne vaut un exemple

Parfois, certaines fonctions sont assez compliquées à utiliser et avoir un exemple fonctionnel sous les yeux aide vraiment. Vous-même l'avez certainement remarqué en vous plongeant dans la documentation C++. Voilà pourquoi nous avons nous-mêmes choisi de séparer cette section du reste de la présentation de Doxygen, pour que vous compreniez bien que c'est important.

Une façon de faire consiste à utiliser l'attribut `@note`, pour bien le différencier des autres sections, puis à fournir un court exemple entre les attributs `@code` et `@endcode`.

```
/** @brief Détermine si une expression est correctement parenthésée.  
* @details La fonction regarde si on a bien le même nombre de  
  parenthèses  
* ouvrantes et fermantes et si celles-ci sont bien ordonnées.  
*  
* @param[in] expression Une chaîne de caractères à analyser.  
*  
* @returns `true` si l'expression est bien parenthésée, `false` sinon.  
*  
* @note Voici un exemple d'utilisation.
```



```

* @code
* std::string const test { "(((( ))" };
* parentheses(test);
* @endcode
*/
bool parentheses(std::string const & expression)
{
    // Implémentation.
}

```

Commentaire vs documentation

La découverte et l'utilisation de Doxygen signifie-t-elle la fin des commentaires comme nous avons appris dès le [chapitre introductif à C++](#) ? Bien sûr que non. En fait, les deux ne sont pas en concurrence, ils servent des buts différents.

La documentation vise à être autant que possible exhaustive et à décrire le quoi et le comment. On s'en sert pour expliquer ce que fait une fonction, les paramètres qu'elle attend, ce qu'elle renvoie, les exceptions qu'elle peut lever, ses contrats, etc. On s'en sert aussi pour illustrer le fonctionnement du code par des exemples, comme vu précédemment. *Si la documentation est bien faite, un développeur n'aura jamais besoin d'aller lire le code source pour comprendre.*

Les commentaires, eux, sont plutôt à usage interne et expliquent le pourquoi. On s'en sert pour *expliquer ce que le code ne dit pas*, pour décrire les raisons qui ont poussé à faire ou ne pas faire telle ou telle chose. Ils seront donc moins présents certes, mais pas moins utiles.

De bons commentaires...

Par exemple, on peut vouloir expliquer le choix d'une structure de données qui sort de l'ordinaire. Vous rappelez-vous de `std::map` et `std::unordered_map` ? La deuxième est souvent choisie pour des raisons de performance. Ainsi, vous voudrez peut-être expliquer pourquoi vous avez choisi la première avec un bref commentaire.

```

// Utilisation de `std::map` car les clés doivent rester
// triées dans l'ordre.
using Data = std::map<std::string, int>;

```

On peut aussi s'en servir pour justifier un code un peu sale ou exotique.

```
int const x { 0 };
// Le +1 est nécessaire car fonction_externe est boguée quand x vaut 0.
int resultat { fonction_externe(x) + 1 };
```

De la même manière, admettons que vous voulez tester si un nombre N est premier ou non, et appliquer une astuce consistant à ne tester que de 2 à $\sqrt{N}$. Cela pourrait faire l'objet d'un commentaire explicatif.

```
int const N { 24 };
bool est_premier { true };

// On ne teste que jusqu'à racine de N pour gagner du temps.
for (int i { 2 }; i < std::sqrt(N); ++i)
{
    if (N % i == 0)
    {
        est_premier = false;
        break;
    }
}
```

Ces commentaires sont utiles, car seulement en lisant le code, vous n'auriez pas compris pourquoi il a été écrit de cette façon.

...et de mauvais commentaires

Malheureusement, ce qu'on voit le plus souvent, ce sont les mauvais commentaires. Qu'ils se contentent de dire ce que dit déjà le code, qu'ils soient mal écrits ou encore pas clairs, le fait est qu'ils n'apportent rien de bon. Un exemple ? La plupart des commentaires écrits dans ce livre répètent ce que le code dit. Bien sûr, dans votre cas, c'est nécessaire car vous êtes en plein apprentissage. Par contre, pour des développeurs chevronnés, ce genre de commentaires est inutile.

Un autre type de commentaire problématique : celui qui explique le code parce que ce dernier n'est pas clair. C'est le cas dans l'exemple suivant.

```
// Si la vie atteint une valeur critique (<100).
if (vie < 100)
{
    // Du code.
}
```

Le code pourrait être amélioré en utilisant une constante, ce qui supprimerait le commentaire au passage.

```
int const niveau_vie_critique { 100 };

// ...

if (vie < niveau_vie_critique)
{
    // Du code.
}
```

De même, les commentaires signalant une accolade fermante, ceux signalant la date de dernière modification, ceux retraçant l'historique des modifications d'un fichier, tout ça fait partie des commentaires qui n'ont pas lieu d'être.

***Note** > La distinction peut vous sembler dure à comprendre et la frontière entre un bon et un mauvais commentaire bien mince. Si vous souhaitez aller plus loin, un chapitre du livre [Coder proprement de Robert C. Martin](#) lui est complètement dédié.*

19.5. Récapitulons !

- La documentation est une des meilleures amies du développeur.
- La documentation C++ est classée en divers thèmes, allant du langage lui-même aux algorithmes, en passant par `std::string` et les fonctions mathématiques.
- On retrouve souvent les mêmes sections, décrivant les différents prototypes, les paramètres, les exceptions possibles, ainsi que des exemples concrets.
- Doxygen nous permet d'écrire la documentation de nos programmes. C'est une bonne habitude à prendre.
- Certaines bonnes pratiques peuvent nous aider à écrire des commentaires et de la documentation de qualité.

SOLUTIONS

section Remplacer une chaîne de caractères par une autre

Comme on parle de chaînes de caractères, on va donc chercher du côté de `basic_string`. On remarque, dans la liste des fonctions disponibles, une certaine `replace`. Le premier prototype est justement celui qui répond à l'énoncé. On obtient donc ce qui suit.

```
#include <cassert>
#include <iostream>
#include <string>

int main()
{
    std::string phrase { "J'aimerais annuler ce texte." };
    std::string const mot { "remplacer" };

    // Le code.
    phrase.replace(10, 7, mot);

    std::cout << phrase << std::endl;
    assert(phrase == "J'aimerais remplacer ce texte." && "Le texte
n'a pas été modifié correctement.");
    return 0;
}
```

section Norme d'un vecteur

En parcourant la liste des fonctions contenues sur cette page de documentation, on note le triplet `hypot`, `hypotf` et `hypotl`. La description associée dit que ces fonctions calculent la racine carrée de la somme des carrés de deux nombres, soit $\sqrt{x^2 + y^2}$. Bien que, de par son nom, cette fonction semble plutôt destinée à calculer une hypoténuse, elle se révèle quand même adaptée. La solution est toute simple.

```
#include <cassert>
#include <cmath>

int main()
{
    int const x { 5 };
    int const y { 12 };
    int norme { 0 };

    // Le code pour résoudre l'exercice.
    norme = std::hypot(x, y);
}
```

SOLUTIONS

```
    assert(norme == 13 && "La norme d'un vecteur v(5;12) vaut 13.");
    return 0;
}
```

section Nombres complexes

Sous la rubrique NUMERICS LIBRARY, vous noterez l'entrée `complex`. Sur cette page, vous découvrirez un nouveau type : `complex`. C'est un *template*, donc on va devoir lui passer un type, en l'occurrence `double`. On voit ensuite, plus bas, la fonction `sqrt`, qui répond à la première partie de l'exercice. C'est une fonction non membre, donc qui s'utilise en se préfixant de `std::`. Plus haut, parmi les fonctions membres, on voit `real` et `imag`, qui donnent respectivement la partie réelle et la partie imaginaire de notre résultat complexe. On a donc tout ce qu'il nous faut.

```
#include <complex>
#include <iostream>

int main()
{
    auto resultat = std::sqrt(std::complex<double>{ -1, 0 });
    std::cout << "La racine carrée de -1 donne (" << resultat.real()
    << "," << resultat.imag() << "i)." << std::endl;
    return 0;
}
```

section Transformations

Qui dit algorithme dit bien souvent le fichier correspondant `<algorithm>`. C'est effectivement dans celui-ci que nous allons trouver ce qu'il nous faut et l'algorithme en question se nomme `std::transform`. Avez-vous écrit un code comme celui-ci ?

```
std::transform(std::begin(phrase), std::end(phrase),
    std::begin(sortie), [](char c) -> char
    {
        return std::toupper(c);
    });
```

Si oui, alors vous n'avez pas bien lu la documentation. En effet, lancez ce programme et il plantera. Pourquoi ? Parce que la chaîne de sortie est vide, donc on ne peut pas modifier les éléments d'une chaîne vide puisqu'il n'y en a pas. La documentation apporte une précision :

SOLUTIONS

`std::transform` applies the given function to a range and stores the result in another range, beginning at `d_first`.

—Documentation de `std::transform`

Il faut que le conteneur de sortie ait la même taille que celui d'entrée. Ou, tout du moins, en théorie. Si vous lisez le code d'exemple situé en fin de page, vous allez remarquer la présence d'une fonction inconnue jusque-là : `std::back_inserter`. En lisant [sa documentation](#), on apprend qu'elle crée un `std::back_inserter_iterator`. Qu'à cela ne tienne, allons lire la documentation de ce nouveau genre d'itérateur. Celle-ci nous donne une information très intéressante.

`std::back_inserter_iterator` is an *LegacyOutputIterator* that appends to a container for which it was constructed. The container's `push_back()` member function is called whenever the iterator (whether dereferenced or not) is assigned to.

—Documentation `std::back_inserter`

Traduit, cela signifie qu'à chaque fois que l'itérateur est modifié, la fonction `push_back` du conteneur en question est appelée. C'est exactement ce qu'il nous faut pour notre conteneur de sortie vide. Voici donc le code final.

```
#include <algorithm>
#include <cctype>
#include <iostream>
#include <string>

int main()
{
    std::string const phrase { "Voici une phrase longue que je veux
en majuscules." };
    std::string sortie { "" };

    std::transform(std::begin(phrase), std::end(phrase),
std::back_inserter(sortie), [](char c) -> char
    {
        return std::toupper(c);
    });

    std::cout << sortie << std::endl;
    return 0;
}
```

INDICES

section Norme d'un vecteur

Il faut chercher dans les [fonctions mathématiques](#).

section Nombres complexes

Vous devez chercher du côté de la page [complex](#).

20

Compilation en cours...

Depuis le début de ce livre, nous avons compilé et recompile de nombreuses fois nos programmes. Vous savez que la compilation transforme notre code source en un programme exécutable qui sera compris par l'ordinateur. Mais, concrètement, que se passe-t-il pendant la compilation ? Quelles en sont les différentes étapes ? Le but de ce chapitre est de vous présenter en détail la compilation d'un code C++.

20.1. Le préprocesseur

La toute première étape est celle du préprocesseur. Ce nom vous est familier ? Normal, nous avons déjà fait appel à lui pour chaque ligne qui commence par `#`.

Inclure des fichiers

Dans le cas de la directive `#include`, il s'agit d'inclure tout le fichier d'en-tête demandé dans le fichier source en question. C'est pour cette raison que ceux qui compilent en ligne de commande ne doivent pas ajouter les `.hpp` à la liste des fichiers à compiler, puisqu'ils sont déjà inclus dans tous les fichiers `.cpp`.

Il est possible de voir à quoi ressemble notre code après son passage.

- Avec Visual Studio, faire un clic droit sur le nom du projet et cliquer sur PROPRIÉTÉS. Là, on va dans PROPRIÉTÉS DE CONFIGURATION > C/C++ > PRÉPROCESSEUR, puis on met le champ PRÉTRAITER DANS UN FICHIER à OUI (/P). Attention cependant, cette option bloque la suite de la compilation. Donc désactivez la sortie du préprocesseur une fois que vous avez fini d'explorer cette option.
- Avec Qt Creator, il faut modifier le fichier `.pro` en lui ajoutant la ligne `QMAKE_CXX_FLAGS += -save-temps`. Le fichier obtenu après traitement a une extension en `.ii`.
- Pour ceux qui travaillent en ligne de commande, pour GCC c'est `g++ -E fichier.cpp -o fichier.i` et pour Clang, `clang++ -E fichier.cpp -o fichier.i`.

Le résultat n'est pas le même en fonction du compilateur, mais le fichier devient conséquent après le passage du préprocesseur. Pour un simple `#include <iostream>`, on

obtient plus de 17.800 lignes pour Clang, plus de 18.000 lignes avec GCC et même plus de 50.000 lignes avec Visual Studio (dont pas mal de vides, certes) !

Conditions

Mais le préprocesseur ne se contente pas d'inclure bêtement ce qu'on lui demande et c'est tout. Il est capable d'exécuter d'autres instructions, notamment celles permettant d'éviter les inclusions multiples. Ce sont des *instructions conditionnelles* pour le préprocesseur. Voici un exemple.

```
#ifndef TEST_HPP
#define TEST_HPP

#endif
```

La première ligne teste si une constante de préprocesseur `TEST_HPP` existe. Si ce n'est pas le cas, alors on exécute le code qui suit. Justement, la ligne d'après définit la constante `TEST_HPP`. Ainsi, si l'on tente d'inclure de nouveau le fichier, la première condition devient fausse et rien n'est inclus. Enfin, l'instruction `#endif` permet de fermer la portée du `#ifndef`, tout comme les accolades fermantes pour `if`.

Ce mécanisme de conditions est très utilisé au sein des fichiers d'en-tête de la bibliothèque standard. En effet, cela permet d'adapter le code en fonction de la version du compilateur ou de la norme C++, entre autres. Par exemple, la bibliothèque standard fournie avec Visual Studio utilise le code suivant pour ne définir la fonction `std::for_each_n` que si l'on compile en C++17.

```
#if _HAS_CXX17

// Le code de la fonction std::for_each_n,
// disponible uniquement à partir de C++17.

#endif /* _HAS_CXX17 */
```

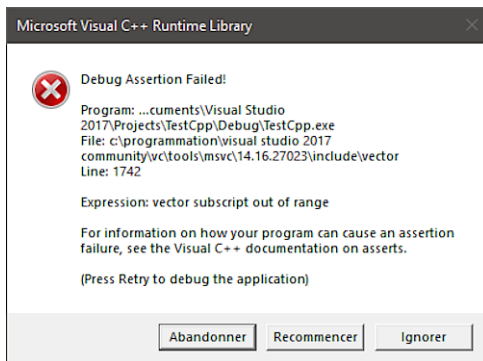
Debug ou release ?

Le préprocesseur est également utilisé pour modifier la bibliothèque standard selon qu'on compile en *debug* ou en *release*. Il s'agit de modes de compilation qui sont utilisés en fonction du moment.

Debug — quand on code

Le mode *debug* inclut dans le code de la bibliothèque standard de nombreuses vérifications et autres tests. C'est le mode dans lequel est compilé le code quand on est encore en plein développement, qu'on cherche à corriger des bugs ou qu'on fait des tests. Depuis le début du livre, nous ne compilons qu'en mode *debug*. C'est pour cette raison que, lorsque nous utilisons [un indice trop grand](#), le programme plante avec un message d'erreur.

Figure 20.1 : Erreur affichée en mode *debug*.



Si l'on examine le code de `std::vector` tel qu'il est fourni avec Visual Studio, on trouve les lignes suivantes. C'est le message qu'on voit à la [Figure 20.1](#).

```
#if _ITERATOR_DEBUG_LEVEL != 0
    _STL_VERIFY(_Pos < size(), "vector subscript out of range");
#endif /* _ITERATOR_DEBUG_LEVEL != 0 */
```

La macro `_ITERATOR_DEBUG_LEVEL` est elle-même définie uniquement si une macro `_HAS_ITERATOR_DEBUGGING` est définie comme valant autre chose que `0`. Cette dernière dépend de l'existence de la macro `_DEBUG`, qui est définie quand on compile en mode *debug*.

Release — quand on distribue

Le mode *release*, lui, se débarrasse de toutes ces vérifications, optimise le code pour le rendre plus rapide à exécuter et réduit la taille de l'exécutable qu'on obtient en sortie.

On compile avec ce mode quand le programme est prêt et qu'on veut le distribuer à d'autres personnes. À ce stade, on juge que le programme est suffisamment bon et correctement testé. Il est donc inutile de continuer à inclure tous les tests et autres vérifications, qui rendent le code plus gros et plus lent.

Typiquement, les assertions sont désactivées en *release*. Bien souvent, `assert` n'est pas implémentée en tant que fonction, comme on aurait pu le penser à première vue, mais à l'aide du préprocesseur, ce qui permet de la supprimer du code, ou de la rendre inopérante, quand on passe en *release*.

20.2. La compilation

Maintenant que le préprocesseur a fini son travail, place à la compilation à proprement parler. Chaque fichier obtenu après le passage du préprocesseur, qu'on appelle *unité de compilation*, est traité individuellement. Le but de cette phase est de *transformer le C++ en code exécutable* par l'ordinateur.

C'est lors de cette étape que beaucoup de vérifications ont lieu et donc d'erreurs. Par exemple, dans le cas où le compilateur ne trouve pas la déclaration d'une fonction que vous appelez, il décidera d'arrêter la compilation. Vous aurez alors un message d'erreur vous indiquant ce qui n'a pas fonctionné.

Les templates

Comme dit au chapitre [Envoyez le générique !](#), le compilateur va instancier chaque template qu'il trouve avec les bons types. C'est pendant cette étape que des erreurs incompréhensibles de templates peuvent arriver, par exemple quand on tente d'appliquer `std::sort` sur une `std::list`.

Prenez l'exemple suivant d'un code C++ très simple.

```
template <typename T>
void fonction(T const & t)
{
    T const resultat { t * 2 };
}

int main()
{
    fonction(42);
    fonction(3.1415);
    return 0;
}
```

Avec Clang, il est possible d'utiliser les options `-xclang -ast-print -fsyntax-only` pour afficher le code après l'instanciation des templates. Avec GCC, c'est la commande `-fdump-tree-original` qui génère des fichiers `.original` qu'on peut afficher.

On voit dans le résultat suivant, obtenu avec Clang, que le template est instancié deux fois, avec `int` et avec `double`.

```
template <typename T = int> void fonction(const int &t) {
    const int resultat{t * 2};
}

template <typename T = double> void fonction(const double &t) {
    const double resultat{t * 2};
}

template <typename T> void fonction(const T &t) {
    const T resultat{t * 2};
}

int main() {
    fonction(42);
    fonction(3.1415);
    return 0;
}
```

constexpr

Nous avons dit au chapitre [Mais où est la doc ?](#) que nous reviendrions sur ce mot clé en temps voulu. Le moment est venu d'en parler, car il a un rapport avec la compilation. En effet, il indique au compilateur qu'une variable ou une fonction peut déjà être évaluée pendant la compilation et non plus seulement à l'exécution du programme. L'avantage, c'est que le compilateur effectue le travail une fois pour toutes, ce qui rend l'exécution du programme plus rapide.

Des constantes

En déclarant des constantes avec le mot clé `constexpr`, on les rend utilisables à la compilation. Typiquement, dans le cas où l'on fixe d'avance la hauteur et la largeur d'une matrice, utiliser `std::array` est tout à fait possible.

```
constexpr std::size_t hauteur { 5 };
constexpr std::size_t largeur { 4 };

std::array<int, hauteur * largeur> matrice {};
```

Note > Implicitement, utiliser `constexpr` sur une variable revient à la déclarer `const`, ce qui est logique. Personnellement, dans le but d'être explicite, nous précisons également `const`, mais ce n'est pas une obligation, simplement une question de goût.

Par contre, pour que l'instruction soit valide et que le programme puisse compiler, il faut qu'elle soit *évaluable à la compilation*. Cela signifie que toute l'expression doit être comprise et utilisable par le compilateur pendant la compilation et que rien ne doit dépendre de variables non constantes, d'entrées/sorties, etc.

```
int main()
{
    // Valide parce que 5 est un littéral évaluable à la compilation.
    constexpr int const x { 5 };
    // Valide parce que x est constexpr.
    constexpr int const y { 2 * x };

    // Invalide car b n'est pas une constante.
    int b { 10 };
    constexpr int const non { b - 5 };

    return 0;
}
```

Des fonctions

Grâce au mot clé `constexpr`, nous sommes capables d'écrire des fonctions qui s'exécuteront pendant la compilation, ce dont nous étions pas capables jusque-là, tout du moins sans utiliser les templates. En plus, la syntaxe est très simple, puisqu'il suffit d'ajouter `constexpr` au prototype de la fonction.

```
constexpr int add(int a, int b)
{
    return a + b;
}
```

Cela a néanmoins un effet un peu particulier.

- Si la fonction est appelée avec des arguments `constexpr`, comme des littéraux ou des variables `constexpr`, alors elle est exécutée à la compilation.
- Dans le cas contraire, avec des arguments non-`constexpr`, elle se comporte normalement et est exécutée pendant l'exécution du programme.

Prenons un exemple en calculant la **factorielle** d'un nombre entier. C'est une opération mathématique simple, qui se définit comme le produit de tous les entiers de 1 à n . La formule est la suivante.

$$n! = \prod_{i=1}^n i = 1 \times 2 \times \dots \times n$$

Ainsi, 2! vaut 2, mais 10! vaut déjà 3628800. C'est donc une opération qu'il convient de faire calculer au compilateur si cela est possible. En plus, son implémentation est toute simple, comme vous le montre le code suivant. Notez que dans le premier cas la fonction est appelée pendant la compilation, alors que dans le second elle l'est pendant l'exécution.

```
#include <array>
#include <iostream>

constexpr int factorielle(int n)
{
    int resultat { 1 };
    for (int i { 1 }; i <= n; ++i)
    {
        resultat *= i;
    }
    return resultat;
}

int main()
{
    // Exécuté à la compilation.
    std::array<int, factorielle(3)> tableau {};

    // Exécuté quand le programme est lancé.
    int entier { 0 };
    std::cout << "Donne un entier entre 1 et 10 : ";
    std::cin >> entier;
    std::cout << "Sa factorielle vaut "
              << factorielle(entier) << "." << std::endl;

    return 0;
}
```



Alors pourquoi ne pas déclarer toutes nos fonctions `constexpr` ? Comme ça on gagne en performance.

Parce que `constexpr` est soumis à des restrictions. En C++11, il ne pouvait pas y avoir de boucles ou de `if/else`. Depuis C++14, ces restrictions ont été levées, ce qui permet l'implémentation de la factorielle telle que vous la voyez plus haut. Et au-delà de ces considérations, une fonction ne peut être `constexpr` que si toutes ses instructions sont évaluables pendant la compilation. Cela nous interdit donc d'utiliser les entrées/sorties

ou des fonctions qui ne sont pas `constexpr`. Voilà pourquoi toutes nos fonctions ne peuvent pas être exécutées pendant la compilation.

La compilation à proprement parler

Maintenant, il s'agit de transformer tout ce code C++ en quelque chose de compréhensible par l'ordinateur. Lui ne comprend que les 0 et les 1. Il n'a que faire de nos noms de variables, de fonctions ou de types. La transformation du C++ en code machine se fait en plusieurs étapes. Nous pouvons en citer quelques-unes :

- **Analyse lexicale**, où le compilateur déduit que `auto` ou `int` sont des mots clés, `main` ou `x` des identificateurs, `=` ou `+` des opérateurs, etc.
- **Analyse syntaxique**, où il vérifie que la syntaxe a du sens et ce qu'elle signifie. Ainsi, écrire `auto = 5` sera rejeté pendant la compilation car cette syntaxe n'a aucun sens.
- **Optimisations**, notamment en mode *release*. Cette étape supprime du code inutile, modifie certaines portions pour qu'elles s'exécutent plus vite, etc.
- **Génération du code pour la plateforme cible**. Le `fichier objet` ne sera pas le même selon la plateforme ciblée, ce qui signifie que le compilateur fera des adaptations à destination de Windows, GNU/Linux, etc.

Note > Nous ne détaillerons pas plus la compilation du C++ à proprement parler car c'est affreusement compliqué. C++ a la réputation d'être un langage dont l'écriture du compilateur est très complexe et cette réputation n'est pas volée.

Le compilateur va au final générer des *fichiers objets*, un pour chaque fichier `.cpp`, chaque *unité de compilation*, qu'il trouve. Si aucune modification n'est détectée depuis la dernière compilation, alors le compilateur réutilise le fichier objet précédemment créé, ce qui accélère la compilation d'un projet.

Un fichier objet, reconnaissable à son extension en `.o` ou `.obj`, n'est rien d'autre que du code machine mais seul, sans aucun lien avec le reste du monde. Tel quel, il n'est pas exécutable, car il lui manque des informations. Pour cela, il faut utiliser le *linker*, ce que nous allons voir à la section suivante.

Dans le cas de Visual Studio, les fichiers objets sont présents dans le dossier de sortie. Dans le cas de GCC et Clang, il faut utiliser l'option `-c` lors de la compilation.

```
> g++ -std=c++20 -c test.cpp -o test.o
> clang++ -std=c++20 -c test.cpp -o test.o
```

Une étape intermédiaire cachée

Beaucoup de compilateurs génèrent maintenant directement des fichiers objets, rendant invisible une étape intermédiaire, la transformation de C++ en [Assembleur](#), un langage de bas niveau très proche du langage machine. Ce code était ensuite passé à un *assembleur*¹, qui s'occupait de le transformer en fichier objet.

Il est cependant possible d'obtenir le code Assembleur généré avec la bonne option.

- Avec Visual Studio, allez dans les propriétés du projet, puis dans C/C++ > FICHIERS DE SORTIE. Là, réglez l'option SORTIE DE L'ASSEMBLEUR sur *Assembleur, code machine et source (/FAcs)*.
- Avec Qt Creator, si vous avez déjà ajouté la ligne `QMAKE_CXXFLAGS += -save-temps` à votre fichier .pro, vous n'avez rien d'autre à faire. Le fichier assembleur est celui avec une extension .s.
- Avec GCC et Clang, il faut compiler avec l'option `-S`.

Attention, il s'agit d'un langage très... brut de décoffrage. Si vous ne nous croyez pas, jetez un œil par vous-même, vous allez être surpris.

Influer sur la compilation

Il est possible de passer d'autres informations au compilateur, notamment pour lui demander d'effectuer certaines opérations ou des tests supplémentaires. C'est important car plus le compilateur fait de vérifications, plus il est en mesure de nous signaler d'éventuels problèmes dans notre code : les fameux *warnings*. Et ne vous y trompez pas, ces derniers ne sont pas là pour vous embêter, le compilateur est là pour vous aider.

Un mot sur les outils

Comme le compilateur fourni avec Visual Studio est différent de GCC ou de Clang, les options de compilation et les *warnings* sont aussi différents et ne s'activent pas forcément de la même façon.

Avec Visual Studio, allez dans les propriétés du projet, puis C/C++ > GÉNÉRAL et là, mettez le champ NIVEAU D'AVERTISSEMENT à la valeur NIVEAU 4 (/W4). Cela augmente le nombre de vérifications effectuées et donc de *warnings* générés. Nous ne conseillons pas (/WALL)

¹Oui, il s'agit du même nom pour le langage et pour l'outil qui compile ce langage. On les différencie toutefois en employant un A majuscule pour parler du langage.

parce que cette option est si sévère qu'elle va même générer des avertissements dans le code de la bibliothèque standard.

Avec GCC ou Clang, il faut rajouter des options supplémentaires, qu'on appelle couramment des *flags*. Il en existe énormément, tant pour [GCC](#) que pour [Clang](#). Mais les deux plus utilisés sont ceux que nous connaissons déjà, à savoir `-Wall` et `-Wextra`, qui activent en sous-main les *warnings* les plus fréquents.

```
> g++ -std=c++20 -Wall -Wextra test.cpp -o test.out
> clang++ -std=c++20 -Wall -Wextra test.cpp -o test.out
```

***Note** > Nous vous conseillons de toujours activer les warnings suggérés ci-dessus. Plus le compilateur travaille pour vous, plus vous pouvez corriger tôt d'éventuels problèmes et améliorer ainsi la qualité de votre code.*

Conversion entre types

Dès le début de ce livre, nous avons vu des types simples, permettant de stocker des caractères (`char`), des entiers (`int`, `std::size_t`) et des réels (`double`). Ils sont bien pratiques, mais si on commence à les mélanger, on peut avoir des problèmes. En effet, comme signalé au chapitre [Erreur, erreur, erreur...](#), tous n'ont pas la même taille. Un `int`, par exemple, ne peut pas stocker des nombres aussi grands qu'un `double`. Ainsi, convertir un `double` en `int` entraîne une perte d'information. C'est un peu comme vouloir transvaser de l'eau depuis un seau dans un gobelet.

Le code suivant illustre bien le principe, puisque en passant d'un `double` à un `int`, on perd comme information la partie décimale du nombre.

```
#include <iostream>

double fonction()
{
    return 3.141582;
}

int main()
{
    int const x { fonction() };
    std::cout << "Voici un nombre : " << x << std::endl;

    return 0;
}
```

D'ailleurs, le compilateur ne va pas se gêner pour vous le dire.

```
[Visual Studio]
Erreur C2397 la conversion de 'double' en 'int' requiert une conversion
restrictive

-----

[GCC]
test.cpp: In function 'int main()':
test.cpp:10:27: warning: narrowing conversion of 'fonction()' from
'double' to 'int' inside { } [-Wnarrowing]
    int const x { fonction() };
                   ~~~~~^~
-----

[Clang]
test.cpp:10:19: error: type 'double' cannot be narrowed to 'int' in
initializer list [-Wc++11-narrowing]
    int const x { fonction() };
                   ^~~~~~
test.cpp:10:19: note: insert an explicit cast to silence this issue
    int const x { fonction() };
                   ^~~~~~
                   static_cast<int>( )
```

Le compilateur nous prévient que c'est une erreur et qu'il faut une *conversion explicite* [ou en anglais *cast*] pour résoudre le problème. Clang est le plus sympa puisqu'il propose une solution, qui consiste à utiliser le mot clé `static_cast`. Ce dernier permet de dire explicitement au compilateur qu'on veut convertir un type vers un autre, en l'occurrence ici un `double` vers un `int`. Il s'utilise comme suit.

`static_cast<le type qu'on veut en résultat>(une variable ou une littérale)`

Une fois la conversion explicite appliquée, le code fonctionne sans problème.

```
#include <iostream>

double fonction()
{
    return 3.141582;
}

int main()
{
    int const x { static_cast<int>(fonction()) };
    std::cout << "Voici un nombre : " << x << std::endl;

    return 0;
}
```

Sans les avertissements du compilateur, l'erreur serait sans doute passée inaperçue. Si l'on comptait faire des calculs avec nos valeurs réelles, ceux-ci auraient été faussés puisque les parties décimales auraient disparu. Grâce à l'avertissement, nous pouvons soit corriger notre erreur, soit mettre le `cast` pour montrer que notre conversion est voulue et volontaire.

Initialisation oubliée

Imaginez que vous oubliez d'écrire les accolades `{}` après avoir déclaré une variable. C'est moyennement cool, puisque du coup on ne lui donne pas de valeur par défaut. Elle peut donc valoir tout et n'importe quoi. Le message du code suivant peut, ou non, s'afficher. C'est impossible à savoir.

```
#include <iostream>

int main()
{
    int x;

    if (x > 5)
    {
        std::cout << "J'ai l'impression d'oublier quelque chose..."
                   << std::endl;
    }

    return 0;
}
```

Heureusement, le compilateur ne manque pas de le souligner.

```
[Visual Studio]
Erreur C4700 variable locale 'x' non initialisée utilisée

-----

[GCC]
test.cpp: In function 'int main()':
test.cpp:7:5: warning: 'x' is used uninitialized in this function [-Wuninitialized]
    if (x > 5)
    ^~
-----

[Clang]
test.cpp:7:9: warning: variable 'x' is uninitialized when used here [-Wuninitialized]
    if (x > 5)
    ^
test.cpp:5:10: note: initialize the variable 'x' to silence this warning
```

```
int x;
  ^
  = 0
```

Paramètres inutilisés

Imaginez que vous déclarez une fonction attendant deux paramètres, mais que vous n'en utilisez qu'un seul. C'est peut-être volontaire, parce que vous n'avez pas encore fini de l'implémenter, mais c'est peut-être une erreur, soit que vous avez oublié une portion de code à écrire, soit oublié de supprimer un paramètre désormais inutile.

```
#include <iostream>
#include <string>

void fonction(std::string const & chaine, bool en_majuscule)
{
    std::cout << chaine << std::endl;
}

int main()
{
    std::string const une_phrase { "Du texte." };
    fonction(une_phrase, true);

    return 0;
}
```

Votre compilateur se demande quelle est votre intention et vous affiche donc un message.

```
[Visual Studio]
Warning C4100: 'en_majuscule' : paramètre formel non référencé

-----

[GCC]
test.cpp: In function 'void fonction(const string&, bool)':
test.cpp:4:48: warning: unused parameter 'en_majuscule' [-Wunused-
parameter]
    void fonction(std::string const & chaine, bool en_majuscule)
                                                    ^~~~~~
-----

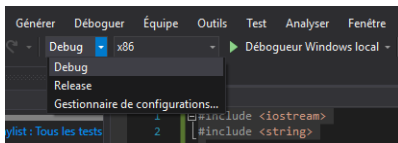
[Clang]
test.cpp:4:48: warning: unused parameter 'en_majuscule' [-Wunused-
parameter]
void fonction(std::string const & chaine, bool en_majuscule)
                                                    ^
1 warning generated.
```

Release et debug

Les options de compilation servent aussi à indiquer au compilateur le mode choisi. Il est ainsi capable d'optimiser le programme pour le rendre plus petit et plus rapide. Une partie de ce travail est faite quand le préprocesseur passe, mais d'autres améliorations et optimisations sont effectuées par le compilateur.

Avec Visual Studio, il faut simplement sélectionner l'entrée correspondante dans le menu déroulant DEBUG/RELEASE, qui se trouve près du triangle vert DÉBOGUEUR WINDOWS LOCAL.

Figure 20.2 : Choix du mode de compilation avec Visual Studio 2019



Avec GCC et Clang, c'est l'option de compilation `-O2` qu'il faut passer. Celle-ci cherche à optimiser le programme pour rendre son exécution plus rapide.

```
g++ -std=c++20 -Wall -Wextra -O2 test.cpp -o test.out
```

20.3. Le linker

Note > Les propos ci-dessous sont illustrés sous GNU/Linux, mais le principe et la plupart des explications sont valables quel que soit le système d'exploitation.

Maintenant que nous avons des fichiers objets, il faut les *lier* pour obtenir notre exécutable final. C'est justement le travail accompli pendant l'étape du *linkage*. Et pour illustrer mes explications, nous allons travailler avec le code ci-dessous, divisé en trois fichiers.

D'abord, le fichier d'en-tête `test.hpp`, qui ne contient qu'un simple prototype.

```
#ifndef TEST_HPP
#define TEST_HPP

int fonction();

#endif
```

Ensuite, le fichier `test.cpp`, qui est l'implémentation du prototype déclaré précédemment.

```
#include "test.hpp"

int fonction()
{
    return 38;
}
```

Et enfin le fichier `main.cpp`, le point d'entrée.

```
#include <iostream>
#include "test.hpp"

int main()
{
    int const x { 5 };
    std::cout << "Voici un calcul : " << x + fonction() << std::endl;

    return 0;
}
```

On génère donc le code. Que ce soit avec GCC, Clang ou en utilisant Visual Studio, on obtient deux fichiers objets, un par fichier source.

```
> g++ -std=c++20 -c test.cpp main.cpp
> clang++ -std=c++20 -c test.cpp main.cpp
```

Une question de symboles

Un fichier objet contient, entre autres, des symboles. Qu'est-ce que c'est ? Un symbole correspond au nom d'une fonction ou d'une variable. Les noms de toutes les fonctions et variables utilisées dans chaque fichier source sont donc présents dans les fichiers objets correspondants. On peut le voir en utilisant la commande `objdump -t fichier.o -C`.

```
> objdump -t test.o -C

test.o:      format de fichier elf64-x86-64

SYMBOL TABLE:
0000000000000000 1  df *ABS* 0000000000000000 test.cpp
0000000000000000 1  d  .text 0000000000000000 .text
0000000000000000 1  d  .data 0000000000000000 .data
0000000000000000 1  d  .bss  0000000000000000 .bss
0000000000000000 1  d  .note.GNU-stack
0000000000000000 .note.GNU-stack
0000000000000000 1  d  .eh_frame 0000000000000000 .eh_frame
0000000000000000 1  d  .comment 0000000000000000 .comment
0000000000000000 g  F  .text 000000000000000b fonction()
```

De nombreuses informations sont affichées, qui nécessiteraient beaucoup d'explications. Notons simplement qu'on retrouve notre identificateur `fonction` sur la dernière ligne. La lettre **F** indique qu'il s'agit d'une fonction. Devant, on note la présence de `.text`, la section dans laquelle `fonction` se trouve.

Sans rentrer dans les détails, il faut savoir qu'un programme est découpé en plusieurs sections. Une section regroupe certaines informations ensemble. Citons la section `.text` qui contient le code et `.rodata`, qui contient toutes les données non modifiables, comme les chaînes de caractères par exemple. Dans le cas du fichier objet `test.o`, comme `fonction` désigne une fonction, donc du code exécutable, il est normal que sa section soit `.text`. Et si on faisait la même analyse sur `main.o` ?

```
objdump -t main.o -C

main.o:      format de fichier elf64-x86-64

SYMBOL TABLE:
0000000000000000 1   df *ABS* 0000000000000000 main.cpp
0000000000000000 1   d  .text 0000000000000000 .text
0000000000000000 1   d  .data 0000000000000000 .data
0000000000000000 1   d  .bss 0000000000000000 .bss
0000000000000000 1   O .bss 0000000000000001 std::__iobuf
0000000000000000 1   d  .rodata 0000000000000000 .rodata
0000000000000059 1   F .text 0000000000000049
__static_initialization_and_destruction_0(int, int)
00000000000000a2 1   F .text 0000000000000015 _GLOBAL__sub_I_main
0000000000000000 1   d  .init_array 0000000000000000 .init_array
0000000000000000 1   d  .note.GNU-stack 0000000000000000 .note.GNU-stack
0000000000000000 1   d  .eh_frame 0000000000000000 .eh_frame
0000000000000000 1   d  .comment 0000000000000000 .comment
0000000000000000 g   F .text 0000000000000059 main
0000000000000000 *UND* 0000000000000000 std::cout
0000000000000000 *UND* 0000000000000000 _GLOBAL_OFFSET_TABLE_
0000000000000000 *UND* 0000000000000000 std::basic_ostream<char,
std::char_traits<char> >& std::operator<< <std::char_traits<char>
>(std::basic_ostream<char, std::char_traits<char> >&, char const*)
0000000000000000 *UND* 0000000000000000 fonction()
0000000000000000 *UND* 0000000000000000
std::ostream::operator<<(int)
0000000000000000 *UND* 0000000000000000 std::basic_ostream<char,
std::char_traits<char> >& std::endl<char, std::char_traits<char>
>(std::basic_ostream<char, std::char_traits<char> >&)
0000000000000000 *UND* 0000000000000000
std::ostream::operator<<(std::ostream& (*) (std::ostream&))
0000000000000000 *UND* 0000000000000000
std::ios_base::Init::Init()
0000000000000000 *UND* 0000000000000000 .hidden __dso_handle
0000000000000000 *UND* 0000000000000000
std::ios_base::Init::~Init()
0000000000000000 *UND* 0000000000000000 __cxa_atexit
```

Certaines choses ne nous surprennent pas, comme `main` désignée comme étant une fonction située dans `.text`. Par contre, d'autres lignes n'ont pour seule information que `*UND*`. Et on trouve des éléments familiers, comme `std::cout` ou encore `std::ostream::operator<<(int)`.

En fait, toutes les lignes contenant `*UND*` désignent des symboles n'ayant *aucune section liée*. Ce sont des symboles qui ne sont pas définis dans notre fichier objet. Il faut donc *lier celui-ci à d'autres fichiers objets*, notamment ceux qui contiennent ce qui nous manque.

Dans le cas de la bibliothèque standard, c'est fait automatiquement par le compilateur sans avoir besoin de ne rien faire. Mais pour résoudre le symbole `fonction()`, il faut lier nos deux fichiers objets. C'est très simple, la commande suivante vous le montre. Notez que l'option `-std=c++20` est absente, car ici on ne compile rien, on *li*ne [assemble] deux fichiers objets.

```
g++ test.o main.o -o mon_super_programme.out
clang++ test.o main.o -o mon_super_programme.out
```

Maintenant, on obtient un exécutable parfaitement fonctionnel. La compilation est terminée.

```
./super_programme.out
Voici un calcul : 43
```

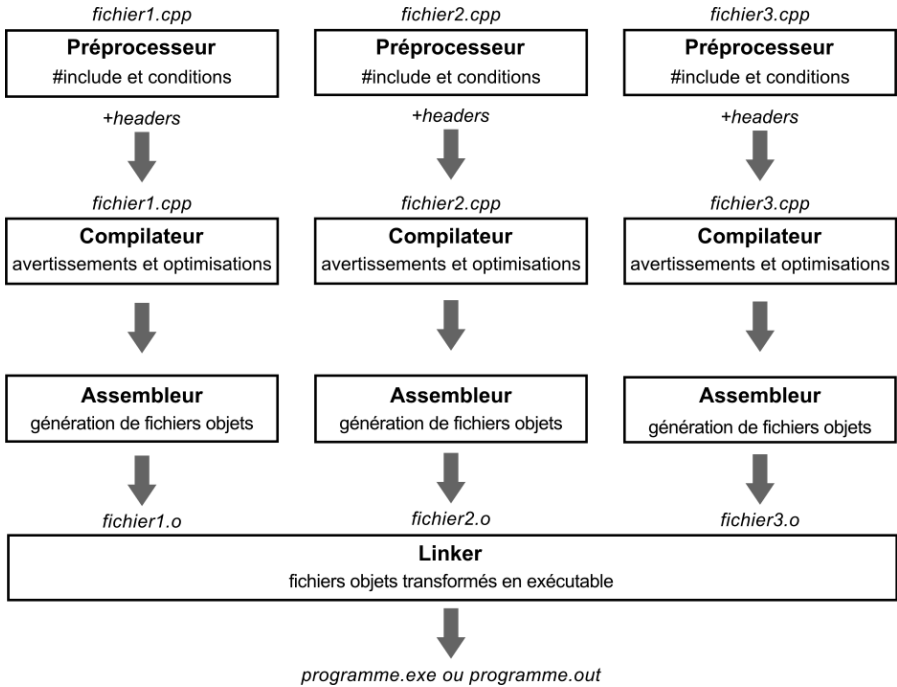
En revanche, si on tente de compiler seulement `main.o`, le *linkage* échouera avec une erreur comme celle ci-dessous.

```
> g++ main.o -o super_programme.out
main.o : Dans la fonction « main » :
main.cpp:(.text+0x27) : référence indéfinie vers « fonction() »
collect2: error: ld returned 1 exit status

> clang++ main.o -o super_programme.out
main.o : Dans la fonction « main » :
main.cpp:(.text+0x27) : référence indéfinie vers « fonction() »
clang: error: linker command failed with exit code 1 (use -v to see invocation)
```

20.4. Schéma récapitulatif

Le processus de compilation d'un programme C++ peut sembler vraiment complexe quand on démarre. C'est vrai qu'il s'agit d'un domaine avancé, mais avoir une idée globale du processus et de ce qui se passe sous le capot est important. Voici donc un schéma, car une image vaut mieux que mille mots.

Figure 20.3 : Processus de compilation

20.5. Récapitulons !

- Le préprocesseur commence le travail en remplaçant chaque directive `#include` par le contenu du fichier demandé et en exécutant des tests conditionnels pour activer ou désactiver certains morceaux de code.
- Le compilateur instancie les templates, exécute les fonctions `constexpr`, vérifie la qualité de notre code puis, en plusieurs phases, transforme le code C++ en fichier objet.
- Le *linker* assemble plusieurs fichiers objets pour résoudre des symboles manquants et achever le processus de compilation en générant un exécutable fonctionnel.
- Il n'est pas obligatoire de connaître chaque détail, mais avoir une vision globale est important.

21

Chasse aux bugs !

Malgré tout le soin apporté à la conception, malgré tous les *warnings* activés pendant la compilation, même avec toutes les bonnes pratiques du monde, nous faisons des erreurs et nous créons des *bugs* [ou *bogues* en français]. Il nous faut donc des outils supplémentaires pour les traquer et les éliminer. Ce chapitre va nous faire découvrir l'usage des débogueurs.

21.1. Le principe

À l'exécution d'un programme, il se passe beaucoup de choses et très vite. Une fonction en appelle une autre, une variable est modifiée, une exception lancée, etc. Il est donc difficile, d'un simple coup d'œil, de savoir d'où survient un problème. Le principe d'un débogueur est donc de dérouler l'exécution du programme pas à pas, ce qui permet à chaque instant de voir où on en est et quel est l'état de nos variables et de la mémoire.

Pour indiquer au débogueur de s'arrêter à un endroit précis dans le code, on pose ce qu'on appelle un *point d'arrêt* [ou *breakpoint* en anglais]. Chaque fois que le débogueur passe par un point d'arrêt, le programme en cours de débogage s'interrompt jusqu'à nouvel ordre et toutes les informations courantes sont affichées, comme la liste des fonctions appelées jusque-là [la *stack trace* en anglais], la valeur de toutes les variables de la portée courante, etc.

Pour les débogueurs les plus avancés, il est également possible de modifier des données du programme, de poursuivre l'exécution à un autre endroit du code pour sauter un crash ou une exception, observer le code assembleur généré et même de déboguer des programmes à distance, par Internet notamment.

Pour faire tout ce travail, le débogueur se repose sur les *symboles de débogage*, qui sont présents quand on compile en *debug* mais souvent éliminés quand on passe en *release*. Donc, afin de pouvoir utiliser cet outil puissant, veillez à passer en mode `DEBUG` si vous utilisez Visual Studio, ou bien à compiler votre programme avec l'option `-g`, pour charger ces fameux symboles.

```
> g++ -std=c++20 -g test.cpp -o test.out
> clang++ -std=c++20 -g test.cpp -o test.out
```

21.2. Un code d'exemple

Pour illustrer ce chapitre, nous allons utiliser le morceau de code suivant. Il est très basique mais il nous permettra de découvrir le principe du débogage en douceur.

```
#include <iostream>
#include <vector>

void fonction()
{
    std::vector<int> tableau {};

    tableau.push_back(10);
    tableau.push_back(20);

    int & a { tableau.front() };
    int & b { tableau.back() };

    std::cout << "Front : " << tableau.front() << std::endl;
    std::cout << "Back : " << tableau.back() << std::endl;

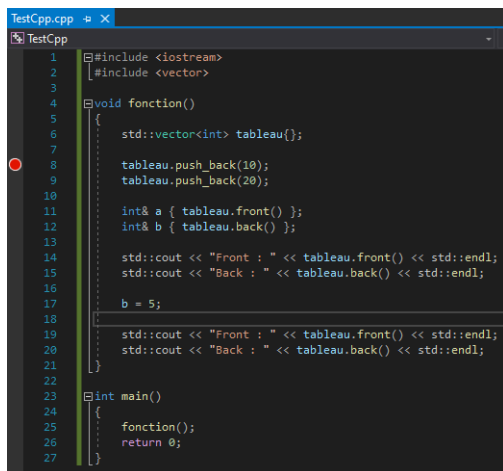
    b = 5;

    std::cout << "Front : " << tableau.front() << std::endl;
    std::cout << "Back : " << tableau.back() << std::endl;
}

int main()
{
    fonction();
    return 0;
}
```

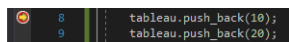
21.3. Avec Visual Studio

Le débogueur inclus avec Visual Studio se lance très simplement en faisant F5 ou en cliquant sur l'option DÉBOGUEUR WINDOWS LOCAL. Mais si vous lancez le code tel quel en *debug*, la console va s'ouvrir puis se refermer aussitôt. Vous devez d'abord poser des points d'arrêt. Pour ça, c'est très simple : il suffit de cliquer dans la colonne à gauche du numéro de la ligne où vous voulez que le programme s'arrête (voir [Figure 21.1](#)).

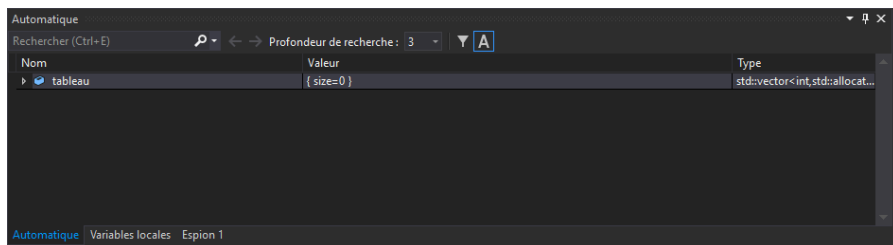
Figure 21.1 : Placement d'un point d'arrêt à la ligne 8

Interface

Une fois que le programme est lancé, l'aspect de Visual Studio change un peu. Tout d'abord, vous remarquerez la petite flèche jaune qui est apparue (ici devant la ligne 8) : elle indique l'instruction à laquelle on se trouve.

Figure 21.2 : Curseur de debug

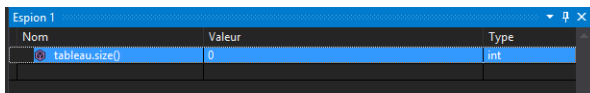
En bas à gauche, c'est la fenêtre des variables, grâce à laquelle on peut voir, à chaque étape, l'état des variables du programme.

Figure 21.3 : Fenêtre des variables

Elle est organisée en trois onglets :

- AUTOMATIQUE liste toutes les variables existantes jusqu'à la ligne où l'on se trouve. Celles situées après ne sont pas encore connues donc pas encore affichées. Pour l'instant, il n'y a que `tableau`.
- VARIABLES LOCALES liste toutes les variables de la portée actuelle, existant déjà en mémoire ou non. Dans notre exemple, on retrouve donc `tableau`, ainsi que `a` et `b`, même si celles-ci n'ont pas encore de valeur mais juste une ligne *Impossible de lire la mémoire*.
- ESPION 1 permet de garder un œil sur une variable donnée et d'être informé de toutes ses modifications. On peut, par exemple, garder une trace de la taille du tableau en créant un espion `tableau.size()`.

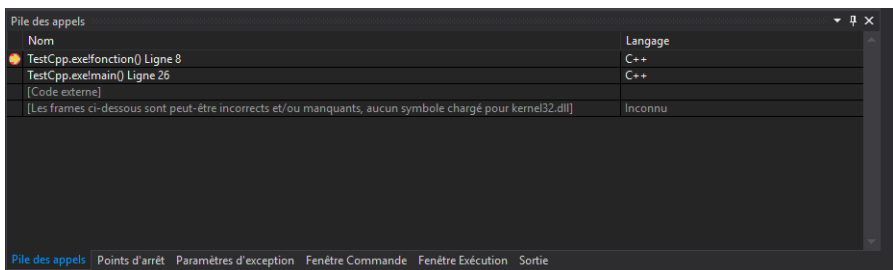
Figure 21.4 : Exemple d'espion. Quand le tableau sera agrandi ou rétréci, la valeur de l'espion sera automatiquement modifiée.



Nom	Valeur	Type
tableau.size()	0	int

En bas à droite, une autre fenêtre permet d'accéder notamment à la *pile des appels*, c'est-à-dire quelle fonction appelle quelle autre fonction. Ici, il n'y en a que deux, donc ce n'est pas étonnant de voir `main` appeler `fonction`. Mais quand le code se complexifie, il est très utile de voir par quelle fonction on passe et laquelle déclenche le bug.

Figure 21.5 : Fenêtre de la pile des appels.



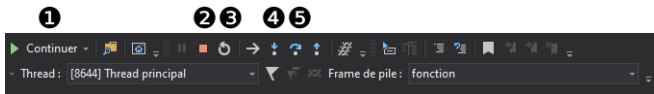
Nom	Langage
TestCpp.exe\fonction() Ligne 8	C++
TestCpp.exe\main() Ligne 26	C++
[Code externe]	
[Les frames ci-dessous sont peut-être incorrectes et/ou manquantes, aucun symbole chargé pour kernel32.dll]	Inconnu

Pile des appels | Points d'arrêt | Paramètres d'exception | Fenêtre Commande | Fenêtre Exécution | Sortie

Pas-à-pas

Que diriez-vous maintenant de passer à l'instruction suivante ? Plusieurs choix s'offrent à nous.

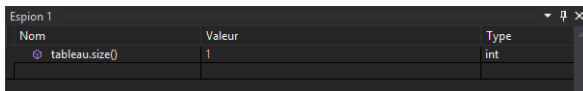
Figure 21.6 : Barre d'outils avec les différentes actions possibles



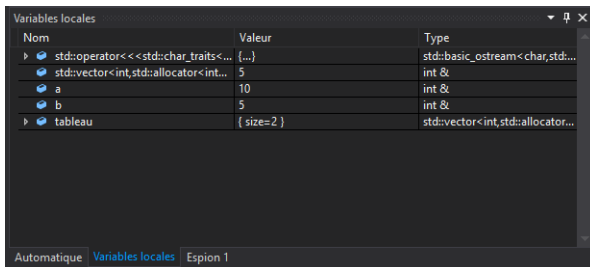
- ❶ Le bouton CONTINUER permet d'exécuter le code jusqu'au prochain point d'arrêt. Comme ici il n'y en a qu'un seul, cela signifie jusqu'à la fin du programme.
- ❷ Le carré rouge (ou Maj+F5) arrête le débogage.
- ❸ La flèche cerclée (ou Ctrl+Maj+F5) redémarre le débogage.
- ❹ Pas-à-pas principal (ou F10) : permet de passer à l'instruction suivante, mais sans explorer la pile d'appel. Cela signifie que si l'instruction à exécuter est un appel de fonction, le pas-à-pas principal n'ira pas voir à l'intérieur.
- ❺ Pas-à-pas détaillé (ou F11) : permet d'examiner en détail les instructions. Ainsi, si vous passez sur une fonction, le pas-à-pas détaillé ira dans cette fonction examiner ses instructions.

En cliquant sur le pas-à-pas principal, nous voyons la flèche jaune se déplacer d'une ligne et passer à l'instruction suivante. Dans la fenêtre des variables locales, `tableau` s'est mis à jour et sa taille est passée à 1, tout comme l'espion `tableau.size()`. On a donc bien exécuté une instruction `push_back` sur le tableau.

Figure 21.7 : Variable espionne bien mise à jour après exécution de la première instruction, qui ajoute un élément au tableau.



La Figure 21.8 montre l'état des variables quand on fait un pas-à-pas principal jusqu'à la fin de la fonction. On voit que `a` et `b` existent bien maintenant. On voit également deux autres variables, dues à la manipulation du flux de sortie, qu'on peut ignorer parce qu'elles ne nous intéressent pas.

Figure 21.8 : Variables après le pas-à-pas principal jusqu'à la fin de la fonction.

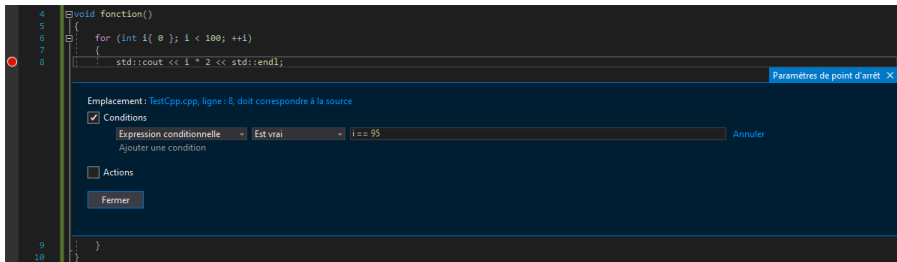
Point d'arrêt conditionnel

Faire un pas-à-pas classique, c'est bien, mais quand on doit dérouler une boucle de 100 éléments et qu'on veut la valeur à la 95^e itération, c'est embêtant. Sauf qu'on dispose d'un moyen pratique qui consiste à dire au débogueur de *s'arrêter quand une condition particulière est vérifiée*.

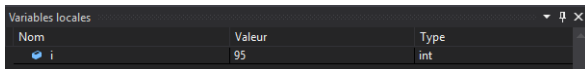
```

1. #include <iostream>
2. #include <vector>
3.
4. void fonction()
5. {
6.     for (int i { 0 }; i < 100; ++i)
7.     {
8.         std::cout << i * 2 << std::endl;
9.     }
10. }
11.
12. int main()
13. {
14.     fonction();
15.     return 0;
16. }
```

Prenons un code simple comme celui ci-dessus et posons un point d'arrêt sur la ligne 7. Cliquez droit sur le point d'arrêt, puis sélectionnez **CONDITIONS...** Vous allez voir un cadre apparaître, où vous pourrez définir une condition à respecter pour que le débogueur s'arrête sur le point d'arrêt désigné. Par exemple, sur la [Figure 21.9](#), le débogueur s'arrêtera à la ligne 8 uniquement quand **i** vaudra **95**.

Figure 21.9 : Mise en place d'une condition d'arrêt à un point donné

Lancez le débogage, vous verrez que le code s'exécute jusqu'au moment où `i` vaut 95. Là, le programme est stoppé et le débogueur attend vos instructions. Pour preuve, examinez les variables locales.

Figure 21.10 : Notre variable locale `i` vaut bien 95.

Note > Le débogueur de Visual Studio est outil très riche, et nous vous encourageons à davantage le découvrir en consultant les [tutoriels de la documentation officielle Microsoft](#). Apprendre à bien le maîtriser ne vous servira pas qu'en C++, mais également en C# ou en Python, si vous continuez à utiliser cet environnement pour développer avec ces langages.

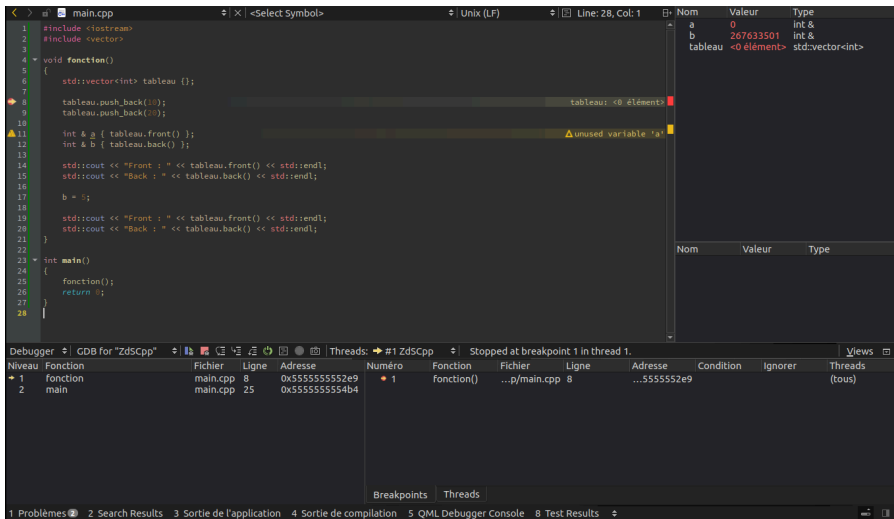
21.4. Avec Qt Creator

Le débogueur de Qt Creator se base sur l'outil libre `gdb`, qui s'utilise normalement en ligne de commande (voir [Section 21.5, En ligne de commande avec gdb](#)). Son mode de fonctionnement est assez semblable à celui de Visual Studio.

De la même façon, vous devez commencer par poser des points d'arrêt, en cliquant sur la colonne à gauche à la hauteur des lignes concernées. On lance ensuite le débogage avec `F5`.

Interface

Examinons l'interface de débogage pour le même code que ci-dessus. Une petite flèche jaune indique la ligne sur laquelle on se trouve.

Figure 21.11 : Vue de Qt Creator pendant un débogage

En haut à droite, on a d'abord la liste des variables locales et les paramètres. On retrouve bien `tableau` qui est vide ; `a` et `b` contiennent des valeurs aléatoires, car elles sont encore non-initialisées.

Figure 21.12 : Les variables locales

Nom	Valeur	Type
a	0	int &
b	267633501	int &
tableau	<0 élément>	std::vector<int>

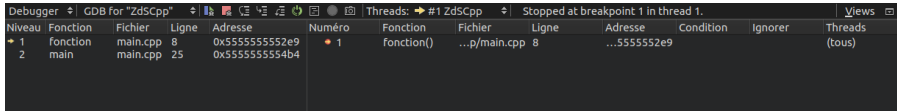
Juste en dessous se trouve la fenêtre des expressions. On peut ici écrire n'importe quelle expression C++ valide, ce qui permet de toujours garder un œil sur la valeur d'une variable en particulier, comme ici la taille du tableau.

Figure 21.13 : Une expression évaluant la taille du tableau, automatiquement mise à jour quand le tableau est modifié

Nom	Valeur	Type
tableau.size()	0	long

Enfin, plus bas, en dessous du code, on trouve la *pile des appels*, c'est-à-dire quelle fonction appelle quelle autre fonction. Également, on trouve la liste de tous les points d'arrêt, ce qui est plutôt utile quand on en a beaucoup.

Figure 21.14 : Liste des appels et des points d'arrêt.



Pas-à-pas

Maintenant, il serait de bon ton de continuer le débogage de notre application. Faisons donc du pas-à-pas.

Figure 21.15 : Les différentes options de pas-à-pas



- ❶ Le bouton CONTINUER (ou F5) permet d'exécuter le code jusqu'au point d'arrêt suivant.
- ❷ Le carré rouge arrête le débogage.
- ❸ Pas-à-pas principal (ou F10) : permet de se rendre à l'instruction suivante sans explorer la pile d'appel. Cela signifie que si l'instruction à exécuter est un appel de fonction, le pas-à-pas principal n'ira pas voir à l'intérieur.
- ❹ Pas-à-pas détaillé (ou F11) : permet de se rendre à l'instruction suivante en explorant la pile d'appel. Ainsi, si vous passez sur une fonction, le pas-à-pas détaillé examinera ses instructions.
- ❺ L'interrupteur redémarre la session de débogage.

En cliquant sur le pas-à-pas principal, on voit bien la flèche jaune se déplacer à l'instruction suivante, ajoutant ainsi une valeur au tableau, ce qui met également l'expression évaluée `tableau.size()` à jour.

Figure 21.16 : L'expression a bien été mise à jour

Nom	Valeur	Type
tableau.size()	1	long

Si on poursuit le pas-à-pas principal jusqu'à la fin de la fonction, on voit que toutes nos variables sont mises à jour et ont maintenant des valeurs cohérentes.

Figure 21.17 : Les valeurs quand le pas-à-pas atteint la fin de la fonction

Nom	Valeur	Type
a	10	int &
b	5	int &
▸ tableau	<2 éléments>	std::vector<int>

Point d'arrêt conditionnel

Avec Qt Creator, on peut tout à fait poser une condition sur un point d'arrêt, si l'on veut qu'il s'exécute à une condition précise. Dans le cas d'une boucle, on peut ainsi s'épargner de faire du pas-à-pas et aller directement à une valeur qui nous intéresse. Reprenons le code de la partie précédente et faisons le test.

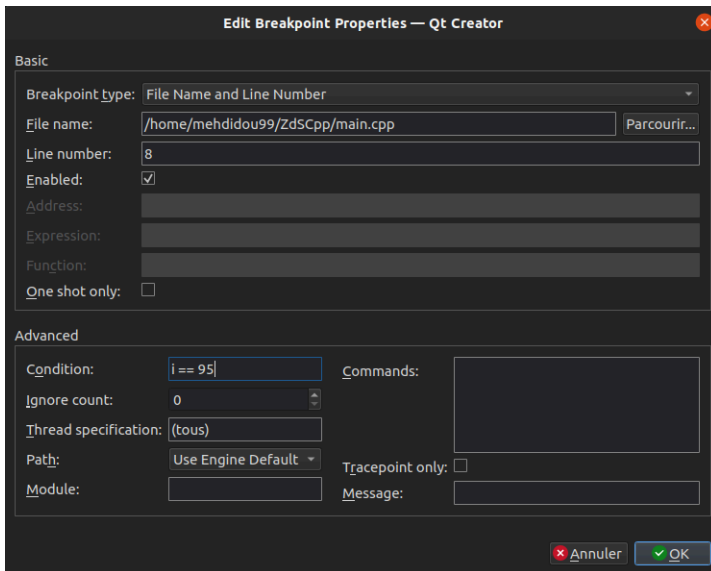
```
#include <iostream>
#include <vector>

void fonction()
{
    for (int i { 0 }; i < 100; ++i)
    {
        std::cout << i * 2 << std::endl;
    }
}

int main()
{
    fonction();
    return 0;
}
```

Pour poser une condition, il suffit de cliquer droit sur un point d'arrêt et de choisir EDIT BREAKPOINT.... Une fenêtre apparaît (voir [Figure 21.18](#)). Dans la section ADVANCED, la ligne CONDITION permet de définir la condition à respecter pour que le débogueur s'arrête sur ce point d'arrêt. On peut ainsi demander à attendre que `i` vaille `95`.

Lancez le débogage et vous verrez que le programme s'exécutera sans interruption jusqu'à ce que `i` atteigne la valeur `95`, auquel cas le débogueur attendra vos instructions pour continuer.

Figure 21.18 : Fenêtre de paramétrage d'un point d'arrêt

Note > Pour aller plus loin, vous pouvez consulter la [documentation Qt officielle \(en anglais\)](#).

21.5. En ligne de commande avec gdb

Nous avons dit, à la section précédente, que Qt Creator se basait sur gdb pour offrir une expérience de débogage. Ce dernier s'utilise avec de nombreuses interfaces graphiques différentes, mais aussi en ligne de commande. Alors rouvrons notre console et plongeons-nous dans la découverte de gdb.

Pour lancer le débogage d'un programme, il suffit de taper `gdb` suivi du nom de ce programme. Celui-ci doit avoir été compilé avec les symboles de débogage. L'option `-quiet` évite l'affichage de toutes les informations légales et de licence.

```
> gdb -quiet programme.out
Reading symbols from programme.out...done.
(gdb)
```

Poser un point d'arrêt

Poser un point d'arrêt avec gdb est faisable de trois façons différentes :

- en indiquant un endroit précis, comme un numéro de ligne, une fonction, un fichier, etc. C'est un point d'arrêt classique (*break point*) ;
- lors de la modification ou de la lecture d'une variable. C'est ce qu'on appelle un *point de surveillance* (*watch point*) ;
- lors d'un certain événement particulier, comme le lancement ou le rattrapage d'une exception, entre autres. C'est ce qu'on appelle un *point d'interception* (*catch point*).

Dans notre cas, contentons-nous de poser un point d'arrêt à une ligne précise (la 8) de notre code. Cela se fait en utilisant la commande `break` et en indiquant le numéro de la ligne voulue.

```
(gdb) break 8
Breakpoint 1 at 0xd0e: file main.cpp, line 8.
```

Maintenant, lançons l'exécution du programme avec la commande `run`.

```
(gdb) run
Starting program: /home/informaticienzero/Documents/programme.out

Breakpoint 1, fonction () at main.cpp:8
8      tableau.push_back(10);
```

Le débogueur s'est bien arrêté à la ligne 8, comme demandé. Il nous affiche en prime la fonction dans laquelle on se trouve, ainsi que le fichier, puis la ligne de code qui se trouve à la position demandée.

Dans le cas où l'on souhaite poser un point d'arrêt dans un autre fichier, il suffit de préciser celui-ci avant le numéro de la ligne, en les séparant par deux-points `:`.

```
(gdb) break autre.cpp:5
Breakpoint 2 at 0xd0e: file autre.cpp, line 5.
```

Supprimer des points d'arrêt

Quand un point d'arrêt n'est plus utile, on peut le supprimer. On a pour cela plusieurs possibilités. Soit on se souvient de la position du point d'arrêt, auquel cas on le supprime avec la commande `clear position`, soit on se souvient de son numéro et on peut le supprimer avec `delete numéro`.

Et comme se souvenir du numéro est compliqué, on peut tout à fait retrouver la liste des points d'arrêt avec la commande `info breakpoints`.

```
(gdb) info breakpoints
Num      Type           Disp Enb Address                What
1        breakpoint     keep y   0x0000000000000d0e in test() at
         autre.cpp:5
2        breakpoint     keep y   0x0000000000000d9e in fonction() at
         main.cpp:6
```

Désactiver des points d'arrêt

Parfois, on ne veut pas s'arrêter à un point d'arrêt donné, mais on ne veut pas non plus le supprimer car on en a besoin plus tard. Il suffit de le désactiver avec la commande `disable numéro`. Lorsqu'on en aura besoin, il suffira de le réactiver avec `enable numéro`.

Afficher l'état d'une variable

Si on regardait l'état de notre tableau ? Celui-ci est bien évidemment vide, mais demandons quand même à gdb de nous l'afficher. Il faut pour cela utiliser la commande `print` avec le nom de la variable voulue. Celle-ci doit exister à ce moment-là.

```
(gdb) print tableau
$1 = std::vector of length 0, capacity 0
(gdb) print a
$2 = (int &) @0x7ffffffede90: 0
(gdb) print x
No symbol "x" in current context.
```

Notre référence `a` existe, mais n'a pas encore de valeur, d'où le nombre aléatoire que vous verrez. Quant à `x`, le symbole n'existe pas car il n'y a aucune variable portant ce nom.

Pas-à-pas

Bon, tout ça c'est bien, mais comment on passe à la suite ? Comment exécuter les instructions qui suivent quand on est à un point d'arrêt ? Nous avons deux choix : faire du pas-à-pas détaillé en explorant la pile des appels avec `step` ou passer d'instruction en instruction sans explorer le code des fonctions appelées avec `next`.

La sortie suivante montre que, quand on décide de faire `step` sur le point d'arrêt, gdb commence à montrer en détail ce qui se passe et explore la fonction `std::vec-`

`tor::push_back`. Si on se contente de faire `next`, alors on passe simplement à la ligne d'après, soit le deuxième `push_back`.

```
(gdb) step
std::vector<int, std::allocator<int> >::push_back (this=0x7fffffe90,
__x=@0x7fffffe7c: 10) at /usr/include/c++/7/bits/stl_vector.h:954
954      { emplace_back(std::move(__x)); }
(gdb) next
fonction () at main.cpp:9
9      tableau.push_back(20);
```

Chacune de ces commandes peut recevoir un entier en paramètre, indiquant de combien d'étapes on souhaite avancer. Ainsi, si on est à la ligne 9 et qu'on fait `next 3`, on avancera de trois instructions. On saute les lignes 10 et 13 qui sont vides, on saute deux instructions aux lignes 11 et 12 et on arrive donc à la troisième après la ligne 9, qui se trouve ligne 14.

```
fonction () at main.cpp:9
9      tableau.push_back(20);
(gdb) next 3
14      std::cout << "Front : " << tableau.front() << std::endl;
```

Conditions

Dans le cas où l'on souhaiterait s'arrêter sur un point d'arrêt uniquement à une certaine condition, c'est possible en rajoutant celle-ci après `break`. Si on veut, dans le code suivant, s'arrêter quand `i` vaut 95, il n'y a qu'à taper la commande ci-après.

```
#include <iostream>
#include <vector>

void fonction()
{
    for (int i { 0 }; i < 100; ++i)
    {
        std::cout << i * 2 << std::endl;
    }
}

int main()
{
    fonction();
    return 0;
}
```

```

> gdb -quiet programme.out
Reading symbols from programme.out...done.
(gdb) break 8 if i == 95
Breakpoint 1 at 0x89f: file main.cpp, line 8.
(gdb) run
Starting program: /home/informaticienzero/Documents/programme.out
0
2
// Des nombres.
186
188

Breakpoint 1, fonction () at main.cpp:8
8      std::cout << i * 2 << std::endl;

```

***Note** > gdb est vraiment complet et doté de nombreuses possibilités. Nous vous encourageons à en apprendre plus de votre côté, que vous vouliez l'utiliser en ligne de commande ou à travers l'interface graphique de votre choix.*

21.6. Récapitulons !

- Le débogueur est un précieux allié qui peut nous aider à traquer les bugs sournois cachés dans nos codes.
- En posant des points d'arrêt, on demande au débogueur d'arrêter l'exécution du programme à une ligne précise.
- Certains points d'arrêt ne s'activeront que si une condition que nous avons définie devient vraie.
- On peut afficher la pile d'appels pour voir la liste des fonctions appelées avant d'arriver à une ligne quelconque.
- On peut voir à tout moment l'état des variables et même définir nos propres expressions, ou espions, qui seront automatiquement mis à jour à la moindre modification.
- Il ne faut pas oublier de compiler le programme avec ses symboles de débogage.

22

Une foule de bibliothèques

La bibliothèque standard, vous commencez à la connaître et à vous sentir à l'aise avec. Mais soyons honnêtes, elle est limitée. Par exemple, impossible de faire communiquer deux programmes par le réseau ou de créer un jeu 2D avec. Qu'à cela ne tienne, il suffit de recourir à des bibliothèques externes, déjà programmées et testées par d'autres personnes. Dans ce chapitre, nous allons voir comment installer et utiliser deux bibliothèques externes.

22.1. Quelles bibliothèques choisir ?

La réponse à cette question dépend de ce que vous cherchez. C++ étant un langage populaire, vous trouverez des bibliothèques externes dans [de nombreux domaines](#), comme les graphismes 2D, la communication réseau, la manipulation de sons, des outils mathématiques supplémentaires, etc. Dans ce chapitre, nous apprendrons à installer et utiliser brièvement deux bibliothèques externes : Boost et SFML. Vous verrez ainsi deux façons de procéder. Une fois le principe compris, installer et utiliser d'autres bibliothèques externes n'aura rien de compliqué, il suffira de lire la documentation.

Boost

Boost, c'est un peu comme l'extension officielle de la bibliothèque standard C++. Au fil des années, de nombreuses fonctionnalités implémentées dans Boost se sont retrouvées intégrées à la bibliothèque standard C++. Citons par exemple `std::array`, `std::tuple` ou encore diverses fonctions mathématiques.

L'installer, c'est s'offrir les services d'une bibliothèque de grande qualité, conçue par des développeurs experts, donc vous avez tout à gagner à apprendre à l'utiliser. Elle propose, entre autres, des tests unitaires, mais également un moyen d'interfacer son code avec Python, la possibilité de travailler avec des [quaternions](#) et bien d'autres choses encore.

SFML

SFML est une bibliothèque 2D très connue et utilisée en C++. Elle vous permet de gérer tout ce qui touche aux jeux 2D, comme le son, les graphismes et le réseau. Et ne vous inquiétez pas, *elle est portable* et fonctionne sur de nombreuses plateformes, dont Windows et GNU/Linux.

22.2. Généralités

La plupart des bibliothèques externes fonctionnent sur le même principe que la bibliothèque standard, c'est-à-dire qu'on a des fichiers d'en-tête et des fichiers objets, entre autres. Les premiers sont nécessaires pour la compilation, les seconds pour le *linker*, comme nous l'avons vu au chapitre [Compilation en cours....](#) Mais ce n'est pas tout. Souvent sont joints des exemples, ainsi qu'une version hors-ligne de la documentation.

Statique ou dynamique ?

Les fichiers objets d'une bibliothèque externe peuvent être joints au programme de deux manières différentes :

- **bibliothèque statique** – les fichiers objets sont directement intégrés au programme. Celui-ci devient plus gros, mais est utilisable tel quel, sans fichier supplémentaire à joindre avec l'exécutable ;
- **bibliothèque dynamique** – les fichiers objets sont chargés à l'exécution du programme. L'avantage, c'est qu'une même copie de la bibliothèque peut être utilisée par différents programmes. L'inconvénient, c'est qu'il faut fournir ces fichiers avec l'exécutable du programme. Sous Windows, ce sont les fameux fichiers `.dll`.

La très grande majorité des bibliothèques externes sont utilisables de façon statique ou dynamique, au choix.

Debug ou release ?

De la même manière que les programmes, les bibliothèques externes sont compilées aussi bien en *debug* qu'en *release*. Cela permet d'offrir une expérience de débogage et des tests pendant qu'on développe notre programme, avec les symboles de débogage présents et une expérience optimisée et plus rapide une fois qu'on passe en *release*.

32 ou 64 bits ?

Sans trop rentrer dans les détails, il faut savoir qu'un processeur (le cerveau de l'ordinateur) peut être conçu avec une architecture 32 bits ou une architecture 64 bits. Grossièrement, un processeur 64 bits (qui équipe presque l'intégralité des ordinateurs aujourd'hui) peut traiter plus d'informations d'un coup qu'un processeur 32 bits.

La très grande majorité des bibliothèques externes sont fournies et/ou peut être compilées tant en version 32 bits que 64 bits. Par contre, on ne peut pas mélanger et utiliser une bibliothèque 32 bits avec un programme 64 bits et vice-versa. Les instructions d'installation des différentes bibliothèques externes que vous utiliserez vous indiqueront s'il y a des subtilités à ce niveau-là.

22.3. Installer Boost

Commençons donc par Boost. Cette bibliothèque est composée majoritairement de fichiers d'en-tête utilisables tels quels, c'est-à-dire sans avoir à lier un quelconque fichier objet pour s'en servir. Cependant, quelques parties demandent à être compilées. Nous allons donc voir comment installer la version complète de Boost sur chaque plateforme. Et pour tester que tout est fonctionnel, nous compilerons le code d'illustration de [Boost.Log](#), un système d'écriture de messages de journalisation [en anglais *logging*], que nous vous remettons ci-dessous.

```
#include <boost/log/trivial.hpp>

int main()
{
    BOOST_LOG_TRIVIAL(trace) << "A trace severity message";
    BOOST_LOG_TRIVIAL(debug) << "A debug severity message";
    BOOST_LOG_TRIVIAL(info) << "An informational severity message";
    BOOST_LOG_TRIVIAL(warning) << "A warning severity message";
    BOOST_LOG_TRIVIAL(error) << "An error severity message";
    BOOST_LOG_TRIVIAL(fatal) << "A fatal severity message";

    return 0;
}
```

```
[2021-06-27 22:28:43.426821] [0x00007f3422ef9740] [trace] A trace
severity message
[2021-06-27 22:28:43.426865] [0x00007f3422ef9740] [debug] A debug
severity message
[2021-06-27 22:28:43.426871] [0x00007f3422ef9740] [info] An
informational severity message
[2021-06-27 22:28:43.426874] [0x00007f3422ef9740] [warning] A warning
severity message
```

```
[2021-06-27 22:28:43.426877] [0x00007f3422ef9740] [error]    An error
severity message
[2021-06-27 22:28:43.426880] [0x00007f3422ef9740] [fatal]    A fatal
severity message
```

GNU/Linux

Vous êtes dans le cas le plus facile, puisqu'il y a de grandes chances que votre distribution fournisse des paquets prêts à l'emploi. Reportez-vous à la documentation de celle-ci pour connaître le nom exact du paquet. Par exemple, pour Ubuntu, c'est la commande suivante qui installe l'intégralité de Boost.

```
sudo apt-get install libboost-all-dev
```

Nous allons maintenant compiler le programme tant en version statique que dynamique, pour nous assurer que tout est fonctionnel. Dans le premier cas, il faut lancer la commande suivante. On obtient alors un exécutable qui n'a besoin d'aucune dépendance pour être exécuté.

```
g++ -static main.cpp -lboost_log -lboost_thread -lboost_system -lpthread
```

Notez qu'on transmet pour le linker non seulement `-lboost_log`, mais également d'autres bibliothèques, qui sont utilisées par Boost.Log en interne et qui ont donc besoin d'être incluses pour que la compilation se déroule sans problème.

- `-lboost_thread` et `-lboost_system` correspondent à Boost.Thread et Boost.System, d'autres composants Boost.
- `-lpthread` est une dépendance à une [bibliothèque commune](#) à toutes les distributions GNU/Linux et UNIX.

Dans le cas où l'on veut lier dynamiquement Boost à notre programme, la commande change un peu.

```
g++ -DBOOST_ALL_DYN_LINK main.cpp -lboost_log -lpthread
```

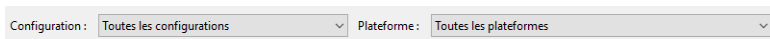
Logiquement, l'option `-static` disparaît. De plus, nous n'avons plus besoin de demander la liaison avec Boost.Thread et Boost.System. En effet, l'option `-DBOOST_ALL_DYN_LINK` que nous passons à la compilation indique que nous voulons une liaison dynamique et Boost s'en occupera lui-même en interne. Nous n'avons donc plus que `-lpthread` à ajouter.

Visual Studio 2019

Commencez par [télécharger Boost](#). Sur la page des versions, prenez la dernière disponible, puis téléchargez les fichiers `boost_{dernière version}-msvc-14.2-32.exe` et `boost_{dernière version}-msvc-14.2-64.exe`. Une fois les exécutables téléchargés, installez-les dans le dossier de votre choix. Ne vous en faites pas, il n'y a pas de risque d'écrasement si vous sélectionnez le même dossier pour la version 32 bits et la version 64 bits.

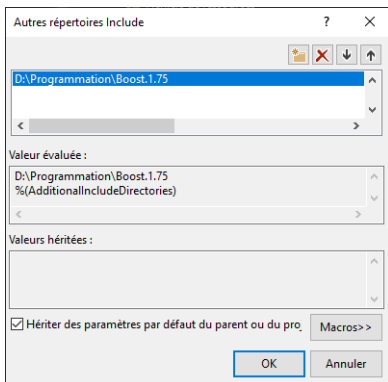
Maintenant, ouvrez un projet quelconque et rendez-vous dans ses propriétés en cliquant droit sur son nom. Il va falloir en effet indiquer au compilateur où se trouvent les fichiers de Boost. Commencez par vous mettre en configuration générale en réglant le champ CONFIGURATION sur *TOUTES LES CONFIGURATIONS* et PLATEFORME sur *TOUTES LES PLATEFORMES*.

Figure 22.1 : Configuration générale du projet



Incluez ensuite le dossier contenant tous les fichiers d'en-tête en modifiant la propriété AUTRES RÉPERTOIRES INCLUDE dans C/C++ > GÉNÉRAL et en renseignant dans le cadre du haut le chemin du dossier où vous avez installé Boost.

Figure 22.2 : Inclusion du dossier contenant les fichiers d'en-tête



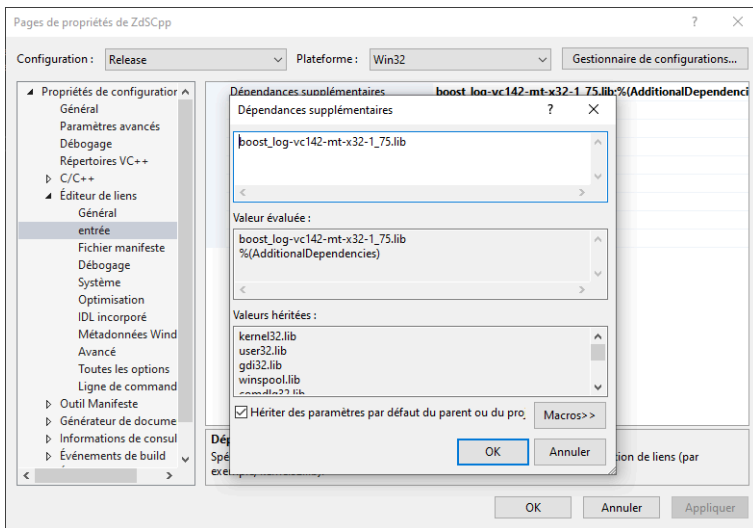
Maintenant, nous allons nous rendre dans ÉDITEUR DE LIENS > GÉNÉRAL. Comme nous avons deux versions, 32 bits et 64 bits, nous allons *adapter le chemin donné*.

Tout d'abord, modifiez le champ `PLATEFORME` en haut pour le mettre à `WIN32`. Puis, dans `ÉDITEUR DE LIENS > GÉNÉRAL`, modifiez le champ `RÉPERTOIRES DE BIBLIOTHÈQUES SUPPLÉMENTAIRES`. Ajoutez-y le chemin où vous avez installé Boost plus `lib32-msvc-14.2`. Une fois fait, cliquez sur `APPLIQUER` puis retournez modifier le champ `PLATEFORME` en le mettant cette fois à `x64`. Faites la même chose que précédemment mais en sélectionnant cette fois le dossier `lib64-msvc-14.2`.

Validez et tout est bon. Il ne nous reste plus qu'à compiler l'exemple pour vérifier. Par défaut, la liaison avec Boost est statique. Cela signifie que vous pouvez lancer l'exécutable en dehors de Visual Studio sans problème. C'est la méthode recommandée sous Windows.

Si vous souhaitez une liaison dynamique, il y a plusieurs étapes. D'abord, rendez-vous dans les propriétés, puis `C/C++ > PRÉPROCESSEUR` et modifiez le champ `DÉFINITIONS DE PRÉPROCESSEUR` pour ajouter la valeur `BOOST_ALL_DYN_LINK`. Ensuite, il faut dire à Visual Studio de quels composants nous avons besoin. Allez dans `ÉDITEUR DE LIENS > ENTRÉE` et modifiez le champ `DÉPENDANCES SUPPLÉMENTAIRES` pour ajouter le fichier `boost_log-vc142-mt-x32-1_75.lib` (à adapter selon que vous êtes en *debug* ou *release*, ainsi que 32 bits ou 64 bits).

Figure 22.3 : Ajout du fichier `.lib` pour Boost.Log en 32 bits Release.



Pour que l'exécutable puisse être exécuté, il faudra maintenant ajouter dans son dossier les DLL suivantes, sous peine d'erreur au lancement. Encore une fois, les DLL exactes à

ajouter changent si vous voulez compiler en *debug* ou en *release*, ainsi qu'en 32 bits ou 64 bits. Ci-dessous, nous vous mettons par exemple les DLL pour compiler en *release* 32 bits.

- `boost_filesystem-vc142-mt-x32-1_75.dll`
- `boost_log-vc142-mt-x32-1_75.dll`
- `boost_thread-vc142-mt-x32-1_75.dll`

Qt Creator et MinGW sous Windows

Ceux qui utilisent Qt Creator avec MinGW sous Windows gagne le droit de compiler eux-mêmes Boost. Ne vous inquiétez pas, c'est moins compliqué qu'il n'y paraît et c'est le seul moyen d'avoir Boost à jour pour MinGW. Commencez par télécharger la [dernière version disponible](#) et décompressez-la dans le dossier de votre choix (par exemple `C:/ArchiveBoost`).

Maintenant, lancez la console installée en même temps que Qt Creator. Vous pouvez normalement la retrouver dans MENU DÉMARRER > QT > (VERSION QT) > MINGW (VERSION MINGW) (64 BITS) > QT (VERSION QT) (MINGW (VERSION MINGW) 64-BIT).

Via cette console, rendez-vous dans le dossier où vous avez décompressé Boost avec la commande `cd chemin_boost`.

```
cd C:/ArchiveBoost
```

Pour compiler Boost, nous allons suivre la méthode habituelle et utiliser l'outil dédié `b2`. La première étape consiste à le configurer en se rendant dans le sous-dossier `tools/build` puis en lançant la commande `bootstrap gcc`. Ensuite, nous lançons `b2` avec la commande `b2 --prefix="chemin" install`. L'option `--prefix="chemin"` indique où sera stocké le résultat de l'appel à `b2`, dont nous aurons ensuite besoin pour lancer la compilation de Boost. Vous pouvez par exemple créer un dossier `C:/ArchiveBoost/B2Build`.

```
> cd C:/ArchiveBoost/boost_{numéro version}/tools/build
> bootstrap gcc
> b2 --prefix="C:/ArchiveBoost/B2Build" install
```

Maintenant, nous allons créer deux nouveaux dossiers, qui serviront pour le premier à stocker les fichiers objets résultants de la compilation de Boost (par exemple `C:/ArchiveBoost/Objets`) et pour le deuxième à stocker les bibliothèques statiques et dynamiques générées (par exemple `C:/Boost`).

Une fois cette étape terminée, nous ajoutons l'exécutable `b2` au `PATH`, de manière à pouvoir l'appeler depuis n'importe quel autre dossier. Cela se fait avec la commande `set PATH=%PATH%;C:/chemin/donne/a/b2/prefix/bin` (exemple `C:/ArchiveBoost/B2Build/bin`). Il ne reste plus qu'à se rendre dans le dossier où l'archive Boost a été décompressée (par exemple `C:/ArchiveBoost/boost_{numéro version}/`) et lancer la compilation à proprement parler.

```
> set PATH=%PATH%;C:/ArchiveBoost/B2Build/bin
> cd C:/ArchiveBoost/boost_{numéro version}/
> b2. --build-type=complete toolset=gcc --build-dir="C:/ArchiveBoost/
Objets" --prefix="C:/Boost" install
```

- `--build-type=complete` permet de construire toutes les variantes de la bibliothèque Boost (statique, dynamique, 32 bits, 64 bits, etc.).
- `toolset=gcc` indique qu'on veut compiler Boost avec MinGW (inclut par `gcc`).
- `--build-dir="C:/ArchiveBoost/Objets"` indique où seront stockés les fichiers objets générés.
- `--prefix="C:/Boost"` indique où seront stockés les fichiers d'en-tête et les bibliothèques issues de la compilation de Boost.

La compilation peut être longue, donc lancez-la et faites autre chose en attendant. Une fois terminée, vous pouvez supprimer le dossier créé pour la décompression de Boost, celui stockant les fichiers objets et celui dans lequel est installé `b2`.

Le seul dossier qui nous intéresse est celui dans lequel est stocké le résultat de la compilation de Boost (`C:/Boost`), où l'on retrouve un sous-dossier `include` qui contient les fichiers d'en-tête et un autre sous-dossier `lib`, qui contient les bibliothèques que nous allons ensuite lier à nos programmes lors de sa génération.

Nous allons maintenant configurer Qt Creator. Cela se passe encore une fois dans le fichier `.pro`. On commence par indiquer où trouver les fichiers d'en-tête.

```
# Adaptez en fonction du chemin d'installation.
INCLUDEPATH += C:/Boost/include/boost-{numéro de version de Boost}
```

Maintenant, nous allons déclarer de quels composants Boost nous avons besoin, en liaison dynamique tout d'abord. Notez que les lignes suivantes font la distinction entre `debug` et `release`. Remarquez aussi que nous utilisons `DEFINES` pour définir la constante de préprocesseur `BOOST_ALL_DYN_LINK` dont Boost a besoin lorsqu'on souhaite effectuer une liaison dynamique.


```

DEFINES += BOOST_ALL_DYN_LINK
# Adaptez en fonction du chemin d'installation.
win32:CONFIG(release, debug|release): LIBS += -LC:/c++/Boost/lib -
lboost_log-mgw8-mt-x64-1_76
else:win32:CONFIG(debug, debug|release): LIBS += -LC:/c++/Boost/lib -
lboost_log-mgw8-mt-d-x64-1_76

```

Maintenant, au tour de la liaison statique. Tout d'abord, il faut définir l'option `CONFIG += static`, puis ajouter explicitement toutes les bibliothèques dont Boost a besoin. Cela inclut `Boost.Thread`, mais également plusieurs bibliothèques fournies avec MinGW dont le programme aura besoin, notamment `pthread` et `stdc++`. La commande `-Wl, -Bstatic` force à ce que ces deux bibliothèques soient liées statiquement.

```

CONFIG += static

win32:CONFIG(release, debug|release): LIBS += -LC:/c++/Boost/lib -
lboost_log-mgw8-mt-s-x64-1_76 \
-lboost_thread-mgw8-mt-s-x64-1_76 \
-static-libgcc \
-static-libstdc++ \
-Wl, -Bstatic -lstdc++ -lpthread -Wl, -Bdynamic

else:win32:CONFIG(debug, debug|release): LIBS += -LC:/c++/Boost/lib -
lboost_log-mgw8-mt-sd-x64-1_76 \
-lboost_thread-mgw8-mt-sd-x64-1_76 \
-static-libgcc \
-static-libstdc++ \
-Wl, -Bstatic -lstdc++ -lpthread -Wl, -Bdynamic

```

Vous obtenez maintenant un exécutable complet, qui n'a besoin d'aucune DLL pour s'exécuter.

22.4. Installer SFML

Voyons maintenant comment installer la bibliothèque SFML. La [documentation officielle](#) est très bien faite et détaille notamment l'installation pour Visual Studio et pour Linux. N'hésitez donc pas à la lire en complément.

Voyons d'abord le code source qui nous servira de test pour vérifier si l'installation s'est bien passée. Il est issu de la documentation.

```

#include <SFML/Graphics.hpp>

int main()
{
    sf::RenderWindow window(sf::VideoMode(200, 200), "SFML works!");

```

```

sf::CircleShape shape(100.f);
shape.setFillColor(sf::Color::Green);

while (window.isOpen())
{
    sf::Event event;
    while (window.pollEvent(event))
    {
        if (event.type == sf::Event::Closed)
        {
            window.close();
        }
    }

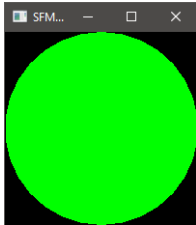
    window.clear();
    window.draw(shape);
    window.display();
}

return 0;
}

```

Si l'installation se passe bien, vous obtiendrez à la fin le résultat suivant.

Figure 22.4 : Un beau cercle vert en guise de résultat.



GNU/Linux

Vous êtes encore des chanceux, puisque la plupart des distributions proposent des paquets permettant d'avoir une SFML à jour en une commande. Ainsi, avec Ubuntu, il suffit de taper la commande qui suit (renseignez-vous pour le nom exact du paquet sur votre distribution).

```
sudo apt-get install libsFML-dev
```

Maintenant, voyons comment compiler ce programme de test. Le package officiel SFML n'inclut que les bibliothèques dynamiques, donc par défaut nous ne pouvons que compiler dynamiquement. Lancez la commande suivante et exécutez le programme obtenu.

```
g++ main.cpp -lsfml-graphics -lsfml-window -lsfml-system
```

Windows

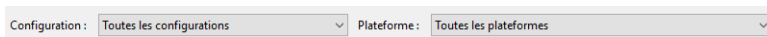
Plusieurs versions sont disponibles au téléchargement, en fonction de la version de Visual Studio que vous utilisez. Dans notre cas, il s'agira de prendre la version `Visual C++ 15` (2017) qui, malgré son nom, fonctionne parfaitement avec Visual Studio 2019. Pour Qt Creator avec MinGW, il faudra vérifier si la version de MinGW qui est proposée est *exactement la même* que celle qui vient avec Qt Creator. Si elles sont différentes, alors il faudra compiler la SFML.

Nous avons le choix entre les versions 32 bits et 64 bits. Nous allons vous montrer la façon de faire pour la première architecture, car la procédure est quasiment identique dans les deux cas. De plus, la documentation recommande pour Windows d'utiliser la version 32 bits. Nous obtenons donc une archive que nous allons décompresser dans le dossier de notre choix, par exemple `C:/SFML`.

Visual Studio

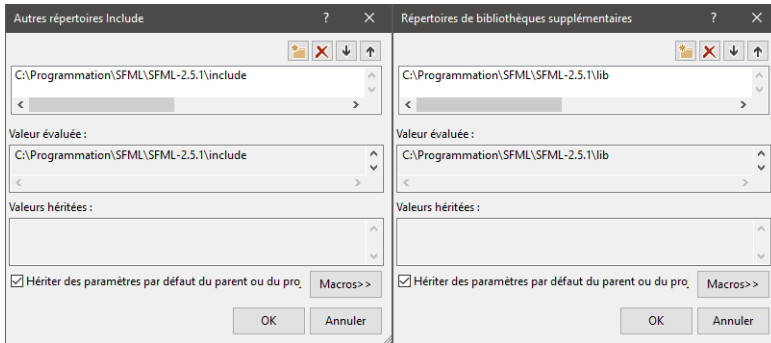
D'abord, nous allons modifier les propriétés de configuration du projet, en cliquant droit dessus. Tout en haut à gauche, assurez-vous d'être en mode **TOUTES LES CONFIGURATIONS** ainsi que **TOUTES LES PLATEFORMES**.

Figure 22.5 : Configuration générale



Il y a deux choses à faire qui sont communes à *debug* et *release*.

- Dans **C/C++ > GÉNÉRAL**, modifiez la propriété **AUTRES RÉPERTOIRES INCLUDE** et renseignez dans le cadre du haut le chemin du dossier `include` du répertoire où vous avez installé SFML.
- Dans **ÉDITEURS DE LIENS > GÉNÉRAL**, modifiez la propriété **RÉPERTOIRES DE BIBLIOTHÈQUES SUPPLÉMENTAIRES**, renseignez dans le cadre du haut le chemin du dossier `lib` du répertoire où vous avez installé SFML.

Figure 22.6 : Inclusion des dossiers *include* et *lib*

Comme SFML est organisée en plusieurs modules, il faut ajouter plusieurs bibliothèques en particulier pour que notre code fonctionne. À l'étape précédente, nous avons indiqué au linker dans quel dossier les chercher. Maintenant, nous allons lui indiquer exactement celles qu'il doit prendre. L'opération sera différente pour *debug* et *release*.

En mode *debug*, passez en configuration *DEBUG* puis allez dans *ÉDITEUR DE LIENS > ENTRÉE*. Là, modifiez la propriété *DÉPENDANCES SUPPLÉMENTAIRES*, puis ajoutez dans le cadre du haut les valeurs suivantes :

- `sfml-graphics-d.lib`
- `sfml-system-d.lib`
- `sfml-window-d.lib`

Note > Le *-d* dans le nom des fichiers indique qu'on a affaire à la version *debug* de la bibliothèque. Cette convention de nommage est répandue.

En mode *release*, procédez de même mais en prenant cette fois les versions sans *-d*.

- `sfml-graphics.lib`
- `sfml-system.lib`
- `sfml-window.lib`

Testons ensuite que notre programme compile et s'exécute bien avec la liaison dynamique. Pour cela, il faut copier-coller les DLL contenues dans le dossier *bin* du dossier d'installation de la SFML dans le dossier de génération de notre projet C++ (donc *Debug* ou *Release*). Les DLL qui contiennent *-d* sont pour le *debug*, les autres pour *release*.

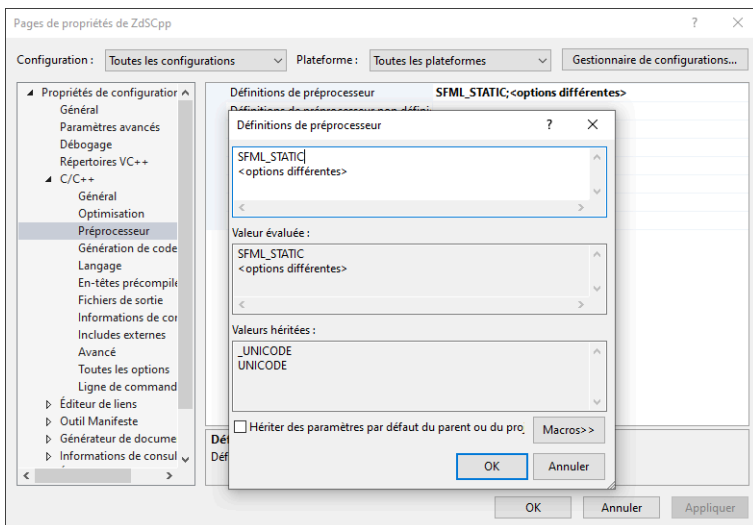
Pour tester la liaison statique, il va falloir procéder à quelques changements. Tout d'abord, il faut indiquer au linker (dans ÉDITEUR DE LIENS > ENTRÉE > DÉPENDANCES SUPPLÉMENTAIRES) qu'il doit chercher les bibliothèques statiques (qui contiennent `-s` dans leur nom). Mais ce n'est pas tout. Il faut explicitement indiquer toutes les autres dépendances qu'ont les composants de SFML. Heureusement, la documentation est complète et nous indique [quoi faire](#).

- `sfml-graphics-s-d.lib`
- `sfml-system-s-d.lib`
- `sfml-window-s-d.lib`
- `winmm.lib`
- `gdi32.lib`
- `opengl32.lib`

Note > Pour la version release, supprimez `-d`.

Dernière étape : il faut ajouter une constante de préprocesseur, qui indiquera à la SFML qu'on veut une liaison statique. Pour cela, il faut se rendre dans `C/C++ > PRÉPROCESSEUR` et modifier le champ DÉFINITIONS DE PRÉPROCESSEUR pour ajouter la valeur `SFML_STATIC`.

Figure 22.7 : Définition de préprocesseur pour la liaison statique de la SFML.



Vous pouvez maintenant générer un exécutable et le lancer.

Qt Creator

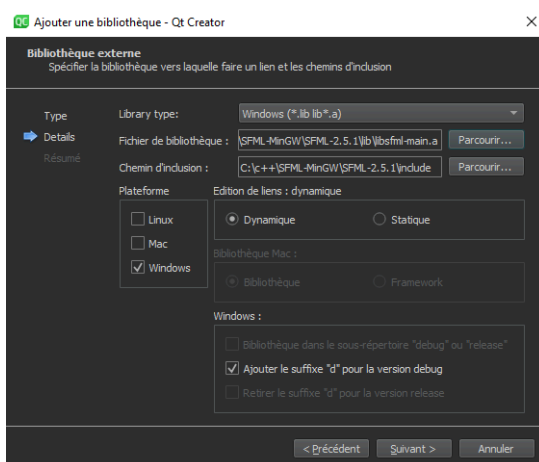
Si vous êtes dans l'obligation de compiler vous-même la SFML, sachez que ce n'est pas compliqué. Vous aurez simplement besoin d'[installer le logiciel CMake](#).

Commencez par télécharger le [code source](#) (rubrique TOUS OS > CODE SOURCE) et décompressez-le dans un dossier de votre choix. Ensuite, ouvrez la console MinGW fournie avec Qt Creator [MENU DÉMARRER > QT > (VERSION QT) > MINGW (VERSION MINGW) (64 BITS) > QT (VERSION QT) (MINGW (VERSION MINGW) 64-BIT)] et rendez-vous dans le dossier où a été décompressé SFML, puis lancez les commandes suivantes, d'abord pour générer les fichiers pour le *debug* puis pour *release*.

```
> cmake -DCMAKE_BUILD_TYPE=Debug -G "MinGW Makefiles" .
> mingw32-make
> cmake -DCMAKE_BUILD_TYPE=Release -G "MinGW Makefiles" .
> mingw32-make
```

Ouvrez votre projet, cliquez droit dessus puis sur AJOUTER UNE BIBLIOTHÈQUE.... Dans la fenêtre qui s'ouvre, cliquez sur BIBLIOTHÈQUE EXTERNE puis sur FICHIER DE BIBLIOTHÈQUE. Rendez-vous dans le dossier lib de la SFML et choisissez libsfml-main.a. Dans la liste des plateformes, décochez LINUX et MAC. Enfin, vérifiez que l'option AJOUTER UN SUFFIXE "D" POUR LA VERSION DEBUG est bien cochée, puis validez.

Figure 22.8 : Ajout de SFML sous Qt Creator



Remarquez que cette étape a modifié le fichier `.pro` en ajoutant plusieurs lignes. Deux lignes notamment indiquent maintenant où se trouvent les fichiers d'en-tête. Elles nous conviennent très bien et nous n'allons pas y toucher.

```
INCLUDEPATH += C:/c++/SFML-MinGW/SFML-2.5.1/include
DEPENDPATH += C:/c++/SFML-MinGW/SFML-2.5.1/include
```

Les deux lignes suivantes se chargent d'indiquer les composants de la SFML dont nous avons besoin, avec une distinction *debug/release*.

```
win32:CONFIG(release, debug|release): LIBS += -LC:/c++/SFML-MinGW/
SFML-2.5.1/lib/ -lsfml-main
else:win32:CONFIG(debug, debug|release): LIBS += -LC:/c++/SFML-MinGW/
SFML-2.5.1/lib/ -lsfml-maind
```

La version de *debug* ne nous convient pas d'ailleurs, car il manque un tiret - avec le *d*. Il faut donc remplacer `-lsfml-maind` par `-lsfml-main-d`. Ensuite, il faut rajouter les autres bibliothèques, à savoir `-lsfml-window`, `-lsfml-graphics` et `-lsfml-system` (avec les versions *-d* pour *debug*).

```
win32:CONFIG(release, debug|release): LIBS += -LC:/c++/SFML-2.5.1/lib -
lsfml-main -lsfml-window -lsfml-graphics -lsfml-system
else:win32:CONFIG(debug, debug|release): LIBS += -LC:/c++/SFML-2.5.1/lib
-lsfml-main-d -lsfml-window-d -lsfml-graphics-d -lsfml-system-d
```

Nous sommes maintenant en mesure de tester l'installation en compilant et en lançant notre programme, avec liaison dynamique. Il faut simplement penser à ajouter les fichiers DLL `sfml-graphics-2.dll`, `sfml-system-2.dll` et `sfml-window-2.dll` dans le dossier du projet Qt Creator (avec les versions *-d* pour *debug*).

Pour ceux qui veulent tester la liaison statique, quelques modifications doivent être apportées. Tout d'abord, il va falloir utiliser les bibliothèques statiques, qui contiennent *-s*. Si vous avez compilé SFML vous-même, utilisez les commandes suivantes.

```
> cmake -DBUILD_SHARED_LIBS=FALSE -DCMAKE_BUILD_TYPE=Debug -G "MinGW
Makefiles" .
> mingw32-make
> cmake -DBUILD_SHARED_LIBS=FALSE -DCMAKE_BUILD_TYPE=Release -G "MinGW
Makefiles" .
> mingw32-make
```

Il faut ensuite mettre à jour le fichier `.pro` comme suit. D'abord, nous indiquons que nous voulons une liaison statique, notamment à la SFML avec la constante `SFML_STATIC`. En-

suite, comme indiqué dans la [documentation](#), nous ajoutons les bibliothèques `-lopengl32`, `-lgdi32` et `-lwinmm`.

```
CONFIG += static
DEFINES += SFML_STATIC

win32:CONFIG(release, debug|release): LIBS += -LC:/c++/SFML-2.5.1/lib
-lsfml-window-s -lsfml-graphics-s -lsfml-system-s -lopengl32 -lgdi32 -
lwinmm
else:win32:CONFIG(debug, debug|release): LIBS += -LC:/c++/SFML-2.5.1/lib
-lsfml-window-s-d -lsfml-graphics-s-d -lsfml-system-s-d -lopengl32 -
lgdi32 -lwinmm
```

Vous pouvez maintenant recompiler et relancer : le programme fonctionnera sans avoir besoin de fournir la moindre DLL.

22.5. Récapitulons !

- Quand une fonctionnalité dont nous avons besoin n'existe pas dans la bibliothèque standard, nous pouvons installer des bibliothèques externes.
- Boost est une des bibliothèques externes les plus célèbres et les plus utilisées en C++. Elle comporte de nombreuses fonctionnalités comme le *logging*, des fonctions mathématiques ou la communication réseau.
- SFML est une bibliothèque réputée permettant de faire notamment des jeux 2D.
- Chaque bibliothèque externe vient avec sa documentation expliquant comment l'installer et l'utiliser. C'est votre meilleure amie et le premier réflexe à avoir quand vous avez besoin d'une information.

23

Améliorer ses projets

Vous avez, au cours de cette troisième partie, énormément appris sur ce que signifie être un développeur. Bien entendu, il y aurait encore de nombreuses choses à évoquer et cette partie ne saurait être exhaustive. Il est temps pour nous de la conclure avec un dernier chapitre présentant quelques outils qui vous seront quasi indispensables pour passer au niveau supérieur et améliorer le développement de vos projets.

23.1. git — sauvegarder et versionner son code

Sauvegarder, c'est important. Peu importe qu'on parle de code, de photos, de documents, ne pas avoir de sauvegarde peut nous créer de gros problèmes quand un document est corrompu ou effacé par erreur. Ça nous est tous arrivé au moins une fois et c'est très embêtant. Nos codes n'échappent pas à cette logique. Il est important de les sauvegarder régulièrement. Quand nos programmes sont tout petits, une solution consistant à copier/coller le dossier du projet peut suffire. Mais elle apporte en réalité plus d'inconvénients qu'autre chose :

- nous n'avons aucun moyen de voir l'historique des modifications d'un fichier ;
- c'est très lourd et encombrant puisque le nombre de dossiers augmente de plus en plus ;
- si deux dossiers ne contiennent pas les mêmes fichiers, les fusionner peut s'avérer complexe ;
- dans le cas où on travaille à plusieurs sur le projet, impossible de savoir qui a quelle version ;
- en cas de *bug*, revenir à une ancienne version est complexe, voire impossible.

Heureusement, comme il arrive souvent, des gens très doués ont créé des outils adaptés et efficaces : il ne nous reste qu'à les utiliser. Ces outils s'appellent des [gestionnaires de code source](#). Ils répondent à toutes les problématiques que nous venons de soulever. Le plus célèbre, utilisé et apprécié s'appelle [git](#).

Git est vraiment un outil fondamental pour le développeur. Nous allons ici vous en présenter les bases, mais nous vous invitons à aller plus loin : vous gagnerez beaucoup à le

maîtriser. Pour démarrer, vous pouvez lire le [petit guide de Roger Dudler](#) ; et pour une référence complète, consulter le [Pro Git Book](#), traduit en français qui plus est.

Installer git

Dans le cas où vous êtes sous Windows, téléchargez la version la plus récente et installez-la dans le répertoire de votre choix. Pour ceux utilisant GNU/Linux, le paquet `git` est très certainement déjà installé. Dans le cas contraire (ce qui serait extrêmement étonnant), vous pouvez l'obtenir avec votre gestionnaire de paquets.

Bien qu'il existe des surcouches graphiques, nous vous conseillons d'apprendre à utiliser git en ligne de commande. De cette manière, vous serez autonome quel que soit le système d'exploitation ou l'environnement sur lequel vous travaillerez, et vous en aurez une utilisation plus souple.

Sous GNU/Linux, vous n'avez qu'à lancer les commandes directement depuis le terminal. Pour Windows, il faut lancer `Git Bash`, qui se trouve dans le dossier `git` du menu DÉMARRER.

Initialiser git

La première chose à faire est d'entrer un identifiant, une adresse e-mail. Cela permet de tracer qui a fait quelle modification. Vous pouvez également préciser l'éditeur de votre choix (par défaut c'est vim).

```
> git config --global user.name "nom"
> git config --global user.email "mail"
> git config --global core.editor "éditeur"
```

Créer un dépôt

Maintenant, nous allons indiquer à git que nous voulons *sauvegarder le dossier contenant notre code source*. Il suffit de se positionner dans le répertoire voulu et de lancer la commande `git init`.

```
$ cd ~/superProjet
$ git init
```

Votre projet est maintenant géré par git. C'est aussi simple que ça.

Connaître l'état de ses fichiers

Et si nous demandions à git de nous indiquer les fichiers modifiés ? Cela se fait très simplement en tapant `git status`. Pour le cas d'un projet fraîchement initialisé, aucun fichier n'étant suivi, on obtient un message proche du message suivant.

```
$ git status
On branch master
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)

    superProjet/main.cpp

nothing added to commit but untracked files present (use "git add" to track)
```

Modifions donc un fichier quelconque. Par exemple, le contenu de `main.cpp`. On relance la commande précédente et cette fois il y a du changement.

```
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   superProjet/main.cpp

no changes added to commit (use "git add" and/or "git commit -a")
```

On voit bien que git a noté la modification du fichier `main.cpp`. Et si nous ajoutons un nouveau fichier, comme un simple document texte ?

```
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   superProjet/main.cpp

Untracked files:
  (use "git add <file>..." to include in what will be committed)

    superProjet/Fichier_texte.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Cette fois, il nous indique qu'un fichier non suivi a été ajouté. Cela signifie que git ne le versionnera pas.

Ajouter un fichier

Il faut donc être en mesure de dire à git de *suivre*, ou de *ne plus suivre*, tel ou tel fichier. Dans le premier cas, cela se fait avec la commande `git add nom_fichier`.

```
$ git add superProjet/Fichier_texte.txt
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

        new file:   superProjet/Fichier_texte.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working
  directory)

        modified:   superProjet/main.cpp
```

Le fichier ajouté se trouve maintenant sous la section *Changes to be committed*, soit les changements à enregistrer. Au contraire, le fichier `main.cpp` ne le sera pas. Il faut l'ajouter lui aussi pour que les modifications apportées à ce fichier soient elles aussi enregistrées.

```
$ git add superProjet/main.cpp
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

        new file:   superProjet/Fichier_texte.txt
        modified:   superProjet/main.cpp
```

Valider les modifications

Nous avons vérifié, tout est bon, nous voulons enregistrer ce que nous avons fait. On parle de *commiter* ou de *faire un commit*. On utilise la commande suivante `git commit -m "Message"`, qui permet d'enregistrer les modifications avec un message explicatif.

```
$ git commit -m "Réécriture de main.cpp et ajout d'un document texte."
[master d83131f] Réécriture de main.cpp et ajout d'un document texte.
 2 files changed, 7 insertions(+), 13 deletions(-)
```

```
create mode 100644 superProjet/Fichier_texte.txt
rewrite superProjet/main.cpp (93%)
```

Attention › Le message accompagnant le commit doit être de qualité, car il indique ce que vous avez fait. Des messages comme *correction*, *xxx* ou *changement* sont donc très mauvais puisqu'ils ne sont pas clairs du tout.

Voir l'historique

Vous pouvez voir l'évolution de votre code en demandant à git de vous afficher l'historique des *commits*. Dans notre cas, nous en avons trois. Le premier affiché est le dernier en date, celui que nous venons de faire. Les deux autres ont automatiquement été faits par git lors de l'initialisation du dépôt.

```
$ git log
commit d83131f1a0f860d40e06921c113affd16700c3e7 (HEAD -> master)
Author: Super codeur <super.codeur@mail.com>
Date:   Mon Mar 25 12:05:25 2019 +0100

    Réécriture de main.cpp et ajout d'un document texte.

commit 242b99d89769b2d409213db3d25c1f60c6c9088a
Author: Super codeur <super.codeur@mail.com>
Date:   Sun Mar 24 12:13:46 2019 +0100

    Ajoutez des fichiers projet.

commit 40f39687c1ad0ef616c7d6fae8549865cb5c4c54
Author: Super codeur <super.codeur@mail.com>
Date:   Sun Mar 24 12:13:42 2019 +0100

    Ajoutez les fichiers .gitignore et .gitattributes.
```

Notez que l'auteur et la date sont affichés, ainsi que le *checksum*, la somme de contrôle identifiant de manière unique chaque *commit*.

Bloquer certains fichiers

Si versionner le code est une bonne pratique, certains fichiers ne doivent pas être intégrés au dépôt. C'est le cas des fichiers objets, entre autres. Ceux-ci sont des produits de la compilation, donc rapidement périmés, pas adaptés en cas de passage à un autre système d'exploitation ou un autre compilateur, etc. On veut donc les exclure. De même, on n'inclut généralement pas les exécutables, les fichiers de configuration de l'éditeur utilisé, etc.

Pour ce faire, on utilise un fichier `.gitignore`, qui n'est ni plus ni moins un fichier texte indiquant les fichiers ou les extensions à exclure. Il suffit de le créer et d'y ajouter les

règles souhaitées. Par exemple, exclure tous les fichiers objets avec l'extension `.o` se fait à l'aide de la ligne `*.o`.

Note > Si vous tentez de créer un fichier nommé `.gitignore` avec l'explorateur Windows, celui-ci protestera que vous n'avez pas entré de nom de fichier. Une astuce consiste à créer un fichier s'appelant `.gitignore`. (notez le point final supplémentaire), que Windows va automatiquement renommer correctement.

Voici un exemple de fichier `.gitignore` :

```
# Fichiers objets.
*.slo
*.lo
*.o
*.obj

# Bibliothèques dynamiques.
*.so
*.dll

# Bibliothèques statiques.
*.a
*.lib

# Exécutables
*.exe
*.out
```

Comment ça se passe dans la pratique ? Par exemple, si on compile un fichier source pour obtenir l'exécutable, sans fichier `.gitignore`, il sera aussi inclus, or ce n'est pas souhaité.

```
$ git status
Sur la branche master

Aucun commit

Modifications qui seront validées :
(utilisez "git rm --cached <fichier>..." pour désindexer)

    nouveau fichier : main.cpp
    nouveau fichier : programme.out
```

Si on ajoute le fichier `.gitignore` écrit précédemment, cette fois l'exécutable est bien exclu et on peut commiter le code.

```
$ git status
Sur la branche master
```

```
Aucun commit
```

```
Modifications qui seront validées :  
(utilisez "git rm --cached <fichier>..." pour désindexer)
```

```
nouveau fichier : .gitignore  
nouveau fichier : main.cpp
```

Aller plus loin

Comment faire pour supprimer un fichier ? Comment revenir à un commit précédent ? Qu'est-ce que le concept de branches et comment l'utiliser ? Toutes ces questions trouvent leurs réponses dans le [Pro Git Book](#), que nous vous encourageons une fois encore à lire. Il est d'excellente qualité, progressif, et vous enseignera tout ce que vous devez savoir sur git.

23.2. GitLab — partager son code

Notez que, dans le nom de cet outil, on retrouve Git. C'est normal, car les deux outils sont fortement liés. Si Git permet de sauvegarder l'état de son code, d'en vérifier l'historique, d'annuler ou de retravailler des commits, etc., il n'en reste pas moins que votre code n'est disponible que sur votre ordinateur. GitLab est là en complément.

Note > [GitHub](#) propose un service équivalent, au fonctionnement analogue.

[GitLab](#) est un service en ligne qui permet d'héberger ses dépôts. Puisque ceux-ci sont sur Internet, librement disponibles, vous êtes en mesure de partager votre code avec le monde entier. D'autres personnes peuvent ainsi le récupérer, le modifier et envoyer des corrections de bugs, de nouvelles fonctionnalités, etc. C'est le fonctionnement et la base des projets dits *open-source*. Par exemple, c'est le cas des logiciels [Inkscape](#) ou [F-Droid](#), hébergés directement par la compagnie derrière GitLab. De même, VLC, le célèbre lecteur multimédia, utilise en interne GitLab [sur ses propres serveurs](#).

Parmi les autres avantages qu'on peut citer, utiliser un service en ligne nous protège des pannes et autres effacements sur notre poste de travail. Si demain votre disque dur meurt, le fait d'utiliser git seul pour versionner un projet ne vous sera d'aucune aide. En revanche, s'il est hébergé sur un service en ligne comme GitLab, il suffira de récupérer les sources et le développement pourra repartir.

Pour illustrer le fonctionnement de cet outil, nous allons voir comment créer un projet sur le serveur distant et le connecter à notre projet local *Discographie*, écrit dans la deuxième partie de ce livre. Vous aurez pour cela besoin de vous créer un compte, si ce n'est pas déjà le cas. Pour ce faire, rendez-vous sur la page [de création de compte](#).

Rentrez les informations demandées, en pensant bien à utiliser la même adresse e-mail que celle renseignée lors de la configuration de git.

Création d'un projet

La première étape consiste à créer un nouveau projet. Pour cela, quand vous êtes sur votre page principale, cliquez sur **NEW PROJECT**. Parmi les catégories proposées, nous partirons sur **CREATE BLANK PROJECT**.

Figure 23.1 : Page de création d'un nouveau projet

Nouveau projet > Create blank project

Nom du projet

URL du projet **Identifiant « slug » du projet**

Want to house several dependent projects under the same namespace? [Create a group](#).

Project description (optional)

Visibility Level ⓘ

☐ **Privé**
 Project access must be granted explicitly to each user. If this project is part of a group, access will be granted to members of the group.

☒ **Public**
 Votre projet est accessible sans aucune authentification.

☒ **Initialize repository with a README**
 Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.

Plusieurs choses sont à indiquer :

- le nom du projet (PROJECT NAME) ;
- l'identifiant du projet tel qu'il sera référencé par GitLab (PROJECT SLUG) ;
- une courte description, qui indique à quoi on a affaire (PROJECT DESCRIPTION) ;
- le choix de la visibilité : **PUBLIC** signifie que n'importe qui pourra voir votre projet, tandis que **PRIVATE** permet de la restreindre à vous et aux utilisateurs que vous autoriserez ;
- **INITIALIZE THIS REPOSITORY WITH A README** permet de créer en même temps que le projet un fichier spécial, qui contient bien souvent la description du projet, les aspects légaux, les informations d'installation, etc.

Quand tout est bon, vous n'avez plus qu'à cliquer sur le bouton bleu **CREATE PROJECT**.

Partons du principe que vous avez initialisé le dépôt localement, créé un `.gitignore` et que tout est commité. Nous allons maintenant indiquer à git, en local, où se trouve le répertoire distant dans lequel stocker notre projet. Cela se fait avec la commande `git remote add origin`, que vous pourrez retrouver sous la rubrique *Push an existing Git repository*, qui donne des explications sur comment se synchroniser avec un répertoire git local déjà existant.

```
/d/Projects/Discographie (master)
$ git remote add origin git@gitlab.com:d-booker_public/Discographie.git
```

Pousser nos modifications

Il faut maintenant *pousser nos modifications locales* sur le dépôt distant, afin que celles-ci soient disponibles pour le reste du monde. Cela se fait avec la commande `git push -u origin --all`.

```
git push -u origin --all
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 8 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (15/15), 41.47 KiB | 4.61 MiB/s, done.
Total 15 (delta 0), reused 0 (delta 0)
To gitlab.com:d-booker_public/Discographie.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
```

Actualisez votre page GitLab et vous verrez que le code est apparu. Félicitations, votre code est maintenant publiquement disponible et accessible à tous.

Figure 23.2 : Fichiers sur le dépôt distant

Nom	Dernier commit	Dernière mise à jour
..		
 .gitkeep	Ajoute le projet Discographie.	1 day ago
 README.md	Add new file	1 day ago
 README.md Le T.P de discographie du livre le Guide du C++ moderne.		

Aller plus loin

Nous n'avons fait que découvrir une fonctionnalité, importante certes, mais basique de GitLab. Il vous reste encore beaucoup à apprendre. Comme ce service est très populaire, vous trouverez beaucoup de tutoriels et d'aide sur Internet. Nous vous encourageons à commencer [par la documentation officielle](#).

Dans celle-ci, vous apprendrez notamment à récupérer les projets des autres (on appelle ça *forker*), à leur proposer des modifications (des *pull requests*), à ouvrir et traiter des rapports de *bugs* (des *issues*) et bien d'autres choses encore.

23.3. CMake — automatiser la compilation de ses programmes

Nous avons du code versionné, en ligne pour plus de sécurité et d'accessibilité et c'est déjà très bien. Mais si tout le monde n'a pas le même environnement ? Si on utilise Visual Studio ? Ou bien la ligne de commande ? Quelles commandes doit-on taper ? C'est assez laborieux, disons-le. Et dans le cas de projets complexes comme Boost, ça peut être difficile de savoir quelles commandes écrire.

CMake fait partie de la famille des outils permettant ce qu'on appelle en anglais du *build automation*, soit de l'automatisation de la compilation. Il est disponible notamment pour GNU/Linux et Windows, ce qui le rend intéressant, car à partir d'un fichier d'instructions écrit dans un langage propre à CMake, l'outil est capable de générer les instructions de compilation, selon le système d'exploitation et le compilateur utilisés, sans que l'on ait à s'occuper de quoi que ce soit. Il est capable de compiler d'autres langages que C++, donc apprendre à l'utiliser vous sera utile d'une manière générale.

Un exemple simple

Reprenons, dans un dossier à part, nos fichiers d'en-tête et sources qui composent le projet *Discographie*, puis ajoutons un fichier `CMakeLists.txt`, qui est le fichier expliquant comment compiler notre projet. La première chose à faire consiste à définir *quelle version minimale de CMake est autorisée*. Cela permet d'imposer l'utilisation d'une version récente, ce qui est nécessaire pour compiler en C++20 et versions supérieures.

```
cmake_minimum_required (VERSION 3.14.0)
```

Ensuite, autre étape importante, le nom du projet. Dans notre cas, il est tout trouvé. On pourrait aussi ajouter une description, un numéro de version, etc.

```
project (Discographie)
```

Maintenant, il faut lister les fichiers qui constituent notre projet, en séparant fichiers d'en-tête et fichiers sources. CMake utilise des variables pour ça. On va définir, grâce à la fonction `set`, une variable `HEADERS` regroupant tous nos fichiers d'en-tête et une variable `SOURCES` regroupant les fichiers sources.

```
set (HEADERS
    discographie.hpp
    donnees_disco.hpp
    donnees_disco_tests.hpp
    systeme_commandes.hpp
    utils.hpp
)

set (SOURCES
    discographie.cpp
    donnees_disco.cpp
    donnees_disco_tests.cpp
    systeme_commandes.cpp
    utils.cpp
    main.cpp
)
```

Terminons en indiquant comment construire une cible nommée `Discographie` à l'aide de nos variables préalablement définies. Une cible n'est rien de plus qu'un nom unique désignant l'exécutable à produire et auquel on peut appliquer des propriétés comme définir la norme C++ à utiliser, lier des bibliothèques externes, etc.

Dans notre exemple, nous voulons simplement compiler avec C++20. On se contente alors de définir l'option `CXX_STANDARD` à `20` avec la commande `set_target_properties`. Dans le cas où le compilateur utilisé ne supporte pas C++20, il essaiera de basculer sur une norme plus ancienne. Comme notre projet `Discographie` ne contient pas de code issu de C++20, cela ne nous pose pas de problème. Par contre, si l'on voulait empêcher de revenir à des normes antérieures car notre code ne serait pas compilable, il faudrait définir l'option `CXX_STANDARD_REQUIRED` à `ON`.

```
add_executable (Discographie ${SOURCES} ${HEADERS})

set_target_properties(Discographie PROPERTIES
    CXX_STANDARD 20
)
```

Au final, voilà le `CMakeLists.txt` qu'on obtient :

```
# On n'autorise l'utilisation qu'à
# partir de la version 3.14.0.
cmake_minimum_required (VERSION 3.14.0)

# Il s'agit du projet Discographie.
project (Discographie)

# Les fichiers d'en-tête.
set (HEADERS
    discographie.hpp
    donnees_disco.hpp
    donnees_disco_tests.hpp
    systeme_commandes.hpp
    utils.hpp
)

# Les fichiers sources.
set (SOURCES
    discographie.cpp
    donnees_disco.cpp
    donnees_disco_tests.cpp
    systeme_commandes.cpp
    utils.cpp
    main.cpp
)

# On veut un exécutable Discographie compilé à partir
# des fichiers décrits par SOURCES et HEADERS.
# Ce nom doit être unique, car CMake s'en sert ensuite
# pour définir des cibles, pour lesquelles on pourra
# préciser la norme C++ à utiliser, les éventuelles
# bibliothèques externes, etc.
add_executable (Discographie ${SOURCES} ${HEADERS})

# On veut compiler avec C++20. Si ce n'est pas possible,
# le compilateur rebasculera sur une norme plus
# ancienne et réessayera. Dans les cas où l'on veut
# forcer l'utilisation de la norme C++20, on peut
# définir en plus la propriété CXX_STANDARD_REQUIRED
# à ON.
set_target_properties(Discographie PROPERTIES
    CXX_STANDARD 20
)
```

Il ne reste plus qu'à lancer `cmake` en lui précisant le générateur, c'est-à-dire la façon dont il doit générer le projet. Par exemple, pour ceux qui utilisent Visual Studio, CMake s'occupera de créer un fichier `.sln`, une solution Visual Studio qu'il n'y a qu'à ouvrir et compiler.

```
cmake . -G "Visual Studio 16 2019"
```

Sous GNU/Linux, on obtient un fichier Makefile, qu'il n'y a plus qu'à exécuter en lançant la commande **make**, qui va construire l'exécutable.

```
$ cmake . -G "Unix Makefiles"
-- The C compiler identification is GNU 9.3.0
-- The CXX compiler identification is GNU 9.3.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/Discographie

$ make
[ 14%] Building CXX object CMakeFiles/Discographie.dir/
discographie.cpp.o
[ 28%] Building CXX object CMakeFiles/Discographie.dir/
donnees_disco.cpp.o
[ 42%] Building CXX object CMakeFiles/Discographie.dir/
donnees_disco_tests.cpp.o
[ 57%] Building CXX object CMakeFiles/Discographie.dir/
systeme_commandes.cpp.o
[ 71%] Building CXX object CMakeFiles/Discographie.dir/Utils.cpp.o
[ 85%] Building CXX object CMakeFiles/Discographie.dir/main.cpp.o
[100%] Linking CXX executable Discographie
[100%] Built target Discographie
```

Dans le cas de Qt Creator, nous allons nous baser sur le compilateur fourni. Pour que CMake puisse l'utiliser, il faut lancer la console incluse avec Qt, qui devrait se trouver normalement dans MENU DÉMARRER > QT > VERSION DE QT INSTALLÉE > MINGW (VERSION DE MINGW) (32 BIT) > QT (VERSION DE QT INSTALLÉE) (MINGW (VERSION DE MINGW) 32-BIT). Le chemin est à adapter selon votre version de Qt et de MinGW, bien évidemment.

Ensuite on précise à CMake que l'on veut utiliser **"MinGW Makefiles"**, ce qui va produire un fichier Makefile, comme sous l'environnement GNU/Linux, mais adapté au compilateur MinGW.

En sortie, nous obtenons le résultat suivant. Le code est généré en lançant `mingw32-make`.

```
$ cmake . -G "MinGW Makefiles"
-- The C compiler identification is GNU 9.2.0
-- The CXX compiler identification is GNU 9.2.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: C:/Programming/MinGW/bin/gcc.exe
-- Check for working C compiler: C:/Programming/MinGW/bin/gcc.exe --
works
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: C:/Programming/MinGW/bin/g++.exe
-- Check for working CXX compiler: C:/Programming/MinGW/bin/g++.exe --
works
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: C:/SuperProjet

C:\SuperProjet>mingw32-make
Scanning dependencies of target Discographie
[ 14%] Building CXX object CMakeFiles/Discographie.dir/
discographie.cpp.obj
[ 28%] Building CXX object CMakeFiles/Discographie.dir/
donnees_disco.cpp.obj
[ 42%] Building CXX object CMakeFiles/Discographie.dir/
donnees_disco_tests.cpp.obj
[ 57%] Building CXX object CMakeFiles/Discographie.dir/
systeme_commandes.cpp.obj
[ 71%] Building CXX object CMakeFiles/Discographie.dir/Utils.cpp.obj
[ 85%] Building CXX object CMakeFiles/Discographie.dir/main.cpp.obj
[100%] Linking CXX executable Discographie.exe
[100%] Built target Discographie
```

Lier des bibliothèques externes

Très bien, nous sommes capables de compiler notre projet quelle que soit la plateforme. C'est déjà une très bonne base. Mais allons plus loin en intégrant une bibliothèque externe, par exemple Boost. Admettons pour l'exemple qu'on veuille un simple programme qui utilise Boost.Log.

```
#include <iostream>
#include <boost/log/trivial.hpp>

int main()
{
    BOOST_LOG_TRIVIAL(info) << "Lancement du programme.";
}
```

```
std::cout << "Bonjour !\n";

BOOST_LOG_TRIVIAL(info) << "Fin du programme.";
return 0;
}
```

CMake vient avec [pleins de modules fournis](#). Chaque module définit un certain nombre de variables qui indiquent, par exemple, si une bibliothèque externe a été trouvée, son chemin d'installation, etc. On en trouve notamment un pour Boost, nommé [FindBoost](#). On pourra ainsi vérifier si Boost a été trouvé sur le système de l'utilisateur et si oui où sont stockés les fichiers d'en-tête, les fichiers objets, etc. On pourra également définir dans quels dossiers chercher les fichiers d'en-tête et les fichiers objets.

CMake fournit aussi un certain nombre de constantes permettant de savoir sur quel système d'exploitation il est exécuté, si on est sur du 32 bits ou du 64 bits, etc. En combinant ces constantes et la fonctionnalité de modules, nous sommes en mesure d'adapter nos instructions pour GNU/Linux, Windows, etc.

Liaison statique

Commençons par définir une variable bien utile pour la suite, qui indique qu'on veut lier statiquement Boost à notre programme, ce qui nous simplifiera la vie sous Windows. Cela se fait avec le mot clé `set`.

```
set (Boost_USE_STATIC_LIBS ON)
```

Visual Studio

Quand on veut viser le compilateur fourni avec Visual Studio, il faut utiliser la constante `MSVC`. Pour définir des conditions, rien de compliqué, un simple bloc conditionnel `if (CONDITION)` suffit, avec un `endif ( )` à la fin, comme les accolades en C++.

À l'intérieur de ce bloc, nous allons préciser où CMake doit chercher Boost à travers le module `FindBoost`. Sans cette information, ce dernier sera complètement perdu et la génération ne pourra pas se faire. Il nous suffit pour cela de définir la variable `BOOST_ROOT` grâce à la commande `set`.

```
if (MSVC)
    set (BOOST_ROOT "C:/Chemin/Installation/Boost/VisualStudio")
endif ( )
```

MinGW

Pour ceux qui utilisent Qt Creator et MinGW sous Windows, la procédure est la même : nous allons indiquer où trouver Boost. Seule la constante `MINGW` est différente.

```
if (MINGW)
    set (BOOST_ROOT "C:/Chemin/Installation/Boost/MingW")
endif ()
```

GNU/Linux

Pour GNU/Linux et les autres UNIX (FreeBSD, macOS, etc.), la procédure est plus simple puisqu'il n'y a rien à faire. Avec ces systèmes, chaque fois qu'on installe une bibliothèque externe, celle-ci est rangée dans un dossier bien précis et commun (souvent dans `/usr/`). Le module FindBoost ira chercher par défaut dans ce dossier-là, donc pas besoin de lui donner plus d'instructions.

A-t-on trouvé Boost ?

Maintenant que nous avons configuré où chercher Boost, il est temps de demander à CMake de le charger. Pour cela nous utilisons la commande `find_package`. Cette dernière attend qu'on lui donne une liste de composants à charger au sein de Boost. Pour notre exemple qui n'utilise que les logs, nous avons besoin de `log` et `log_setup`.

```
find_package (Boost REQUIRED COMPONENTS log log_setup)
```

Quand la commande `find_package` est invoquée pour trouver une bibliothèque, elle définit une variable `XXX_FOUND`, un booléen indiquant si oui ou non la bibliothèque XXX est présente. Dans notre cas, c'est `Boost_FOUND`. Nous pouvons ainsi gérer le cas où Boost n'a pas été détecté et ainsi afficher une erreur pour l'utilisateur.

```
if (Boost_FOUND)
    # Nous sommes dans le cas où Boost a été trouvé.
else ()
    message (FATAL_ERROR "Erreur : Boost est introuvable !")
endif ()
```

Nous allons maintenant nous concentrer sur le cas où Boost a été trouvé. Toutes les commandes que nous allons vous montrer maintenant seront donc à écrire à l'intérieur du `if (Boost_FOUND)`.

La construction de l'exécutable

Notre projet n'est constitué que d'un seul fichier source très petit, la construction de l'exécutable avec la commande `add_executable` va donc être très simple. Nous n'allons même pas prendre la peine de définir une variable regroupant nos fichiers sources !

```
add_executable (TestBoostLog main.cpp)
```

Maintenant, nous voulons configurer la compilation pour qu'elle se base sur la norme C++20, comme lors de l'[exemple simple](#). Nous allons donc reprendre la même commande que plus haut.

```
set_target_properties(TestBoostLog PROPERTIES
CXX_STANDARD 20
)
```

Ensuite, il faut lier Boost à notre cible. Si nous ne faisons appel qu'aux composants implémentés sous forme de fichiers d'en-tête, nous n'aurions qu'à indiquer où se trouvent ces derniers avec la commande `target_include_directories` et la constante `Boost_INCLUDE_DIRS`.

```
target_include_directories(TestBoostLog PUBLIC ${Boost_INCLUDE_DIRS})
```

Ici, nous avons besoin d'indiquer qu'il faudra `linker log` et `log_setup` au moment de la génération de l'exécutable. Nous allons donc utiliser la commande `target_link_libraries`. Notez comment, ci-dessous, chaque élément est préfixé d'un `Boost::` afin de faire une distinction entre plusieurs fichiers objets qui viendraient de modules différents mais qui porteraient le même nom. Ce n'est pas sans nous rappeler la bibliothèque standard C++ et le préfixe `std::`.

```
target_link_libraries (TestBoostLog Boost::log Boost::log_setup)
```

Fichier final

Et voici le fichier au final, commenté et complet.

```
# On n'autorise l'utilisation qu'à partir de la version 3.14.0.
cmake_minimum_required(VERSION 3.14)

# Il s'agit du projet TestBoostLog.
project (TestBoostLog)
```

```

# On veut lier Boost de façon statique.
set (Boost_USE_STATIC_LIBS ON)

# Configuration MSVC.
if (MSVC)
    set (BOOST_ROOT "C:/c++/BoostMscv")
endif()

# Configuration MinGW.
if (MINGW)
    set (BOOST_ROOT "C:/c++/MinGW")
endif ()

find_package (Boost REQUIRED COMPONENTS log log_setup)

# Teste si Boost a bien été trouvé.
if (Boost_FOUND)

    # On veut un exécutable TestBoostLog compilé
    # à partir du seul fichier main.cpp.
    # Le nom de l'exécutable doit être unique dans
    # le fichier CMakeLists.txt et désigne une cible,
    # pour laquelle nous pourrons ensuite appliquer
    # des propriétés, comme par exemple la norme C++
    # à utiliser, les bibliothèques externes à lier, etc.
    add_executable (TestBoostLog main.cpp)

    # On demande à inclure les fichiers d'en-tête Boost
    # pour notre cible TestBoostLog.
    target_include_directories(TestBoostLog PUBLIC
        ${Boost_INCLUDE_DIRS}
    )

    # On veut compiler avec C++20.
    set_target_properties(TestBoostLog PROPERTIES CXX_STANDARD 20)

    # On indique au linker qu'il faudra inclure les
    # fichiers objets correspondants aux composants
    # Boost::log et Boost::log_setup.
    target_link_libraries (TestBoostLog Boost::log Boost::log_setup)

else ()
    message (FATAL_ERROR "Erreur : Boost est introuvable !")
endif ()

```

Exécuter CMake

Maintenant que le fichier `CMakeLists.txt` est prêt, il ne nous reste plus qu'à lancer la génération de notre programme. Avec Visual Studio, on obtient une solution `.sln` prête à être ouverte et compilée.

```

> cmake . -G "Visual Studio 16 2019"
-- Building for: Visual Studio 16 2019

```

```
-- Selecting Windows SDK version 10.0.19041.0 to target Windows
10.0.19043.
-- The C compiler identification is MSVC 19.29.30037.0
-- The CXX compiler identification is MSVC 19.29.30037.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: C:/Program Files (x86)/Microsoft Visual
Studio/2019/Community/VC/Tools/MSVC/14.29.30037/bin/Hostx64/x64/cl.exe
- skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: C:/Program Files (x86)/Microsoft
Visual Studio/2019/Community/VC/Tools/MSVC/14.29.30037/bin/Hostx64/x64/
cl.exe - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Looking for pthread.h
-- Looking for pthread.h - not found
-- Found Threads: TRUE
-- Found Boost: C:/c++/boost_1_76_0_142 (found version "1.76.0") found
components: log log_setup date_time filesystem thread regex chrono
atomic
-- Boost Fichiers en-têtes - C:/c++/boost_1_76_0_142
-- Boost Fichiers objets - C:/c++/boost_1_76_0_142/lib64-msvc-14.2
-- Configuring done
-- Generating done
-- Build files have been written to: C:/c++/testVS
```

Sous GNU/Linux, on obtient un fichier **Makefile**. La compilation de notre programme de test se lance avec la commande **make**. On voit au final que le programme est fonctionnel.

```
$ cmake . -G "Unix Makefiles"
-- The C compiler identification is GNU 9.3.0
-- The CXX compiler identification is GNU 9.3.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Found Boost: /usr/lib/x86_64-linux-gnu/cmake/Boost-1.71.0/
BoostConfig.cmake (found version "1.71.0") found components: log
log_setup
-- Configuring done
-- Generating done
-- Build files have been written to: /home/user/TestCpp
```

```
$ make
Scanning dependencies of target TestBoostLog
[ 50%] Building CXX object CMakeFiles/TestBoostLog.dir/main.cpp.o
[100%] Linking CXX executable TestBoostLog
[100%] Built target TestBoostLog

$ ./TestBoostLog
[2021-06-27 18:36:07.938973] [0x00007fe99ae83740] [info]    Lancement du
programme.
Bonjour !
[2021-06-27 18:36:07.939026] [0x00007fe99ae83740] [info]    Fin du
programme.
```

Avec MinGW, pareil, nous relançons une console Qt et exécutons la commande `cmake` avec les bons paramètres.

```
>cmake . -G "MinGW Makefiles"
-- The C compiler identification is GNU 9.2.0
-- The CXX compiler identification is GNU 9.2.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: C:/c++/MinGW/bin/gcc.exe - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: C:/c++/MinGW/bin/g++.exe - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Looking for pthread.h
-- Looking for pthread.h - found
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success
-- Found Threads: TRUE
-- Found Boost: C:/c++/MinGW/include (found version "1.71.0") found
components: log log_setup date_time filesystem thread regex chrono
atomic
-- Configuring done
-- Generating done
-- Build files have been written to: C:/c++/MinGW/e

>mingw32-make
[ 50%] Building CXX object CMakeFiles/TestBoostLog.dir/main.cpp.obj
[100%] Linking CXX executable TestBoostLog.exe
[100%] Built target TestBoostLog

>TestBoostLog.exe
[2021-06-27 18:40:05.543387] [0x000051d4] [info]    Lancement du
programme.
Bonjour !
[2021-06-27 18:40:05.549507] [0x000051d4] [info]    Fin du programme.
```

Définir des variables dynamiquement

Que se passe-t-il si le chemin d'installation de Boost chez un utilisateur n'est pas le même que chez vous ? Mieux vaudrait qu'il ne modifie pas lui-même le fichier `CMakeLists.txt`, il pourrait casser quelque chose. L'idéal serait de lui laisser la main au moment de lancer CMake juste sur la variable `BOOST_ROOT`.

Nous allons donc supprimer de notre `CMakeLists.txt` les deux blocs conditionnels pour `MSVC` et `MINGW`. L'utilisateur devra alors définir dynamiquement la variable `BOOST_ROOT` lors de l'exécution de `cmake` avec la commande suivante.

```
cmake . -G "générateur" -DBOOST_ROOT="chemin/vers/Boost"
```

Nous pouvons bien entendu définir dynamiquement la valeur de beaucoup d'autres variables grâce à l'option `-D{NOM}`, où `{NOM}` est à remplacer par la variable souhaitée. On pourrait par exemple permettre à l'utilisateur de choisir s'il veut lier Boost de manière statique (`-DBOOST_USE_STATIC_LIBS="ON"`) ou dynamique (`-DBOOST_USE_STATIC_LIBS="OFF"`), s'il veut du 32 bits ou du 64 bits, etc.

L'option `-DNOM` permet de définir une variable `NOM` avec la valeur de notre choix. On peut ainsi préciser explicitement `"-x64"` quand on le veut, sans être forcé comme avant.

Aller plus loin

CMake est un outil puissant, qui permet de gagner en souplesse, mais le maîtriser demande bien plus que de savoir les quelques commandes que nous avons vues ici. De nombreuses ressources sont disponibles, à commencer par [le cours officiel](#) sur le site de CMake. Vous pouvez également consulter le [tutoriel de Alexandre Laurent, qui est en français](#).

Par exemple, il est possible de lancer automatiquement Doxygen en même temps que la compilation, afin de disposer également de la documentation du projet. Vous pouvez aussi apprendre des techniques pour ne pas avoir à mettre à jour à la main la liste des fichiers d'en-tête et sources.

23.4. Encore quelques outils

Si nous devons vous présenter tous les outils utiles d'un développeur C++, ce livre serait au moins deux fois plus gros et ce chapitre bien indigeste. Nous en avons déjà vu trois, importants et très largement répandus. Mais il en existe encore de nombreux autres, que vous serez amené à utiliser (ou pas) au fur et à mesure que vous progresserez.

GitLab CI — de l'intégration continue

Imaginez qu'à chaque commit sur GitLab, votre code soit automatiquement compilé, les tests unitaires lancés et la documentation automatiquement générée et mise à jour ? C'est ce que permet de faire [GitLab CI/CD](#) et c'est ce qui s'appelle *l'intégration continue*. En plus, l'outil est gratuit jusqu'à 400 minutes de CI/CD par mois.

CppCheck — vérification du code

Un autre outil qui rend de fiers services : [CppCheck](#). Cet outil effectue de nombreuses vérifications sur votre code pour vous aider à le rendre plus robuste. On peut citer notamment la vérification des limites de tableaux, les divisions par zéro, ou encore la vérification des exceptions lancées.

```
$ cppcheck ./
Checking discographie.cpp ...
1/6 files checked 31% done
Checking donnees_disco.cpp ...
2/6 files checked 43% done
Checking donnees_disco_tests.cpp ...
3/6 files checked 62% done
Checking main.cpp ...
4/6 files checked 77% done
Checking systeme_commandes.cpp ...
5/6 files checked 95% done
Checking utils.cpp ...
6/6 files checked 100% done
```

Beaucoup d'informations, mais aucun problème trouvé sur le projet Discographie. Ouf !

StackOverflow — la réponse à quasi toutes les questions

S'il y a un site que tous les développeurs connaissent et lisent, c'est bien [StackOverflow](#). C'est LE site de questions-réponses sur Internet. Si vous n'avez aucune réponse à votre problème sur StackOverflow, alors vous n'avez vraiment pas de chance du tout. Peu importe le langage, peu importe la technologie, tout est abordé sur ce site. Mettez-le dès maintenant en favori, il va vous sauver la vie !

23.5. Récapitulons !

- Le gestionnaire de code source `git` permet de sauvegarder et versionner son code source, et ainsi de tracer un historique du projet, de revenir à des versions antérieures et de développer plusieurs fonctionnalités en parallèle.
- Le service en ligne `GitLab` permet de partager son code source avec les autres, et ainsi une sauvegarde pérenne, la collecte de bugs, de proposition de correctifs ou d'améliorations, entre autres choses.
- `CMake` automatise la génération du code, permettant, avec un unique fichier de configuration, de générer des projets `Visual Studio`, des `Makefiles` et bien d'autres choses encore.
- Chacun de ces outils offre de nombreuses possibilités qui sortent du cadre de ce livre mais que vous gagnerez à approfondir par vous-même.
- De nombreux outils pour C++ sont disponibles, qu'un développeur découvre et utilise au fil du temps.

La programmation orientée objet

Au chapitre [Un coup d'œil dans le rétro](#), nous avons vu que le C++ est un langage très riche qui intègre plusieurs manières de programmer, plusieurs paradigmes. Nous avons notamment vu les paradigmes impératif et générique.

Maintenant, nous allons en découvrir un troisième : la *programmation orientée objet*, ou POO pour les intimes. C'est un paradigme puissant qui, bien utilisé et combiné avec ceux que l'on a déjà étudiés, peut permettre une meilleure conception en termes d'élégance, d'expressivité et de robustesse.

24

Premiers pas avec la POO

La programmation orientée objet (abrégée en POO), c'est un terme qu'on rencontre souvent en programmation. Sa définition exacte et son utilisation varient en fonction des points de vue, des langages qui l'utilisent, des contraintes. Une chose est sûre : il s'agit d'un paradigme très répandu et très utilisé.

Ce chapitre a pour but de vous introduire à la programmation orientée objet en définissant les principes qui la régissent et en expliquant ce qu'elle peut nous apporter.

La façon dont nous allons vous présenter la POO va peut-être vous surprendre, comparée à d'autres ouvrages. Cela est dû à une volonté de bien revenir sur ses fondements, de manière à l'exploiter avantageusement.

Comme toute solution, la programmation orientée objet a été conçue pour répondre à certains besoins. Il y aura donc des cas où elle sera plus adaptée et dans d'autres plus limitative. Il convient donc de bien en cerner l'esprit et les principes de base afin de l'utiliser à bon escient. C'est ce que nous essayerons de vous transmettre au cours de cette partie.

24.1. Le principe : des objets bien serviables

Penser en termes de données...

Depuis que nous avons commencé ce livre, nous avons pensé et codé en termes de *structures de données* et d'*algorithmes*. Nous avons conçu notre code comme une suite d'instructions et d'opérations, avec tel algorithme appliqué à telle structure ou tel type, afin d'obtenir telle information en sortie.

Prenons un exemple tiré de la vie réelle. Une laverie met à la disposition des clients des machines à laver, mais à leur charge de s'occuper du reste. C'est à chacun de préparer son linge, d'amener sa lessive, de trier par couleur, de vérifier le poids inséré dans la machine, de lancer celle-ci avec le programme adapté au type de vêtements, de s'occuper

du séchage, etc. On est typiquement dans le cas d'une solution assez complexe, avec plusieurs opérations à effectuer les unes à la suite des autres, des traitements particuliers (tri du linge, tri par couleur, etc.). Tout tourne autour du linge, la "donnée centrale" de notre programme de laverie. Si on devait coder ça, on aurait sans doute une structure pour le linge et plusieurs fonctions comme `tri_linge_par_couleur`, `ajouter_lessive`, etc.

...ou de services ?

La notion de service est un des fondements de la POO. Plutôt que de fournir une liste de fonctions, d'opérations, d'algorithmes à appliquer, on va abstraire cette complexité technique, la cacher. On va enfermer, camoufler tout ce qui est complexe et n'offrir qu'une interface beaucoup plus simple à utiliser.

Appliquer ce principe à notre laverie revient à créer un pressing. Toutes les opérations de tri, de dosage et autres sont alors cachées, elles sont déléguées. L'utilisation est beaucoup plus simple. Il suffit de venir, déposer le linge sale et repartir avec du linge propre. Nous n'avons rien d'autre à faire qu'attendre (et, accessoirement, payer). Le pressing s'occupe de nous rendre un service "Laver notre linge" et nous nous contentons de l'utiliser, sans chercher plus loin. On n'a pas besoin de maîtriser les arcanes du nettoyage ; des spécialistes en qui on peut avoir confiance vont en assumer la responsabilité. On passe par une abstraction et ce principe est une des pierres angulaires de la programmation orientée objet.

Pour résumer, la programmation orientée objet, c'est créer des services.

Exemple concret

Vous voulez un exemple concret, avec du code ? Ça tombe bien, la bibliothèque standard en est remplie ! Prenez le cas des chaînes de caractères. Plusieurs fois dans le livre, nous avons mentionné qu'il existe en C++ un type de chaînes de caractères hérité du C. Dans ce langage, manipuler les chaînes est plus compliqué. Il faut passer par des fonctions pour les concaténer ou supprimer un caractère, on doit gérer explicitement la mémoire, etc. Bref, pas très pratique.

Le type `std::string` permet quant à lui de manipuler les chaînes de caractères de façon beaucoup plus aisée. C'est la raison pour laquelle on l'utilise tout le temps. Sauf qu'en interne, `std::string` manipule des chaînes de caractères version C. Simplement voilà, la cuisine qu'il fait pour concaténer deux chaînes ou en vider une troisième ne nous intéresse pas, et d'ailleurs elle n'est pas exposée. C'est donc un très bon exemple de service.

24.2. Un peu de vocabulaire

L'objet

Un objet n'est ni plus ni moins qu'une donnée qu'on manipule. Ainsi, dans le code `std::string chaine`, `chaine` est tout simplement un objet de type `std::string`. De même pour `std::vector<int> tableau`, où notre variable `tableau` est un objet de type `std::vector<int>`.

Le type

Afin de définir quels services sont rendus par un objet, il faut définir son modèle, son type. On parle de la *classe d'un objet*. À partir de cette classe, on pourra créer autant d'objets de ce type qu'on veut. On parle d'*instancier la classe*. Quand on écrit `std::string chaine`, on crée une instance du type `std::string`, instance qui se nomme `chaine` et qui est notre objet.

Note > Des variables de types basiques, comme `int`, `double` ou `bool`, peuvent être considérés comme des objets d'un point de vue théorique. En pratique, en C++, on fera la distinction entre ces types dits natifs et les instances de classes, comme `std::string`.

L'interface d'une classe

Un objet peut nous rendre plus ou moins de services, en fonction de son type. Dans le cas de `std::string`, on peut accéder au premier caractère avec `front`, ajouter un caractère à la fin avec `push_back`, etc. Ces fonctions, qui s'appliquent à une instance donnée d'une classe, sont appelées les *fonctions membres*. Nous en avons déjà un peu parlé au chapitre [Mais où est la doc ?](#). Ce sont elles que nous retrouvons sous la rubrique MEMBER FUNCTIONS.

Ce terme "fonction membre" est à mettre en relation avec "fonction libre" [ou *Non-member functions* dans la documentation], qui désigne les fonctions ne faisant partie d'aucun objet. Toutes les fonctions que nous avons écrites jusque-là étaient des fonctions libres. Beaucoup de celles que nous avons utilisées aussi, comme la fonction `std::size`.

Toutes les fonctions membres d'une classe, ainsi que toutes les fonctions libres qui peuvent manipuler cette classe, composent ce qu'on nomme *l'interface d'une classe*.

Note > L'interface d'une classe regroupe également d'autres concepts, pas uniquement les fonctions membres et les fonctions libres qui manipulent des instances de cette classe. On peut citer, entre autres, la surcharge d'opérateurs. Nous verrons le reste plus tard dans ce livre.

24.3. En C++, ça donne quoi ?

Créer des types, ça, nous savons le faire depuis que nous avons découvert les structures. Jusqu'ici, nous avons traitées ces dernières comme de simples agrégats de données. Il est temps de changer notre vision des choses et de passer à une approche objet.

Penser services

Reprenons l'exemple du chapitre [Reprendrez-vous un peu de sucre syntaxique ?, où nous avons manipulé des fractions](#). Quels services peut-on attendre de la part d'une fraction ? Comment celle-ci se manipule-t-elle ? Qu'est-il pertinent de faire avec une fraction ? La liste ci-dessous décrit les services sur lesquels nous allons travailler. Elle n'est pas exhaustive, mais constitue un bon début.

- Obtenir sa valeur réelle.
- La simplifier.
- Calculer sa puissance au degré x .

Penser tests

Maintenant que nous avons défini une liste des services que nous aimerions appliquer à un objet de type `Fraction`, il faut réfléchir aux tests que nous allons écrire pour vérifier que notre implémentation est correcte. Prenez le temps de les écrire, ce sont eux qui vous apporteront la garantie que votre code est de qualité. Voici un exemple.

```
#include <cassert>

int main()
{
    Fraction f1 { 5, 2 };
    assert(f1.valeur_reelle() == 2.5 && "5/2 équivaut à 2.5.");
    f1.simplifier();
    assert(f1.numérateur == 5 && f1.dénominateur == 2 && "5/2 après
simplification donne 5/2.");
    f1 = pow(f1, 2);
    assert(f1.numérateur == 25 && f1.dénominateur == 4 && "5/2 au carré
équivaut à 25/4.");

    Fraction f2 { 258, 43 };
    assert(f2.valeur_reelle() == 6 && "258/43 équivaut à 6.");
    f2.simplifier();
    assert(f2.numérateur == 6 && f2.dénominateur == 1 && "258/43 après
simplification donne 6/1.");
    f2 = pow(f2, 2);
```

```

    assert(f2.numerateur == 36 && f2.denominateur == 1 && "6/1 au carré
    équivaut à 36/1.");

    return 0;
}

```

Notez que les tests montrent qu'on aura à manipuler tant des fonctions membres que des fonctions libres.

Fonctions membres

Nous sommes maintenant prêts à définir notre classe. Commençons par les fonctions membres. Celles-ci se définissent au sein même de la classe et s'écrivent comme des fonctions libres.

```

struct Fraction
{
    double valeur_reelle();
    void simplifier();
};

```

Le prototype n'a rien d'extraordinaire, mais l'implémentation est quand même un poil inhabituelle. En effet, pour écrire l'implémentation d'une fonction membre, il faut préfixer son identifiant par le nom de la classe, avec l'opérateur `::` en guise de séparateur. C'est nécessaire pour que le compilateur sache que la fonction définie est membre d'une classe.

```

struct Fraction
{
    double valeur_reelle();
    void simplifier();
};

double Fraction::valeur_reelle()
{
}

void Fraction::simplifier()
{
}

```

UN MOT SUR L'OPÉRATEUR ::

Vous avez déjà croisé cet opérateur avec des objets courants comme `std::string` ou `std::vector`. C'est aussi lui qu'on utilise avec les énumérations. L'opérateur `::` est ce qu'on appelle un *opérateur de résolution de portée*. Il permet de lever toute ambiguïté éventuelle sur l'origine d'un type, d'une fonction ou d'une variable.

Dans le cas de `Fraction::valeur_reelle`, il indique qu'il existe une fonction `valeur_reelle` dans la portée `Fraction`, qui est ici une classe. Et dans le cas de `std::string`, qu'il existe un type `string` dans une portée `std`. Cette fois, ce n'est pas d'une classe dont il s'agit, mais d'un espace de noms [en anglais *namespace*], qui permet de regrouper plusieurs identificateurs sous une portée commune.

Grâce à ces portées, on peut mieux cloisonner son code, ce qui est plus propre. L'espace de nom `std` permet, par exemple, de regrouper tout ce qui fait partie de la bibliothèque standard. De même, `Fraction::` regroupe tout ce qui a trait à notre classe `Fraction`.

Fonctions libres

Continuons sur notre lancée en définissant des fonctions libres. Tout comme `size` est à la fois une fonction membre des classes `std::string` et `std::vector` et à la fois une fonction libre, nous pouvons opter pour l'une ou l'autre solution avec la classe `Fraction`. Nous devons simplement prendre en compte les points suivants.

- Certaines fonctions (comme l'opérateur `[]`) sont obligatoirement implémentées sous la forme membre.
- D'autres (comme l'opérateur `<<`) sont obligatoirement implémentées sous la forme libre.
- Pour celles qui peuvent être implémentées aussi bien libres que membres, il faut évaluer quelle façon de faire est la plus simple. Notamment, si un service peut être rendu en utilisant uniquement d'autres services, mieux vaut l'écrire sous forme de fonction libre. Si, au contraire, il a besoin de manipuler des détails internes, alors il vaut mieux l'implémenter en tant que fonction membre.

Dans le cas d'une fraction, il semble logique d'attendre d'une instance qu'elle nous donne sa valeur réelle et sa version simplifiée. Par contre, mettre une fraction à une puissance x , c'est comme `std::pow`, mais pour une fraction. Il paraît donc bien d'offrir

ce service sous forme de fonction libre. Cela uniformise l'utilisation de `pow`, qui a alors la même utilisation que ce soit pour un entier ou pour une `Fraction`.

```
Fraction pow(Fraction const & fraction, int puissance)
{
}
```

Les attributs

Maintenant que nous avons défini l'interface, il est temps de nous plonger dans les détails d'implémentation. N'oublions pas que, même si nous pensons et concevons services, il faut bien des données à manipuler. Dans notre cas, il s'agit d'un numérateur et d'un dénominateur. On peut désormais compléter la définition de la classe.

```
#include <cassert>
#include <cmath>
#include <numeric>

struct Fraction
{
    // Nos fonctions membres.
    double valeur_reelle();
    void simplifier();

    // Nos attributs.
    int numérateur;
    int dénominateur;
};

double Fraction::valeur_reelle()
{
    return static_cast<double>(numérateur) / dénominateur;
}

void Fraction::simplifier()
{
    int const pgcd { std::gcd(numérateur, dénominateur) };
    numérateur /= pgcd;
    dénominateur /= pgcd;
}

// Déclaration et implémentation d'un service sous forme libre.
Fraction pow(Fraction const & fraction, int puissance)
{
    Fraction copie { fraction };
    copie.numérateur = std::pow(fraction.numérateur, puissance);
    copie.dénominateur = std::pow(fraction.dénominateur, puissance);
    return copie;
}
```

LE MOT CLÉ `STATIC_CAST`

Si l'on divise un entier par un autre entier, on obtient un entier. Notre résultat serait alors faussé, puisqu'on perdrait toute la partie décimale. Pour éviter ce problème, il suffit que l'un des deux opérandes soit de type `double`. C'est ce que fait `static_cast`, en indiquant au compilateur que cette variable doit être considérée comme un réel et non plus un entier.

Notez que les fonctions membres manipulent sans problème les attributs de la classe. Chaque instance aura ses propres attributs, donc quand on écrit `f1.simplifier()`, ce sont le numérateur et le dénominateur de `f1` qui sont modifiés et uniquement eux.

24.4. Désolé, cet objet n'est pas modifiable

Revenons à un mot clé que nous connaissons depuis longtemps, que l'on utilise beaucoup mais que nous avons volontairement laissé de côté depuis l'introduction à la programmation orientée objet. Il s'agit de `const`. Il nous permet de déclarer qu'un objet est constant et ainsi empêcher d'éventuelles modifications. Sauf que si on déclare notre objet `Fraction` comme étant `const`, ça ne compile plus.

```
#include <cmath>
#include <iostream>
#include <numeric>

struct Fraction
{
    double valeur_reelle();
    void simplifier();

    int numerateur;
    int denominateur;
};

double Fraction::valeur_reelle()
{
    return static_cast<double>(numerateur) / denominateur;
}

void Fraction::simplifier()
{
    int const pgcd { std::gcd(numerateur, denominateur) };
    numerateur /= pgcd;
    denominateur /= pgcd;
}

Fraction pow(Fraction const & fraction, int puissance)
```



```

{
    Fraction copie { fraction };
    copie.numerateur = std::pow(fraction.numerateur, puissance);
    copie.denominateur = std::pow(fraction.denominateur, puissance);
    return copie;
}

int main()
{
    Fraction const f1 { 5, 2 };
    std::cout << "Voici la valeur réelle de 5/2 : " << f1.valeur_reelle()
    << "\n";
    return 0;
}

```

```

[Visual Studio]
E1086 l'objet a des qualificateurs de type incompatibles avec le membre
fonction "Fraction::valeur_reelle"
C2662 'double Fraction::valeur_reelle(void)' : impossible de convertir
un pointeur 'this'
de 'const Fraction' en 'Fraction &'

-----

[GCC]
tests.cpp: In function 'int main()':
tests.cpp:48:74: error: passing 'const Fraction' as 'this' argument
discards qualifiers [-fpermissive]
   48 |         std::cout << "Voici la valeur réelle de 5/2 : " <<
      |         ^
      |         ^
tests.cpp:18:8: note:   in call to 'double Fraction::valeur_reelle()'
   18 | double Fraction::valeur_reelle()
      |         ^~~~~~

-----

[Clang]
prog.cc:43:57: error: 'this' argument to member function 'valeur_reelle'
has type 'const Fraction',
but function is not marked const
   std::cout << "Voici la valeur réelle de 5/2 : " <<
   f1.valeur_reelle() << "\n";
                                     ^~

prog.cc:15:18: note: 'valeur_reelle' declared here
double Fraction::valeur_reelle()
                  ^
1 error generated.

```

Le message émis par GCC est un peu moins clair que les autres, mais les trois indiquent que notre fonction membre n'est pas utilisable si l'instance est déclarée comme constante, parce que cette fonction membre ne l'est pas.

Une fonction membre constante, c'est une fonction qui *promet de ne pas modifier l'objet* auquel elle appartient. Par exemple, dans le cas de `std::vector`, la fonction membre `size` retourne simplement la taille du tableau. Cette fonction donc est déclarée comme étant constante et on peut l'utiliser sur des tableaux constants. Au contraire, `push_back` modifie le tableau et n'est donc utilisable que sur un tableau non `const`.

Le mieux est d'avoir *le plus de fonctions constantes possibles*. Moins on modifie de choses, moins on a de risques d'introduire des erreurs et des bugs. Il convient donc d'y réfléchir dès la conception d'une classe. Ici, on voit que `valeur_reelle` est un très bon candidat pour ça.

Pour qu'une fonction membre soit utilisable même quand l'objet est constant, il faut qu'elle soit explicitement marquée comme constante. Cela se fait très simplement, en rajoutant le mot clé `const` à la fin de la signature de la fonction en question, tant dans le prototype que dans l'implémentation.

```
struct Fraction
{
    double valeur_reelle() const;
};

// Ne pas oublier const dans l'implémentation aussi.
double Fraction::valeur_reelle() const
{
    return static_cast<double>(numérateur) / dénominateur;
}
```

Bien sûr, certaines fonctions ne pourront jamais être `const`. Par exemple, vouloir déclarer `simplifier` comme étant constante n'a aucun sens, puisque c'est sa raison d'être que de modifier l'instance sur laquelle elle est appelée. Si vous essayez, le compilateur va vous envoyer balader.

```
[Visual Studio]
C3490 impossible de modifier 'numérateur' car il est accessible via un
objet const
C3490 impossible de modifier 'dénominateur' car il est accessible via
un objet const

-----

[GCC]
tests.cpp: In member function 'void Fraction::simplifier() const':
tests.cpp:24:16: error: assignment of member 'Fraction::numérateur' in
read-only object
   24 |         numérateur /= pgcd;
      |         ~~~~~^~~~~~
tests.cpp:25:18: error: assignment of member 'Fraction::dénominateur' in
read-only object
```

```

25 |     denominateur /= pgcd;
    |     ~~~~~^~~~~~
    |
-----

[Clang]
prog.cc:24:16: error: cannot assign to non-static data member within
const member function 'simplifier'
    numerateur /= pgcd;
    ~~~~~^
prog.cc:21:16: note: member function 'Fraction::simplifier' is declared
const here
void Fraction::simplifier() const
~~~~~^~~~~~
prog.cc:25:18: error: cannot assign to non-static data member within
const member function 'simplifier'
    denominateur /= pgcd;
    ~~~~~^
prog.cc:21:16: note: member function 'Fraction::simplifier' is declared
const here
void Fraction::simplifier() const
~~~~~^~~~~~
2 errors generated.

```

24.5. On ne fait pas d'exception

Au chapitre [Mais où est la doc ?](#), nous avons vu qu'il existait pour les fonctions une section `EXCEPTIONS` qui nous indique si elles sont susceptibles de lancer une exception ou non. Pour celles qui ont la garantie de ne rien lancer, il y a le mot clé `noexcept`. La même chose est possible pour les classes que nous créons.

Par exemple, la fonction membre `valeur_reelle` consiste en une simple division et ne lèvera donc jamais d'exception. Par contre, même si `simplifier` ne fait appel qu'à `std::gcd` en interne, nous n'avons pas de garantie, car sa [documentation](#) ne précise pas que `std::gcd` est `noexcept`. Quant à `std::pow`, aucune information n'est donnée. Nous la laisserons telle quelle.

Comme pour `const`, le mot clé `noexcept` s'écrit deux fois, tant après le prototype au sein de la classe qu'après la signature au moment de l'implémentation.

```

#include <cmath>
#include <iostream>
#include <numeric>

struct Fraction
{
    double valeur_reelle() const noexcept;
    void simplifier();

    int numerateur;

```

```

    int denominateur;
};

double Fraction::valeur_reelle() const noexcept
{
    return static_cast<double>(numérateur) / denominateur;
}

void Fraction::simplifier()
{
    int const pgcd { std::gcd(numérateur, denominateur) };
    numérateur /= pgcd;
    denominateur /= pgcd;
}

Fraction pow(Fraction const & fraction, int puissance)
{
    Fraction copie { fraction };
    copie.numérateur = std::pow(fraction.numérateur, puissance);
    copie.denominateur = std::pow(fraction.denominateur, puissance);
    return copie;
}

int main()
{
    Fraction const f1 { 5, 2 };
    std::cout << "Voici la valeur réelle de 5/2 : "
               << f1.valeur_reelle() << "\n";
    return 0;
}

```

24.6. Découpage en fichiers

Bien que tout écrire dans un fichier soit plus rapide (en particulier dans le cas de nos exemples), dans la réalité, rappelez-vous, on sépare les définitions des implémentations, ainsi que nous l'avons vu au chapitre [Découpons du code – les fichiers](#). Le principe est valable aussi ici.

Fraction.hpp

```

#ifndef FRACTION_HPP
#define FRACTION_HPP

struct Fraction
{
    double valeur_reelle() const noexcept;
    void simplifier();

    int numérateur;
    int denominateur;
};

```

```

    Fraction pow(Fraction const & fraction, int puissance);

#endif

```

Fraction.cpp

```

#include <cmath>
#include <numeric>

#include "Fraction.hpp"

double Fraction::valeur_reelle() const noexcept
{
    return static_cast<double>(numérateur) / dénominateur;
}

void Fraction::simplifier()
{
    int const pgcd { std::gcd(numérateur, dénominateur) };
    numérateur /= pgcd;
    dénominateur /= pgcd;
}

Fraction pow(Fraction const & fraction, int puissance)
{
    Fraction copie { fraction };
    copie.numérateur = std::pow(fraction.numérateur, puissance);
    copie.dénominateur = std::pow(fraction.dénominateur, puissance);
    return copie;
}

```

main.cpp

```

#include <iostream>
#include "Fraction.hpp"

int main()
{
    Fraction const f1 { 5, 2 };
    std::cout << "5/2 = "
               << f1.valeur_reelle() << "\n";
    return 0;
}

```

Pour la suite du livre, la majorité des exemples resteront écrits dans un seul fichier, par simplicité et facilité. Néanmoins, quand vous commencerez des projets plus gros, découpez et vous gagnerez en lisibilité et compréhension.

24.7. Entraînez-vous !

Cercle

Imaginons qu'on veuille créer un cercle. Celui-ci doit pouvoir faire l'objet d'un certain nombre d'opérations.

- Calculer son aire.
- Calculer son périmètre.
- Déterminer si un point fait partie du cercle ou non.

Si on se base sur ces informations, alors on peut déjà imaginer les tests qui suivent.

```
#include <cassert>
#include <iostream>

bool compare(double lhs, double rhs)
{
    return std::abs(lhs - rhs) < 0.001;
}

int main()
{
    Point const centre { 0, 0 };
    Cercle const c1 { 1, centre };

    std::cout << "Voici l'aire du cercle : "
               << c1.aire() << "\n";
    assert(compare(c1.aire(), 3.1415)
           && "L'aire du cercle c1 doit valoir pi.");

    std::cout << "Voici le périmètre du cercle : "
               << c1.perimetre() << "\n";
    assert(compare(c1.perimetre(), 3.1415 * 2)
           && "Le périmètre du cercle doit valoir pi * 2.");

    std::cout << std::boolalpha;
    Point const p1 { 2, -18 };
    std::cout << "Est-ce que le point p1 appartient au cercle c1 ? "
               << appartient(c1, p1) << "\n";
    assert(!appartient(c1, p1)
           && "Le point p1 n'appartient pas au cercle c1.");

    return 0;
}
```

Nous vous avons ajouté une fonction `compare`, qui permet de dire si deux nombres réels sont égaux en ne se basant que sur les trois premières décimales. Sans ça, si on les

compare directement avec `==`, alors l'opération se fera sur les milliers de décimales qui composent nos `double`. On risque d'avoir des résultats faussés, car le programme nous dira alors que deux nombres sont différents si par exemple leur 20000^e décimale sont différentes, ce qui est négligeable à notre niveau.

✧ [Voir la solution](#)

? [Obtenir un indice](#)

Informations personnelles

Vous souvenez-vous de la structure `InformationsPersonnelles`, que nous avons écrite dans le chapitre [De nouvelles structures de données](#)? Pourrait-on la réécrire avec un point de vue objet? Quels services pourrait-elle rendre? Quels invariants pourraient être mis en place? Nous laissons ces questions ouvertes, à vous de décider.

24.8. Récapitulons !

- La programmation orientée objet, c'est penser en termes de services rendus et non en termes de données.
- Une classe est un type personnalisé que l'on définit comme on veut.
- On parle d'instancier une classe quand on crée un objet de ce type.
- L'interface d'une classe comprend notamment les fonctions membres et les fonctions libres qui manipulent les instances de cette classe.
- Seules les fonctions membres constantes peuvent manipuler des instances déclarées `const`.
- Les fonctions membres doivent être déclarées constantes le plus possible.
- De la même manière, il est bien de déclarer `noexcept` toutes les fonctions membres qui ne lèvent pas d'exception.
- La séparation en fichiers nous permet de séparer la définition d'une classe de l'implémentation de ses fonctions membres.

SOLUTION

section Cercle

Comme un cercle a un point central, les opérations concernant les points ont été séparées de celles concernant les cercles. En effet, le calcul de la distance entre deux points n'est pas une opération réservée aux seuls cercles, mais peut être effectué pour n'importe quel point. Nous avons aussi décidé de fournir les services `distance` et `appartient` comme des fonctions libres, car ces opérations ne sont pas liées à une instance en particulier, mais sont générales. C'est un choix de conception, nullement une obligation.

Point.hpp

```
#ifndef POINT_HPP
#define POINT_HPP

struct Point
{
    int x;
    int y;
};

double distance(Point const & lhs, Point const & rhs);
#endif
```

Point.cpp

```
#include <cmath>
#include "Point.hpp"

double distance(Point const & lhs, Point const & rhs)
{
    return std::sqrt(std::pow(lhs.x - rhs.x, 2) + std::pow(lhs.y -
        rhs.y, 2));
}
```

Cercle.hpp

```
#ifndef CERCLE_HPP
#define CERCLE_HPP

#include <cmath>
#include "Point.hpp"

struct Cercle
{
    double aire() const noexcept;
    double perimetre() const noexcept;
```


SOLUTION

```

    int rayon;
    Point centre;
};

bool appartient(Cercle const & cercle, Point const & point);

#endif

```

Cercle.cpp

```

#include "Cercle.hpp"

// Avec C++17 et inférieur.
double const pi { std::acos(-1) };
// Si vous compilez avec C++20, vous pouvez utiliser
// le fichier <numbers>.
//
// using namespace std::numbers;
// La valeur pi est maintenant accessible.

double Cercle::aire() const noexcept
{
    return rayon * rayon * pi;
}

double Cercle::perimetre() const noexcept
{
    return 2 * rayon * pi;
}

bool appartient(Cercle const & cercle, Point const & point)
{
    return distance(cercle.centre, point) <= cercle.rayon;
}

```

INDICE

section Cercle

Petits rappels mathématiques :

- L'aire d'un cercle de rayon r vaut $r^2 \times \pi$.
- Le périmètre d'un cercle de rayon r vaut $2 \times r \times \pi$.
- La distance entre deux points $P_1 (x; y)$ et $P_2 (a; b)$ se calcule avec la formule $\sqrt{(x-a)^2 + (y-a)^2}$.

25

Et qui va construire tout ça ?

Dans le chapitre précédent, nous avons appris à réfléchir en termes de services et nous avons vu comment le faire concrètement. Mais la programmation orientée objet, ce n'est pas que ça, autrement elle ne vaudrait pas le détour. Dans ce chapitre, nous allons examiner quelques problèmes qui se posent avec notre classe `Fraction` et les techniques qui existent pour les résoudre et améliorer la qualité de notre code.

Ce chapitre va nous faire découvrir les *invariants*, les *constructeurs*, ainsi qu'approfondir la *gestion d'erreurs*.

25.1. Encapsulation et invariants

Cette classe, c'est open bar

Notre classe `Fraction` a été définie et implémentée suivant une logique de services et c'est une bonne chose. Reste que, pour l'instant, nos attributs sont toujours visibles et *librement modifiables*. Quel intérêt d'aller au pressing si n'importe qui peut avoir accès aux machines à laver et mettre le bazar ?

Vous voulez un exemple d'erreur qu'on peut obtenir en faisant n'importe quoi ? Regardez le code ci-dessous.

```
int main()
{
    Fraction f1 { 4, 2 };
    std::cout << "4/2 = " << f1.valeur_reelle() << std::endl;

    // Erreur !
    f1.denominateur = 0;
    // Ça va faire boom !
    std::cout << "4/0 = " << f1.valeur_reelle() << std::endl;

    return 0;
}
```

```
4/2 = 2
4/0 = inf
```

Rappelons que diviser un entier par 0 est mathématiquement indéfini. On peut donc obtenir tout et n'importe quoi comme résultat. Ici, c'est l'infini, mais ça aurait très bien pu être autre chose (d'où le résultat peut-être différent que vous avez chez vous).

Le vrai problème, c'est qu'en modifiant directement un attribut, on s'arroge des droits qui ne sont pas les nôtres et on peut potentiellement *tout* casser. Si on part du principe que cette classe doit représenter fidèlement le concept mathématique de fraction, alors on est face à un problème, puisque injecter directement un dénominateur nul invaliderait l'objet.

Tout est question d'invariant

Justement, qu'est-ce qui permet de dire que cet objet n'est plus valide ? C'est qu'il ne respecte plus une condition qui devrait toujours être vraie, celle que le dénominateur ne doit jamais valoir zéro. Cette condition, qu'on appelle *invariant*, c'est le concepteur de la classe qui la définit.

Nous aurions pu aussi très bien dire qu'on s'en fiche et que diviser par zéro n'est pas une erreur. Ce sont des choix de conception. Disons qu'on veut implémenter une classe qui offre des services proches de ceux d'une fraction mathématiquement parlant. On décide donc qu'il y a toujours un invariant à respecter, celui que le dénominateur ne soit jamais nul.

Tant que cet invariant est respecté, chaque instance de la classe `Fraction` est supposée rendre les services qu'elle expose (simplifier, puissance, etc.). En revanche, s'il vient à être transgressé, alors tout peut arriver. C'est typiquement de la *programmation par contrat*, déjà abordée dans la deuxième partie. Souvenez-vous, nous avons vu qu'il existe des *préconditions*, qui doivent être respectées avant l'appel à une fonction, et des *postconditions*, que cette dernière doit garantir quand elle termine de s'exécuter. Le principe est le même ici.

Vous voulez d'autres exemples ? Revenons à notre pressing. Son invariant est que votre linge ne doit pas être abîmé. Si vous vous amusez à changer par vous-même, sans rien y connaître, la lessive ou la programmation de la machine à laver, il ne faudra pas vous étonner que le linge soit décoloré ou rétréci.

L'invariant d'un carré, c'est que la longueur soit égale à la largeur. L'invariant d'un jeu de tarot, c'est qu'il y ait 78 cartes. L'invariant d'une date, c'est que les jours, les mois et les

années renseignés soient valides. L'invariant d'une machine à café, c'est qu'elle ne fasse pas de court-jus. L'invariant d'une variable, c'est qu'elle soit prête à l'emploi sans qu'on ait à se poser de question (imaginez créer un `std::string` et devoir manuellement initialiser la chaîne comme étant vide, pas très pratique). En fait, dès qu'il y a des conditions à respecter quand on veut créer un objet, c'est qu'il y a un ou plusieurs invariants.

Encapsulons tout ça

La solution pour respecter nos invariants s'appelle l'*encapsulation*. Plutôt que de laisser nos attributs librement modifiables par le premier venu, nous allons les protéger, les enrober d'une couche protectrice, d'une capsule. Seuls les services seront publics et seront à même de modifier les attributs. Il faut pour cela changer la visibilité des attributs.

Il existe trois statuts de visibilité, qui peuvent s'appliquer tant aux attributs qu'aux fonctions membres.

- `public` : l'élément est visible et accessible par tout le monde.
- `private` : l'élément ne peut être manipulé que par une instance de la classe.
- Ni l'un, ni l'autre : nous verrons cela [en temps voulu](#).

UN MOT SUR LA VISIBILITÉ

Ce concept de visibilité est à mettre en lien avec le découpage en fichiers d'en-tête et fichiers sources. Dans les premiers, on trouve les fonctions, les classes, etc. qu'un code peut appeler et utiliser. Ce sont les *interfaces publiques*, ce que le reste du monde voit. Au contraire, dans les fichiers sources, qui contiennent les détails d'implémentation, tout est masqué.

De la même manière, nos services seront publiquement exposés, alors que les détails d'implémentation seront cachés et inaccessibles par le reste du monde.

Par défaut, une structure a tous ses éléments publics, d'où le problème soulevé précédemment. Heureusement, la visibilité n'est pas figée dans le marbre.

Modifier sa visibilité

Deux mots clés, `public` et `private`, permettent respectivement de rendre un élément public ou privé. C'est justement ce dont nous avons besoin. Grâce à `public`, nous pouvons

permettre à tout le monde de faire appel à nos services. Grâce à `private`, nous protégeons nos invariants de toute manipulation directe malencontreuse.

Chaque mot clé s'applique jusqu'à ce qu'on rencontre un autre modificateur de visibilité, ou jusqu'à la fin de la classe s'il n'y en a pas d'autres.

```
class Fraction ❶
{
    // Tout ce qui est en dessous est maintenant public.
public:
    double valeur_reelle() const noexcept;
    void simplifier();

    // Tout ce qui est en dessous est maintenant privé et caché.
private:
    int m_numerator; ❷
    int m_denominateur;
};
```

- ❷ Les attributs sont maintenant préfixés par `m_` [de l'anglais *member*, membre]. Cela permet de voir d'un seul coup d'œil qu'on a affaire à un attribut privé. Cette convention n'est pas obligatoire, mais est très répandue dans le monde C++. Nous l'utiliserons dans la suite de cet ouvrage.

Note > Certains utilisent `my`, d'autres postfixent avec `_`. Il y en a pour tous les goûts.

- ❶ Le mot clé `struct` a été remplacé par `class`. Les deux sont interchangeables, la seule différence étant que, par défaut, tout est public avec `struct`, alors que tout est privé avec `class`.

STRUCT OU CLASS ?

Les deux sont très proches et on peut sans problème adapter la visibilité dans les deux cas. Il y a néanmoins une convention répandue, qui stipule d'utiliser `struct` dans le cas d'une simple *agrégation de données* et de réserver `class` quand il y a présence d'*au moins un invariant*.

Par exemple, une structure `Point`, regroupant des coordonnées (x, y, z) , n'a pas vraiment d'invariant et peut être laissée comme telle. Par contre, une fraction peut présenter au moins un invariant, celui de ne jamais avoir de dénominateur à zéro. Une classe sera donc plus adaptée.

25.2. On cherche un constructeur

Le principe

Si vous essayez de compiler le code ci-dessous, qui reprend toutes les modifications que nous venons de faire, vous allez obtenir une erreur. En effet, nous ne pouvons plus initialiser notre objet avec les valeurs que l'on souhaite.

```
#include <cmath>
#include <iostream>
#include <numeric>

class Fraction
{
public:
    double valeur_reelle() const noexcept;
    void simplifier();

private:
    int m_numérateur;
    int m_dénominateur;
};

double Fraction::valeur_reelle() const noexcept
{
    return static_cast<double>(m_numérateur) / m_dénominateur;
}

void Fraction::simplifier()
{
    int const pgcd { std::gcd(m_numérateur, m_dénominateur) };
    m_numérateur /= pgcd;
    m_dénominateur /= pgcd;
}

int main()
{
    Fraction const f1 { 5, 2 };
    std::cout << "5/2 = "
                << f1.valeur_reelle() << "\n";
    return 0;
}
```

```
[Visual Studio]
Erreur C2440 'initialisation' : impossible de convertir de 'initializer
list' en 'Fraction' TestCpp.cpp 30
Erreur E0146 trop de valeurs d'initialiseur TestCpp.cpp 30
```

```
[GCC]
prog.cc: In function 'int main()':
prog.cc:44:30: error: no matching function for call to
  'Fraction::Fraction(<brace-enclosed initializer list>)'
   44 |         Fraction const f1 { 5, 2 };
       |                        ^
prog.cc:5:7: note: candidate: 'Fraction::Fraction()'
   5 | class Fraction
     | ^~~~~~
prog.cc:5:7: note: candidate expects 0 arguments, 2 provided
prog.cc:5:7: note: candidate: 'constexpr Fraction::Fraction(const
  Fraction&)'
prog.cc:5:7: note: candidate expects 1 argument, 2 provided
prog.cc:5:7: note: candidate: 'constexpr Fraction::Fraction(Fraction&&)'
prog.cc:5:7: note: candidate expects 1 argument, 2 provided
-----

[Clang]
prog.cc:44:20: error: no matching constructor for initialization of
  'const Fraction'
   44 |         Fraction const f1 { 5, 2 };
       |                        ^ ~~~~~~
prog.cc:5:7: note: candidate constructor (the implicit copy constructor)
not viable: requires 1 argument, but 2 were provided
class Fraction
  ^
prog.cc:5:7: note: candidate constructor (the implicit move constructor)
not viable: requires 1 argument, but 2 were provided
prog.cc:5:7: note: candidate constructor (the implicit default
constructor) not viable: requires 0 arguments, but 2 were provided
1 error generated.
```

Le message d'erreur de Clang est plus clair que celui de GCC, puisqu'il nous indique clairement qu'il ne trouve pas le bon constructeur.

Un *constructeur* est une fonction membre spéciale appelée lors de l'instanciation d'une classe et qui s'occupe d'*initialiser* l'objet. Comme il s'agit du point d'entrée depuis l'extérieur, c'est ici que nous allons pouvoir définir nos invariants :

- si tous les invariants sont respectés, alors l'objet est correctement initialisé et utilisable ;
- si un des invariants est transgressé, alors on va avorter la construction de l'objet. À noter que si les préconditions du constructeur ne sont pas respectées, on ne peut même pas définir d'invariant, ce qui signifie qu'on a donc affaire à une erreur de programmation.

Le compilateur génère implicitement et automatiquement trois constructeurs :

- le constructeur par défaut, que nous allons voir tout de suite. C'est celui que GCC désigne par `Fraction::Fraction()` ;
- le constructeur de copie, que nous verrons au [chapitre suivant](#). Il est de la forme `constexpr Fraction::Fraction(const Fraction&)` ;
- le constructeur de déplacement, que nous verrons au chapitre [La sémantique de déplacement](#). Il est de la forme `constexpr Fraction::Fraction(Fraction&&)`.

La syntaxe

Tous les constructeurs d'une classe partagent la même syntaxe, ce qui va simplifier les choses pour vous. En fait, seuls leurs paramètres et leur nombre changent.

Tout d'abord, aucun constructeur n'a de type de retour écrit, pas même `void`. Le prototype du constructeur est donc constitué uniquement du nom de la classe, plus les éventuels arguments. Et comme pour les autres fonctions membres, on retrouve le nom de la classe et l'opérateur de résolution de portée `::`. Enfin, on peut retrouver `constexpr` et/ou `noexcept`, si besoin.

L'autre particularité va vous surprendre : on initialise les attributs directement en dessous de la signature du constructeur. On appelle cette syntaxe la *liste d'initialisation des membres* [en anglais *member initializer lists*]. Voyez par vous-même comment cela s'applique à notre classe `Fraction`.

```
#include <cassert>
#include <cmath>
#include <iostream>
#include <numeric>

class Fraction
{
public:
    // Pas d'exception car on ne fait qu'initialiser deux entiers.
    Fraction(int numérateur, int dénominateur) noexcept;
    double valeur_reelle() const noexcept;
    void simplifier();

private:
    int m_numérateur;
    int m_dénominateur;
};

// Le qualificateur noexcept se répète aussi ici.
Fraction::Fraction(int numérateur, int dénominateur) noexcept
    : m_numérateur(numérateur), m_dénominateur(dénominateur)
{
    assert(m_dénominateur != 0 && "Le dénominateur ne peut pas valoir 0.");
}
```

```

}

double Fraction::valeur_reelle() const noexcept
{
    return static_cast<double>(m_numerateur) / m_denominateur;
}

void Fraction::simplifier()
{
    int const pgcd { std::gcd(m_numerateur, m_denominateur) };
    m_numerateur /= pgcd;
    m_denominateur /= pgcd;
}

int main()
{
    Fraction const f1 { 5, 2 };
    std::cout << "5/2 = "
                << f1.valeur_reelle() << "\n";
    return 0;
}

```

L'avantage de passer par un constructeur pour faire ça, c'est qu'on peut vérifier nos contrats AVANT de créer l'objet. Ainsi, si on reçoit un argument incorrect ou si une erreur externe survient, il est possible de réagir immédiatement, avec une assertion ou une exception.

Tout comme nos fonctions ont des préconditions qu'on vérifie avec `assert`, nos objets ont des préconditions qui seront vérifiées dans le constructeur. Son but n'est donc pas simplement d'initialiser, mais aussi de vérifier qu'on est en droit de le faire.

25.3. Constructeur par défaut

Nommé en anglais *default constructor*, c'est lui qui est appelé quand on instancie une classe sans fournir d'argument. Par exemple, quand nous écrivons `std::string chaine {}` ou `std::vector tableau {}`, nos instances sont initialisées par défaut. Dans le premier cas, on obtient une chaîne de caractères vide, dans le second un tableau vide.

Le compilateur en génère automatiquement un (sauf si un ou plusieurs constructeurs personnalisés ont déjà été définis). Le problème, c'est que le compilateur se contente de tout initialiser à la valeur par défaut : chaîne vide pour `std::string`, 0 pour `int`, etc. Cela peut nous jouer de mauvais tours, par exemple dans le cas de la classe `Fraction`.

```

#include <iostream>

class Fraction
{

```

```

// Sans autre constructeur personnalisé,
// le compilateur implémente le constructeur par défaut.
};

int main()
{
    Fraction const f1 {};
    std::cout << "Valeur réelle par défaut = "
              << f1.valeur_reelle() << "\n";
    return 0;
}

```

```

[Visual Studio]
Valeur réelle par défaut = -nan(ind)

```

```

[GCC]
Valeur réelle par défaut = -nan

```

```

[Clang]
Valeur réelle par défaut = -nan

```

Le code ci-dessus est bogué parce que `m_numérateur` et `m_dénominateur` valent tous les deux `0`, et nous avons donc affaire à une division non définie. Notre programme n'est en conséquence pas très robuste. Moralité, le constructeur par défaut implicite ne nous plaît pas.

Nous devons commencer par nous interroger si créer une *fraction par défaut* a, ou non, du sens. À partir de cette réflexion, deux directions sont possibles.

Initialiser une instance par défaut a du sens

On peut décider, à bon droit, qu'une fraction sans plus autre détail vaut `0`, ou `1`, ou `42`, ou encore `97455`. C'est notre choix en tant que concepteur. Partons du principe qu'une fraction vaut par défaut zéro.

La solution consiste donc à initialiser directement les attributs dont on veut changer la valeur par défaut et laisser le constructeur par défaut s'occuper du reste. On peut simplement demander explicitement au compilateur de le générer, ce qui rend le code plus clair. Cela se fait en rajoutant `= default` après la signature.

```

class Fraction
{
public:
    Fraction() = default;

```

```
private:
    // On peut laisser le compilateur le mettre
    // à 0, ou l'écrire nous-mêmes.
    int m_numérateur = 0;
    int m_dénominateur = 1;
};
```

Initialiser une instance par défaut n'a pas de sens

On peut aussi très bien décider, à l'opposé, que l'utilisateur doit explicitement initialiser toute fraction qu'il souhaite créer. C'est aussi un choix valable. Dans ce cas, on va lui interdire l'utilisation du constructeur par défaut.

Comme dit précédemment, si la classe définit des constructeurs personnalisés, alors le constructeur par défaut n'est implicitement pas créé, donc impossible de l'utiliser. Mais comme la clarté est une bonne pratique que nous nous efforçons de vous transmettre, nous allons dire explicitement au compilateur que nous n'en voulons pas. Cela se fait en ajoutant `= delete` après la signature.

```
class Fraction
{
public:
    Fraction() = delete;

private:
    int m_numérateur;
    int m_dénominateur;
};
```

Voici l'erreur que vous obtiendrez si vous essayez quand même d'utiliser le constructeur par défaut.

```
[Visual Studio]
Erreur C2280 'Fraction::Fraction(void)' : tentative de référencement
d'une fonction supprimée
Erreur (active) E1790 impossible de faire référence au constructeur
par défaut de "Fraction"
```

```
-----
[GCC]
main.cpp: In function 'int main()':
main.cpp:26:18: error: use of deleted function
'Fraction::Fraction()'
   26 |         Fraction f1 {};
      |         ^
main.cpp:10:5: note: declared here
   10 |         Fraction() = delete;
      |         ^~~~~~
```

```
-----
[Clang]
prog.cc:26:14: error: call to deleted constructor of 'Fraction'
    Fraction f1 {};
              ^  ~~
prog.cc:10:5: note: 'Fraction' has been explicitly marked deleted here
    Fraction() = delete;
    ^
1 error generated.
```

POUR OU CONTRE ?

Proposer ou non un constructeur par défaut dépend de vos choix de conception. Dans certains cas, cela a du sens, comme une chaîne de caractères vide. Dans d'autres cas, c'est absurde, comme pour un livre, une voiture, un humain.

25.4. Soyons explicites

Pour pouvoir obtenir le nombre réel correspondant à la division de deux entiers, nous avons utilisé `static_cast`, qui permet de faire une *conversion explicite* d'un type vers un autre. Mais qui dit explicite dit souvent implicite et, en effet, C++ permet de nombreuses *conversions implicites*. C'est par exemple le cas dans la ligne suivante, où un caractère est implicitement converti en entier.

```
int const entier { 'A' };
```

De telles conversions peuvent aussi se produire lors de l'appel d'un constructeur à un argument (ou à plusieurs arguments, mais avec des valeurs par défaut). Parfois, ça nous rend service, comme lorsqu'on manipule des chaînes de caractères avec `std::string`. Ainsi dans l'exemple suivant, les conversions implicites sont utiles et même désirées. Ici, l'utilisateur peut passer à son gré un littéral comme en C, ou bien un littéral de type `std::string`. On lui offre un plus grand choix, une plus grande liberté, sans rendre le code plus complexe ou plus obscur.

```
#include <string>

class Mot
{
public:
    Mot(std::string const & mot);
```

```

private:
    std::string m_mot;
};

Mot::Mot(std::string const & mot)
    : m_mot(mot)
{
}

using namespace std::string_literals;

int main()
{
    // Conversion implicite d'une chaîne de caractères C
    // en std::string au moment de l'appel au constructeur.
    Mot const arbre { "arbre" };

    // Pas de conversion car argument de type std::string.
    Mot const agrume { "agrume"s };

    return 0;
}

```

Mais d'autres fois, cette conversion implicite pose problème. Imaginez, par exemple, une classe représentant un colis. En fonction de son poids et de plein d'autres paramètres, comme l'adresse, le mode de livraison, etc., le prix à payer varie. Modélisons tout ça (le strict minimum).

```

#include <cassert>
#include <iostream>

enum class ModeLivraison
{
    Standard,
    Express,
    Urgent,
    Economique
};

class Adresse
{
    // Juste pour l'exemple.
};

class Colis
{
public:
    Colis(double poids);
private:
    double m_poids { 0. };
};

Colis::Colis(double poids)

```

```

    : m_poids(poids)
{
    assert(poids >= 0.1 && "Le poids ne peut pas être inférieur à 100
    g.");
}

double calcul_prix(Colis const & colis, Adresse const & adresse,
    ModeLivraison mode)
{
    // Plein d'opérations complexes pour
    // calculer le prix à payer pour le colis.
    return 10;
}

int main()
{
    Adresse adresse_livraison {};
    std::cout << "Voici le prix à payer : "
    << calcul_prix(4, adresse_livraison, ModeLivraison::Express)
    << "\n";
    return 0;
}

```

Notez l'appel à la fonction libre `calcul_prix`, car c'est ici que se niche le problème. En effet, le premier argument est censé être une référence sur un objet `Colis const`, pourtant le compilateur ne bronche pas alors qu'on lui donne un entier. Normal, il fait appel à un *constructeur de conversion*.

Dans notre cas, la conversion implicite qu'il fait d'un entier vers un `Colis` nuit à la lisibilité du code. On ne comprend pas tout de suite à quoi on a affaire en voyant ce littéral `4`. Pire, ça va à l'encontre du principe de moindre surprise, puisque ce genre de comportement en douce peut être difficile à détecter et mener à des bugs apparemment incompréhensibles. Mieux vaut interdire la conversion implicite, ce qui se fait en utilisant le mot clé `explicit` (uniquement devant la déclaration, pas devant l'implémentation).

```
explicit Colis(double poids);
```

Maintenant, le compilateur va nous signaler que passer un entier est invalide.

```
[Visual Studio]
Erreur C2664 'double calcul_prix(const Colis &,const Adresse
&,ModeLivraison)' : impossible de convertir l'argument 1 de 'int' en
'const Colis &'
Erreur (active) E0415 il n'existe aucun constructeur approprié pour la
conversion de "int" en "Colis"

-----

[GCC]
main.cpp: In function 'int main()':
main.cpp:41:61: error: invalid initialization of reference of type
'const Colis&' from expression of type 'int'
  41 |         std::cout << "Voici le prix à payer : " << calcul_prix(4,
      |         adresse_livraison, ModeLivraison::Express) << "\n";
      |                                     ^
main.cpp:32:34: note: in passing argument 1 of 'double calcul_prix(const
Colis&, const Adresse&, ModeLivraison)'
  32 | double calcul_prix(Colis const & colis, Adresse const & adresse,
      |                    ~~~~~~^~~~~~
      |                    Modelivraison mode)

-----

[Clang]
prog.cc:41:49: error: no matching function for call to 'calcul_prix'
  std::cout << "Voici le prix à payer : " << calcul_prix(4,
  adresse_livraison, ModeLivraison::Express) << "\n";
                                     ^~~~~~
prog.cc:32:8: note: candidate function not viable: no known conversion
from 'int' to 'const Colis' for 1st argument
double calcul_prix(Colis const & colis, Adresse const & adresse,
  Modelivraison mode)
      ^
1 error generated.
```

Alors, **explicit** ou pas ? Là encore, le choix de déclarer un constructeur comme étant explicite ne dépend que de vous et de vos choix de conception. On recommande souvent de *définir comme **explicit** tout constructeur prenant un seul argument* (ce qui inclut ceux ayant plusieurs paramètres optionnels), car les conversions implicites peuvent nuire à la compréhension du code.

Néanmoins, dans certains cas, elles sont appréciables. Par exemple, on peut très bien passer un entier 5 à une fonction attendant une fraction, car un entier est un cas particulier de fraction. On pourrait dire pareil pour les nombres complexes (un entier est un nombre complexe avec une partie imaginaire nulle), les quaternions, etc. Il faut donc peser à chaque fois le pour et le contre.

25.5. En toute amitié

Avez-vous noté que, depuis le début de ce chapitre, nous avons volontairement omis la fonction `pow` ? C'est que, maintenant que nous avons encapsulé nos attributs, nous n'y avons plus accès en dehors de la classe. Comme `pow` est une fonction libre, elle ne peut pas accéder aux attributs `m_numérateur` et `m_dénominateur`.

```
#include <cassert>
#include <cmath>
#include <iostream>
#include <numeric>

class Fraction
{
public:
    Fraction(int numérateur, int dénominateur) noexcept;
    double valeur_reelle() const noexcept;
    void simplifier();

private:
    int m_numérateur;
    int m_dénominateur;
};

Fraction::Fraction(int numérateur, int dénominateur) noexcept
    : m_numérateur(numérateur), m_dénominateur(dénominateur)
{
    assert(m_dénominateur != 0 && "Le dénominateur ne peut pas valoir 0.");
}

double Fraction::valeur_reelle() const noexcept
{
    return static_cast<double>(m_numérateur) / m_dénominateur;
}

void Fraction::simplifier()
{
    int const pgcd { std::gcd(m_numérateur, m_dénominateur) };
    m_numérateur /= pgcd;
    m_dénominateur /= pgcd;
}

Fraction pow(Fraction const & fraction, int puissance)
{
    Fraction copie { fraction };
    copie.m_numérateur = std::pow(fraction.m_numérateur, puissance);
    copie.m_dénominateur = std::pow(fraction.m_dénominateur, puissance);
    return copie;
}

int main()
```

```

{
    Fraction const f1 { 5, 2 };
    Fraction const f2 { pow(f1, 3) };
    return 0;
}

```

```

[Visual Studio]
Erreur (active) E0265 membre "Fraction::m_numérateur" (déclaré à la
ligne 15) est inaccessible
Erreur (active) E0265 membre "Fraction::m_dénominateur" (déclaré à la
ligne 16) est inaccessible
Erreur (active) E0265 membre "Fraction::m_dénominateur" (déclaré à la
ligne 16) est inaccessible

```

```

-----

[GCC]
prog.cc: In function 'Fraction pow(const Fraction&, int)':
prog.cc:40:11: error: 'int Fraction::m_numérateur' is private within
this context
   40 |     copie.m_numérateur = std::pow(fraction.m_numérateur,
      |           ^~~~~~
      |           ^~~~~~
prog.cc:15:9: note: declared private here
   15 |     int m_numérateur;
      |           ^~~~~~
prog.cc:40:44: error: 'int Fraction::m_dénominateur' is private within
this context
   40 |     copie.m_numérateur = std::pow(fraction.m_numérateur,
      |           ^~~~~~
      |           ^~~~~~
prog.cc:15:9: note: declared private here
   15 |     int m_numérateur;
      |           ^~~~~~
prog.cc:41:11: error: 'int Fraction::m_dénominateur' is private within
this context
   41 |     copie.m_dénominateur = std::pow(fraction.m_dénominateur,
      |           ^~~~~~
      |           ^~~~~~
prog.cc:16:9: note: declared private here
   16 |     int m_dénominateur;
      |           ^~~~~~
prog.cc:41:46: error: 'int Fraction::m_dénominateur' is private within
this context
   41 |     copie.m_dénominateur = std::pow(fraction.m_dénominateur,
      |           ^~~~~~
      |           ^~~~~~
prog.cc:16:9: note: declared private here
   16 |     int m_dénominateur;
      |           ^~~~~~

```

```

-----

[Clang]
prog.cc:40:11: error: 'm_numérateur' is a private member of 'Fraction'

```

```

    copie.m_numérateur = std::pow(fraction.m_numérateur, puissance);
    ^
prog.cc:15:9: note: declared private here
    int m_numérateur;
    ^
prog.cc:40:44: error: 'm_numérateur' is a private member of 'Fraction'
    copie.m_numérateur = std::pow(fraction.m_numérateur, puissance);
                                   ^
prog.cc:15:9: note: declared private here
    int m_numérateur;
    ^
prog.cc:41:11: error: 'm_dénominateur' is a private member of 'Fraction'
    copie.m_dénominateur = std::pow(fraction.m_dénominateur, puissance);
                                   ^
prog.cc:16:9: note: declared private here
    int m_dénominateur;
    ^
prog.cc:41:46: error: 'm_dénominateur' is a private member of 'Fraction'
    copie.m_dénominateur = std::pow(fraction.m_dénominateur, puissance);
                                   ^
prog.cc:16:9: note: declared private here
    int m_dénominateur;
    ^

4 errors generated.

```

Pour résoudre ce problème, on peut partir sur plusieurs solutions :

- rendre les attributs publics ;
- ajouter des services `numérateur` et `dénominateur` qui retournent les attributs respectifs, ainsi que des services `changer_numérateur` et `changer_dénominateur` ;
- utiliser l'amitié.

Attributs publics

Concernant la première solution, autant pour `m_numérateur`, cela ne causerait pas de problème, autant pour `m_dénominateur`, nous prendrions le risque que l'invariant puisse être transgressé. Cette solution ne convient donc pas.

Accesseurs

La deuxième solution peut sembler bonne, à première vue. Seulement voilà, il faut pour cela ajouter quatre nouveaux services à notre classe : deux accesseurs ou *getters* [de l'anglais *to get*, obtenir], qui permettent d'obtenir un accès en lecture aux attributs `m_numérateur` et `m_dénominateur`, ainsi que deux *mutateurs* ou *setters* [de l'anglais *to set*, positionner], qui permettent de les modifier.

Déjà, on sait que plus on écrit de code, plus on augmente le risque d'introduire des bugs par inadvertance. Et plus une classe contient de services, plus elle sera difficilement maintenable et plus il faudra d'efforts pour s'assurer qu'elle est correcte.

De plus, en utilisant cette solution, on dévoile des détails d'implémentation, puisque chaque attribut fait l'objet d'un accesseur et d'un mutateur. On ne pense plus en termes de comportements, de services, mais de nouveau en simple agrégat de données.

Enfin, les services que l'on est amené à écrire n'ont pas toujours du sens. Prenons l'exemple d'une classe `Lampe`. On peut imaginer récupérer la puissance de la lampe, c'est un service qui peut être attendu. Par contre, que dire d'un mutateur qui permettrait de modifier sa puissance ? Soit le setter ne fait aucune vérification, auquel cas pourquoi ne pas rendre l'attribut public ? Soit on a des préconditions, mais qui auront vraisemblablement déjà été vérifiées dans le constructeur, ce qui veut dire duplication de code, risque plus grand de bugs, difficulté de maintenance, etc.

Parfois, cette solution n'est tout simplement pas envisageable. C'est le cas si on a plusieurs invariants à respecter. Si on rajoute à la classe `Fraction` l'invariant "La fraction doit toujours être sous sa forme simplifiée" alors le code suivant présente des problèmes.

```
Fraction f1 { 3, 2 };
// On obtient 4/2 qui se simplifie tout de suite en 2/1.
f1.setNumérateur(4);

Fraction f2 { 3, 2 };
// On obtient 3/3 qui se simplifie tout de suite en 1/1.
f2.setDénominateur(3);

Fraction f3 { 3, 2 };
// On obtient 2/3.
f3.setNumérateur(4);
f3.setDénominateur(3);

Fraction f4 { 3, 2 };
// On obtient au final une fraction valant 4/1.
f4.setDénominateur(3);
f4.setNumérateur(4);

// Le seul moyen possible de passer à 4/3.
f1 = Fraction { 4, 3 };
```

La troisième, c'est la bonne

Il nous reste donc la troisième solution, l'amitié. Dire d'une fonction qu'elle est amie de la nôtre signifie qu'elle a un droit d'accès privilégié à ladite classe, tant en lecture qu'en écriture. Elle fait partie intégrante de l'interface, des services offerts par cette classe.

L'amitié se traduit en C++ avec le mot clé `friend`, qui s'écrit devant le prototype, que l'on place à l'intérieur de la classe et non plus à l'extérieur, comme pour une fonction libre non-amie. Ensuite, il n'y a plus qu'à implémenter la fonction de la même manière qu'auparavant.

```
class Fraction
{
public:
    // pow est maintenant amie de Fraction.
    friend Fraction pow(Fraction const & fraction, int puissance);
};

// Aucun problème maintenant.
Fraction pow(Fraction const & fraction, int puissance)
{
    Fraction copie { fraction };
    copie.numerateur = std::pow(fraction.numerateur, puissance);
    copie.denominateur = std::pow(fraction.denominateur, puissance);
    return copie;
}
```

Une précision s'impose : les fonctions amies ne sont pas des fonctions externes venues squatter notre classe. Bien au contraire, elles sont les services que rend la classe, au même titre que les fonctions membres. Elles font donc partie de ce qu'on a défini comme l'interface de la classe, tout en gardant l'aspect syntaxique d'une fonction libre.

Bien entendu, avec de grands pouvoirs viennent de grandes responsabilités. Nous avons certes un accès privilégié à la classe, mais cela implique que nous devons respecter ses invariants, comme n'importe quelle autre fonction membre.

25.6. Entraînez-vous !

Un autre constructeur pour *Fraction*

On pourrait très bien vouloir instancier une fraction à partir d'une chaîne de caractères telle que "35/8". Il suffit d'écrire un autre constructeur. Demandons-nous simplement quels pourraient être les cas d'erreurs possibles :

- pas de numérateur ;
- pas de dénominateur ;
- pas de séparateur ou caractère invalide ;
- trop de séparateur ;
- dénominateur qui vaut 0.

```
int main()
{
    // Doit compiler.
    Fraction f1 { "4/2" };
    assert(f1.valeur_reelle() == 2 && "La valeur réelle de f1 doit être 2.");

    // Doit générer une erreur.
    // Fraction f2 { "4\2" };

    // Doit générer une erreur.
    // Fraction f3 { "4//2" };

    // Doit générer une erreur.
    // Fraction f4 { "4/" };

    // Doit générer une erreur.
    // Fraction f5 { "/2" };

    // Doit générer une erreur.
    // Fraction f6 { "4/0" };

    return 0;
}
```

✧ Voir la solution

Une pile

Une *pile* est une structure de données qui fonctionne sur le principe du "dernier arrivé, premier parti". On peut lui ajouter ou supprimer des éléments, mais chaque fois c'est le

dernier élément qui est modifié. Par exemple, on ne peut pas directement supprimer le premier élément ; il faut d'abord supprimer tous les autres.

On peut modéliser l'ordre des appels à des fonctions en C++ avec une pile. D'abord, il y aura `main`, puis une autre fonction appelée dans `main`, puis une autre, etc. Lorsqu'une fonction a fini son exécution, on la retire de la pile et on revient à la fonction précédente, jusqu'à ce que la pile soit vide, quand le programme est terminé.

Voici une liste des opérations que nous allons implémenter :

- ajouter un élément ;
- retirer le dernier élément ;
- obtenir un accès au premier élément (qu'on appelle le sommet) ;
- obtenir le nombre total d'éléments ;
- déterminer si la pile est vide ou non.

Avec ces spécifications, nous pouvons maintenant définir un jeu de tests, qui nous servira à déterminer si notre code est correct et fonctionnel.

```
int main()
{
    Pile pile {};
    assert(pile.est_vide() && "La pile est vide.");
    assert(pile.taille() == 0 && "Au début, la pile n'a aucun élément.");

    pile.ajouter(5);
    pile.ajouter(10);
    pile.ajouter(15);
    assert(pile.taille() == 3 && "Maintenant, la pile a trois éléments.");

    int const premier { pile.sommet() };
    assert(premier == 5 && "Le premier élément vaut 5.");
    assert(pile.taille() == 3 && "La pile a toujours trois éléments.");

    pile.supprimer();
    assert(pile.taille() == 2 && "Maintenant, la pile n'a plus que deux éléments.");
    assert(!pile.est_vide() && "La pile n'est pas vide.");

    pile.supprimer();
    pile.supprimer();
    assert(pile.est_vide() && "Maintenant la pile est vide.");

    return 0;
}
```

✧ [Voir la solution](#)

25.7. Récapitulons !

- Les classes peuvent posséder des invariants et des contrats, qui doivent être respectés.
- Grâce à l'encapsulation et aux différentes visibilité, il est possible de n'exposer que les services que l'on veut, tout en protégeant les invariants de la classe.
- Une convention C++ répandue incite à préfixer les attributs privés avec `m_`.
- Le constructeur permet d'initialiser l'objet, tout en définissant quels seront les invariants de la classe.
- On peut écrire plusieurs constructeurs différents, de manière à permettre plusieurs façons d'initialiser l'objet.
- Si aucun constructeur personnalisé n'est défini, le compilateur implémente un constructeur par défaut.
- On peut initialiser directement des attributs, ce qui permet dans certains cas de se passer du constructeur par défaut.
- On peut aussi demander explicitement au compilateur de supprimer le constructeur par défaut.
- Le mot clé `explicit` évite les conversions implicites qui peuvent nuire à la compréhension et à la lisibilité du code.
- L'amitié permet à des fonctions libres d'accéder aux parties privées d'une classe de la même manière que les fonctions membres.
- Une fonction libre amie d'une classe fait partie intégrante de son interface.

SOLUTIONS

section Un autre constructeur pour *Fraction*

Voici une façon de faire parmi d'autres. Notez qu'on n'utilise pas la liste d'initialisation, car le code est plus complexe qu'une simple affectation et qu'il y a un peu de travail avant d'extraire le numérateur et le dénominateur. Comme certaines fonctions de la bibliothèque standard n'ont pas la garantie `noexcept` (comme `std::begin`) et que nous n'essayons pas de rattraper une potentielle exception, nous ne pouvons pas garantir que notre constructeur est `noexcept`.

```
Fraction::Fraction(std::string const & fraction)
{
    // Vérification que la chaîne n'est pas vide.
    assert(!std::empty(fraction) && "La chaîne ne peut pas être vide.");

    // Vérification de la présence du séparateur /.
    auto debut = std::begin(fraction);
    auto fin = std::end(fraction);
    assert(std::count(debut, fin, '/') == 1 && "Il ne doit y avoir qu'un seul séparateur '/'.");
    auto itereur = std::find(debut, fin, '/');

    // Récupération du numérateur.
    std::string const n { debut, debut + itereur };
    assert(!std::empty(n) && "Le numérateur doit être renseigné.");
    m_numérateur = std::stoi(n);

    // Pour sauter le caractère /.
    ++itereur;
    // Récupération du dénominateur.
    std::string const d { itereur, fin };
    assert(!std::empty(d) && "Le dénominateur doit être renseigné.");
    m_dénominateur = std::stoi(d);
    assert(m_dénominateur != 0 && "Le dénominateur ne peut pas être nul.");
}
```

section Une pile

En soi, le code n'est pas très compliqué car nous nous basons entièrement sur `std::vector`. Nous nous contentons juste de rajouter des contraintes, celles inhérentes aux piles. Sachez que les piles existent déjà en C++, avec `std::stack`. Donc, à l'avenir, quand vous utiliserez des piles, ne reprenez pas cet exercice, utilisez la bibliothèque standard.

Pile.hpp

```
#ifndef PILE_HPP
```

SOLUTIONS

```
#define PILE_HPP
#include <vector>
class Pile
{
public:
    void ajouter(int valeur);
    void supprimer();
    // Pas de noexcept car en interne, std::vector ne fournit pas la
    // garantie que récupérer un élément ne lancera pas d'exception.
    int sommet() const;
    bool est_vide() const noexcept;
    std::size_t taille() const noexcept;

private:
    std::vector<int> m_pile;
};
#endif
```

Pile.cpp

```
#include <cassert>
#include "Pile.hpp"
void Pile::ajouter(int valeur)
{
    m_pile.push_back(valeur);
}

void Pile::supprimer()
{
    assert(m_pile.size() > 0 && "Impossible de supprimer un élément d'une pile vide.");
    m_pile.pop_back();
}

int Pile::sommet() const
{
    assert(m_pile.size() > 0 && "Impossible de récupérer le sommet d'une pile vide.");
    return m_pile.back();
}

bool Pile::est_vide() const noexcept
{
    return m_pile.empty();
}

std::size_t Pile::taille() const noexcept
{
    return m_pile.size();
}
```

26

Une classe de grande valeur

Dans le chapitre sur les opérateurs, nous avons dit, sans plus détailler, que ce n'était pas parce que l'on pouvait surcharger des opérateurs qu'on devait forcément le faire. En effet, ça n'a de sens que pour les classes à *sémantique de valeur*. Et nous allons voir que ce n'est pas la seule implication qu'a cette sémantique.

26.1. Une histoire de sémantique

Depuis le début de ce livre, nous avons manipulé de nombreux types, comme des entiers, des réels, des chaînes de caractères, des tableaux, etc. Mais vous êtes-vous déjà demandé pourquoi il y en avait autant ? Puisqu'au final le processeur ne travaille que sur des nombres, on pourrait se dire que c'est un peu inutile. Cependant, s'il existe tant de types différents, c'est parce qu'ils ont des *sémantiques différentes*.

La sémantique, c'est *l'étude du sens*. Étudier la sémantique d'un type, c'est donc regarder ce qui a du sens de faire et ce qui n'en a pas. Le type `int` est un type simple, avec une sémantique simple, celle de manipuler un entier. L'exemple suivant est donc parfaitement compilable et exécutable.

```
int const temperature { 37 };
int const poids { 70 };
auto const calcul_idiot { poids + temperature };
```

Pourtant, on sait bien qu'additionner un poids et une température n'a pas de sens. D'ailleurs, quelle unité est-on censé obtenir ? Le compilateur lui ne le sait pas. On lui donne deux entiers et on lui demande de les additionner, il n'y voit aucun problème. Alors que si l'on définit des types plus spécialisés, avec une sémantique plus précise, on empêche ce comportement.

```
Temperature const temperature { 37 };
Poids const poids { 70 };
// Erreur à la compilation.
auto const calcul_idiot { poids + temperature };
```

Définir une sémantique permet donc d'éviter des bugs, tout en rendant le code plus clair, puisque celui qui vous relit (quelqu'un d'autre, vous dans six mois, etc.) ne se pose pas

de question sur vos intentions ni sur le sens de votre code. Par exemple, quel code vous semble le plus clair, parmi les deux ci-dessous ?

```
std::tuple<int, int, int> date_jour()
{
}

auto const date { date_jour() };
std::cout << "Aujourd'hui, nous sommes le "
<< std::get<0>(date) << "/"
<< std::get<1>(date) << "/"
<< std::get<2>(date) << "\n";
```

VS

```
struct Date
{
    int jour;
    int mois;
    int annee;
}

Date date_jour()
{
}

auto const date { date_jour() };
std::cout << "Aujourd'hui, nous sommes le "
<< date.jour << "/"
<< date.mois << "/"
<< date.annee << "\n";
```

Dans le premier, il faut se souvenir constamment de l'ordre des éléments du tuple : est-ce que c'est jour, mois puis année, ou l'inverse ? Dans le deuxième, tout est transparent et clair, on sait directement qu'on manipule une date. Le code est donc meilleur.

Ce qu'il convient de retenir, c'est que définir la sémantique d'un type que l'on crée a des conséquences sur son utilisation, car certaines opérations auront du sens et d'autres non. Créer des objets, on sait faire. Maintenant, nous allons voir plusieurs sémantiques possibles et ce qu'elles vont impliquer dans la conception de nos classes : la sémantique de valeur, celle que nous allons voir dans ce chapitre ; la [sémantique d'entité](#) et la [sémantique de déplacement](#) que nous verrons plus loin, dans les chapitres éponymes.

26.2. La sémantique de valeur, c'est quoi ?

On dit d'une classe qu'elle a une sémantique de valeur si deux objets au contenu identique peuvent être considérés comme égaux. Par exemple, une classe modélisant une fraction est une classe à sémantique de valeur. Si deux fractions valent $\frac{1}{2}$ chacune, elles sont égales. On a bien deux objets en mémoire, mais leur valeur est égale et donc on considère que ces objets sont égaux.

Note > Si deux objets ayant la même valeur peuvent être considérés comme égaux, on est typiquement dans de la sémantique de valeur.

Quand c'est pertinent, on peut aussi additionner ou soustraire plusieurs valeurs entre elles, les multiplier, les diviser, etc. D'où l'intérêt de la surcharge d'opérateurs, qui nous permet d'écrire ça sous forme élégante.

Note > La surcharge d'opérateurs arithmétiques prend tout son sens dans le cas de la sémantique de valeur.

Prenons un autre exemple, une somme d'argent. C'est bien une valeur. Mais que 100 € soient sous forme de liquide, de chèque ou d'une ligne sur votre compte en banque, c'est toujours la même valeur. On peut ajouter une somme d'argent à une autre et dans ce cas on obtient une nouvelle valeur.

Note > Modifier une valeur revient à en créer une nouvelle.

26.3. Égalité

Commençons par le premier point caractéristique des valeurs, l'égalité. Derrière ce mot simple se cachent des concepts plus complexes, qu'on aborde notamment en théorie des langages. En C++, pour être en mesure de définir l'égalité, on doit respecter les conditions suivantes.

- Pour n'importe quel objet `a`, la comparaison `a == a` est toujours vraie. C'est ce qu'on appelle la *réflexivité*.
- Pour deux objets `a` et `b` de même type, si `a == b` alors `b == a`. La comparaison doit être *commutative* et *symétrique*.
- Pour des objets `a`, `b` et `c` de même type, si on a `a == b` et `b == c`, alors on doit avoir `a == c`. On appelle ça la *transitivité*.

Note > En mathématiques, on parle de relation d'équivalence.

Au chapitre [Reprenez-vous un peu de sucre syntaxique ?](#), nous avons vu les [opérateurs de comparaison](#) `==`, `!=`, `>`, `<`, `>=` et `<=`. Nous allons maintenant à nouveau nous pencher sur eux.

Libre ou membre ?

Nous avons appris à les écrire sous forme libre, mais ces opérateurs existent aussi sous la forme de fonctions membres. C'est ce qu'on apprend en lisant [la documentation](#).

```
class Classe
{
    bool operator==(Classe a);
};

Classe a {};
Classe b {};
a == b; // Appel de == membre.

class Classe
{
};

bool operator==(Classe a, Classe b);

Classe a {};
Classe b {};
a == b; // Appel de == libre.
```

Entre les deux formes, il y a une grande différence. Dans le premier cas, l'appel `a == b` va être remplacé par `a.operator==(b)`. Cela signifie que le premier paramètre sera toujours du même type que la classe. Dans le deuxième cas, ce n'est pas obligatoire, car il peut y avoir une conversion implicite réalisée.

Imaginons que nous voulions comparer une fraction avec un entier. Cette opération a du sens, puisqu'un entier est un cas particulier de fraction, avec le dénominateur valant 1. Avec la forme membre, l'instruction `entier == fraction` ne compile pas, car le premier argument doit obligatoirement être une instance de `Fraction`.

```
#include <cassert>
#include <iostream>
#include <numeric>

class Fraction
{
public:
    // Constructeur de conversion car un seul paramètre obligatoire.
    Fraction(int numérateur, int dénominateur = 1) noexcept;

    void simplifier();

    // Version membre.
    bool operator==(Fraction const & rhs) const noexcept;
```

```

private:
    int m_numérateur { 0 };
    int m_dénominateur { 1 };
};

Fraction::Fraction(int numérateur, int dénominateur) noexcept
    : m_numérateur(numérateur), m_dénominateur(dénominateur)
{
    assert(dénominateur != 0
        && "Le dénominateur ne peut pas valoir 0.");
    simplifier();
}

// Implémentation de la version membre.
bool Fraction::operator==(Fraction const & rhs) const noexcept
{
    return m_numérateur == rhs.m_numérateur
        && m_dénominateur == rhs.m_dénominateur;
}

void Fraction::simplifier()
{
    int const pgcd { std::gcd(m_numérateur, m_dénominateur) };
    m_numérateur /= pgcd;
    m_dénominateur /= pgcd;
}

int main()
{
    Fraction const fraction { 4, 2 };
    int const entier { 2 };

    if (entier == fraction)
    {
        std::cout << "4/2 est égal à 2.\n";
    }
    else
    {
        std::cout << "4/2 est différent de 2.\n";
    }

    return 0;
}

```

```
[Visual Studio]
Erreur (active) E0349 aucun opérateur "==" ne correspond à ces opérandes
Erreur C2677 '==' binaire : aucun opérateur global trouvé qui accepte
le type
    'const Fraction' (ou il n'existe aucune conversion acceptable)

-----

[GCC]
main.cpp: In function 'int main()':
main.cpp:34:16: error: no match for 'operator==' (operand types are
'const int' and 'const Fraction')
34 |         if (entier == fraction)
    |             ~~~~~ ^~ ~~~~~
    |             |         |
    |             const int const Fraction

-----

[Clang]
prog.cc:34:16: error: invalid operands to binary expression ('const int'
and 'const Fraction')
    if (entier == fraction)
        ~~~~~ ^ ~~~~~
1 error generated.
```

Avec la version libre, le problème ne se pose plus, puisqu'il y a conversion implicite de l'entier en fraction. Notez au passage l'utilisation de l'[amitié](#) pour surcharger l'opérateur `==` libre.

```
#include <cassert>
#include <iostream>
#include <numeric>

class Fraction
{
public:
    // Constructeur de conversion car un seul paramètre obligatoire.
    Fraction(int numérateur, int dénominateur = 1) noexcept;

    void simplifier();

    // Version libre amie.
    friend bool operator==(Fraction const & lhs, Fraction const & rhs);

private:
    int m_numérateur { 0 };
    int m_dénominateur { 1 };
};

Fraction::Fraction(int numérateur, int dénominateur) noexcept
    : m_numérateur(numérateur), m_dénominateur(dénominateur)
{
}
```



```

    assert(denominateur != 0 && "Le dénominateur ne peut pas valoir 0.");
    simplifier();
}

bool operator==(Fraction const & lhs, Fraction const & rhs)
{
    return lhs.m_numérateur == rhs.m_numérateur && lhs.m_dénominateur ==
           rhs.m_dénominateur;
}

void Fraction::simplifier()
{
    int const pgcd { std::gcd(m_numérateur, m_dénominateur) };
    m_numérateur /= pgcd;
    m_dénominateur /= pgcd;
}

int main()
{
    Fraction const fraction { 4, 2 };
    int const entier { 2 };

    if (entier == fraction)
    {
        std::cout << "4/2 est égal à 2.\n";
    }
    else
    {
        std::cout << "4/2 est différent de 2.\n";
    }

    return 0;
}

```

4/2 est égal à 2.



Quelle version dois-je utiliser ? Laquelle est la meilleure ?

Il faut en effet en choisir une des deux, car il est impossible d'utiliser les deux en même temps. En effet, en écrivant `fraction == autre_fraction`, le compilateur ne sait pas si vous cherchez à utiliser la version libre ou la version membre. Cela se voit dans les erreurs qu'on obtient si on tente malgré tout la compilation.

```

[Visual Studio]
Erreur (active) E0350 plusieurs opérateurs "==" correspondent à ces
opérandes
Erreur C2593 'operator ==' est ambigu

```

```

-----
[GCC]
main.cpp: In function 'int main()':
main.cpp:42:18: error: ambiguous overload for 'operator=='
(operand types are 'const Fraction' and 'const Fraction')
 42 |         if (fraction == autre_fraction)
    |             ^~ ^~~~~~
    |             |      |
    |             |      | const Fraction
    |             |      | const Fraction
main.cpp:27:6: note: candidate: 'bool Fraction::operator==(const
Fraction&) const'
 27 | bool Fraction::operator==(Fraction const & rhs) const noexcept
    |     ^~~~~~
main.cpp:32:6: note: candidate: 'bool operator==(const Fraction&, const
Fraction&)'
 32 | bool operator==(Fraction const & lhs, Fraction const & rhs)
    |     ^~~~~~
-----

[Clang]
prog.cc:42:18: error: use of overloaded operator '==' is ambiguous
(with operand types 'const Fraction' and 'const Fraction')
    if (fraction == autre_fraction)
        ^~ ^~~~~~
prog.cc:27:16: note: candidate function
bool Fraction::operator==(Fraction const & rhs) const noexcept
    ^
prog.cc:32:6: note: candidate function
bool operator==(Fraction const & lhs, Fraction const & rhs)
    ^
1 error generated.

```

La forme la plus pratique et la plus simple reste la forme libre, quitte à utiliser `friend` si besoin d'accéder à des attributs privés. La forme membre est permise, mais est plus limitative.

Soyons intelligents, réutilisons

Ce conseil vous a déjà été donné lors de la découverte de la surcharge d'opérateurs, mais il est bon de vous le rappeler. En effet, réutiliser le code permet de réduire le risque de bugs tout en le rendant plus léger. Nombreux développeurs prennent l'habitude de définir les opérateurs `==` et `<` puis de les utiliser pour définir les autres (voir [Section 15.3, La surcharge d'opérateurs](#)).

Dès que l'on définit un opérateur d'égalité ou de comparaison, il semble logique de définir les autres. Si vous fournissez `<` et `>`, l'utilisateur s'attend aussi à pouvoir utiliser `<=` et `>=`. D'où l'intérêt d'utiliser `<` pour définir les trois autres, ainsi que `==` pour définir `!=`.

26.4. Le retour des opérateurs

Lors de notre découverte de la [surcharge d'opérateurs](#), nous avons appris à implémenter l'addition, la soustraction, etc. Grâce à nos nouvelles connaissances sur les objets, nous allons en aborder d'autres.

***Note** > La surcharge d'opération n'est pas une caractéristique essentielle des classes à sémantique de valeur, plutôt un sous-ensemble qu'on rencontre souvent, mais pas systématiquement.*

Les opérateurs d'affectation intégrés

Sous ce terme se cachent les opérateurs `+=`, `-=`, `*=`, `/=` et `%=`, qui permettent les raccourcis du type `f1 += f2`, ou bien encore `f3 -= f4`. On les appelle en anglais les *compound assignment operator*. Notez que, comme pour des types natifs, ils modifient l'instance qui les appelle. La forme canonique voudra donc qu'on les écrive sous la forme membre. Voici à quoi l'opérateur `+=` de la classe `Fraction` doit ressembler. La forme est la même pour tous les autres, au signe près.

```
class Fraction
{
public:
    Fraction& operator+=(Fraction const & origine) noexcept;
};

Fraction& Fraction::operator+=(Fraction const & origine) noexcept
{
    // Implémentation de l'addition.
    return *this;
}
```

Normalement, l'instruction `return *this` devrait vous surprendre. En effet, nous ne l'avons encore jamais rencontrée. De plus, la fonction renvoie une référence. Or, [nous avons vu](#) que cela peut être dangereux dans le cas où celle-ci pointe sur un objet qui n'existe plus.

L'explication est simple. L'expression `return *this` permet de renvoyer l'objet courant. Sans cela, il serait impossible d'écrire quelque chose comme ce qui suit.

```
Fraction const f1 { 5, 4 };
Fraction f2 { 1, 1 };
Fraction f3 { 1, 1 };

// On additionne f1 à f2, puis on soustrait ce nouveau f2 à f3.
f3 -= f2 += f1;
```

Comme la ligne `f3 -= f2 += f1` revient à écrire `f3.operator-=(f2.operator+=(f1))`, si la fonction ne renvoyait rien, nous ne pourrions pas chaîner ainsi les modifications (bien que cet exemple soit très peu lisible). Il faut donc renvoyer l'objet après modification. Comme celui-ci est toujours valide, aucun risque à renvoyer une référence sur cet objet, ce qui est moins long qu'une recopie.

Cela nous permet d'avoir le même comportement pour notre objet que s'il s'agissait d'un type natif comme `int` (donc pas de surprise), tout en gardant une similitude avec le langage C.

L'implémentation en elle-même ne devrait pas vous surprendre, il ne s'agit que d'une simple addition de fractions.

```
Fraction& Fraction::operator+=(Fraction const & origine) noexcept
{
    m_numerateur = m_numerateur * origine.m_denominateur +
    origine.m_numerateur * m_denominateur;
    m_denominateur *= origine.m_denominateur;
    return *this;
}
```

Note > Vous pouvez vous entraîner en implémentant `-=`, `*=` et `/=`.

Du deux en un

Pour chaque opérateur d'affectation intégré, il existe l'équivalent binaire, qui prend deux arguments et renvoie une nouvelle valeur qui est le résultat. Ainsi, pour `+=`, il existe `+`; pour `-=` il existe `-`; etc. Logiquement, si l'on fournit l'une des deux formes, on s'attend à ce que l'autre soit disponible. Dans un esprit de réutilisation du code déjà écrit et validé, nous allons en implémenter un à l'aide de l'autre.

L'usage en C++, bien que ce ne soit pas une règle absolue, est d'écrire l'implémentation dans l'opérateur intégré, puis de définir l'opérateur libre associé en fonction du premier. Dans le cas de la classe `Fraction`, cela veut dire écrire la logique de l'addition dans `operator+=` puis d'appeler celui-ci dans `operator+`.

```
class Fraction
{
public:
    Fraction& operator+=(Fraction const & origine) noexcept;

private:
    int m_numerateur { 0 };
    int m_denominateur { 1 };
};
```

```

Fraction& Fraction::operator+=(Fraction const & origine) noexcept
{
    m_numerator = m_numerator * origine.m_denominateur +
    origine.m_numerator * m_denominateur;
    m_denominateur *= origine.m_denominateur;
    return *this;
}

// L'argument lhs est recopié, donc on peut le modifier sans problème.
Fraction operator+(Fraction lhs, Fraction const & rhs) noexcept
{
    lhs += rhs;
    return lhs;
}

```

On utilise le principe de la copie. Puisque le paramètre `lhs` est une copie, on peut le modifier en utilisant `+=` sans risque. On obtient un opérateur `+` correct et non redondant avec `+=`.

Note > Vous pouvez vous entraîner en implémentant `-`, `*` et `/`.

Les opérateurs de manipulation des flux

Grâce à eux, on peut afficher un objet sur la sortie standard, en créer un depuis un fichier, etc. En lisant [la documentation](#), on note que leur forme canonique est la suivante.

```

std::ostream& operator<<(std::ostream & os, T const & obj)
{
    // Écriture sur le flux de sortie.
    return os;
}

std::istream& operator>>(std::istream & is, T & obj)
{
    // Lecture depuis le flux d'entrée.
    if ( /* Erreur, impossible de construire T. */ )
    {
        is.setstate(std::ios::failbit);
    }
    return is;
}

```

Ces opérateurs s'écrivent toujours sous la forme libre car leur premier argument est toujours un flux. Ils sont souvent *friend*, car ils doivent accéder aux attributs de l'instance. Enfin, notez qu'on signale que l'entrée est invalide en mettant le flux dans un état invalide avec `std::ios::failbit`, ce qui permet à l'utilisateur de vérifier le flux en écrivant `if (std::cin.fail())`, comme nous l'avons appris lors de l'écriture de la fonction de saisie sécurisée (voir [Section 11.7](#), [\[Pratique\] Gérer les erreurs d'entrée • Partie V](#)).

L'exemple suivant montre une implémentation possible pour la classe `Fraction`. Notez que nous avons déplacé dans une nouvelle fonction l'extraction des données depuis une chaîne de caractères. Cela permet de ne pas dupliquer la logique et le code entre le constructeur et `operator>>`.

`Fraction.hpp`

```
#ifndef FRACTION_HPP
#define FRACTION_HPP

#include <istream>
#include <ostream>
#include <string>

class Fraction
{
public:
    Fraction(int numérateur, int dénominateur);
    Fraction(std::string const & chaîne);

    friend std::ostream& operator<<(std::ostream & os, Fraction const & fraction);
    friend std::istream& operator>>(std::istream & is, Fraction & fraction);

private:
    int m_numérateur { 1 };
    int m_dénominateur { 1 };
};

#endif
```

`Fraction.cpp`

```
#include <algorithm>
#include <cassert>
#include <exception>
#include "Fraction.hpp"

/**
 * @brief Extrait le numérateur et le dénominateur d'une chaîne, si possible.
 * @param[in] chaîne La chaîne de caractères à analyser.
 * @param[out] numérateur Le numérateur extrait.
 * @param[out] dénominateur Le dénominateur extrait.
 * @exception std::runtime_error si la chaîne n'est pas correcte.
 */
void extraction_depuis_chaine(std::string const & chaîne, int & numérateur, int & dénominateur)
{
    if (std::empty(chaîne))
    {
```

```

    throw std::runtime_error("La chaîne ne peut pas être vide.");
}

auto const debut = std::begin(chaine);
auto const fin = std::end(chaine);
if (std::count(debut, fin, '/') != 1)
{
    throw std::runtime_error("La chaîne ne peut contenir qu'un seul
séparateur '/'.");
}
auto itérateur = std::find(debut, fin, '/');

// Récupération du numérateur.
std::string const n { debut, debut + std::distance(debut,
itérateur) };
if (std::empty(n))
{
    throw std::runtime_error("Le numérateur ne peut pas être vide.");
}

// Pour sauter le caractère /.
++itérateur;
// Récupération du dénominateur.
std::string const d { itérateur, fin };
if (std::empty(d) || std::stoi(d) == 0)
{
    throw std::runtime_error("La dénominateur ne peut pas être vide ou
nul.");
}

// Mise à jour uniquement si tout s'est bien passé.
numérateur = std::stoi(n);
dénominateur = std::stoi(d);
}

Fraction::Fraction(int numérateur, int dénominateur)
: m_numérateur(numérateur), m_dénominateur(dénominateur)
{
    assert(dénominateur != 0 && "Le dénominateur doit toujours être
différent de 0.");
}

Fraction::Fraction(std::string const & origine)
{
    extraction_depuis_chaine(origine, m_numérateur, m_dénominateur);
}

// Rien de particulier, le code est simple.
std::ostream& operator<<(std::ostream & os, Fraction const & fraction)
{
    os << fraction.m_numérateur << "/" << fraction.m_dénominateur;
    return os;
}

// Ici, on a un peu plus de travail.
```

```

std::istream& operator>>(std::istream & is, Fraction & fraction)
{
    std::string entree {};
    is >> entree;

    try
    {
        int numerateur { 0 };
        int denominateur { 0 };
        extraction_depuis_chaine(entree, numerateur, denominateur);
        // Si on arrive ici, tout va bien, on peut créer le nouvelle objet.
        fraction = Fraction { numerateur, denominateur };
    }
    catch (std::exception const & e)
    {
        // On attrape n'importe quelle exception, que ce soit parce
        // que la chaîne est invalide ou parce que stoi a échoué
        // lors de la conversion.
        is.setstate(std::ios::failbit);
    }

    return is;
}

```

main.cpp

```

#include <iostream>
#include "Fraction.hpp"

template<typename T>
void entree_securisee(T & variable)
{
    while (!(std::cin >> variable))
    {
        if (std::cin.eof())
        {
            break;
        }
        else if (std::cin.fail())
        {
            std::cout << "Entrée invalide. Recommence." << std::endl;
            std::cin.clear();

            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        }
        else
        {
            std::cout << "Le prédicat n'est pas respecté !" << std::endl;
        }
    }
}

int main()
{

```



```

Fraction const f1 { "5/2" };
std::cout << f1 << "\n";

Fraction f2 {};
// Grâce à notre écriture de operator>> conforme aux normes, on peut
// réutiliser la fonction entree_securisee sans plus d'effort.
entree_securisee(f2);
std::cout << f2 << "\n";
return 0;
}

```

Quelques bonnes pratiques

Finissons cette section en détaillant trois bonnes pratiques que vous devez garder à l'esprit chaque fois que vous surchargerez des opérateurs. Comme d'habitude, ces principes n'ont rien d'obligatoire, mais ils rendront votre code de meilleure qualité.

D'abord, respectez toujours la sémantique d'un opérateur. Si vous utilisez l'opérateur `+`, ne calculez pas une division. C++ vous y autorise, mais les utilisateurs de votre code risquent fort de ne pas apprécier. Si un opérateur a une sémantique déjà définie pour un domaine, tenez-vous-y. Que votre `+` signifie toujours `+` !

Ensuite, si la signification d'un opérateur n'est pas claire et indiscutable, vous ne devriez pas l'utiliser. Par exemple, bien qu'on puisse surcharger l'opérateur `*`, ne vous en servez pas pour retourner l'inverse d'une fraction, cela surprendrait tout le monde car personne ne s'attend à ce que cet opérateur fasse ça. Si la signification de l'opérateur n'est pas clairement comprise et largement acceptée pour l'opération que vous voulez réaliser, mieux vaut utiliser une fonction bien nommée à la place.

***Note** > Le choix de `<<` et `>>` pour les flux ne respecte pas ce principe, car cet opérateur n'a pas la même sémantique ici que celle qu'il a en C.*

Enfin, certains opérateurs viennent par groupe et si vous implémentez un opérateur d'une catégorie, implémentez aussi tous les autres de cette catégorie. On l'a déjà dit, mais si vous implémentez `<`, les utilisateurs s'attendent à pouvoir utiliser aussi `>`, `<=` et `>=`. Si vous définissez `+=`, on peut vraisemblablement s'attendre à pouvoir utiliser aussi `+`, etc.

Les autres opérateurs

Faire le tour de tous les opérateurs serait long et peu instructif. En effet, maintenant que nous avons vu plus en détail comment procéder, ainsi que les formes canoniques, le principe est le même pour tous les autres. Vous serez donc capables de vous débrouiller seul avec les opérateurs que nous n'avons pas abordés.

Dans ce livre, vous trouverez également les opérateurs `()` et `[]`. Ceux qui restent (à part `++` et `--`) sont assez rarement surchargés.

L'opérateur `()` est par exemple utilisé avec les *foncteurs*, dont nous avons parlé en découvrant les conteneurs (voir [section Trier une liste dans l'ordre décroissant](#)). Lorsqu'on passe un foncteur à un algorithme, celui-ci se contente de faire appel à son opérateur `()` pour en déduire si le prédicat est valide ou non.

L'opérateur `[]` vous est familier, car il est présent dans plusieurs conteneurs comme `std::array`, `std::string` ou `std::vector`, pour accéder directement à un élément précis.

26.5. Copier des objets

On peut créer autant de duplicatas du même objet : s'il a une sémantique de valeur, ils seront tous égaux. On peut copier, par exemple, notre fraction 1/2 dix fois, à la fin il ne reste que la même valeur. Mais justement, comment on copie les objets en C++ ?

Constructeur de copie

C'est ce constructeur qui est appelé quand on crée une nouvelle instance à partir d'une instance déjà existante. C'est lui que nous avons utilisé, sans le savoir, en écrivant [la fonction libre pow](#).

```
Fraction copie { fraction };
```

Par défaut, le compilateur en génère un si on ne lui indique pas de ne pas le faire. Ce constructeur s'occupe comme un grand de copier chaque attribut, en faisant appel à son constructeur de copie s'il en définit un. Dans notre cas, nos attributs sont de simples entiers, donc la copie est très simple.

Le compilateur fait très bien son travail, donc pas besoin d'écrire nous-mêmes le constructeur de copie. On peut cependant utiliser le mot clé `default` si l'on veut être plus explicite.

```
class Fraction
{
public:
    Fraction(Fraction const & origine) = default;
};
```

Il ne prend qu'un argument, l'instance à copier, sous forme de référence constante.

Opérateur d'affectation par copie

Ce que nous venons de voir fonctionne lors de l'instanciation d'une classe, puisque c'est à ce moment précis que le constructeur est appelé. Or, il se peut que l'on veuille affecter une nouvelle valeur à notre objet en dupliquant celle d'un autre, alors que celui-là est déjà créé. Pour cela, nous avons besoin de l'opérateur d'affectation par copie.

```
Fraction const fraction { 3, 4 };
Fraction autre_fraction { 1, 1 };

// Du code.

autre_fraction = fraction;
```

Comme pour le constructeur de copie, son implémentation est fournie implicitement par le compilateur. On peut là encore choisir de la rendre explicite.

```
class Fraction
{
public:
    Fraction & operator=(Fraction const & origine) = default;
};
```

À l'instar des opérateurs d'affectation intégrés, cet opérateur renvoie une référence sur l'objet courant, ce qui permet de chaîner les affectations, comme le montre l'exemple ci-dessous.

```
Fraction const f1 { 87, 9 };
Fraction f2 { 1, 1 };
Fraction f3 { 1, 1 };

f3 = f2 = f1;
```

26.6. Récapitulons !

- Si deux objets, ayant la même valeur, peuvent être considérés comme égaux, on est typiquement dans de la sémantique de valeur.
- Modifier une valeur revient à en créer une nouvelle.
- L'égalité de deux objets est définie en C++ par des règles précises.
- On peut utiliser la surcharge d'opérateurs pour faciliter la manipulation de nos objets à sémantique de valeur.

- En réutilisant intelligemment certains opérateurs, on peut écrire le reste plus simplement et de façon plus concise.
- La copie, ou duplication, fait aussi partie des services qu'on attend de classes à sémantique de valeur.
- Le constructeur de copie et l'opérateur d'affection par copie peuvent être générés automatiquement par le compilateur, soit implicitement en n'écrivant rien, soit explicitement en utilisant `default`.

27

[Pratique] Entrons dans la matrice

Les objets mathématiques se prêtent bien à la sémantique de valeur. Alors, pour ne pas déroger à la tradition, nous allons implémenter un nouveau concept, les matrices. Si vous ne connaissez pas, aucun problème, nous allons aborder rapidement le sujet. Sinon, vous n'avez plus qu'à coder.

Ce chapitre a pour but de vous faire pratiquer toutes les notions vues précédemment.

27.1. Qu'est-ce qu'une matrice ?

Note > Ce livre n'a pas vocation à être un livre de mathématiques, donc les explications qui suivent seront succinctes.

On peut définir une matrice de manière simple comme étant un tableau de nombres à n lignes et m colonnes. Ces nombres sont appelés dimension de la matrice. Ci-dessous, vous avez un exemple de matrice de dimension 2×3 .

$$\begin{pmatrix} 1 & 2 & 3 \\ 3 & 4 & 6 \end{pmatrix}$$

Si une matrice n'est composée que d'une ligne, on parle de vecteur ligne. De même, s'il n'y a qu'une colonne, on parle de vecteur colonne.

Comme pour beaucoup d'objets mathématiques, il est possible d'appliquer un certain nombre d'opérations élémentaires, comme l'addition ou la multiplication. Il existe aussi quelques opérations plus spécifiques aux matrices, comme la transposition.

Addition

L'addition de deux matrices se fait terme à terme. Les deux matrices additionnées doivent être de même dimension.

$$\begin{pmatrix} 1 & 2 & 3 \\ 3 & 4 & 6 \end{pmatrix} + \begin{pmatrix} 0 & 4 & 9 \\ 4 & -7 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 6 & 12 \\ 7 & -3 & 7 \end{pmatrix}$$

Le même principe s'applique à la soustraction.

Multiplication par un nombre

Multiplier une matrice par un nombre revient à multiplier chaque terme par ce nombre.

$$2 \times \begin{pmatrix} 1 & 2 & 3 \\ 3 & 4 & 6 \end{pmatrix} = \begin{pmatrix} 2 & 4 & 6 \\ 6 & 8 & 12 \end{pmatrix}$$

Transposition

Transposer une matrice revient à échanger les lignes et les colonnes.

$$\begin{pmatrix} 1 & 2 & 3 \\ 3 & 4 & 6 \end{pmatrix} \Rightarrow \begin{pmatrix} 1 & 3 \\ 2 & 4 \\ 3 & 6 \end{pmatrix}$$

Multiplication de deux matrices

Pour définir la multiplication dans le cas général, intéressons-nous d'abord à la multiplication d'un vecteur colonne par un vecteur ligne, qu'on appelle produit scalaire. Pour deux matrices de même dimension, calculer leur produit scalaire revient à faire l'opération suivante.

$$(x_1 \ x_2 \ \dots \ x_n) \cdot \begin{pmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{pmatrix} = x_1 y_1 + x_2 y_2 + \dots + x_n y_n = \sum_{i=1}^n x_i y_i$$

De manière plus générale, la multiplication d'une matrice $A(n \times m)$ par une matrice $A(m \times p)$ donne une matrice $A(c \times p)$ dont chaque élément C_{ij} est le produit scalaire de la ligne i de la matrice A avec la colonne j de la matrice B . L'exemple ci-dessous va vous aider à saisir le principe.

$$\begin{array}{rcl} A \times B = C & \begin{pmatrix} 0 & 4 \\ 4 & -7 \\ 9 & 1 \end{pmatrix} B & \\ \begin{matrix} A \\ \begin{pmatrix} 1 & 2 & 3 \\ 3 & 4 & 6 \end{pmatrix} \end{matrix} & = & \begin{pmatrix} 1 \times 0 + 2 \times 4 + 3 \times 9 & 1 \times 4 + 2 \times -7 + 3 \times 1 \\ 3 \times 0 + 4 \times 4 + 6 \times 9 & 3 \times 4 + 4 \times -7 + 6 \times 1 \end{pmatrix} C \\ & = & \begin{pmatrix} 35 & -7 \\ 70 & -10 \end{pmatrix} C \end{array}$$

Les matrices sont un vaste sujet et il y aurait encore beaucoup à dire. Vous pouvez continuer votre découverte, si le cœur vous en dit, sur de nombreux sites, à commencer par la [page Wikipédia](#). Quant à nous, nous nous contenterons de ces notions.

27.2. Cahier des charges

Comme vous l'avez compris, le but est de pouvoir manipuler des matrices en C++. Mais avant de vous lancer corps et âme dans la réalisation, prenons le temps de réfléchir à la conception de notre programme, notamment aux services que l'on souhaite offrir.

Les services

Le premier est bien évidemment la création d'une matrice, de dimension quelconque, avec éventuellement une valeur par défaut pour chacun de ses éléments. De plus, on doit pouvoir la copier, car une matrice a une sémantique de valeur.

Ensuite, on aimerait pouvoir accéder à un élément de la matrice, tant en lecture qu'en écriture, de la même manière que cela est possible avec `std::string` ou `std::vector`.

Puisqu'on a des additions et des multiplications à effectuer, il semble évident que nous allons utiliser la surcharge d'opérateurs. Vous devez maintenant avoir l'habitude. Néanmoins, n'oubliez pas que l'on veut proposer la multiplication tant par un entier que par une autre matrice.

Enfin, en vrac, proposons de récupérer la dimension de la matrice, de l'afficher et puis d'extraire une ligne ou une colonne spécifique, pour en faire respectivement un vecteur ligne et un vecteur colonne.

Les détails d'implémentation

Maintenant que nous avons bien fixé nos services, plongeons-nous dans des détails d'implémentation. La première question qui se pose est *comment stocker les différents nombres* qui composent une matrice, en sachant que nous ne connaissons pas sa dimension à l'avance.

S'il semble clair que `std::vector` est taillé pour le job, cela ne répond pas à la question de savoir comment procéder. Un `std::vector` dont chaque élément est lui-même un `std::vector<int>` ? C'est une solution, mais il y a plus simple encore.

Il suffit d'utiliser un tableau, de taille `lignes*colonnes` tout simplement. Une petite astuce vous aidera à accéder directement au bon élément. Pour la comprendre, nous allons reprendre notre matrice d'introduction de taille 2×3 , soit une matrice de deux lignes par

trois colonnes, donc un tableau de 6 éléments. Si on stocke tous les éléments par ligne, alors l'élément $M_{0,0}$ correspond à l'indice 0, l'élément $M_{1,0}$ à l'indice 1, l'élément $M_{0,1}$ à l'indice 2, etc. Supposons que nous voulons l'élément $M_{1,1}$. Avec ce système, nous devons accéder à l'élément d'indice 3. Or, si on y prête attention, ce chiffre s'obtient en multipliant le nombre de lignes (qui vaut 2) par l'indice y voulu (soit 1), puis en rajoutant l'indice x voulu (soit 1). L'astuce se répète pour n'importe quel indice (x, y) , ce qui donne la formule suivante : $y \times \text{lignes} + x$.

L'autre question à laquelle il convient de répondre est *comment accéder à un élément de la matrice* ? Contrairement à `std::string` ou `std::vector`, nous ne pouvons pas utiliser l'opérateur `[]`, car celui-ci n'accepte qu'un seul argument, alors que nous devons en fournir deux, la ligne et la colonne. La solution s'appelle `operator()`, le même opérateur que pour les `foncteurs`. Ce dernier peut prendre autant d'arguments que l'on souhaite. On pourra donc écrire `matrice(0, 0)`, par exemple.

Les tests

Bien entendu, nous voulons nous assurer que notre code fonctionne bien. Nous avons vu quelques exemples mathématiques pour illustrer chacune des opérations, nous allons donc pouvoir nous baser sur les valeurs obtenues pour écrire quelques tests.

```
void test_addition()
{
    Matrice m1 { 2, 3 };
    m1(0, 0) = 1;
    m1(0, 1) = 2;
    m1(0, 2) = 3;
    m1(1, 0) = 3;
    m1(1, 1) = 4;
    m1(1, 2) = 6;

    Matrice m2 { 2, 3 };
    m2(0, 0) = 0;
    m2(0, 1) = 4;
    m2(0, 2) = 9;
    m2(1, 0) = 4;
    m2(1, 1) = -7;
    m2(1, 2) = 1;

    Matrice resultat { 2, 3 };
    resultat(0, 0) = 1;
    resultat(0, 1) = 6;
    resultat(0, 2) = 12;
    resultat(1, 0) = 7;
    resultat(1, 1) = -3;
    resultat(1, 2) = 7;

    Matrice const addition { m1 + m2 };
```



```

assert(addition.nb_lignes() == resultat.nb_lignes()
    && "L'addition n'est pas correcte.");
assert(addition.nb_colonnes() == resultat.nb_colonnes()
    && "L'addition n'est pas correcte.");

for (int i { 0 }; i < addition.nb_lignes(); ++i)
{
    for (int j { 0 }; j < addition.nb_colonnes(); ++j)
    {
        assert(resultat(i, j) == addition(i, j)
            && "L'addition n'est pas correcte.");
    }
}

void test_multiplication_entier()
{
    Matrice m1 { 2, 3 };
    m1(0, 0) = 1;
    m1(0, 1) = 2;
    m1(0, 2) = 3;
    m1(1, 0) = 3;
    m1(1, 1) = 4;
    m1(1, 2) = 6;

    Matrice resultat { 2, 3 };
    resultat(0, 0) = 2;
    resultat(0, 1) = 4;
    resultat(0, 2) = 6;
    resultat(1, 0) = 6;
    resultat(1, 1) = 8;
    resultat(1, 2) = 12;

    Matrice const multiplication { m1 * 2 };

    assert(resultat.nb_lignes() == multiplication.nb_lignes()
        && "La multiplication par un entier n'est pas correcte.");
    assert(resultat.nb_colonnes() == multiplication.nb_colonnes()
        && "La multiplication par un entier n'est pas correcte.");

    for (int i { 0 }; i < multiplication.nb_lignes(); ++i)
    {
        for (int j { 0 }; j < multiplication.nb_colonnes(); ++j)
        {
            assert(resultat(i, j) == multiplication(i, j)
                && "La multiplication par un entier n'est pas correcte.");
        }
    }
}

void test_multiplication_matrice()
{
    Matrice m1 { 2, 3 };
    m1(0, 0) = 1;
    m1(0, 1) = 2;

```

```

m1(0, 2) = 3;
m1(1, 0) = 3;
m1(1, 1) = 4;
m1(1, 2) = 6;

Matrice m2 { 3, 2 };
m2(0, 0) = 0;
m2(0, 1) = 4;
m2(1, 0) = 4;
m2(1, 1) = -7;
m2(2, 0) = 9;
m2(2, 1) = 1;

Matrice resultat { 2, 2 };
resultat(0, 0) = 35;
resultat(0, 1) = -7;
resultat(1, 0) = 70;
resultat(1, 1) = -10;

Matrice const multiplication { m1 * m2 };

assert(resultat.nb_lignes() == multiplication.nb_lignes()
    && "La multiplication par une matrice n'est pas correcte.");
assert(resultat.nb_colonnes() == multiplication.nb_colonnes()
    && "La multiplication par une matrice n'est pas correcte.");

for (int i { 0 }; i < multiplication.nb_lignes(); ++i)
{
    for (int j { 0 }; j < multiplication.nb_colonnes(); ++j)
    {
        assert(resultat(i, j) == multiplication(i, j)
            && "La multiplication par une matrice n'est pas correcte.");
    }
}

void test_transposition()
{
    Matrice m1 { 2, 3 };
    m1(0, 0) = 1;
    m1(0, 1) = 2;
    m1(0, 2) = 3;
    m1(1, 0) = 3;
    m1(1, 1) = 4;
    m1(1, 2) = 6;

    Matrice resultat { 3, 2 };
    resultat(0, 0) = 1;
    resultat(0, 1) = 3;
    resultat(1, 0) = 2;
    resultat(1, 1) = 4;
    resultat(2, 0) = 3;
    resultat(2, 1) = 6;

    Matrice const transposee { m1.transpose() };

```

```

assert(resultat.nb_lignes() == transposee.nb_lignes()
    && "La transposition n'est pas correcte.");
assert(resultat.nb_colonnes() == transposee.nb_colonnes()
    && "La transposition n'est pas correcte.");

for (int i { 0 }; i < transposee.nb_lignes(); ++i)
{
    for (int j { 0 }; j < transposee.nb_colonnes(); ++j)
    {
        assert(resultat(i, j) == transposee(i, j)
            && "La transposition n'est pas correcte.");
    }
}

void test_affichage()
{
    Matrice m1 { 2, 2 };
    m1(0, 0) = 1;
    m1(0, 1) = 2;
    m1(1, 0) = 3;
    m1(1, 1) = 3;

    Matrice m2 { 2, 2 };
    m2(0, 0) = 0;
    m2(0, 1) = 4;
    m2(1, 0) = 4;
    m2(1, 1) = -7;

    std::cout << m1 << "\n";
    std::cout << m1 + m2 << "\n";
    std::cout << m1 + m2 + m2 << "\n";
}

int main()
{
    test_addition();
    test_multiplication_entier();
    test_multiplication_matrice();
    test_transposition();
    test_affichage();

    return 0;
}

```

27.3. Réalisation

Voyons maintenant en détail comment mettre cela en œuvre.

Les services

Constructeurs

Commençons par réfléchir à la façon dont nous voulons créer des matrices. Il paraît évident de renseigner le nombre de lignes et de colonnes quand on crée une matrice. On peut vouloir aussi initialiser toute la matrice à une valeur précise.

De plus, comme nous sommes en plein dans la sémantique de valeur, on aimerait pouvoir copier une matrice, ce qui signifie que nous avons besoin d'un constructeur de copie et un opérateur d'affectation par copie. On en déduit donc le code suivant.

```
Matrice(std::size_t nb_lignes, std::size_t nb_colonnes, int
    valeur_initiale = 0);
// Constructeur de copie.
Matrice(Matrice const & copie) = default;
// Opérateur d'affectation par copie
Matrice& operator=(Matrice const & autre_matrice) = default;
```

Accès aux éléments d'une matrice

De manière similaire à `std::vector` ou `std::string`, il faut qu'on puisse accéder à un élément de la matrice, tant en lecture qu'en écriture. On doit donc penser à gérer une matrice constante. Puisqu'on sait qu'on doit utiliser l'opérateur `()`, on obtient les services suivants.

```
// Version d'accès en lecture, marche avec une matrice const.
int const & operator()(std::size_t x, std::size_t y) const;
// Version d'accès en écriture.
int& operator()(std::size_t x, std::size_t y);
```

Surcharges des opérateurs d'addition et de multiplication

On a vu en introduction qu'on pouvait additionner deux matrices de même dimension, multiplier une matrice par un entier ou par une autre matrice. On en déduit donc plusieurs surcharges d'opérateurs, en n'oubliant pas bien sûr leurs versions libres.

```

// Additionne une matrice à l'objet courant.
Matrice& operator+=(Matrice const & autre_matrice);
// Opérateur d'addition.
friend Matrice operator+(Matrice lhs, Matrice const & rhs);

// Multiplie la matrice courante par un entier.
Matrice& operator*=(int multiplicateur);
// Opérateur de multiplication par un entier.
friend Matrice operator*(Matrice autre_matrice, int multiplicateur);

// Multiplie l'objet courant par une autre matrice.
Matrice& operator*=(Matrice const & autre_matrice);
// Opérateur de multiplication de deux matrices.
friend Matrice operator*(Matrice lhs, Matrice const & rhs);

```

Autres fonctions utiles

Terminons la définition de notre classe `Matrice` par quelques services supplémentaires qu'elle peut être amenée à fournir. Nous avons déjà la fonction permettant de faire une transposition, qui apparaît dans le cahier des charges. Nous avons personnellement choisi de faire une fonction membre retournant une copie de l'objet courant, après transposition, mais d'autres pourraient considérer que `transposition` s'applique à l'objet courant. C'est une question de choix.

```

Matrice transposition() const;

```

On peut aussi se dire qu'il serait bien d'accéder à une colonne ou une ligne précise de la matrice, les fameux vecteurs lignes et vecteurs colonnes. Cela ajoute deux nouvelles fonctions membres à la liste de nos services.

```

// Pour obtenir un vecteur ligne.
Matrice ligne(std::size_t index_ligne) const;
// Pour obtenir un vecteur colonne.
Matrice colonne(std::size_t index_colonne) const;

```

Enfin, il paraît important de connaître la dimension de notre matrice, ce qui nous amène aux deux dernières fonctions membres à définir.

```

// Connaître le nombre de lignes de la matrice courante.
std::size_t nb_lignes() const noexcept;
// Connaître le nombre de colonnes de la matrice courante.
std::size_t nb_colonnes() const noexcept;

```

Les détails d'implémentation

Fichiers d'en-tête standards

Notre fichier d'en-tête `Matrice.hpp` utilise trois composants de la bibliothèque standard, à savoir `assert`, `std::vector` et `std::ostream`. Il faut donc inclure les fichiers d'en-tête correspondants pour que le nôtre soit valide. On s'assure ainsi qu'il est *auto-suffisant*, qu'il n'y a pas besoin d'ajouts externes ni de manipulations supplémentaires pour pouvoir l'utiliser.

Les fichiers `<cassert>` et `<vector>` ne devraient pas vous surprendre. Par contre, `<iosfwd>` est nouveau. On l'utilise à la place de `<iostream>` quand on a simplement besoin d'un type I/O comme `std::ostream`, `std::istream`, `std::stringstream`, etc., sans son implémentation complète. Ce fichier d'en-tête est donc très léger. À l'opposé, `<iostream>` est très lourd car il contient l'implémentation complète de ces (nombreux) types.

Constructeurs

En plus des lignes et des colonnes, il faut un tableau dans lequel stocker les nombres de la matrice. Comme nous ne savons pas à l'avance sa taille, un `std::vector` est tout désigné pour ça. Et comme [un de ses constructeurs](#) permet d'initialiser tous les éléments à une certaine valeur, on peut déjà écrire le nôtre.

```
Matrice::Matrice(std::size_t nb_lignes, std::size_t colonnes, int
    valeur_initiale)
: m_nb_lignes(nb_lignes), m_nb_colonnes(colonnes),
  m_tableau_interne(nb_lignes * colonnes, valeur_initiale)
{
    assert(m_nb_lignes > 0 && m_nb_colonnes > 0
        && "On ne peut pas avoir une matrice de dimension 0.");
}
```

Notez qu'on utilise ici une assertion pour s'assurer qu'on fournit bien des dimensions correctes. Une matrice de dimension 0 ou moins n'ayant pas de sens, ce serait une erreur de programmation que de l'accepter.

Le constructeur de copie et l'opérateur d'affectation par copie seront très bien gérés par le compilateur, donc on se contente de les définir avec `default`.

Accès aux éléments d'une matrice

L'accès à un élément d'une matrice n'est pas très compliqué à écrire. Nous avons choisi de passer par une fonction privée de manière à ne pas dupliquer la vérification des

contrats s'assurant qu'on ne demande pas à accéder à un élément inexistant de la matrice.

```
std::size_t Matrice::offset(std::size_t ligne, std::size_t
    colonne) const noexcept
{
    assert(ligne < m_nb_lignes && "Ligne demandée invalide.");
    assert(colonne < m_nb_colonnes && "Colonne demandée invalide.");
    return colonne * m_nb_lignes + ligne;
}

// Accès en lecture seule.
int const & Matrice::operator()(std::size_t x, std::size_t y) const
{
    return m_tableau_interne[offset(x, y)];
}

// Accès en lecture et écriture.
int& Matrice::operator()(std::size_t x, std::size_t y)
{
    return m_tableau_interne[offset(x, y)];
}
```

Surcharge des opérateurs d'addition et de multiplication

Attaquons maintenant un gros morceau. Pour commencer doucement, parlons de l'addition. On rappelle que celle-ci ne peut se faire que si les deux matrices ont la même dimension, d'où le `assert` pour s'en assurer. L'addition en elle-même est faite avec l'algorithme `std::transform`, que nous connaissons déjà et qu'on utilise pour écrire dans notre tableau interne le résultat de l'addition.

```
Matrice& Matrice::operator+=(Matrice const& autre_matrice)
{
    assert(nb_lignes() == autre_matrice.nb_lignes()
        && nb_colonnes() == autre_matrice.nb_colonnes()
        && "Impossible d'additionner deux matrices de dimensions
différentes.");

    std::transform(std::cbegin(m_tableau_interne),
        std::cend(m_tableau_interne),
        std::cbegin(autre_matrice.m_tableau_interne),
        std::begin(m_tableau_interne),
        [](int lhs, int rhs) -> int { return lhs + rhs; });

    return *this;
}
```

La version libre de l'addition est elle aussi très simple, puisqu'elle se contente de réutiliser `operator+=`.

```
Matrice operator+(Matrice lhs, Matrice const & rhs)
{
    lhs += rhs;
    return lhs;
}
```

Passons maintenant à la multiplication par un entier. Là encore, l'algorithme est simple puisqu'il suffit de multiplier chaque élément de la matrice par l'entier en question. Il n'y a pas de précaution ni de vérification particulière à faire. Un seul rappel, il faut capturer le paramètre `multiplicateur` dans la zone de capture de la lambda.

```
Matrice& Matrice::operator*=(int multiplicateur)
    Matrice& Matrice::operator*=(int multiplicateur)
{
    std::transform(std::cbegin(m_tableau_interne),
        std::cend(m_tableau_interne),
        std::begin(m_tableau_interne),
        [multiplicateur](int element) -> int { return element *
multiplicateur; });
    return *this;
}
```

Et la version libre est également très simple.

```
Matrice operator*(Matrice matrice, int multiplicateur)
{
    matrice *= multiplicateur;
    return matrice;
}
```

Maintenant, augmentons la difficulté et parlons de la multiplication de deux matrices. On doit commencer par vérifier que la précondition sur la dimension des matrices multipliées est respectée. Ensuite, on applique l'algorithme, à savoir multiplier chaque ligne de la matrice actuelle par chaque colonne de la matrice passée en paramètre. Sauf que voilà, si on modifie la matrice courante, cela va fausser les résultats puisque des éléments auront été modifiés en cours de route puis réutilisés dans les calculs suivants. On doit donc *utiliser une copie*. C'est justement ce que fait l'opérateur libre de multiplication. Cela veut dire que, de manière exceptionnelle, nous allons écrire l'opérateur intégré `*=` en fonction de l'opérateur `*`.

Sur cette copie, on va pouvoir faire la multiplication de chaque vecteur ligne par chaque vecteur colonne, qu'on peut obtenir avec les fonctions membres `lignes` et `colonnes`, que l'on définira juste après.

Sachant que cette multiplication revient à faire une somme de produits et qu'un algorithme comme celui-ci est assez commun, commençons par regarder dans la bibliothèque standard. Ça tombe bien, la fonction `std::inner_product` sert justement à ça. C'est ainsi qu'on peut obtenir le résultat de la multiplication d'un vecteur ligne par un vecteur colonne.

```
const int valeur { std::inner_product(std::begin(ligne_courante),
    std::end(ligne_courante),
    std::begin(colonne_courante),
    0) };

copie(i, j) = valeur;
```

On obtient au final le code suivant pour l'opérateur libre.

```
Matrice operator*(Matrice const & lhs, Matrice const & rhs)
{
    assert(lhs.nb_colonnes() == rhs.nb_lignes()
        && "Le nombre de colonnes de la matrice multipliée doit être égal au
        nombre de lignes de la matrice multipliant.");

    Matrice copie { lhs.nb_lignes(), rhs.nb_colonnes() };
    for (std::size_t i { 0 }; i < copie.nb_lignes(); ++i)
    {
        auto const ligne_courante { lhs.ligne(i).m_tableau_interne };
        for (std::size_t j { 0 }; j < copie.nb_colonnes(); ++j)
        {
            auto const colonne_courante { rhs.colonne(j).m_tableau_interne };
            const int valeur {
                std::inner_product(
                    std::begin(ligne_courante),
                    std::end(ligne_courante),
                    std::begin(colonne_courante),
                    0
                )
            };
            copie(i, j) = valeur;
        }
    }

    return copie;
}
```

Reste à écrire l'opérateur intégré. Comment mettre à jour notre objet avec le nouveau résultat ? En l'échangeant. Nous pourrions le faire nous-mêmes mais, encore une fois, la bibliothèque standard fournit ce qu'il faut, avec `std::swap`.

```
Matrice& Matrice::operator*=(Matrice const & autre_matrice)
{
```

```

Matrice copie { *this * autre_matrice };
// On échange l'objet courant avec la copie.
std::swap(*this, copie);
// L'objet courant est à jour et contient bien le résultat attendu, on
// peut donc le renvoyer.
return *this;
}

```

Autres fonctions utiles

Pour que la surcharge de l'opérateur `*` puisse fonctionner, nous devons définir les fonctions membres `lignes` et `colonnes`. Celles-ci sont simples et ne consistent qu'en un simple tour de boucle.

```

Matrice Matrice::ligne(std::size_t index_ligne) const
{
    assert(index_ligne < nb_lignes()
        && "L'index demandé pour la ligne est plus grand que la dimension de
        la matrice.");

    Matrice matrice_ligne { 1, nb_colonnes() };
    for (std::size_t j { 0 }; j < nb_colonnes(); ++j)
    {
        matrice_ligne(0, j) = (*this)(index_ligne, j);
    }
    return matrice_ligne;
}

Matrice Matrice::colonne(std::size_t index_colonne) const
{
    assert(index_colonne < nb_colonnes()
        && "L'index demandé pour la colonne est plus grand que la dimension
        de la matrice.");

    Matrice matrice_colonne { nb_lignes(), 1 };
    for (std::size_t i { 0 }; i < nb_lignes(); ++i)
    {
        matrice_colonne(i, 0) = (*this)(i, index_colonne);
    }
    return matrice_colonne;
}

```

Ensuite, la transposition ne devrait pas vous causer trop de problème, puisqu'il s'agit simplement d'inverser les indices de lignes et de colonnes.

```

Matrice Matrice::transposition() const
{
    Matrice transposee { nb_colonnes(), nb_lignes() };
    for (std::size_t i { 0 }; i < nb_lignes(); ++i)
    {
        for (std::size_t j { 0 }; j < nb_colonnes(); ++j)

```

```

        {
            transposee(j, i) = (*this)(i, j);
        }
    }

    return transposee;
}

```

Enfin, il reste l’affichage, qui nous permettra de visualiser les résultats. Dans une classe plus complète, destinée à être utilisée par d’autres, nous ne l’écririons pas, car le formatage et l’affichage des données dépend beaucoup du contexte et des besoins de l’utilisateur, donc nous ne voudrions pas le forcer à afficher ses matrices d’une certaine façon. Pour cet exercice, ce n’est pas un problème.

```

std::ostream& operator<<(std::ostream & stream, Matrice const & matrice)
{
    for (std::size_t i { 0 }; i < matrice.nb_lignes(); ++i)
    {
        for (std::size_t j { 0 }; j < matrice.nb_colonnes(); ++j)
        {
            stream << matrice(i, j) << " ";
        }
        stream << "\n";
    }
    return stream;
}

```

27.4. Code complet

Voici le code que nous avons écrit et qui reflète notre analyse du cahier des charges et notre réflexion.

Matrice.hpp

```

#ifndef MATRICE_HPP
#define MATRICE_HPP

#include <iosfwd>
#include <vector>

/**
 * @brief Manipulation de matrices.
 */
class Matrice
{
public:
    /**
     * @brief Créé une nouvelle instance de Matrice.
     * @param[in] nb_lignes Le nombre de lignes de la matrice à créer.
     */

```

```

    * @param[in] nb_colonnes Le nombre de colonnes de la matrice à
    créer.
    * @pre Le nombre de lignes et de colonnes doit être strictement
    supérieur à 0.
    * @param[in] valeur_initiale La valeur à donner à chaque élément de
    la matrice.
    */
    Matrice(std::size_t nb_lignes, std::size_t nb_colonnes, int
    valeur_initiale = 0);

    /**
    * @brief Constructeur de copie.
    * @param[in] copie La matrice à copier.
    */
    Matrice(Matrice const & copie) = default;

    /**
    * @brief Opérateur d'affectation par copie.
    * @param[in] autre_matrice La matrice à copier.
    * @return La matrice actuelle après copie.
    */
    Matrice& operator=(Matrice const & autre_matrice) = default;

    /**
    * @brief Opérateur d'accès en lecture seule à un élément de la
    matrice.
    * @param[in] x L'indice de la ligne voulue.
    * @param[in] y L'indice de la colonne voulue.
    * @pre Les indices de ligne et de colonne doivent être strictement
    inférieurs à la dimension de la matrice.
    * @return Une référence constante sur l'élément trouvé.
    */
    int const & operator()(std::size_t x, std::size_t y) const;

    /**
    * @brief Opérateur d'accès en lecture/écriture à un élément de la
    matrice.
    * @param[in] x L'indice de la ligne voulue.
    * @param[in] y L'indice de la colonne voulue.
    * @pre Les indices de ligne et de colonne doivent être strictement
    inférieurs à la dimension de la matrice.
    * @return Une référence sur l'élément trouvé.
    */
    int& operator()(std::size_t x, std::size_t y);

    /**
    * @brief Additionne une matrice à la matrice courante.
    * @param autre_matrice La matrice à additionner à la matrice
    courante.
    * @pre La matrice passée en paramètre doit être de même dimension
    que la matrice courante.
    * @post La nouvelle valeur est égale à l'ancienne plus `rhs`
    * @return Matrice& L'objet modifié après addition.
    */
    Matrice& operator+=(Matrice const & autre_matrice);

```

```

/**
 * @brief Opérateur d'addition de deux matrices.
 * @param[in] lhs La matrice de gauche.
 * @param[in] rhs La matrice de droite.
 * @pre Les deux matrices doivent être de même dimension.
 * @return Le résultat de l'addition des deux matrices.
 */
friend Matrice operator+(Matrice lhs, Matrice const & rhs);

/**
 * @brief Multiplie la matrice courante par un entier.
 * @param[in] multiplicateur Un entier, par lequel la matrice est
multipliée.
 * @return L'objet modifié après multiplication.
 */
Matrice& operator*=(int multiplicateur);

/**
 * @brief Opérateur de multiplication d'une matrice par un entier.
 * @param[in] m La matrice à multiplier.
 * @param[in] multiplicateur L'entier multiplicateur.
 * @return Le résultat de la multiplication d'une matrice par un
entier.
 */
friend Matrice operator*(Matrice m, int multiplicateur);

/**
 * @brief Opérateur de multiplication d'une matrice par un entier.
 * @param[in] multiplicateur L'entier multiplicateur.
 * @param[in] m La matrice à multiplier.
 * @return Le résultat de la multiplication d'une matrice par un
entier.
 */
friend Matrice operator*(int multiplicateur, Matrice m);

/**
 * @brief Multiplie la matrice courante par une autre matrice.
 * @param[in] autre_matrice La matrice par laquelle la matrice
courante sera multipliée.
 * @return L'objet modifié après multiplication.
 */
Matrice& operator*=(Matrice const & autre_matrice);

/**
 * @brief Opérateur de multiplications de matrices.
 * @param[in] lhs La matrice de gauche.
 * @param[in] rhs La matrice de droite.
 * @return Le résultat de la multiplication d'une matrice par une
autre.
 */
friend Matrice operator*(Matrice const & lhs, Matrice const & rhs);

/**
 * @brief Affiche une matrice sur un flux de sortie.

```

```

    * @param[in,out] stream Le flux sur lequel sera affichée la matrice.
    * @param[in] matrice La matrice à afficher.
    * @return Le flux de sortie utilisé.
    */
    friend std::ostream& operator<<(std::ostream & stream, Matrice const &
matrice);

    /**
    * @brief Retourne un vecteur ligne, selon l'indice demandé.
    * @param[in] index_ligne L'index de la ligne désirée.
    * @pre L'index de la ligne désirée doit être strictement inférieur
au nombre de lignes de la matrice.
    * @return Un vecteur ligne de type Matrice, correspondant à la ligne
demandée.
    */
    Matrice ligne(std::size_t index_ligne) const;

    /**
    * @brief Retourne un vecteur colonne, selon l'indice demandé.
    * @param[in] index_colonne L'index de la colonne désirée.
    * @pre L'index de la colonne désirée doit être strictement inférieur
au nombre de colonnes de la matrice.
    * @return Un vecteur colonne de type Matrice, correspondant à la
colonne demandée.
    */
    Matrice colonne(std::size_t index_colonne) const;

    /**
    * @brief Obtient la transposée de la matrice courante.
    * @return La matrice courante une fois transposée.
    */
    Matrice transpose() const;

    /**
    * @brief Donne le nombre de lignes de la matrice actuelle.
    * @return Le nombre de lignes de la matrice actuelle.
    */
    std::size_t nb_lignes() const noexcept;

    /**
    * @brief Donne le nombre de colonnes de la matrice actuelle.
    * @return Le nombre de colonnes de la matrice actuelle.
    */
    std::size_t nb_colonnes() const noexcept;

private:
    /**
    * Permet de calculer la position dans le tableau interne de
l'élément qui se trouve à la ligne et à la colonne demandées.
    * @param[in] ligne La ligne demandée.
    * @param[in] colonne La colonne demandée.
    * @pre La ligne et la colonne doivent être inférieures à la
dimension de la matrice.
    * @returns L'index dans le tableau auquel se trouve l'élément.
    */

```

```

    std::size_t offset(std::size_t ligne, std::size_t colonne) const
    noexcept;

    std::size_t m_nb_lignes;
    std::size_t m_nb_colonnes;
    std::vector<int> m_tableau_interne;
};

#endif

```

Matrice.cpp

```

#include "Matrice.hpp"

#include <algorithm>
#include <cassert>
#include <ostream>
#include <numeric>

Matrice::Matrice(std::size_t nb_lignes, std::size_t nb_colonnes, int
    valeur_initiale)
: m_nb_lignes(nb_lignes), m_nb_colonnes(nb_colonnes),
  m_tableau_interne(nb_lignes * nb_colonnes, valeur_initiale)
{
    assert(m_nb_lignes > 0 && m_nb_colonnes > 0 && "On ne peut pas avoir
    une matrice de dimension 0.");
}

std::size_t Matrice::offset(std::size_t ligne, std::size_t
    colonne) const noexcept
{
    assert(ligne < m_nb_lignes && "Ligne demandée invalide.");
    assert(colonne < m_nb_colonnes && "Colonne demandée invalide.");
    return colonne * m_nb_lignes + ligne;
}

int const & Matrice::operator()(std::size_t x, std::size_t y) const
{
    return m_tableau_interne[offset(x, y)];
}

int& Matrice::operator()(std::size_t x, std::size_t y)
{
    return m_tableau_interne[offset(x, y)];
}

Matrice& Matrice::operator+=(Matrice const& autre_matrice)
{
    assert(nb_lignes() == autre_matrice.nb_lignes()
        && nb_colonnes() == autre_matrice.nb_colonnes()
        && "Impossible d'additionner deux matrices de dimensions
        différentes.");
}

```

```

        std::transform(std::cbegin(m_tableau_interne),
            std::cend(m_tableau_interne),
            std::cbegin(autre_matrice.m_tableau_interne),
            std::begin(m_tableau_interne),
            [](int lhs, int rhs) -> int { return lhs + rhs; });

        return *this;
    }

    Matrice operator+(Matrice lhs, Matrice const & rhs)
    {
        lhs += rhs;
        return lhs;
    }

    Matrice& Matrice::operator*=(int multiplicateur)
    {
        std::transform(std::cbegin(m_tableau_interne),
            std::cend(m_tableau_interne),
            std::begin(m_tableau_interne),
            [multiplicateur](int element) -> int { return element *
                multiplicateur; });
        return *this;
    }

    Matrice operator*(Matrice matrice, int multiplicateur)
    {
        matrice *= multiplicateur;
        return matrice;
    }

    Matrice operator*(int multiplicateur, Matrice autre_matrice)
    {
        return autre_matrice * multiplicateur;
    }

    Matrice& Matrice::operator*=(Matrice const & autre_matrice)
    {
        Matrice copie { *this * autre_matrice };
        std::swap(*this, copie);
        return *this;
    }

    Matrice operator*(Matrice const & lhs, Matrice const & rhs)
    {
        assert(lhs.nb_colonnes() == rhs.nb_lignes()
            && "Le nombre de colonnes de la matrice multipliée doit être égal au
            nombre de lignes de la matrice multipliant.");

        Matrice copie { lhs.nb_lignes(), rhs.nb_colonnes() };
        for (std::size_t i { 0 }; i < copie.nb_lignes(); ++i)
        {
            auto const ligne_courante { lhs.ligne(i).m_tableau_interne };
            for (std::size_t j { 0 }; j < copie.nb_colonnes(); ++j)
            {

```



```

        auto const colonne_courante { rhs.colonne(j).m_tableau_interne };
        const int valeur {
            std::inner_product(
                std::begin(ligne_courante),
                std::end(ligne_courante),
                std::begin(colonne_courante),
                0
            )
        };
        copie(i, j) = valeur;
    }
}

return copie;
}

std::ostream& operator<<(std::ostream & stream, Matrice const & matrice)
{
    for (std::size_t i { 0 }; i < matrice.nb_lignes(); ++i)
    {
        for (std::size_t j { 0 }; j < matrice.nb_colonnes(); ++j)
        {
            stream << matrice(i, j) << " ";
        }
        stream << "\n";
    }
    return stream;
}

Matrice Matrice::ligne(std::size_t index_ligne) const
{
    assert(index_ligne < nb_lignes()
        && "L'index demandé pour la ligne est plus grand que la dimension de la matrice.");

    Matrice matrice_ligne { 1, nb_colonnes() };
    for (std::size_t j { 0 }; j < nb_colonnes(); ++j)
    {
        matrice_ligne(0, j) = (*this)(index_ligne, j);
    }
    return matrice_ligne;
}

Matrice Matrice::colonne(std::size_t index_colonne) const
{
    assert(index_colonne < nb_colonnes()
        && "L'index demandé pour la colonne est plus grand que la dimension de la matrice.");

    Matrice matrice_colonne { nb_lignes(), 1 };
    for (std::size_t i { 0 }; i < nb_lignes(); ++i)
    {
        matrice_colonne(i, 0) = (*this)(i, index_colonne);
    }
    return matrice_colonne;
}

```

```

}

Matrice Matrice::transpose() const
{
    Matrice transposee { nb_colonnes(), nb_lignes() };
    for (std::size_t i { 0 }; i < nb_lignes(); ++i)
    {
        for (std::size_t j { 0 }; j < nb_colonnes(); ++j)
        {
            transposee(j, i) = (*this)(i, j);
        }
    }

    return transposee;
}

std::size_t Matrice::nb_lignes() const noexcept
{
    return m_nb_lignes;
}

std::size_t Matrice::nb_colonnes() const noexcept
{
    return m_nb_colonnes;
}

```

27.5. Entraînez-vous !

Travailler sur les matrices vous a mis l'eau à la bouche ? Vous voulez d'autres idées pour pratiquer la sémantique de valeur ? Les exercices suivants vont vous plaire.

Entiers infinis

Note > L'exercice qui suit est un classique, vous le retrouverez dans différents langages et sous différents énoncés. Merci à [Matthieu Schaller](#) de nous avoir inspiré cette version !

Comme vous le savez, les types comme `int` ou `double` ont une taille maximale, que vous pouvez retrouver dans l'en-tête `<limits>`. Et si jamais on veut manipuler des entiers encore plus grands ? Il faut coder.

Le but de cet exercice est de pouvoir manipuler des entiers très très grands, comme cela est possible en Python ou en JavaScript. Pour cela, nous allons utiliser des chaînes de caractères. Mais plutôt que de les manipuler nous-mêmes, nous allons encapsuler tout ça dans une belle classe, qui nous fournira des services simples à utiliser, comme l'addition, la soustraction, etc.

Le code suivant vous servira de base pour vérifier les quatre opérations élémentaires.

```
#include <iostream>

int main()
{
    Nombre a { 420 };
    Nombre b { "48879554321858363547485554545557454555445" };

    // Doit afficher 48879554321858363547485554545557454555865.
    std::cout << a + b << std::endl;

    // Doit afficher 48879554321858363547485554545557454555025.
    std::cout << a - b << std::endl;

    // Doit afficher 20529412815180512689943932909134130913286900.
    std::cout << a * b << std::endl;

    // Doit afficher 116379891242519913208298939394184415608.
    std::cout << a / b << std::endl;

    return 0;
}
```

Des polynômes

Continuons dans notre voyage mathématique en parlant maintenant des polynômes, ces outils d'algèbre qu'on commence à découvrir (en France) au lycée et qui ont la forme suivante.

$$P(x) = \sum_{i=0}^n a_i x^i = a_0 + a_1 x + a_2 x^2 + \dots + a_n x^n, n \geq 0$$

De même que pour des réels ou des entiers, il est possible de leur appliquer plusieurs opérations, dont les quatre élémentaires. Le but de l'exercice est donc de créer une classe encapsulant ces différents services.

***Note** > Si vous ne connaissez pas les polynômes ou n'êtes pas très à l'aise en mathématiques, ne vous en faites pas. Vous pouvez vous contenter d'implémenter l'addition et la soustraction, qui sont les opérations les plus simples.*

Voici un exemple d'utilisation, pour vous inspirer.

```
#include <iostream>

int main()
{
    Polynome p1 { "3x^2+x+7" };
    Polynome p2 { "4x^3-3x^2+x+1" };

    // Doit afficher 4x^3+2x+8.
    std::cout << p1 + p2 << std::endl;
    // Doit afficher -4x^3+6x^2+6.
    std::cout << p1 - p2 << std::endl;
    // Doit afficher 12x^5-5x^4+28x^3-17x^2+8x+7.
    std::cout << p1 * p2 << std::endl;

    return 0;
}
```

27.6. Récapitulons !

- Nous avons pu pratiquer toutes les notions vues précédemment.
- Nous avons revu l'importance de chercher au préalable si une fonction ou un algorithme n'existe pas déjà dans la bibliothèque standard.
- Nous avons pu voir l'importance de d'abord réfléchir aux services que propose une classe et seulement après de penser aux détails d'implémentation.

28

Classes à sémantique d'entité

Précédemment, nous avons appris à créer des objets respectant la sémantique de valeur. Nous allons voir maintenant une autre sémantique majeure du C++ qu'on appelle la *sémantique d'entité*. Elle est à contre-pied de la sémantique de valeur, mais comme cette dernière permet de se simplifier la vie et d'éviter des erreurs. Pour cela, nous allons voir ce qu'elle signifie, les conséquences sur le code, dans quels cas l'utiliser et les réflexes à avoir.

28.1. Des objets uniques !

Dire qu'une classe est à sémantique d'entité veut dire que les instances de cette classe sont des entités.

Une entité est un objet tel qu'on le conçoit dans la vie réelle. Deux entités ont beau avoir exactement les mêmes caractéristiques, elles ne sont pas les mêmes. Par exemple, si vous avez un stylo identique à celui de votre voisin, cela reste bien deux stylos distincts. En termes de programmation, si on implémente une classe `Stylo`, deux instances dans le même état (c'est-à-dire ayant exactement les mêmes attributs, comme la couleur ou le niveau d'encre) seront chacune uniques et ne représenteront en aucun cas le même objet.

En particulier, chaque instance va vivre sa vie indépendamment. C'est logique, la quantité d'encre dans votre stylo peut changer, ça n'altérera pas le stylo du voisin. La sémantique d'entité est donc radicalement différente de la sémantique de valeur, où deux instances identiques représentent forcément la même valeur.

Conséquences sur l'interface

Quand on écrit une classe à sémantique d'entité, il y a trois choses à prendre en compte :

- on ne surcharge plus les opérateurs arithmétiques (`+`, `-`, etc.) ;
- on ne surcharge plus les opérateurs de comparaison ;
- on n'utilise plus la copie.

La première règle est assez évidente. Ça n'a pas de sens de vouloir soustraire ou multiplier des chaises, des stylos ou des comptes en banque.

La deuxième provient de ce que chaque instance est unique. Dès lors, à quoi bon les comparer dans leur ensemble puisqu'elles ne seront jamais égales. En revanche, on peut très bien les comparer en utilisant certains attributs à sémantique de valeur. Par exemple, on peut comparer des comptes en banque suivant leur montant, des chaises en fonction de leur poids, etc.

La troisième règle est aussi liée à l'unicité de chaque instance. On ne peut pas créer une copie à partir d'un autre objet. On ne peut pas copier un personnage vers un autre, ou bien copier une lampe vers une autre. Cette raison est philosophique, mais nous allons voir en fin de chapitre que cela a des *conséquences pratiques* et que vouloir implémenter la copie d'entité est une source sans fin de bugs vicieux.

Pour être en mesure de désactiver la copie, on utilise le mot clé `delete` sur le constructeur de copie et l'opérateur d'affectation par copie, comme nous l'avons déjà vu pour le [constructeur par défaut](#). On se protège ainsi de la copie, qu'elle soit explicite (`Entite b { a };`) ou implicite (appel à `push_back` par exemple).

```
class NonCopiable
{
public:
    NonCopiable(NonCopiable const & copie) = delete;
    NonCopiable& operator=(NonCopiable const & copie) = delete;
};
```

La visibilité n'a dans ce cas aucune importance. Que le constructeur et l'opérateur de copie soient `public` ou `private`, la copie est désactivée dans les deux cas.

Un exemple

Supposons que vous écriviez un programme qui émule le fonctionnement d'une entreprise. Un besoin possible est de représenter les personnes qui y travaillent. Des personnes humaines sont un cas évident de sémantique d'entité. Un début de classe pourrait donc ressembler à cela.

```
#include <cassert>
#include <iostream>
#include <string>

class Personne
{
public:
```

```

    Personne() = delete;
    Personne(std::string const& nom, std::string const& prenom, int
salaire);

    Personne(Personne const& copie) = delete;
    Personne& operator=(Personne const& copie) = delete;

    void travailler() const;

private:
    std::string m_nom;
    std::string m_prenom;
    int m_salaire;
};

Personne::Personne(std::string const& nom, std::string const&
prenom, int salaire)
    : m_nom(nom), m_prenom(prenom), m_salaire(salaire)
{
    assert(!std::empty(m_nom) && "Le nom de famille est obligatoire.");
    assert(!std::empty(m_prenom) && "Le prénom est obligatoire.");
    assert(m_salaire >= 0 && "C'est vous qui payez la personne, pas
l'inverse !");
}

void Personne::travailler() const
{
    std::cout << "Je suis " << m_prenom << " "
        << m_nom << " et j'effectue différentes tâches.\n";
}

using namespace std::literals;

int main()
{
    Personne const patron { "Ron"s, "Pat"s, 2500 };
    patron.travailler();

    // Pauvre Bob...
    Personne const stagiaire { "Le Stagiaire"s, "Bob"s, 0 };
    stagiaire.travailler();
    return 0;
}

```

```

Je suis Pat Ron et j'effectue différentes tâches.
Je suis Bob Le Stagiaire et j'effectue différentes tâches.

```

Cet exemple de personne et d'entreprise va servir de fil rouge pour ce chapitre et illustrera très bien ce à quoi servent les entités et comment les manipuler.

Note > Les entités en C++ ne correspondent pas nécessairement à des entités de la vraie vie. Dans ce premier exemple, c'est le cas, mais nous verrons *plus bas* un cas où nos entités ne sont pas des objets réels.

28.2. Des sous-types...

Dans une entreprise, tout le monde n'occupe pas le même poste. Par conséquent, il y a plusieurs types de travailleurs. Bien qu'ils soient différents, ils n'en restent pas moins tous des personnes. Il serait intéressant de pouvoir représenter cette hiérarchie. C++ offre justement une façon de représenter une relation EST-UN et qu'on appelle *l'héritage*. Grâce à cette technique, nous pouvons créer des sous-types plus spécialisés de nos entités de base.

Exemple

Nous pourrions commencer en définissant un type `Directeur`, qui sera une spécialisation (un sous-type) de `Personne`.

```
class Personne
{
    //...
};

// Directeur hérite de Personne.
// Directeur est un sous-type de Personne.
class Directeur : public Personne
{
    //...
};
```

En plus des propriétés que le sous-type a en commun avec sa classe parent, on peut aussi lui *ajouter des services propres*. Dans le cas d'une chaise, s'asseoir est un service commun à tous les types de chaises, mais rouler devient un service spécifique aux chaises à roulettes, se balancer aux chaises à bascule, etc. L'exemple suivant montre par exemple que notre nouveau type `Directeur`, en plus de travailler, pourra assister à des réunions. En effet, un directeur EST-UNE personne, mais pas que.

```
class Directeur : public Personne
{
public:
    void assister_aux_reunions() const;
};

void Directeur::assister_aux_reunions() const
{
    std::cout << "Des décisions importantes doivent être prises
    aujourd'hui !\n";
}
```


Héritage et constructeurs

Vous le savez déjà, quand on instancie une classe, son constructeur est automatiquement appelé. Et, comme nous dérivons de la classe `Personne`, nous devrions logiquement avoir accès aux constructeurs que cette dernière définit. Pourtant, si nous tentons d'instancier un directeur, nous allons rencontrer une erreur bien embêtante.

```
int main()
{
    Directeur const patron { "Ron"s, "Pat"s, 2500 };
    patron.travailler();
    patron.assister_aux_reunions();

    return 0;
}
```

```
[Visual Studio]
Erreur C2440 'initialisation' : impossible de convertir de 'std::string'
en 'Personne'
Erreur C2078 initialiseurs trop nombreux
```

```
-----

[GCC]
prog.cc: In function 'int main()':
prog.cc:53:51: error: too many initializers for 'const Directeur'
   53 |     Directeur const patron { "Ron"s, "Pat"s, 2500 };
      |
```

```
-----

[Clang]
prog.cc:53:30: error: no viable conversion from 'basic_string
<char>'
  to 'Personne'
    Directeur const patron { "Ron"s, "Pat"s, 2500 };
                        ^~~~~~
prog.cc:11:5: note: candidate constructor not viable: no known
conversion from 'basic_string
<char>' to 'const Personne &' for 1st argument
    Personne(Personne const& copie) = delete;
    ^
1 error generated.
```

En effet, contrairement aux autres fonctions membres, les constructeurs ne sont pas hérités si on ne le demande pas explicitement. Pour cela, il faut utiliser un mot clé qui ne nous est pas inconnu : `using`. Ainsi, le compilateur saura qu'il existe un constructeur correspondant aux arguments donnés et le code suivant pourra compiler.

```

#include <cassert>
#include <iostream>
#include <string>

class Personne
{
    // L'implémentation vue précédemment.
};

class Directeur : public Personne
{
public:
    // On demande à réexposer les constructeurs fournis
    // dans la classe parent.
    using Personne::Personne;
    void assister_aux_reunions() const;
};

void Directeur::assister_aux_reunions() const
{
    std::cout << "Des décisions importantes doivent être prises
    aujourd'hui !\n";
}

using namespace std::literals;

int main()
{
    Directeur const patron { "Ron"s, "Pat"s, 2500 };
    // Comportement hérité de Personne.
    patron.travailler();
    // Comportement propre à un Directeur.
    patron.assister_aux_reunions();

    Personne const stagiaire { "Le Stagiaire"s, "Bob"s, 0 };
    stagiaire.travailler();
    return 0;
}

```

```

Je suis Pat Ron et j'effectue différentes tâches.
Des décisions importantes doivent être prises aujourd'hui !
Je suis Bob Le Stagiaire et j'effectue différentes tâches.

```

Bien entendu, rien ne nous interdit de définir nos propres constructeurs. Par exemple, pour un `Directeur`, on pourrait imaginer qu'on fournisse au moment de l'instancier le nom du service sous sa responsabilité. Afin de ne pas dupliquer le code déjà écrit dans le constructeur de la classe parent, C++ nous permet d'appeler un constructeur parent directement depuis la liste d'initialisation.

```

class Directeur : public Personne
{

```

```

public:
    Directeur(std::string const& nom, std::string const& prenom, int
    salaire, std::string const& direction);
    // ...
};

Directeur::Directeur(std::string const& nom, std::string const&
    prenom, int salaire, std::string const& direction)
    : Personne(nom, prenom, salaire), m_direction(direction)
    // Appel du constructeur de Personne.
{
    assert(!std::empty(m_direction) && "On doit avoir une direction
    définie.");
}

// ...

using namespace std::literals;

int main()
{
    Directeur const patron_dsi { "Ron"s, "Pat"s, 2500, "DSI"s };
    patron_dsi.travailler();
    patron_dsi.assister_aux_reunions();

    Directeur const patron_marketing { "Eting"s, "Mark"s,
    5000, "Marketing"s };
    patron_marketing.travailler();
    patron_marketing.assister_aux_reunions();

    return 0;
}

```

```

Je suis Pat Ron et j'effectue différentes tâches.
Des décisions importantes doivent être prises aujourd'hui pour le
service DSI !
Je suis Mark Eting et j'effectue différentes tâches.
Des décisions importantes doivent être prises aujourd'hui pour le
service Marketing !

```

Note > Les constructeurs et opérateurs supprimés dans la classe parent sont implicitement supprimés dans les classes enfants aussi. Il n'y a donc pas besoin de le faire explicitement pour conserver la sémantique d'entité.

Une question de visibilité

Depuis que nous avons commencé cette partie sur la programmation orientée objet, nous avons parlé de deux visibilité, représentées en C++ par les mots clés `public` et `private`. Ce qui est public peut être manipulé par n'importe quel morceau de code, alors que ce qui est privé ne peut l'être que par une instance de la classe en question (plus les éventuels amis). Cette règle est valable aussi pour les classes enfants. Autrement

dit, je ne peux pas accéder aux éléments privés d'une classe parent depuis une classe enfant. L'exemple suivant ne peut tout simplement pas compiler.

```
void Directeur::assister_aux_reunions() const
{
    std::cout << "Des décisions importantes doivent être prises
    aujourd'hui pour le service " << m_direction << " !\n";
    std::cout << "Signé le directeur, " << m_prenom << " " << m_nom << ".
    \n";
}
```

```
[Visual Studio]
Erreur (active) E0265 membre "Personne::m_prenom" (déclaré à la ligne
22) est inaccessible
Erreur (active) E0265 membre "Personne::m_nom" (déclaré à la ligne 21)
est inaccessible
Erreur C2248 'Personne::m_prenom' : impossible d'accéder à private
membre déclaré(e) dans la classe 'Personne'
Erreur C2248 'Personne::m_nom' : impossible d'accéder à private membre
déclaré(e) dans la classe 'Personne'
```

```
-----

[GCC]
prog.cc: In member function 'void Directeur::assister_aux_reunions()
const':
prog.cc:58:45: error: 'std::string Personne::m_prenom' is private within
this context
 58 |     std::cout << "Signé le directeur, " << m_prenom << " " <<
    |                                     ^~~~~~
    m_nom << ".\n";
    |
prog.cc:22:17: note: declared private here
 22 |     std::string m_prenom;
    |     ^~~~~~
prog.cc:58:64: error: 'std::string Personne::m_nom' is private within
this context
 58 |     std::cout << "Signé le directeur, " << m_prenom << " " <<
    |                                     ^~~~~~
    m_nom << ".\n";
    |
    ^~~~~
prog.cc:21:17: note: declared private here
 21 |     std::string m_nom;
    |     ^~~~~
```

```
-----

[Clang]
prog.cc:58:45: error: 'm_prenom' is a private member of 'Personne'
    std::cout << "Signé le directeur, " << m_prenom << " " << m_nom <<
    ".\n";
                                ^
prog.cc:22:17: note: declared private here
    std::string m_prenom;
```

```

      ^
prog.cc:58:64: error: 'm_nom' is a private member of 'Personne'
    std::cout << "Signé le directeur, " << m_prenom << " " << m_nom <<
    ".\n";
                                     ^
prog.cc:21:17: note: declared private here
    std::string m_nom;
                ^
2 errors generated.

```



Comment faire dans ce cas pour accéder au nom et au prénom d'un directeur ?

En l'occurrence, il conviendrait d'implémenter ces services qu'on pourrait naturellement attendre d'une personne. Nous ne l'avons pas fait pour ne pas alourdir les codes donnés en exemple. C'est donc une solution possible.

Par contre, de manière plus générale, accéder à un élément privé d'une classe parent n'est pas une bonne idée. En effet, si son auteur a jugé qu'un attribut ou qu'une fonction membre doit être **private**, c'est qu'il a sans doute une bonne raison. Il peut y avoir un invariant à respecter, ou bien il ne s'agit peut-être que d'un détail interne d'implémentation, qu'il ne convient pas de révéler au monde extérieur.

28.3. ...substituables

Les relations d'héritage ne servent pas qu'à factoriser des services. En effet, peu importe les sous-types qu'on définit, peu importe leurs caractéristiques et services propres, ils n'en restent pas moins tous des types dérivés d'une classe parent, ce qui signifie qu'ils ont tous des propriétés en commun.

Si l'on reprend notre exemple d'entreprise, un **Directeur** est un type particulier de **Personne**. On devrait donc logiquement pouvoir substituer le premier là où on attend le deuxième. L'exemple suivant en donne la confirmation.

```

#include <cassert>
#include <iostream>
#include <string>

class Personne
{
public:
    Personne() = delete;
    Personne(std::string const& nom, std::string const& prenom, int
    salaire);

    Personne(Personne const& copie) = delete;
    Personne& operator=(Personne const& copie) = delete;

```

```

    void se_presenter() const;
    void travailler() const;

private:
    std::string m_nom;
    std::string m_prenom;
    int m_salaire;
};

Personne::Personne(std::string const& nom, std::string const&
    prenom, int salaire)
    : m_nom(nom), m_prenom(prenom), m_salaire(salaire)
{
    assert(!std::empty(m_nom) && "Le nom de famille est obligatoire.");
    assert(!std::empty(m_prenom) && "Le prénom est obligatoire.");
    assert(m_salaire >= 0 && "C'est vous qui payez la personne, pas
        l'inverse !");
}

void Personne::se_presenter() const
{
    std::cout << "Je suis " << m_prenom <<
        " " << m_nom << ".\n";
}

void Personne::travailler() const
{
    std::cout << "J'effectue différentes tâches.\n";
}

class Directeur : public Personne
{
public:
    Directeur(std::string const& nom, std::string const& prenom, int
        salaire, std::string const& direction);
    void assister_aux_reunions() const;

private:
    std::string m_direction;
};

Directeur::Directeur(std::string const& nom, std::string const&
    prenom, int salaire, std::string const& direction)
    : Personne(nom, prenom, salaire), m_direction(direction)
{
    assert(!std::empty(m_direction) && "On doit avoir une direction
        définie.");
}

void Directeur::assister_aux_reunions() const
{
    std::cout << "Des décisions importantes doivent être prises
        aujourd'hui pour le service "
        << m_direction << " !\n";
}

```

```

}

// Cette fonction doit fonctionner, peu importe que je lui passe une
// Personne directement, ou un sous-type.
void commencer_journee(Personne const & personne)
{
    personne.se_presenter();
    personne.travailler();
}

using namespace std::literals;

int main()
{
    Directeur const patron_dsi { "Ron"s, "Pat"s, 2500, "DSI"s };
    commencer_journee(patron_dsi);

    Directeur const patron_marketing { "Eting"s, "Mark"s,
    5000, "Marketing"s };
    commencer_journee(patron_marketing);

    Personne const stagiaire { "Le Stagiaire", "Bob", 0 };
    commencer_journee(stagiaire);
    return 0;
}

```

```

Je suis Pat Ron.
J'effectue différentes tâches.
Je suis Mark Eting.
J'effectue différentes tâches.
Je suis Bob Le Stagiaire.
J'effectue différentes tâches.

```

Une histoire de manchots

Pourtant, le simple fait que **B** soit un sous-type de **A** ne garantit pas l'absence de problème. Imaginons que nous voulions représenter une hiérarchie d'oiseaux. Nous savons qu'une des propriétés de base d'un oiseau est de voler. Alors codons et voyons ça.

```

#include <iostream>
#include <stdexcept>

class Oiseau
{
public:
    Oiseau() = default;

    Oiseau(Oiseau const& copie) = delete;
    Oiseau& operator=(Oiseau const& autre_oiseau) = delete;

    virtual void voler() const;

```

```

};

void Oiseau::voler() const
{
    std::cout << "Je vole !\n";
}

// Un manchot est bien un oiseau, n'est-ce-pas ?
class Manchot : public Oiseau
{
public:
    void voler() const override;
};

void Manchot::voler() const
{
    // Pourtant un manchot ne sait pas voler.
    throw std::logic_error("Non je nage ou je me dandine en marchant!");
}

void voler_gracieusement(Oiseau const & oiseau)
{
    std::cout << "Et c'est le décollage !\n";
    oiseau.voler();
}

int main()
{
    Manchot empereur {};
    voler_gracieusement(empereur);
    return 0;
}

```

On le voit avec cet exemple, il y a un problème : on perd un service, une possibilité. `Manchot` a beau être un sous-type de `Oiseau`, on ne peut pas dire qu'il soit vraiment substituable. Si la lecture d'un code aussi simple nous montre qu'il ne faut pas utiliser le service `voler` sur un `Manchot`, il n'empêche que la syntaxe est valide et n'empêchera pas de se tromper.

Le principe de substitution de Liskov

C'est là qu'intervient un principe énoncé dans les années 1980 par une informaticienne américaine du nom de Barbara Liskov et qui depuis porte son nom, abrégé LSP en anglais. Ce dernier énonce que si une propriété P est vraie pour une instance x d'un type T , alors elle doit rester vraie pour toute instance y d'un sous-type de T .

Autrement dit, puisqu'un `Manchot` ne respecte pas toutes les propriétés qu'on attend d'un `Oiseau`, il ne respecte pas le LSP et n'a donc pas sa place dans une relation d'héritage.

Au contraire, dans l'exemple d'entreprise, un **Directeur** est parfaitement substituable à une **Personne** et respecte bien le LSP.

Le principe de substitution de Liskov nous est d'une grande aide pour déterminer si hériter d'une classe a un sens ou non. Ainsi, si un **Ouvrier** est une personne, on ne peut pas dire la même chose d'un **Chat**, d'une **Voiture** ou d'une **GrandeQuestionUniverselle**. Ces classes n'ont donc pas leur place dans un héritage avec le type **Personne**.

Substitution dans la bibliothèque standard

Quand nous avons parlé pour la première fois des exceptions, nous avons dit que `std::exception` était un type générique qui permet de rattraper n'importe quelle autre exception, comme `std::runtime_exception` ou `std::invalid_argument`. C'est parce que ces classes sont des sous-types de la classe parent `std::exception`. Comme elles ont les mêmes propriétés, qu'elles exposent les mêmes services, le principe de substitution de Liskov s'applique et on peut transmettre une exception enfant à un `catch` attrapant une exception parent, sans aucun problème.

```
#include <iostream>
#include <stdexcept>
#include <string>

int main()
{
    try
    {
        // Va lancer une exception std::out_of_range.
        int entier {

std::stoi("1000000000000000000000000000000000000000000000000000000000000000")
        };
        std::cout << "Entier : " << entier << "\n";
    }
    catch (std::exception const & exception)
    {
        std::cout << "Une erreur est survenue : "
            << exception.what() << "\n";
    }

    return 0;
}
```

Substitution et programmation par contrat

Parfois, il peut être difficile de déterminer si un héritage a du sens ou non. Pour nous aider, il convient de réfléchir à ce que nous avons appris sur la programmation par

contrat et notamment sur les invariants et les préconditions/postconditions. En effet, le principe de substitution de Liskov indique qu'une classe enfant doit respecter certaines règles concernant les invariants et les contrats exposés par la classe parent. Elles sont au nombre de trois :

- les préconditions ne peuvent pas être renforcées dans une sous-classe ;
- les postconditions ne peuvent pas être affaiblies dans une sous-classe ;
- les invariants doivent être préservés.

En gardant le principe de substitutalité en tête, on comprend facilement la raison d'être de ces règles. En effet, avec des préconditions plus fortes dans le sous-type, nous risquons de voir celui-ci refusé là où une instance de la classe parent serait acceptée. De même, avec des postconditions trop faibles, nous n'aurons pas les garanties auxquelles nous nous attendrions avec une instance de la classe parent.

Un bon exemple

Prenons un exemple tiré de l'aviation civile, inventé par Luc Hermitte, à propos des compagnies aériennes. Quand vous achetez un billet, vous avez une taille réglementaire de bagage. Les compagnies low-cost font en général payer si vous voulez embarquer avec des bagages plus gros, alors que les compagnies traditionnelles les prennent souvent sans supplément. La précondition sur les bagages est donc *plus faible* dans le cas d'une compagnie traditionnelle que pour une compagnie low-cost.

De plus, vous n'avez aucune garantie de manger gratuitement sur un vol low-cost, alors que les compagnies traditionnelles vous offrent souvent un petit repas. Vous avez donc une postcondition *plus forte* sur les compagnies traditionnelles, où vous arrivez à destination le ventre plein, ce qui n'est pas certain avec un vol low-cost.

Dans tous les cas, peu importe le type de compagnie, l'invariant "Arriver à destination en toute sécurité" doit être respecté. Si la compagnie ne respecte pas cet invariant, il y a assez peu de chance que vous souhaitiez voyager de nouveau avec eux.

Cette réflexion nous permet de valider que nous pourrions partir sur une relation d'héritage entre une compagnie low-cost minimaliste et une compagnie traditionnelle plus chère, mais à la prestation plus développée.

Un mauvais exemple

Imaginez un instant que nous souhaitions stocker un ensemble de valeurs dans un tableau, de manière à ce que ces valeurs soient toujours triées en ordre croissant et automatiquement à chaque ajout, modification ou suppression d'une valeur. Comme ce

genre de conteneur n'existe pas dans la bibliothèque standard C++ (mais dans [Boost](#) [oui](#)), nous allons écrire le nôtre, qui dérive de `std::vector`. Après tout, nous voulons avoir la même interface, à base de `push_back`, `clear`, etc.

Mais si on regarde plus en détail, ce n'est pas une bonne idée. Par exemple, avec `std::vector`, quand on exécute un `push_back`, on a comme postcondition que l'élément qui vient d'être inséré est le même que celui renvoyé par `std::back`. Avec une liste triée, ce n'est pas forcément vrai, car si on ajoute 3 à un tableau qui contient déjà 1 et 5, `std::back` renverra toujours 5 et non 3.

Nous avons affaibli une postcondition, ce qui est une violation du LSP. On ne peut donc pas faire de tableau automatiquement trié en héritant de `std::vector`.

RÉFLEXION PRÉALABLE

Cette réflexion est importante à mener car C++ ne vous empêchera pas de faire un héritage à partir de types n'ayant rien en commun, ou qui ne respectent pas les principes de la programmation par contrat. Pour le compilateur, si la syntaxe est respectée, il n'y a rien à redire.

Si vous êtes tenté de créer un sous-type par héritage, il faut vous poser les deux questions suivantes, dans l'ordre.

- Cet héritage respecte-t-il bien une relation EST-UN ?
- Cet héritage respecte-t-il le LSP du point de vue de la programmation par contrat, c'est-à-dire une relation EST-SUBSTITUABLE-À ?

Héritage public et LSP

Lorsqu'on définit une relation d'héritage entre `class`, on utilise le mot clé `public` entre les deux-points : et l'identificateur de la classe parent. Sa présence n'est pas anodine, car il indique qu'on veut réaliser un *héritage public*, où les visibilitées de la classe parent seront conservées dans la classe enfant. Autrement dit, ce qui est public dans la classe parent le reste dans la classe enfant. Grâce à ça, on garde la même interface et on permet la substitution (d'un point de vue syntaxique seulement, pas forcément d'un point de vue LSP).

Sans la présence de ce mot clé, nous aurions alors affaire à un *héritage privé*, dans lequel tout ce qui est public dans la classe parent devient privé dans la classe enfant. Les services exposés par la classe parent ne seraient plus visibles ni utilisables par les

utilisateurs de la classe enfant et nous ne serions donc plus dans une relation EST-UN. L'exemple de substitution donné au début de cette section ne compilerait donc plus.

```
// ...

// Héritage privé.
class Directeur : Personne
{
    // ...
};

// ...
```

```
[Visual Studio]
Erreur C2243 'cast de type' : la conversion de 'const Directeur *' en
'const Personne &' existe, mais n'est pas accessible

-----

[GCC]
prog.cc: In function 'int main()':
prog.cc:74:33: error: 'Personne' is an inaccessible base of 'Directeur'
   74 |         commencer_journee(patron_dsi);
      |                             ^
prog.cc:77:39: error: 'Personne' is an inaccessible base of 'Directeur'
   77 |         commencer_journee(patron_marketing);
      |                             ^
-----

[Clang]
prog.cc:74:23: error: cannot cast 'const Directeur' to its private base
class 'const Personne'
    commencer_journee(patron_dsi);
                        ^
prog.cc:43:19: note: implicitly declared private here
class Directeur : Personne
    ^~~~~~
prog.cc:77:23: error: cannot cast 'const Directeur' to its private base
class 'const Personne'
    commencer_journee(patron_marketing);
                        ^
prog.cc:43:19: note: implicitly declared private here
class Directeur : Personne
    ^~~~~~
2 errors generated.
```

Les implications de ce type d'héritage et les cas d'utilisation sortent du cadre de ce livre, nous n'en parlerons donc pas plus, mais nous nous devons de faire ce petit aparté.

Note > Dans d'autres langages comme Java ou Python, il n'y a pas de notion d'héritage public/privé. C'est un concept spécifique à C++.

Petite subtilité, si nous remplaçons `class` par `struct`, alors l'héritage par défaut est public. Le mot clé `public` ne serait alors plus nécessaire, bien que nous vous encourageons à être explicite. Cette différence vient du fait que tout est public par défaut pour une `struct`, alors que tout est privé par défaut pour une `class`, y compris en ce qui concerne l'héritage.

```
// Héritage public, aucun problème.
struct Directeur : Personne
{
    // ...
};
```

28.4. Un monde virtuel

L'intérêt de définir des relations d'héritage et d'utiliser la substituabilité, c'est qu'on peut créer des *points de variation dynamiques*. Cela nous donnera enfin l'occasion de créer des objets au comportement qui varie dynamiquement. Nous aurons ce qu'on appelle des *entités polymorphiques*.

Un problème...

Commençons donc par poser le problème que l'on veut résoudre et imaginons un programme qui contrôle un robot nettoyeur. Ce robot peut en particulier nettoyer le sol et il doit être capable d'utiliser plusieurs méthodes, qui varient selon l'outil, comme par exemple le balai, l'aspirateur, etc.

On pourrait y aller brutalement et mettre des conditions pour choisir la méthode. C'est le style qu'on aurait sans doute choisi si cet exemple avait été développé dans la première partie.

```
#include <iostream>

enum class MethodeNettoyage
{
    AvecBalai, AvecAspirateur
};

void preparer_outil(MethodeNettoyage methode)
{
    if (methode == MethodeNettoyage::AvecBalai)
    {
        std::cout << "Je saisis le balai.\n";
    }
    else if (methode == MethodeNettoyage::AvecAspirateur)
    {
        std::cout << "Je branche l'aspirateur.\n";
    }
}
```

```

        std::cout << "J'allume l'aspirateur.\n";
        std::cout << "Je saisis le manche de l'aspirateur.\n";
    }
}

void nettoyer(MethodeNettoyage methode)
{
    if (methode == MethodeNettoyage::AvecBalai)
    {
        std::cout << "Je nettoie avec un mouvement de balancier.\n";
        std::cout << "Je me déplace.\n";
    }
    else if (methode == MethodeNettoyage::AvecAspirateur)
    {
        std::cout << "Je nettoie avec des aller-retours.\n";
        std::cout << "Je me déplace.\n";
    }
}

void ranger_outil(MethodeNettoyage methode)
{
    if (methode == MethodeNettoyage::AvecBalai)
    {
        std::cout << "Je remets le balai dans le placard.\n";
    }
    else if (methode == MethodeNettoyage::AvecAspirateur)
    {
        std::cout << "J'éteins l'aspirateur.\n";
        std::cout << "Je débranche l'aspirateur.\n";
        std::cout << "Je range l'aspirateur.\n";
    }
}

void nettoyer_sol(MethodeNettoyage methode)
{
    preparer_outil(methode);
    nettoyer(methode);
    ranger_outil(methode);
}

int main()
{
    std::cout << "Je commence par le salon.\n";
    nettoyer_sol(MethodeNettoyage::AvecAspirateur);

    std::cout << "\nJe passe au garage.\n";
    nettoyer_sol(MethodeNettoyage::AvecBalai);

    return 0;
}

```

Tout a l'air de bien aller... jusqu'au jour où la personne qui a commandé le robot souhaite que ce dernier sache aussi utiliser un aspirateur automatique. Cela implique alors de modifier les fonctions `preparer_outil`, `nettoyer` et `ranger_outil`.

```

void preparer_outil(MethodeNettoyage methode)
{
    if (methode == MethodeNettoyage::AvecBalai)
    {
        std::cout << "Je saisis le balai.\n";
    }
    else if (methode == MethodeNettoyage::AvecAspirateur)
    {
        std::cout << "Je branche l'aspirateur.\n";
        std::cout << "J'allume l'aspirateur.\n";
        std::cout << "Je saisis le manche de l'aspirateur.\n";
    }
    else if (methode == MethodeNettoyage::AvecAspirateurAuto)
    {
        // On imagine une logique derrière pour connaître le niveau de
        charge.
        bool const est_charge { false };

        if (!est_charge)
        {
            std::cout << "Je recharge l'aspirateur automatique.\n";
        }
        std::cout << "Je branche l'aspirateur automatique.\n";
        std::cout << "Je démarre l'aspirateur automatique.\n";
    }
}

void nettoyer(MethodeNettoyage methode)
{
    if (methode == MethodeNettoyage::AvecBalai)
    {
        std::cout << "Je nettoie avec un mouvement de balancier.\n";
        std::cout << "Je me déplace.\n";
    }
    else if (methode == MethodeNettoyage::AvecAspirateur)
    {
        std::cout << "Je nettoie avec des aller-retours.\n";
        std::cout << "Je me déplace.\n";
    }
    else if (methode == MethodeNettoyage::AvecAspirateurAuto)
    {
        std::cout << "Je regarde faire.\n";
    }
}

void ranger_outil(MethodeNettoyage methode)
{
    if (methode == MethodeNettoyage::AvecBalai)
    {
        std::cout << "Je remets le balai dans le placard.\n";
    }
    else if (methode == MethodeNettoyage::AvecAspirateur)
    {
        std::cout << "J'éteins l'aspirateur.\n";
    }
}

```

```

        std::cout << "Je débranche l'aspirateur.\n";
        std::cout << "Je range l'aspirateur.\n";
    }
    else if (methode == MethodeNettoyage::AvecAspirateurAuto)
    {
        std::cout << "J'éteins l'aspirateur automatique.\n";
        std::cout << "Je range l'aspirateur automatique.\n";
    }
}

```

On voit déjà que le code se complique. D'autant que ce n'est que le début, votre commanditaire pouvant sûrement avoir l'idée soudaine de prendre en charge les systèmes d'aspiration centralisée, les aspirateurs auto-porteurs de chats, etc.

Il existe un principe qui s'appelle l'*open-closed principle* [soit en français le *principe ouvert-fermé*]. Celui-ci stipule qu'une classe doit être ouverte aux évolutions, aux extensions, mais rester fermée aux modifications. En d'autres termes, une fois qu'on a testé et validé le comportement d'une classe, on ne devrait plus avoir à la modifier (sauf pour corriger des bugs). Par contre, elle doit être ouverte aux extensions dans le sens qu'on devrait pouvoir prendre en compte des nouveaux comportements sans avoir à tout chambouler derrière.

Dans le cadre de notre robot nettoyeur, cela veut dire qu'on veut pouvoir ajouter une nouvelle méthode de nettoyage sans avoir à modifier celles qui existent déjà. Notre code actuel ne le permet pas.

Note > Nous aurons l'occasion de reparler plus en détail de ce principe au chapitre Ça, c'est du SOLID !.

...une solution

Une solution possible pour respecter ce principe est de se servir de ces fameux points de variation dynamiques. Nous allons créer une classe représentant une méthode de nettoyage et nous ferons une classe dérivée pour chaque méthode de nettoyage possible.

Et pour que nos points de variation soient dynamiques, nous allons avoir besoin de deux nouveaux mots clés :

- **virtual**, qui indique au compilateur qu'une fonction membre peut voir son comportement supplanté/redéfini par celui d'une classe enfant ;
- **override**, qui indique au compilateur qu'une fonction membre supplante/redéfinit le comportement d'une fonction membre de sa classe parent.

Notez que ces deux mots clés ne s'appliquent qu'à la déclaration d'une fonction membre, pas à son implémentation.


```

#include <iostream>

class MethodeNettoyage
{
public:
    MethodeNettoyage() = default;

    MethodeNettoyage(MethodeNettoyage const& copie) = delete;
    MethodeNettoyage& operator=(MethodeNettoyage const& copie) = delete;

    virtual void preparer_outil() const;
    virtual void nettoyer() const;
    virtual void ranger_outil() const;
};

void MethodeNettoyage::preparer_outil() const
{
    std::cout << "Préparation de l'outil de nettoyage.\n";
}

void MethodeNettoyage::nettoyer() const
{
    std::cout << "Nettoyage de la pièce.\n";
}

void MethodeNettoyage::ranger_outil() const
{
    std::cout << "Rangement de l'outil de nettoyage.\n";
}

class AvecBalai : public MethodeNettoyage
{
public:
    void preparer_outil() const override;
    void nettoyer() const override;
    void ranger_outil() const override;
};

void AvecBalai::preparer_outil() const
{
    std::cout << "Je saisis le balai.\n";
}

void AvecBalai::nettoyer() const
{
    std::cout << "Je nettoie avec un mouvement de balancier.\n";
    std::cout << "Je me déplace.\n";
}

void AvecBalai::ranger_outil() const
{
    std::cout << "Je remets le balai dans le placard.\n";
}

```

```

class AvecAspirateur : public MethodeNettoyage
{
public:
    void preparer_outil() const override;
    void nettoyer() const override;
    void ranger_outil() const override;
};

void AvecAspirateur::preparer_outil() const
{
    std::cout << "Je branche l'aspirateur.\n";
    std::cout << "J'allume l'aspirateur.\n";
    std::cout << "Je saisis le manche de l'aspirateur.\n";
}

void AvecAspirateur::nettoyer() const
{
    std::cout << "Je nettoie avec des aller-retours.\n";
    std::cout << "Je me déplace.\n";
}

void AvecAspirateur::ranger_outil() const
{
    std::cout << "J'éteins l'aspirateur.\n";
    std::cout << "Je débranche l'aspirateur.\n";
    std::cout << "Je range l'aspirateur.\n";
}

class AvecAspirateurAuto : public MethodeNettoyage
{
public:
    void preparer_outil() const override;
    void nettoyer() const override;
    void ranger_outil() const override;

private:
    bool m_est_charge{ false };
};

void AvecAspirateurAuto::preparer_outil() const
{
    if (!m_est_charge)
    {
        std::cout << "Je recharge l'aspirateur automatique.\n";
    }
    std::cout << "Je démarre l'aspirateur automatique.\n";
}

void AvecAspirateurAuto::nettoyer() const
{
    std::cout << "Je regarde faire.\n";
}

void AvecAspirateurAuto::ranger_outil() const
{

```

```

    std::cout << "J'éteins l'aspirateur automatique.\n";
    std::cout << "Je range l'aspirateur automatique.\n";
}

void nettoyer_sol(MethodeNettoyage const & methode)
{
    methode.preparer_outil();
    methode.nettoyer();
    methode.ranger_outil();
}

int main()
{
    std::cout << "Je commence par le salon.\n";
    AvecAspirateur const pour_le_salon {};
    nettoyer_sol(pour_le_salon);

    std::cout << "\nJe passe au garage.\n";
    AvecBalai const pour_le_garage {};
    nettoyer_sol(pour_le_garage);

    std::cout << "\nMaintenant la chambre.\n";
    AvecAspirateurAuto const pour_la_chambre {};
    nettoyer_sol(pour_la_chambre);

    return 0;
}

```

```

Je commence par le salon.
Je branche l'aspirateur.
J'allume l'aspirateur.
Je saisis le manche de l'aspirateur.
Je nettoie avec des aller-retours.
Je me déplace.
J'éteins l'aspirateur.
Je débranche l'aspirateur.
Je range l'aspirateur.

Je passe au garage.
Je saisis le balai.
Je nettoie avec un mouvement de balancier.
Je me déplace.
Je remets le balai dans le placard.

Maintenant la chambre.
Je recharge l'aspirateur automatique.
Je démarre l'aspirateur automatique.
Je regarde faire.
J'éteins l'aspirateur automatique.
Je range l'aspirateur automatique.

```

Note > Voici un exemple où nos classes ne représentent pas des objets de la vie réelle.

Avec cette architecture, pour faire un nettoyage avec une méthode, il suffit d'instancier une classe dérivant de `MethodeNettoyage` et de la passer à la fonction `nettoyer_sol`. On règle ainsi le problème de la violation de l'OCP, puisque pour ajouter l'aspiration centralisée, par exemple, il suffit de créer la classe enfant correspondante, sans avoir besoin de modifier les autres classes existantes.

En interne

Quand on passe un objet à la fonction `nettoyer_sol`, le compilateur ne voit qu'un type `MethodeNettoyage`, ce qui est le type statique de l'objet. Mais il est possible que son type réel soit différent. Ça peut être `AvecBalai`, `AvecAspirateur`, etc.

Pour que le compilateur soit capable de faire appel aux services du type réel qui se cache derrière, nous devons demander une *résolution dynamique* des appels de fonction, qui se fera à l'exécution. C'est ce que permet de faire le mot clé `virtual`. Toute fonction membre devant laquelle on écrit ce mot clé est dite virtuelle.

Note > Dans la terminologie spécifique C++, on parle également de méthode, mais nous n'emploierons pas ce terme, pour éviter tout risque de confusion.

Sans ce mot clé, le compilateur opère une *résolution statique* des appels, c'est-à-dire à la compilation, ce qui exécutera les méthodes que fournit le type de base, à savoir ici `MethodeNettoyage`. Nous n'aurons donc aucun point de variation dynamique, puisque seul le type de base sera appelé. Pour tester, enlevez tous les `virtual` et `override` puis recompilez.

```
Je commence par le salon.
Préparation de l'outil de nettoyage.
Nettoyage de la pièce.
Rangement de l'outil de nettoyage.

Je passe au garage.
Préparation de l'outil de nettoyage.
Nettoyage de la pièce.
Rangement de l'outil de nettoyage.

Maintenant la chambre.
Préparation de l'outil de nettoyage.
Nettoyage de la pièce.
Rangement de l'outil de nettoyage
```

Alors que l'utilisation des mots clés `virtual` et `override` permet une *redéfinition* d'un comportement par une classe enfant, ce qui est voulu, leur absence entraîne un *masquage* de la fonction définie dans la classe parent par celle définie dans la classe enfant. Sauf dans de rares cas que nous n'aborderons pas, c'est une mauvaise idée.

D'ailleurs, `override`, introduit avec C++11, nous donne une sécurité supplémentaire puisqu'il ne peut s'appliquer que pour une fonction membre d'une classe enfant qui supprime/redéfinit une fonction membre d'une classe parent. Si on utilise ce mot clé alors que la fonction membre de la classe parent n'est pas virtuelle ou n'a pas la même signature, la compilation échoue.

```
class A
{
public:
    // Fonction membre virtuelle.
    virtual void f1(int entier);
    // Autre fonction membre, non-virtuelle.
    void f2(int entier);
};

class B : public A
{
public:
    // Compile et fonctionne.
    void f1(int entier) override;
    // Erreur car surcharge non-virtuelle (présence de const).
    void f1(int entier) const override;
    // Erreur car non-virtuelle dans la classe parent.
    void f2(int entier) override;
};

int main()
{
    A a {};
    B b {};

    return 0;
}
```

```
[Visual Studio]
Erreur (active) E1455 une fonction membre déclarée avec le modificateur
'override' ne remplace pas un membre de la classe de base
Erreur (active) E1455 une fonction membre déclarée avec le modificateur
'override' ne remplace pas un membre de la classe de base
Erreur C3668 'B::f1' : la méthode comprenant le spécificateur de
substitution 'override' n'a substitué aucune méthode de la classe de
base
Erreur C3668 'B::f2' : la méthode comprenant le spécificateur de
substitution 'override' n'a substitué aucune méthode de la classe de
base
```

```
-----
[GCC]
prog.cc:16:10: error: 'void B::f1(int) const' marked 'override', but
does not override
   16 |     void f1(int entier) const override;
      |         ^~
```

```

prog.cc:18:10: error: 'void B::f2(int)' marked 'override', but does not
override
  18 |         void f2(int entier) override;
     |         ^~
-----

[Clang]
prog.cc:16:31: error: only virtual member functions can be marked
'override'
    void f1(int entier) const override;
                        ^~~~~~
prog.cc:18:25: error: only virtual member functions can be marked
'override'
    void f2(int entier) override;
                        ^~~~~~
2 errors generated.

```

Faisons le point

Nous avons découvert un nouveau mécanisme qui permet de faire varier dynamiquement le comportement de notre programme et cela nous donne un double avantage par rapport à l'approche naïve avec laquelle nous avons débuté :

- l'OCP est bien mieux respecté, comme nous l'avons déjà remarqué ;
- cela nous donne une beaucoup plus grande puissance d'abstraction. Le code de nettoyage n'a pas besoin de connaître le détail des fonctions appelées et surtout n'a pas besoin de connaître toutes les façons possibles de nettoyer. Cette responsabilité est transférée à la classe `MethodeNettoyage` et aux classes qui en dérivent.

Ce nouveau mécanisme s'appelle le *polymorphisme d'inclusion*, qu'on abrège très souvent dans la terminologie orientée objet par *polymorphisme*, bien que ce terme puisse recouper plusieurs notions en C++, dont la surcharge de fonctions et les templates.

ET CHEZ LES AUTRES ?

Certains langages, comme Python ou Java, ne font que des liaisons dynamiques par défaut. N'importe quelle fonction membre peut donc être supplantée. D'autres, comme C#, ont fait les mêmes choix que C++, à savoir liaison statique par défaut et liaison dynamique sur demande explicite.

28.5. Interfaces et classes abstraites

Des fonctions virtuelles vraiment pures

Parfois, lorsqu'on réalise un héritage, on se rend compte que l'on ne peut pas vraiment attribuer de comportement à la classe parent. Par exemple, la classe `MethodeNettoyage` n'a pas de comportement propre, cohérent. Elle sert simplement à indiquer aux classes enfants ce qu'elles doivent définir, quelles interfaces elles doivent exposer. Ce besoin de définition d'interface sans implémentation a une solution en C++, qui s'appelle *classes abstraites*. Elles sont non-instanciables — d'où la qualification d'abstraite — et n'auront vocation qu'à être héritées.

Pour rendre une classe abstraite, il suffit que l'une de ses fonctions membres soit virtuelle pure. Cela se fait en ajoutant `= 0` à la fin de son prototype. Il sera désormais impossible d'instancier cette classe, sous peine de voir le compilateur se plaindre.

```
class MethodeNettoyage
{
public:
    // ...

    virtual void preparer_outil() const = 0;
    virtual void nettoyer() const = 0;
    virtual void ranger_outil() const = 0;
};

// Pas d'implémentation pour ces trois fonctions membres.

int main()
{
    MethodeNettoyage methode {};
    return 0;
}
```

```
[Visual Studio]
Erreur (active) E0322 objet de type classe abstraite "MethodeNettoyage"
non autorisé
Erreur C2259 'MethodeNettoyage' : impossible d'instancier une classe
abstraite
```

```
-----

[GCC]
prog.cc: In function 'int main()':
prog.cc:20:28: error: cannot declare variable 'methode' to be of
abstract type 'MethodeNettoyage'
   20 |     MethodeNettoyage const methode {};
      |                               ^~~~~~
```

```

prog.cc:3:7: note:   because the following virtual functions are pure
      within 'MethodeNettoyage':
      3 | class MethodeNettoyage
        |      ^~~~~~
prog.cc:13:18: note:   'virtual void
MethodeNettoyage::preparer_outil() const'
      13 |     virtual void preparer_outil() const = 0;
        |           ^~~~~~
prog.cc:14:18: note:   'virtual void MethodeNettoyage::nettoyer()
const'
      14 |     virtual void nettoyer() const = 0;
        |           ^~~~~~
prog.cc:15:18: note:   'virtual void MethodeNettoyage::ranger_outil()
const'
      15 |     virtual void ranger_outil() const = 0;
        |           ^~~~~~
-----

[Clang]
prog.cc:20:28: error: variable type 'const MethodeNettoyage' is an
      abstract class
      MethodeNettoyage const methode {};
        ^
prog.cc:13:18: note: unimplemented pure virtual method 'preparer_outil'
in 'MethodeNettoyage'
      virtual void preparer_outil() const = 0;
        ^
prog.cc:14:18: note: unimplemented pure virtual method 'nettoyer' in
'MethodeNettoyage'
      virtual void nettoyer() const = 0;
        ^
prog.cc:15:18: note: unimplemented pure virtual method 'ranger_outil' in
'MethodeNettoyage'
      virtual void ranger_outil() const = 0;
        ^
1 error generated.

```

À l'inverse, pour qu'un type dérivé d'une classe abstraite soit instanciable, il faut qu'il redéfinisse *toutes* les fonctions virtuelles pures de la classe parent. Dans le cas où certaines d'entre elles ne sont pas supplantées, le type dérivé reste lui aussi abstrait.

Classe abstraite et implémentation

Une classe devient abstraite dès lors que l'une de ses fonctions membres est déclarée virtuelle pure, mais cela ne veut pas dire que toutes les autres doivent forcément l'être. Rien ne nous empêche d'implémenter certaines fonctions membres virtuelles, qui pourront alors être réutilisées par les classes enfants.

Par exemple, si on code un système de comptes bancaires, on voit bien qu'un compte est quelque chose d'abstrait. Par contre, un livret A, un compte d'épargne, une assu-

rance-vie sont des objets concrets. On pourrait ainsi définir une classe `Compte` comme étant abstraite, tout en proposant une implémentation du service `crediter`, qui est une opération commune à tous les comptes. On évite une duplication du code et celui-ci s'en trouve amélioré.

ET CHEZ LES AUTRES ?

D'autres langages, comme C# ou PHP, font une différence entre ce qu'ils appellent une interface et une classe abstraite. Dans le premier cas, on ne peut définir qu'une liste de services à rendre (des signatures uniquement), alors que dans le deuxième, on peut implémenter une partie du comportement (comme en C++).

28.6. La copie, une vraie source de problème

En début de chapitre, nous avons vu une raison philosophique d'interdire la copie d'entités. Mais votre compilateur C++ n'en a que faire de la philosophie et vous laissera définir un constructeur et un opérateur d'affectation par copie si vous le souhaitez. Si vous le faites, vous allez avoir une mauvaise surprise.

```
#include <iostream>

struct Base
{
    // Attribut commun à tous les descendants.
    int i;

    friend std::ostream& operator<<(std::ostream& os, Base const& valeur);
};

std::ostream& operator<<(std::ostream& os, Base const& valeur)
{
    return os << "i = " << valeur.i;
}

struct Enfant : public Base
{
    // Un attribut spécifique à Enfant.
    int j;

    friend std::ostream& operator<<(std::ostream& os, Enfant const&
    valeur);
};

std::ostream& operator<<(std::ostream& os, Enfant const& valeur)
{
    return os << "i = " << valeur.i << " - j = " << valeur.j;
```

```

}

int main()
{
    // On crée et on affiche deux enfants, OK.
    Enfant e1 { 1, 42 };
    Enfant e2 { 2, 84 };

    std::cout << "e1 : " << e1 << std::endl;
    std::cout << "e2 : " << e2 << std::endl;

    // Est-ce que je peux faire appel au constructeur par copie ?
    Base b1 { e1 };
    // Mais où est passé 'j' ?
    std::cout << "b1 : " << b1 << std::endl;

    // Bon, est-ce que je peux au moins avoir une référence
    // sur Base qui soit en fait un Enfant ? Oui.
    Base& b2 { e2 };
    // Ok, à travers la référence, je vais assigner une
    // nouvelle valeur à e2.
    b2 = e1;
    // Hein ?
    std::cout << "e2 : " << e2 << std::endl;

    return 0;
}

```

```

e1 : i = 1 - j = 42
e2 : i = 2 - j = 84
b1 : i = 1
e2 : i = 1 - j = 84

```

Ce que vous venez de voir est une célèbre erreur en C++ qu'on appelle le *slicing* [ou en français *tronçonnage*, *découpage*]. Elle vient du fait que la taille que va occuper une instance en mémoire varie d'un sous-type à un autre. C'est ce que démontre le code suivant, qui fait appel à l'opérateur `sizeof`, lequel renvoie la taille en bytes qu'occupe un type ou une expression.

```

#include <iostream>

struct Base
{
    int i;
};

struct Enfant : public Base
{
    int j;
};

```

```
int main()
{
    std::cout << "Taille de Base : "
               << sizeof(Base) << "\n";
    std::cout << "Taille de Enfant : "
               << sizeof(Enfant) << "\n";
    return 0;
}
```

```
Taille de Base : 4
Taille de Enfant : 8
```

En plus de la taille qu'occupe en mémoire l'attribut `i`, commun à tous les enfants, une instance de la classe `Enfant` a besoin de stocker l'attribut `j` qui lui est propre. Elle occupe donc plus de place en mémoire que son parent.

Or, les constructeurs d'une classe, dont le constructeur de copie, ne peuvent pas être virtuels car on connaît forcément le type réel que l'on veut instancier. Donc lorsqu'on passe une classe enfant au constructeur d'une classe parent, le type réel est *perdu*. Seuls les attributs hérités de la classe parent seront copiés, ceux spécifiques aux classes enfants seront ignorés. C'est pourquoi l'affichage de `e2` après `b2 = e1` est erroné, puisque la copie de `e1.j` vers `e2.j` n'a jamais eu lieu.

28.7. Récapitulons !

- Les entités sont uniques et ne sont pas comparables entre elles.
- La surcharge des opérateurs arithmétiques et de comparaison n'a pas de sens dans le cas de la sémantique d'entité.
- La copie est désactivée grâce au mot clé `delete`.
- On peut définir des sous-types, une catégorie d'entités, grâce à l'héritage.
- Les constructeurs d'une classe parent sont exposés dans les classes enfants avec le mot clé `using`.
- Le principe de substitution de Liskov indique qu'on doit pouvoir utiliser n'importe quel sous-type à la place de la classe parent sans que ça ne pose de problème.
- Les conséquences sur la programmation par contrat sont qu'on ne peut pas durcir les préconditions, affaiblir les postconditions ou transgresser les invariants.
- Grâce à l'héritage et au polymorphisme d'inclusion, on peut définir des points de variation dynamiques, qui permettent de mieux respecter le principe ouvert/fermé.
- Certains types ne définissent aucun comportement propre et ne font qu'indiquer aux classes enfants ce qu'elles doivent implémenter elles-mêmes.

- Une fonction virtuelle est dite pure si elle est suivie de `= 0`.
- La présence d'une seule fonction virtuelle pure suffit à rendre un type abstrait, ce qui empêchera son instanciation.
- La copie d'objets dans une relation d'héritage pose le problème du slicing, qui rend la copie partielle et donc incorrecte.

29

Ressources, indirections et handles

C++ est un langage très puissant mais complexe. Ce chapitre est le plus difficile du livre. En effet, il va nous plonger dans des concepts et des détails bien souvent masqués dans d'autres langages. Nous avons fait tout notre possible pour rendre les explications claires et progressives, autant que faire se peut. Si vous avez du mal à comprendre, ne vous inquiétez pas. Relisez autant de fois que nécessaire et prenez votre temps.

Ce chapitre va parler de *mémoire*, d'*indirections*, de *ressources* et de *handles*.

29.1. Retour sur la mémoire

Du stockage pour tous !

Créer des variables, nous savons le faire depuis le début du livre. Quand nous avons abordé le sujet pour la première fois, nous avons simplement dit qu'une variable était stockée en mémoire. C'est vrai, mais il est temps d'étayer un peu cette affirmation. Il faut en effet savoir qu'on distingue essentiellement trois zones :

L'automatic storage

C'est le type de stockage que nous avons le plus manipulé depuis le début du livre. La mémoire est réservée automatiquement par le compilateur au moment où l'on rentre dans la portée courante (le bloc d'accolades) dans laquelle est déclarée la variable concernée. Inversement, elle est libérée quand on sort de cette même portée. C'est pourquoi, dans le [chapitre sur les conditions](#), nous avons vu qu'on ne pouvait pas réutiliser une variable en dehors de sa portée.

```
int main()
{
    if (true)
    {
        // La mémoire est réservée pour la variable 'age'.
        int const age { 42 };
    }
}
```

```

    // Du code...

    if (age == 42)
    {
    }

    // Du code...

    // On atteint la fin de la portée,
    // la mémoire occupée par 'age' est libérée.
}

// Erreur, le compilateur ne connaît pas la variable 'age'.
age += 5;
return 0;
}

```

C'est également la raison pour laquelle renvoyer à un appelant une référence portant sur une variable locale est une erreur, car on sort de la portée dans laquelle cette variable locale est définie et la mémoire qu'elle occupait est libérée. On enverrait alors une référence sur n'importe quoi.

```

int& fonction()
{
    int variable_locale { 4 };
    int& reference_variable_locale { variable_locale };
    // Erreur, 'variable_locale' n'existera plus.
    // La référence ne sera plus valide.
    return reference_variable_locale;
}

int main()
{
    // Référence sur n'importe quoi.
    int& reference { fonction() };
    return 0;
}

```

Le static storage

Ce deuxième type de stockage regroupe des variables qui sont allouées une seule fois en tout début de programme et qui seront libérées uniquement quand l'exécution du programme s'achèvera. Nous n'en avons jamais défini nous-mêmes, mais nous savons en utiliser et ce, depuis le premier chapitre. En effet, `std::cout` est stocké dans le *static storage*.

En nous basant sur cette propriété, nous sommes en mesure d'écrire le code suivant. Notez que, puisque `compteur_statique` est une variable statique, elle est conser-

vée tout au long du programme, ce qui explique que sa valeur augmente au fur et à mesure. Au contraire, `compteur` est sur l'*automatic storage* et donc disparaît à la fin de chaque appel à *fonction* et sa valeur reste toujours à 1.

```
#include <iostream>

void fonction()
{
    static int compteur_statique { 0 };
    int compteur { 0 };

    ++compteur_statique;
    ++compteur;

    std::cout << "Le compteur statique vaut maintenant "
                << compteur_statique << std::endl;
    std::cout << "Le compteur vaut maintenant "
                << compteur << std::endl;
}

int main()
{
    fonction();
    fonction();
    fonction();

    return 0;
}
```

```
Le compteur statique vaut maintenant 1
Le compteur vaut maintenant 1
Le compteur statique vaut maintenant 2
Le compteur vaut maintenant 1
Le compteur statique vaut maintenant 3
Le compteur vaut maintenant 1
```

Le dynamic storage

Ce troisième type de stockage permet une gestion explicite de l'allocation et de la libération de la mémoire (on parle d'ailleurs d'*allocation dynamique*). Ce n'est plus le compilateur qui gère implicitement la durée de vie de la variable stockée. Par exemple, quand on crée un tableau redimensionnable avec `std::vector`, la zone mémoire qui contient notre tableau est dans le *dynamic storage*.

Jusqu'ici, nous n'avions nous-mêmes jamais manipulés explicitement d'éléments stockés dedans. Cette complexité était masquée par les différentes classes de la bibliothèque standard, auxquelles nous faisons appel.

C'est à quelle adresse ?

Qui dit emplacement en mémoire dit en fait *adresse*. Chaque objet créé a son adresse propre, qu'on peut récupérer en C++ en utilisant l'opérateur `&` devant un identificateur. Attention, cette syntaxe n'a rien à voir avec les références !

```
#include <iostream>

void fonction()
{
    static int compteur_statique { 0 };
    std::cout << "Adresse de 'compteur_statique' : "
               << &compteur_statique << "\n";
}

int main()
{
    int const variable_locale { 3 };
    // Adresse d'une variable locale.
    std::cout << "Adresse de 'variable_locale' : "
               << &variable_locale << "\n";
    // Les fonctions aussi ont des adresses.
    std::cout << "Adresse de 'fonction' : "
               << &fonction << "\n";

    fonction();
    return 0;
}
```

```
Adresse de 'variable_locale' : 008FF750
Adresse de 'fonction' : 00FB3DED
Adresse de 'compteur_statique' : 00FE4618
```

Les valeurs que vous obtiendrez seront différentes, mais elles ne sont pas importantes en elles-mêmes. Ce qu'il convient de retenir, c'est que chaque élément a son adresse propre. Et tout comme C++ fournit des types pour manipuler des entiers ou des réels, il fournit aussi un type pour manipuler les adresses, qu'on appelle *pointeur*.

29.2. Les indirections

Une *indirection* est une référence à une donnée stockée quelque part, qui permet de la désigner indirectement. Par exemple, "Numéro 6" est une indirection qui fait référence à un [prisonnier bien connu](#). Le surnom "mon cœur" fait référence à la personne avec qui vous partagez votre vie. Pour un policier regardant la base de données des infractions routières, votre plaque d'immatriculation est une indirection faisant référence à votre

voiture, ses caractéristiques, son propriétaire (vous) et ses éventuelles amendes (pas bien !), etc.

D'un point de vue C++, nous connaissons déjà plusieurs types indirections.

- les *itérateurs*, qui sont des indirections vers une donnée qui fait partie d'une séquence (tableau, chaîne de caractères, etc.). On peut, en fonction de leur nature, lire la donnée en question, la modifier, se déplacer au sein de la séquence en avant, en arrière, à une position arbitraire, etc. ;
- les *références*, qui sont des indirections permanentes vers une information en mémoire, là encore qu'on peut lire, voire modifier si la référence n'est pas `const` ;
- les *pointeurs*, que nous allons voir maintenant.

Un pointeur sur une variable de type `T` se déclare avec la syntaxe `T*`. Pour l'initialiser, on peut soit lui donner l'adresse d'un objet avec l'esperluette `&`, soit `nullptr`, qui signifie "adresse nulle" et qu'on utilise quand on veut dire qu'un pointeur ne pointe sur rien. Une fois qu'un pointeur est initialisé avec une adresse valide, on peut accéder à la valeur qu'il pointe en utilisant l'opérateur `*`, comme pour les itérateurs. On parle de *déréférencement*.

```
#include <iostream>

int main()
{
    int variable_modifiable { 3 };
    // Le pointeur est maintenant initialisé
    // avec l'adresse de 'variable_modifiable'.
    int* pointeur { &variable_modifiable };

    std::cout << "Adresse de 'variable_modifiable' : "
               << &variable_modifiable << "\n";
    std::cout << "Valeur du pointeur : "
               << pointeur << "\n";
    // Déréférencement pour accéder à la valeur pointée.
    std::cout << "Valeur pointée : "
               << *pointeur << "\n";

    // Comme pour une référence modifiable, on
    // peut changer la valeur derrière l'indirection.
    *pointeur = 98;

    std::cout << "variable_modifiable : "
               << variable_modifiable << "\n";
    std::cout << "Valeur pointée : "
               << *pointeur << "\n";

    int autre_variable { -7 };
```

```
// Contrairement à une référence qui est permanente,
// on peut changer la valeur sur laquelle pointer.
pointeur = &autre_variable;
std::cout << "Valeur pointée après modification : "
  << *pointeur << "\n";

return 0;
}
```

```
Adresse de 'variable_modifiable' : 0x7ffd18e32f98
Valeur du pointeur : 0x7ffd18e32f98
Valeur pointée : 3
variable_modifiable : 98
Valeur pointée : 98
Valeur pointée après modification : -7
```

Mais des indirections, il en existe potentiellement plein d'autres. D'ailleurs, les informations auxquelles elles font référence ne sont *pas toujours en mémoire*. Ce peut être un fichier présent sur le disque dur (auquel on accède avec `std::ifstream` et `std::ofstream`, indirections vues lors du [chapitre sur les flux](#)), une base de données, une connexion à un serveur quelque part sur Internet, etc.

Parmi ces nouvelles indirections, certaines le sont sur des ressources, qu'il faut acquérir avant de pouvoir les manipuler. Pour pouvoir ouvrir un livre et le lire, il faut bien d'abord le saisir. Pour repeindre le garage, il faut bien d'abord prendre le pot de peinture sur l'étagère. Pour faire la vaisselle, il faut d'abord enfiler une paire de gants.

En C++, pour ce faire, nous allons avoir besoin d'un nouvel outil : les *handles*.

29.3. Les ressources et les handles

Un *handle*, qu'on pourrait traduire par *poignée*, est une indirection vers une ressource qui a été acquise, afin de permettre la manipulation de cette dernière. Ce sont généralement des fonctions d'acquisition au nom évocateur (ouvrir, acquérir, créer, etc.) qui nous renvoient des handles.

Par exemple, si vous examinez la documentation de `std::ifstream`, vous noterez l'existence d'une fonction membre `open`, qui s'occupe d'ouvrir en lecture le fichier demandé. De même, pour `std::vector`, il existe la fonction membre `reserve`, qui s'occupe d'acquérir de la mémoire pour stocker nos éléments. Nous ne nous y étions jamais intéressés avant, car l'acquisition de la ressource était faite directement dans le constructeur, ce qui nous masquait la complexité sous-jacente.

Ensuite, ce que l'on peut faire avec un handle dépend de la sémantique de la ressource associée. Certes tout n'est que nombres pour le processeur d'un ordinateur, mais de

notre point de vue nous avons des possibilités différentes. Est-ce dans un fichier que nous allons lire et/ou écrire des informations ? Est-ce à un périphérique que nous voulons envoyer des signaux ? Est-ce une portion de mémoire ?

Certains de ces handles seront en réalité des indirections vers des zones de mémoire allouées dynamiquement par le programme (donc dans le *dynamic storage*). C'est le cas pour `std::vector` par exemple, dont la taille occupée en mémoire peut être amenée à changer en fonction des données qu'on y stocke. Pour manipuler cette zone mémoire, il est fait en interne utilisation d'un pointeur. Une référence ne conviendrait pas, car l'adresse de la-dite zone peut changer, voire devenir invalide (si on vide le tableau), alors qu'une référence est permanente.

Dans tous les cas, peu importe la façon dont la ressource est acquise et comment elle est manipulée, une fois qu'on a fini, il faut penser à la restituer. Dans le cas contraire, nous avons ce qu'on appelle une *fuite*, ce qui veut potentiellement dire saturation de la mémoire (avec potentiellement crash total de la machine dans le pire des cas), impossibilité de se reconnecter ultérieurement à une base de données (car la connexion actuelle reste ouverte indéfiniment), etc.

De même, il faut penser à ne restituer la ressource acquise *qu'une seule et unique fois*. La norme C++ précise que tenter de libérer une ressource déjà restituée est un comportement indéterminé. Pour rappel, tout peut potentiellement arriver. Autant il faut éviter comme la peste les fuites mémoires pour ne pas saturer la mémoire, autant les suppressions multiples font office de choléra...

29.4. Restitution garantie 100% satisfaction

Le travail manuel ne paie pas toujours...

Nous avons besoin d'un mécanisme de restitution déterministe, c'est-à-dire qui offre la garantie qu'une ressource acquise sera bien libérée une seule et unique fois. Dans des exemples très simples, c'est possible de faire ça manuellement, mais dès que la complexité augmente, ça devient infernal, voire impossible. C'est ce que nous allons démontrer.

Le code suivant a été imaginé, mais il ressemble à ce qu'on écrirait si on utilisait des bibliothèques graphiques comme [SDL](#) ou [OpenGL](#). Sur le principe, il est très simple, puisqu'on se contente de créer une fenêtre puis d'afficher une image dedans.

```

int main()
{
    // D'abord on doit initialiser la bibliothèque
    // de manière globale.
    initialisation_bibliotheque_graphique();

    // On crée ensuite une fenêtre dans laquelle
    // apparaîtra une image.
    Fenetre* fenetre { creation_fenetre() };

    // On acquiert l'image pour l'afficher.
    Image* image { chargement_image("SuperbeImage.png") };
    afficher_image(fenetre, image);

    // On finit le programme et on s'en va.
    // On libère tout ce qu'il faut.
    detruire_image(image);
    detruire_fenetre(fenetre);
    desallocation_bibliotheque_graphique();

    return 0;
}

```

Pourtant, il n'est pas du tout sécurisé, car il ne fait aucune vérification d'erreur. Nulle part on ne regarde si une ressource a bien été initialisée et en cas d'erreur on ne pense pas à libérer les ressources déjà acquises. Il va donc falloir penser à tous les cas d'erreurs et bien les couvrir.

```

int main()
{
    // D'abord on doit initialiser la bibliothèque de manière globale.
    try
    {
        // Il faut gérer le cas où on ne peut même pas utiliser
        // la bibliothèque. Peut-être qu'elle n'est pas utilisable
        // sur notre système ou bien manque-t-il des dépendances.
        initialisation_bibliotheque_graphique();
    }
    catch (std::exception const& e)
    {
        std::cout << e.what() << std::endl;
        // Toute valeur différente de 0 signifie
        // qu'on a rencontré une erreur.
        return 1;
    }

    // On crée ensuite une fenêtre dans laquelle
    // apparaîtra une image.
    Fenetre* fenetre { nullptr };
    try
    {
        fenetre = creation_fenetre();
    }
}

```

```

    }
    catch (std::exception const& e)
    {
        std::cout << e.what() << std::endl;
        desallocation_bibliotheque_graphique();
        return 1;
    }

    Image* image { nullptr };

    try
    {
        image = chargement_image("SuperbeImage.png");
    }
    catch (std::exception const& e)
    {
        std::cout << e.what() << std::endl;
        detruire_fenetre(fenetre);
        desallocation_bibliotheque_graphique();
        return 1;
    }

    afficher_image(fenetre, image);

    // On finit le programme et on s'en va.
    // On libère tout ce qu'il faut.
    detruire_image(image);
    detruire_fenetre(fenetre);
    desallocation_bibliotheque_graphique();

    return 0;
}

```

Ouch ! Le code est déjà bien plus lourd et bien moins lisible. Et le pire, c'est qu'il ne couvre pas toutes les possibilités. En effet, `afficher_image` pourrait aussi lancer une exception, qui ne serait alors pas rattrapée et donc fuite de mémoire. Pareil pour `detruire_image`, `detruire_fenetre` et `desallocation_bibliotheque_graphique`. Autant dire qu'on a affaire à un cauchemar de code. Il nous faut une autre solution.

Resource Acquisition Is Initialization

L'idée est que chaque ressource allouée doit être restituée. Si on reprend l'exemple de la ressource gants de vaisselle et qu'on complexifie avec la ressource éponge, alors on doit s'assurer dans le scénario suivant que peu importe le problème qui se pose, à la fin, l'éponge et les gants doivent être rincés et rangés.

Mettre les gants.

Attraper une assiette.

```

// Et si l'éponge a disparu?
Attraper l'éponge.

// Et s'il n'y plus de savon?
S'assurer qu'il y a du liquide vaisselle sur l'éponge.

// Et si l'assiette glisse ?
Savonner l'assiette.

// Et s'il n'y a plus d'eau claire?
Reposer l'éponge.

Rincer l'assiette.

// Et si le téléphone sonne ?
// Et si on frappe à la porte ?
// Etc.

```

Chaque langage apporte sa solution à cet épineux problème. En C, où les exceptions n'existent pas, on fait un grand usage de `goto`. En Python, il y a `with` ; en C#, il y a `using` ; en Java, c'est le mécanisme dit du *try-with-resources*. Dans de nombreux langages (JavaScript, PHP, etc.), on a aussi `finally`, des instructions qui s'exécutent peu importe si le `try` lance ou non une exception. La solution qui vient avec C++ s'appelle RAII, un acronyme en anglais qu'on pourrait traduire par *acquisition de ressources lors de l'initialisation*. Avec ce principe, on ne surveille plus tous les chemins possibles d'exécution (complexe voire impossible, comme vu plus haut), mais plutôt on *surveille chaque ressource*.

Pour y arriver, on va encapsuler l'acquisition et la libération de la ressource (qui n'est pas dans l'*automatic storage* et donc qui n'est pas restituée automatiquement) au sein d'une classe dédiée qu'on appelle *gestionnaire* ou encore *capsule RAII*. Celle-ci sera stockée dans l'*automatic storage* et s'occupera d'acquérir la ressource dans son constructeur et de la libérer dans son *destructeur*. La norme C++ garantissant que le destructeur d'un objet stocké dans l'*automatic storage* sera appelé, peu importe si le programme s'est correctement déroulé ou si une exception ou une autre erreur est survenue, on règle notre problème.

Concrètement, les classes `std::vector` et `std::string`, que nous manipulons depuis le début de cet ouvrage, sont des capsules RAII. En effet, le premier alloue et libère le tronçon de mémoire où est stocké le tableau et le deuxième fait de même avec le tronçon de mémoire où est stocké le texte. Et si vous lisez la documentation, vous verrez par exemple une page dédiée au [destructeur de `std::vector`](#).

Vous noterez que cette fonction spéciale (dans laquelle sont réalisées les opérations de libération) commence par un tilde `~` et porte le même nom que la classe dans laquelle elle est définie. D'autre part, elle est `noexcept`. En effet, un destructeur ne devrait jamais lancer d'exception, autrement c'est un comportement indéterminé.

```

class A
{
public:
    // On définit explicitement le destructeur de la classe A.
    ~A() noexcept = default;
};

```

L'exemple corrigé

Avec les nouvelles notions que nous connaissons, nous sommes maintenant en mesure de corriger notre code d'exemple plus haut pour garantir la bonne libération de toutes les ressources. Le code est bien plus simple et ressemble davantage à ce qu'on écrirait avec SFML, la bibliothèque graphique que vous connaissez. Cette dernière étant écrite en C++ (à la différence de la SDL ou d'OpenGL écrites en C), elle profite de ce mécanisme de RAII et c'est la raison pour laquelle, dans l'exemple du [chapitre sur les bibliothèques externes](#), vous ne voyez pas une ligne de libération de ressource.

```

// Capsule RAII pour la bibliothèque graphique.
class BibliothequeGraphique
{
public:
    BibliothequeGraphique();

    // Ici, on effectuera toutes les opérations
    // nécessaires pour libérer la mémoire
    // utilisée par la bibliothèque.
    ~BibliothequeGraphique() noexcept;
};

BibliothequeGraphique::BibliothequeGraphique()
{
    initialisation_bibliotheque_graphique();
}

BibliothequeGraphique::~BibliothequeGraphique() noexcept
{
    desallocation_bibliotheque_graphique();
}

// Capsule RAII responsable d'acquérir et de
// libérer une ressource de type Fenetre.
class Fenetre
{
public:
    Fenetre();

    void afficher_image(Image const& image)
    {
    }
}

```

```

        // Ici, on effectuera toutes les opérations
        // nécessaires pour libérer la mémoire
        // utilisée par une instance de Fenetre.
        ~Fenetre() noexcept;

private:
    FenetreTypeInterne* m_fenetre;
};

Fenetre::Fenetre()
{
    m_fenetre = creation_fenetre();
}

Fenetre::~Fenetre() noexcept
{
    detruire_fenetre(fenetre);
}

// Capsule RAII responsable d'acquérir et de
// libérer une ressource de type Image.
class Image
{
public:
    Image(std::string const& chemin);

    // Ici, on effectuera toutes les opérations
    // nécessaires pour libérer la mémoire
    // utilisée par une instance d'Image.
    ~Image() noexcept;

private:
    ImageTypeInterne* m_image;
};

Image::Image()
{
    m_image = chargement_image(chemin);
}

Image::~Image() noexcept
{
    detruire_image(image);
}

int main()
{
    BibliothequeGraphique bibliotheque {};
    Fenetre fenetre {};
    Image image { "SuperbeImage.png" };

    // Même si une exception est lancée ci-après,
    // toutes les ressources allouées seront libérées.
    fenetre.afficher_image(image);

    // Pas besoin de libérer explicitement la mémoire

```



```
// dans le cas où tout va bien non plus, car les
// différents destructeurs s'en chargent.
return 0;
}
```

29.5. `std::unique_ptr` — pour surveiller la mémoire

Mais quel rôle ce pointeur joue-t-il ?

Revenons un instant sur les pointeurs. Nous avons dit que cette indirection, faite pour stocker une adresse en mémoire, peut être utilisée pour acquérir une ressource dynamiquement. Historiquement, c'est le plus ancien type d'indirection qui existe en C++, hérité directement de son prédécesseur le langage C. C'est aussi celui qui pose le plus de problème, puisqu'il peut avoir plusieurs rôles. Il peut tout aussi bien désigner :

- une zone mémoire qui nous appartient (`Image* image { chargement_image("SuperbeImage.png") }`);
- une zone que l'on peut regarder, sans qu'on en soit propriétaire (`int* pointeur { &variable }`);
- le début d'une zone sur laquelle on peut itérer (pointeur interne à `std::vector`).

Cette confusion fait que beaucoup voient les pointeurs d'un mauvais œil. Heureusement, le C++ moderne apporte une solution en permettant de mieux définir les responsabilités dans notre code.

Un seul propriétaire...

Depuis C++11, nous avons une capsule RAII conçue exprès pour servir d'indirection vers une zone mémoire dont nous sommes les seuls et uniques propriétaires. Il s'agit de `std::unique_ptr`, disponible dans le fichier d'en-tête `<memory>`. On parle de *pointeurs intelligents*, terme à opposer aux *pointeurs nus*, qui désignent les pointeurs historiques avec l'étoile `*`.

Dans le cas où l'on ne pointe sur rien, quand on n'a aucune ressource à acquérir, on peut initialiser notre pointeur intelligent à `nullptr`. Sinon, quand on a besoin d'un bloc de mémoire, on utilise la fonction spéciale `std::make_unique`, à qui on passe les paramètres pour initialiser la ressource gérée.

```

#include <memory>

struct A
{
    int entier;
    double reel;
};

int main()
{
    // Sur un type natif.
    auto handle_int { std::make_unique<int>(42) };
    // Sur un type que nous avons nous-mêmes défini.
    auto handle_a { std::make_unique<A>(42, 3.1415) };

    // La mémoire est automatiquement libérée,
    // 'handle_a' ne gère plus rien.
    handle_int = nullptr;

    // 'handle_a' est de nouveau propriétaire
    // d'une ressource de type 'int'.
    handle_int = std::make_unique<int>(0);

    return 0;
}

```

***Note** > Exceptionnellement, nous n'utiliserons pas le constructeur de `std::unique_ptr` mais la fonction `std::make_unique`. Les raisons de cette particularité sortent du cadre de ce livre. Pour l'instant, prenez cette règle comme absolue, vous pourrez par la suite creuser de votre côté.*

La syntaxe pour accéder à la valeur pointée va vous rappeler des souvenirs, puisqu'elle est identique à celle des itérateurs. Ainsi, il faut utiliser l'opérateur `*` pour obtenir l'accès à l'objet sous-jacent et l'opérateur `->` pour l'accès aux éléments de l'interface que cet objet expose.

```

#include <iostream>
#include <memory>

struct A
{
    void parler() const
    {
        std::cout << "Bonjour, je suis A.\n";
    }
};

int main()
{
    auto const pointeur_double { std::make_unique<double>(3.1415) };
    std::cout << *pointeur_double << std::endl;

    auto const pointeur_a { std::make_unique<A>() };
}

```

```

    pointeur_a ->parler();
    return 0;
}

```

```

3.1415
Bonjour, je suis A.

```

...et plusieurs observateurs

Quand on veut simplement observer une ressource qui ne nous appartient pas, nous devons utiliser des *indirections non-propriétaires*, c'est-à-dire qui ne s'occuperont pas de libérer cette ressource. Dans le monde des adresses mémoire, nous avons notamment le choix entre les références et les pointeurs nus.

Le premier choix est le plus immédiat. En effet, la référence se prête parfaitement à ce rôle. C'est typiquement la solution qu'on a toujours utilisée pour transmettre des objets dans des fonctions, sans passer par la copie. Cependant, les références présentent aussi des inconvénients. D'abord, *on ne peut pas les stocker dans un tableau*, c'est interdit par la norme. De plus, elles doivent forcément faire référence à un objet et ne peuvent donc pas servir pour désigner le vide.

Le deuxième choix que nous avons, les pointeurs nus, résout justement ce problème. Par exemple, dans le cadre d'un programme affichant des objets 3D à l'écran, on pourra au choix assigner une texture à un objet visible, ou bien lui enlever quand il sort de l'écran.

```

#include <string>
#include <unordered_map>

struct Texture
{
    // Toute la logique et les services
    // qu'on attend d'une texture.
};

struct Objet3D
{
    Texture* m_texture = nullptr;
};

int main()
{
    // Chargement des textures, etc.
    std::unordered_map<std::string, Texture> textures {
        charge_textures()
    };

    // Pas de texture de base,

```

```

// la balle est invisible.
Objet3D balle {};

// On utilise la texture 'balle bleue'.
// La balle est maintenant visible.
balle.m_texture = &textures["balle_bleue"];

// On peut changer autant qu'on veut.
// Il n'y a pas de risque de fuite, car
// 'm_texture' n'est qu'un observateur.
balle.m_texture = &textures["balle_rouge"];
balle.m_texture = nullptr;
balle.m_texture = &textures["balle_verte"];

return 0;
}

```

Pour rendre plus explicite l'utilisation des pointeurs nus comme de simples observateurs, il est possible de créer un *alias générique*, dont la syntaxe n'est pas sans rappeler celle utilisée pour les fonctions génériques.

```

template<typename T>
using ObserverPtr = T*;

struct Objet3D
{
    // Rôle explicite, pas propriétaire mais simple observateur.
    ObserverPtr<Texture> m_texture;
};

```

Liens entre propriétaire et observateurs

L'intérêt des observateurs serait nul s'ils ne pouvaient pas se lier à un propriétaire. Mais `std::unique_ptr` fournit tout ce qu'il faut. Pour obtenir une référence sur l'objet géré par un `std::unique_ptr`, il suffit de le déréférencer avec `*`. Pour un pointeur nu, on peut faire appel à la fonction membre `get`.

```

#include <memory>

template<typename T>
using ObserverPtr = T*;

int main()
{
    // Handle propriétaire.
    auto handle_proprietaire { std::make_unique<int>(42) };

    // Observateur à l'aide d'une référence.
    int& reference { *handle_proprietaire };
}

```

```
// Observateur à l'aide d'un pointeur.
ObserverPtr<int> pointeur { handle_proprietaire.get() };

return 0;
}
```

Transfert de responsabilité

La classe `std::unique_ptr` désactive le constructeur et l'opérateur d'affectation de copie, donc il est impossible d'écrire ce qui suit.

```
auto pointeur { std::make_unique<int>(42) };
auto copie { pointeur };
```

La raison est la suivante : puisque `std::unique_ptr` est une capsule RAII gérant une ressource allouée dynamiquement, si on pouvait bêtement le copier, alors on se retrouverait avec plusieurs propriétaires d'une même ressource. Quand vient le moment de la libérer, le premier `std::unique_ptr` s'exécute, mais le deuxième tentera alors de désallouer une ressource déjà libérée... Heureusement, plutôt que de copier la ressource, il suffit simplement de la transférer.

Imaginons que nous voulions représenter une voiture en C++. Celle-ci contient en interne un moteur. Ce dernier peut être de plusieurs types différents : V8, V6, R4, etc. Et bien souvent, quand on emmène la voiture à la casse, le moteur est détruit avec. Mais rien ne nous empêche, avant que la voiture ne parte à la broyeuse, de le récupérer s'il est en bon état et de l'installer dans un autre véhicule.

Si d'un point de vue C++ notre classe `Voiture` contient un `std::unique_ptr<Moteur>`, comment représente-t-on le fait d'enlever le moteur d'une voiture `v1` pour l'installer dans une voiture `v2` ? En d'autres termes, comment transfère-t-on la responsabilité de notre ressource `m_moteur` ?

Ce besoin se résout grâce à la fonction `std::move` (disponible dans le fichier d'en-tête `<utility>`). Elle indique au compilateur qu'il peut réaliser un transfert de propriété sur l'objet passé en argument.

```
#include <memory>
#include <utility>

class Moteur
{
    // ...
};
```

```

class V6 : public Moteur
{
};

class V8 : public Moteur
{
};

struct Voiture
{
    // public pour l'exemple.
    std::unique_ptr<Moteur> m_moteur;
};

void changer_moteur(Voiture& donneuse, Voiture& receveuse)
{
    receveuse.m_moteur = std::move(donneuse.m_moteur);
}

int main()
{
    // N'a plus de moteur.
    Voiture a_reparer { nullptr };
    // Le V8 de cette voiture est encore bon.
    Voiture epave { std::make_unique<V8>() };

    changer_moteur(epave, a_reparer);
    // La voiture à réparer a maintenant
    // un V8 qui ronronne.
    // L'épave peut être broyée sans regret.
    return 0;
}

```

Une fois son contenu déplacé, le handle `m_moteur` de la voiture `epave` vaut `nullptr` et n'est plus responsable du moteur. Il n'y a donc aucun problème de double libération de la mémoire.

Plusieurs propriétaires, plusieurs observateurs

Il existe une autre stratégie, qui consiste à partager la gestion d'une ressource entre plusieurs propriétaires. On ne détaillera pas cette approche dans le cadre de ce livre, car l'utilisation de `std::unique_ptr` permet d'avoir une architecture plus simple et claire, qui suffit dans la majorité des cas, ce qui en fait la solution privilégiée.

Pour les plus curieux, vous pouvez consulter la documentation de `std::shared_ptr` et de `std::weak_ptr`, qui s'utilisent de concert.

29.6. Les entités et les handles

Maintenant que nous connaissons le principe des handles et des indirections, avec la logique qui va derrière, nous allons revenir à nos entités. Alors qu'on peut manipuler des sous-types comme un type de base, à cause du problème posé par le *slicing*, nous ne pouvons malheureusement pas les stocker dans un tableau du type de base.

```
struct A
{
    // On laisse volontairement de côté
    // l'interdiction de copie pour l'exemple.
    int entier;
};

struct B : public A
{
    double reel;
};

// Chaque élément du tableau occupe sizeof(A)
// en mémoire. Comme sizeof(B) > sizeof(A),
// on a encore le problème de slicing.
std::vector<A> tableau {};
```

Si la phrase précédente est écrite au passé, c'est que nous avons maintenant une solution. On peut en effet créer un tableau de `std::unique_ptr`. En utilisant une indirection, on permet que chaque élément du tableau pointe vers une zone mémoire *pile de la bonne taille* pour le sous-type réel. De plus, la sémantique est claire : ce tableau est propriétaire des ressources manipulées.

Le code suivant reprend en simplifiant un peu l'exemple d'entreprise vu au chapitre précédent.

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

class Personne
{
public:
    Personne() = delete;
    ~Personne() noexcept = default;

    Personne(std::string const& prenom,
             std::string const& nom,
             int salaire)
        : m_prenom(prenom), m_nom(nom), m_salaire(salaire)
    {
```

```

    }

    Personne(Personne const& copie) = delete;
    Personne& operator=(Personne const& autre_personne) = delete;

    virtual void travailler() const = 0;

private:
    std::string m_nom;
    std::string m_prenom;
    int m_salaire;
};

class Directeur : public Personne
{
public:
    using Personne::Personne;

    ~Directeur() noexcept = default;

    Directeur(std::string const& prenom,
               std::string const& nom,
               int salaire,
               std::string const& service)
        : Personne(prenom, nom, salaire), m_service(service)
    {

    }

    void travailler() const override
    {
        std::cout << "J'assiste à des réunions.\n";
    }

private:
    std::string m_service;
};

enum class EquipementProtection
{
    Gants,
    Casque,
    Chassures,
    Lunettes,
    Masque
};

using EquipementsProtection = std::vector<EquipementProtection>;

class Ouvrier : public Personne
{
public:
    using Personne::Personne;

```



```

~Ouvrier() noexcept = default;

Ouvrier(std::string const& prenom,
        std::string const& nom,
        int salaire,
        EquipementsProtection const& epi)
    : Personne(prenom, nom, salaire), m_epi(epi)
{
}

void travailler() const override
{
    std::cout << "Je travaille avec mes outils.\n";
}

private:
    EquipementsProtection m_epi;
};

using Entreprise = std::vector<std::unique_ptr<Personne>>;

int main()
{
    std::cout << sizeof(Personne) << std::endl;
    std::cout << sizeof(Directeur) << std::endl;
    std::cout << sizeof(Ouvrier) << std::endl;

    Entreprise usine {};

    usine.push_back(std::make_unique<Directeur>(
        "Eting",
        "Mark",
        8000,
        "Marketing"
    ));

    std::vector<EquipementProtection> const epi_soudeur {
        EquipementProtection::Gants,
        EquipementProtection::Casque,
        EquipementProtection::Lunettes,
        EquipementProtection::Chaussures
    };

    usine.push_back(std::make_unique<Ouvrier>(
        "Le soudeur fou",
        "Bob",
        2300,
        epi_soudeur
    ));

    // Nous sommes en mesure d'avoir une collection
    // d'entités polymorphiques.
    for (auto const& personne : usine)
    {

```

```

        std::cout << "\n";
        personne->travailler();
    }

    return 0;
}

```

29.7. Un problème de destruction

Le [polymorphisme d'inclusion](#) nous a appris qu'une indirection sur une instance de type `T` peut en réalité cacher une instance d'un sous-type `U`. C'est le cas aussi quand on utilise `std::unique_ptr` et c'est tout l'intérêt des objets polymorphes. Cependant, cela renferme un gros problème. En effet, quand notre entité est libérée, peu importe son sous-type réel, elle est traitée comme une instance de la classe de base. Or, en C++, détruire un objet d'une classe dérivée comme un objet de la classe de base est un comportement indéterminé ! Le plus probable dans ce cas est une fuite de mémoire due à une destruction partielle de l'objet — seule la partie commune avec la classe de base étant détruite — mais tout peut théoriquement arriver.

La solution pour permettre la résolution dynamique des liens ne devrait pas vous surprendre, puisqu'il suffit d'utiliser `virtual`, ce qui aura comme conséquence d'appeler aussi le destructeur de la classe enfant. L'exemple suivant nous illustre bien le problème et sa solution.

```

#include <iostream>
#include <memory>

class A
{
public:
    ~A() noexcept;
};

A::~A() noexcept
{
    std::cout << "Destructeur de A non-virtuel.\n";
}

class B : public A
{
public:
    ~B() noexcept;
};

B::~B() noexcept
{
    std::cout << "Destructeur de B non-appelé.\n";
}

```

```

class C
{
public:
    virtual ~C() noexcept;
};

C::~C() noexcept
{
    std::cout << "Destructeur de C virtuel.\n";
}

class D : public C
{
public:
    ~D() noexcept override;
};

D::~D() noexcept
{
    std::cout << "Destructeur de D appelé.\n";
}

int main()
{
    // Comportement indéterminé au moment de la libération car partielle.
    std::unique_ptr<A> const objet_a { std::make_unique<B>() };

    // Libération complète et correcte.
    std::unique_ptr<C> const objet_c { std::make_unique<D>() };

    return 0;
}

```

```

Destructeur de D appelé.
Destructeur de C virtuel.
Destructeur de A non-virtuel.

```



Le destructeur devrait-il toujours être virtuel, pour parer à tous les cas ?

Eh bien non, pas forcément. Ça n'a d'intérêt qu'à partir du moment où l'on sait que des instances de notre classe seront manipulées de manière polymorphique à travers une indirection sur la classe de base. Si l'on n'est pas dans ce cas, ou bien si l'on est dans le cas de la sémantique de valeur (qui n'utilise pas l'héritage), pas besoin de déclarer le destructeur comme étant virtuel.

29.8. Les pointeurs nus, on oublie !

Nous l'avons vu, les pointeurs rendent de grands services, mais le cumul des rôles rend difficile la bonne compréhension de leur sémantique. Heureusement, le C++ moderne contient plusieurs outils pour nous faciliter la tâche. Nous avons vu `std::unique_ptr`, qui remplace le pointeur vers une zone mémoire à surveiller. Nous avons aussi défini un alias pour remplacer le pointeur nu quand il n'est que simple observateur. Voyons encore quelques cas.

`std::function` — un remplaçant pour manipuler des fonctions

En C et avant que la norme C++11 ne sorte, on était obligé de passer par des pointeurs pour manipuler des fonctions, par exemple pour passer une fonction en argument à une autre. Maintenant, la bibliothèque standard inclut un type plus pratique et plus intelligent, capable de gérer les fonctions libres, les fonctions membres, les lambdas, etc : `std::function`.

Par exemple, quand on développe un programme avec une interface graphique, il est très courant que celle-ci génère des événements, en réaction à un clic sur la croix pour fermer la fenêtre, du texte entré dans un encart, etc. Le composant graphique (bouton, champ texte, etc.) peut alors s'abonner à ces événements. On lui passe simplement une fonction qu'il va exécuter à chaque fois qu'un événement auquel il est abonné est levé.

```
#include <functional>
#include <iostream>

struct Bouton
{
    void onClick(std::function<void()> callback);
};

void Bouton::onClick(std::function<void()> callback)
{
    std::cout << "Reçu un événement de clic.\n";
    callback();
}

void fonction()
{
    std::cout << "Clic sur le bouton avec une fonction libre.\n";
}

int main()
{
    Bouton bouton {};

    bouton.onClick([]() -> void {
```

```

    std::cout << "Clic sur le bouton avec une lambda.\n";
  });

  bouton.onClick(fonction);
  return 0;
}

```

```

Reçu un évènement de clic.
Clic sur le bouton avec une lambda.
Reçu un évènement de clic.
Clic sur le bouton avec une fonction libre.

```

std::optional — un remplaçant pour valeur optionnelle

Quand une fonction retourne un pointeur nu, ce peut être aussi pour gérer le cas d'une *valeur optionnelle*. On pourrait imaginer une fonction qui convertit une chaîne de caractères en entier, mais qui prend soin de ne pas utiliser d'exception. Il nous faut alors une valeur à utiliser en cas d'erreur, qui ne puisse pas être confondue avec une valeur valable comme 0, -1, etc.

Note > Cet exemple s'inspire de la fonction `int.TryParse` en C#.

Ça tombe bien, la bibliothèque standard propose une alternative nommé `std::optional`, qu'on utilise quand la valeur manipulée est stockée dans l'*automatic storage*. Le concept de "rien" est représentée par la valeur spéciale `std::nullopt`. L'exemple suivant nous montre comment on pourrait s'y prendre.

```

#include <exception>
#include <iostream>
#include <optional>
#include <string>

std::optional<int> chaine_vers_entier(std::string const & chaine)
noexcept
{
    try
    {
        return std::stoi(chaine);
    }
    catch (std::exception const & exception)
    {
        return std::nullopt;
    }
}

int main()
{
    auto const conversion_reussie { chaine_vers_entier("100") };
    std::cout << conversion_reussie.value_or(-1) << "\n";
}

```

```

auto const conversion_echouee { chaine_vers_entier("ABCDEF6") };
std::cout << conversion_echouee.value_or(-1) << "\n";
return 0;
}

```

```

100
-1

```

29.9. De l'importance de prendre ses responsabilités

Ce chapitre a pu vous paraître difficile et c'est normal. Beaucoup de nouvelles notions ont été introduites, qui sont pour quelques-unes spécifiques à C++. Il était néanmoins important d'en parler, car elles sont étroitement liées à la qualité logicielle.

Quand nous écrivons des programmes, nous voulons que ceux-ci soient de qualité et aient le moins de bugs possibles. Cela implique notamment que les objets que nous manipulons soient valides. Nous voulons donc à tout prix éviter de manipuler un objet qui n'est pas encore initialisé ou qui est déjà détruit. De même, nous ne voulons pas libérer la mémoire si nous ne sommes pas le propriétaire de la ressource. Il faut donc maîtriser deux notions :

- l'*ownership*, c'est-à-dire qui est responsable de la ressource et de sa libération ;
- la *durée de vie* des objets, c'est-à-dire quand l'objet est créé et quand il est détruit.

Dans beaucoup de langages, cette complexité est masquée, mais pas en C++. D'où l'importance des outils que nous avons découverts dans ce chapitre et la réflexion associée, l'idéal étant d'avoir le moins de responsabilités possible quand on manipule une ressource.

La première solution à envisager, la plus simple et la plus facile, reste la *composition directe*, c'est-à-dire manipuler un objet par *valeur*, sans quelconque indirection. C'est la solution que nous utilisons depuis le début. Ainsi, dans le cadre de l'entreprise, les attributs `m_nom` et `m_prenom` de la classe `Personne` sont des exemples d'utilisation de la composition directe. L'utilisation de `std::string` nous facilite la vie et nous masque les opérations d'acquisition et de restitution de la mémoire.

Si l'on doit acquérir une ressource, on utilisera alors `std::unique_ptr` ou bien une autre capsule RAII dédiée (comme `std::fstream`, `std::vector`, etc.). À l'inverse, si nous ne sommes pas propriétaires, alors on fera notre choix parmi les différents observateurs qui existent : référence pour observer une valeur qui ne peut être nulle, pointeur nu (avec éventuellement un alias), etc.

Grâce à cette réflexion, nous pouvons écrire du code plus résistant et de meilleure qualité, puisque les durées de vie et les responsabilités des ressources sont clairement définies. Le code est aussi plus explicite, ce qui facilite sa compréhension.

29.10. Récapitulons !

- En fonction de ses propriétés, un objet peut se retrouver dans une zone ou une autre de stockage en mémoire.
- Ce qui se trouve dans l'*automatic storage* est géré automatiquement par le compilateur.
- Ce qui se trouve dans le *static storage* est alloué une seule et unique fois en tout début de programme et n'est libéré que quand celui-ci s'achève.
- Ce qui se trouve dans le *dynamic storage* est alloué et libéré explicitement, à la demande.
- Un pointeur est un type conçu pour stocker l'adresse d'un objet.
- Les indirections (dont font parties entre autres les références, les pointeurs et les itérateurs) permettent de désigner indirectement une donnée stockée quelque part.
- Les handles sont des indirections vers des ressources acquises.
- Quand une ressource a été acquise, il faut s'assurer qu'elle est libérée une seule et unique fois.
- La gestion manuelle des ressources est très complexe, voire impossible, à cause de tous les chemins d'exécution possibles.
- Le principe RAII indique qu'une ressource doit être acquise dans le constructeur et libérée dans le destructeur.
- Un pointeur peut avoir plusieurs rôles très différents, il faut donc des alternatives pour expliciter nos intentions.
- La bibliothèque standard fournit le handle `std::unique_ptr` dans le cas d'une ressource avec un unique propriétaire.
- On utilise les références quand on veut un observateur sur une ressource qui n'est jamais nulle.
- On utilise les pointeurs quand on veut un observateur sur une ressource optionnelle, ou bien pour manipuler un tableau d'observateurs.
- Afin de ne pas provoquer de comportement indéterminé au moment de la libération d'un objet manipulé à travers un pointeur sur son type parent, il faut que son destructeur soit virtuel.

- Ces notions d'*ownership* et de gestion de la durée de vie sont importantes pour écrire du code sans bug, d'autant plus qu'elles ne sont pas masquées en C++ comme dans d'autres langages.

30

La sémantique de déplacement

Sémantique. Le titre du chapitre peut laisser penser qu'on va aborder une troisième façon d'organiser nos classes, différente des deux autres, avec des incompatibilités notoires. Ce n'est pas le cas. Nous allons justement voir que la sémantique de déplacement se marie bien avec la sémantique de valeur et les capsules RAll.

30.1. T'as saisi la référence ?

Le besoin...

Continuons avec notre exemple d'entreprise, mais cette fois, nous n'allons plus utiliser un simple alias de `std::vector` mais bien un type à part, une classe `Entreprise`. Ainsi, nous pourrions choisir les services exposés, les invariants, etc. Seulement voilà, le code suivant (simplifié), ne compile pas.

```
#include <memory>
#include <vector>

class Employe
{
public:
    Employe(Employe const& origine) = delete;
    Employe& operator=(Employe const& autre_employe) = delete;

    virtual ~Employe() noexcept = default;

    // Autres services...
};

class Entreprise
{
public:
    void embaucher_employe_const_ref(std::unique_ptr<Employe> const&
employe);
    void embaucher_employe_ref(std::unique_ptr<Employe>& employe);

private:
    std::vector<std::unique_ptr<Employe>> m_employes;
};
```

```

void
Entreprise::embaucher_employe_const_ref(std::unique_ptr<Employe> const&
employe)
{
    m_employes.push_back(employe);
}

void Entreprise::embaucher_employe_ref(std::unique_ptr<Employe>&
employe)
{
    m_employes.push_back(employe);
}

int main()
{
    Entreprise entreprise {};

    // Les deux lignes suivantes posent problème.
    entreprise.embaucher_employe_const_ref(std::make_unique<Employe>());
    entreprise.embaucher_employe_ref(std::make_unique<Employe>());

    return 0;
}

```

```

[Visual Studio]
# Ligne 35
Erreur C2280
'std::unique_ptr<Employe, std::default_delete<Employe>>::unique_ptr(const
unique_ptr<Employe, std::default_delete<Employe>> &)' : tentative
de référencement d'une fonction supprimée

# Ligne 36.
Erreur E0461 la valeur initiale d'une référence non const doit être une
lvalue
Erreur C2664 'void
Entreprise::ajouter_employe_ref(std::unique_ptr<Employe, std::default_delete<
&)' : impossible de convertir l'argument 1 de
'std::unique_ptr<Employe, std::default_delete<Employe>>' en
'std::unique_ptr<Employe, std::default_delete<Employe>> &'

-----

[GCC]
# Ligne 35.
/opt/wandbox/gcc-head/include/c++/11.0.0/bits/stl_construct.h:96:17:
error: use of deleted function 'std::unique_ptr<_Tp,
_Dp>::unique_ptr(const std::unique_ptr<_Tp, _Dp>&) [with _Tp = Employe;
_Dp = std::default_delete<Employe>]'
   96 |         -> decltype(::new((void*)0) _Tp(std::declval< Args>()...))
      |         ^~~~~~
In file included from /opt/wandbox/gcc-head/include/c++/11.0.0/
memory:76,
               from prog.cc:2:
/opt/wandbox/gcc-head/include/c++/11.0.0/bits/unique_ptr.h:468:7: note:
declared here

```

```

468 |         unique_ptr(const unique_ptr&) = delete;
    |         ^~~~~~

# Ligne 36.
prog.cc: In function 'int main()':
prog.cc:36:61: error: cannot bind non-const lvalue reference
of type 'std::unique_ptr<Employee>&' to an rvalue of
type 'std::_MakeUniq<Employee>::__single_object' {aka
'std::unique_ptr<Employee>'}
   36 |         entreprise.ajouter_employe_ref(std::make_unique<Employee>());
      |                                         ~~~~~^~
prog.cc:26:65: note:   initializing argument 1 of 'void
Entreprise::ajouter_employe_ref(std::unique_ptr<Employee>&)'
   26 | void Entreprise::ajouter_employe_ref(std::unique_ptr<Employee>
      |                                         & employe)
      |                                         ~~~~~^~~~~~
-----

[Clang]
# Ligne 35.
/opt/wandbox/clang-head/include/c++/v1/__memory/base.h:103:16:
note: candidate template ignored: substitution failure [with _Tp =
std::unique_ptr<Employee>, _Args = <const std::unique_ptr<Employee>
&>]: call to implicitly-deleted copy constructor of
'std::unique_ptr<Employee>'
constexpr _Tp* construct_at(_Tp* __location, _Args&& ...__args) {

# Ligne 36.
prog.cc:36:36: error: non-const lvalue reference to type
'unique_ptr<...>' cannot bind to a temporary of type 'unique_ptr<...>'
   entreprise.ajouter_employe_ref(std::make_unique<Employee>());

prog.cc:26:65: note: passing argument to parameter 'employe' here
void Entreprise::ajouter_employe_ref(std::unique_ptr<Employee> & employe)

```

Peu importe qu'on essaye de passer une référence constante ou une référence modifiable, le code ne compile pas. Dans le premier cas, on tente de faire appel au constructeur de copie de `std::unique_ptr`, qui est supprimé. Dans le deuxième, le compilateur se plaint que "la valeur initiale d'une référence non `const` doit être une *lvalue*" (pour reprendre le message en français de Visual Studio).

...la cause...

En C++, une expression relève toujours d'une catégorie de valeur. Parmi celles-ci, on peut citer la *lvalue*. Comprendre comment fonctionnent ces catégories va nous aider à saisir le cœur du problème.

Une *lvalue* (acronyme de *left-value*) désigne, sans entrer dans les détails, une expression qui peut se trouver à gauche de l'opérateur d'affectation `=` et dont on peut prendre l'adresse avec l'opérateur `&`. Par exemple, dans le code suivant, `a` et `b` sont des *lvalues*.

```
// On peut lui assigner une valeur.
int a { 5 };
// On peut prendre son adresse.
int* b { &a };
```

À l'inverse, le littéral `5` n'est pas une *lvalue*, ce qui interdit notamment d'écrire `5 = a`. Cette expression est une *rvalue* (acronyme de *right-value*) ce qui désigne, là encore sans entrer dans les détails, les expressions dont on ne peut pas prendre l'adresse ou qui arrivent en fin de vie.

RÈGLES EXACTES DES CATÉGORIES DE VALEURS

En réalité, les règles sont plus complexes que ça. Ce que nous vous avons donné n'est qu'une version incomplète, mais qui suffit largement pour comprendre ce chapitre. Vous pourrez chercher par la suite les explications complètes.

Cette notion de durée de vie, dont on a déjà parlé au chapitre précédent, prend ici une plus grande importance encore. En effet, dans notre exemple, l'objet retourné par `std::make_unique` disparaît une fois que le `push_back` termine. Il a donc une durée de vie très courte. Comme ce n'est pas une *lvalue*, l'appel à `ajouter_employe_ref` échoue donc.

...et la solution

Avec C++11, la solution a été d'introduire un nouveau type de références, qu'on appelle les *rvalues references*, des références sur des *rvalues*. On les écrit avec deux esperluettes `&&` au lieu d'une. Il est alors possible de corriger le code précédent.

```
#include <memory>
#include <vector>

class Employe
{
};

class Entreprise
{
public:
    void ajouter_employe_ref(std::unique_ptr<Employe> && employee);

private:
    std::vector<std::unique_ptr<Employe>> m_employes;
```

```

};

void Entreprise::ajouter_employe_ref(std::unique_ptr<Employe> &&
    employe)
{
    // std::move permet le transfert de responsabilité
    // entre le paramètre et le std::vector.
    m_employes.push_back(std::move(employe));
}

int main()
{
    Entreprise entreprise {};
    entreprise.ajouter_employe_ref(std::make_unique<Employe>());

    return 0;
}

```

Ce sont ces fameuses *rvalues references* qui apparaissent quand on lit la documentation de certaines fonctions comme [push_back](#). Nous n'avions pas abordé le sujet au moment du [chapitre sur la documentation](#), car vous n'aviez alors pas les connaissances nécessaires pour comprendre.

Ce sont elles aussi qui apparaissent dans le constructeur de déplacement, que nous avons entrevu dans le [chapitre sur les constructeurs](#).

`std::move`, la mal-nommée

Justement, reparlons un peu de la fonction `std::move`, qu'on a vue dans le chapitre précédent [au moment de la transmission de propriété entre deux `unique\_ptr`](#) et qu'on utilise ici encore. En interne, elle fait appel à ce concept de *rvalues references* puisqu'elle se contente de transformer ce qu'on lui passe en référence sur *rvalue*, en utilisant `static_cast`.

```

#include <iostream>
#include <string>

std::string source() noexcept
{
    return "src";
}

void f(std::string && s) noexcept
{
    std::cout << "f(&&)\n";
}

int main()
{

```

```

std::string varloc { "v1" };
std::cout << "f(varloc) -> ";
// On utilise std::move, ce qui provoque un appel à f(&&).
f(std::move(varloc));

std::cout << "f(source()) -> ";
// Source est déjà une rvalue, le cast est donc inutile.
f(source());

// Mais du coup, rien n'a changé ?
std::cout << "varloc -> " << varloc << std::endl;

std::string un_puit { std::move(varloc) };
// Pourtant ici, on voit bien que le contenu
// de varloc a été déplacé, qu'il n'y a plus rien.
std::cout << "un_puit -> " << un_puit << std::endl;
std::cout << "varloc -> " << varloc << std::endl;

return 0;
}

```

```

f(varloc) -> f(&&)
f(source()) -> f(&&)
varloc -> v1
un_puit -> v1
varloc ->

```

Mais alors comment se passe la transmission de propriété entre deux `unique_ptr` si `std::move` n'est qu'un cast ? Et pourquoi y a-t-il déplacement dans le cas `std::string un_puit { std::move(varloc) }` ?

30.2. ♪ I like to move it, move it ♪

Pour preuve que ces *rvalues* *references* sont utiles, lisez la documentation de types que nous connaissons maintenant bien, comme `std::string` et `std::vector` et vous verrez qu'ils en font un large usage. Remarquez en particulier les surcharges du constructeur et de l'opérateur d'affectation pour prendre en compte les *rvalues*.

```

// std::string
basic_string(basic_string&& other) noexcept;
basic_string& operator=(basic_string&& str);

// std::vector
vector(const vector& other) noexcept;
vector& operator=(vector&& other);

```

Ce sont eux qui sont à la base de la sémantique de déplacement. Puisque nous avons affaire à des *rvalues*, à des objets temporaires qui sont voués à disparaître, autant ne

pas perdre de temps à les copier. On va plutôt les dépouiller de tous leurs attributs et les donner tels quels à l'objet en train d'être initialisé/modifié. Et pour bien comprendre en quoi c'est intéressant, examinons une mise en scène que l'on doit à Luc Hermitte.

Imaginons que vous vouliez transmettre un document très important, enfermé dans une mallette, à quelqu'un. Comment allez-vous faire ? Trouveriez-vous sensé d'acheter une deuxième mallette tout à fait semblable à la vôtre, de recopier le document à transmettre à la virgule près, puis de détruire la première mallette une fois fini ? Il serait bien plus simple de donner la mallette directement ou simplement transmettre son contenu.



D'accord, mais que devient l'objet une fois vidé de ses attributs ? Qu'en fait-on ?

La norme exige que l'objet qui a été déplacé reste valide, c'est-à-dire qu'il ne risque pas de provoquer de crash ou autre. Par contre, son contenu est indéfini. Il est peut-être inchangé, ou bien vide, ou bien peut contenir les codes de lancement d'un missile nucléaire. En fait, on ne peut faire que deux choses avec un objet qui a été déplacé : lui affecter une nouvelle valeur ou le détruire.

Note > Utiliser les attributs d'un objet qui a été déplacé aura un comportement indéterminé, en plus de ne pas avoir de sens !

Pour que nos propres types puissent eux aussi être déplaçables [movable en anglais], il suffit qu'ils implémentent le constructeur de déplacement et l'opérateur d'affectation par déplacement. Reprenons notre classe `Matrice` et simplifions-la un peu pour illustrer ce qui se passe.

```
#include <iostream>
#include <vector>

class Matrice
{
public:
    Matrice() = default;
    // Constructeur de copie.
    Matrice(Matrice const& matrice);
    // Constructeur de déplacement.
    Matrice(Matrice&& matrice) noexcept;

    Matrice& operator+=(Matrice const& autre);
};

Matrice::Matrice(Matrice const& matrice)
{
    std::cout << "Je suis dans le constructeur de copie de la classe
    Matrice.\n";
}
```

```

}

Matrice::Matrice(Matrice&& matrice) noexcept
{
    std::cout << "Je suis dans le constructeur de déplacement de la classe
    Matrice.\n";
}

Matrice& Matrice::operator+=(Matrice const& autre)
{
    std::cout << "3\n";
    std::cout << "Je suis dans l'opérateur += de la classe Matrice.\n";
    return *this;
}

Matrice operator+(Matrice lhs, Matrice const& rhs)
{
    std::cout << "2\n";
    lhs += rhs;
    std::cout << "4\n";
    return lhs;
}

int main()
{
    Matrice const m1{};
    Matrice const m2{};

    std::vector<Matrice> matrices{};
    std::cout << "1\n";
    matrices.push_back(m1 + m2);

    return 0;
}

```

```

1
Je suis dans le constructeur de copie de la classe Matrice.
2
3
Je suis dans l'opérateur += de la classe Matrice.
4
Je suis dans le constructeur de déplacement de la classe Matrice.
Je suis dans le constructeur de déplacement de la classe Matrice.

```

Dans cet exemple, le constructeur de déplacement n'effectue strictement rien, à part afficher un message. Mais comment donc s'implémentent concrètement ces opérations de déplacement ? En fait, nous n'avons pas à nous casser la tête à les écrire à la main. En effet, de même que pour la copie, il est possible de demander au compilateur de générer pour nous ces opérations.

```
Matrice(Matrice&& matrice) noexcept = default;
```



```
Matrice& operator=(Matrice&& matrice) noexcept = default;
```

On peut également lui interdire explicitement de le faire, avec `delete`. Mais comment savoir si on doit interdire ou non le déplacement pour nos classes ? Déjà, cela dépendra de la sémantique que vous souhaitez donner à votre classe. Autant déplacer une classe à sémantique de valeur ou permettre la transmission de propriétés entre deux indirections a du sens, autant vouloir déplacer les attributs d'une entité vers une autre n'en a généralement pas.

Pour aborder l'autre partie de la réponse, il faut nous pencher sur ce qui se passe en interne lors d'un déplacement.

30.3. En interne

La question importante à se poser quand on réfléchit à la pertinence d'implémenter le déplacement est la suivante : *Y a-t-il dans la classe un(e) indirection/handle qui est responsable d'une ressource ?* De cette question découlent quatre réponses :

1. Oui, il y a une ressource directement dans la classe.
2. Oui, il y a une ressource qui est gérée par une capsule RAII.
3. Oui, mais il y a d'autres attributs qui eux ne sont pas déplaçables.
4. Non, il n'y a aucune indirection, ce n'est que de la composition.

Le premier cas correspond grosso-modo à *écrire une capsule RAII*, un peu comme dans l'exemple de la bibliothèque graphique du chapitre précédent. Il faut alors manuellement déplacer tous les attributs d'une instance vers une autre. Nous n'en parlerons pas dans ce livre.

Le deuxième cas est bien connu, puisque on tombe dessus dès lors que *notre objet a comme attribut une capsule RAII*, telle que `std::string`, `std::vector`, `std::unique_ptr`, etc. Pour implémenter le déplacement pour une telle classe, il suffit de déplacer chacun des attributs capsules avec `std::move`.

Le troisième cas répond presque immédiatement à la question : comment rendre la classe déplaçable alors qu'un des attributs ne peut pas être déplacé ? Là encore, nous sortons du cadre de ce livre et ne nous étendons pas plus sur le sujet.

Enfin, le quatrième cas est celui dans lequel on peut implémenter le déplacement, mais *sans forcément en tirer bénéfice*. En effet, dans le cas d'une indirection, le déplacement équivaut en quelque sorte à lui dire *Tu ne regardes plus la donnée A stockée ici, mais*

maintenant tu regardes la donnée *B* stockée là. En revanche, s'il n'y a pas d'indirection, alors la donnée est tout simplement copiée.

Pour illustrer les cas 2 et 4, nous allons utiliser le code suivant, qui affiche les adresses de chaque attribut avant et après déplacement, respectivement d'une capsule RAI1 de type `std::vector`, de la ressource que surveille cette capsule et d'un entier utilisé en composition directe.

```
#include <iostream>
#include <string>
#include <utility>
#include <vector>

struct Test
{
    Test(std::vector<int> const& tableau, int entier);
    ~Test() noexcept = default;

    // On demande la génération explicite du déplacement.
    Test(Test&& déplacement) noexcept = default;
    Test& operator=(Test&& autre_test) noexcept = default;

    friend std::ostream& operator<<(std::ostream& os, Test const& test);

private:
    std::vector<int> m_tableau;
    int m_entier;
};

Test::Test(std::vector<int> const& tableau, int entier)
    : m_tableau(tableau), m_entier(entier)
{
}

std::ostream& operator<<(std::ostream& os, Test const& test)
{
    return os << "adresse m_tableau -> " << &test.m_tableau
        << " | adresse ressource gérée -> " << test.m_tableau.data()
        << " | adresse m_entier -> " << &test.m_entier;
}

int main()
{
    std::vector<int> const entiers { 1, 2, 3, 4, 5 };
    Test t1 { entiers, 42 };

    std::cout << "t1 : " << t1 << std::endl;

    std::vector<int> const autres_entiers { 0 };
    Test t2 { autres_entiers, -7 };

    std::cout << "t2 : " << t2 << std::endl;
```

```
std::cout << "\nOn déplace.\n";
t2 = std::move(t1);
std::cout << "t2 : " << t2 << std::endl;

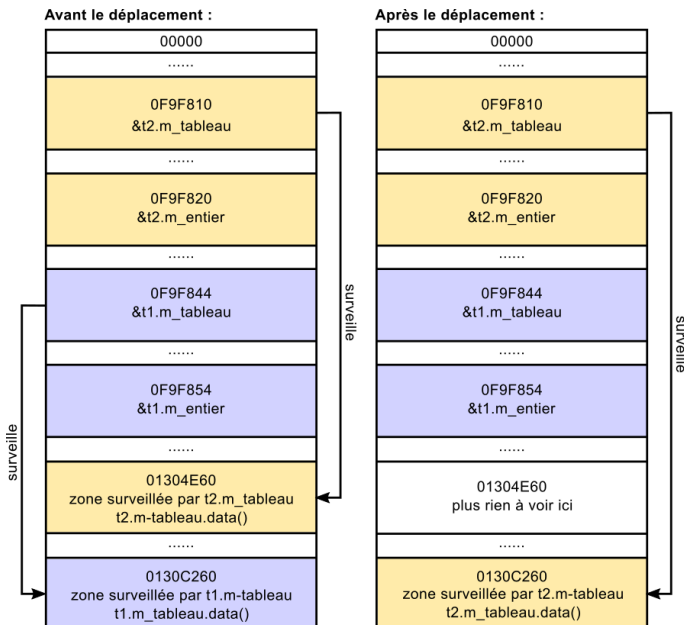
return 0;
}
```

```
t1 : adresse m_tableau -> 00F9F844 | adresse ressource gérée -> 0130C260
  | adresse m_entier -> 00F9F854
t2 : adresse m_tableau -> 00F9F810 | adresse ressource gérée -> 01304E60
  | adresse m_entier -> 00F9F820

On déplace.
t2 : adresse m_tableau -> 00F9F810 | adresse ressource gérée -> 0130C260
  | adresse m_entier -> 00F9F820
```

Les adresses changeront à chaque exécution, donc les valeurs en elles-mêmes ne sont pas très intéressantes. Par contre, on voit bien qu'après le déplacement, l'attribut `t2.m_tableau` n'a pas changé d'adresse (la capsule n'a pas bougé) mais surveille la zone mémoire qui était auparavant surveillée par `t1.m_tableau`. Il y a donc eu *déplacement* pour l'indirection. À l'inverse, `m_entier` n'a pas bougé et a simplement été copié.

Figure 30.1 : Ce qui se passe avant et après le déplacement.



Dans les cas où la sémantique de déplacement est justifiée pour notre classe, on ne perd rien à demander au compilateur de la générer, même si, dans le pire des cas, il ne se passe que des copies en interne. Sachant que le compilateur est assez malin pour mieux optimiser que nous, on est souvent gagnant.

30.4. Comme les cinq doigts de la main

Terminons ce chapitre sur une simple note. Il existe une règle en C++, qui est une bonne pratique à suivre et qui concerne un groupe de cinq fonctions, dont nous venons de voir les deux manquantes :

- le constructeur de copie ;
- l'opérateur d'affectation par copie ;
- le constructeur de déplacement ;
- l'opérateur d'affectation par déplacement ;
- le destructeur.

Cette règle explique que si vous définissez explicitement une des opérations ci-dessus, vous devriez le faire pour toutes les autres. Sinon, le compilateur ne générera que celles demandées explicitement et ne fournira pas toujours les autres (les règles de génération/suppression exactes sont complexes). On en déduit donc deux possibilités :

- *Ne rien définir explicitement* : le compilateur va donc toutes les générer, mais cela n'apparaît pas à l'œil d'un non-habitué. Dans le jargon anglophone, c'est la *Rule of Zero* [règle du zéro]. Elle n'est pas toujours possible, comme vous le voyez dans le cas d'entités polymorphes.
- *Être explicite*, et donc tout définir avec `default/delete`. C'est ce qu'on appelle la *Rule of Five* [règle des cinq].

30.5. Récapitulons !

- C++ fait une distinction entre les *lvalues* et les *rvalues*, termes qui englobent, entre autres, les littéraux et les objets temporaires ou en fin de vie.
- Depuis C++11, il est plus aisé de manipuler des *rvalues*.
- Il existe des situations où des copies inutiles peuvent se produire, comme lorsqu'on passe le résultat d'un calcul à une autre variable.
- Depuis C++11, la sémantique de déplacement peut optimiser le code en remplaçant les copies par des déplacements.

- Un objet déplacé se voit vidé de ses attributs. Il est valide mais son état est indéfini. On ne doit plus l'utiliser, car ce serait un comportement indéterminé. La seule chose possible est de lui réaffecter une nouvelle valeur.
- Le fait qu'un attribut soit simplement copié ou effectivement déplacé dépend notamment s'il s'agit d'une indirection ou d'une composition directe.
- La règle des cinq incite à définir ou supprimer explicitement le constructeur de copie, l'opérateur d'affectation par copie, le constructeur de déplacement, l'opérateur d'affectation par déplacement et le destructeur, dès lors que l'un d'entre eux l'est.

31

Oh, le bel héritage

Vous avez déjà eu un aperçu de l'héritage et de ce qu'il permet, mais nous sommes loin d'en avoir fait le tour. Justement, dans ce chapitre, nous allons approfondir ce point et aborder une nouvelle façon de concevoir ses classes appelée NVI.

31.1. NVI — Un contrat est un contrat

Des problèmes d'héritage...

Dans l'[exemple du robot nettoyeur](#), nous n'avons aucun invariant ni problème et le code fonctionne très bien. C'est normal, quand on introduit un concept, de ne pas le rendre trop complexe. Mais ce n'est malheureusement jamais aussi simple dans la réalité.

Vous l'avez compris, un point extrêmement important à respecter en cas d'héritage est le *principe de substitution de Liskov*. En effet, si l'on n'y prend garde, on peut facilement violer ce principe, sans que cela ne semble évident de prime abord. Après tout, *comment garantir que les classes enfants vont penser aux contrats ?*

Si dans le projet de nettoyage, on a la postcondition absolument vitale que tous les outils sont rangés une fois le ménage fini, qui nous dit que six mois plus tard, quand on va créer un aspirateur automatique auto-porteur de chats, nous n'allons pas oublier ? Après tout, personne n'est parfait. Si c'est quelqu'un d'autre qui dérive notre classe, on a encore moins de garanties qu'il respectera les contrats, volontairement ou involontairement.

Non Virtual Interface

Pour remédier à ce problème, il existe une façon de concevoir ses classes un peu différente qu'on appelle le *Non Virtual Interface*. Le principe se résume en deux points :

- l'interface de la classe est publique et non-virtuelle ;
- l'implémentation et les détails qui peuvent varier sont privés et virtuels, pour être supplantés/étendus par les classes enfants.

Grâce à cette nouvelle façon de faire, on centralise dans la classe parent les contrats et leur vérification et on laisse aux classes enfants les détails d'implémentation. On sépare l'interface publique (que les clients de la classe vont utiliser) de celle privée (qui ne concernent que les classes enfants et qui sert à implémenter/modifier le comportement).

Pour rendre ce principe plus clair et plus compréhensible, nous allons prendre l'exemple d'une classe permettant d'écrire une donnée dans un fichier, mais selon un format bien particulier (on parle de *sérialisation*). Cela peut être du texte séparé par des points-virgules (CSV), du XML, du JSON, etc. Mais avant d'écrire quoi que ce soit, on doit vérifier que le flux est valide et que les données le sont aussi. Et quand on a fini, il faut s'assurer qu'on a laissé le flux dans un état cohérent et utilisable. Grâce au *Non Virtual Interface*, on peut centraliser ces vérifications à un seul endroit, dans la classe parent.

```
#include <cassert>
#include <iostream>
#include <string>

struct InformationsPersonnelles
{
    std::string prenom;
    std::string nom;
    int age;
};

class Serializer
{
public:
    void ecrire(std::ostream & output,
        InformationsPersonnelles const & informations) const;

private:
    // Notez que l'écriture en elle-même est virtuelle pure
    // car variant en fonction des sous-types.
    virtual void ecrire_implementation(
        std::ostream & output,
        InformationsPersonnelles const & informations) const = 0;
};

void Serializer::ecrire(std::ostream & output,
    InformationsPersonnelles const & informations) const
{
    // Préconditions : que les données soient cohérentes.
    assert(!std::empty(informations.prenom)
        && "Le prénom doit être renseigné.");
    assert(!std::empty(informations.nom)
        && "Le nom doit être renseigné.");
    assert(informations.age > 0
        && "L'âge doit être correct.");
    ecrire_implementation(output, informations);
}
```

```

class XmlSerializer : public Serializer
{
private:
    void ecrire_implementation(std::ostream & output,
        InformationsPersonnelles const & informations) const override;
};

void XmlSerializer::ecrire_implementation(std::ostream & output,
    InformationsPersonnelles const & informations) const
{
    output << "<information>\n";
    output << "\t<prenom>" << informations.prenom << "</prenom>\n";
    output << "\t<nom>" << informations.nom << "</nom>\n";
    output << "\t<age>" << informations.age << "</age>\n";
    output << "</information>";
}

class JsonSerializer : public Serializer
{
private:
    void ecrire_implementation(std::ostream & output,
        InformationsPersonnelles const & informations) const override;
};

void JsonSerializer::ecrire_implementation(std::ostream & output,
    InformationsPersonnelles const & informations) const
{
    output << "{\n";
    output << "\t" << R"("prenom": )" << informations.prenom << ",\n";
    output << "\t" << R"("nom": )" << informations.nom << ",\n";
    output << "\t" << R"("age": )" << informations.age << "\n";
    output << "}";
}

using namespace std::literals;

void serialiser_donnees(Serializer const & serializer,
    InformationsPersonnelles const & informations)
{
    serializer.ecrire(std::cout, informations);
}

int main()
{
    InformationsPersonnelles const nicky_larson
    {
        "Nicky"s,
        "Larson"s,
        20
    };

    XmlSerializer const xml {};
    std::cout << "En XML : \n";
    serialiser_donnees(xml, nicky_larson);
}

```



```

std::cout << "\n\n";

InformationsPersonnelles const samus_aran
{
    "Samus"s,
    "Aran"s,
    20
};

JsonSerializer const json {};
std::cout << "En JSON : \n";
serialiser_donnees(json, samus_aran);

return 0;
}

```

```

En XML :
<information>
  <prenom>Nicky</prenom>
  <nom>Larson</nom>
  <age>20</age>
</information>

```

```

En JSON :
{
  "prenom": Samus,
  "nom": Aran,
  "age": 20
}

```

Notez que maintenant, nous avons bien une claire distinction entre l'interface publique (sérialiser les données) qui est destinée aux utilisateurs de notre classe et les détails d'implémentation qui ne concernent que les classes enfants (comment cette sérialisation est effectuée). D'un côté, *les clients ne voient que `ecrire`*, de l'autre, les classes enfants ne peuvent supplanter que les fonctions membres virtuelles, qui sont privées.



Comment le code peut-il compiler et s'exécuter alors que *`ecrire_implementation`* est privée ?

La classe enfant ne peut pas accéder ni manipuler la fonction membre *`ecrire_implementation`* de la classe parent *`Serializer`*, c'est vrai, mais ça n'empêche pas la résolution dynamique de se faire, qui appelle la fonction membre du type réel, à savoir la classe enfant (*`XmlSerializer`* ou *`JsonSerializer`* dans cet exemple).

Il ne faut pas confondre l'accessibilité d'une fonction membre, qui est vérifiée à la compilation, avec la résolution dynamique de liens, qui est fait à l'exécution.



Mais du coup, quelle est la meilleure façon de faire ? Est-ce que je dois oublier ce que j'ai appris au chapitre [Classes à sémantique d'entité](#) et ne jurer que par le NVI ?

Encore une fois, C++ vous laisse une grande liberté, qui peut laisser perplexe, car la réponse est que ça *dépend des cas*. Comme on vient de le voir, le *Non Virtual Interface* présente des avantages quand on a des contrats et des invariants à faire respecter par les enfants. Il vient au prix de plus de code à écrire. Dans les cas où il n'y a aucun contrat, on peut se contenter d'utiliser des fonctions membres publiques virtuelles. C'est ce que fait la classe `std::exception` avec la fonction membre `what`, par exemple.

Changements dans les contrats

Le principe de substitution de Liskov précise que nous pouvons affaiblir les préconditions ou renforcer les postconditions. Ces modifications sont possibles si nos contrats sont disponibles à travers des fonctions membres virtuelles, qui pourront être supplantées si nécessaire. Reprenons notre exemple des compagnies aériennes pour l'illustrer.

```
#include <cassert>
#include <iostream>

struct Bagage
{
    int poids;
    int hauteur;
    int largeur;
    int profondeur;
};

class CompagnieAerienne
{
public:
    void enregistrer_bagage(Bagage const& bagage) const noexcept;

private:
    virtual void verifier_taille_bagage(Bagage const& bagage) const
        noexcept;
};

void CompagnieAerienne::enregistrer_bagage(Bagage const & bagage) const
    noexcept
{
    // Vérification des invariants à travers une
    // fonction membre éventuellement supplantable.
    verifier_taille_bagage(bagage);
    std::cout << "Bagage bien enregistré.\n";
}
```

```

void CompagnieAerienne::verifier_taille_bagage(Bagage const &
    bagage) const noexcept
{
    assert(bagage.poids <= 8 && "Un bagage cabine ne peut pas dépasser
    8kg.");
    assert(bagage.hauteur <= 55 && bagage.largeur <= 40
        && bagage.profondueur <= 20 && "Dimensions autorisées : 55x40x20");
}

class CompagnieTraditionnelle : public CompagnieAerienne
{
private:
    void verifier_taille_bagage(Bagage const & bagage) const noexcept
        override;
};

void CompagnieTraditionnelle::verifier_taille_bagage(Bagage const &
    bagage) const noexcept
{
    // Préconditions affaiblies dans un sous-type, en respect avec le LSP.
    assert(bagage.poids <= 12 && "Un bagage cabine ne peut pas dépasser
    12kg.");
    assert(bagage.hauteur <= 55 && bagage.largeur <= 40
        && bagage.profondueur <= 25 && "Dimensions autorisées : 55x40x25");
}

int main()
{
    Bagage const bagage { 10, 50, 40, 25 };
    CompagnieTraditionnelle const compagnie {};
    compagnie.enregistrer_bagage(bagage);

    return 0;
}

```

Bien entendu, avec cette solution, rien n'empêche le développeur qui codera une classe enfant de faire n'importe quoi avec les contrats et les invariants, mais dans ce cas, c'est hors de votre portée. Vous avez fait votre part du travail, lui seul sera fautif.

31.2. *protected* — la portée intermédiaire

Quand nous avons introduit `public` et `private`, nous avons évoqué l'existence d'une troisième portée: `protected`. Cette dernière est intermédiaire puisque plus restrictive que `public` mais plus permissive que `private`. Tout attribut ou toute fonction membre marqué comme protégé n'est pas accessible en dehors de la classe ET de ses enfants et c'est toute la différence avec les éléments privés.

```

#include <iostream>

class Mere

```

```

{
public:
    virtual void dire_bonjour() const noexcept;

    // Accessible à tout le monde.
    int a;

protected:
    // Accessible seulement aux instances de Mere,
    // de Fille et même de PetiteFille.
    int b;

private:
    // Accessible seulement aux instances de Mere.
    int c;
};

void Mere::dire_bonjour() const noexcept
{
    std::cout << "a = " << a << std::endl;
    std::cout << "b = " << b << std::endl;
    std::cout << "c = " << c << std::endl;
}

class Fille : public Mere
{
public:
    void dire_bonjour() const noexcept override;
};

void Fille::dire_bonjour() const noexcept
{
    // Possible car public.
    std::cout << "a = " << a << std::endl;
    // Possible car protected.
    std::cout << "b = " << b << std::endl;
    // Impossible car private.
    std::cout << "c = " << c << std::endl;
}

class PetiteFille : public Fille
{
public:
    void dire_bonjour() const noexcept override;
};

void PetiteFille::dire_bonjour() const noexcept
{
    // Possible car public.
    std::cout << "a = " << a << std::endl;
    // Possible car protected marche avec les héritages successifs.
    std::cout << "b = " << b << std::endl;
    // Impossible car private dès la classe de base.
    std::cout << "c = " << c << std::endl;
}

```

```
[Visual Studio]
E0265 membre "Mere::c" (déclaré à la ligne 18) est inaccessible
C2248 'Mere::c' : impossible d'accéder à private membre déclaré(e) dans
la classe 'Mere'

-----

[GCC]
prog.cc: In member function 'virtual void Fille::dire_bonjour() const':
prog.cc:41:28: error: 'int Mere::c' is private within this context
 41 |         std::cout << "c = " << c << std::endl;
    |                               ^
prog.cc:18:9: note: declared private here
 18 |         int c;
    |         ^
prog.cc: In member function 'virtual void PetiteFille::dire_bonjour()
const':
prog.cc:57:28: error: 'int Mere::c' is private within this context
 57 |         std::cout << "c = " << c << std::endl;
    |                               ^
prog.cc:18:9: note: declared private here
 18 |         int c;
    |         ^

-----

[Clang]
prog.cc:41:28: error: 'c' is a private member of 'Mere'
    std::cout << "c = " << c << std::endl;
                        ^
prog.cc:18:9: note: declared private here
    int c;
    ^
prog.cc:57:28: error: 'c' is a private member of 'Mere'
    std::cout << "c = " << c << std::endl;
                        ^
prog.cc:18:9: note: declared private here
    int c;
    ^
2 errors generated.
```

Les cas d'utilisation de **protected** sont plus limités que pour **public** et **private**, donc on le croise plus rarement. Il existe néanmoins une règle qui va nous donner une application concrète et utile de ce nouveau mot clé. Elle a été énoncée en 2001, une fois encore, par **Herb Sutter** :

Le destructeur d'une classe de base, destinée à être héritée, doit être soit *public et virtuel*, soit *protégé et non-virtuel*.

Comme nous l'avons vu ensemble au chapitre [Ressources, indirections et handles](#) dès lors qu'il y a héritage, on a la possibilité de manipuler une instance d'une classe enfant à

travers un pointeur (ou une référence) sur la classe parent. Dans ce cas, il est nécessaire que le destructeur soit *virtuel*, sinon, au moment de la libération de la mémoire, nous aurons un comportement indéterminé. Cependant, on peut aussi vouloir interdire cette possibilité. Nous ne pouvons pas rendre le destructeur *private*, ce qui empêcherait les classes enfants de l'appeler au moment de leur libération. La solution tient donc dans cette troisième portée, *protected*.



Quel peut bien être l'intérêt de cette interdiction ?

Examinons un exemple concret, tiré de la bibliothèque standard avant que le C++ moderne ne nous apporte les *std::function* et les lambdas. À l'époque, il fallait dériver du type *std::unary_function* pour pouvoir passer ses propres prédicats aux algorithmes de la bibliothèque standard. Comme ce genre de type n'a pas vocation à être manipulé de manière polymorphique, il n'y a aucun intérêt à ce que son destructeur soit *virtuel*.

Par contre, en opposition avec l'exemple introductif, utiliser *protected* sur des attributs de la classe est rarement une bonne idée. En effet, ceux-ci pourraient alors être directement modifiés par les classes enfants, ce qui pourrait transgresser d'éventuels contrats et invariants liés à cet attribut et donc irait en opposition avec le LSP.

Dans le cas où l'on veut laisser une certaine liberté de modification aux descendants, il faut mieux laisser les attributs privés et passer par des fonctions dédiées protégées, ce qui assurera alors le respect des contrats.

31.3. Mettons un point final à cet héritage

Cette section peut sembler ironique, puisque nous allons voir comment bloquer un héritage... alors même que le présent chapitre porte sur son mode de fonctionnement. Cependant, il est très pratique de pouvoir interdire un héritage.

Depuis C++11, nous disposons d'un nouveau mot clé pour ce faire : *final*. Celui-ci peut s'appliquer à une fonction membre virtuelle. Dans ce cas, il signifie que la redéfinition qu'on effectue dans la classe enfant sera la *dernière possible*. Aucune autre redéfinition ne sera possible dans les classes descendantes.

```
class Mere
{
public:
    // Peut être redéfinie.
```

```

    virtual void fonction() const noexcept;
    // Peut être redéfinie aussi.
    virtual void autre_fonction() const noexcept;
};

class Fille : public Mere {
public:
    // redéfinie, mais qui ne pourra plus
    // l'être dans les classes descendantes.
    void fonction() const noexcept override final;
    // Possible.
    void autre_fonction() const noexcept override;
};

class PetiteFille : public Fille {
public:
    // Impossible !
    void fonction() const noexcept override;
    // Toujours possible.
    void autre_fonction() const noexcept override;
};

```

```

[Visual Studio]
E1850 impossible de substituer 'final' fonction
"Fille::fonction" (déclaré à la ligne 14)
C3248 'Fille::fonction' : fonction déclarée comme 'final' ; ne peut pas
être substituée par 'PetiteFille::fonction'

```

```

-----

[GCC]
prog.cc:22:10: error: virtual function 'virtual void
PetiteFille::fonction() const' overriding final function
   18 |     void fonction() const noexcept override;
      |         ^~~~~~
prog.cc:14:10: note: overridden function is 'virtual void
Fille::fonction() const'
   12 |     void fonction() const noexcept override final;
      |         ^~~~~~

```

```

-----

[Clang]
prog.cc:22:10: error: declaration of 'fonction' overrides a 'final'
function
    void fonction() const noexcept override;
      ^
prog.cc:14:10: note: overridden virtual function is here
    void fonction() const noexcept override final;
      ^
1 error generated.

```

L'autre possibilité consiste à rendre toute la classe finale, ce qui empêche qu'elle soit dérivée. Dans ce cas, le mot clé **final** se marque après l'identifiant de la classe souhaitée.

```

class Mere
{
};

class Fille final : public Mere {

};

class PetiteFille : public Fille {

};

```

```

[Visual Studio]
E1904 un type de classe 'final' ne peut pas être utilisé comme une
classe de base
C3246 'PetiteFille' : ne peut pas hériter de 'Fille', car il a été
déclaré comme 'final'

-----

[GCC]
prog.cc:9:7: error: cannot derive from 'final' base 'Fille' in derived
type 'PetiteFille'
   9 | class PetiteFille : public Fille {
     |         ^~~~~~
-----

[Clang]
prog.cc:9:28: error: base 'Fille' is marked 'final'
class PetiteFille : public Fille {
                        ^
prog.cc:5:7: note: 'Fille' declared here
class Fille final : public Mere {
    ^      ~~~~~
1 error generated.

```

Enfin, comme vous vous en doutez peut-être, **final** ne peut pas être appliqué sur les fonctions membres non-virtuelles. C'est normal, celles-ci ne sont pas prévues pour être supplantées, donc la norme a prévu que ce cas soit interdit, tout comme pour **override**.

31.4. L'héritage multiple

Attention aux âmes sensibles, nous allons maintenant aborder un sujet qui est à l'origine de beaucoup de débats âpres et passionnés entre aficionados de différents langages de programmation. La possibilité que nous offre C++ à ce sujet n'a pas été reprise telle quelle dans beaucoup de langages qui sont apparus après lui, dont PHP, Ruby ou encore Java, bien que ces derniers proposent en général un mécanisme similaire. Il s'agit de *l'héritage multiple*.

Le principe est simple : il consiste à hériter de plusieurs classes à la fois. Prenons une analogie automobile. Nous croisons de plus en plus souvent sur nos routes des véhicules hybrides, qui combinent certains aspects des véhicules 100% électriques avec ceux des véhicules équipés de moteurs thermiques. Nous pourrions alors très bien imaginer le code suivant.

```
class VehiculeElectrique
{
public:
    void rouler_en_utilisant_la_batterie() const noexcept;
};

class VehiculeEssence
{
public:
    void rouler_en_utilisant_le_carburant() const noexcept;
};

// Notez la répétition de public, ce qui est
// important pour utiliser l'héritage public.
class VehiculeHybride : public VehiculeElectrique, public
    VehiculeEssence
{
};
```

Posons-nous un instant et réfléchissons. Peut-on vraiment substituer un véhicule hybride à un véhicule thermique ou électrique ? Cela va dépendre des contraintes, mais imaginons que notre véhicule hybride recharge sa batterie en utilisant l'énergie produite par le moteur thermique. Nous ne sommes plus compatibles avec la voiture 100% électrique qui se branche sur le secteur. Ce serait dans ce cas une violation du LSP.

Comme nous l'avons déjà dit et redit, le LSP est vraiment un principe très important à respecter. Sauf que si un héritage simple est déjà en opposition avec la substitution, que dire d'un héritage multiple ? Les problèmes qui en découlent se trouvent multipliés.

D'un point de vue technique, l'héritage multiple peut mener à d'autres types de problèmes. Par exemple, il y a ce qu'on appelle *l'héritage en diamant* : que se passe-t-il si une classe hérite de deux classes différentes, qui elles-mêmes héritent d'une même classe commune qui définit un certain attribut ou fonction membre ?

```
#include <iostream>

class A
{
public:
    int entier { 42 };
};
```

```

};

class B : public A
{
};

class C : public A
{
};

class D : public B, public C
{
};

int main()
{
    D const d {};
    std::cout << d.entier << std::endl;
    return 0;
}

```

```

[Visual Studio]
E0266 "D::entier" est ambigu
C2385 accès ambigu de 'entier'

```

```

-----

[GCC]
prog.cc: In function 'int main()':
prog.cc:27:20: error: request for member 'entier' is ambiguous
   27 |         std::cout << d.entier << std::endl;
      |                ^~~~~~
prog.cc:6:9: note: candidates are: 'int A::entier'
    6 |         int entier { 42 };
      |         ^~~~~~
prog.cc:6:9: note:           'int A::entier'

```

```

-----

[Clang]
prog.cc:27:20: error: non-static member 'entier' found in multiple base-
class subobjects of type 'A':
    class D -> class B -> class A
    class D -> class C -> class A
    std::cout << d.entier << std::endl;
                  ^
prog.cc:6:9: note: member found by ambiguous name lookup
    int entier { 42 };
    ^

```

Comment est censé réagir le compilateur ? La réponse est compliquée et dépend des cas. Vous pourrez néanmoins creuser le sujet à l'issue de ce livre. Les mots clés pour trouver la solution sont *héritage virtuel* et *diamant de la mort*.

Il n'empêche que ce genre de problèmes, tant techniques que conceptuels, a amené certains langages à bannir la possibilité d'hériter de plusieurs classes en même temps. C'est le cas par exemple de Java, PHP, C# ou encore Ruby.



Du coup, si les langages venus après C++ ont pour la plupart supprimé l'héritage multiple, pourquoi nous en parler ? N'est-ce pas une mauvaise pratique ?

Comme beaucoup de solution technique, l'héritage multiple a ses avantages et ses inconvénients. Et bien que Java ou PHP aient fait ce choix, ils implémentent des comportements qui restent proches, dans l'idée, de l'héritage multiple. Ainsi, Java autorise qu'une classe implémente plusieurs interfaces et PHP fournit un mécanisme similaire nommé *trait*.

Sachant que C++ ne distingue pas les classes des interfaces, l'héritage multiple nous est nécessaire. Et comme toujours, C++ vous laissera une grande liberté dans ce que vous décidez de faire.

EXEMPLE DANS LA BIBLIOTHÈQUE STANDARD

La bibliothèque standard nous fournit un flux d'entrée/sortie pour les chaînes de caractères nommé `std::stringstream`, qui hérite de `std::basic_istream`, qui lui-même hérite de deux classes, `std::basic_istream` et `std::basic_ostream`.

31.5. L'héritage, solution miracle ?

L'héritage et le polymorphisme d'inclusion offrent de grandes possibilités, c'est indéniable. Néanmoins ce n'est pas toujours la bonne solution. C'est ce que nous allons voir, en prenant un jeu de rôle comme exemple.

Dans ce genre de jeux, on a souvent plusieurs classes de personnages, plusieurs armes, objets, etc. On pourrait essayer de tout modéliser avec des hiérarchies de classes, comme par exemple une classe `Personnage` dont hériteraient `Mage`, `Guerrier`, `Voleur`, etc. Ou bien une classe `Arme` qui servirait de parent à `Lance`, `Hache` ou `Dague`. Cette solution pose pourtant pleins de problème. Concentrons-nous un instant sur les armes en particulier.

Déjà, l'héritage ne nous permet pas de tout représenter. Si on prend l'exemple d'une hallebarde, on peut la voir tant comme une hache que comme une lance. Pourtant, on ne peut pas la lancer, ce qui est une perte de service par rapport à une lance. On va alors se retrouver à dupliquer du code.

De plus, on peut vouloir ajouter un poids à l'arme, ce qui pourra forcer le personnage à utiliser ses deux mains au lieu d'une pour la manipuler... sauf si on a une potion de force ou que notre personnage fait partie d'une race de géants ou d'une race ayant plus de deux mains, etc. On pourrait se retrouver à créer autant de classes qu'on a de variations, c'est-à-dire beaucoup trop.

Sans parler que certaines armes peuvent être des objets de quêtes. Ce sont des armes comme les autres, à ceci près qu'on ne peut pas s'en débarrasser, ni les vendre, car elles sont nécessaires pour terminer une ou plusieurs quêtes. Il faudrait donc créer une autre classe pour gérer ce nouveau concept. Le même problème se pose avec les races, les équipements, etc. On le comprend très vite, le nombre de classes explose et devient ingérable. Nous avons donc un cas où l'héritage n'est pas adapté.

UNE SOLUTION

Une possible solution à ce problème s'appelle l'*Entities Components Systems*, qui permet une architecture bien plus souple. Les plus curieux pourront aller chercher des informations eux-mêmes sur Internet.

31.6. Récapitulons !

- Si l'on n'y prend pas garde, une classe enfant peut violer les contrats que la classe parent définit, ou tout du moins ne pas les respecter.
- Le NVI permet de séparer l'interface publique (pour les clients du code) de l'interface privée (qui sera supplantée/définie par les classes enfants).
- Le NVI permet de centraliser la vérification des contrats dans la classe parent.
- Il existe une troisième portée, **protected**, qui rend accessible un attribut ou une fonction membre dans les classes enfants.
- La portée **protected** est, très souvent, une mauvaise idée pour les attributs, puisque ceux-ci peuvent alors être modifiés sans vérification des contrats par les descendants, ce qui est une rupture d'encapsulation.
- C++11 a introduit le mot clé **final**, qui empêche de supplanter une fonction ou dériver d'une classe ainsi qualifiée.

- C++ nous permet d'hériter d'autant de classes qu'on le souhaite, ce qu'on appelle l'héritage multiple.
- Comme beaucoup de solutions, l'héritage multiple a ses avantages dans certains cas, mais peut mener à des difficultés dans d'autres, notamment avec le problème de l'héritage en diamant.
- L'héritage et le polymorphisme d'inclusion sont pratiques mais ne sont néanmoins pas des solutions universelles qui correspondent à tous les cas et à toutes les situations.

32

Les classes templates

Nous avons déjà utilisé de nombreuses fois des classes templates (comme `std::vector`) et appris à écrire des fonctions libres templates. Il nous reste à voir comment créer des classes génériques. Ce chapitre va nous introduire quelques techniques avancées.

32.1. Les classes génériques

Définir une classe générique

La syntaxe pour rendre une classe ou une structure générique est la même que pour une fonction, donc elle ne vous surprendra pas. Pour l'illustrer, nous allons reprendre notre `Pile` et rendre le type de données stocké paramétrable.

```
#ifndef PILE_HPP
#define PILE_HPP

#include <vector>

template <typename T>
class Pile
{
public:
    // Le type T est utilisable dans nos fonctions membres.
    void ajouter(T const & valeur);
    void supprimer();
    T const & sommet() const;

private:
    // Ainsi que dans les attributs.
    std::vector<T> m_pile;
};

#endif
```

Exactement comme pour les fonctions libres templates, la déclaration et l'implémentation de nos fonctions membres au sein d'une classe template doivent être *dans le même fichier*. Nous allons donc écrire le corps des fonctions directement au sein de la classe.

```

#ifndef PILE_HPP
#define PILE_HPP

#include <cassert>
#include <vector>

template <typename T>
class Pile
{
public:
    void ajouter(T const & valeur)
    {
        m_pile.push_back(valeur);
    }

    void supprimer()
    {
        assert(m_pile.size() > 0
            && "Impossible de supprimer un élément d'une pile vide.");
        m_pile.pop_back();
    }

    T const & sommet() const
    {
        assert(m_pile.size() > 0
            && "Impossible de récupérer le sommet d'une pile vide.");
        return m_pile[0];
    }

private:
    std::vector<T> m_pile;
};

#endif

```

L'utilisation de notre classe générique, quant à elle, ne change pas des conteneurs de la bibliothèque standard, que nous sommes maintenant habitués à manipuler.

```

#include <iostream>
#include "Pile.hpp"

int main()
{
    Pile<int> pile {};
    pile.ajouter(3);
    std::cout << pile.premier() << std::endl;

    return 0;
}

```

Référencer une classe générique

Quand on référence notre classe à l'intérieur d'une fonction membre, que ce soit pour l'utiliser comme un paramètre ou comme une valeur de retour, nous ne sommes pas obligés d'utiliser la syntaxe `<>` pour préciser qu'il s'agit d'une classe template. Le compilateur est intelligent et va le déduire tout seul.

```
template <typename T>
class Pile
{
public:
    // Pas besoin de rajouter <T>.
    Pile() = default;
    ~Pile() noexcept = default;

    Pile(Pile const& pile) = default;
    Pile& operator=(Pile const & pile) = default;

    Pile(Pile&& pile) = default;
    Pile& operator=(Pile&& pile) = default;
};
```

Par contre, en dehors de ce cas, nous sommes obligés de donner un type explicite ou générique à chaque fois qu'on référence la classe. Cette règle est donc valable si on utilise la solution consistant à séparer la déclaration d'une classe générique de sa définition.

```
template <typename T>
class Pile
{
public:
    Pile();
};

// Obligation d'utiliser <T>
template <typename T>
Pile<T>::Pile()
{
}
```

Arguments types et non-types

Le mot clé `typename` signifie littéralement *nom de type* ce qui permet au compilateur de comprendre que ce qu'on lui passe est un type. Cependant, ce n'est pas notre seule option quand on joue avec les templates. On peut en effet définir un template qui attend autre chose qu'un type, par exemple une valeur entière. C'est ainsi que procède

`std::array`, qui attend non seulement le type du tableau statique qu'on veut créer, mais aussi sa taille.

```
template <typename T, std::size_t N>
class array;
```

Grâce à cette technique, nous pourrions modifier notre classe `Matrice` pour que sa taille soit connue à la compilation, ce qui permettrait d'implémenter la plupart des services avec `constexpr`. Les différents calculs seraient alors réalisés pendant la compilation, rendant l'exécution de notre code encore plus rapide.

Valeur par défaut

Tout comme il est possible de donner une valeur par défaut aux paramètres d'une fonction, on peut très bien définir une valeur par défaut pour un paramètre de template. Dans ce cas, nous ne sommes plus obligés de le préciser à l'utilisation.

```
template <typename T = int>
class Pile
{

};

int main()
{
    // Plus besoin de préciser le type.
    Pile<> const p1 {};
    // Et à partir de C++17, il n'y a même
    // plus besoin d'écrire les chevrons <>.
    Pile const p2 {};
    return 0;
}
```

C'est le cas par exemple pour `std::vector`. Depuis le début, nous ne précisons que le type de données stockées, mais si l'on regarde sa définition, on voit bien qu'il y a une deuxième valeur, liée à la gestion de la mémoire sur le tas (technique avancée qui ne nous concerne pas à notre niveau).

```
template <typename T, typename Allocator = std::allocator<T>>
class vector;
```

Le fait de rendre ce paramètre optionnel car ayant déjà une valeur par défaut rend le code plus simple pour nous, tout en laissant la liberté à ceux dont le besoin est justifié de modifier la gestion de la mémoire à l'intérieur du conteneur.

32.2. [Pratique] Un wrapper pour les pointeurs nus

Grâce aux connaissances que nous avons maintenant, nous pouvons écrire un petit utilitaire pour manier des pointeurs nus de manière plus sécurisée. En effet, bien qu'ils rendent des services, certains comportements hérités du C ne nous intéressent pas quand on cherche à avoir un *observateur non-propriétaire*.

Cette classe `ObserverPtr` pourra être utilisable avec n'importe quel type, sera copiable et déplaçable et implémentera les opérateurs `*` pour accéder à la ressource pointée et `->` pour obtenir le pointeur lui-même. Enfin, par défaut, il sera initialisé à `nullptr`.

Note > Un `type semblable` a été proposé pour être ajouté dans la bibliothèque standard. Le futur nous dira s'il est accepté ou rejeté.

```
#include <cassert>

template <typename T>
// Notez l'utilisation de final pour empêcher
// tout héritage à partir de cette classe.
class ObserverPtr final
{
public:
    // Un seul constructeur, par défaut initialise à nullptr.
    ObserverPtr(T* ptr = nullptr) noexcept
        : m_ptr(ptr)
    {

    }

    ~ObserverPtr() noexcept = default;

    ObserverPtr(ObserverPtr const& observer_ptr) noexcept = default;
    ObserverPtr& operator=(ObserverPtr const& observer_ptr) noexcept
        = default;

    ObserverPtr(ObserverPtr&& observer_ptr) noexcept = default;
    ObserverPtr& operator=(ObserverPtr&& observer_ptr) noexcept = default;

    T& operator*() const noexcept
    {
        assert(m_ptr != nullptr && "Impossible d'utiliser l'opérateur * sur
un pointeur null.");
        return *m_ptr;
    }

    T* operator->() const noexcept
    {
        assert(m_ptr != nullptr && "Impossible d'utiliser l'opérateur -> sur
un pointeur null.");
        return m_ptr;
    }
}
```

```
private:
    T* m_ptr;
};
```

32.3. Les traits

Nous avons appris au cours de ce livre l'importance que jouent les contrats dans la qualité du code qu'on écrit. Cependant, nos assertions ne sont vérifiées qu'à l'exécution, alors que les templates sont instanciés à la compilation, ce qui veut dire que `assert` ne peut pas nous aider. Qu'à cela ne tienne, le C++ contient désormais les outils dont nous avons besoin : les *traits*.

Disponibles depuis la norme de 2011 dans le fichier d'en-tête `<type_traits>` et ne cessant de s'enrichir au fil du temps, les traits permettent de répondre à des questions sur un type telles que savoir si celui-ci est un entier, un nombre réel, s'il possède un destructeur, s'il est copiable, qu'elle est la valeur maximum assignable, etc. On peut ainsi *affiner le comportement* de nos templates et de nos fonctions `constexpr`.

Note > Quand nous avons utilisé `std::numeric_limits` pour sécuriser nos entrées, nous avons en réalité fait appel à un trait.

Exemple simple

Pour utiliser les traits, il suffit de passer en paramètre template le type ou l'expression à évaluer et de regarder la valeur du membre `value`, qui vaudra `true` ou `false` si l'expression donnée respecte ou non la propriété demandée. L'exemple suivant, très simple, permet de savoir si le type qu'on manipule est un entier ou non. Notez l'existence d'une syntaxe raccourcie, avec `_v` à la place de `::value`.

```
#include <iostream>
#include <string>
#include <type_traits>

int main()
{
    std::cout << std::boolalpha;
    // Syntaxe complète.
    std::cout << std::is_integral<int>::value << std::endl;
    // Syntaxe raccourcie.
    std::cout << std::is_integral_v<std::string> << std::endl;
    return 0;
}
```

```
true
false
```

L'intérêt de cet exemple est limité, mais il va servir comme base pour apprendre la programmation par contrat à la compilation.

Traits et programmation par contrat

Hérité du C, nous avons `assert` pour vérifier des assertions à l'exécution. C++11 a apporté le mot clé `static_assert`, qui va quant à lui permettre la vérification de contrats au moment de compiler. Notez bien qu'il n'y a pas besoin d'inclure de fichier d'en-tête, puisque c'est un mot clé. Toute expression constante qu'on lui donne sera évaluée à `true` ou `false`, auquel cas la compilation échouera, avec éventuellement un message d'erreur. On peut l'appliquer tant pour une classe entière que pour une fonction en particulier.

Dans l'exemple suivant, nous utilisons le trait `std::is_arithmetic` de manière à limiter l'instanciation d'une classe `Matrice` générique aux seuls types entiers et réels. Le type `std::string` provoque donc une erreur de compilation, claire grâce au message qui accompagne `static_assert`.

```
#include <string>
#include <type_traits>

template <typename T>
class Matrice
{
    static_assert(std::is_arithmetic_v<T>,
        "Entiers ou flottants uniquement.");
};

int main()
{
    // Pas de problème.
    Matrice<int> const matrice_entiers {};
    Matrice<double> const matrice_reels {};
    // Ne compile pas.
    Matrice<std::string> const matrice_string {};
    return 0;
}
```

```
[Visual Studio]
Erreur C2338 Entiers ou flottants uniquement.

-----

[GCC]
prog.cc: In instantiation of 'class
Matrice<std::__cxx11::basic_string<char> >':
prog.cc:17:32:   required from here
prog.cc:8:24: error: static assertion failed: Entiers ou flottants
uniquement.
    8 |     static_assert(std::is_arithmetic_v<T>, "Entiers ou flottants
uniquement.");
      |               ^~~~~~
prog.cc:8:24: note:
'std::is_arithmetic_v<std::__cxx11::basic_string<char> >' evaluates to
false
      |               ^~~~~~
-----

[Clang]
prog.cc:8:5: error: static_assert failed due to requirement
'std::is_arithmetic_v<std::string>' "Entiers ou flottants uniquement."
    static_assert(std::is_arithmetic_v<T>, "Entiers ou flottants
uniquement.");
      ^~~~~~
prog.cc:17:32: note: in instantiation of template class
'Matrice<std::string>' requested here
    Matrice<std::string> const matrice_string {};
      ^
1 error generated.
```

Imaginez maintenant que vous vouliez écrire une fonction qui compare deux nombres pour déterminer s'ils sont égaux. Pour des entiers, c'est très simple, car il suffit d'utiliser l'opérateur d'égalité `==`. Mais pour des nombres réels, cet opérateur va comparer toutes les décimales ; or, nous n'avons pas besoin d'une aussi grande précision.

Nous allons donc réutiliser l'*astuce vue dans le chapitre introductif à la POO*, à savoir comparer seulement selon un nombre défini de décimales. Grâce aux traits, nous pouvons nous assurer que les deux arguments sont comparables et nous faisons varier l'algorithme utilisé si nous avons affaire ou non à un nombre réel.

```
#include <iostream>
#include <string>
#include <type_traits>

template <typename T, typename U>
auto comparer(T const& lhs, U const& rhs,
              double ecart_minimal = 0.001)
{
```

```

// On vérifie que le test d'égalité est possible.
static_assert(std::is_convertible_v<T, U>
    && std::is_convertible_v<U, T>,
    "T et U doivent être convertibles.");

if (std::is_floating_point_v<T>
    || std::is_floating_point_v<U>)
{
    return std::abs(lhs - rhs) < ecart_minimal;
}
else
{
    return lhs == rhs;
}
}

using namespace std::literals;

int main()
{
    std::cout << std::boolalpha;

    std::cout << comparer(3, 3) << std::endl;
    std::cout << comparer(3.00001, 3.00009) << std::endl;
    std::cout << comparer(3.00001, 3.00009, 0.0000001);
    // Erreur, un int et une std::string ne sont pas convertibles.
    // Si on tente de compiler, on aura l'erreur :
    // T et U doivent être convertibles.
    //std::cout << comparer(3, "3"s) << std::endl;

    return 0;
}

```

En interne

Les traits offrent bien des services, mais peut-être vous êtes-vous demandé comment fonctionnent-ils vraiment ? Par exemple, comment fait `std::is_floating_point` pour nous dire que `double` est un réel mais pas `int` ? Ou bien comment `std::is_const_v` peut-il être `true` pour `int const` mais `false` pour `int` ?

Sans rentrer trop profondément dans les détails des traits, qui mériteraient un livre à eux seuls, la réponse s'appelle *la spécialisation*, que nous avons déjà abordée [dans le chapitre sur les fonctions templates](#). La technique est simple : il suffit de définir le cas général comme étant faux, puis de spécialiser pour définir les cas vrais. Examinons une implémentation possible de `std::is_const`, tirée de la [documentation](#).

```

template <typename T>
struct is_const : std::false_type {};

template <typename T>

```

```
struct is_const<const T> : std::true_type {};
```

La bibliothèque standard offre une `base` en la présence de `std::false_type` et `std::true_type` afin de représenter les concepts de *faux* et de *vrai*. Le cas général est donc faux et seul la spécialisation pour les types `const T` est vraie.

Dans le cas de traits simples, comme celui ci-dessus, on peut s'en sortir uniquement avec du code. D'autres, plus complexes, ne sont possibles que grâce à des informations fournies par des fonctions internes au compilateur. C++11 et les normes suivantes ont donc apporté une grande aide de ce côté en standardisant un grand nombres de traits.

32.4. Ces modèles sont bien trop génériques...

Pendant longtemps, C++ a souffert d'une réputation de langage trop complexe à causes des templates et des erreurs incompréhensibles qui leur sont liées. Il est vrai que lorsqu'on écrit une fonction ou une classe template et qu'on lui passe un type incompatible, le compilateur s'emballe et nous sort une flopée de lignes d'erreurs.

Le code suivant est un bon exemple. Nous vous laissons l'exécuter pour voir par vous-même le résultat, car la liste des erreurs est bien trop longue pour être incluse ici.

```
#include <algorithm>
#include <vector>

int main()
{
    int const entier { 42 };
    std::vector<std::vector<int>> const tableau_2d;
    // Oups...
    std::find(std::cbegin(tableau_2d), std::cend(tableau_2d), entier);

    return 0;
}
```

Le problème c'est que les paramètres des templates sont *trop génériques*. On aimerait pouvoir les restreindre un peu, être certain qu'ils vérifient certaines propriétés ou possèdent certaines caractéristiques. Certes, les traits et `static_assert` nous aident, mais c'est avec C++20 qu'est arrivée la véritable solution.

En voilà un bon concept !

C'est cette norme qui a enfin apporté les concepts dans la bibliothèque standard. On les retrouve dans le fichier d'en-tête `<concepts>`. Un concept regroupe l'ensemble des prérequis que doit respecter un type. Comme les traits et `static_assert`, ils sont évalués à la compilation, ce qui provoquera une erreur si un concept n'est pas respecté.

Voici déjà trois exemples, chacun illustrant un concept et une syntaxe différents. Chacun d'entre eux est utilisé deux fois, pour montrer ce qui se passe avec un argument respectant les contraintes imposées et avec un second qui ne colle pas.

```
#include <concepts>
#include <iostream>
#include <memory>
#include <string>

// Syntaxe la plus explicite, l'écriture est habituelle
// et on rajoute après le template les prérequis
// avec le mot clé 'requires' et ensuite les contraintes
// que l'on veut imposer.
template <typename T> requires std::copyable<T>
auto additionner(T const & lhs, T const & rhs)
{
    return lhs + rhs;
}

// Syntaxe intermédiaire, où le concept remplace
// le mot clé typename dans l'écriture du template.
template <std::convertible_to<double> T>
double conversion_double(T const & t) noexcept
{
    return t * 1.0;
}

// Syntaxe la plus raccourcie, le concept est utilisé
// directement dans la liste des paramètres.
// Notez que nous sommes obligés d'utiliser auto en lieu
// et place des templates, pour garder la généricité.
auto ajouter_42(std::integral auto const & parametre) noexcept
{
    return parametre + 42;
}

int main()
{
    using namespace std::literals;

    // Exemple valide car les std::string sont concaténables.
    additionner("Du"s, "texte"s);
    // Exemple invalide car les std::unique_ptr ne sont pas concaténables.
    additionner(std::make_unique<int>(0), std::make_unique<int>(42));
}
```



```

// Exemple valide car un int peut être converti.
std::cout << conversion_double(0) << std::endl;
// Exemple invalide car une std::string ne peut pas être convertie.
std::cout << conversion_double("std::string"s) << std::endl;

// Exemple valide car int est un type entier.
std::cout << ajouter_42(0) << std::endl;
// Exemple invalide car std::string n'est pas un type entier.
std::cout << ajouter_42("s") << std::endl;

return 0;
}

```

En plus de rendre les erreurs plus compréhensibles, on améliore la qualité du code puisque celui-ci est plus explicite et l'intention de son auteur plus claire. Au lieu d'un simple `typename T` qui signifie *tout*, on précise ce qu'on attend exactement.

Créer ses propres concepts

Jusqu'ici, on pourrait se dire qu'il n'y a pas vraiment plus d'intérêt que les traits. C'est en définissant nos propres concepts que vous allez vous rendre compte de la puissance qu'ils permettent. La syntaxe est précisée ci-dessous.

```

template <typename T>
concept NomDuConcept = requires(T /* un ou plusieurs paramètres */)
{
    // Les propriétés à respecter.
};

```

Imaginons qu'on veuille que n'importe quel type qu'on passe respecte les quatre opérations de base que sont l'addition, la soustraction, la multiplication et la division. Cela signifie qu'on pourra accepter des `int`, des `double` mais aussi n'importe quel type qui implémente cette arithmétique, comme `Fraction` ou `std::complex`. Pour cela, il suffit d'écrire ces opérations à l'intérieur du concept et le compilateur vérifiera ensuite, pour n'importe quel type `T`, qu'elles existent et qu'elles sont possibles.

```

#include <complex>
#include <concepts>
#include <iostream>
#include <string>

template <typename T>
concept ArithmetiqueBasique = requires (T lhs, T rhs)
{
    lhs + rhs;
    lhs - rhs;
    lhs * rhs;
    lhs / rhs;
}

```

```

};

template <ArithmetiqueBasique T>
T calcul(T const& premier, T const& deuxieme, T const& troisieme)
{
    return premier + deuxieme - troisieme;
}

int main()
{
    // Fonctionne pour des types natifs.
    std::cout << calcul(8, 2, 3) << std::endl;

    // Fonction pour des types créés qui implémentent
    // les opérateurs demandés.
    using namespace std::complex_literals;
    std::complex<double> const z1 { 5 };
    std::complex<double> const z2 { 0. + 1i };
    std::complex<double> const z3 { -8. + -3i };
    std::cout << calcul(z1, z2, z3) << std::endl;

    // Ne fonctionne pas pour des chaînes de caractères.
    using namespace std::literals;
    std::cout << calcul("A"s, "B"s, "C"s) << std::endl;

    return 0;
}

```

On peut aussi attendre d'un type qu'il offre un certain service ou qu'un appel à une fonction avec ce type réussisse. Par exemple, le concept ci-dessous ne sera utilisable qu'avec des types qui définissent un service `size`.

```

template <typename T>
concept Taille = requires(T parametre)
{
    parametre.size();
};

```

Il est également possible d'appliquer des contraintes sur le type de retour. Par exemple, on peut vouloir que le template ne fonctionne qu'avec des objets qui sont capables de donner leur ID. On peut tout à fait écrire ce genre de concept et donner comme contrainte que le type de retour soit exactement `std::size_t` en utilisant `std::same_as`.

```

#include <concepts>
#include <iostream>

template <typename T>
concept Id = requires(T parametre)
{
    // Notez que la contrainte sur le paramètre est entourée d'accollades
    {}.
    // La contrainte de retour est à la suite.
    // Le type de retour doit être exactement un std::size_t.
    { parametre.id() } -> std::same_as<std::size_t>;
};

struct A
{
    // Ok.
    std::size_t id() const noexcept
    {
        return 0;
    }
};

struct B
{
    // Ne marchera pas car int != std::size_t.
    int id() const noexcept
    {
        return 1;
    }
};

struct C
{
    // Ne marchera pas car par le bon nom.
    std::size_t obtenir_id() const noexcept
    {
        return 2;
    }
};

template <Id T>
void afficher_id(T const& parametre)
{
    std::cout << parametre.id() << std::endl;
}

int main()
{
    afficher_id(A{});
    afficher_id(B{});
    afficher_id(C{});

    return 0;
}

```

On pourrait aussi utiliser `std::convertible_to` pour être un peu moins restrictif et accepter des types qui peuvent être convertis en `std::size_t`, comme `int`. Il est également possible d'utiliser des traits pour forcer d'autres propriétés, comme être copiable, être un entier, etc.

32.5. Un peu de politique

Le paramétrage par politique est une technique de programmation conçue par Andrei Alexandrescu (un développeur très impliqué dans les langages C++ et D) et popularisée par son livre *Modern C++ Design: Generic Programming and Design Patterns Applied*, datant de 2001. Les classes de politiques sont proches des traits, à ceci près qu'elles ne définissent pas des propriétés sur des types mais des comportements. On parle alors de *programmation par politique* [en anglais, *policy-based design*].

Ainsi, les classes de politiques permettent d'introduire des points de variation dans le comportement d'un programme.



Ne sait-on pas déjà faire cela avec de l'héritage et des fonctions membres virtuelles ?

Très bonne remarque ! Effectivement, on sait déjà introduire des points de variation *dynamiques* dans le comportement d'un programme. Ce que l'on va aborder dans la suite, c'est une manière d'introduire des points de variation *à la compilation*. L'avantage, c'est qu'ils n'auront aucun impact sur les performances à l'exécution. Le paramétrage par politique est donc particulièrement utilisé par les auteurs de bibliothèque, puisqu'il permet de fournir des fonctionnalités adaptables par l'utilisateur sans qu'il y ait le moindre impact sur les performances.

En particulier, la bibliothèque standard ne s'en prive pas. Ainsi, le deuxième argument template d'un `std::vector`, dont nous avons parlé [plus haut](#), n'est rien d'autre qu'une classe de politique qui permet de définir le comportement à adopter lors des opérations d'allocation et de libération de mémoire. Par défaut, il y a une politique pour les utilisations normales mais la classe est adaptable pour les utilisateurs aux besoins particuliers.

Un bon tremplin pour aborder les politiques serait de reprendre notre [exemple de robot nettoyeur](#) et de le réécrire. Pour ce faire, il est intéressant d'avoir en tête les principales composantes d'un point de variation dynamique.

Élément	Rôle
Classe parent	Définit les comportements communs ainsi que les points de variation.

Élément	Rôle
Classe enfant	Implémente les points de variation définis par la classe parent.
Fonction membre virtuelle	Constitue le point de variation en lui-même.
Héritage	Fait le lien entre la définition (classe parent) et les implémentations (classes enfants) des points de variation.

À la base, une classe très hospitalière

La partie commune va être implémentée dans ce que l'on appelle une *classe hôte*, c'est-à-dire une classe qui va accueillir la politique à utiliser. Pour ce faire, il suffit de passer cette politique en argument template. Allons droit au but, voici un exemple pour le nettoyage.

```
template<typename PolitiqueNettoyage>
class Nettoyeur
{
public:
    Nettoyeur(PolitiqueNettoyage const& politique)
        : m_politique{ politique }
    {
    }

    void nettoyer_sol()
    {
        m_politique.preparer_outil();
        m_politique.nettoyer();
        m_politique.ranger_outil();
    }

private:
    PolitiqueNettoyage m_politique;
};
```

Contrairement à l'équivalent dynamique, les points de variation ne sont pas définis explicitement dans la classe parent. Ils sont définis implicitement par l'utilisation qui en est faite, ici dans la fonction `nettoyer_sol`. En d'autres termes, on ne connaît pas l'interface exacte, on fait simplement des suppositions sur ce qu'on attend du paramètre template `PolitiqueNettoyage`.

Points de variation

Les points de variation seront implémentés dans des classes, appelées *classes de politiques* ou simplement *politiques*. Chacune d'entre elles implémentera les différents services qui sont utilisés dans la classes hôtes. À chaque variante de comportement correspondra donc une classe de politique.

```

#include <iostream>

class AvecBalai
{
public:
    void preparer_outil() const noexcept;
    void nettoyer() const noexcept;
    void ranger_outil() const noexcept;
};

void AvecBalai::preparer_outil() const noexcept
{
    std::cout << "Je saisi le balai.\n";
}

void AvecBalai::nettoyer() const noexcept
{
    std::cout << "Je nettoie avec un mouvement de balancier.\n";
    std::cout << "Je me déplace.\n";
}

void AvecBalai::ranger_outil() const noexcept
{
    std::cout << "Je remets le balai dans le placard.\n";
}

class AvecAspirateur
{
public:
    void preparer_outil() const noexcept;
    void nettoyer() const noexcept;
    void ranger_outil() const noexcept;
};

void AvecAspirateur::preparer_outil() const noexcept
{
    std::cout << "Je branche l'aspirateur.\n";
    std::cout << "J'allume l'aspirateur.\n";
    std::cout << "Je saisi le manche de l'aspirateur.\n";
}

void AvecAspirateur::nettoyer() const noexcept
{
    std::cout << "Je nettoie avec des aller-retours.\n";
    std::cout << "Je me déplace.\n";
}

void AvecAspirateur::ranger_outil() const noexcept
{
    std::cout << "J'éteins l'aspirateur.\n";
    std::cout << "Je débranche l'aspirateur.\n";
    std::cout << "Je range l'aspirateur.\n";
}

```

```

class AvecAspirateurAuto
{
public:
    void preparer_outil() const noexcept;
    void nettoyer() const noexcept;
    void ranger_outil() const noexcept;

private:
    bool m_est_charge { false };
};

void AvecAspirateurAuto::preparer_outil() const noexcept
{
    if (!m_est_charge)
    {
        std::cout << "Je recharge l'aspirateur automatique.\n";
    }
    std::cout << "Je démarre l'aspirateur automatique.\n";
}

void AvecAspirateurAuto::nettoyer() const noexcept
{
    std::cout << "Je regarde faire.\n";
}

void AvecAspirateurAuto::ranger_outil() const noexcept
{
    std::cout << "J'éteins l'aspirateur automatique.\n";
    std::cout << "Je range l'aspirateur automatique.\n";
}

```

Comme vous pouvez le constater, l'implémentation en elle-même des différents algorithmes de nettoyage ne change pas. Seulement, il n'y a plus de relation d'héritage et donc plus besoin de fonctions membres virtuelles.

Lier l'implémentation et les points de variation

La dernière pièce manquante au puzzle est donc la mise en lien de la classe hôte avec les classes de politiques. C'est un mécanisme que vous connaissez déjà : l'[instanciation de template](#). Il suffit d'instancier la classe hôte avec une politique au choix et le tour est joué. On obtient alors le même résultat qu'avec les points de variation dynamiques utilisant l'héritage.

```

#include <iostream>

template<typename PolitiqueNettoyage>
class Nettoyeur
{
public:
    Nettoyeur(PolitiqueNettoyage const& politique)

```

```

        : m_politique{ politique }
    {
    }

    void nettoyer_sol()
    {
        m_politique.preparer_outil();
        m_politique.nettoyer();
        m_politique.ranger_outil();
    }

private:
    PolitiqueNettoyage m_politique;
};

class AvecBalai
{
public:
    void preparer_outil() const noexcept;
    void nettoyer() const noexcept;
    void ranger_outil() const noexcept;
};

void AvecBalai::preparer_outil() const noexcept
{
    std::cout << "Je saisi le balai.\n";
}

void AvecBalai::nettoyer() const noexcept
{
    std::cout << "Je nettoie avec un mouvement de balancier.\n";
    std::cout << "Je me déplace.\n";
}

void AvecBalai::ranger_outil() const noexcept
{
    std::cout << "Je remets le balai dans le placard.\n";
}

class AvecAspirateur
{
public:
    void preparer_outil() const noexcept;
    void nettoyer() const noexcept;
    void ranger_outil() const noexcept;
};

void AvecAspirateur::preparer_outil() const noexcept
{
    std::cout << "Je branche l'aspirateur.\n";
    std::cout << "J'allume l'aspirateur.\n";
    std::cout << "Je saisi le manche de l'aspirateur.\n";
}

void AvecAspirateur::nettoyer() const noexcept

```



```

{
    std::cout << "Je nettoie avec des aller-retours.\n";
    std::cout << "Je me déplace.\n";
}

void AvecAspirateur::ranger_outil() const noexcept
{
    std::cout << "J'éteins l'aspirateur.\n";
    std::cout << "Je débranche l'aspirateur.\n";
    std::cout << "Je range l'aspirateur.\n";
}

class AvecAspirateurAuto
{
public:
    void preparer_outil() const noexcept;
    void nettoyer() const noexcept;
    void ranger_outil() const noexcept;

private:
    bool m_est_charge{ false };
};

void AvecAspirateurAuto::preparer_outil() const noexcept
{
    if (!m_est_charge)
    {
        std::cout << "Je recharge l'aspirateur automatique.\n";
    }
    std::cout << "Je démarre l'aspirateur automatique.\n";
}

void AvecAspirateurAuto::nettoyer() const noexcept
{
    std::cout << "Je regarde faire.\n";
}

void AvecAspirateurAuto::ranger_outil() const noexcept
{
    std::cout << "J'éteins l'aspirateur automatique.\n";
    std::cout << "Je range l'aspirateur automatique.\n";
}

int main()
{
    std::cout << "Je commence par le salon.\n";
    AvecAspirateur const pour_le_salon {};
    Nettoyeur<AvecAspirateur> nettoyeur_salon { pour_le_salon };
    nettoyeur_salon.nettoyer_sol();

    std::cout << "\nJe passe au garage.\n";
    AvecBalai const pour_le_garage {};
    Nettoyeur<AvecBalai> nettoyeur_garage { pour_le_garage };
    nettoyeur_garage.nettoyer_sol();
}

```

```
std::cout << "\nMaintenant la chambre.\n";
AvecAspirateurAuto const pour_la_chambre {};
Nettoyeur<AvecAspirateurAuto> nettoyeur_chambre { pour_la_chambre };
nettoyeur_chambre.nettoyer_sol();

return 0;
}
```

Il y a un parallèle très naturel entre les deux techniques, qui transparaît à travers leurs différentes composantes.

Points de variation dynamiques	Politiques
Classe parent	Classe hôte
Classe enfant	Classe de politique
Fonction membre virtuelle	Fonction de la classe de politique
Héritage	Instanciation de template



Pourquoi ne pas toujours utiliser du paramétrage par politique, alors ?

On pourrait être tenté d'abuser de cette nouvelle technique. Cependant, le paramétrage par politique a aussi ses limites.

D'une part, dès lors que le type réel ne peut pas être connu à la compilation (par exemple car il dépend d'une décision de l'utilisateur du programme ou du contenu d'un fichier, ou tout simplement au cœur d'un programme complexe), on ne peut plus utiliser des politiques et le caractère dynamique des points de variation devient une nécessité.

D'autre part, l'utilisation de politiques vient avec un coût en complexité du code, en temps de compilation (un grand nombre d'instanciations de templates peut faire exploser le temps de compilation) ainsi qu'en complexité des erreurs générées à la compilation en cas d'erreur dans le code (par exemple une classe de politique qui n'implémenterait pas l'un des comportements utilisés dans la classe hôte).

En d'autres termes, le paramétrage par politique traîne avec lui tous les inconvénients des templates. Ceux-ci peuvent être limités avec les apports du C++ moderne comme les traits ou les concepts, mais ne disparaissent pas complètement non plus. On retombe sur la réponse que C++ vous donne le plus souvent : ça dépendra des cas.

32.6. Récapitulons !

- Il est tout à fait possible de créer une classe générique, de la même manière qu'une fonction ou qu'un alias.
- L'implémentation des fonctions membres se font directement à l'intérieur de la classe.
- Un template peut prendre en argument non seulement des types, mais aussi des valeurs, comme par exemple un entier.
- Un template peut avoir une valeur par défaut.
- Les traits permettent de vérifier les propriétés d'un type, comme par exemple s'il est copiable, déplaçable, etc.
- Les traits se combinent bien avec `static_assert` pour faire de la programmation par contrat à la compilation.
- C++20 a introduit les concepts pour répondre au besoin de forcer des propriétés sur un type template.
- Lorsqu'une propriété d'un concept n'est pas respectée, la compilation échoue avec une erreur bien plus précise et lisible qu'avant.
- La programmation par politique permet d'introduire des points de variation à la compilation.
- La partie commune est implémentée dans une classe hôte.
- Les points de variation se font dans des classes de politique.
- La programmation par politique a aussi des contraintes, notamment en terme de complexité et de temps de compilation.

33

Ça, c'est du SOLID !

Non, il n'y a pas de faute, c'est bien SOLID sans e final. Il s'agit d'un acronyme regroupant cinq principes génériques, qui, bien compris et bien utilisés, permettent d'écrire du code plus robuste, plus facilement maintenable et modifiable. Promus par Robert C. Martin, vous retrouverez ces principes notamment dans son livre sur les bonnes pratiques de développement *Clean Code* (également disponible en français), que nous vous invitons à parcourir dès ce livre terminé.

Note > Ce que nous allons voir maintenant n'est absolument pas spécifique à C++. Au contraire, connaître et appliquer ces bonnes pratiques vous aidera dans n'importe quel langage.

33.1. S — Single responsibility principle

Le principe

En français, on le traduirait par *Principe de responsabilité unique*. Il énonce qu'une classe, ou une fonction, ne devrait avoir qu'une seule et unique responsabilité, bien définie et clairement identifiable. En d'autres mots, elle ne doit avoir qu'une seule raison de changer.

Dans la vie d'un programme, il est extrêmement rare d'écrire une classe ou une fonction une seule fois, sans plus jamais lui modifier la moindre ligne de code. Déjà, il y a les bugs et correctifs à appliquer, car personne ne peut écrire du code parfait. Et puis il y a aussi les modifications suite à des changements techniques, métiers, à une nouvelle compréhension du projet, etc. En bref, le code évolue plus ou moins régulièrement. Quand une classe, ou une fonction, n'a qu'une raison de changer, ça procure plusieurs avantages :

- les tests sont plus faciles à écrire ;
- l'organisation du code est meilleure, puisque chaque classe est plus petite ;
- moins notre classe a de responsabilités, moins elle dépend des autres.

À l'inverse, si on lui a attribué trop de responsabilités, apporter des modifications à l'une d'entre elles *peut créer des bugs ou casser des fonctionnalités* sur d'autres, que nous n'avons pourtant pas touchées. De plus, les tests seront plus gros et plus complexes à

écrire, avec la possibilité qu'ils ne soient pas très efficaces. Quant au code, il devient moins compréhensible, car trop d'éléments se retrouvent en un même endroit.

LE SRP ET `STD::STRING`

Si une grande partie de la bibliothèque standard respecte ce principe, ce n'est pas le cas de `std::string`, qui contient plusieurs fonctions de recherche de caractères, contrairement aux autres conteneurs, qui ne contiennent que ce qu'il faut pour l'ajout, la suppression et l'accès aux éléments.

Un exemple problématique...

Imaginons que l'on veuille se connecter à une base de données afin d'en extraire certaines informations brutes, qui seront transformées et retraitées avant d'être affichées. La classe fournissant ces services pourrait prendre la forme suivante.

```
#include <string>
#include <vector>

class Processus
{
public:
    void connexion_base_donnees(std::string const & utilisateur,
                                std::string const & mot_de_passe);
    std::vector<int> recuperation_donnees();
    std::vector<int> transformation(std::vector<int> const &
                                    donnees_brutes);
    void generation_sortie(std::vector<int> const & donnees_transformees);
};
```

On voit tout de suite que cette classe cumule trop de responsabilités. Elle doit en effet gérer la base de données, avec la connexion et la récupération des données, leur transformation et leur export. Si demain on décide de changer de base de données et d'en utiliser une autre, on doit modifier le code. Si après-demain on change le format de sortie du code, on doit encore une fois modifier le code. Si dans une semaine on apprend qu'il y a d'autres opérations de transformation à prendre en compte, hop, encore une modification du code.

Vous noterez d'ailleurs que son identificateur est peu clair. `Processus`, ça n'indique pas ce qu'est censé faire la classe, ça n'aide pas à comprendre son rôle. Et bien souvent, quand on a du mal à trouver un nom assez clair, c'est le signe que la classe a trop de responsabilités.

...et sa solution

Étant donné qu'on a trois grandes responsabilités, on peut les répartir en trois classes ayant chacune son rôle bien défini : une pour gérer la base de données, une autre pour gérer les diverses transformations à appliquer aux données et une dernière pour l'affichage des données retraitées.

```
class BaseDonnees
{
public:
    BaseDonnees(std::string const & utilisateur, std::string const &
mot_de_passe);
    void connexion_base_donnees();
    std::vector<int> recuperation_donnees();
};

class TransformationDonnees
{
public:
    TransformationDonnees(std::vector<int> const & donnees_brutes);
    std::vector<int> transformation();
};

class GenerationSortie
{
public:
    GenerationSortie(std::vector<int> const & donnees_transformees);
    void generation_sortie();
};
```

Le fait que chaque classe a une responsabilité claire se voit dans leurs noms, qui sont bien plus parlants qu'un vague *Processus*.

33.2. O — Open-closed principle

Le principe

Nous l'avons brièvement abordé [dans le chapitre sur la sémantique d'entité](#). Il s'agit du *principe ouvert-fermé* qui énonce qu'une classe (ou une fonction), dont le comportement a été validé et vérifié, doit être à la fois *ouverte aux extensions* et *fermée aux modifications*. Autrement dit, on doit pouvoir étendre son comportement sans avoir besoin de modifier son code source.

Comme nous l'avons dit plus haut, le code évolue fréquemment, tant pour des raisons techniques (comme un bug ou un problème de performance à corriger, etc.) que pour

des raisons métiers (un nouveau besoin, une meilleure compréhension d'un besoin existant, etc.). Cependant, l'évolution du code ne doit pas casser l'existant. Si le comportement d'une classe ou d'une fonction a été testé et validé, le modifier pourrait introduire des bugs et des régressions.

L'OCF ET LA BIBLIOTHÈQUE STANDARD

La bibliothèque standard est typiquement fermée aux modifications mais ouverte aux extensions. Par exemple, vous pouvez créer vos propres conteneurs et utiliser les algorithmes standards dessus, dès lors que vos conteneurs utilisent le concept des itérateurs.

De la même manière, les flux sont également interchangeables et sont de bons exemples de conception selon le principe ouvert-fermé.

Un exemple problématique...

Nous avons déjà abordé ce point avec notre [exemple de nettoyage](#), nous allons donc en voir un autre. Admettons que l'on veuille écrire une fonction qui calcule la TVA applicable selon le type de produit qu'on a. Le code suivant est une solution possible, qui répond au besoin exprimé.

```
#include <iostream>

enum class Tva
{
    Tva5Pourcent,
    Tva10Pourcent,
    Tva20Pourcent
};

struct Produit
{
    double prix;
    Tva taxe = Tva::Tva20Pourcent;
};

double prix_ttc(Produit const& produit)
{
    if (produit.taxe == Tva::Tva5Pourcent)
    {
        return produit.prix + (produit.prix * 5 / 100);
    }
    else if (produit.taxe == Tva::Tva10Pourcent)
```

```

    {
        return produit.prix + (produit.prix * 10 / 100);
    }
    else
    {
        return produit.prix + (produit.prix * 20 / 100);
    }
}

int main()
{
    Produit const voiture { 10000 };
    Produit const bois_chauffage { 100, Tva::Tva10Pourcent };
    Produit const pates { 3, Tva::Tva5Pourcent };

    std::cout << "Prix TTC de cette superbe voiture : "
    << prix_ttc(voiture) << std::endl;
    std::cout << "Prix TTC de ce bois pour vous réchauffer en hiver : "
    << prix_ttc(bois_chauffage) << std::endl;
    std::cout << "Prix TTC de ces délicieuses pâtes : "
    << prix_ttc(pates) << std::endl;
    return 0;
}

```

Seulement voilà, si demain un nouveau taux est ajouté, on doit alors changer le code de la fonction `prix_ttc`. Elle n'est donc pas fermée aux modifications. De plus, le code pourrait vite devenir difficile à maintenir.

...et sa solution

Plusieurs solutions s'offrent à nous. Nous pourrions utiliser des points de variations dynamiques avec le polymorphisme, de la même manière que ce que nous avons vu avec notre robot nettoyeur, ou bien utiliser les templates. C'est cette deuxième solution qui est implémentée ci-dessous.

```

#include <iostream>

struct Taxe5Pourcent
{
    double prix_ttc(double prix_ht) const noexcept;
};

double Taxe5Pourcent::prix_ttc(double prix_ht) const noexcept
{
    return prix_ht + (prix_ht * 5 / 100);
}

struct Taxe10Pourcent
{
    double prix_ttc(double prix_ht) const noexcept;
};

```



```

double Taxe10Pourcent::prix_ttc(double prix_ht) const noexcept
{
    return prix_ht + (prix_ht * 10 / 100);
}

struct Taxe20Pourcent
{
    double prix_ttc(double prix_ht) const noexcept;
};

double Taxe20Pourcent::prix_ttc(double prix_ht) const noexcept
{
    return prix_ht + (prix_ht * 20 / 100);
}

struct Produit
{
    double prix;
};

template <typename CalculateurTaxe>
double prix_ttc(Produit const& produit, CalculateurTaxe const &
calculateur)
{
    return calculateur.prix_ttc(produit.prix);
}

int main()
{
    Taxe5Pourcent const taxe5Pourcent {};
    Taxe10Pourcent const taxe10Pourcent {};
    Taxe20Pourcent const taxe20Pourcent {};

    Produit const voiture{ 10000 };
    Produit const bois_chauffage { 100 };
    Produit const pates { 3 };

    std::cout << "Prix TTC de cette superbe voiture : "
        << prix_ttc(voiture, taxe20Pourcent) << std::endl;
    std::cout << "Prix TTC de ce bois pour vous l'hiver : "
        << prix_ttc(bois_chauffage, taxe10Pourcent) << std::endl;
    std::cout << "Prix TTC de ces délicieuses pâtes : "
        << prix_ttc(pates, taxe5Pourcent) << std::endl;

    return 0;
}

```

L'OCP trouve tout son intérêt dans les portions de code qui s'interfaçent avec l'extérieur, comme la base de données, le format de sortie, etc., ainsi qu'avec les algorithmes métiers, comme le calcul de scores de jeux, d'impôts, etc., en bref, tout ce qui est susceptible d'évoluer, que ce soit pour implémenter de nouveaux besoins ou suite à une meilleure compréhension des besoins actuels.

33.3. L — Liskov substitution principle

Le principe

Ce principe, énoncé par Barbara Liskov en 1987, vous le connaissez déjà, mais on va en remettre une couche car il est vraiment d'une grande importance dans la phase de conception et de réflexion : *Si une propriété P est vraie pour une instance x d'un type T , alors cette propriété P doit rester vraie pour toute instance y d'un sous-type de T .*

Le compilateur ne vous empêchera jamais de faire des héritages qui n'ont aucune logique (comme un `Ordinateur` avec une `Brosse` ou bien `Chien` avec `BaseDeDonnees`). Pour lui, tant que la syntaxe est respectée, tout va bien. C'est pour ça qu'il faut se poser deux questions avant d'envisager tout héritage :

- Cet héritage respecte-t-il bien une relation EST-UN ?
- Cet héritage respecte-t-il le LSP du point de vue de la programmation par contrat ? Autrement dit, y a-t-il une relation EST-SUBSTITUABLE-À ?

Se poser uniquement la première question ne suffit pas, car la programmation par contrat demande que *certaines conditions soient respectées*. Il faut absolument s'assurer qu'un type est substituable dans le respect du LSP.

- Les préconditions ne peuvent pas être renforcées dans une sous-classe.
- Les postconditions ne peuvent pas être affaiblies dans une sous-classe.
- Les invariants doivent être préservés.

Un exemple problématique...

L'exemple que nous allons prendre est réel et est tiré de la bibliothèque standard Java, dans laquelle la classe `Stack`, qui représente une pile, est dérivée de la classe `Vector`, qui est semblable à notre `std::vector` de C++. Puisque nous sommes dans une relation d'héritage, *toute fonction membre de `Vector` est disponible dans `Stack`*, dont `insertElementAt`, qui permet d'ajouter un élément à un emplacement particulier.

Sauf que, dans le cas d'une pile, nous ne sommes capables en théorie que d'ajouter un élément au-dessus du dernier arrivé et nulle part ailleurs. Le code suivant, écrit en Java, est donc une violation du LSP.

```
// On crée une pile de chaînes de caractères, OK.
Stack<String> stack = new Stack<String>();
```

```
// J'entasse les éléments un par un, OK.
stack.push("1");
stack.push("2");
stack.push("3");
// J'insère le "4" à l'emplacement 1.
// Viol évident du LSP, BOOM !
stack.insertElementAt("4", 1);
```

Ce n'est pas la seule fonction membre dans ce cas, on peut aussi citer `get`, pour obtenir une valeur à un emplacement précis, ou encore `removeElementAt`, qui supprime l'élément à la position demandée.

...et sa solution

Dans cet exemple, la solution la plus simple est de *supprimer l'héritage*, puisqu'il n'y a pas de lien entre une pile et un tableau dynamique. Par contre, on pourrait très bien envisager que `Stack` utilise en interne un tableau dynamique (composition directe), comme nous l'avons fait avec notre classe `Pile` dans le [chapitre sur les constructeurs](#).

Si l'on insiste vraiment autant sur l'importance du LSP, c'est que le non-respect de cette loi est un *bug*, alors que les autres principes sont plus des aides pour faciliter la maintenance et l'évolution du code dans l'avenir.

33.4. I — Interface segregation principle

Le principe

En français, on parle du *principe de ségrégation des interfaces*. Bien que C++ n'ait pas d'interface dans le même sens que C# ou Java, ce principe s'applique néanmoins. Il stipule *qu'aucun client ne devrait dépendre de méthode qu'il n'utilise pas*.

Plus on dépend d'autres fonctions ou classes, plus on est fortement couplé et plus les modifications seront nombreuses si un paramètre change, avec tous les risques de bugs et de régressions que cela implique. D'où l'intérêt de ne dépendre que de ce qui nous est utile, ce qui réduit le couplage. Mais pour cela, il faut que les interfaces dont nous dépendons ne soient pas trop grosses.

On utilise donc le célèbre principe du *diviser pour mieux régner* avec nos interfaces, quitte à en avoir plus, de manière à ce que chacune ne contienne que le nécessaire.

SRP ET ISP, DES COUSINS PAS SI ÉLOIGNÉS

Ce principe rappelle sous certains aspects le *Single Responsibility Principle*. En effet, dans les deux cas, on veut limiter les responsabilités que porte un concept. La nuance entre les deux est néanmoins plus ténue en C++, qui, on le rappelle, ne dispose pas d'un concept d'interfaces comme celui de Java.

L'inconvénient, c'est qu'il est facile de mal l'appliquer en créant du coup trop de petites interfaces. Pour évaluer si une interface doit être séparée en plusieurs morceaux, il faut se poser la question de savoir si on aura vraiment besoin d'utiliser un de ces morceaux pris séparément. Si oui, on peut le faire, sinon c'est qu'on tombe dans le piège d'en faire trop.

Un exemple problématique...

L'exemple qu'on retrouve souvent quand on veut illustrer l'ISP est celui d'une interface graphique. Au sein de cette dernière, on retrouve les boutons, qui sont cliquables mais pas copiables, les labels, qui sont copiables mais pas cliquables, les barres de chargement qui ne sont ni cliquables ni copiables, etc.

Le code suivant illustre le non-respect de l'ISP, avec une interface `ElementGraphique` trop grosse. En effet, elle nous force à implémenter des services dont nous ne voulons pas.

```
#include <iostream>

class ElementGraphique
{
public:
    virtual void copier() const noexcept = 0;
    virtual void clic() const noexcept = 0;
    virtual void afficher() const noexcept = 0;
    // ...
    // Encore d'autres services.
};

class Bouton : public ElementGraphique
{
public:
    void afficher() const noexcept override;
    void clic() const noexcept override;
    // On est forcé d'implémenter les autres fonctions membres.
    // Pourtant, elles ne nous intéressent pas.
    void copier() const noexcept override;
};
```

```

void Bouton::afficher() const noexcept
{
    std::cout << "Je suis un bouton qui s'affiche.\n";
}

void Bouton::clic() const noexcept
{
    std::cout << "On me clique dessus, je réagis.\n";
}

void Bouton::copier() const noexcept
{
    // Que mettre ?
    // Laisser le corps de la fonction vide ?
    // Lancer une exception ? Une assertion ?
}

class Label : public ElementGraphique
{
public:
    void afficher() const noexcept override;
    void copier() const noexcept override;
    // On est forcé d'implémenter les autres fonctions membres.
    // Pourtant, elles ne nous intéressent pas.
    void clic() const noexcept override;
};

void Label::afficher() const noexcept
{
    std::cout << "Je suis un label qui s'affiche.\n";
}

void Label::copier() const noexcept
{
    std::cout << "L'utilisateur copie mon contenu.\n";
}

void Label::clic() const noexcept
{
    // Même problème, que mettre ici ?
}

```

...et sa solution

Comme indiqué plus haut, une solution envisageable est de diviser nos interfaces en d'autres plus fines. Ici, une division possible serait d'avoir trois interfaces : pour les concepts d'affichage, de clics et de copies. Les classes `Bouton` et `Label` sont désormais moins couplées et n'implémentent que ce qui les concerne.

```

#include <iostream>

class ElementAffichable
{
public:
    virtual void afficher() const noexcept = 0;
};

class ElementCopiable
{
public:
    virtual void copier() const noexcept = 0;
};

class ElementCliquable
{
public:
    virtual void clic() const noexcept = 0;
};

class Bouton : public ElementAffichable, public ElementCliquable
{
public:
    void afficher() const noexcept override;
    void clic() const noexcept override;
};

void Bouton::afficher() const noexcept
{
    std::cout << "Je suis un bouton qui s'affiche.\n";
}

void Bouton::clic() const noexcept
{
    std::cout << "On me clique dessus, je réagis.\n";
}

class Label : public ElementAffichable, public ElementCopiable
{
public:
    void afficher() const noexcept override;
    void copier() const noexcept override;
};

void Label::afficher() const noexcept
{
    std::cout << "Je suis un label qui s'affiche.\n";
}

void Label::copier() const noexcept
{
    std::cout << "L'utilisateur copie mon contenu.\n";
}

```

33.5. D — Dependency inversion principe

Le principe

Terminons le tour des principes SOLID avec l'*inversion des dépendances*, qui comporte deux points. Ce principe est généralement le plus difficile à comprendre quand on découvre SOLID. Pourtant, si vous codez en suivant l'OCP et le LSP, vous avez déjà presque tout fait.

Le premier point stipule que *les modules de haut niveau ne doivent pas dépendre de modules de bas niveau*, mais plutôt que les deux devraient *dépendre d'abstractions*. Reste à savoir ce qu'on entend par modules de haut ou de bas niveau.

- Les modules de *haut niveau* sont ceux qui contiennent la logique de l'application (comme le calcul de taxes, la mise à jour des stocks de chaussettes jaunes, etc.).
- Les modules de *bas niveau* implémentent des fonctionnalités de base qui seront utilisées par les modules de haut niveau (comme la manipulation d'une base de données, d'une connexion HTTP, etc.). Les conteneurs de la bibliothèque standard sont de bons exemples de modules de bas niveau.

Si on ne procède pas de la sorte, alors nos classes sont fortement couplées et un changement de bas niveau provoquera des changements dans les modules de haut niveau. De plus, le code devient moins réutilisable puisqu'on dépend de détails techniques concrets.

Le deuxième point complète le premier en précisant que *les dites abstractions ne doivent pas dépendre de détails*, mais que ce sont *les détails qui doivent dépendre d'abstractions*. Et comme pour les autres principes, les interfaces vont jouer un rôle essentiel là-dedans.

Pour prendre une comparaison tirée de la vie réelle, personne ne soude directement les fils d'une lampe sur le câblage électrique de la maison (détails de bas niveau). Au contraire, les deux se basent sur une interface commune, la prise, ce qui permet de brancher la lampe n'importe où et d'utiliser la prise libérée pour un autre appareil.

Un exemple problématique...

Dans le code suivant, on retrouve plusieurs violations du principe d'inversion des dépendances. Saurez-vous les identifier ?

```
#include <string>

// Service de bas niveau pour utiliser le protocole HTTP.
class ServiceHttp
{
public:
    void requete() const noexcept;
    void envoyer_reponse(std::string const & reponse) const noexcept;
};

// Service de bas niveau pour manipuler une base de données.
class BaseDonnees
{
public:
    void ecrire_donnees(int donnee) const noexcept;
};

// Service de haut niveau, qui contient la logique métier.
class TraitementEntiers
{
public:
    void traiter_reception_entier(int entier) const noexcept;

private:
    ServiceHttp m_service_http;
    BaseDonnees m_base_donnees;
};

void TraitementEntiers::traiter_reception_entier(int entier) const
noexcept
{
    using namespace std::literals;
    m_base_donnees.ecrire_donnees(entier);
    m_service_http.envoyer_reponse("Bien reçu !"s);
}
```

D'abord, notre classe de haut niveau dépend de services de bas niveau, à savoir la manipulation de la base de données et le service HTTP. Apporter des changements à l'un d'entre eux, que ce soit pour une évolution des besoins métiers, ou parce qu'on a trouvé une solution plus performante, entraînera alors des modifications dans nos modules de haut niveau.

De plus, la réponse envoyée dépend de détails d'implémentation (`std::string`) et non d'abstractions. Si demain, au lieu de renvoyer une chaîne de caractères, on souhaite

répondre avec du JSON, du XML, une image, ou autre chose, nous sommes obligés de modifier deux services, violant au passage l'OCP.

...et sa solution

Le code suivant corrige les problèmes remontés en faisant en sorte que tant les modules de haut niveau que ceux de bas niveau se basent sur des interfaces communes, grâce à `IReponse` et ses descendants. De plus, notre service `TraitementEntiers` n'est plus fortement lié à une base de données et à un service HTTP, mais à leurs abstractions.

```
#include <memory>

struct IReponse
{
};

struct ReponseString : public IReponse
{
};

struct ReponseJson : public IReponse
{
};

struct IServiceHttp
{
    virtual void requete() const noexcept = 0;
    // On ne dépend plus de détails bas niveau pour répondre
    // à une requête.
    virtual void envoyer_reponse(IReponse const& reponse) const noexcept =
    0;
};

// Service bas niveau pour utiliser le protocole HTTP.
class ServiceHttp : public IServiceHttp
{
public:
    void requete() const noexcept override;
    void envoyer_reponse(IReponse const & reponse) const noexcept
    override;
};

struct IBaseDonnees
{
    virtual void ecrire_donnees(int donnee) const noexcept = 0;
};

// Service bas niveau pour manipuler une base de données.
class BaseDonnees : public IBaseDonnees
```

```

{
public:
    void ecrire_donnees(int donnee) const noexcept override;
};

// Service haut niveau, qui contient la logique métier.
class TraitementEntiers
{
public:
    void traiter_reception_entier(int entier) const noexcept;

private:
    // On dépend maintenant d'interfaces.
    // Notez l'utilisation de std::unique_ptr
    // pour permettre le polymorphisme.
    std::unique_ptr<IServiceHttp> m_service_http;
    std::unique_ptr<IBaseDonnees> m_base_donnees;
};

void TraitementEntiers::traiter_reception_entier(int entier) const
noexcept
{
    m_base_donnees->ecrire_donnees(entier);

    ReponseString response {};
    m_service_http->envoyer_reponse(response);
}

```

Cette souplesse permet de les remplacer plus facilement, notamment dans le cadre des tests unitaires, où l'on aurait non plus une vraie base de données mais une simple capsule au comportement prédéterminé. Dans notre exemple, nous pourrions remplacer le service HTTP par un faux service dont les résultats sont fixes et connus d'avance.

33.6. Pour quelques principes de plus

Il est impossible de faire le tour de tous les principes qui existent. De plus, certains sont spécifiques et leur intérêt dépend donc de la situation. Néanmoins, il nous paraît bien de vous en lister encore quelques-uns assez connus. Vous trouverez une littérature abondante sur le sujet, tant en anglais qu'en français, aussi bien sur Internet que dans les livres.

- *STUPID*, qui est un acronyme à la manière de SOLID et qui regroupe un ensemble de mauvaises pratiques qui rendront votre code plus difficile à maintenir et à faire évoluer.
- *KISS*, qui est l'acronyme de *Keep it simple, stupid* [ne complique pas les choses], encourage à éviter toute complexité non indispensable. La plupart des programmes UNIX suivent ce principe, avec beaucoup de petits utilitaires différents (*ls*, *grep*, *wc*, etc.) qui se combinent entre eux avec *|*.

- *DRY*, qui est l'acronyme de *Don't repeat yourself* [ne te répète pas], invite à se méfier des copier-coller trop fréquents dans le code, qui sont le signe qu'on répète un même concept, ce qui peut introduire des bugs et autres problèmes. À ce principe s'oppose *WET* (*Write Everything Twice*) qui désigne le code qui n'est pas *DRY* et donc duplique énormément.
- *YAGNI*, qui est l'acronyme de *You ain't gonna need it* [tu n'en auras pas besoin], insiste sur la nécessité de ne pas implémenter en avance une fonctionnalité dont on n'a pas la certitude qu'elle sera utilisée plus tard.
- La *règle du boy-scout* consiste à rendre le code un peu meilleur que tel qu'on l'a trouvé, que ce soit en renommant une variable peu claire, en séparant en plusieurs morceaux une fonction trop grande, etc.

33.7. Récapitulons !

- Les principes *SOLID* nous aident à écrire un code meilleur, plus robuste et plus facilement adaptable.
- Le *S* incite à ce que chaque classe ou fonction n'ait qu'une seule responsabilité, clairement définie.
- Le *O* incite à fermer une classe ou fonction aux modifications (sauf bug) tout en la gardant ouverte aux extensions, notamment grâce aux templates ou à l'héritage.
- Le *L* incite à bien réfléchir à la substituabilité d'un type à un autre et à ce que ça implique sur le respect des préconditions, postconditions et invariants.
- Le *I* incite à ne pas avoir de trop grosses interfaces, ce qui nous forcerait à implémenter des services que nous ne voulons pas rendre.
- Le *D* incite à ne pas dépendre de détails de bas niveau mais d'abstractions de haut niveau.
- Il existe beaucoup d'autres principes.

34

Le voyage ne fait que commencer

Nous voilà arrivés au dernier chapitre de ce livre. Ensemble, nous avons appris beaucoup de choses sur le C++, l'environnement et les outils qui vont avec, mais aussi les bonnes pratiques de développement. Bien sûr, il vous reste encore beaucoup à apprendre, mais vous avez déjà un bon bagage, suffisant pour commencer à programmer par vous-même et poursuivre le voyage seul. Désormais il s'agit de vos choix, de votre chemin. Nous allons simplement encore vous donner quelques pistes pour aller plus loin.

34.1. Davantage sur C++

C++ est un langage tellement riche qu'une vie entière ne suffirait pas pour en maîtriser toutes les subtilités. Néanmoins, vous progresserez beaucoup au contact des autres développeurs et en vous formant auprès d'experts. Voici quelques livres, parmi beaucoup d'autres, qui vous aideront à affiner vos connaissances.

Coder efficacement – Bonnes pratiques et erreurs à éviter en C++ par Philippe Dunki est né de plusieurs années d'intervention sur les forums francophones (sous le pseudo koala01) pour aider les autres développeurs à comprendre leurs erreurs. On y apprend énormément sur le C++ et le pourquoi de beaucoup de bonnes pratiques, avec en prime une étude de cas concrète. C'est vraiment un livre de qualité, qui fera un bon complément à ce que vous avez vu à nos côtés.

C++ Primer par Stanley B. Lippman, Josée Lajoie et Barbara E. Moo est un livre introductif qui vous apprendra néanmoins beaucoup. Gros pavé de plus de 1000 pages, il est recommandé pour apprendre le C++ si l'on a déjà une expérience en programmation. Vous y découvrirez assurément des choses que nous n'avons pas abordées ici. Il n'est cependant disponible qu'en anglais.

Pour aller plus en profondeur, Scott Meyers a écrit une série de livres intitulés *Effective C++*, *More Effective C++* et *Effective Modern C++*, ce dernier étant conçu spécifiquement pour C++11 et au-delà. Vous pourrez les trouver en français ou en anglais, selon vos goûts. Chacun d'entre eux est découpé en items ou points, abordant une règle ou une bonne pratique et les raisons de celle-ci.

Quand on parle de templates, il est rare de ne pas citer Andrei Akexandrescu, l’auteur de *Modern C++ Design*. Bien que ce livre ait plus de 20 ans, il reste une bonne référence (en anglais) pour se plonger dans les templates et mieux comprendre la puissance qu’ils nous offrent. C’est également le livre qui a introduit la programmation par politique.

Il est impossible de citer tous les livres. Beaucoup d’entre eux ne sont disponibles dans leur dernière édition qu’en anglais, voire exclusivement dans cette langue. Cela peut être un frein pour certains, mais nous vous encourageons malgré tout à tenter de les lire, vous verrez qu’ils sont souvent écrits dans un style relativement simple, et sont plus accessibles qu’ils n’y paraissent.

34.2. Davantage sur le développement

Notre but n’était pas uniquement de vous enseigner C++, sa syntaxe, ses règles. Nous avons cherché également à vous apprendre à penser comme un développeur. Nous vous encourageons vivement à approfondir le sujet. Ces connaissances vous serviront, que vous continuiez avec C++ ou que vous passiez à un autre langage.

L’un des livres les plus imposants sur la programmation orientée objet est celui de Bertrand Meyer, intitulé sobrement *Conception et programmation orientées objet*. Sachez que le livre compte plus de 1000 pages et qu’il va en profondeur dans la théorie. Notez également que son auteur est un grand promoteur de la programmation par contrat.

The Pragmatic Programmer, écrit par Andrew Hun et David Thomas, reste un des plus recommandés malgré son ancienneté. Il contient beaucoup de bons conseils non seulement sur la programmation, mais aussi sur la communication avec les autres membres d’une équipe, l’efficacité, etc. Ce livre constitue une des bases du mouvement du [software craftsmanship](#), une approche du développement qui met l’accent sur les compétences du développeur et la qualité du code.

Toujours dans cette optique de produire le meilleur code possible, les livres *Clean code* et *The Clean Coder* de Robert C. Martin apportent beaucoup, en détaillant les pratiques et principes qu’un programmeur peut utiliser pour rendre son code encore plus élégant et propre, avec plusieurs études de cas.

Enfin, citons *Design Patterns – Tête la première*, par Eric Freeman, Elisabeth Freeman, Kathy Sierra et Bert Bates, un ouvrage assez ancien mais dont les principes restent valables. Il apporte des solutions [nommées en français *patrons de conception*] pour aider à résoudre des problèmes récurrents d’architecture et de conception.

Il ne nous reste plus qu’à vous souhaiter bon voyage !

[Pratique] Gérer les erreurs d'entrée • Solutions

Section 5.6, [Pratique] Gérer les erreurs d'entrée • Partie I

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Entre ton age : ";
    unsigned int age { 0 };

    if (std::cin >> age)
    {
        // Tout va bien.
        std::cout << "Tu as " << age << " ans.\n";
    }
    else
    {
        // Si std::cin a rencontré une erreur quelconque.
        std::cout << "Tu n'as pas rentré un entier, il y a eu une
erreur." << std::endl;
        std::cin.clear(); // On remet std::cin dans un état fonctionnel.
        std::cin.ignore(255, '\n'); // On vide les caractères mémorisés.
    }

    std::cout << "Maintenant, vérifions que tout va bien."
        << std::endl;
    std::cout << "Entre ton nom : ";
    std::string nom { "" };
    std::cin >> nom;
    std::cout << "Tu t'appelles " << nom << ".\n";

    return 0;
}
```

Section 6.4, [Pratique] Gérer les erreurs d'entrée • Partie II

```
#include <iostream>

int main()
{
    std::cout << "Quel jour es-tu né ? ";
    int jour { 0 };
}
```

```

// Comme std::cin >> jour renvoie false s'il y a eu
// une erreur, afin de rentrer dans la boucle,
// on prend la négation de l'expression avec !
while (!(std::cin >> jour))
{
    std::cout << "Entrée invalide. Recommence." << std::endl;
    std::cin.clear();
    std::cin.ignore(255, '\n');
}

std::cout << "Quel mois es-tu né ? ";
int mois { 0 };

// Même version avec un do while.
do
{
    if (std::cin.fail())
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();
        std::cin.ignore(255, '\n');
    }
} while (!(std::cin >> mois));

std::cout << "Tu es né le " << jour << "/" << mois << "."
    << std::endl;
return 0;
}

```

Les deux versions sont légèrement différentes dans ce qu'elles font. La première demande l'entrée, puis tant qu'elle est invalide, elle nettoie puis demande à nouveau. La seconde vérifie s'il y a eu une quelconque erreur, la nettoie puis demande l'entrée et recommence si l'entrée est invalide. Peu importe celle que vous avez choisie d'utiliser, l'essentiel est de nettoyer nos entrées et d'avoir des programmes robustes. Et c'est justement ce que nous faisons.

Section 10.5, [Pratique] Gérer les erreurs d'entrée • Partie III

```

#include <iostream>
#include <string>

void entree_securisee(int & variable)
{
    while (!(std::cin >> variable))
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();
        std::cin.ignore(255, '\n');
    }
}

```

```

void entree_securisee(double & variable)
{
    while (!(std::cin >> variable))
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();
        std::cin.ignore(255, '\n');
    }
}

int main()
{
    int jour { 0 };
    std::cout << "Quel jour es-tu né ? ";
    entree_securisee(jour);

    int mois { 0 };
    std::cout << "Quel mois ? ";
    entree_securisee(mois);

    int annee { 0 };
    std::cout << "Quelle année ? ";
    entree_securisee(annee);

    double taille { 0.0 };
    std::cout << "Quelle taille ? ";
    entree_securisee(taille);

    std::cout << "Tu es né le " << jour << "/" << mois << "/" << annee
        << " et tu mesures " << taille << "m." << std::endl;
    return 0;
}

```

Vous remarquerez que même si le code n'est plus répété à chaque entrée, nos fonctions restent quasiment identiques, la seule différence résidant dans le paramètre qu'elles attendent. Finalement, on duplique toujours le code. Mais comme vous vous en doutez, il y a une solution pour ça, que nous verrons en temps voulu.

Section 11.5, [Pratique] Gérer les erreurs d'entrée • Partie IV

```

#include <iostream>
#include <limits>

void entree_securisee(int & variable)
{
    while (!(std::cin >> variable))
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();

        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
}

```



```

    }
}

void entree_securisee(double & variable)
{
    while (!(std::cin >> variable))
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();

        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
}

int main()
{
    int annee { 0 };
    std::cout << "Donne-moi une année (essaye de taper un long texte,
je ne broncherais pas) : ";
    entree_securisee(annee);

    return 0;
}

```

Notez que le code est toujours dupliqué. Nous verrons progressivement comment améliorer ce problème dans les chapitres qui suivent.

Section 11.7, [Pratique] Gérer les erreurs d'entrée • Partie V

```

#include <iostream>
#include <limits>

int main()
{
    std::cout << "Quel jour es-tu né ? ";
    int jour { 0 };
    while (!(std::cin >> jour) || jour < 1 || jour > 31)
    {
        if (std::cin.eof())
        {
            // Dans le cas où le flux est fermé,
            // on renvoie une exception.
            throw std::runtime_error("Le flux a été fermé !");
        }
        else if (std::cin.fail())
        {
            // En cas d'entrée invalide.
            std::cout << "Entrée invalide. Recommence." << std::endl;
            std::cin.clear();

            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        }
        else

```

```

    {
        // Cas où l'entrée n'est pas correcte.
        std::cout << "Le jour doit être compris entre 1 et 31."
                  << std::endl;
    }
}

std::cout << "Quel mois es-tu né ? ";
int mois { 0 };
while (!(std::cin >> mois) || mois < 1 || mois > 12)
{
    if (std::cin.eof())
    {
        throw std::runtime_error("Le flux a été fermé !");
    }
    else if (std::cin.fail())
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();
    }

    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    else
    {
        std::cout << "Le mois doit être compris entre 1 et 12."
                  << std::endl;
    }
}

std::cout << "Tu es né le " << jour << "/" << mois << "."
          << std::endl;
return 0;
}

```

Vous remarquerez qu'on a toujours le dilemme entre fonction ou gestion des erreurs plus personnalisée. Bientôt, nous trouverons une solution à ce problème.

Attention > Ceci n'est qu'un petit exercice illustratif. Notez bien que dans un programme réel, on ne laisserait pas l'exception faire planter le programme : on la gèrerait de manière agréable pour l'utilisateur. Ici, typiquement, on pourrait afficher un message avant de fermer le programme, pour expliquer à l'utilisateur pourquoi on ne peut plus continuer.

Section 12.7, [Pratique] Gérer les erreurs d'entrée • Partie VI

```

#include <iostream>
#include <limits>
#include <stdexcept>

int main()
{
    auto entree_securisee = [](auto & variable) -> void
    {

```

```

while (!(std::cin >> variable))
{
    if (std::cin.eof())
    {
        throw std::runtime_error("Le flux a été fermé !");
    }
    else if (std::cin.fail())
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();
    }
}

std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

};

int jour { 0 };
std::cout << "Quel jour es-tu né ? ";
entree_securisee(jour);

int mois { 0 };
std::cout << "Quel mois ? ";
entree_securisee(mois);

int annee { 0 };
std::cout << "Quelle année ? ";
entree_securisee(annee);

double taille { 0.0 };
std::cout << "Quelle taille ? ";
entree_securisee(taille);

std::cout << "Tu es né le " << jour << "/" << mois << "/" << annee
    << " et tu mesures " << taille << "m." << std::endl;
return 0;
}

```

Le code est déjà mieux qu'avant. Il reste cependant un problème : la lambda est locale à la fonction, donc si nous voulons utiliser notre fonction sécurisée à plein d'endroits différents, il va falloir trouver une autre solution. Nous verrons cela très bientôt.

Section 12.8, [Pratique] Gérer les erreurs d'entrée • Partie VII

```

#include <iostream>
#include <limits>

int main()
{
    auto entree_securisee = [](auto & variable, auto predicat) -
    > void
    {

```

```

while (!(std::cin >> variable) || !predicat(variable))
{
    if (std::cin.eof())
    {
        throw std::runtime_error("Le flux a été fermé !");
    }
    else if (std::cin.fail())
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();

        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
    else
    {
        std::cout << "Le prédicat n'est pas respecté !"
                    << std::endl;
    }
}
};

int jour { 0 };
std::cout << "Quel jour es-tu né ? ";
entree_securisee(jour, [](int jour) -> bool { return jour > 0 &&
jour <= 31; });

int mois { 0 };
std::cout << "Quelle mois ? ";
entree_securisee(mois, [](int mois) -> bool { return mois > 0 &&
mois <= 12; });

int annee { 0 };
std::cout << "Quelle année ? ";
entree_securisee(annee, [](int annee) -> bool { return annee >
1900; });

double taille { 0.0 };
std::cout << "Quelle taille ? ";
entree_securisee(taille, [](double taille) -> bool { return taille
> 0.0; });

std::cout << "Tu es né le " << jour << "/" << mois << "/" << annee
        << " et tu mesures " << taille << "m." << std::endl;
return 0;
}

```

Même si le problème soulevé à l'étape précédente reste vrai (à savoir qu'on ne peut pas utiliser la lambda ailleurs), notez comme nous avons déjà amélioré notre fonction pour lui faire vérifier que non seulement la valeur est du bon type, mais qu'elle est correcte.

Section 13.2, [Pratique] Gérer les erreurs d'entrée • Partie VIII

```
#include <iostream>
#include <limits>

template <typename T, typename Predicat>
void entree_securisee(T & variable, Predicat predicat)
{
    while (!(std::cin >> variable) || !predicat(variable))
    {
        if (std::cin.eof())
        {
            throw std::runtime_error("Le flux a été fermé !");
        }
        else if (std::cin.fail())
        {
            std::cout << "Entrée invalide. Recommence." << std::endl;
            std::cin.clear();

            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        }
        else
        {
            std::cout << "Le prédicat n'est pas respecté !" << std::endl;
        }
    }
}

int main()
{
    int jour { 0 };
    std::cout << "Quel jour es-tu né ? ";
    entree_securisee(jour, [](int jour) -> bool { return jour > 0 &&
    jour <= 31; });

    int mois { 0 };
    std::cout << "Quel mois ? ";
    entree_securisee(mois, [](int mois) -> bool { return mois > 0 &&
    mois <= 12; });

    int annee { 0 };
    std::cout << "Quelle année ? ";
    entree_securisee(annee, [](int annee) -> bool { return annee >
    1900; });

    double taille { 0.0 };
    std::cout << "Quelle taille ? ";
    entree_securisee(taille, [](double taille) -> bool { return taille
    > 0.0; });

    std::cout << "Tu es né le " << jour << "/" << mois << "/" << annee
    << " et tu mesures " << taille << "m." << std::endl;
    return 0;
}
```

```
}
```

TEMPLATES ET ALGORITHMES STANDARDS

Avez-vous remarqué les deux lignes du début de notre modèle ?

```
template <typename T, typename Predicat>
void entree_securisee(T & variable, Predicat predicat)
```

Grâce aux templates, nous pouvons passer une fonction sans effort ni difficulté. La bibliothèque standard procède de la même manière pour les algorithmes personnalisables.

Enfin, grâce à la surcharge, on peut combiner les deux versions de notre fonction, pour choisir de passer ou non un prédicat.

```
template<typename T>
void entree_securisee(T& variable)
{
    while (!(std::cin >> variable))
    {
        if (std::cin.eof())
        {
            break;
        }
        else if (std::cin.fail())
        {
            std::cout << "Entrée invalide. Recommence." << std::endl;
            std::cin.clear();

            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        }
        else
        {
            std::cout << "Le prédicat n'est pas respecté !" << std::endl;
        }
    }
}

template <typename T, typename Predicat>
void entree_securisee(T & variable, Predicat predicat)
{
    while (!(std::cin >> variable) || !predicat(variable))
    {
        if (std::cin.eof())
        {
            break;
        }
    }
}
```

```
    else if (std::cin.fail())
    {
        std::cout << "Entrée invalide. Recommence." << std::endl;
        std::cin.clear();

        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
    else
    {
        std::cout << "Le prédicat n'est pas respecté !" << std::endl;
    }
}
}
```

Index

Symboles

!, 56
", 24
#, 19
%, 28
&, 576, 577, 603
&&, 53, 604
(), 513, 524
*, 108
++, 108
+=", 36
--, 109
::, 461
;, 20
<<, 20, 138, 268
=, 35
[, 85, 95, 98, 513
{}, 31
||, 54

A

Accesseur, 491
Adresse, 576, 603
Affectation
 par copie, 515, 612
 par déplacement, 607
Afficher
 élément de tableau, 88
 message, 19
 sur la sortie standard, 268, 509
Algorithme, 114, 362
Algorithme standard, 678
Alias de type, 257
Amitié, 493
AND, opérateur, 53

Argument, 155
ASCII, 136
Assembleur, 392
Assertion, 178, 387, 636
assign, fonction, 92
Atom, 16
auto, mot clé, 38, 164, 203
Automatic storage, 573

B

back, fonction, 87, 96, 98
Bibliothèque, 417
 compilation, 446
 statique/dynamique, 418
bibliothèque
 32 bits/64 bits, 419
Bibliothèque standard, 19, 218, 257, 271, 358, 553
 (voir aussi std::)
 algorithme, 114
 concepts, 640
Booléen, 44
 opérateurs, 53
Boost, 417, 419
Boucle, 67
 for, 72, 88, 100
 imbriquée, 73
 infinie, 68
 while, 67
break, mot clé, 74
Breakpoint, 402
Buffer, 143

C

Chaîne brute, 133
Chaîne de caractères, 24, 38, 97, 363

- accéder à un caractère, 98
- boucler sur les caractères, 100
- concaténation, 271
- flux, 145
- ligne, 139
- nombre, 166
- remplacer, 364
- taille, 97
- char, mot clé, 31
- Chemin d'accès, 132
- Clang, 9, 16, 384, 387, 392
 - flags, 393
- class, mot clé, 478
 - ou struct, 478, 556
- Classe, 459
 - abstraite, 567
 - de politique, 644, 645
 - générique, 630
 - hôte, 645
 - instanciation, 480, 482
 - interface, 459, 493
- clear, fonction, 90
- CMake, 442
- Collection, 106
 - (voir aussi Conteneur)
 - hétérogène, 234
- Coloration syntaxique, 16
- Commentaire, 21
 - vs documentation, 377
- Compilateur, 9
 - avertissement, 25, 392
 - (voir aussi Warning)
 - fonction générique, 213, 387
 - optimisation, 59
- Compilation, 387
 - automatiser, 442
 - bibliothèque, 446
 - Linux, 16, 393
 - modes, 385, 397

- Comportement indéterminé, 86, 91, 164, 579, 594, 607, 621
- Composition directe, 598
- Compound assignment operator, 507
- Compter des occurrences, 115
- Concept, 640
- Condition, 46
 - imbriquée, 51
 - invariant, 476
- Console
 - accents (Windows), 15
 - sortie standard, 20
- const, mot clé, 34, 464, 466
- Constante, 34
 - compilation, 388
- constexpr, mot clé, 362, 388
- Constructeur, 480
 - de conversion, 487
 - de copie, 514, 524, 612
 - de déplacement, 607
 - héritage, 545
 - par défaut, 482
- Conteneur, 83, 106, 239
 - à accès direct, 110
 - bidirectionnel, 110
 - ordre des éléments, 118, 250
 - parcourir, 88, 106
 - supprimer des éléments, 114, 119
 - taille, 87
 - trier des éléments, 117
- continue, mot clé, 76
- Conversion de types, 393
 - constructeur, 487
 - implicite/explicite, 485
- Copier des objets, 514, 542, 569
- CppCheck, 454

D

- Déboguer, 178

Débogueur, 402
 Debug, mode, 386, 397, 402
 bibliothèque, 418
 decltype, mot clé, 165
 Décomposition, 258
 Décrémentation, 37
 opérateur, 109
 default, mot clé, 514
 delete, mot clé, 542, 609
 Délimiteur, 139
 chaîne brute, 133
 Déréférencer, 108, 577
 Destructeur, 612
 DIP, 663
 Directive de préprocesseur, 19, 318, 384
 do while, mot clé, 70
 Documenter, 366
 bug, 374
 exception, 372
 fonction, 370
 paramètres, 371
 type de retour, 373
 type personnalisé, 374
 double, mot clé, 31
 Doxygen, 366
 attributs, 370
 Durée, 272
 Durée de vie, 604
 Dynamic storage, 575

E

Échappement, 26, 133
 Écrire dans un fichier, 134
 Éditeur de texte, 16
 Égalité, 501
 else if, mot clé, 50
 else, mot clé, 48
 Encapsulation, 477
 Encodage, 135

Ensemble d'éléments, 248
 Entier, nombre, 31
 Entité, 541
 polymorphique, 557
 Entities Components Systems, 628
 Entrée standard, 39
 Énumération, 251
 Espace de nom, 462
 Évaluation en court-circuit, 59
 Évolutivité, 233
 Exception, 187, 361, 363, 553
 catch, 189
 documenter, 372
 générique, 192
 type, 188
 Exécutable, 397
 explicit, mot clé, 487
 Extension, 131

F

Fichier d'en-tête, 19, 313
 (voir aussi Inclusion)
 Fichier objet, 391, 398
 Fichier source, 316
 fill, fonction, 95
 final, mot clé, 622
 Flottant, 27, 31
 Flux, 131, 142, 194, 268, 509
 modificateur de, 140, 144
 Foncteur, 122
 Fonction, 19, 150
 amie, 493
 appels croisés, 168
 constante, 465
 contrat, 177
 de hachage, 250
 documenter, 370
 et constexpr, 389
 générique, 213

- identificateur, 153
- libre, 459, 502
- membre, 360, 459, 502, 622
- non-membre, 360
- paramètres, 154
- signature, 166
- surcharge, 166, 226
- template, 213
- virtuelle pure, 567
- vs lambda, 199
- for, mot clé, 72, 100
 - parcourir conteneur, 88
- friend, mot clé, 493
- front, fonction, 87, 96, 98
- Fuite de mémoire, 579

G

- GCC, 10, 16, 384, 388, 392
 - flags, 393
- gdb, 408, 412
- Gestion des erreurs, 40, 52, 71, 176, 312
- Getter, 491
- git, 433
- GitLab CI, 454

H

- Handle, 578
- Hash, 250
- Héritage, 544, 614, 658
 - bloquer, 622
 - multiple, 624
 - public/privé, 555
 - virtuel, 626
- Héritage du C, 39, 85, 124, 458
- .hpp, 317

I

- IDE, 11

- Identificateur, 150, 152
- if, mot clé, 46
- Inclusion, 313, 384
 - algorithm, 114
 - array, 94
 - cassert, 178
 - cctype, 124
 - ciso646, 53
 - cmath, 203
 - concepts, 640
 - fichier source, 316
 - fstream, 131
 - functional, 122
 - iosfwd, 526
 - iostream, 19
 - map, 250
 - numeric, 123
 - set, 250
 - sstream, 145
 - stdexcept, 188
 - string, 31
 - tuple, 234
 - unordered_map, 239
 - unordered_set, 248
 - utility, 589
- Incrémentation, 37
 - opérateur, 108
- Indirection, 576
 - non-propriétaire, 587
- Inférence de type, 38, 107
- Instanciation, 218
 - classe, 459, 482
 - template, 213, 387
- Instructions conditionnelles, 385
- int, mot clé, 31
- Intégration continue, 454
- Interface de classe, 459, 493
 - non-virtuelle, 614
 - publique, 477

Invariant, 476
 istream, 19
 ISP, 659
 Itérateur, 106, 577
 constance, 111
 déréférencer, 108
 distance entre deux, 126
 Itération, 67
 variable i, 74
 variable it, 109

L

Lambda, 199
 réutiliser, 203
 zone de capture, 207
 Linker, 397, 428
 Lire
 dans un fichier, 138
 ligne entière, 139
 Lisibilité, 21
 Liste d'initialisation des membres, 481
 Littéral, 24
 LSP, 552, 658
 (voir aussi Principe de substitution de
 Liskov)
 lvalue, 603

M

m_, préfixe, 478
 main, fonction, 19, 152
 Makefile, 445
 Matrice, 517
 Mémoire, 573
 adresse, 576
 fuite, 579
 template, 359
 MinGW
 bibliothèque, 448
 CMake, 445

Modèle de fonction, 213
 (voir aussi Template)
 Modificateur de flux, 140, 144
 boolalpha, 45, 144
 hex, 145
 Modulo, 28
 Moins unaire, 269
 mutable, mot clé, 209
 Mutateur, 491

N

Namespace, 462
 noexcept, mot clé, 467
 Nombre, 27
 chaîne de caractères, 166
 Non Virtual Interface (NVI), 614
 NOT, opérateur, 55
 nullptr, 577, 585

O

.o, extension, 391
 .obj, 391
 Objet, 459
 copier, 514
 courant, 507
 initialisation, 480
 OCP, 654 (voir Principe ouvert-fermé)
 Opérateur, 506
 !, 56
 %, 28
 &&, 53
 arithmétique, 264
 booléen, 53
 d'affectation, 35, 507, 515, 524, 612
 de comparaison, 44, 266, 360, 363
 de décrément, 109
 de résolution de portée, 462
 d'incrément, 108
 manipulation des flux, 268, 509

- raccourcis, 36
- sizeof, 570
- surcharge, 262, 360, 513, 524
- unaire, 269
- ||, 54
- Optimisation
 - par le compilateur, 59, 391
 - programme, 397
 - référence, 163
- OR, opérateur, 54
 - inclusif/exclusif, 60
- Ouvrir un fichier, 134
- Overloading, 166
 - (voir aussi Surcharge)
- override, mot clé, 560

P

- Paradigme, 352
- Paramétrage par politique, 644
- Paramètre
 - documenter, 371
 - et argument, 155
 - portée, 155
- Parcourir
 - conteneur, 106
 - de la fin vers le début, 112
 - tableau, 88
- Passage
 - par copie, 158, 253
 - par référence, 160
- Pile, 494
 - d'appels, 405, 409
- Plus unaire, 269
- Point d'arrêt, 402, 413
 - conditionnel, 411
- Point de variation, 645
 - compilation, 644
 - dynamique, 557, 644
- Pointeur, 576, 577, 634
 - nu/intelligent, 585
 - rôles, 585
- Politique, 645
 - (voir aussi Classe de politique)
- Polymorphisme d'inclusion, 566, 627
- pop_back, fonction, 90, 99
- Portée, 47, 462, 619
- Postcondition, 177, 476
- Précondition, 177, 476
- Prédicat, 121
- Préprocesseur, 19, 384
- Principe
 - inversion des dépendances, 663
 - ouvert-fermé, 560, 654
 - responsabilité unique, 652
 - ségrégation des interfaces, 659
 - substitution de Liskov, 552, 614, 658
- private, mot clé, 477
- Procédure, 151
- Produit, 123
- Programmation
 - fonctionnelle, 353
 - générique, 215, 354
 - impérative, 352
 - orientée objet, 457
 - par contrat, 355, 476, 553, 636
 - par politique, 644
 - récursive, 169
- Protected, mot clé, 619
- Prototype, 169
- public, mot clé, 477, 555
- push_back, fonction, 89, 99

Q

- Qt Creator, 314, 384, 392
 - CMake, 445

R

- R"(...) ", 133

Racine, 132
 Random access, 110
 Raw strings, 133
 Rechercher un élément, 116, 119
 Référence, 159, 577
 et auto, 164
 rvalue, 604
 Release, mode, 386, 397
 bibliothèques, 418
 Remplir un tableau, 92, 95
 Résolution de portée, 462
 return, mot clé, 21, 153
 *this, 507
 Rule of Zero, 612
 rvalue, 603

S

.s, extension, 392
 s, suffixe, 38, 234, 271
 Section d'un programme, 399
 Sémantique, 499
 d'entité, 541
 de déplacement, 601, 606
 de valeur, 501
 Séparateur, 139
 (voir aussi Délimiteur)
 Séquence d'échappement, 26
 Setter, 491
 SFML, 418
 Signature d'une fonction, 166
 sizeof, 570
 Slicing, 570
 .sln, 444
 SOLID
 DIP, 663
 ISP, 659
 LSP, 658
 (voir aussi Principe de substitution
 de Liskov)

OCP, 654
 SRP, 652
 Somme, 123
 Sortie standard, 20, 268, 509
 Spécialisation, 226, 638
 SRP, 652, 660
 Stack trace, 402
 StackOverflow, 454
 Static storage, 574
 static_assert, 636
 static_cast, mot clé, 464, 485, 605
 std::, 20
 abs, 203
 accumulate, 123
 all_of, 124
 any_of, 203
 array, 94, 632
 begin, 107
 boolalpha, 45, 144
 cbegin, 111
 cend, 111
 cin, 40, 52
 cin.eof, 194
 cin.fail, 194
 count, 115
 cout, 20
 distance, 126
 empty, 88, 97, 99
 end, 107
 endl, 20
 equal, 120
 espace de nom, 462
 exception, 192
 find, 116
 find_if, 202
 find_if_not, 126
 for_each, 201
 function, 596
 get, 234

- getline, 139
- greater, 122
- hex, 145
- ifstream, 138
- inner_product, 528
- is_arithmetic, 636
- is_const, 638
- list, 110, 116, 118
- make_unique, 585
- map, 250
- move, 589, 605
- multiplies, 123
- noboolalpha, 144
- nullopt, 597
- numeric_limits, 185
- ofstream, 134
- optional, 597
- rbegin, 112
- remove, 119
- rend, 112
- reverse, 118
- runtime_error, 188
- scientific, 145
- search, 119
- set, 250
- showpos, 145
- size, 87, 96, 97
- size_t, 87
- sort, 117, 122, 201, 362
- stod, 167
- stoi, 167, 191
- streamsize, 185
- string, 97, 363, 458
- swap, 529
- tie, 260
- tuple, 234
- unary_function, 622
- unique_ptr, 605
- unordered_map, 239

- unordered_set, 248
- vector, 83
- ws, 140
- string, mot clé, 31
- struct, mot clé, 230, 478
 - ou class, 478, 556
- Structure, 230
 - déclaration variable, 259
 - membre, 232
 - syntaxe, 231
 - vs tuple, 239
- Structure de contrôle, 46
- Structure de données, 83, 283
 - struct, 230
- Sublime Text, 16
- Substitution, principe de, 552, 658
- Sucre syntaxique, 257
- Surcharge
 - fonction, 166, 226
 - opérateur, 262, 284, 513, 524
- Système de fichiers, 132
- système hexadécimal, 145

T

- Table associative, 239
- Table de vérité, 54
- Tableau, 83
 - (voir aussi Conteneur)
 - accéder à un élément, 85, 95, 271
 - ajouter un élément, 89
 - dynamique, 83
 - modifier un élément, 91
 - parcourir, 88
 - remplir, 95
 - statistique, 94
 - supprimer un élément, 90, 114
 - taille, 87, 96
- Taille
 - chaîne de caractères, 97

- conteneur, 87
- tableau, 96
- Tampon, 143
- Template, 213, 218, 323, 359, 656, 678
 - classe, 630
 - compilation, 387
- template, mot clé, 213
- Test unitaire, 182, 284
- Tester
 - caractère, 124
 - éléments d'un ensemble, 124
 - équivalence, 120
- throw, mot clé, 188
- Trait, 635
- Trier des éléments, 117, 122
- TU (voir Test unitaire)
- Tuple, 234
 - déclaration variable, 258
 - vs structure, 239
- Type
 - alias, 257
 - conversion, 393, 485
 - exception, 188
 - fonction, 153
 - générique, 213
 - inférence, 38, 107
 - paramètre, 154
 - personnalisé, 374
 - sémantique, 499
 - variable, 31, 459
- typename, mot clé, 213, 632

U

- UB (Undefined behavior), 86, 361
- Unité de compilation, 387
- using, mot clé, 257, 545
- UTF-8, 136

V

- Valeur par défaut, 33
- Variable, 29
 - constexpr, 389
 - déclaration collective, 258
 - déclarer, 30
 - mémoire, 573
 - modifier valeur, 35
 - portée, 47
 - valeur par défaut, 33
- virtual, mot clé, 560
- Visibilité, 477, 477, 547
- Visual Studio, 208, 314, 384, 391, 392
 - bibliothèque, 447
 - CMake, 444
 - fichier SLN, 444
- Visual Studio Code, 16
- Visual Studio Community, 11
- void, mot clé, 153

W

- Wandbox, 10
- Warning, 25, 392
 - multi-character character constant, 25
 - unknown escape sequence, 26, 26
- while, mot clé, 67
- Windows
 - accents, 15, 20
 - encodage, 135

X

- XOR, 60